

# **Topics on econometrics and causal inference**

Xiang Ao

2026-09-03



# Table of contents

|                                                           |               |
|-----------------------------------------------------------|---------------|
| <b>Preface</b>                                            | <b>1</b>      |
| How to cite . . . . .                                     | 1             |
| <br><b>I. Regression Foundations</b>                      | <br><b>3</b>  |
| <b>1. Interpreting interaction in a regression model</b>  | <b>5</b>      |
| 1.1. Interaction with two binary variables . . . . .      | 5             |
| 1.2. Interaction with continuous variables . . . . .      | 7             |
| <b>2. Interaction term in a non-linear model</b>          | <b>13</b>     |
| 2.1. The issue . . . . .                                  | 13            |
| 2.2. A worked example . . . . .                           | 14            |
| <b>3. Chow test and more</b>                              | <b>17</b>     |
| 3.1. Chow test . . . . .                                  | 17            |
| 3.2. A comparison with two different outcomes . . . . .   | 20            |
| 3.3. Overlapping samples . . . . .                        | 25            |
| 3.4. IV regression . . . . .                              | 29            |
| 3.5. IV with fixed effects . . . . .                      | 31            |
| 3.6. Chow test with different covariates . . . . .        | 42            |
| 3.7. Conclusion . . . . .                                 | 46            |
| <b>4. Weights in OLS in causal inference</b>              | <b>49</b>     |
| 4.1. Unit level weights for OLS . . . . .                 | 49            |
| 4.2. Strata level weights . . . . .                       | 51            |
| 4.3. Recommended estimation approaches . . . . .          | 51            |
| 4.4. A worked example . . . . .                           | 51            |
| 4.5. When the weights actually bite . . . . .             | 54            |
| <br><b>II. Marginal Effects &amp; Fixed Effects</b>       | <br><b>57</b> |
| <b>5. Marginal effects in models with fixed effects</b>   | <b>59</b>     |
| 5.1. Marginal effects in a linear model . . . . .         | 59            |
| 5.2. Marginal effects in a non-linear model . . . . .     | 61            |
| <b>6. Comparing Marginal effects with margins command</b> | <b>71</b>     |
| 6.1. Comparing marginal effects . . . . .                 | 71            |
| 6.2. Summary . . . . .                                    | 79            |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| <b>7. Fixed or Random Effect, or Both?</b>                                      | <b>81</b>  |
| 7.1. Panel data . . . . .                                                       | 81         |
| 7.2. Time-invariant variables . . . . .                                         | 81         |
| 7.3. Between-within model . . . . .                                             | 81         |
| 7.4. BW model in R . . . . .                                                    | 85         |
| 7.5. BW model in Stata . . . . .                                                | 88         |
| 7.6. BW model in non-linear models . . . . .                                    | 90         |
| <b>8. The Mundlak Device in Panel Data and Difference-in-Differences</b>        | <b>91</b>  |
| 8.1. Introduction: Fixed Effects vs. Mundlak Regression . . . . .               | 91         |
| 8.2. Empirical Demonstration: One-Way Panel ( <code>nlswork</code> ) . . . . .  | 91         |
| 8.3. Two-Way Mundlak (TWM) and the Balanced Panel Condition . . . . .           | 94         |
| 8.4. Multi-Way Fixed Effects (> 2 Dimensions) . . . . .                         | 96         |
| 8.5. Application to Difference-in-Differences (Common Timing) . . . . .         | 98         |
| 8.6. Summary . . . . .                                                          | 102        |
| <b>9. Correlated Random Effect</b>                                              | <b>103</b> |
| 9.1. Panel data . . . . .                                                       | 103        |
| 9.2. Fixed effect . . . . .                                                     | 103        |
| 9.3. Random effect . . . . .                                                    | 103        |
| 9.4. Correlated random effect . . . . .                                         | 103        |
| 9.5. Example . . . . .                                                          | 104        |
| <b>10. Correlated Random Effects (CRE) in Linear and Nonlinear Models</b>       | <b>109</b> |
| 10.1. Background: Unobserved Heterogeneity and CRE . . . . .                    | 109        |
| 10.2. Linear Models: Equivalence to Fixed Effects . . . . .                     | 109        |
| 10.3. Nonlinear Panel Models: Binary Outcomes . . . . .                         | 111        |
| 10.4. Summary . . . . .                                                         | 112        |
| <b>III. Treatment Effects &amp; Matching</b>                                    | <b>113</b> |
| <b>11. Treatment effects and matching</b>                                       | <b>115</b> |
| 11.1. Treatment effects in observational studies . . . . .                      | 115        |
| 11.2. Matching . . . . .                                                        | 129        |
| 11.3. Modern matching in R: balance diagnostics and entropy weighting . . . . . | 133        |
| <b>12. When is OLS coefficient the ATE</b>                                      | <b>139</b> |
| 12.1. OLS cannot be ATE (most of the time) . . . . .                            | 139        |
| 12.2. Regression adjustment . . . . .                                           | 143        |
| <b>13. Matching and Weighting Part 1: matching</b>                              | <b>145</b> |
| 13.1. Assumptions . . . . .                                                     | 145        |
| 13.2. Distance . . . . .                                                        | 145        |
| 13.3. matching methods . . . . .                                                | 146        |
| 13.4. Example . . . . .                                                         | 147        |
| 13.5. Estimation . . . . .                                                      | 163        |

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| <b>14. Matching and Weighting - Part 2: weighting</b>                      | <b>165</b> |
| 14.1. Weighting methods . . . . .                                          | 165        |
| 14.2. Example . . . . .                                                    | 166        |
| 14.3. comparing matching and weighting . . . . .                           | 171        |
| <b>15. Sensitivity analysis of OLS with sensemakr</b>                      | <b>173</b> |
| 15.1. OVB . . . . .                                                        | 173        |
| 15.2. Partial $R^2$ of treatment on outcome . . . . .                      | 173        |
| 15.3. Robust value . . . . .                                               | 173        |
| 15.4. Bounds using observed covariate(s) . . . . .                         | 174        |
| 15.5. Example . . . . .                                                    | 174        |
| 15.6. sensitivity analysis for IV . . . . .                                | 178        |
| <b>16. Gender Wage Gap Analysis: Oaxaca-Blinder and Causal Estimation</b>  | <b>183</b> |
| 16.1. Introduction: Oaxaca-Blinder as Regression Adjustment . . . . .      | 183        |
| 16.2. Empirical Application: 2015 CPS Data . . . . .                       | 183        |
| 16.3. Adding Occupation and Industry: Mediators vs. Bad Controls . . . . . | 186        |
| 16.4. Nonparametric Doubly Robust Estimation (AIPW) . . . . .              | 187        |
| 16.5. Summary . . . . .                                                    | 188        |
| <b>IV. Panel Data &amp; DiD</b>                                            | <b>189</b> |
| <b>17. Causal Forest in panel data</b>                                     | <b>191</b> |
| 17.1. Introduction . . . . .                                               | 191        |
| 17.2. Linear effect . . . . .                                              | 191        |
| 17.3. Nonlinear effect case 1 . . . . .                                    | 198        |
| 17.4. Nonlinear effect case 2 . . . . .                                    | 204        |
| 17.5. Summary . . . . .                                                    | 210        |
| <b>18. Synthetic Control in R and Stata</b>                                | <b>211</b> |
| 18.1. Synthetic control . . . . .                                          | 211        |
| 18.2. inference . . . . .                                                  | 211        |
| 18.3. an example . . . . .                                                 | 212        |
| 18.4. Generalized Synthetic Control . . . . .                              | 218        |
| <b>19. Same Data, Different Estimators: A Synthetic Control Comparison</b> | <b>225</b> |
| 19.1. One panel, four estimators . . . . .                                 | 225        |
| 19.2. Four estimators, one panel . . . . .                                 | 225        |
| 19.3. What changes across estimators . . . . .                             | 226        |
| 19.4. The figure . . . . .                                                 | 226        |
| 19.5. What the comparison teaches . . . . .                                | 228        |
| 19.6. Setup code . . . . .                                                 | 228        |
| <b>20. Bartik Instrument and Rotemberg weights</b>                         | <b>231</b> |
| 20.1. Bartik Instrument . . . . .                                          | 231        |
| 20.2. Bartik Instrument in special cases . . . . .                         | 232        |
| 20.3. Rotemberg weights . . . . .                                          | 232        |
| 20.4. Simulations . . . . .                                                | 233        |

## Table of contents

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| 20.5. Why is good to have Rotemberg weights? . . . . .                      | 239        |
| 20.6. What if you are using a reduced form? . . . . .                       | 240        |
| <b>21. Extended TWFE and other DiD estimators</b>                           | <b>243</b> |
| 21.1. Staggered DiD without treatment reversal . . . . .                    | 243        |
| 21.2. Callaway and Sant'Anna . . . . .                                      | 249        |
| 21.3. Comparing the estimators . . . . .                                    | 261        |
| 21.4. Staggered DiD with treatment reversal . . . . .                       | 262        |
| <b>22. Causal Panel (why DDDiD)</b>                                         | <b>263</b> |
| 22.1. Why DDDiD . . . . .                                                   | 263        |
| 22.2. Synthetic Control . . . . .                                           | 263        |
| 22.3. Synthetic DiD . . . . .                                               | 263        |
| 22.4. Example from synthdid . . . . .                                       | 264        |
| 22.5. Summary . . . . .                                                     | 266        |
| <b>V. Count Data &amp; Specialized Models</b>                               | <b>267</b> |
| <b>23. Which count data model to use</b>                                    | <b>269</b> |
| 23.1. A comparison of various count data models with extra zeros . . . . .  | 269        |
| 23.2. Quick worked example . . . . .                                        | 272        |
| 23.3. Mixed count models with glmmTMB . . . . .                             | 274        |
| <b>24. What model to use for rare events</b>                                | <b>277</b> |
| 24.1. Introduction . . . . .                                                | 277        |
| 24.2. Simulation . . . . .                                                  | 278        |
| <b>25. Poisson model with exposure</b>                                      | <b>281</b> |
| 25.1. Poisson with exposure or Poisson rate . . . . .                       | 281        |
| 25.2. Simulation . . . . .                                                  | 282        |
| 25.3. Fitting the three models . . . . .                                    | 282        |
| 25.4. When $\gamma \neq 1$ . . . . .                                        | 283        |
| 25.5. Stata equivalent . . . . .                                            | 284        |
| 25.6. Which model to use? . . . . .                                         | 285        |
| <b>26. Instrumental Variables in Fixed-Effects Poisson Models</b>           | <b>287</b> |
| 26.1. The Problem: Endogeneity with Exponential Conditional Means . . . . . | 287        |
| 26.2. The Two-Step Control Function Recipe . . . . .                        | 287        |
| 26.3. Continuous Endogenous Variable: Application in Stata . . . . .        | 288        |
| 26.4. Implementation in R: <code>fixest</code> . . . . .                    | 295        |
| 26.5. Extension: Binary Endogenous Regressor . . . . .                      | 297        |
| 26.6. Summary . . . . .                                                     | 300        |
| <b>VI. Causal Inference Methods</b>                                         | <b>301</b> |
| <b>27. Using machine learning for causal effect in observational study</b>  | <b>303</b> |
| 27.1. A simulation for an OLS model . . . . .                               | 303        |

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>28. Mediation analysis in R and Stata</b>                       | <b>309</b> |
| 28.1. Mediation analysis . . . . .                                 | 309        |
| 28.2. Causal Mediation analysis . . . . .                          | 312        |
| 28.3. Binary outcome . . . . .                                     | 315        |
| 28.4. Going further: formal causal mediation . . . . .             | 316        |
| <b>29. Endogeneity with censored variable</b>                      | <b>319</b> |
| 29.1. Bunching for the outcome variable . . . . .                  | 319        |
| 29.2. Bunching for the treatment variable . . . . .                | 319        |
| 29.3. Simulation . . . . .                                         | 321        |
| 29.4. Conclusion . . . . .                                         | 324        |
| <b>30. G-computation (g-formula) with time varying covariates</b>  | <b>325</b> |
| 30.1. The problem . . . . .                                        | 325        |
| 30.2. Assumptions . . . . .                                        | 326        |
| 30.3. Identifiability . . . . .                                    | 327        |
| 30.4. Estimation . . . . .                                         | 327        |
| 30.5. Parametric g-formula . . . . .                               | 327        |
| 30.6. an example . . . . .                                         | 328        |
| 30.7. A second example . . . . .                                   | 334        |
| <b>31. Policy learning by policytree</b>                           | <b>339</b> |
| 31.1. Optimal policy . . . . .                                     | 339        |
| 31.2. IPW loss . . . . .                                           | 340        |
| 31.3. AIPW loss . . . . .                                          | 340        |
| 31.4. Policytree . . . . .                                         | 341        |
| <b>32. Recent causal inference tools</b>                           | <b>347</b> |
| 32.1. Identification . . . . .                                     | 347        |
| 32.2. Nonparametric estimation in a nutshell . . . . .             | 347        |
| 32.3. Simulation with a known treatment effect . . . . .           | 349        |
| 32.4. EAGeR trial simulation (AIPW package) . . . . .              | 357        |
| <b>33. Intro to proximal causal inference</b>                      | <b>365</b> |
| 33.1. Unobserved confounders . . . . .                             | 365        |
| 33.2. Proximal causal inference . . . . .                          | 365        |
| 33.3. example . . . . .                                            | 369        |
| <b>34. Simulation on proximal causal inference with panel data</b> | <b>373</b> |
| 34.1. A simulation with panel data . . . . .                       | 374        |
| <b>35. longitudinal modified treatment policy (LMTP)</b>           | <b>379</b> |
| 35.1. Definitions and assumptions . . . . .                        | 380        |
| 35.2. Using lmtip package . . . . .                                | 386        |
| 35.3. another example . . . . .                                    | 388        |

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| <b>VII. Advanced Topics</b>                                                  | <b>393</b> |
| <b>36. Doing the same multi-level model with R and stata</b>                 | <b>395</b> |
| 36.1. Stata and R syntax comparison for the same three level model . . . . . | 395        |
| 36.2. A growth model in R and stata . . . . .                                | 400        |
| 36.3. cem in R and stata . . . . .                                           | 404        |
| <b>37. Modeling Variation, Not Just the Mean, in Likert-Scale Data</b>       | <b>407</b> |
| 37.1. The Problem: The Mechanical Ceiling on Dispersion . . . . .            | 407        |
| 37.2. The Solution: Ordinal Cumulative-Probit Location-Scale Model . . . . . | 408        |
| 37.3. Simulation: Identical Dispersion with Different Means . . . . .        | 409        |
| 37.4. Model Comparison: Gaussian vs. Ordinal Location-Scale . . . . .        | 411        |
| 37.5. Benchmarking Relative Dispersion: The <i>R</i> Metric . . . . .        | 412        |
| 37.6. Summary . . . . .                                                      | 413        |
| <b>38. Conjoint design and analysis using R</b>                              | <b>415</b> |
| 38.1. Conjoint Analysis . . . . .                                            | 415        |
| 38.2. Example . . . . .                                                      | 416        |
| 38.3. Conjoint design of experiment . . . . .                                | 417        |
| 38.4. Conjoint analysis . . . . .                                            | 420        |
| 38.5. Causal effect in conjoint analysis . . . . .                           | 423        |
| <b>39. Stata 19 CATE command</b>                                             | <b>425</b> |
| 39.1. Stata 19 CATE command . . . . .                                        | 425        |
| 39.2. AIPW . . . . .                                                         | 433        |
| <b>40. Spatial econometrics introduction</b>                                 | <b>437</b> |
| 40.1. Why do we need spatial econometrics? . . . . .                         | 437        |
| 40.2. A general linear spatial model . . . . .                               | 437        |
| 40.3. Estimation with S2SLS for a less general model . . . . .               | 438        |
| 40.4. Estimation with a more general model . . . . .                         | 438        |
| 40.5. an example . . . . .                                                   | 439        |
| <b>41. Causal simulation</b>                                                 | <b>443</b> |
| 41.1. examples . . . . .                                                     | 444        |
| 41.2. a more complicated model . . . . .                                     | 448        |
| <b>42. Frengression and Engression</b>                                       | <b>453</b> |
| 42.1. Engression . . . . .                                                   | 453        |
| 42.2. From Engression to Frengression . . . . .                              | 455        |
| 42.3. The Architecture . . . . .                                             | 455        |
| 42.4. Using Rfrengression . . . . .                                          | 455        |
| 42.5. Frengression as DGP . . . . .                                          | 458        |
| 42.6. Real Data: LaLonde . . . . .                                           | 458        |
| 42.7. Comparison: causl vs frengression . . . . .                            | 460        |
| 42.8. Benchmarking: Frengression vs Semiparametric Methods . . . . .         | 460        |
| 42.9. Summary . . . . .                                                      | 472        |



|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| <b>43. Partial Interference</b>                                          | <b>473</b> |
| 43.1. The Setup . . . . .                                                | 473        |
| 43.2. Estimation . . . . .                                               | 474        |
| 43.3. Using CausalModel . . . . .                                        | 475        |
| 43.4. Monte Carlo Validation . . . . .                                   | 479        |
| 43.5. What If You Ignore Interference? . . . . .                         | 482        |
| 43.6. Summary . . . . .                                                  | 484        |
| <b>44. Power analysis with list experiment</b>                           | <b>485</b> |
| 44.1. List experiment . . . . .                                          | 485        |
| 44.2. Power analysis . . . . .                                           | 485        |
| <b>45. Causal mediation analysis</b>                                     | <b>489</b> |
| 45.1. Traditional mediation analysis . . . . .                           | 489        |
| 45.2. Causal mediation analysis . . . . .                                | 489        |
| 45.3. Summary . . . . .                                                  | 498        |
| <b>46. Uplift Modeling and CATE</b>                                      | <b>499</b> |
| 46.1. Python: causalml uplift trees . . . . .                            | 499        |
| 46.2. R: causal forest for CATE . . . . .                                | 501        |
| 46.3. Uplift vs CATE: the same thing with different vocabulary . . . . . | 503        |
| <b>47. Using Numpyro</b>                                                 | <b>505</b> |
| 47.1. Example 1 . . . . .                                                | 505        |
| 47.2. Example 2: Numpyro hierarchical model . . . . .                    | 507        |
| 47.3. Comparing the two approaches . . . . .                             | 509        |
| <b>48. Equivalence Testing and Two One-Sided Tests (TOST)</b>            | <b>511</b> |
| 48.1. The Problem: Non-Significance Is Not Evidence of Absence . . . . . | 511        |
| 48.2. Two One-Sided Tests (TOST) . . . . .                               | 511        |
| 48.3. Implementation in R: TOSTER . . . . .                              | 511        |
| 48.4. Bayesian Equivalence Testing: The Unit Trap . . . . .              | 513        |
| 48.5. Specifying the Equivalence Bound ( $\Delta$ ) . . . . .            | 514        |
| 48.6. Summary . . . . .                                                  | 515        |



# Preface

This is a working notebook rather than a textbook. The chapters are posts I wrote while reading, teaching, and consulting on applied econometrics, often because a colleague or a student asked a question that did not have a tidy answer in the books I had. Each one stands on its own. The order did not follow a plan, and the same topic sometimes appears more than once as my understanding changed.

The common question is the one Angrist and Pischke emphasize: what is being identified, by what variation, under what assumptions. The posts ask that question in different settings: interactions and marginal effects, fixed and correlated random effects, matching and weights, difference-in-differences, synthetic control, count models, TMLE and AIPW, mediation, proximal causal inference, partial interference, conjoint and list experiments, uplift modeling, and policy trees.

Most examples are in R. I use Stata where it is more natural, and sometimes Python or Julia when the package is better there. The two companion books, [Introduction to Causal Econometrics](#) and [Causal Econometrics with Julia](#), organize the same material more systematically. This collection is the messier version: useful when I want to see how a specific method behaves on a specific problem.

## How to cite

Ao, Xiang. *Topics on Econometrics and Causal Inference*. [https://xiangao.github.io/blog\\_book/](https://xiangao.github.io/blog_book/).

```
@book{ao_topics_econometrics,  
  author = {Ao, Xiang},  
  title  = {Topics on Econometrics and Causal Inference},  
  year   = {2026},  
  url    = {https://xiangao.github.io/blog_book/}  
}
```



**Part I.**

# **Regression Foundations**



# 1. Interpreting interaction in a regression model

## 1.1. Interaction with two binary variables

Start with the simplest case: two binary variables  $x_1$  and  $x_2$  coded 0/1.

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 \quad (1.1)$$

The model is saturated — three coefficients for three contrasts from the baseline  $(x_1, x_2) = (0, 0)$ , one for each of the other three cells. Often only the interaction term gets attention. When the two variables are treatment and group, the interaction is the difference in treatment effects between groups, and that may be all that matters. But in other designs all four cells are substantively interesting, and then  $\beta_1$  and  $\beta_2$  matter too.

Consider the model:

$$E(y) = \beta_0 + \beta_1 \text{female} + \beta_2 \text{treatment} + \beta_{12} \text{female} * \text{treatment} \quad (1.2)$$

The four cell means are:

|           | male                |       | female                                     |       |
|-----------|---------------------|-------|--------------------------------------------|-------|
| control   | $\beta_0$           | (1.2) | $\beta_0 + \beta_1$                        | (1.2) |
| treatment | $\beta_0 + \beta_2$ | (1.2) | $\beta_0 + \beta_1 + \beta_2 + \beta_{12}$ | (1.2) |

Each  $\beta$  reads off the table:  $\beta_0$  is the baseline cell mean,  $\beta_1$  the female premium among controls,  $\beta_2$  the treatment effect among males. The interaction

$$\beta_{12} = (E(Y|(1, 1)) - E(Y|(0, 1))) - (E(Y|(1, 0)) - E(Y|(0, 0))) \quad (1.3)$$

is the difference in differences — the treatment effect for females minus the treatment effect for males. When the research question is “does the treatment effect differ by group?”, the interaction

## 1. Interpreting interaction in a regression model

term is the answer. Replace female/male with before/after and the same expression is the standard DiD estimator.

But sometimes the interest is in all four cells. Union membership crossed with race, for example — we may want all six pairwise comparisons, not just the interaction. Stata's `margins` command computes these directly:

```
webuse union3
reg ln_wage i.union##i.black, r
margins union#black
margins union#black, pwcompare
```

```
. webuse union3
(NLS Women 14-24 in 1968)
```

```
. reg ln_wage i.union##i.black, r
```

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 1,244  |
|                   | F(3, 1240)    | = | 34.76  |
|                   | Prob > F      | = | 0.0000 |
|                   | R-squared     | = | 0.0762 |
|                   | Root MSE      | = | .37699 |

| -----       |             |                     |        |       |                      |           |  |
|-------------|-------------|---------------------|--------|-------|----------------------|-----------|--|
| ln_wage     | Coefficient | Robust<br>std. err. | t      | P> t  | [95% conf. interval] |           |  |
| -----       |             |                     |        |       |                      |           |  |
| 1.union     | .2045053    | .0291682            | 7.01   | 0.000 | .1472808             | .2617298  |  |
| 1.black     | -.1709034   | .0308067            | -5.55  | 0.000 | -.2313425            | -.1104644 |  |
| union#black |             |                     |        |       |                      |           |  |
| 1 1         | .0386275    | .0516609            | 0.75   | 0.455 | -.062725             | .13998    |  |
|             |             |                     |        |       |                      |           |  |
| _cons       | 1.657525    | .0138278            | 119.87 | 0.000 | 1.630396             | 1.684653  |  |
| -----       |             |                     |        |       |                      |           |  |

```
. margins union#black
```

|                      |                       |
|----------------------|-----------------------|
| Adjusted predictions | Number of obs = 1,244 |
| Model VCE: Robust    |                       |

Expression: Linear prediction, predict()

| ----- |              |           |   |      |                      |
|-------|--------------|-----------|---|------|----------------------|
|       | Delta-method |           |   |      |                      |
|       | Margin       | std. err. | t | P> t | [95% conf. interval] |
| ----- |              |           |   |      |                      |



| union#black |  |          |          |        |       |          |          |
|-------------|--|----------|----------|--------|-------|----------|----------|
| 0 0         |  | 1.657525 | .0138278 | 119.87 | 0.000 | 1.630396 | 1.684653 |
| 0 1         |  | 1.486621 | .027529  | 54.00  | 0.000 | 1.432613 | 1.54063  |
| 1 0         |  | 1.86203  | .0256822 | 72.50  | 0.000 | 1.811644 | 1.912415 |
| 1 1         |  | 1.729754 | .0325611 | 53.12  | 0.000 | 1.665873 | 1.793635 |

```
. margins union#black, pwcompare
```

Pairwise comparisons of adjusted predictions  
Model VCE: Robust

Number of obs = 1,244

Expression: Linear prediction, predict()

|                |           | Delta-method | Unadjusted           |
|----------------|-----------|--------------|----------------------|
|                | Contrast  | std. err.    | [95% conf. interval] |
| union#black    |           |              |                      |
| (0 1) vs (0 0) | -.1709034 | .0308067     | -.2313425 -.1104644  |
| (1 0) vs (0 0) | .2045053  | .0291682     | .1472808 .2617298    |
| (1 1) vs (0 0) | .0722294  | .0353756     | .0028268 .141632     |
| (1 0) vs (0 1) | .3754087  | .0376487     | .3015466 .4492709    |
| (1 1) vs (0 1) | .2431328  | .0426388     | .1594807 .326785     |
| (1 1) vs (1 0) | -.1322759 | .0414705     | -.2136359 -.0509159  |

.

The `margins` call reports the four cell means; `pwcompare` gives all six pairwise comparisons. In this example, all six pairwise contrasts are significant at 95% — yet the interaction term is insignificant ( $p = 0.455$ ). The two facts are compatible. Each pairwise comparison asks whether two cells differ from each other. The interaction asks whether the *difference between two of those differences* is non-zero — a second-order quantity with its own, larger standard error. Here the union premium is clearly positive for both groups, but the gap between the two premiums (0.039) is not distinguishable from zero.

## 1.2. Interaction with continuous variables

With continuous  $x_1$  and  $x_2$ ,

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 \quad (1.4)$$

centering both variables — subtracting each mean before forming the product — is a linear transformation that leaves fitted values,  $R^2$ , and everything about  $\beta_{12}$  unchanged. What it changes is  $\beta_1$  and  $\beta_2$ .

## 1. Interpreting interaction in a regression model

First, it reduces collinearity. When  $x_1$  and  $x_2$  are positive,  $x_1x_2$  is mechanically correlated with both, inflating the standard errors of  $\beta_1$  and  $\beta_2$ . Centering breaks most of that correlation (all of it under normality), stabilizing the main-effect estimates without altering the interaction.

Second, it fixes the interpretation. In the uncentered model,  $\beta_1$  is the effect of  $x_1$  when  $x_2 = 0$ , and zero may be outside the support of  $x_2$ . After centering,  $\beta_1$  is the effect of  $x_1$  evaluated at the mean of  $x_2$  — a quantity that is always interpretable. When a dummy interacts with a continuous variable, only the continuous variable should be centered.

Stata's `margins` command computes the conditional predictions directly. The data are the built-in `auto` file: 74 1978 model cars, with `price` in dollars, `mpg` in miles per gallon, and `foreign` marking the 22 imports. We centre `mpg` at its mean, regress price on `foreign` interacted with centred `mpg`, and then ask for predicted prices by origin across a range of `mpg` values.

```
sysuse auto
sum mpg
gen mpg_centered=mpg-r(mean)
sum mpg_centered
reg price i.foreign##c.mpg_centered
margins foreign, at(mpg_centered=(-3 (1) 3))
marginsplot
graph export "marginsplot-interaction.svg", as(svg) replace
```

```
. sysuse auto
(1978 automobile data)
```

```
. sum mpg
```

| Variable    | Obs | Mean    | Std. dev. | Min | Max |
|-------------|-----|---------|-----------|-----|-----|
| -----+----- |     |         |           |     |     |
| mpg         | 74  | 21.2973 | 5.785503  | 12  | 41  |

```
. gen mpg_centered=mpg-r(mean)
```

```
. sum mpg_centered
```

| Variable     | Obs | Mean      | Std. dev. | Min       | Max     |
|--------------|-----|-----------|-----------|-----------|---------|
| -----+-----  |     |           |           |           |         |
| mpg_centered | 74  | -4.03e-08 | 5.785503  | -9.297297 | 19.7027 |

```
. reg price i.foreign##c.mpg_centered
```

| Source      | SS        | df | MS         | Number of obs | = | 74     |
|-------------|-----------|----|------------|---------------|---|--------|
| -----+----- |           |    |            | F(3, 70)      | = | 9.48   |
| Model       | 183435285 | 3  | 61145094.9 | Prob > F      | = | 0.0000 |
| Residual    | 451630112 | 70 | 6451858.74 | R-squared     | = | 0.2888 |
| -----+----- |           |    |            | Adj R-squared | = | 0.2584 |

## 1.2. Interaction with continuous variables

Total | 635065396 73 8699525.97 Root MSE = 2540.1

| price        | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |           |
|--------------|-------------|-----------|-------|-------|----------------------|-----------|
| foreign      |             |           |       |       |                      |           |
| Foreign      | 1666.519    | 717.217   | 2.32  | 0.023 | 236.0751             | 3096.963  |
| mpg_centered | -329.2551   | 74.98545  | -4.39 | 0.000 | -478.8088            | -179.7013 |
| foreign#     |             |           |       |       |                      |           |
| c.           |             |           |       |       |                      |           |
| mpg_centered |             |           |       |       |                      |           |
| Foreign      | 78.88826    | 112.4812  | 0.70  | 0.485 | -145.4485            | 303.225   |
| _cons        | 5588.295    | 369.0945  | 15.14 | 0.000 | 4852.159             | 6324.431  |

. margins foreign, at(mpg\_centered=(-3 (1) 3))

Adjusted predictions

Number of obs = 74

Model VCE: OLS

Expression: Linear prediction, predict()

1.\_at: mpg\_centered = -3  
 2.\_at: mpg\_centered = -2  
 3.\_at: mpg\_centered = -1  
 4.\_at: mpg\_centered = 0  
 5.\_at: mpg\_centered = 1  
 6.\_at: mpg\_centered = 2  
 7.\_at: mpg\_centered = 3

|             | Delta-method |           |       |       |                      |          |
|-------------|--------------|-----------|-------|-------|----------------------|----------|
|             | Margin       | std. err. | t     | P> t  | [95% conf. interval] |          |
| _at#foreign |              |           |       |       |                      |          |
| 1#Domestic  | 6576.06      | 370.446   | 17.75 | 0.000 | 5837.229             | 7314.891 |
| 1#Foreign   | 8005.915     | 766.8178  | 10.44 | 0.000 | 6476.545             | 9535.284 |
| 2#Domestic  | 6246.805     | 354.4734  | 17.62 | 0.000 | 5539.83              | 6953.78  |
| 2#Foreign   | 7755.548     | 709.9327  | 10.92 | 0.000 | 6339.632             | 9171.464 |
| 3#Domestic  | 5917.55      | 354.0032  | 16.72 | 0.000 | 5211.513             | 6623.587 |
| 3#Foreign   | 7505.181     | 658.8306  | 11.39 | 0.000 | 6191.185             | 8819.177 |
| 4#Domestic  | 5588.295     | 369.0945  | 15.14 | 0.000 | 4852.159             | 6324.431 |
| 4#Foreign   | 7254.814     | 614.9548  | 11.80 | 0.000 | 6028.325             | 8481.303 |
| 5#Domestic  | 5259.04      | 397.981   | 13.21 | 0.000 | 4465.292             | 6052.788 |
| 5#Foreign   | 7004.447     | 579.9479  | 12.08 | 0.000 | 5847.778             | 8161.117 |
| 6#Domestic  | 4929.785     | 437.9413  | 11.26 | 0.000 | 4056.338             | 5803.231 |
| 6#Foreign   | 6754.081     | 555.4891  | 12.16 | 0.000 | 5646.192             | 7861.969 |

## 1. Interpreting interaction in a regression model

|            |  |          |          |       |       |          |          |
|------------|--|----------|----------|-------|-------|----------|----------|
| 7#Domestic |  | 4600.53  | 486.253  | 9.46  | 0.000 | 3630.729 | 5570.331 |
| 7#Foreign  |  | 6503.714 | 543.0057 | 11.98 | 0.000 | 5420.723 | 7586.704 |

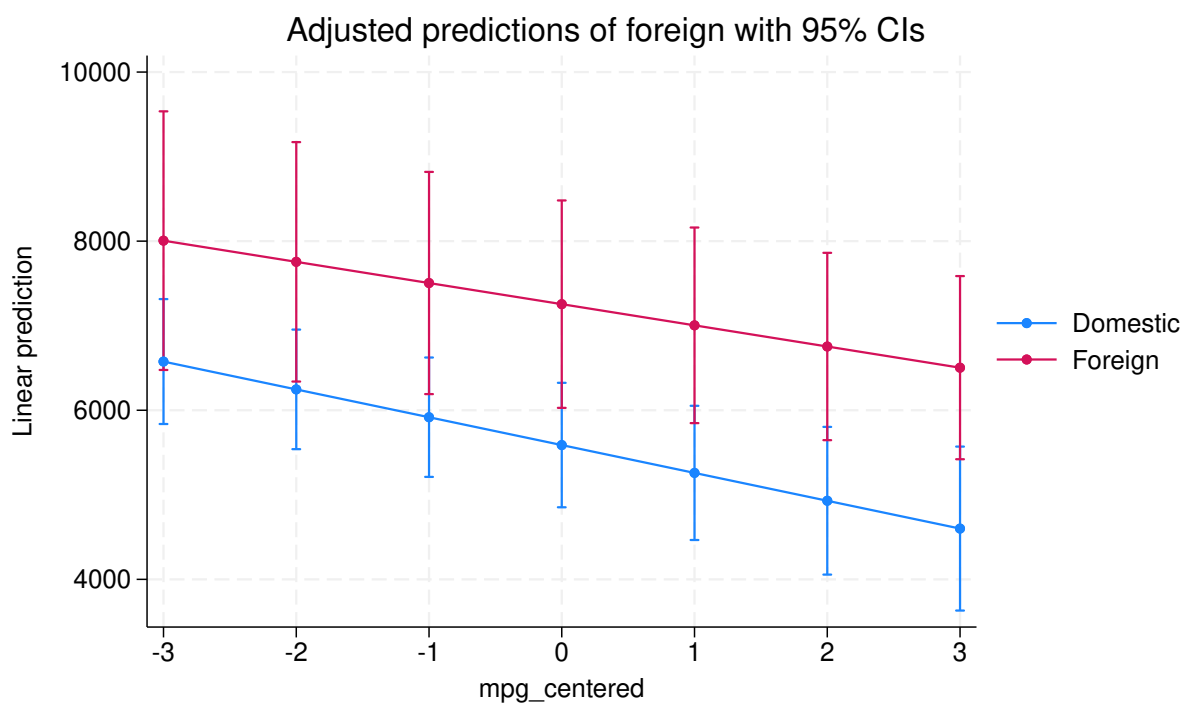
---

```
. marginsplot
```

Variables that uniquely identify margins: mpg\_centered foreign

```
. graph export "marginsplot-interaction.svg", as(svg) replace
file marginsplot-interaction.svg saved as SVG format
```

.



Mean mpg is 21.3, so the centred variable runs from  $-9.3$  to  $19.7$  and has mean zero to seven decimals. That is what makes the coefficients readable. The constant of \$5,588 is the predicted price of a domestic car at *average* fuel economy. In the uncentred model it would instead be the predicted price at  $\text{mpg} = 0$ , which is nine standard deviations below the smallest car in the data and has no meaning.

The **foreign** coefficient is 1666.5 with a standard error of 717.2 and  $p = 0.023$ : at average mpg, imports cost about \$1,667 more than domestic cars. The slope on centred mpg is  $-329.3$  with a standard error of 75.0, so among domestic cars each extra mile per gallon is worth about \$329 less in price — efficient cars in 1978 were the cheap ones. The interaction is 78.9 with a standard error of 112.5 and  $p = 0.485$ , so the mpg slope for imports ( $-329.3 + 78.9 = -250.4$ ) is not distinguishable from the domestic slope.

### *1.2. Interaction with continuous variables*

The graph shows predicted price for foreign and domestic cars across the range of mpg. Both lines slope down, the import line sits above the domestic one, and because the interaction is small and insignificant the two are close to parallel. The visible gap between them is the \$1,667 main effect, not evidence of a differing slope.



## 2. Interaction term in a non-linear model

### 2.1. The issue

In a linear model with interaction,

$$E(y) = \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2, \quad (2.1)$$

the cross partial derivative is simply the coefficient:

$$\frac{\partial^2 E(y)}{\partial x_1 \partial x_2} = \beta_{12}. \quad (2.2)$$

In a nonlinear model — logit, Poisson, or any link function  $F$  — the same coefficient  $\beta_{12}$  sits on the *link* scale:

$$F(E(y)) = \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2. \quad (2.3)$$

The cross partial of  $F(E(y))$  is still  $\beta_{12}$ . But the cross partial of  $E(y)$  itself — the quantity most researchers want — is not:

$$\frac{\partial^2 E(y)}{\partial x_1 \partial x_2} = \beta_{12} G'(\eta) + (\beta_1 + \beta_{12} x_2)(\beta_2 + \beta_{12} x_1) G''(\eta), \quad (2.4)$$

where  $G = F^{-1}$  and  $\eta$  is the linear predictor. This is the point of [Ai and Norton \(2003\)](#): the probability-scale interaction effect depends on every coefficient, varies across the covariate space, and can differ in sign and significance from  $\beta_{12}$ .

The deeper issue is that nonlinearity alone generates a nonzero cross partial — even without an interaction term. Drop  $\beta_{12}$  and the cross partial on  $E(y)$  is still  $\beta_1 \beta_2 G''(\eta) \neq 0$  whenever the link is curved. The marginal effect of  $x_1$  depends on  $x_2$  in any nonlinear model, interaction or not.

Greene (*Econometric Analysis*, 8th ed., sec. 17.3) and [Buis \(2010\)](#) draw the same conclusion: the model is specified on the link scale, and that is where coefficients and their tests are interpretable. In a logit,  $\beta_{12}$  is the interaction on log-odds; exponentiating gives an odds-ratio interaction that is multiplicative. Partial effects on the probability scale are *implications* of the model, not parameters of it — they should be computed from the fitted model (via `margins` or `marginaleffects`), not read off the coefficient table.

## 2.2. A worked example

Let's illustrate the Ai-Norton point directly: in a logit model with an interaction term, the cross partial derivative  $\frac{\partial^2 E(y)}{\partial x_1 \partial x_2}$  (what we actually care about) is not the same object as the estimated coefficient  $\beta_{12}$  on  $x_1 x_2$  in the linear index. We simulate data with a true interaction effect on the log-odds scale, fit a logit model, and then use the `marginalEffects` package for two distinct things: first the marginal effect of  $x_1$  on the probability scale at several values of  $x_2$ , and then the cross partial derivative itself.

We draw  $n = 2000$  observations with  $x_1, x_2 \sim N(0, 1)$ , independent of each other. The linear index is  $\eta = -0.3 + 0.8x_1 + 0.5x_2 + 0.6x_1x_2$ , the success probability is  $p = \Lambda(\eta)$ , and  $y \sim \text{Bernoulli}(p)$ . So the true interaction on the log-odds scale is  $\beta_{12} = 0.6$ , a single constant — which is exactly the object the coefficient table estimates, and exactly the object that does *not* transfer to the probability scale.

```
library(dplyr)
library(marginalEffects)

set.seed(42)
n <- 2000
x1 <- rnorm(n)
x2 <- rnorm(n)
eta <- -0.3 + 0.8 * x1 + 0.5 * x2 + 0.6 * x1 * x2
p <- plogis(eta)
y <- rbinom(n, 1, p)
dat <- data.frame(y, x1, x2)

m <- glm(y ~ x1 * x2, data = dat, family = binomial)
summary(m)$coefficients
```

|             | Estimate   | Std. Error | z value   | Pr(> z )     |
|-------------|------------|------------|-----------|--------------|
| (Intercept) | -0.2348658 | 0.05017155 | -4.681255 | 2.851237e-06 |
| x1          | 0.7792602  | 0.05930028 | 13.140920 | 1.918795e-39 |
| x2          | 0.5296149  | 0.05537644 | 9.563903  | 1.133977e-21 |
| x1:x2       | 0.6624955  | 0.06403580 | 10.345705 | 4.376826e-25 |

The coefficient on `x1:x2` is 0.662 with a standard error of 0.064, against a true 0.6 — recovered to within one standard error, as it should be, since the logit is correctly specified. The other three are 0.779, 0.530 and  $-0.235$  against true values of 0.8, 0.5 and  $-0.3$ .

That coefficient is  $\hat{\beta}_{12}$ , the interaction effect on the log-odds (linear index) scale — constant by construction. On the *probability* scale nothing is constant. Start with the marginal effect of  $x_1$  itself, evaluated at three values of  $x_2$ :

```
slopes(m, variables = "x1", by = "x2", newdata = datagrid(x2 = c(-1, 0, 1)))
```



| x2 | Estimate | Std. Error | z     | Pr(> z ) | S     | 2.5 %    | 97.5 % |
|----|----------|------------|-------|----------|-------|----------|--------|
| -1 | 0.0253   | 0.0159     | 1.59  | 0.112    | 3.2   | -0.00591 | 0.0565 |
| 0  | 0.1919   | 0.0147     | 13.09 | <0.001   | 127.6 | 0.16314  | 0.2206 |
| 1  | 0.3538   | 0.0243     | 14.59 | <0.001   | 157.7 | 0.30630  | 0.4014 |

Term: x1

Type: response

Comparison: dY/dX

The marginal effect of  $x_1$  on  $P(y = 1)$  is 0.025 at  $x_2 = -1$ , 0.192 at  $x_2 = 0$ , and 0.354 at  $x_2 = 1$  — a fourteen-fold change across the three values, where the log-odds interaction is one fixed number. Notice too that at  $x_2 = -1$  the effect is not significant ( $p = 0.112$ ) even though  $\hat{\beta}_{12}$  is significant at better than  $10^{-24}$ . Already the probability-scale answer and the coefficient tell different stories.

The cross partial derivative is a *further* step: not the effect of  $x_1$  at a given  $x_2$ , but how much that effect changes as  $x_2$  moves. That is a difference of slopes, so it needs a hypothesis test on top of the `by` argument rather than a second call to `slopes()`:

```
slopes(m, variables = "x1", by = "x2",
       newdata = datagrid(x2 = c(-1, 1)),
       hypothesis = "b2 - b1 = 0")
```

| Hypothesis | Estimate | Std. Error | z  | Pr(> z ) | S    | 2.5 % | 97.5 % |
|------------|----------|------------|----|----------|------|-------|--------|
| b2-b1=0    | 0.329    | 0.0299     | 11 | <0.001   | 90.7 | 0.27  | 0.387  |

Type: response

The cross partial is 0.329 with a standard error of 0.030,  $z = 11$  — that is the difference between the slope at  $x_2 = 1$  and the slope at  $x_2 = -1$ ,  $0.354 - 0.025$ , with a proper standard error attached. It is large and highly significant, which is worth being honest about: in *this* example it agrees with  $\hat{\beta}_{12}$  in significance, differing only in scale and magnitude. Note that it is nowhere near 0.662; the two numbers live on different scales and there is no reason to expect them to match. Ai and Norton's warning is that the two quantities *need not* agree — not that they never do. The dependable lesson is the one visible in the table above: on the probability scale the answer depends on where you evaluate it, so a single coefficient cannot summarise it.



## 3. Chow test and more

### 3.1. Chow test

To compare a coefficient across two subsamples, interact the subsample indicator with the covariates and run one pooled regression. If only the treatment is interacted, all other coefficients are constrained equal across subsamples; interacting everything relaxes that. This is the Chow test. (The overlapping-samples extension below follows Austin Nichols's Statalist code.)

The data are Stata's `nlsw88`, an extract from the 1988 National Longitudinal Survey of Young Women. With non-missing wage and hours, 938 of the women live in the South and 1,304 elsewhere, 2,242 in all. We regress hourly `wage` on weekly `hours`, and ask whether the slope differs by region. The chunk runs that four ways — two separate regressions, `suest`, and the pooled interacted regression — so the equivalences are visible side by side.

```
est clear
sysuse nlsw88, clear
reg wage hours if south
est sto south
reg wage hours if !south
est sto nonsouth
suest south nonsouth
est sto suest
gen hours1=hours*(south==1)
gen hours2=hours*(south==0)
reg wage south hours?
est sto chow
test _b[hours1]-_b[hours2]=0
esttab south nonsouth suest chow, nogaps mti
```

(NLSW, 1988 extract)

| Source   | SS         | df  | MS         | Number of obs | = | 938    |
|----------|------------|-----|------------|---------------|---|--------|
| Model    | 344.732583 | 1   | 344.732583 | F(1, 936)     | = | 12.47  |
| Residual | 25866.3404 | 936 | 27.634979  | Prob > F      | = | 0.0004 |
| Total    | 26211.0729 | 937 | 27.973397  | R-squared     | = | 0.0132 |
|          |            |     |            | Adj R-squared | = | 0.0121 |
|          |            |     |            | Root MSE      | = | 5.2569 |

| wage | Coefficient | Std. err. | t | P> t | [95% conf. interval] |
|------|-------------|-----------|---|------|----------------------|
|------|-------------|-----------|---|------|----------------------|

### 3. Chow test and more

|       |  |          |          |      |       |          |          |
|-------|--|----------|----------|------|-------|----------|----------|
| hours |  | .0623497 | .0176532 | 3.53 | 0.000 | .0277053 | .0969941 |
| _cons |  | 4.520583 | .6957145 | 6.50 | 0.000 | 3.155242 | 5.885923 |

|          |  |            |       |            |               |   |        |
|----------|--|------------|-------|------------|---------------|---|--------|
| Source   |  | SS         | df    | MS         | Number of obs | = | 1,304  |
|          |  |            |       |            | F(1, 1302)    | = | 55.93  |
| Model    |  | 1929.41943 | 1     | 1929.41943 | Prob > F      | = | 0.0000 |
| Residual |  | 44919.1023 | 1,302 | 34.5000785 | R-squared     | = | 0.0412 |
|          |  |            |       |            | Adj R-squared | = | 0.0404 |
| Total    |  | 46848.5217 | 1,303 | 35.9543528 | Root MSE      | = | 5.8737 |

|       |  |             |           |      |       |                      |
|-------|--|-------------|-----------|------|-------|----------------------|
| wage  |  | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |
| hours |  | .1107536    | .01481    | 7.48 | 0.000 | .0816995 .1398076    |
| _cons |  | 4.357811    | .564756   | 7.72 | 0.000 | 3.24988 5.465743     |

Simultaneous results for south, nonsouth

Number of obs = 2,242

|              |  |             |           |       |       |                      |
|--------------|--|-------------|-----------|-------|-------|----------------------|
|              |  | Robust      |           |       |       |                      |
|              |  | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |
| south_mean   |  |             |           |       |       |                      |
| hours        |  | .0623497    | .0174426  | 3.57  | 0.000 | .0281629 .0965366    |
| _cons        |  | 4.520583    | .6599379  | 6.85  | 0.000 | 3.227128 5.814037    |
| south_lnvar  |  |             |           |       |       |                      |
| _cons        |  | 3.319082    | .1435305  | 23.12 | 0.000 | 3.037768 3.600397    |
| nonsouth_m~n |  |             |           |       |       |                      |
| hours        |  | .1107536    | .0134936  | 8.21  | 0.000 | .0843066 .1372006    |
| _cons        |  | 4.357811    | .4633326  | 9.41  | 0.000 | 3.449696 5.265927    |
| nonsouth_l~r |  |             |           |       |       |                      |
| _cons        |  | 3.540962    | .1006119  | 35.19 | 0.000 | 3.343766 3.738157    |

(4 missing values generated)

(4 missing values generated)

|        |  |    |    |    |               |   |       |
|--------|--|----|----|----|---------------|---|-------|
| Source |  | SS | df | MS | Number of obs | = | 2,242 |
|--------|--|----|----|----|---------------|---|-------|

|             |  |            |                  |               |   |        |
|-------------|--|------------|------------------|---------------|---|--------|
| -----+----- |  |            |                  | F(3, 2238)    | = | 36.91  |
| Model       |  | 3502.37892 | 3 1167.45964     | Prob > F      | = | 0.0000 |
| Residual    |  | 70785.4426 | 2,238 31.6288841 | R-squared     | = | 0.0471 |
| -----+----- |  |            |                  | Adj R-squared | = | 0.0459 |
| Total       |  | 74287.8215 | 2,241 33.1494072 | Root MSE      | = | 5.624  |

|             |  |             |           |      |       |                      |
|-------------|--|-------------|-----------|------|-------|----------------------|
| -----+----- |  |             |           |      |       |                      |
| wage        |  | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |
| -----+----- |  |             |           |      |       |                      |
| south       |  | .1627711    | .9199871  | 0.18 | 0.860 | -1.641346 1.966888   |
| hours1      |  | .0623497    | .0188858  | 3.30 | 0.001 | .0253142 .0993852    |
| hours2      |  | .1107536    | .0141803  | 7.81 | 0.000 | .0829456 .1385616    |
| _cons       |  | 4.357811    | .5407453  | 8.06 | 0.000 | 3.297397 5.418226    |
| -----+----- |  |             |           |      |       |                      |

( 1) hours1 - hours2 = 0

F( 1, 2238) = 4.20  
 Prob > F = 0.0405

|              |           |          |           |           |
|--------------|-----------|----------|-----------|-----------|
| -----+-----  |           |          |           |           |
|              | (1)       | (2)      | (3)       | (4)       |
|              | south     | nonsouth | suest     | chow      |
| -----+-----  |           |          |           |           |
| main         |           |          |           |           |
| hours        | 0.0623*** | 0.111*** | 0.0623*** |           |
|              | (3.53)    | (7.48)   | (3.57)    |           |
| south        |           |          |           | 0.163     |
|              |           |          |           | (0.18)    |
| hours1       |           |          |           | 0.0623*** |
|              |           |          |           | (3.30)    |
| hours2       |           |          |           | 0.111***  |
|              |           |          |           | (7.81)    |
| _cons        | 4.521***  | 4.358*** | 4.521***  | 4.358***  |
|              | (6.50)    | (7.72)   | (6.85)    | (8.06)    |
| -----+-----  |           |          |           |           |
| south_lnvar  |           |          |           |           |
| _cons        |           |          | 3.319***  |           |
|              |           |          | (23.12)   |           |
| -----+-----  |           |          |           |           |
| nonsouth_m~n |           |          |           |           |
| hours        |           |          | 0.111***  |           |
|              |           |          | (8.21)    |           |
| _cons        |           |          | 4.358***  |           |
|              |           |          | (9.41)    |           |

### 3. Chow test and more

```

-----
nonsouth_1~r
_cons                      3.541***
                           (35.19)
-----
N                938                1304                2242                2242
-----
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001

```

The separate regressions give an hours slope of 0.0623 in the South and 0.1108 elsewhere. The pooled regression with `hours1` and `hours2` returns 0.0623497 and 0.1107536 — the *same numbers to seven digits*, not merely similar ones. That is algebra, not luck: a fully interacted regression is the two subsample regressions written as one.

What does differ is the standard errors. The South slope carries 0.0177 in its own regression and 0.0189 in the pooled one, because the pooled model estimates a single error variance across both groups while the separate regressions each get their own (root MSE 5.26 in the South, 5.87 elsewhere). If that pooling is unattractive, `suest` avoids it: it stacks the two stored estimates with their joint covariance and reports robust standard errors, 0.0174 and 0.0135 here.

`test _b[hours1]-_b[hours2]=0` is the Chow test, and it gives  $F(1, 2238) = 4.20$  with  $p = 0.0405$ . The returns to hours are steeper outside the South, and the gap of 0.048 dollars per hour is just significant at 5%. Note that the `south` dummy itself is 0.163 with  $p = 0.860$  — the two regions differ in slope, not in intercept, which is precisely the pattern a treatment-only interaction would have missed. Stata's `suest` does the same job by stacking the two stored estimates and their covariance structure; the interaction approach is more flexible since it works with any estimator.

## 3.2. A comparison with two different outcomes

The same idea compares a coefficient across two different outcomes. Stack the two outcome variables into a long format, interact the covariates with the stacking indicator, and run one regression.

The chunk below does that in seven steps, and it is worth reading them in order because the payoff is the last two columns of the table agreeing.

1. `reg south wage hours` and `reg smsa wage hours`, each stored with `est sto`. These are the two equations we want to compare, fitted separately.
2. `suest south smsa` refits them jointly and stacks the two coefficient vectors into one, with a robust covariance matrix across equations. This is the benchmark: it is the standard way to test a cross-equation hypothesis.
3. `preserve`, because the next steps reshape the data and we want it back.
4. `gen Y1=south, gen Y2=smsa, gen id=_n`. Copy the two outcomes into a numbered pair and give every woman an id.
5. `reshape long Y, i(id) j(subsample)` turns each woman's one row into two: one carrying `Y = south` with `subsample == 1`, one carrying `Y = smsa` with `subsample == 2`. The stacked outcome is now a single variable `Y`.

6. `gen wage1=wage*(subsample==1)` and its three siblings. Each covariate is split into the part that acts in equation 1 and the part that acts in equation 2. A single regression on `wage1 wage2 hours1 hours2` therefore estimates a separate slope per equation, which is what the two separate regressions did.
7. `reg Y wage? hours? subsample, cluster(id)` fits it, and `test _b[wage1]-_b[wage2]=0` is the cross-equation test. The `?` is a Stata wildcard, so `wage?` expands to `wage1 wage2`.

The whole construction exists so that a hypothesis *across* two equations becomes a hypothesis *within* one, where `test` can reach both coefficients.

```
est clear
sysuse nlsw88, clear
reg south wage hours
est sto south
reg smsa wage hours
est sto smsa
suest south smsa
est sto suest
preserve
gen Y1=south
gen Y2=smsa
gen id=_n
reshape long Y, i(id) j(subsample)
gen wage1=wage*(subsample==1)
gen wage2=wage*(subsample==2)
gen hours1=hours*(subsample==1)
gen hours2=hours*(subsample==2)
* Each individual contributes TWO rows here (one per stacked outcome), so the
* errors are correlated within id. Clustering on the stacking id is what makes
* the cross-equation test comparable to suest -- see the note below.
reg Y wage? hours? subsample, cluster(id)
test _b[wage1]-_b[wage2]=0
est sto stacked
* Match the preserve above, so the reshape does not leak into later chunks.
restore
* Note: suest stores coefficients under equation-qualified names (e.g.
* south:hours), while the single-sample models store the same
* coefficient simply as "hours"; esttab matches by exact coefficient
* name, so these land on separate rows instead of being aligned. Use
* esttab's coeqlabel()/eqlabel() options, or rename coefficients on the
* stored estimates first, to get a directly comparable table.
esttab south smsa suest stacked, nogaps mti
```

(NLSW, 1988 extract)

|        |  |    |    |    |               |   |       |
|--------|--|----|----|----|---------------|---|-------|
| Source |  | SS | df | MS | Number of obs | = | 2,242 |
|--------|--|----|----|----|---------------|---|-------|

### 3. Chow test and more

|          |            |       |            |               |   |        |
|----------|------------|-------|------------|---------------|---|--------|
|          |            |       |            | F(2, 2239)    | = | 30.60  |
| Model    | 14.513685  | 2     | 7.25684251 | Prob > F      | = | 0.0000 |
| Residual | 531.049205 | 2,239 | .237181423 | R-squared     | = | 0.0266 |
|          |            |       |            | Adj R-squared | = | 0.0257 |
| Total    | 545.56289  | 2,241 | .24344618  | Root MSE      | = | .48701 |

| south | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|-------|-------|----------------------|----------|
| wage  | -.0124053   | .0018099  | -6.85 | 0.000 | -.0159545            | -.008856 |
| hours | .0047722    | .0009916  | 4.81  | 0.000 | .0028277             | .0067167 |
| _cons | .3372108    | .0387354  | 8.71  | 0.000 | .2612497             | .4131718 |

|          |            |       |            |               |   |        |
|----------|------------|-------|------------|---------------|---|--------|
| Source   | SS         | df    | MS         | Number of obs | = | 2,242  |
|          |            |       |            | F(2, 2239)    | = | 36.17  |
| Model    | 14.6274602 | 2     | 7.31373012 | Prob > F      | = | 0.0000 |
| Residual | 452.719551 | 2,239 | .202197209 | R-squared     | = | 0.0313 |
|          |            |       |            | Adj R-squared | = | 0.0304 |
| Total    | 467.347012 | 2,241 | .208543959 | Root MSE      | = | .44966 |

| smsa  | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|-------|-------|----------------------|----------|
| wage  | .0139103    | .0016711  | 8.32  | 0.000 | .0106332             | .0171873 |
| hours | .0003663    | .0009155  | 0.40  | 0.689 | -.0014291            | .0021616 |
| _cons | .5820585    | .0357648  | 16.27 | 0.000 | .511923              | .6521941 |

Simultaneous results for south, smsa

Number of obs = 2,242

|             | Coefficient | Robust std. err. | z       | P> z  | [95% conf. interval] |           |
|-------------|-------------|------------------|---------|-------|----------------------|-----------|
| south_mean  |             |                  |         |       |                      |           |
| wage        | -.0124053   | .0019287         | -6.43   | 0.000 | -.0161854            | -.0086251 |
| hours       | .0047722    | .0009825         | 4.86    | 0.000 | .0028465             | .0066978  |
| _cons       | .3372108    | .0379918         | 8.88    | 0.000 | .2627483             | .4116733  |
| south_lnvar |             |                  |         |       |                      |           |
| _cons       | -1.43893    | .0096645         | -148.89 | 0.000 | -1.457872            | -1.419988 |
| smsa_mean   |             |                  |         |       |                      |           |
| wage        | .0139103    | .0018313         | 7.60    | 0.000 | .010321              | .0174996  |
| hours       | .0003663    | .0009099         | 0.40    | 0.687 | -.0014172            | .0021497  |



### 3.2. A comparison with two different outcomes

|            |  |           |          |        |       |           |           |
|------------|--|-----------|----------|--------|-------|-----------|-----------|
| _cons      |  | .5820585  | .0361037 | 16.12  | 0.000 | .5112965  | .6528205  |
| -----      |  |           |          |        |       |           |           |
| smsa_lnvar |  |           |          |        |       |           |           |
| _cons      |  | -1.598512 | .0195103 | -81.93 | 0.000 | -1.636751 | -1.560272 |
| -----      |  |           |          |        |       |           |           |

(j = 1 2)

|                        |  |       |    |           |
|------------------------|--|-------|----|-----------|
| Data                   |  | Wide  | -> | Long      |
| -----                  |  |       |    |           |
| Number of observations |  | 2,246 | -> | 4,492     |
| Number of variables    |  | 23    | -> | 23        |
| j variable (2 values)  |  |       | -> | subsample |
| xij variables:         |  | Y1 Y2 | -> | Y         |
| -----                  |  |       |    |           |

(8 missing values generated)

(8 missing values generated)

|                   |  |               |   |        |
|-------------------|--|---------------|---|--------|
| Linear regression |  | Number of obs | = | 4,484  |
|                   |  | F(5, 2241)    | = | 96.34  |
|                   |  | Prob > F      | = | 0.0000 |
|                   |  | R-squared     | = | 0.1091 |
|                   |  | Root MSE      | = | .46871 |

(Std. err. adjusted for 2,242 clusters in id)

|           |  |             |           |       |       |                       |
|-----------|--|-------------|-----------|-------|-------|-----------------------|
|           |  |             | Robust    |       |       |                       |
| Y         |  | Coefficient | std. err. | t     | P> t  | [95% conf. interval]  |
| -----     |  |             |           |       |       |                       |
| wage1     |  | -.0124053   | .0019298  | -6.43 | 0.000 | -.0161896    -.008621 |
| wage2     |  | .0139103    | .0018323  | 7.59  | 0.000 | .010317    .0175035   |
| hours1    |  | .0047722    | .000983   | 4.85  | 0.000 | .0028444    .0066999  |
| hours2    |  | .0003663    | .0009104  | 0.40  | 0.688 | -.0014191    .0021517 |
| subsample |  | .2448477    | .0548754  | 4.46  | 0.000 | .1372358    .3524597  |
| _cons     |  | .092363     | .0872219  | 1.06  | 0.290 | -.0786812    .2634072 |
| -----     |  |             |           |       |       |                       |

### 3. Chow test and more

( 1) wage1 - wage2 = 0

F( 1, 2241) = 72.14  
 Prob > F = 0.0000

|             | (1)<br>south          | (2)<br>smsa         | (3)<br>suest           | (4)<br>stacked        |
|-------------|-----------------------|---------------------|------------------------|-----------------------|
| main        |                       |                     |                        |                       |
| wage        | -0.0124***<br>(-6.85) | 0.0139***<br>(8.32) | -0.0124***<br>(-6.43)  |                       |
| hours       | 0.00477***<br>(4.81)  | 0.000366<br>(0.40)  | 0.00477***<br>(4.86)   |                       |
| wage1       |                       |                     |                        | -0.0124***<br>(-6.43) |
| wage2       |                       |                     |                        | 0.0139***<br>(7.59)   |
| hours1      |                       |                     |                        | 0.00477***<br>(4.85)  |
| hours2      |                       |                     |                        | 0.000366<br>(0.40)    |
| subsample   |                       |                     |                        | 0.245***<br>(4.46)    |
| _cons       | 0.337***<br>(8.71)    | 0.582***<br>(16.27) | 0.337***<br>(8.88)     | 0.0924<br>(1.06)      |
| south_lnvar |                       |                     |                        |                       |
| _cons       |                       |                     | -1.439***<br>(-148.89) |                       |
| smsa_mean   |                       |                     |                        |                       |
| wage        |                       |                     | 0.0139***<br>(7.60)    |                       |
| hours       |                       |                     | 0.000366<br>(0.40)     |                       |
| _cons       |                       |                     | 0.582***<br>(16.12)    |                       |
| smsa_lnvar  |                       |                     |                        |                       |
| _cons       |                       |                     | -1.599***<br>(-81.93)  |                       |
| N           | 2242                  | 2242                | 2242                   | 4484                  |

---

t statistics in parentheses  
 \* p<0.05, \*\* p<0.01, \*\*\* p<0.001

Read the table by columns. The first two are the separate regressions, the third is **suest**, and the fourth is the stacked regression. The **wage1** coefficient in column four equals the **wage** coefficient in column one, and **wage2** equals the **wage** coefficient in column two: stacking has reproduced both single-equation fits exactly, in one regression, with both coefficients now addressable by **test**. Note the sample size, 4,484 in the stacked column against 2,242 in the others — the reshape doubled the rows, which is the next point.

One detail matters a great deal here, and it is easy to skip. Stacking puts **two rows per person** in the data, so the two error terms belonging to the same person are correlated by construction. The cross-equation test is precisely a comparison *across* those two rows, so its standard error has to account for that correlation. Clustering on the stacking id does it, and it is not a cosmetic adjustment:

|                               | test of the wage coefficient across equations |
|-------------------------------|-----------------------------------------------|
| <b>suest</b>                  | $\chi^2(1) = 72.22$                           |
| stacked, <b>no</b> clustering | $F(1, 4478) = 114.12$                         |
| stacked, <b>cluster(id)</b>   | $F(1, 2241) = 72.14$                          |

Without clustering the statistic is inflated by more than half. With it, the stacked test reproduces **suest** almost exactly — the small remaining gap is just the  $F$ -versus- $\chi^2$  finite-sample adjustment. Note that **robust** alone will *not* fix this; the cluster variable has to be the id you stacked on.

### 3.3. Overlapping samples

The same method extends to overlapping subsamples. Here **south** and **smsa** overlap — some observations belong to both — so a simple split is not possible.

The reshape trick from the previous section will not work here, because a woman who is both southern and in an SMSA has to appear in both subsamples rather than be assigned to one. **expand** handles that instead:

1. **ta south smsa** shows the overlap, which is the reason for what follows.
2. **preserve**, then **expand 2** duplicates every row, so each woman now has two copies.
3. **bys idcode: g n=\_n** numbers a woman's two copies 1 and 2.
4. **keep if (n==1&south) | (n==2&smsa)** keeps copy 1 only if she is southern and copy 2 only if she is in an SMSA. A woman in both keeps both copies; a woman in neither drops out; a woman in exactly one keeps one. Each surviving row now represents one woman's membership in one subsample.
5. **g hours1=hours\*(n==1)** and **g hours2=hours\*(n==2)** split the covariate by subsample, as before. Within this restricted sample **n==1** means southern and **n==2** means SMSA, so the copy number *is* the subsample indicator.

### 3. Chow test and more

6. `reg wage hours? n, cl(idcode)` fits both slopes at once, clustering on the woman rather than on a stacking id, since a woman can now contribute one row or two.

```

est clear
sysuse nlsw88, clear
ta south smsa
reg wage hours if south
est sto south
reg wage hours if smsa
est sto smsa
suest south smsa
est sto suest
preserve
expand 2
bys idcode: g n=_n
keep if (n==1&south)|(n==2&smsa)
* hours1 is hours in the south subsample (n==1), hours2 is hours in the
* smsa subsample (n==2); within this restricted sample n==1 <=> south==1
* and n==2 <=> smsa==1, so this is equivalent to but clearer than negating
* the complementary condition.
g hours1=hours*(n==1)
g hours2=hours*(n==2)
reg wage hours? n, cl(idcode)
est sto stacked
restore
esttab south smsa suest stacked, nogaps mti

```

(NLSW, 1988 extract)

| Lives in          | Lives in SMSA |       |       |
|-------------------|---------------|-------|-------|
| the south         | Not SMSA      | SMSA  | Total |
| -----+-----+----- |               |       |       |
| Not south         | 308           | 996   | 1,304 |
| South             | 357           | 585   | 942   |
| -----+-----+----- |               |       |       |
| Total             | 665           | 1,581 | 2,246 |

| Source            | SS         | df  | MS         | Number of obs | = | 938    |
|-------------------|------------|-----|------------|---------------|---|--------|
| -----+-----+----- |            |     |            |               |   |        |
| Model             | 344.732583 | 1   | 344.732583 | F(1, 936)     | = | 12.47  |
| Residual          | 25866.3404 | 936 | 27.634979  | Prob > F      | = | 0.0004 |
| -----+-----+----- |            |     |            |               |   |        |
|                   |            |     |            | R-squared     | = | 0.0132 |
|                   |            |     |            | Adj R-squared | = | 0.0121 |
| Total             | 26211.0729 | 937 | 27.973397  | Root MSE      | = | 5.2569 |

### 3.3. Overlapping samples

| wage  | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|------|-------|----------------------|----------|
| hours | .0623497    | .0176532  | 3.53 | 0.000 | .0277053             | .0969941 |
| _cons | 4.520583    | .6957145  | 6.50 | 0.000 | 3.155242             | 5.885923 |

| Source   | SS         | df    | MS         | Number of obs | = | 1,578  |
|----------|------------|-------|------------|---------------|---|--------|
|          |            |       |            | F(1, 1576)    | = | 46.48  |
| Model    | 1594.14881 | 1     | 1594.14881 | Prob > F      | = | 0.0000 |
| Residual | 54048.2539 | 1,576 | 34.2945773 | R-squared     | = | 0.0286 |
|          |            |       |            | Adj R-squared | = | 0.0280 |
| Total    | 55642.4027 | 1,577 | 35.2837049 | Root MSE      | = | 5.8562 |

| wage  | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|------|-------|----------------------|----------|
| hours | .0953636    | .0139872  | 6.82 | 0.000 | .0679281             | .1227991 |
| _cons | 4.861519    | .5443826  | 8.93 | 0.000 | 3.793729             | 5.929309 |

Simultaneous results for south, smsa

Number of obs = 1,934

|             |             | Robust    |       |       |                      |          |
|-------------|-------------|-----------|-------|-------|----------------------|----------|
|             | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |          |
| south_mean  |             |           |       |       |                      |          |
| hours       | .0623497    | .0174432  | 3.57  | 0.000 | .0281617             | .0965378 |
| _cons       | 4.520583    | .6599613  | 6.85  | 0.000 | 3.227082             | 5.814083 |
| south_lnvar |             |           |       |       |                      |          |
| _cons       | 3.319082    | .1435356  | 23.12 | 0.000 | 3.037758             | 3.600407 |
| smsa_mean   |             |           |       |       |                      |          |
| hours       | .0953636    | .0132806  | 7.18  | 0.000 | .069334              | .1213931 |
| _cons       | 4.861519    | .4842914  | 10.04 | 0.000 | 3.912325             | 5.810713 |
| smsa_lnvar  |             |           |       |       |                      |          |
| _cons       | 3.534987    | .0910825  | 38.81 | 0.000 | 3.356469             | 3.713506 |

(2,246 observations created)

### 3. Chow test and more

(1,969 observations deleted)

(7 missing values generated)

(7 missing values generated)

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 2,516  |
|                   | F(3, 1933)    | = | 40.81  |
|                   | Prob > F      | = | 0.0000 |
|                   | R-squared     | = | 0.0399 |
|                   | Root MSE      | = | 5.6403 |

(Std. err. adjusted for 1,934 clusters in idcode)

|        |      | Robust      |           |      |       |                      |
|--------|------|-------------|-----------|------|-------|----------------------|
|        | wage | Coefficient | std. err. | t    | P> t  | [95% conf. interval] |
| hours1 |      | .0623497    | .0174536  | 3.57 | 0.000 | .0281198 .0965796    |
| hours2 |      | .0953636    | .0132886  | 7.18 | 0.000 | .0693022 .121425     |
| n      |      | .3409364    | .6124145  | 0.56 | 0.578 | -.8601261 1.541999   |
| _cons  |      | 4.179646    | 1.177889  | 3.55 | 0.000 | 1.869579 6.489713    |

|             | (1)                 | (2)                 | (3)                 | (4)                 |
|-------------|---------------------|---------------------|---------------------|---------------------|
|             | south               | smsa                | suest               | stacked             |
| main        |                     |                     |                     |                     |
| hours       | 0.0623***<br>(3.53) | 0.0954***<br>(6.82) | 0.0623***<br>(3.57) |                     |
| hours1      |                     |                     |                     | 0.0623***<br>(3.57) |
| hours2      |                     |                     |                     | 0.0954***<br>(7.18) |
| n           |                     |                     |                     | 0.341<br>(0.56)     |
| _cons       | 4.521***<br>(6.50)  | 4.862***<br>(8.93)  | 4.521***<br>(6.85)  | 4.180***<br>(3.55)  |
| south_lnvar |                     |                     |                     |                     |
| _cons       |                     |                     | 3.319***<br>(23.12) |                     |
| smsa_mean   |                     |                     |                     |                     |

|                                  |     |      |           |      |
|----------------------------------|-----|------|-----------|------|
| hours                            |     |      | 0.0954*** |      |
|                                  |     |      | (7.18)    |      |
| _cons                            |     |      | 4.862***  |      |
|                                  |     |      | (10.04)   |      |
| -----                            |     |      |           |      |
| smsa_lnvar                       |     |      |           |      |
| _cons                            |     |      | 3.535***  |      |
|                                  |     |      | (38.81)   |      |
| -----                            |     |      |           |      |
| N                                | 938 | 1578 | 1934      | 2516 |
| -----                            |     |      |           |      |
| t statistics in parentheses      |     |      |           |      |
| * p<0.05, ** p<0.01, *** p<0.001 |     |      |           |      |

The cross-tab shows the overlap: of the 1,304 non-South women, 996 live in an SMSA, so the two subsamples share members and cannot be produced by a simple split. Expanding the data and keeping each observation under whichever subsample it belongs to handles that. The pooled `hours1` is 0.0623497, again reproducing the South regression exactly, and `hours2` is 0.0954 for the SMSA subsample. The `n` indicator is 0.341 with  $p = 0.578$ .

### 3.4. IV regression

The interaction approach works with IV. Interact both the endogenous variable and the instrument with the subsample indicator. Here `hours` is instrumented by `union` within each region.

The point is not to get a different estimate. The two interacted 2SLS coefficients reproduce the two subsample IV regressions exactly, just as they did for OLS. The point is that running them separately leaves the two estimates in different `ivregress` results with no joint covariance between them, so there is nothing to test the difference *with*. Interacting puts both in one estimation, so `test _b[hours1] - _b[hours2] = 0` can reach both coefficients and their covariance. That is the whole reason for the construction, and it is the same reason as in the OLS case.

Two details specific to IV. The instrument must be split alongside the endogenous variable — `union1` and `union2`, not just `hours1` and `hours2` — because an instrument that is not subsample-specific would identify a single pooled first stage and defeat the purpose. And the number of instruments must still match the number of endogenous regressors: two of each here, so the system is exactly identified, as each subsample regression was on its own.

```
sysuse nlsw88, clear
ivregress 2sls wage (hours=union) if south
ivregress 2sls wage (hours=union) if !south
gen hours1=hours*(south==1)
gen hours2=hours*(south==0)
gen union1=union*(south==1)
gen union2=union*(south==0)
```

### 3. Chow test and more

```
ivregress 2sls wage south (hours1 hours2 = union1 union2)
test _b[hours1]-_b[hours2]=0
```

(NLSW, 1988 extract)

|                                        |               |   |        |
|----------------------------------------|---------------|---|--------|
| Instrumental-variables 2SLS regression | Number of obs | = | 798    |
|                                        | Wald chi2(1)  | = | 2.61   |
|                                        | Prob > chi2   | = | 0.1060 |
|                                        | Root MSE      | = | 9.5807 |

| -----+----- |             |           |       |       |                      |          |
|-------------|-------------|-----------|-------|-------|----------------------|----------|
| wage        | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |          |
| -----+----- |             |           |       |       |                      |          |
| hours       | .97819      | .6050822  | 1.62  | 0.106 | -.2077492            | 2.164129 |
| _cons       | -30.96026   | 23.32846  | -1.33 | 0.184 | -76.6832             | 14.76268 |
| -----+----- |             |           |       |       |                      |          |

Endogenous: hours

Exogenous: union

|                                        |               |   |        |
|----------------------------------------|---------------|---|--------|
| Instrumental-variables 2SLS regression | Number of obs | = | 1,079  |
|                                        | Wald chi2(1)  | = | 6.03   |
|                                        | Prob > chi2   | = | 0.0140 |
|                                        | Root MSE      | = | 7.0372 |

| -----+----- |             |           |       |       |                      |          |
|-------------|-------------|-----------|-------|-------|----------------------|----------|
| wage        | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |          |
| -----+----- |             |           |       |       |                      |          |
| hours       | .6255843    | .2546731  | 2.46  | 0.014 | .1264342             | 1.124735 |
| _cons       | -14.9159    | 9.401508  | -1.59 | 0.113 | -33.34252            | 3.510714 |
| -----+----- |             |           |       |       |                      |          |

Endogenous: hours

Exogenous: union

(4 missing values generated)

(4 missing values generated)

(368 missing values generated)

(368 missing values generated)

|                                        |               |   |        |
|----------------------------------------|---------------|---|--------|
| Instrumental-variables 2SLS regression | Number of obs | = | 1,877  |
|                                        | Wald chi2(3)  | = | 21.75  |
|                                        | Prob > chi2   | = | 0.0001 |



Root MSE = 8.2154

|        | wage | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |          |
|--------|------|-------------|-----------|-------|-------|----------------------|----------|
| hours1 |      | .97819      | .5188511  | 1.89  | 0.059 | -.0387394            | 1.995119 |
| hours2 |      | .6255843    | .297311   | 2.10  | 0.035 | .0428654             | 1.208303 |
| south  |      | -16.04435   | 22.81705  | -0.70 | 0.482 | -60.76495            | 28.67625 |
| _cons  |      | -14.9159    | 10.97553  | -1.36 | 0.174 | -36.42754            | 6.595735 |

Endogenous: hours1 hours2

Exogenous: south union1 union2

( 1) hours1 - hours2 = 0

chi2( 1) = 0.35  
Prob > chi2 = 0.5554

The IV estimates are 0.978 in the South and 0.626 elsewhere, against OLS values of 0.062 and 0.111 — an order of magnitude larger, with standard errors to match (0.519 and 0.297). And the Chow test now gives  $\chi^2(1) = 0.35$  with  $p = 0.5554$ , where the OLS version rejected at  $p = 0.0405$ . Instrumenting has not overturned the difference between regions; it has made the data too imprecise to detect it. The lesson is about the test, not the subsamples: a Chow test inherits the precision of whatever estimator it is built on.

### 3.5. IV with fixed effects

With fixed effects the pooled regression needs the fixed effects themselves interacted with the subsample indicator. The first attempt below omits this and fails to reproduce the separate estimates:

#### Warning

The examples below cluster standard errors on **race**, which in **nls88** has only three categories (White, Black, Other). Cluster-robust standard errors rely on asymptotics in the number of clusters  $G \rightarrow \infty$ ; with  $G = 3$  they are severely downward-biased, inflating t-statistics and false-positive rates. These chunks use **race** purely to illustrate the fixed-effect/IV mechanics — in an actual analysis, cluster on a variable with many more categories (e.g., **industry** or **occupation**), or on the true level of treatment assignment/sampling design.

```
sysuse nls88, clear
gen hours1=hours*(south==1)
gen hours2=hours*(south==0)
gen union1=union*(south==1)
```

### 3. Chow test and more

```
gen union2=union*(south==0)
ivreghdfe wage (hours=union) if south, a(race) cluster(race)
ivreghdfe wage (hours=union) if !south, a(race) cluster(race)
ivreghdfe wage south (hours1 hours2 = union1 union2) , a(race) cluster(race)
```

(NLSW, 1988 extract)

(4 missing values generated)

(4 missing values generated)

(368 missing values generated)

(368 missing values generated)

(MWFE estimator converged in 1 iterations)

IV (2SLS) estimation

-----

Estimates efficient for homoskedasticity only

Statistics robust to heteroskedasticity and clustering on race

|                             |             |                 |         |
|-----------------------------|-------------|-----------------|---------|
| Number of clusters (race) = | 3           | Number of obs = | 798     |
|                             |             | F( 1, 2) =      | 69.28   |
|                             |             | Prob > F =      | 0.0141  |
| Total (centered) SS =       | 12186.8806  | Centered R2 =   | -6.4527 |
| Total (uncentered) SS =     | 12186.8806  | Uncentered R2 = | -6.4527 |
| Residual SS =               | 90825.44631 | Root MSE =      | 10.68   |

|  |       | Robust      |           |      |       |                      |
|--|-------|-------------|-----------|------|-------|----------------------|
|  | wage  | Coefficient | std. err. | t    | P> t  | [95% conf. interval] |
|  | hours | 1.107099    | .133012   | 8.32 | 0.014 | .5347947 1.679404    |

Underidentification test (Kleibergen-Paap rk LM statistic): 1.756  
Chi-sq(1) P-val = 0.1852

Weak identification test (Cragg-Donald Wald F statistic): 3.233  
(Kleibergen-Paap rk Wald F statistic): 13.977  
Stock-Yogo weak ID test critical values: 10% maximal IV size 16.38  
15% maximal IV size 8.96  
20% maximal IV size 6.66  
25% maximal IV size 5.53

Source: Stock-Yogo (2005). Reproduced by permission.

NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.

```
-----
Hansen J statistic (overidentification test of all instruments):      0.000
                                                                    (equation exactly identified)
-----
```

```
Instrumented:      hours
Excluded instruments: union
Partialled-out:    _cons
                   nb: total SS, model F and R2s are after partialling-out;
                       any small-sample adjustments include partialled-out
                       variables in regressor count K
-----
```

Absorbed degrees of freedom:

```
-----+-----
Absorbed FE | Categories - Redundant = Num. Coefs |
-----+-----+-----
           race |           3           3           0   *|
-----+-----+-----
```

\* = FE nested within cluster; treated as redundant for DoF computation

(MWFE estimator converged in 1 iterations)

IV (2SLS) estimation

Estimates efficient for homoskedasticity only

Statistics robust to heteroskedasticity and clustering on race

```
Number of clusters (race) =           3           Number of obs =       1079
                                                                    F(   1,   2) =       1.41
                                                                    Prob > F       =       0.3564
Total (centered) SS       =  19086.22115           Centered R2      = -2.3302
Total (uncentered) SS    =  19086.22115           Uncentered R2    = -2.3302
Residual SS              =  63560.97528           Root MSE        =       7.682
```

```
-----
           |               Robust
           | Coefficient std. err.      t    P>|t|    [95% conf. interval]
-----+-----
hours |   .7023322   .5905772     1.19  0.356   -1.838717    3.243381
-----+-----
```

```
Underidentification test (Kleibergen-Paap rk LM statistic):      0.789
                                                                    Chi-sq(1) P-val =      0.3745
-----
```

```
Weak identification test (Cragg-Donald Wald F statistic):      5.501
                                                                    (Kleibergen-Paap rk Wald F statistic):      3.772
Stock-Yogo weak ID test critical values: 10% maximal IV size    16.38
                                           15% maximal IV size    8.96
```

### 3. Chow test and more

20% maximal IV size 6.66  
25% maximal IV size 5.53

Source: Stock-Yogo (2005). Reproduced by permission.

NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.

Hansen J statistic (overidentification test of all instruments): 0.000  
(equation exactly identified)

Instrumented: hours  
Excluded instruments: union  
Partialled-out: \_cons  
nb: total SS, model F and R2s are after partialling-out;  
any small-sample adjustments include partialled-out  
variables in regressor count K

Absorbed degrees of freedom:

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| race        | 3          | 3           | 0 *          |

\* = FE nested within cluster; treated as redundant for DoF computation

(MWFE estimator converged in 1 iterations)

IV (2SLS) estimation

Estimates efficient for homoskedasticity only

Statistics robust to heteroskedasticity and clustering on race

|                             |             |                 |         |
|-----------------------------|-------------|-----------------|---------|
| Number of clusters (race) = | 3           | Number of obs = | 1877    |
|                             |             | F( 3, 2) =      | 1036.24 |
|                             |             | Prob > F =      | 0.0010  |
| Total (centered) SS =       | 32142.319   | Centered R2 =   | -3.9041 |
| Total (uncentered) SS =     | 32142.319   | Uncentered R2 = | -3.9041 |
| Residual SS =               | 157627.5718 | Root MSE =      | 9.174   |

|        | Robust      |           |       |       |                      |          |
|--------|-------------|-----------|-------|-------|----------------------|----------|
| wage   | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |          |
| hours1 | 1.142036    | .0650507  | 17.56 | 0.003 | .8621454             | 1.421927 |
| hours2 | .6880691    | .5265185  | 1.31  | 0.321 | -1.577357            | 2.953495 |
| south  | -19.74773   | 22.10811  | -0.89 | 0.466 | -114.8712            | 75.37577 |

Underidentification test (Kleibergen-Paap rk LM statistic): 1.793

```

Chi-sq(1) P-val =    0.1806
-----
Weak identification test (Cragg-Donald Wald F statistic):    3.584
                    (Kleibergen-Paap rk Wald F statistic):    8.484
Stock-Yogo weak ID test critical values: 10% maximal IV size    7.03
                    15% maximal IV size    4.58
                    20% maximal IV size    3.95
                    25% maximal IV size    3.63

Source: Stock-Yogo (2005).  Reproduced by permission.
NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.
-----
Warning: estimated covariance matrix of moment conditions not of full rank.
        overidentification statistic not reported, and standard errors and
        model tests should be interpreted with caution.
Possible causes:
        number of clusters insufficient to calculate robust covariance matrix
        singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)
partial option may address problem.
-----
Instrumented:          hours1 hours2
Included instruments:  south
Excluded instruments: union1 union2
Partialled-out:        _cons
                    nb: total SS, model F and R2s are after partialling-out;
                        any small-sample adjustments include partialled-out
                        variables in regressor count K
-----

Absorbed degrees of freedom:
-----+-----
Absorbed FE | Categories - Redundant = Num. Coefs |
-----+-----+-----+-----+
      race |           3           3           0    *|
-----+-----+-----+-----+
* = FE nested within cluster; treated as redundant for DoF computation

```

The separate regressions give 1.1071 and 0.7023. The pooled regression gives 1.1420 and 0.6881 — close, but not equal, and that is the point. The pooled coefficients do not match the separate regressions because the fixed effects are shared: one set of race effects is estimated off both subsamples at once, so neither subsample gets the within-group demeaning its own regression used. Interacting them with the subsample indicator fixes it:

```

sysuse nlsw88, clear
gen hours1=hours*(south==1)
gen hours2=hours*(south==0)
gen union1=union*(south==1)
gen union2=union*(south==0)

```

### 3. Chow test and more

```
gen race1=race*(south==1)
gen race2=race*(south==0)

ivreghdfe wage (hours=union) if south, a(race) cluster(race)
ivreghdfe wage (hours=union) if !south, a(race) cluster(race)
ivregress 2sls wage south (hours1 hours2 = union1 union2) i.race1 i.race2, cluster(race)
```

Warning: estimated covariance matrix of moment conditions not of full rank.  
overidentification statistic not reported, and standard errors and  
model tests should be interpreted with caution.

Possible causes:

number of clusters insufficient to calculate robust covariance matrix  
singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)  
partial option may address problem.

(NLSW, 1988 extract)

(4 missing values generated)

(4 missing values generated)

(368 missing values generated)

(368 missing values generated)

(MWFE estimator converged in 1 iterations)

IV (2SLS) estimation

-----

Estimates efficient for homoskedasticity only

Statistics robust to heteroskedasticity and clustering on race

|                             |             |                 |         |
|-----------------------------|-------------|-----------------|---------|
| Number of clusters (race) = | 3           | Number of obs = | 798     |
|                             |             | F( 1, 2) =      | 69.28   |
|                             |             | Prob > F =      | 0.0141  |
| Total (centered) SS =       | 12186.8806  | Centered R2 =   | -6.4527 |
| Total (uncentered) SS =     | 12186.8806  | Uncentered R2 = | -6.4527 |
| Residual SS =               | 90825.44631 | Root MSE =      | 10.68   |

-----

| Robust

```

      wage | Coefficient  std. err.      t    P>|t|     [95% conf. interval]
-----+-----
      hours |    1.107099    .133012    8.32   0.014    .5347947    1.679404
-----+-----
Underidentification test (Kleibergen-Paap rk LM statistic):          1.756
                                                    Chi-sq(1) P-val =    0.1852
-----+-----
Weak identification test (Cragg-Donald Wald F statistic):          3.233
      (Kleibergen-Paap rk Wald F statistic):          13.977
Stock-Yogo weak ID test critical values: 10% maximal IV size      16.38
                                           15% maximal IV size      8.96
                                           20% maximal IV size      6.66
                                           25% maximal IV size      5.53
Source: Stock-Yogo (2005).  Reproduced by permission.
NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.
-----+-----
Hansen J statistic (overidentification test of all instruments):    0.000
      (equation exactly identified)
-----+-----
Instrumented:      hours
Excluded instruments: union
Partialled-out:    _cons
                  nb: total SS, model F and R2s are after partialling-out;
                  any small-sample adjustments include partialled-out
                  variables in regressor count K
-----+-----

Absorbed degrees of freedom:
-----+-----
Absorbed FE | Categories  - Redundant  = Num. Coefs |
-----+-----+-----
      race |          3          3          0    *|
-----+-----+-----
* = FE nested within cluster; treated as redundant for DoF computation

(MWFE estimator converged in 1 iterations)

IV (2SLS) estimation
-----+-----

Estimates efficient for homoskedasticity only
Statistics robust to heteroskedasticity and clustering on race

Number of clusters (race) =          3          Number of obs =      1079
                                           F( 1,      2) =      1.41
                                           Prob > F      =      0.3564
Total (centered) SS      = 19086.22115          Centered R2      = -2.3302
Total (uncentered) SS    = 19086.22115          Uncentered R2    = -2.3302

```

### 3. Chow test and more

Residual SS = 63560.97528                      Root MSE = 7.682

|  |       | Robust      |           |      |       |                       |
|--|-------|-------------|-----------|------|-------|-----------------------|
|  | wage  | Coefficient | std. err. | t    | P> t  | [95% conf. interval]  |
|  | hours | .7023322    | .5905772  | 1.19 | 0.356 | -1.838717    3.243381 |

Underidentification test (Kleibergen-Paap rk LM statistic): 0.789  
Chi-sq(1) P-val = 0.3745

Weak identification test (Cragg-Donald Wald F statistic): 5.501  
(Kleibergen-Paap rk Wald F statistic): 3.772  
Stock-Yogo weak ID test critical values: 10% maximal IV size 16.38  
15% maximal IV size 8.96  
20% maximal IV size 6.66  
25% maximal IV size 5.53

Source: Stock-Yogo (2005). Reproduced by permission.

NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.

Hansen J statistic (overidentification test of all instruments): 0.000  
(equation exactly identified)

Instrumented: hours  
Excluded instruments: union  
Partialled-out: \_cons  
nb: total SS, model F and R2s are after partialling-out;  
any small-sample adjustments include partialled-out  
variables in regressor count K

Absorbed degrees of freedom:

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| race        | 3          | 3           | 0 *          |

\* = FE nested within cluster; treated as redundant for DoF computation

note: 3.race1 omitted because of collinearity.

note: 3.race2 omitted because of collinearity.

Instrumental-variables 2SLS regression                      Number of obs = 1,877  
Wald chi2(7) = 101696.08  
Prob > chi2 = 0.0000  
Root MSE = 9.0693

(Std. err. adjusted for 3 clusters in race)



|        |  | Coefficient | Robust<br>std. err. | z     | P> z  | [95% conf. interval] |           |
|--------|--|-------------|---------------------|-------|-------|----------------------|-----------|
| wage   |  |             |                     |       |       |                      |           |
| hours1 |  | 1.107099    | .1085357            | 10.20 | 0.000 | .8943732             | 1.319825  |
| hours2 |  | .7023322    | .4819806            | 1.46  | 0.145 | -.2423324            | 1.646997  |
| south  |  | -21.52299   | 22.35225            | -0.96 | 0.336 | -65.33259            | 22.28662  |
| race1  |  |             |                     |       |       |                      |           |
| 1      |  | 3.054296    | .1451752            | 21.04 | 0.000 | 2.769758             | 3.338834  |
| 2      |  | 1.987268    | .176538             | 11.26 | 0.000 | 1.64126              | 2.333277  |
| 3      |  | 0           | (omitted)           |       |       |                      |           |
| race2  |  |             |                     |       |       |                      |           |
| 1      |  | -.4816127   | .434723             | -1.11 | 0.268 | -1.333654            | .3704287  |
| 2      |  | -2.065527   | .7694573            | -2.68 | 0.007 | -3.573635            | -.5574181 |
| 3      |  | 0           | (omitted)           |       |       |                      |           |
| _cons  |  | -17.01633   | 18.01689            | -0.94 | 0.345 | -52.32879            | 18.29614  |

Endogenous: hours1 hours2

Exogenous: south 1.race1 2.race1 1.race2 2.race2 union1 union2

This reproduces the separate estimates exactly: 1.107099 and 0.7023322 in the pooled regression against 1.107099 and 0.7023322 separately. The approach builds explicit dummies from the interaction of the fixed-effect variable and the subsample indicator, which becomes impractical when the number of fixed-effect levels is large.

One caution about how those dummies were built. `race1 = race*(south==1)` multiplies a *categorical code* by an indicator, so non-south observations are assigned `race1 = 0` and then `i.race1` treats 0 as its own category. That is safe here only because `race` has no zero level (it is coded 1/2/3). If the absorbed variable did have a 0 category, those observations and all the non-south ones would be silently collapsed into the same dummy. For a general recipe, build the interactions from the category dummies directly (`i.race#i.south`) rather than from the code.

Alternatively, pass the two interaction variables to `reghdfe`'s `absorb` option as a two-way fixed effect:

```
sysuse nlsw88, clear
gen hours1=hours*(south==1)
gen hours2=hours*(south==0)
gen union1=union*(south==1)
gen union2=union*(south==0)
gen race1=race*(south==1)
gen race2=race*(south==0)

ivreghdfe wage south (hours1 hours2 = union1 union2) , a(race1 race2) cluster(race)
test _b[hours1]-_b[hours2]=0
```

### 3. Chow test and more

Warning: estimated covariance matrix of moment conditions not of full rank.  
overidentification statistic not reported, and standard errors and  
model tests should be interpreted with caution.

Possible causes:

number of clusters insufficient to calculate robust covariance matrix  
singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)  
partial option may address problem.

(NLSW, 1988 extract)

(4 missing values generated)

(4 missing values generated)

(368 missing values generated)

(368 missing values generated)

(MWFE estimator converged in 2 iterations)

IV (2SLS) estimation

Estimates efficient for homoskedasticity only

Statistics robust to heteroskedasticity and clustering on race

|                             |             |                 |          |
|-----------------------------|-------------|-----------------|----------|
| Number of clusters (race) = | 3           | Number of obs = | 1877     |
|                             |             | F( 2, 2) =      | 13010.62 |
|                             |             | Prob > F =      | 0.0001   |
| Total (centered) SS =       | 31273.10174 | Centered R2 =   | -3.9367  |
| Total (uncentered) SS =     | 31273.10174 | Uncentered R2 = | -3.9367  |
| Residual SS =               | 154386.4216 | Root MSE =      | 9.089    |

|        | Robust      |           |      |       |                      |
|--------|-------------|-----------|------|-------|----------------------|
| wage   | Coefficient | std. err. | t    | P> t  | [95% conf. interval] |
| hours1 | 1.107099    | .1331773  | 8.31 | 0.014 | .5340838 1.680115    |
| hours2 | .7023322    | .5914076  | 1.19 | 0.357 | -1.84229 3.246954    |
| south  | 0 (omitted) |           |      |       |                      |

Underidentification test (Kleibergen-Paap rk LM statistic): 0.000  
Chi-sq(1) P-val = 1.0000

Weak identification test (Cragg-Donald Wald F statistic): 3.793

```

(Kleibergen-Paap rk Wald F statistic):          0.000
Stock-Yogo weak ID test critical values: 10% maximal IV size      7.03
                                         15% maximal IV size      4.58
                                         20% maximal IV size      3.95
                                         25% maximal IV size      3.63

```

Source: Stock-Yogo (2005). Reproduced by permission.

NB: Critical values are for Cragg-Donald F statistic and i.i.d. errors.

```

-----
Warning: estimated covariance matrix of moment conditions not of full rank.
        overidentification statistic not reported, and standard errors and
        model tests should be interpreted with caution.

```

Possible causes:

```

        number of clusters insufficient to calculate robust covariance matrix
        singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)
partial option may address problem.
-----

```

Collinearities detected among instruments: 1 instrument(s) dropped

Instrumented: hours1 hours2

Included instruments: south

Excluded instruments: union1 union2

Partialled-out: \_cons

```

        nb: total SS, model F and R2s are after partialling-out;
        any small-sample adjustments include partialled-out
        variables in regressor count K
-----

```

Absorbed degrees of freedom:

```

-----+-----
Absorbed FE | Categories - Redundant = Num. Coefs |
-----+-----+-----
      race1 |           4           0           4   |
      race2 |           4           2           2   |
-----+-----+-----

```

( 1) hours1 - hours2 = 0

```

      F( 1,      2) =      0.31
      Prob > F =      0.6325

```

Absorbing `race1` and `race2` as a two-way fixed effect gives the same 1.107099 and 0.7023322 without building the dummies by hand, and scales to many fixed-effect levels. The Chow test is  $F(1, 2) = 0.31$  with  $p = 0.6325$  — but read that denominator: 2 degrees of freedom, because the standard errors are clustered on `race`, which has three categories. This is the problem the callout above flags, and it is why these IV/FE numbers illustrate mechanics rather than support any conclusion about the South.

### 3.6. Chow test with different covariates

The equations need not share the same covariates. If equation 1 includes  $X_2$  and equation 2 does not,

$$Y_1 = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon_1, \quad Y_2 = \gamma_0 + \gamma_1 X_1 + \epsilon_2, \quad (3.1)$$

the second equation implicitly sets the coefficient on  $X_2$  to zero. In the stacked regression, replace  $X_2$  with a constant (e.g. 1) for the  $Y_2$  subsample, so that its “coefficient” is absorbed into the intercept:

```
sysuse nlsw88, clear
reg south wage hours tenure
est sto south
reg smsa wage hours
est sto smsa
suest south smsa
est sto suest
preserve
gen Y1=south
gen Y2=smsa
gen id=_n
reshape long Y, i(id) j(subsample)
gen wage1=wage*(subsample==1)
gen wage2=wage*(subsample==2)
gen hours1=hours*(subsample==1)
gen hours2=hours*(subsample==2)
replace tenure=1 if subsample==2
gen tenure1= tenure*(subsample==1)
gen tenure2= tenure*(subsample==2)
reg Y wage? hours? tenure? subsample, cluster(id)
est sto chow
test _b[hours1]-_b[hours2]=0
esttab suest chow, nogaps mti
restore
```

Warning: estimated covariance matrix of moment conditions not of full rank.  
overidentification statistic not reported, and standard errors and  
model tests should be interpreted with caution.

Possible causes:

number of clusters insufficient to calculate robust covariance matrix  
singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)

partial option may address problem.

Warning: estimated covariance matrix of moment conditions not of full rank.  
overidentification statistic not reported, and standard errors and  
model tests should be interpreted with caution.

### 3.6. Chow test with different covariates

Possible causes:

number of clusters insufficient to calculate robust covariance matrix  
 singleton dummy variable (dummy with one 1 and N-1 0s or vice versa)  
 partial option may address problem.

(NLSW, 1988 extract)

| Source   | SS         | df    | MS         | Number of obs | = | 2,227  |
|----------|------------|-------|------------|---------------|---|--------|
|          |            |       |            | F(3, 2223)    | = | 20.30  |
| Model    | 14.4507388 | 3     | 4.81691294 | Prob > F      | = | 0.0000 |
| Residual | 527.507052 | 2,223 | .23729512  | R-squared     | = | 0.0267 |
|          |            |       |            | Adj R-squared | = | 0.0254 |
| Total    | 541.957791 | 2,226 | .243467112 | Root MSE      | = | .48713 |

| south  | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |           |
|--------|-------------|-----------|-------|-------|----------------------|-----------|
| wage   | -.0119584   | .0018359  | -6.51 | 0.000 | -.0155586            | -.0083583 |
| hours  | .004828     | .0010072  | 4.79  | 0.000 | .0028528             | .0068031  |
| tenure | -.0024032   | .0019221  | -1.25 | 0.211 | -.0061726            | .0013661  |
| _cons  | .346361     | .0392121  | 8.83  | 0.000 | .2694648             | .4232573  |

| Source   | SS         | df    | MS         | Number of obs | = | 2,242  |
|----------|------------|-------|------------|---------------|---|--------|
|          |            |       |            | F(2, 2239)    | = | 36.17  |
| Model    | 14.6274602 | 2     | 7.31373012 | Prob > F      | = | 0.0000 |
| Residual | 452.719551 | 2,239 | .202197209 | R-squared     | = | 0.0313 |
|          |            |       |            | Adj R-squared | = | 0.0304 |
| Total    | 467.347012 | 2,241 | .208543959 | Root MSE      | = | .44966 |

| smsa  | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|-------|-------|----------------------|----------|
| wage  | .0139103    | .0016711  | 8.32  | 0.000 | .0106332             | .0171873 |
| hours | .0003663    | .0009155  | 0.40  | 0.689 | -.0014291            | .0021616 |
| _cons | .5820585    | .0357648  | 16.27 | 0.000 | .511923              | .6521941 |

Simultaneous results for south, smsa

Number of obs = 2,242

|            |             | Robust    |   |      |                      |  |
|------------|-------------|-----------|---|------|----------------------|--|
|            | Coefficient | std. err. | z | P> z | [95% conf. interval] |  |
| south_mean |             |           |   |      |                      |  |

### 3. Chow test and more

|             |  |           |          |         |       |           |           |
|-------------|--|-----------|----------|---------|-------|-----------|-----------|
| wage        |  | -.0119584 | .001953  | -6.12   | 0.000 | -.0157862 | -.0081307 |
| hours       |  | .004828   | .0009971 | 4.84    | 0.000 | .0028737  | .0067822  |
| tenure      |  | -.0024032 | .0018928 | -1.27   | 0.204 | -.0061131 | .0013066  |
| _cons       |  | .346361   | .038473  | 9.00    | 0.000 | .2709554  | .4217667  |
| -----       |  |           |          |         |       |           |           |
| south_lnvar |  |           |          |         |       |           |           |
| _cons       |  | -1.438451 | .0096359 | -149.28 | 0.000 | -1.457337 | -1.419565 |
| -----       |  |           |          |         |       |           |           |
| smsa_mean   |  |           |          |         |       |           |           |
| wage        |  | .0139103  | .0018313 | 7.60    | 0.000 | .010321   | .0174996  |
| hours       |  | .0003663  | .0009099 | 0.40    | 0.687 | -.0014172 | .0021497  |
| _cons       |  | .5820585  | .0361037 | 16.12   | 0.000 | .5112965  | .6528205  |
| -----       |  |           |          |         |       |           |           |
| smsa_lnvar  |  |           |          |         |       |           |           |
| _cons       |  | -1.598512 | .0195103 | -81.93  | 0.000 | -1.636751 | -1.560272 |

$$(j = 1, 2)$$

| Data                   | Wide  | -> | Long      |
|------------------------|-------|----|-----------|
| Number of observations | 2,246 | -> | 4,492     |
| Number of variables    | 23    | -> | 23        |
| j variable (2 values)  |       | -> | subsample |
| xij variables:         | Y1 Y2 | -> | Y         |

(8 missing values generated)

(8 missing values generated)

(2,222 real changes made)

(15 missing values generated)

(15 missing values generated)

note: subsample omitted because of collinearity.

Linear regression                      Number of obs        =        4,469

### 3.6. Chow test with different covariates

```

F(6, 2241)      =      81.10
Prob > F        =      0.0000
R-squared       =      0.1091
Root MSE       =      .4687

```

(Std. err. adjusted for 2,242 clusters in id)

|           |             | Robust    |       |       |                      |          |
|-----------|-------------|-----------|-------|-------|----------------------|----------|
| Y         | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |          |
| wage1     | -.0119584   | .0019543  | -6.12 | 0.000 | -.0157909            | -.008126 |
| wage2     | .0139103    | .0018325  | 7.59  | 0.000 | .0103166             | .0175039 |
| hours1    | .004828     | .0009978  | 4.84  | 0.000 | .0028713             | .0067846 |
| hours2    | .0003663    | .0009105  | 0.40  | 0.688 | -.0014193            | .0021519 |
| tenure1   | -.0024032   | .0018941  | -1.27 | 0.205 | -.0061176            | .0013111 |
| tenure2   | .2356975    | .0550137  | 4.28  | 0.000 | .1278144             | .3435806 |
| subsample | 0           | (omitted) |       |       |                      |          |
| _cons     | .346361     | .0384988  | 9.00  | 0.000 | .270864              | .4218581 |

( 1) hours1 - hours2 = 0

```

F( 1, 2241) = 10.03
Prob > F = 0.0016

```

|         | (1)                   | (2)                   |
|---------|-----------------------|-----------------------|
|         | suest                 | chow                  |
| main    |                       |                       |
| wage    | -0.0120***<br>(-6.12) |                       |
| hours   | 0.00483***<br>(4.84)  |                       |
| tenure  | -0.00240<br>(-1.27)   |                       |
| wage1   |                       | -0.0120***<br>(-6.12) |
| wage2   |                       | 0.0139***<br>(7.59)   |
| hours1  |                       | 0.00483***<br>(4.84)  |
| hours2  |                       | 0.000366<br>(0.40)    |
| tenure1 |                       | -0.00240              |

### 3. Chow test and more

```

                                (-1.27)
tenure2                        0.236***
                                (4.28)
subsample                      0
                                (.)
_cons                        0.346***    0.346***
                        (9.00)    (9.00)
-----
south_lnvar
_cons                -1.438***
                    (-149.28)
-----
smsa_mean
wage                0.0139***
                    (7.60)
hours              0.000366
                    (0.40)
_cons              0.582***
                    (16.12)
-----
smsa_lnvar
_cons                -1.599***
                    (-81.93)
-----
N                2242                4469
-----
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001

```

The stacked model reproduces `suest`'s point estimates exactly: the hours coefficient is 0.004828 in the `south` equation and 0.0003663 in the `smsa` one, in both. Here the covariates differ — `tenure` appears in the first equation only — and the trick of setting `tenure = 1` for the second subsample lets its coefficient be absorbed by that equation's intercept. The Chow test on the hours coefficient gives  $F(1, 2241) = 10.03$  with  $p = 0.0016$ : hours predicts living in the South quite differently from how it predicts living in an SMSA. (The `esttab` columns do not line up because `suest` stores equation-qualified names like `south:wage` while the stacked model stores plain `wage`.)

## 3.7. Conclusion

`suest` and the stacking method both test cross-equation hypotheses with robust standard errors. `sureg` does the same under a GLS framework, but it assumes homoscedastic errors across equations; under misspecification it remains consistent but its standard errors are invalid. When the equations share covariates `sureg` and OLS give the same point estimates; with different covariates they diverge.



The stacking approach is the most portable. It works in any language, extends to IV and fixed-effect models, and requires only that the single-equation estimator works — no special post-estimation infrastructure.

One pattern runs through every example in this chapter, and it is worth stating once on its own. A Chow test is a hypothesis about two coefficients that live in two different regressions. Stata’s `test` can only reach coefficients inside a single stored estimation result, and, more importantly, testing a difference requires the covariance between the two estimates, which separate regressions never produce. Every construction here — the reshape for two outcomes, the `expand` for overlapping samples, the split instruments under IV, the absorbed fixed effects — exists to move both coefficients into one estimation so that the covariance exists and `test` can use it. `suest` does the same thing by a different route, refitting the stored estimates jointly with a robust cross-equation covariance matrix.

So the choice among them is practical rather than statistical. Use `suest` when the estimator supports it and the samples are the same. Stack when the samples overlap, when the covariates differ across equations, or when the estimator is one `suest` will not take — which, in practice, means most fixed-effect and IV commands.



## 4. Weights in OLS in causal inference

### 4.1. Unit level weights for OLS

This chapter follows Chad Hazlett and Tanvi Shinkre (2024), “Demystifying and avoiding the OLS ‘weighting problem’: unmodeled heterogeneity and straightforward solutions,” arXiv:2403.03299.

Under unconfoundedness, the identifying assumption holds at each level of  $X$ . When  $X$  is discrete and low-dimensional, we can stratify and compute the ATE by the g-formula. In practice we usually run a linear regression

$$Y = \alpha + \beta D + \gamma X + \epsilon \quad (4.1)$$

and treat  $\hat{\beta}$  as the treatment effect, relying on linearity to span the covariate space. The question is what this estimator is actually weighting.

Suppose we are interested in the ATE of  $D$  on  $Y$ . Say we have discrete variables  $X$ . Under unconfoundedness, the ATE is

$$\begin{aligned} \tau_{ATE} &= \sum E[Y(1) - Y(0)|X = x]P(X = x) \\ &= \sum (E[Y|D = 1, X = x] - E[Y|D = 0, X = x])P(X = x) \\ &= \sum DIM_x P(X = x) \end{aligned} \quad (4.2)$$

This is basically the g-formula, or regression adjustment formula. We compute the DIM (difference in means) between treated and control for each strata  $X = x$ , then the weighted average is the ATE, with  $\hat{P}(X = x)$  as weights.

If we have high dimensional  $X$ , we usually do

$$Y = \beta_0 + \beta_1 X + \delta_{reg} D + \epsilon \quad (4.3)$$

We would hope  $\delta_{reg}$  is close to  $\delta_{ATE}$ , but we are not sure.

OLS coefficients are:

$$\hat{\beta} = (Z'Z)^{-1}Z'y \quad (4.4)$$

Here I use  $Z$  to represent all regressors. Each coefficient is a weighted sum of the outcome,

$$\hat{\tau}_{reg} = \sum \omega_i Y_i \quad (4.5)$$

#### 4. Weights in OLS in causal inference

By FWL theorem,

$$\hat{\tau}_{reg} = \frac{\sum Y_i(D_i - \hat{d}(X_i))}{\sum (D_i - \hat{d}(X_i))^2} \quad (4.6)$$

Here  $\hat{d}(X_i) = \hat{p}(D_i = 1|X_i)$  which is the predicted value of linear regression of  $D$  on  $X$ . The numerator is the covariance between  $Y$  and residualized  $D$ , the denominator is the variance of the residualized  $D$ . Note that the original FWL would have residualized  $Y$  here, but it turns out it works the same with the original  $Y$ .

Therefore the weights are

$$\omega_i = \frac{D_i - \hat{d}(X_i)}{\sum (D_i - \hat{d}(X_i))^2} \quad (4.7)$$

Note for the denominator, it's a constant. It does *not* scale the weights to sum to one — they sum to **zero**, because  $D_i - \hat{d}(X_i)$  is an OLS residual from a regression that includes an intercept. What the denominator normalizes is the contrast: since  $\sum D_i(D_i - \hat{d}(X_i)) = \sum (D_i - \hat{d}(X_i))^2$  (the residual is orthogonal to the fitted values), we get  $\sum_i \omega_i D_i = 1$ , i.e. the weights on the treated sum to +1 and those on the control sum to -1. That is what makes  $\sum_i \omega_i Y_i$  a weighted *difference* of means. The numerator for treated units are  $1 - \hat{d}(X_i)$ , for control units it's  $-\hat{d}(X_i)$  (negative). It is similar to IPW weights, which are  $1/\hat{d}(X_i)$  for treated and  $1/(1 - \hat{d}(X_i))$  for control. These two sets of weights go the same direction. For the treated, the higher probability of being treated, the lower weight; for the control, the higher probability of being treated, the larger the magnitude of the (negative) weight.

If these weights are the same as the IPW weights, then we'll get  $\tau_{reg} = \tau_{ATE}$ . However, they are usually different.

Another way to write this as weighted difference in means:

$$\hat{\tau}_{wdim} = \sum \omega_i Y_i = \sum_{i:D=1} \tilde{\omega}_i Y_i + \sum_{i:D=0} \tilde{\omega}_i Y_i \quad (4.8)$$

where, with  $c = \sum (D_i - \hat{d}(X_i))^2$ , the per-arm weights are  $\tilde{\omega}_i = (1 - \hat{d}(X_i))/c$  for treated units ( $D_i = 1$ ) and  $\tilde{\omega}_i = -\hat{d}(X_i)/c$  for control units ( $D_i = 0$ )

Because  $\hat{d}(X_i)$  is from a linear probability model, it's possible that it's not between 0 and 1. In that case, we can have negative weights. The negative weights could mean the observations do not have a match in the other group. For example,  $\hat{d}(X_i) > 1$  will cause the weight  $(1 - \hat{d}(X_i))/c$  to flip sign and become negative for a treated unit. That means this unit is more like treated units than any control units. In other words, there is no match in the control group. The mirror image happens for a control unit with  $\hat{d}(X_i) < 0$ : its weight  $-\hat{d}(X_i)/c$  flips to *positive*, so it enters the contrast on the same side as the treated. Either way the unit contributes with the wrong sign.

The negative weights are invisible unless we examine  $\hat{d}(X_i)$  directly — which is to say, unless we look at the propensity score. A linear regression of  $Y$  on  $D$  and  $X$  hides the extrapolation. So  $\hat{\tau}_{reg}$  is a weighted average of outcomes with weights that differ from the ATE weights, and some of those weights may carry the wrong sign without the researcher knowing.

## 4.2. Strata level weights

Angrist (1998, *Econometrica*) has the strata-wise weights for OLS:

$$\hat{\tau}_{reg} = \frac{\sum_x \hat{\tau}_x \hat{d}(x)(1 - \hat{d}(x)) \hat{P}(X = x)}{\sum_x \hat{d}(x)(1 - \hat{d}(x)) \hat{P}(X = x)} \quad (4.9)$$

this shows that different from ATE, these weights are  $\hat{d}(x)(1 - \hat{d}(x)) \hat{P}(X = x)$ . The stratas are weighted not only by  $P(X_i = x)$ , but also by the variance of treatment status. When  $\hat{d}(X_i)$  is close to .5, the weight is the largest.

If there is no treatment effect heterogeneity, then it does not matter how you weight. When there is treatment effect heterogeneity, this estimator could be very different from ATE.

Hazlett and Shinkre show that Angrist's formula is a special case: it is exact when the probability of treatment is linear in  $X$  (as in a saturated model with discrete  $X$ ). They derive a more general formula, but the Angrist version suffices under linearity of  $d(X)$ .

## 4.3. Recommended estimation approaches

Under constant effects and linearity,  $E[Y | X, D] = \beta_0 + \beta_1 X + \beta_2 D$ , a regression of  $Y$  on  $D$  and  $X$  recovers the ATE without further adjustment. When the linearity or constant-effects assumption fails, Hazlett and Shinkre suggest relaxing to *separate* linearity — allowing different intercepts for  $Y(0)$  and  $Y(1)$ . Under that weaker assumption, several estimators are unbiased for the ATE.

The interaction (Lin 2013) estimator regresses  $Y$  on  $D$ ,  $\tilde{X} = X - \bar{X}$ , and  $D\tilde{X}$ ; the coefficient on  $D$  is the ATE. The imputation estimator fits separate regressions for treated and control, predicts counterfactual outcomes for the other group, and takes the average difference — algebraically, it returns the same estimate. Stratification and g-computation do the same for discrete  $X$ : compute the stratum-level difference in means and weight by the stratum probability.

All of these still rely on linearity within each arm. When that fails, matching (e.g. mean-balancing) or nonparametric estimation is needed.

## 4.4. A worked example

We make the regression weights  $\omega_i$  concrete and compare them to IPW weights  $1/\hat{d}(X_i)$  (treated) and  $1/(1 - \hat{d}(X_i))$  (control).

We draw  $n = 500$  observations with a single covariate  $X \sim N(0, 1)$ , treatment  $D \sim \text{Bernoulli}(\Lambda(0.5X))$  — so 238 units are treated — and outcome  $Y = 1 + 2D + 1.5X + \varepsilon$  with  $\varepsilon \sim N(0, 1)$ . The effect is constant at 2, so there is no heterogeneity for the weights to mis-average; the point of this first example is only to show that the weights reproduce the coefficient.

#### 4. Weights in OLS in causal inference

```
library(dplyr)

set.seed(1)
n <- 500
X <- rnorm(n)
d_prob <- plogis(0.5 * X)
D <- rbinom(n, 1, d_prob)
Y <- 1 + 2 * D + 1.5 * X + rnorm(n)
dat <- data.frame(Y, D, X)

# Regression-implied weights via the FWL residualization of D on X.
d_hat <- fitted(lm(D ~ X, data = dat))
resid_D <- dat$D - d_hat
omega_reg <- resid_D / sum(resid_D^2)

# IPW weights (Horvitz-Thompson), using the same linear-probability d_hat
# purely for comparability with the regression weights above.
ipw <- ifelse(dat$D == 1, 1 / d_hat, 1 / (1 - d_hat))
# rescale by a single constant so the treated weights average 1; this only
# fixes the axis scale of the plot below and does not change any comparison
ipw <- ipw / sum(ipw[dat$D == 1]) * sum(dat$D == 1)

tau_reg <- sum(omega_reg * dat$Y)
tau_lm <- coef(lm(Y ~ D + X, data = dat))["D"]

data.frame(tau_reg_from_weights = tau_reg, tau_from_lm_coef = tau_lm)
```

```
      tau_reg_from_weights tau_from_lm_coef
D                2.084644                2.084644
```

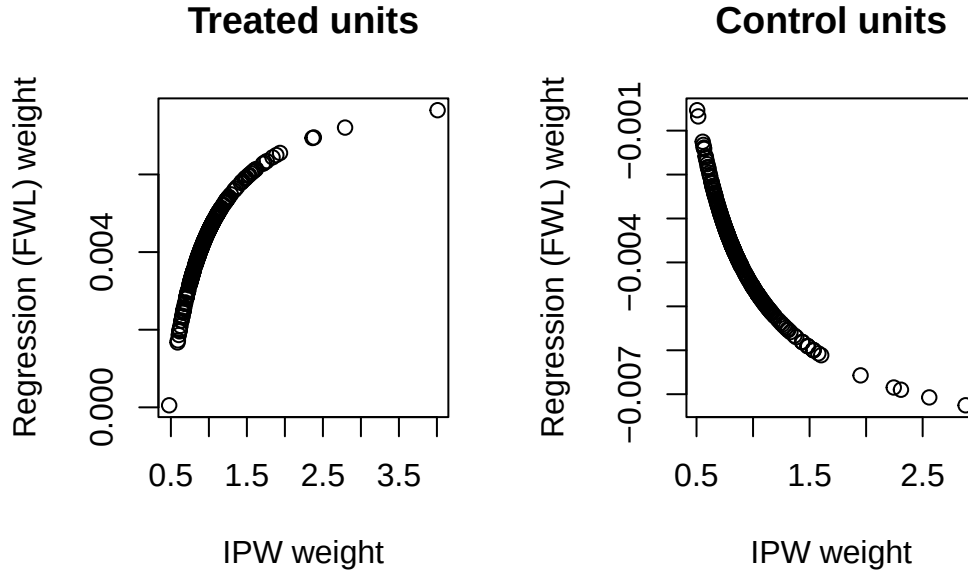
Both give 2.084644, identical to seven digits, confirming  $\hat{\tau}_{reg} = \sum \omega_i Y_i$ . (Both sit a little above the true 2; with  $n = 500$  and a constant effect that is sampling noise, not weighting.)

Plotting `omega_reg` against `ipw` for treated and control units separately shows the two weighting schemes move in the same direction but are not proportional.

Be careful about what that does and does not establish. Non-proportional weights are *necessary* for  $\hat{\tau}_{reg}$  to differ from the ATE, not sufficient. In this example the effect is constant at 2, so every unit's effect is the same number and any weights that form a difference of means return that number:  $\hat{\tau}_{reg}$  and the ATE coincide here no matter how oddly the weights are distributed. A weighting problem needs two ingredients, and this design has only one. The section after next supplies the other.

```
par(mfrow = c(1, 2))
plot(ipw[dat$D == 1], omega_reg[dat$D == 1],
     xlab = "IPW weight", ylab = "Regression (FWL) weight",
     main = "Treated units")
plot(ipw[dat$D == 0], omega_reg[dat$D == 0],
```

```
xlab = "IPW weight", ylab = "Regression (FWL) weight",
main = "Control units")
```



```
par(mfrow = c(1, 1))
```

Both panels slope upward and neither is a straight line through the origin: the two schemes rank units the same way but do not scale together, which is the whole content of  $\hat{\tau}_{reg} \neq \hat{\tau}_{ATE}$ .

Note also what this example does *not* show. Here  $\hat{d}(X_i)$  stays inside  $[0.062, 0.994]$ , so every weight keeps its expected sign and the negative-weight problem discussed above never materializes. That is a property of this particular design, not a reassurance. We now strengthen the selection from  $\Lambda(0.5X)$  to  $\Lambda(3X)$ , holding everything else fixed, and the linear probability model starts predicting outside the unit interval:

```
set.seed(2)
X2 <- rnorm(n)
D2 <- rbinom(n, 1, plogis(3 * X2)) # much stronger selection on X
dat2 <- data.frame(D = D2, X = X2)
d_hat2 <- fitted(lm(D ~ X, data = dat2))
omega2 <- (dat2$D - d_hat2) / sum((dat2$D - d_hat2)^2)

data.frame(
  dhat_below_0 = sum(d_hat2 < 0),
  dhat_above_1 = sum(d_hat2 > 1),
  treated_wrong_sign = sum(dat2$D == 1 & omega2 < 0),
  control_wrong_sign = sum(dat2$D == 0 & omega2 > 0)
)
```

#### 4. Weights in OLS in causal inference

|   | dhat_below_0 | dhat_above_1 | treated_wrong_sign | control_wrong_sign |
|---|--------------|--------------|--------------------|--------------------|
| 1 | 28           | 42           | 42                 | 28                 |

The fitted probabilities now run from  $-0.367$  to  $1.445$ . Twenty-eight units fall below zero and 42 rise above one, and the wrong-sign counts match those two numbers exactly: 42 treated units carry a negative weight and 28 control units carry a positive one. That is not a coincidence but the mechanism described above — every unit with  $\hat{d} > 1$  here is treated, every unit with  $\hat{d} < 0$  is control, and each flips the sign of its own contribution. In total 70 of 500 observations, 14% of the sample, enter the contrast on the wrong side.

Those units are the ones with no counterpart in the other arm, and a plain regression of  $Y$  on  $D$  and  $X$  gives you no indication that they are there. Nothing in the coefficient table changes appearance; only looking at  $\hat{d}(X_i)$  reveals it.

This is a property of the *regression* weights specifically, and the reason is worth being explicit about.  $\hat{d}(X_i)$  here is not a propensity score we chose to estimate with a linear probability model; it is the object OLS implicitly uses, because the FWL residualisation of  $D$  on  $X$  is a linear regression. Linear fitted values are not confined to  $[0, 1]$ , so out-of-range values, and with them wrong-sign weights, are always available to OLS. Genuine IPW weights built from a logit or probit propensity score cannot do this:  $\hat{\pi}(X)$  lies strictly inside  $(0, 1)$  by construction, so  $1/\hat{\pi}$  and  $1/(1 - \hat{\pi})$  stay positive. IPW has its own failure mode when  $\hat{\pi}$  approaches 0 or 1 — weights explode, and the estimate rests on a handful of observations — but it is a visible one. The `ipw` vector plotted above is deliberately built from the same linear  $\hat{d}$  so that the two weighting schemes are comparable unit by unit; it is a diagnostic device, not the IPW estimator one would actually run.

### 4.5. When the weights actually bite

Neither example so far has shown  $\hat{\tau}_{reg}$  miss the ATE, because neither had heterogeneous effects. This one does, and it is built on discrete  $X$  so that every quantity in this chapter can be computed exactly rather than approximated.

Let  $X \in \{0, 1, 2\}$  with probabilities 0.4, 0.3, 0.3; let the treatment probability be  $d(x) = 0.5, 0.1, 0.9$ ; and let the effect be  $\tau(x) = 1, 4, 4$ . The outcome is  $Y = 1 + \tau(X)D + 2X + \varepsilon$  with  $\varepsilon \sim N(0, 1)$ , and we draw  $n = 20,000$ . The design is chosen so that the stratum where treatment is most balanced,  $x = 0$  with  $d = 0.5$ , is also the stratum with the small effect.

The population ATE follows from the definition:  $0.4(1) + 0.3(4) + 0.3(4) = 2.8$ . Angrist's strata-level formula above weights each stratum by  $d(x)\{1 - d(x)\}P(X = x)$  instead — that is 0.10, 0.027, 0.027, or 0.649, 0.175, 0.175 once normalised, against the true shares 0.4, 0.3, 0.3. The  $x = 0$  stratum carries 65% of the regression's weight and 40% of the ATE's, so the regression should be pulled toward that stratum's effect of 1.

```
set.seed(11)
n <- 20000
px <- c(0.4, 0.3, 0.3)      # P(X = 0, 1, 2)
dx <- c(0.5, 0.1, 0.9)     # P(D = 1 | X)
tx <- c(1, 4, 4)           # tau(X)
```



```

X  <- sample(0:2, n, TRUE, prob = px)
D  <- rbinom(n, 1, dx[X + 1])
Y  <- 1 + tx[X + 1] * D + 2 * X + rnorm(n)
dat3 <- data.frame(Y, D, X = factor(X))

# What OLS returns
tau_ols <- coef(lm(Y ~ D + X, data = dat3))["D"]

# Angrist's variance-weighted average of the stratum effects, computed from the
# population quantities rather than estimated
w  <- dx * (1 - dx) * px
ang <- sum(w * tx) / sum(w)

# g-computation on a saturated arm-specific model
m  <- lm(Y ~ D * X, data = dat3)
tau_g <- mean(predict(m, transform(dat3, D = 1)) -
               predict(m, transform(dat3, D = 0)))

# Lin's interaction estimator: D, centred covariates, and their interaction
Xc <- scale(model.matrix(~ X, dat3)[, -1, drop = FALSE], scale = FALSE)
tau_lin <- coef(lm(dat3$Y ~ dat3$D + Xc + dat3$D:Xc))[2]

data.frame(population_ATE = sum(px * tx),
            angrist_formula = ang,
            ols_coefficient = tau_ols,
            g_computation = tau_g,
            lin_interaction = tau_lin, row.names = NULL)

```

|   | population_ATE | angrist_formula | ols_coefficient | g_computation | lin_interaction |
|---|----------------|-----------------|-----------------|---------------|-----------------|
| 1 | 2.8            | 2.051948        | 2.082158        | 2.839965      | 2.839965        |

The OLS coefficient is 2.08 against a population ATE of 2.80. It is not a noisy estimate of the ATE; it is a precise estimate of something else. Angrist's formula predicts 2.052 from the population quantities alone, and the regression delivers 2.082 — the gap between those two is sampling noise, while the gap between either of them and 2.80 is the weighting problem. The regression is answering a question about the stratum where treatment happens to be most balanced, and nothing in its output says so.

The two recommended estimators recover the ATE, and recover it identically: g-computation and Lin's interaction estimator both return 2.840, against a realised sample ATE of 2.814. That they agree to every printed digit is the algebraic equivalence noted in the previous section, not a coincidence of this draw. Both work here because the saturated model in a discrete  $X$  satisfies separate linearity exactly — each arm's conditional mean is linear in the stratum dummies, since it is *saturated* in them.

The practical reading is that the weighting problem needs heterogeneity and non-proportional weights together. The first two examples had the weights; this one adds the heterogeneity, and the

#### 4. *Weights in OLS in causal inference*

cost appears immediately. Since one rarely knows in advance whether effects are heterogeneous, the cheap insurance is to run the interaction or imputation estimator by default: under constant effects it returns what OLS would have returned, and under heterogeneity it returns the ATE that OLS would have missed.

## **Part II.**

# **Marginal Effects & Fixed Effects**



## 5. Marginal effects in models with fixed effects

### 5.1. Marginal effects in a linear model

Stata's `margins` command computes predicted means and marginal effects, but care is needed when fixed effects are present. In a linear model the issue is benign: demeaning and dummy-variable approaches return the same coefficients and the same marginal effects.

We check that on the `auto` data, treating the 5-category repair record `rep78` as the panel identifier and regressing `price` on `mpg` interacted with `trunk` — once with `xtreg, fe` (within-demeaning) and once with `reg` plus `i.rep78` dummies.

```
clear
sysuse auto
xtset rep78
xtreg price c.mpg##c.trunk, fe
margins , dydx(mpg)
reg price c.mpg##c.trunk i.rep78
margins , dydx(mpg)
```

```
. clear

. sysuse auto
(1978 automobile data)

. xtset rep78

Panel variable: rep78 (unbalanced)

. xtreg price c.mpg##c.trunk, fe

Fixed-effects (within) regression              Number of obs   =           69
Group variable: rep78                        Number of groups =            5

R-squared:                                    Obs per group:
    Within = 0.2570                             min =            2
    Between = 0.0653                             avg =           13.8
    Overall = 0.2237                             max =           30

                                                F(3, 61)        =           7.03
```

## 5. Marginal effects in models with fixed effects

corr(u\_i, Xb) = -0.4133 Prob > F = 0.0004

| price   | Coefficient | Std. err.                         | t     | P> t  | [95% conf. interval] |          |
|---------|-------------|-----------------------------------|-------|-------|----------------------|----------|
| mpg     | -98.12003   | 226.8708                          | -0.43 | 0.667 | -551.7763            | 355.5362 |
| trunk   | 295.0544    | 343.3934                          | 0.86  | 0.394 | -391.6032            | 981.712  |
| c.mpg#  |             |                                   |       |       |                      |          |
| c.trunk | -12.23318   | 15.94713                          | -0.77 | 0.446 | -44.12143            | 19.65506 |
| _cons   | 7574.85     | 5321.325                          | 1.42  | 0.160 | -3065.797            | 18215.5  |
| sigma_u | 992.2156    |                                   |       |       |                      |          |
| sigma_e | 2631.2869   |                                   |       |       |                      |          |
| rho     | .12449059   | (fraction of variance due to u_i) |       |       |                      |          |

F test that all u\_i=0: F(4, 61) = 0.86 Prob > F = 0.4948

. margins , dydx(mpg)

Average marginal effects Number of obs = 69  
Model VCE: Conventional

Expression: Linear prediction, predict()  
dy/dx wrt: mpg

|     |           | Delta-method |       |       |                      |           |
|-----|-----------|--------------|-------|-------|----------------------|-----------|
|     | dy/dx     | std. err.    | z     | P> z  | [95% conf. interval] |           |
| mpg | -268.4981 | 74.12513     | -3.62 | 0.000 | -413.7807            | -123.2156 |

. reg price c.mpg##c.trunk i.rep78

| Source   | SS        | df | MS         | Number of obs | = | 69     |
|----------|-----------|----|------------|---------------|---|--------|
| Model    | 154453046 | 7  | 22064720.8 | F(7, 61)      | = | 3.19   |
| Residual | 422343913 | 61 | 6923670.71 | Prob > F      | = | 0.0061 |
|          |           |    |            | R-squared     | = | 0.2678 |
|          |           |    |            | Adj R-squared | = | 0.1838 |
| Total    | 576796959 | 68 | 8482308.22 | Root MSE      | = | 2631.3 |

| price | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------|-------------|-----------|-------|-------|----------------------|----------|
| mpg   | -98.12003   | 226.8708  | -0.43 | 0.667 | -551.7763            | 355.5362 |
| trunk | 295.0544    | 343.3934  | 0.86  | 0.394 | -391.6032            | 981.712  |

|         |           |          |       |       |           |          |
|---------|-----------|----------|-------|-------|-----------|----------|
| c.mpg#  |           |          |       |       |           |          |
| c.trunk | -12.23318 | 15.94713 | -0.77 | 0.446 | -44.12143 | 19.65506 |
| rep78   |           |          |       |       |           |          |
| 2       | 438.0002  | 2161.922 | 0.20  | 0.840 | -3885.031 | 4761.031 |
| 3       | 987.1363  | 2022.606 | 0.49  | 0.627 | -3057.315 | 5031.587 |
| 4       | 1240.944  | 2046.417 | 0.61  | 0.547 | -2851.12  | 5333.008 |
| 5       | 2605.83   | 2161.837 | 1.21  | 0.233 | -1717.031 | 6928.691 |
| _cons   | 6355.731  | 5209.899 | 1.22  | 0.227 | -4062.105 | 16773.57 |

```
. margins , dydx(mpg)
```

Average marginal effects  
Model VCE: OLS

Number of obs = 69

Expression: Linear prediction, predict()  
dy/dx wrt: mpg

|     | Delta-method |           |       |       |                      |
|-----|--------------|-----------|-------|-------|----------------------|
|     | dy/dx        | std. err. | t     | P> t  | [95% conf. interval] |
| mpg | -268.4981    | 74.12513  | -3.62 | 0.001 | -416.7205 -120.2758  |

.

The two agree exactly. Both give  $-98.12003$  on `mpg`,  $295.0544$  on `trunk` and  $-12.23318$  on the interaction, and both average marginal effects are  $-268.4981$  with a standard error of  $74.12513$ . The only difference is that `xtreg` reports a  $z$  statistic and `reg` a  $t$ , so the confidence intervals differ in the fourth digit. Nothing here needs care.

## 5.2. Marginal effects in a non-linear model

In a nonlinear model the two approaches diverge. We now need a genuine panel and a count outcome, so we simulate one: 250 observations, 50 units observed over 5 periods. Within each unit,  $\text{mpg} \sim N(20, 4^2)$  and  $\text{trunk} \sim N(15, 3^2)$  vary period to period, while the unit effect  $\alpha_i \sim N(0, 1)$  is drawn once and held fixed. The conditional mean is  $\lambda_{it} = \exp(0.5 + 0.05 \text{mpg}_{it} - 0.03 \text{trunk}_{it} + \alpha_i)$  and  $\text{accidents}_{it} \sim \text{Poisson}(\lambda_{it})$ . The true coefficients are  $0.05$  and  $-0.03$ .

```
clear
set seed 42
set obs 250
```

## 5. Marginal effects in models with fixed effects

```
gen id = ceil(_n/5)
bys id: gen t = _n
gen mpg = rnormal(20,4)
gen trunk = rnormal(15,3)
gen alpha = rnormal(0,1)
bys id: replace alpha = alpha[1]
gen lambda = exp(0.5 + 0.05*mpg - 0.03*trunk + alpha)
gen accidents = rpoisson(lambda)
xtset id
xtpoisson accidents mpg trunk, fe
margins , dydx(mpg)
margins , dydx(mpg) predict(nu0)
poisson accidents mpg trunk i.id
margins , dydx(mpg)
```

```
. clear

. set seed 42

. set obs 250
Number of observations (_N) was 0, now 250.

. gen id = ceil(_n/5)

. bys id: gen t = _n

. gen mpg = rnormal(20,4)

. gen trunk = rnormal(15,3)

. gen alpha = rnormal(0,1)

. bys id: replace alpha = alpha[1]
(200 real changes made)

. gen lambda = exp(0.5 + 0.05*mpg - 0.03*trunk + alpha)

. gen accidents = rpoisson(lambda)

. xtset id

Panel variable: id (balanced)

. xtpoisson accidents mpg trunk, fe
note: 1 group (5 obs) dropped because of all zero outcomes
```



## 5.2. Marginal effects in a non-linear model

Iteration 0: Log likelihood = -358.34297  
 Iteration 1: Log likelihood = -340.16596  
 Iteration 2: Log likelihood = -340.16413  
 Iteration 3: Log likelihood = -340.16413

Conditional fixed-effects Poisson regression  
 Group variable: id

Number of obs = 245  
 Number of groups = 49

Obs per group:  
     min = 5  
     avg = 5.0  
     max = 5

Log likelihood = -340.16413  
 Wald chi2(2) = 35.90  
 Prob > chi2 = 0.0000

| accidents | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |           |
|-----------|-------------|-----------|-------|-------|----------------------|-----------|
| mpg       | .0487946    | .0090524  | 5.39  | 0.000 | .0310521             | .0665371  |
| trunk     | -.0316066   | .0108931  | -2.90 | 0.004 | -.0529566            | -.0102566 |

. margins , dydx(mpg)

Average marginal effects  
 Model VCE: OIM

Number of obs = 245

Expression: Linear prediction, predict()  
 dy/dx wrt: mpg

|     | Delta-method |           |      |       |                      |          |
|-----|--------------|-----------|------|-------|----------------------|----------|
|     | dy/dx        | std. err. | z    | P> z  | [95% conf. interval] |          |
| mpg | .0487946     | .0090524  | 5.39 | 0.000 | .0310521             | .0665371 |

. margins , dydx(mpg) predict(nu0)

Average marginal effects  
 Model VCE: OIM

Number of obs = 245

Expression: Predicted number of events (assuming u\_i=0), predict(nu0)  
 dy/dx wrt: mpg

|  | Delta-method |  |
|--|--------------|--|
|--|--------------|--|

## 5. Marginal effects in models with fixed effects

|     | dy/dx    | std. err. | z    | P> z  | [95% conf. interval] |          |
|-----|----------|-----------|------|-------|----------------------|----------|
| mpg | .0850287 | .0340547  | 2.50 | 0.013 | .0182826             | .1517747 |

```
. poisson accidents mpg trunk i.id
```

```
Iteration 0: Log likelihood = -485.81671
Iteration 1: Log likelihood = -453.45562
Iteration 2: Log likelihood = -453.02481
Iteration 3: Log likelihood = -452.93985
Iteration 4: Log likelihood = -452.92127
Iteration 5: Log likelihood = -452.91701
Iteration 6: Log likelihood = -452.916
Iteration 7: Log likelihood = -452.91579
Iteration 8: Log likelihood = -452.91575
```

Poisson regression

Number of obs = 250  
LR chi2(51) = 934.93  
Prob > chi2 = 0.0000  
Pseudo R2 = 0.5079

Log likelihood = -452.91575

| accidents | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |           |
|-----------|-------------|-----------|-------|-------|----------------------|-----------|
| mpg       | .0487946    | .0090525  | 5.39  | 0.000 | .031052              | .0665371  |
| trunk     | -.0316067   | .0108931  | -2.90 | 0.004 | -.0529568            | -.0102565 |
| i.id      |             |           |       |       |                      |           |
| 2         | .3464221    | .5592835  | 0.62  | 0.536 | -.7497533            | 1.442598  |
| 3         | 1.123341    | .5131078  | 2.19  | 0.029 | .1176679             | 2.129014  |
| 4         | .1098041    | .6057603  | 0.18  | 0.856 | -1.077464            | 1.297073  |
| 5         | .4069152    | .5702496  | 0.71  | 0.475 | -.7107536            | 1.524584  |
| 6         | 1.791221    | .4776303  | 3.75  | 0.000 | .8550829             | 2.727359  |
| 7         | -14.00991   | 475.5637  | -0.03 | 0.976 | -946.0976            | 918.0778  |
| 8         | .9632705    | .5216941  | 1.85  | 0.065 | -.0592311            | 1.985772  |
| 9         | .1039008    | .6061094  | 0.17  | 0.864 | -1.084052            | 1.291853  |
| 10        | .7748186    | .5273812  | 1.47  | 0.142 | -.2588296            | 1.808467  |
| 11        | .7331684    | .5395719  | 1.36  | 0.174 | -.3243732            | 1.79071   |
| 12        | 1.114943    | .5007908  | 2.23  | 0.026 | .133411              | 2.096475  |
| 13        | -.2054342   | .6343396  | -0.32 | 0.746 | -1.448717            | 1.037848  |
| 14        | 1.248116    | .4945786  | 2.52  | 0.012 | .2787602             | 2.217472  |
| 15        | 1.395876    | .4914484  | 2.84  | 0.005 | .4326552             | 2.359098  |
| 16        | 1.689054    | .4845687  | 3.49  | 0.000 | .7393167             | 2.638791  |
| 17        | .9941942    | .5171224  | 1.92  | 0.055 | -.0193471            | 2.007735  |
| 18        | 1.742704    | .4844911  | 3.60  | 0.000 | .793119              | 2.692289  |
| 19        | .9912213    | .5168563  | 1.92  | 0.055 | -.0217985            | 2.004241  |
| 20        | -1.134862   | .8375742  | -1.35 | 0.175 | -2.776477            | .5067532  |

## 5.2. Marginal effects in a non-linear model

|       |  |           |          |       |       |           |          |
|-------|--|-----------|----------|-------|-------|-----------|----------|
| 21    |  | 1.484731  | .4879013 | 3.04  | 0.002 | .5284622  | 2.441    |
| 22    |  | 2.442088  | .4643803 | 5.26  | 0.000 | 1.531919  | 3.352256 |
| 23    |  | .9732572  | .5265941 | 1.85  | 0.065 | -.0588483 | 2.005363 |
| 24    |  | .280425   | .5859507 | 0.48  | 0.632 | -.8680172 | 1.428867 |
| 25    |  | 2.472329  | .4641555 | 5.33  | 0.000 | 1.562601  | 3.382057 |
| 26    |  | .5602592  | .5485592 | 1.02  | 0.307 | -.5148971 | 1.635416 |
| 27    |  | 1.537242  | .4861325 | 3.16  | 0.002 | .5844401  | 2.490044 |
| 28    |  | 1.34391   | .5026286 | 2.67  | 0.008 | .3587763  | 2.329044 |
| 29    |  | 2.370956  | .4660218 | 5.09  | 0.000 | 1.45757   | 3.284342 |
| 30    |  | 2.453329  | .4642702 | 5.28  | 0.000 | 1.543377  | 3.363282 |
| 31    |  | .9383342  | .5171502 | 1.81  | 0.070 | -.0752616 | 1.95193  |
| 32    |  | .1850002  | .5864331 | 0.32  | 0.752 | -.9643876 | 1.334388 |
| 33    |  | -.2686233 | .6711829 | -0.40 | 0.689 | -1.584118 | 1.046871 |
| 34    |  | 1.547864  | .485423  | 3.19  | 0.001 | .5964525  | 2.499276 |
| 35    |  | 1.557431  | .4878123 | 3.19  | 0.001 | .6013366  | 2.513526 |
| 36    |  | 1.295147  | .4965837 | 2.61  | 0.009 | .3218611  | 2.268433 |
| 37    |  | 2.668556  | .4616456 | 5.78  | 0.000 | 1.763748  | 3.573365 |
| 38    |  | .9287953  | .5326269 | 1.74  | 0.081 | -.1151343 | 1.972725 |
| 39    |  | -.9247308 | .8366882 | -1.11 | 0.269 | -2.56461  | .715148  |
| 40    |  | 1.358213  | .4979867 | 2.73  | 0.006 | .3821769  | 2.334249 |
| 41    |  | -.0472585 | .6070096 | -0.08 | 0.938 | -1.236975 | 1.142458 |
| 42    |  | 1.394504  | .4958685 | 2.81  | 0.005 | .4226201  | 2.366389 |
| 43    |  | -.5414444 | .7303533 | -0.74 | 0.458 | -1.972911 | .8900219 |
| 44    |  | .9881123  | .513744  | 1.92  | 0.054 | -.0188073 | 1.995032 |
| 45    |  | 1.171458  | .5165503 | 2.27  | 0.023 | .1590378  | 2.183878 |
| 46    |  | .817007   | .5269245 | 1.55  | 0.121 | -.2157461 | 1.84976  |
| 47    |  | 2.800173  | .4596719 | 6.09  | 0.000 | 1.899233  | 3.701113 |
| 48    |  | .365911   | .5704157 | 0.64  | 0.521 | -.7520831 | 1.483905 |
| 49    |  | 2.445633  | .4649624 | 5.26  | 0.000 | 1.534324  | 3.356943 |
| 50    |  | .7177863  | .5279692 | 1.36  | 0.174 | -.3170143 | 1.752587 |
|       |  |           |          |       |       |           |          |
| _cons |  | -.4227135 | .5046249 | -0.84 | 0.402 | -1.41176  | .566333  |

---

. margins , dydx(mpg)

Average marginal effects

Number of obs = 250

Model VCE: OIM

Expression: Predicted number of events, predict()

dy/dx wrt: mpg

|       |  | Delta-method |           |      |       |                      |
|-------|--|--------------|-----------|------|-------|----------------------|
|       |  | dy/dx        | std. err. | z    | P> z  | [95% conf. interval] |
| <hr/> |  |              |           |      |       |                      |
| mpg   |  | .2238673     | .0420551  | 5.32 | 0.000 | .1414408 .3062937    |

---

## 5. Marginal effects in models with fixed effects

(We use a simulated count outcome `accidents` and a proper panel with 50 units of 5 periods each, rather than `auto`'s continuous `price` and its 5-category `rep78` — a true panel needs many units, and a fixed-effect count model needs an actual count dependent variable.)

In this example, “`xtpoisson, fe`” and “`poisson ... i.id`” return the same coefficients (both give `mpg` = .0487946 with a standard error of .0090524). Fixed effect Poisson model (sometimes called conditional fixed effect Poisson) is the same models as a Poisson model with dummies, just like a linear model (OLS with dummies is the same as fixed effect OLS). Poisson model and OLS are unique in this sense that there is no “incidental parameter” problem.

Both recover the truth: 0.0488 against a true 0.05 for `mpg`, and  $-0.0316$  against  $-0.03$  for `trunk`. Note the sample sizes differ, 245 against 250 — `xtpoisson, fe` drops the one group whose outcome is zero in every period, because such a group contributes nothing to the conditional likelihood. It changes neither coefficient here.

The `margins` calls return different marginal effects even though the models are the same, and the spread is not small:

| call                                                   | AME of <code>mpg</code> | std. err. |
|--------------------------------------------------------|-------------------------|-----------|
| <code>margins, dydx(mpg) after xtpoisson, fe</code>    | 0.0488                  | 0.0091    |
| <code>margins, dydx(mpg) predict(nu0)</code>           | 0.0850                  | 0.0341    |
| <code>margins, dydx(mpg) after poisson ... i.id</code> | 0.2239                  | 0.0421    |

Three answers from one coefficient, the largest 4.6 times the smallest. In the conditional fixed-effect Poisson the fixed effects are conditioned out — they are not estimated. The first row is not really a marginal effect at all: with no  $\alpha_i$  to exponentiate against, it returns the coefficient itself. `margins, predict(nu0)` sets the fixed effect to zero, which is arbitrary, and note that the conditional model has no intercept either — it was swept out with the  $\alpha_i$ . So `nu0` exponentiates  $X\beta$  alone, giving a baseline count no unit in the data actually has. The Poisson with dummies estimates the fixed effects, so `margins` uses actual estimates and gets the largest value.

The arithmetic is easy to follow, since for a Poisson  $\partial\lambda/\partial x = \beta\lambda$  and therefore every one of these AMEs is  $\beta$  times an average  $\lambda$ . Divide each row by 0.0488 and the implied baselines are 1.00, 1.74 and 4.59. The last matches  $E[\lambda] = e^{1.05}E[e^\alpha] = 4.71$  from the DGP; the middle matches  $e^{X\beta}$  evaluated without intercept or fixed effect, 1.65. The three rows differ only in what they assume about  $\alpha_i$ , and none of them is a modelling subtlety — it is a choice of baseline. The third is the defensible one, but for the conditional model the safest route is to report the original coefficients and avoid `margins` for marginal effects entirely.

The conditional logit is worse. The fixed effects are not estimated, and unlike Poisson there is an incidental-parameter problem, so logit with dummies is inconsistent unless the panel is deep (roughly 20+ observations per unit). With conditional logit the predicted probability depends on the unestimated  $\alpha_i$ ; `margins` after `clogit` or `xtlogit, fe` offers options like `pu0` (set all fixed effects to zero), but none is defensible.

In a fixed effect logit model,

$$\log(P(y=1)/(1-P(y=1))) = \alpha_i + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 * x_2 \quad (5.1)$$

Here  $\alpha_i$  is the unit fixed effect. Therefore

$$P(y=1) = F(\alpha_i + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 * x_2) \quad (5.2)$$

Without estimating  $\alpha_i$  there is no way to predict  $P$  in a meaningful way. But the log-odds are linear in the covariates, and the marginal effect of  $x_1$  or  $x_2$  on the log-odds does not involve  $\alpha_i$ . So `margins` with the `predict(xb)` option — marginal effects on the log-odds scale — is interpretable. Note that `xb` here is  $X_{it}\beta$  only — the individual fixed effect  $\alpha_i$  is not estimated in conditional logit and so is not part of the prediction. The logged-odds *effect* of a covariate ( $\partial(X\beta)/\partial x$ ) therefore matches the original coefficient exactly and does not depend on  $\alpha_i$ , but `xb` itself is a log-odds relative to each individual's (unidentified) baseline, not the individual's absolute log-odds. This lets you make linear extrapolations of relative log-odds across values of the covariates, but not of absolute predicted probabilities, since those would require  $\alpha_i$ .

```
clear
webuse union
clogit union c.age##i.south not_smsa grade, group(idcode)
margins, at( age=(15 20 25 30 35 40) south=(0 1)) predict(xb)
marginsplot
graph export "marginsplot-clogit.svg", as(svg) replace
```

```
. clear

. webuse union
(NLS Women 14-24 in 1968)

. clogit union c.age##i.south not_smsa grade, group(idcode)
note: multiple positive outcomes within groups encountered.
note: 2,744 groups (14,165 obs) omitted because of all positive or
      all negative outcomes.
```

```
Iteration 0:  Log likelihood = -4518.8815
Iteration 1:  Log likelihood = -4512.8224
Iteration 2:  Log likelihood = -4512.8192
Iteration 3:  Log likelihood = -4512.8192
```

|                                                 |                        |
|-------------------------------------------------|------------------------|
| Conditional (fixed-effects) logistic regression | Number of obs = 12,035 |
|                                                 | LR chi2(5) = 74.73     |
|                                                 | Prob > chi2 = 0.0000   |
| Log likelihood = -4512.8192                     | Pseudo R2 = 0.0082     |

```
-----
union | Coefficient Std. err.      z    P>|z|    [95% conf. interval]
```

## 5. Marginal effects in models with fixed effects

|             |  |           |          |       |       |           |           |
|-------------|--|-----------|----------|-------|-------|-----------|-----------|
| age         |  | .0096842  | .0050265 | 1.93  | 0.054 | -.0001676 | .019536   |
| 1.south     |  | -1.382178 | .276966  | -4.99 | 0.000 | -1.925022 | -.8393346 |
| south#c.age |  |           |          |       |       |           |           |
| 1           |  | .0208997  | .0081247 | 2.57  | 0.010 | .0049756  | .0368238  |
| not_smsa    |  | .0195233  | .1131292 | 0.17  | 0.863 | -.2022058 | .2412523  |
| grade       |  | .0822276  | .0419062 | 1.96  | 0.050 | .000093   | .1643622  |

```
. margins, at( age=(15 20 25 30 35 40) south=(0 1)) predict(xb)
```

Predictive margins

Number of obs = 12,035

Model VCE: OIM

Expression: Linear prediction, predict(xb)

```
1._at: age = 15
      south = 0
2._at: age = 15
      south = 1
3._at: age = 20
      south = 0
4._at: age = 20
      south = 1
5._at: age = 25
      south = 0
6._at: age = 25
      south = 1
7._at: age = 30
      south = 0
8._at: age = 30
      south = 1
9._at: age = 35
      south = 0
10._at: age = 35
      south = 1
11._at: age = 40
      south = 0
12._at: age = 40
      south = 1
```

|     |  | Delta-method |           |      |       | [95% conf. interval] |          |
|-----|--|--------------|-----------|------|-------|----------------------|----------|
|     |  | Margin       | std. err. | z    | P> z  |                      |          |
| _at |  |              |           |      |       |                      |          |
| 1   |  | 1.202147     | .5190753  | 2.32 | 0.021 | .184778              | 2.219516 |

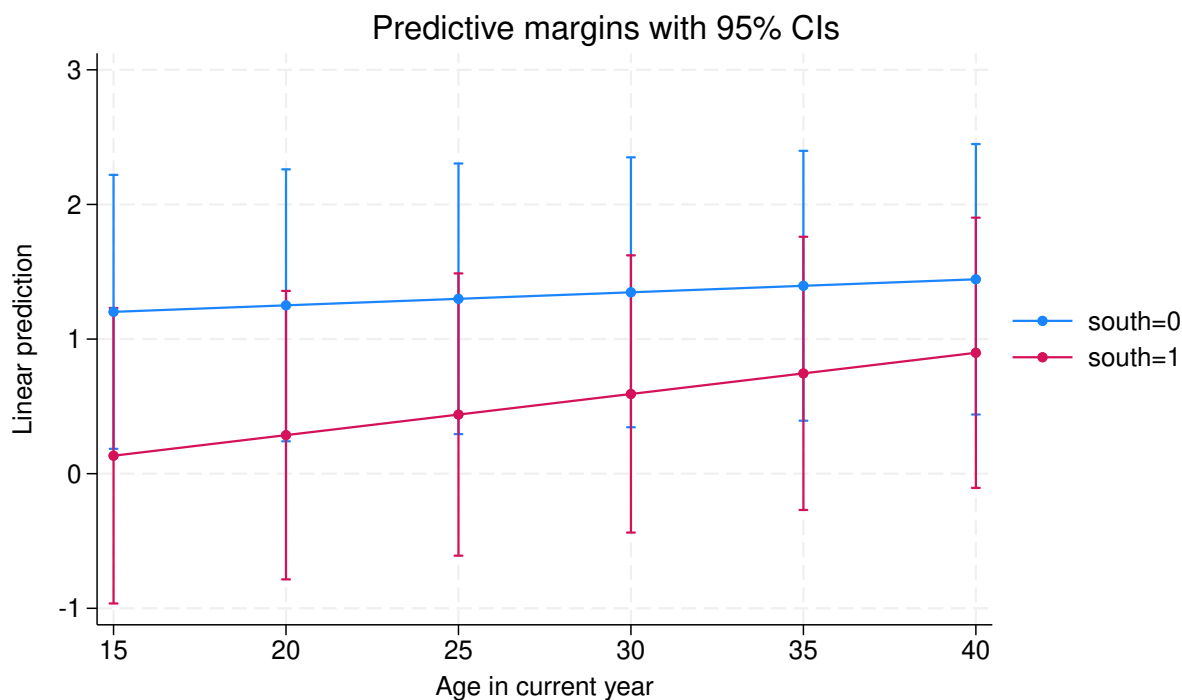
|    |  |          |          |      |       |           |          |
|----|--|----------|----------|------|-------|-----------|----------|
| 2  |  | .133464  | .5599015 | 0.24 | 0.812 | -.9639228 | 1.230851 |
| 3  |  | 1.250568 | .5153257 | 2.43 | 0.015 | .240548   | 2.260588 |
| 4  |  | .2863834 | .5465398 | 0.52 | 0.600 | -.7848148 | 1.357582 |
| 5  |  | 1.298989 | .5127819 | 2.53 | 0.011 | .2939548  | 2.304023 |
| 6  |  | .4393029 | .5349589 | 0.82 | 0.412 | -.6091973 | 1.487803 |
| 7  |  | 1.34741  | .5114619 | 2.63 | 0.008 | .3449629  | 2.349857 |
| 8  |  | .5922223 | .5252767 | 1.13 | 0.260 | -.4373011 | 1.621746 |
| 9  |  | 1.395831 | .5113752 | 2.73 | 0.006 | .3935538  | 2.398108 |
| 10 |  | .7451418 | .5175997 | 1.44 | 0.150 | -.2693351 | 1.759619 |
| 11 |  | 1.444252 | .5125224 | 2.82 | 0.005 | .4397264  | 2.448777 |
| 12 |  | .8980612 | .5120182 | 1.75 | 0.079 | -.1054761 | 1.901598 |

---

```
. marginsplot
```

Variables that uniquely identify margins: age south

```
. graph export "marginsplot-clogit.svg", as(svg) replace
file marginsplot-clogit.svg saved as SVG format
```



This fits a conditional logit of union status on age, south, and their interaction, on 12,035 person-years from the `union` panel. The age coefficient is 0.0097 with a standard error of 0.0050, and the south indicator is  $-1.382$  with a standard error of 0.277. `margins` with `predict(xb)` then returns predicted log-odds at six ages crossed with south — twelve combinations — but not predicted probabilities, since those require  $\alpha_i$ .

## 5. Marginal effects in models with fixed effects

Read the vertical axis of that plot as a *relative* scale. Because  $\alpha_i$  is not estimated,  $\mathbf{x}\mathbf{b}$  is each individual's log-odds measured from an unidentified baseline, so the level of any single point is arbitrary; what is interpretable is how the curves move across age and how far apart the south and non-south curves sit. Differences within the plot are the estimable quantities, not heights.



## 6. Comparing Marginal effects with margins command

### 6.1. Comparing marginal effects

A common practice is to plot marginal effects with `margins` and `marginsplot`, then conclude two effects differ whenever their confidence intervals do not overlap. This can be wrong — overlapping intervals do not imply no significant difference, because the test depends on the joint distribution of the two estimates, not on their marginal distributions separately.

```
webuse nhanes2f, clear
logit diabetes i.female#c.age, nolog
margins female, at(age=(20 30 40 50 60 70))
marginsplot
graph export "marginsplot1.svg", as(svg) replace
margins r.female, at(age=(20 30 40 50 60 70))
marginsplot
graph export "marginsplot2.svg", as(svg) replace
```

|                             |                        |
|-----------------------------|------------------------|
| Logistic regression         | Number of obs = 10,335 |
|                             | LR chi2(3) = 353.06    |
|                             | Prob > chi2 = 0.0000   |
| Log likelihood = -1822.5393 | Pseudo R2 = 0.0883     |

| diabetes     | Coefficient | Std. err. | z      | P> z  | [95% conf. interval] |           |
|--------------|-------------|-----------|--------|-------|----------------------|-----------|
| female       |             |           |        |       |                      |           |
| Female       | 1.404111    | .4858776  | 2.89   | 0.004 | .4518088             | 2.356414  |
| age          | .0714166    | .0063231  | 11.29  | 0.000 | .0590235             | .0838097  |
| female#c.age |             |           |        |       |                      |           |
| Female       | -.0206572   | .0078369  | -2.64  | 0.008 | -.0360172            | -.0052972 |
| _cons        | -7.041892   | .3967299  | -17.75 | 0.000 | -7.819468            | -6.264315 |

|                      |                        |
|----------------------|------------------------|
| Adjusted predictions | Number of obs = 10,335 |
| Model VCE: OIM       |                        |

## 6. Comparing Marginal effects with margins command

Expression: Pr(diabetes), predict()

1.\_at: age = 20  
 2.\_at: age = 30  
 3.\_at: age = 40  
 4.\_at: age = 50  
 5.\_at: age = 60  
 6.\_at: age = 70

|            |  | Delta-method |           | z     | P> z  | [95% conf. interval] |          |
|------------|--|--------------|-----------|-------|-------|----------------------|----------|
|            |  | Margin       | std. err. |       |       |                      |          |
| _at#female |  |              |           |       |       |                      |          |
| 1#Male     |  | .0036348     | .0009895  | 3.67  | 0.000 | .0016953             | .0055743 |
| 1#Female   |  | .0097316     | .0018436  | 5.28  | 0.000 | .0061184             | .0133449 |
| 2#Male     |  | .007396      | .0015622  | 4.73  | 0.000 | .0043341             | .0104579 |
| 2#Female   |  | .0160637     | .0023434  | 6.85  | 0.000 | .0114706             | .0206568 |
| 3#Male     |  | .0149906     | .0022831  | 6.57  | 0.000 | .0105159             | .0194653 |
| 3#Female   |  | .026406      | .0027757  | 9.51  | 0.000 | .0209657             | .0318462 |
| 4#Male     |  | .0301469     | .0029996  | 10.05 | 0.000 | .0242677             | .0360261 |
| 4#Female   |  | .043115      | .0030923  | 13.94 | 0.000 | .0370543             | .0491758 |
| 5#Male     |  | .0596983     | .0040248  | 14.83 | 0.000 | .0518099             | .0675867 |
| 5#Female   |  | .069641      | .0040284  | 17.29 | 0.000 | .0617454             | .0775366 |
| 6#Male     |  | .1147889     | .0089467  | 12.83 | 0.000 | .0972536             | .1323242 |
| 6#Female   |  | .1106004     | .0078699  | 14.05 | 0.000 | .0951757             | .126025  |

Variables that uniquely identify margins: age female

file marginsplot1.svg saved as SVG format

Contrasts of adjusted predictions

Number of obs = 10,335

Model VCE: OIM

Expression: Pr(diabetes), predict()

1.\_at: age = 20  
 2.\_at: age = 30  
 3.\_at: age = 40  
 4.\_at: age = 50  
 5.\_at: age = 60  
 6.\_at: age = 70

|  | df | chi2 | P>chi2 |
|--|----|------|--------|
|  |    |      |        |

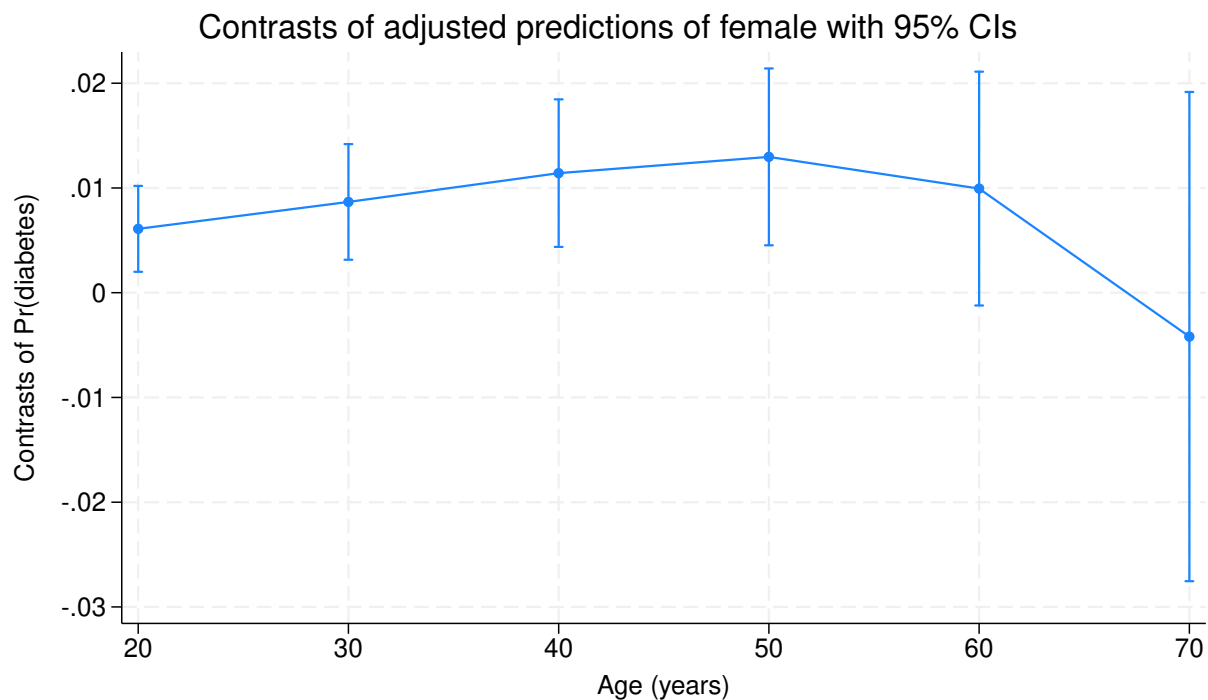
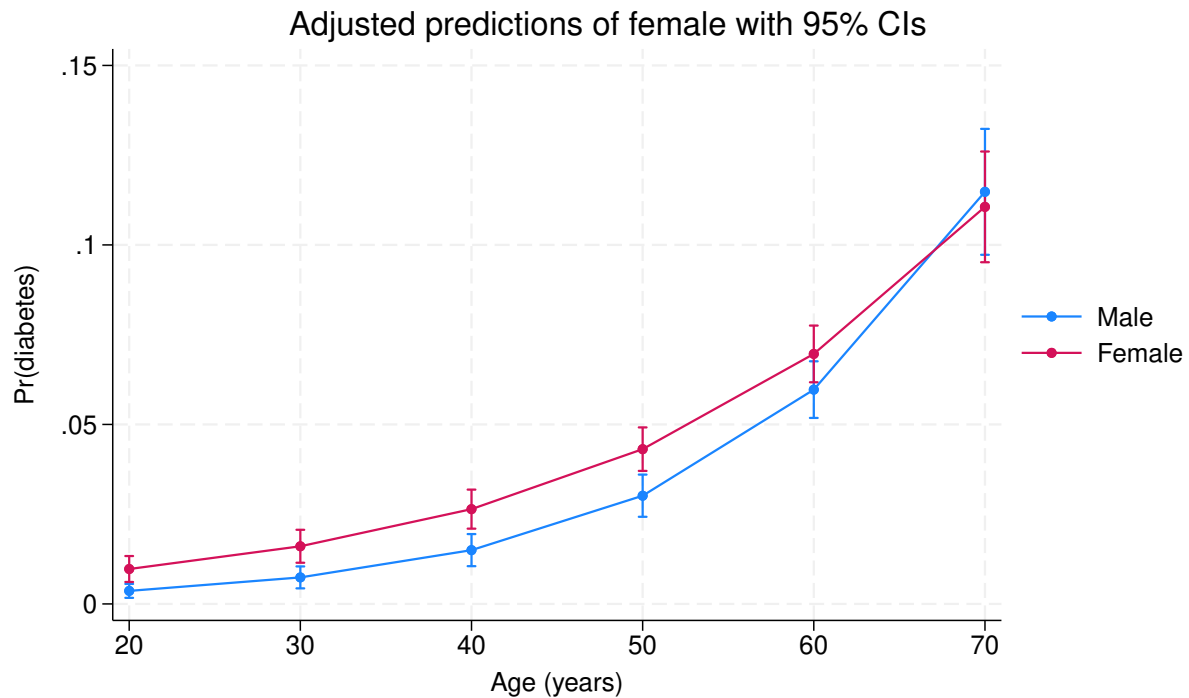
|                    |  |   |       |        |
|--------------------|--|---|-------|--------|
| female@_at         |  |   |       |        |
| (Female vs Male) 1 |  | 1 | 8.49  | 0.0036 |
| (Female vs Male) 2 |  | 1 | 9.47  | 0.0021 |
| (Female vs Male) 3 |  | 1 | 10.09 | 0.0015 |
| (Female vs Male) 4 |  | 1 | 9.06  | 0.0026 |
| (Female vs Male) 5 |  | 1 | 3.05  | 0.0808 |
| (Female vs Male) 6 |  | 1 | 0.12  | 0.7252 |
| Joint              |  | 4 | 40.48 | 0.0000 |

|                    |  | Delta-method |           |                      |
|--------------------|--|--------------|-----------|----------------------|
|                    |  | Contrast     | std. err. | [95% conf. interval] |
| female@_at         |  |              |           |                      |
| (Female vs Male) 1 |  | .0060969     | .0020923  | .0019959 .0101978    |
| (Female vs Male) 2 |  | .0086677     | .0028164  | .0031476 .0141878    |
| (Female vs Male) 3 |  | .0114154     | .003594   | .0043712 .0184595    |
| (Female vs Male) 4 |  | .0129682     | .0043081  | .0045244 .0214119    |
| (Female vs Male) 5 |  | .0099427     | .0056945  | -.0012183 .0211037   |
| (Female vs Male) 6 |  | -.0041885    | .0119155  | -.0275425 .0191654   |

Variables that uniquely identify margins: age

file marginsplot2.svg saved as SVG format

## 6. Comparing Marginal effects with margins command



We fit a logit of diabetes on female, age, and their interaction, on 10,335 NHANES II respondents. The interaction is  $-0.0207$  with a standard error of  $0.0078$ , so the female-male gap narrows with age. The first `margins` call gives predicted probabilities by gender; the second (with `r.female`) gives the female-vs-male contrast at each age. (On the general question of marginal effects in nonlinear models, see the previous chapter — the coefficient on the link scale is usually more interpretable.)

Those contrasts run .0061, .0087, .0114, .0130, .0099 and  $-.0042$  at ages 20 through 70, significant through age 50 and not thereafter — the gap widens, peaks in middle age, then closes and reverses sign.

Now the point of the chapter. The female effect at age 20 is .0061 with a 95% interval of [.0020, .0102], and at age 30 it is .0087 with an interval of [.0031, .0142]. Those intervals overlap across most of their length. It is tempting to conclude the two effects are not significantly different, but that conclusion does not follow. For two effects  $x$  and  $y$  from the same model,

$$\text{Var}(x - y) = \text{Var}(x) + \text{Var}(y) - 2 \text{Cov}(x, y). \quad (6.1)$$

When  $x$  and  $y$  are positively correlated — as they will be, coming from the same fit — the variance of their difference can be much smaller than either individual variance. So the difference can be significant even when the individual intervals overlap.

```
webuse nhanes2f, clear
logit diabetes i.female#c.age, nolog
margins r.female, at(age=(20 30 40 50 60 70)) post
lincom _b[r1vs0.female@2._at] - _b[r1vs0.female@1._at]
```

|                             |                        |
|-----------------------------|------------------------|
| Logistic regression         | Number of obs = 10,335 |
|                             | LR chi2(3) = 353.06    |
|                             | Prob > chi2 = 0.0000   |
| Log likelihood = -1822.5393 | Pseudo R2 = 0.0883     |

| diabetes     | Coefficient | Std. err. | z      | P> z  | [95% conf. interval] |           |
|--------------|-------------|-----------|--------|-------|----------------------|-----------|
| female       |             |           |        |       |                      |           |
| Female       | 1.404111    | .4858776  | 2.89   | 0.004 | .4518088             | 2.356414  |
| age          | .0714166    | .0063231  | 11.29  | 0.000 | .0590235             | .0838097  |
| female#c.age |             |           |        |       |                      |           |
| Female       | -.0206572   | .0078369  | -2.64  | 0.008 | -.0360172            | -.0052972 |
| _cons        | -7.041892   | .3967299  | -17.75 | 0.000 | -7.819468            | -6.264315 |

|                                   |                        |
|-----------------------------------|------------------------|
| Contrasts of adjusted predictions | Number of obs = 10,335 |
| Model VCE: OIM                    |                        |

Expression: Pr(diabetes), predict()

```
1._at: age = 20
2._at: age = 30
3._at: age = 40
4._at: age = 50
```

## 6. Comparing Marginal effects with margins command

5.\_at: age = 60

6.\_at: age = 70

|                    | df | chi2  | P>chi2 |
|--------------------|----|-------|--------|
| female@_at         |    |       |        |
| (Female vs Male) 1 | 1  | 8.49  | 0.0036 |
| (Female vs Male) 2 | 1  | 9.47  | 0.0021 |
| (Female vs Male) 3 | 1  | 10.09 | 0.0015 |
| (Female vs Male) 4 | 1  | 9.06  | 0.0026 |
| (Female vs Male) 5 | 1  | 3.05  | 0.0808 |
| (Female vs Male) 6 | 1  | 0.12  | 0.7252 |
| Joint              | 4  | 40.48 | 0.0000 |

|                    | Contrast  | std. err. | [95% conf. interval] |          |
|--------------------|-----------|-----------|----------------------|----------|
| female@_at         |           |           |                      |          |
| (Female vs Male) 1 | .0060969  | .0020923  | .0019959             | .0101978 |
| (Female vs Male) 2 | .0086677  | .0028164  | .0031476             | .0141878 |
| (Female vs Male) 3 | .0114154  | .003594   | .0043712             | .0184595 |
| (Female vs Male) 4 | .0129682  | .0043081  | .0045244             | .0214119 |
| (Female vs Male) 5 | .0099427  | .0056945  | -.0012183            | .0211037 |
| (Female vs Male) 6 | -.0041885 | .0119155  | -.0275425            | .0191654 |

( 1) - r1vs0.female@1bn.\_at + r1vs0.female@2.\_at = 0

|     | Coefficient | Std. err. | z    | P> z  | [95% conf. interval] |          |
|-----|-------------|-----------|------|-------|----------------------|----------|
| (1) | .0025709    | .0007946  | 3.24 | 0.001 | .0010134             | .0041283 |

The test gives a difference of .0025709 with a standard error of .0007946,  $z = 3.24$  and  $p = 0.001$ . The two effects are different, despite intervals that overlap heavily.

The standard error is what makes the point. Each individual contrast carries a standard error of .0021 and .0028, but their *difference* carries only .00079 — smaller than either. Solving  $\text{Var}(x - y) = \text{Var}(x) + \text{Var}(y) - 2\text{Cov}(x, y)$  for the correlation gives about 0.99: the two contrasts come from the same fitted model evaluated ten years apart, so they move together almost perfectly, and nearly all of their variance cancels in the difference.

Had we ignored the covariance and used  $\sqrt{\text{Var}(x) + \text{Var}(y)} = .0035$ , we would have got  $z = 0.73$

and  $p = 0.46$ . Same data, same two numbers, opposite conclusion. That is the cost of reading overlap off a graph.

We can do this with native Stata syntax: `margins ...`, `post` posts the estimated contrasts as if they were regular coefficients (named `r1vs0.female@1._at`, `r1vs0.female@2._at`, etc. — check `matrix list e(b)` to see the exact names), so `lincom` can test any linear combination of them natively, without depending on a user-written command.

`margins` can compute these differences directly with `contrast(atcontrast(r))`, which compares each evaluation point to the reference:

```
webuse nhanes2f, clear
logit diabetes i.female#c.age, nolog
margins r.female, at(age=(20 30 40 50 60 70)) contrast(atcontrast(r))
marginsplot
graph export "marginsplot3.svg", as(svg) replace
```

Logistic regression

Number of obs = 10,335

LR chi2(3) = 353.06

Prob > chi2 = 0.0000

Log likelihood = -1822.5393

Pseudo R2 = 0.0883

| diabetes     | Coefficient | Std. err. | z      | P> z  | [95% conf. interval] |           |
|--------------|-------------|-----------|--------|-------|----------------------|-----------|
| female       |             |           |        |       |                      |           |
| Female       | 1.404111    | .4858776  | 2.89   | 0.004 | .4518088             | 2.356414  |
| age          | .0714166    | .0063231  | 11.29  | 0.000 | .0590235             | .0838097  |
| female#c.age |             |           |        |       |                      |           |
| Female       | -.0206572   | .0078369  | -2.64  | 0.008 | -.0360172            | -.0052972 |
| _cons        | -7.041892   | .3967299  | -17.75 | 0.000 | -7.819468            | -6.264315 |

Contrasts of adjusted predictions

Number of obs = 10,335

Model VCE: OIM

Expression: `Pr(diabetes), predict()`

1.\_at: age = 20

2.\_at: age = 30

3.\_at: age = 40

4.\_at: age = 50

5.\_at: age = 60

6.\_at: age = 70

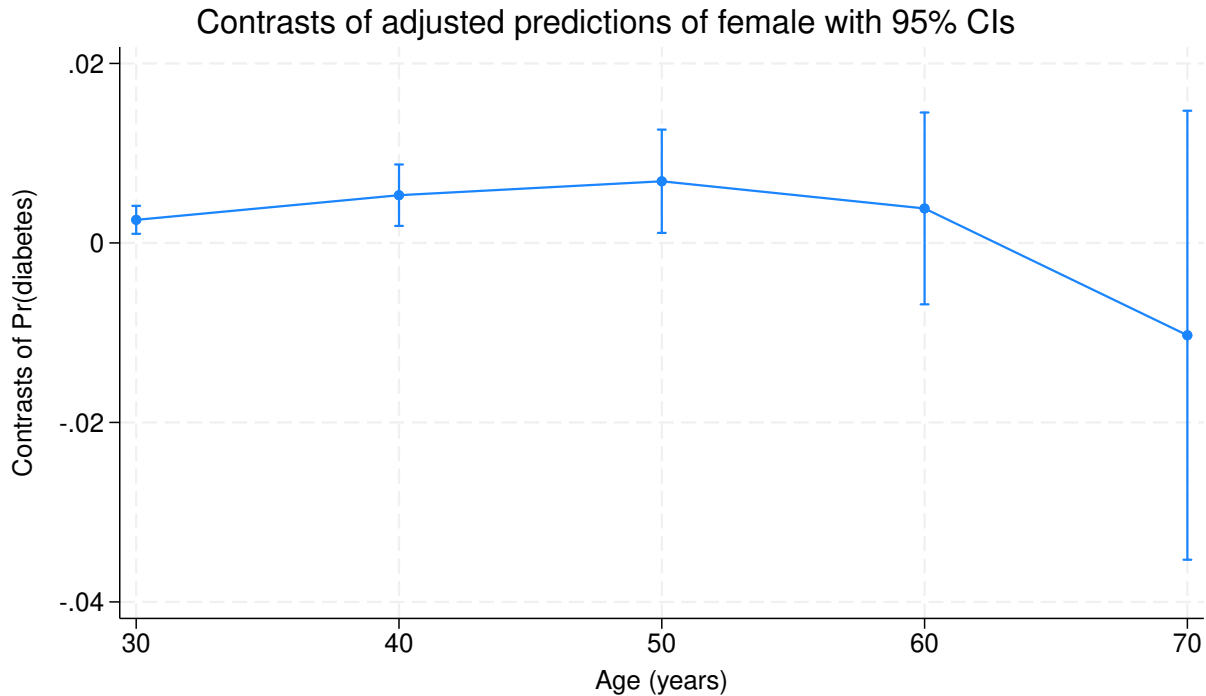
## 6. Comparing Marginal effects with margins command

|                           | df           | chi2      | P>chi2               |
|---------------------------|--------------|-----------|----------------------|
| -----+-----               |              |           |                      |
| _at#female                |              |           |                      |
| (2 vs 1) (Female vs Male) | 1            | 10.47     | 0.0012               |
| (3 vs 1) (Female vs Male) | 1            | 9.28      | 0.0023               |
| (4 vs 1) (Female vs Male) | 1            | 5.48      | 0.0192               |
| (5 vs 1) (Female vs Male) | 1            | 0.50      | 0.4807               |
| (6 vs 1) (Female vs Male) | 1            | 0.65      | 0.4203               |
| Joint                     | 4            | 20.04     | 0.0005               |
| -----+-----               |              |           |                      |
|                           | Delta-method |           |                      |
|                           | Contrast     | std. err. | [95% conf. interval] |
| -----+-----               |              |           |                      |
| _at#female                |              |           |                      |
| (2 vs 1) (Female vs Male) | .0025709     | .0007946  | .0010134 .0041283    |
| (3 vs 1) (Female vs Male) | .0053185     | .001746   | .0018964 .0087407    |
| (4 vs 1) (Female vs Male) | .0068713     | .0029351  | .0011187 .0126239    |
| (5 vs 1) (Female vs Male) | .0038458     | .0054532  | -.0068422 .0145339   |
| (6 vs 1) (Female vs Male) | -.0102854    | .0127614  | -.0352972 .0147264   |
| -----+-----               |              |           |                      |

Variables that uniquely identify margins: age

file marginsplot3.svg saved as SVG format





From this we see the difference of effects between point 2 and 1 is .0026, with  $\chi^2(1) = 10.47$  and  $p = 0.0012$  — the same conclusion the `lincom` test gave above, and indeed the same test:  $3.24^2 = 10.5$ . The remaining rows compare each later age to age 20, and the widening gap is significant through age 50 ((4 vs 1):  $\chi^2 = 5.48$ ,  $p = 0.019$ ) before losing precision.

## 6.2. Summary

Overlapping confidence intervals do not mean two estimates are statistically indistinguishable. A test of the difference depends on the variance of the difference,  $\text{Var}(x - y) = \text{Var}(x) + \text{Var}(y) - 2 \text{Cov}(x, y)$ , and the covariance is nowhere to be seen in a plot of the two intervals separately. Estimates read off the same fitted model at nearby covariate values are strongly positively correlated — 0.99 in this example — so most of their variance cancels in the difference, and the difference is pinned down far more precisely than either level is. The eyeball test is therefore conservative in a specific and misleading way: it reports no difference where a proper test finds one.

The chapter's numbers make the size of the error concrete. The female-vs-male contrasts at ages 20 and 30 have intervals overlapping across most of their length, yet their difference is 0.0026 with a standard error of 0.00079,  $z = 3.24$ ,  $p = 0.001$ . Ignoring the covariance and combining the two standard errors as  $\sqrt{\text{Var}(x) + \text{Var}(y)}$  would give  $z = 0.73$  and  $p = 0.46$  — same data, opposite conclusion.

So do not read differences off a `marginsplot`. Ask for them directly:

- `margins r.varname` returns a contrast rather than two levels, with the covariance already handled.

## 6. Comparing Marginal effects with *margins* command

- `margins ..., contrast(atcontrast(r))` compares every evaluation point to the reference in one call, which is what produced the last table above.
- `margins ..., post` followed by `lincom` or `test` handles any single contrast you can write down, using only built-in commands.

The plot is still the right tool for showing how an effect moves across a covariate. It is the wrong tool for judging whether two points on it differ.

## 7. Fixed or Random Effect, or Both?

### 7.1. Panel data

With panel data — repeated observations over time or observations clustered at a higher level — the standard choice is between fixed and random effects. Consider

$$y_{it} = \beta_0 + \beta_1 x_{it} + c_i + \epsilon_{it} \quad (7.1)$$

for individuals  $i = 1, \dots, n$  measured at  $t = 1, \dots, T$ , where  $c_i$  is an unobserved time-invariant individual effect. The two approaches differ in how they handle  $c_i$ .

Fixed effects eliminate  $c_i$  by demeaning or by including individual dummies (the two are equivalent in a linear model). In nonlinear models only Poisson avoids the incidental-parameter problem; the standard solution for other models is conditional likelihood (conditional logit, for example), which absorbs the fixed effects in the likelihood. When no conditional likelihood exists, the incidental-parameter bias shrinks with panel depth but does not vanish.

Random effects treat  $c_i$  as part of the error term. This is more efficient but requires  $x_{it}$  to be uncorrelated with  $c_i$  — an assumption economists are generally skeptical of.

### 7.2. Time-invariant variables

Sometimes we want the effect of a time-invariant variable, so the model becomes

$$y_{it} = \beta_0 + \beta_1 x_{it} + c_i + \gamma z_i + \epsilon_{it} \quad (7.2)$$

Fixed effects cannot identify  $\gamma$  because  $z_i$  is perfectly collinear with  $c_i$ . Random effects can still estimate both, treating  $z_i$  as another covariate.

### 7.3. Between-within model

The textbook advice is a Hausman test: random effects are more efficient when the uncorrelatedness assumption holds; if not, fixed effects are still consistent. The between-within model (BW) sidesteps the choice. [Neuhaus and Kalbfleisch \(1998\)](#) introduced the BW estimator,

$$y_{it} = \beta_0 + \beta_1(x_{it} - \bar{x}_i) + \beta_2 \bar{x}_i + c_i + \gamma z_i + \epsilon_{it} \quad (7.3)$$

## 7. Fixed or Random Effect, or Both?

Here  $\beta_1$  is the within effect — identical to the fixed-effect coefficient.  $\beta_2$  is the between effect (mean of  $x$  on mean of  $y$ ).  $\gamma$  is the effect of the time-invariant variable.

The other specification of BW estimator is

$$y_{it} = \beta_0 + \beta_1 x_{it} + \beta_2 \bar{x}_i + c_i + \gamma z_i + \epsilon_{it} \quad (7.4)$$

This is the same model reparametrized.  $\beta_1$  is unchanged, and  $\beta_2$  is now the between-minus-within difference (substitute  $\beta_1(x_{it} - \bar{x}_i) + \beta_2 \bar{x}_i = \beta_1 x_{it} + (\beta_2 - \beta_1) \bar{x}_i$  to see this). This is the “contextual model”;  $\beta_2$  is the contextual effect and acts as an embedded Hausman test.

The BW model has two advantages. It recovers the fixed-effect coefficient while estimating time-invariant covariates. And it extends naturally to cross-level interactions, random slopes, and other multilevel structures. Implementation is straightforward: run a random-effects model on either equation above.

### 7.3.1. Is this just Mundlak?

Yes. Equation 7.4 is the Mundlak device: Mundlak’s regression puts  $x_{it}$  and  $\bar{x}_i$  on the right-hand side of a random-effects model, which is exactly what that equation does. So the between-within model and the Mundlak device are one model in two parameterisations, fitted by the same random-effects command. It is worth doing the algebra, because the identity is easy to state loosely and easy to get wrong on the inference side.

Write  $d_{it} = x_{it} - \bar{x}_i$  for the within deviation, and label the two specifications by their coefficients:  $b_1, b_2$  for the hybrid form Equation 7.3 and  $p_1, p_2$  for the contextual form Equation 7.4. Expanding the hybrid regressor block,

$$b_1(x_{it} - \bar{x}_i) + b_2 \bar{x}_i = b_1 x_{it} + (b_2 - b_1) \bar{x}_i, \quad (7.5)$$

so  $p_1 = b_1$  and  $p_2 = b_2 - b_1$ . Substitution shows the map, but the reason behind it is worth stating more strongly: the two regressor matrices are related by an invertible linear transformation. With  $X = [x_{it}, \bar{x}_i]$  and  $W = [d_{it}, \bar{x}_i]$ ,

$$W = XA, \quad A = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}, \quad A^{-1} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad (7.6)$$

and  $\det A = 1$ , so the two designs span the same column space. Everything that depends only on that space is numerically identical: fitted values, residuals, the likelihood, the estimated variance components, and the GLS quasi-demeaning weight  $\theta$ . Only the coordinates change, by  $b = A^{-1}p$ , and the covariance matrix by  $\text{Var}(b) = A^{-1}\text{Var}(p)A^{-1'}$ . Componentwise that is

$$\text{Var}(p_1) = \text{Var}(b_1), \quad \text{Var}(p_2) = \text{Var}(b_1) + \text{Var}(b_2) + 2\text{Cov}(b_1, b_2), \quad (7.7)$$

which already says something useful: the standard error printed next to the between effect is not the standard error you want for the contextual effect.

### 7.3.1.1. Why the within coefficient is the fixed-effect estimator

This part needs a proof rather than a substitution. The relevant fact is that  $\sum_t d_{it} = 0$  within every unit by construction, so  $d$  is orthogonal to any group-invariant column: for  $v_i$  constant within  $i$ ,

$$\sum_{i,t} d_{it} v_i = \sum_i v_i \sum_t d_{it} = 0, \quad (7.8)$$

which covers the intercept,  $\bar{x}_i$  and  $z_i$  alike.

Random effects is OLS after quasi-demeaning,  $\tilde{w}_{it} = w_{it} - \theta \bar{w}_i$  with  $\theta = 1 - \sqrt{\sigma_\epsilon^2 / (\sigma_\epsilon^2 + T\sigma_c^2)}$ . Apply it to the hybrid regressors. Because  $\bar{d}_i = 0$  we get  $\tilde{d}_{it} = d_{it}$ : the within-deviation regressor passes through the GLS transform untouched, whatever  $\theta$  is. The remaining regressors are group-invariant and stay so after scaling by  $1 - \theta$ . Orthogonality therefore survives the transform, and by Frisch-Waugh-Lovell

$$b_1 = \frac{\sum d_{it} \tilde{y}_{it}}{\sum d_{it}^2} = \frac{\sum d_{it} y_{it}}{\sum d_{it}^2} = \frac{\sum (x_{it} - \bar{x}_i)(y_{it} - \bar{y}_i)}{\sum (x_{it} - \bar{x}_i)^2}, \quad (7.9)$$

the second and third equalities both by Equation 7.8. That last expression is the within estimator. Note that  $\theta$  has dropped out, so the identity holds for *any*  $\theta$  — for random effects, for pooled OLS ( $\theta = 0$ ) and for fixed effects ( $\theta = 1$ ). That is why running a random-effects model on either equation returns the fixed-effect coefficient. It also gives  $\text{Cov}(b_1, b_2) = 0$  exactly, which combined with Equation 7.7 yields  $\text{Var}(p_2) = \text{Var}(b_1) + \text{Var}(b_2)$  and  $\text{Cov}(p_1, p_2) = -\text{Var}(b_1)$ .

All of this is checkable on the chapter's own data. We take a single regressor, weeks worked, fit both parameterisations by `plm`, and compare against the within estimator and against the variance map:

```
library(plm); library(panelr); data("WageData")
d <- WageData
d$xbar <- ave(d$wks, d$id)           # unit mean
d$xdev <- d$wks - d$xbar             # within deviation

fe <- plm(lwage ~ wks,               data = d, index = c("id", "t"), model = "within")
hyb <- plm(lwage ~ xdev + xbar,      data = d, index = c("id", "t"), model = "random")
mun <- plm(lwage ~ wks + xbar,      data = d, index = c("id", "t"), model = "random")

b <- coef(hyb); p <- coef(mun); Vh <- vcov(hyb); Vm <- vcov(mun)

data.frame(
  check = c("p1 - b1", "p2 - (b2 - b1)", "b1 - FE within",
            "sum(xdev * xbar)", "Cov(b1, b2)",
            "Var(p2) - [Var(b1) + Var(b2)]", "Cov(p1, p2) + Var(b1)",
            "theta(hybrid) - theta(Mundlak)"),
  value = c(p["wks"] - b["xdev"],
            p["xbar"] - (b["xbar"] - b["xdev"]),
            b["xdev"] - coef(fe)["wks"],
```

## 7. Fixed or Random Effect, or Both?

```
sum(d$xdev * d$xbar),  
Vh["xdev", "xbar"],  
Vm["xbar", "xbar"] - (Vh["xdev", "xdev"] + Vh["xbar", "xbar"]),  
Vm["wks", "xbar"] + Vh["xdev", "xdev"],  
hyb$ercomp$theta - mun$ercomp$theta),  
row.names = NULL)
```

|   | check                          | value         |
|---|--------------------------------|---------------|
| 1 | p1 - b1                        | 2.385245e-18  |
| 2 | p2 - (b2 - b1)                 | 7.459311e-17  |
| 3 | b1 - FE within                 | 1.886512e-17  |
| 4 | sum(xdev * xbar)               | 3.534950e-12  |
| 5 | Cov(b1, b2)                    | -5.676184e-21 |
| 6 | Var(p2) - [Var(b1) + Var(b2)]  | -4.404571e-20 |
| 7 | Cov(p1, p2) + Var(b1)          | 2.117582e-21  |
| 8 | theta(hybrid) - theta(Mundlak) | 1.110223e-16  |

Every entry is zero to machine precision. The two parameterisations are the same fit, the within coefficient is the fixed-effect estimate, the hybrid regressors are orthogonal, and the variance map holds as derived.

### 7.3.1.2. So why keep both

The fit is one object, but the parameterisation settles three practical things.

*Which quantity is printed.*  $b_2$  is the between effect and  $p_2$  is between minus within. They are different parameters with non-interchangeable standard errors, and whichever form you fit, the other costs a `lincom`.

*Which hypothesis is one line.*  $p_2 = 0$  says between equals within, which is exactly the random-effects assumption that the regressors are uncorrelated with  $c_i$ . The Mundlak form prints that as a single  $t$ ; in hybrid form it is the contrast  $b_1 = b_2$ . The two tests agree numerically, but only one is printed. This is the Mundlak form of the Hausman test, asymptotically equivalent to the classical version and with the practical advantage of working directly with cluster-robust standard errors. The [next chapter](#) develops it and its two-way extension for difference-in-differences.

*What the second coefficient is for.* In multilevel work  $p_2$  is not a diagnostic but the estimand: holding a pupil's own status fixed, what does the school mean do? The hybrid's  $b_2$  answers a different question, the total between-unit association. Two literatures ask two questions of one regression, and each named the parameterisation that printed its answer. That is the honest reason both names survive.

There is also a reason to prefer the hybrid form in extensions. Its two regressors are exactly orthogonal, where  $x_{it}$  and  $\bar{x}_i$  are correlated. That does not change the fit, but if you want a random slope on the within component you need the deviation as its own column; a random slope on  $x_{it}$  would mix within and between variation.

### 7.3.1.3. Where the equivalence stops

The reparameterisation is exact for anything with a linear index, logit and Poisson mixed models included, since  $XA$  spans the same space and the index is unchanged. The two names stay interchangeable there.

The fixed-effect identity does not carry over. The proof above used two linear-model facts: that random effects is OLS on quasi-demeaned data, and that  $d$  is orthogonal to group constants. A logit or Poisson mixed model has no such transform, the unit effects do not sweep out, and the hybrid within coefficient is not the conditional-logit estimate but an approximation to it. Keep the two claims separate: the parameterisations are always the same model, while “within coefficient equals fixed effects” is a linear-model result. The last section of this chapter returns to that point.

## 7.4. BW model in R

R’s `panelr` package implements BW models directly. The data are `WageData`, a balanced panel of 595 men observed over 7 years. We model log wage on weeks worked, union membership, marital status and occupation, all time-varying, plus two time-invariant covariates — black and female — that a fixed-effect model could not touch.

```
library(panelr)
data("WageData")
wages <- panel_data(WageData, id = id, wave = t)
model1 <- wbm(lwage ~ wks + union + ms + occ | blk + fem, data = wages)
summary(model1)
```

MODEL INFO:

Entities: 595

Time periods: 1-7

Dependent variable: lwage

Model type: Linear mixed effects

Specification: within-between

MODEL FIT:

AIC = 2036.78, BIC = 2119.13

Pseudo- $R^2$  (fixed effects) = 0.27

Pseudo- $R^2$  (total) = 0.69

Entity ICC = 0.57

WITHIN EFFECTS:

|       | Est.  | S.E. | t val. | p    |
|-------|-------|------|--------|------|
| wks   | 0.00  | 0.00 | 1.06   | 0.29 |
| union | 0.06  | 0.03 | 2.53   | 0.01 |
| ms    | -0.08 | 0.03 | -2.57  | 0.01 |

## 7. Fixed or Random Effect, or Both?

```
occ          -0.08   0.02   -3.32   0.00
```

### BETWEEN EFFECTS:

|              | Est.  | S.E. | t val. | p    |
|--------------|-------|------|--------|------|
| (Intercept)  | 6.30  | 0.20 | 30.85  | 0.00 |
| imean(wks)   | 0.01  | 0.00 | 2.25   | 0.02 |
| imean(union) | 0.15  | 0.03 | 4.67   | 0.00 |
| imean(ms)    | 0.17  | 0.05 | 3.07   | 0.00 |
| imean(occ)   | -0.41 | 0.03 | -13.31 | 0.00 |
| blk          | -0.15 | 0.05 | -2.81  | 0.00 |
| fem          | -0.32 | 0.06 | -4.96  | 0.00 |

p values calculated using df = 4153

### RANDOM EFFECTS:

| Group    | Parameter   | Std. Dev. |
|----------|-------------|-----------|
| id       | (Intercept) | 0.2992    |
| Residual |             | 0.2589    |

The within estimates are 0.00 for weeks, 0.06 for union,  $-0.08$  for married and  $-0.08$  for occupation. The between estimates for the same four variables are 0.01, 0.15, 0.17 and  $-0.41$  — two to five times larger, and for `ms` the opposite sign. That gap is the reason the fixed-versus-random choice matters here: a random-effects model without the decomposition would blend the two and report neither. The time-invariant covariates come out at  $-0.15$  for black and  $-0.32$  for female, quantities a fixed-effect model cannot produce at all.

Compare with `lfe`.

```
library(lfe)
model2 <- felm(lwage ~ wks + union + ms + occ | id, data = wages)
summary(model2)
```

Call:

```
felm(formula = lwage ~ wks + union + ms + occ | id, data = wages)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-1.89500 -0.16174  0.00652  0.17060  1.94521
```



Coefficients:

|       | Estimate  | Std. Error | t value | Pr(> t )     |
|-------|-----------|------------|---------|--------------|
| wks   | 0.001083  | 0.001019   | 1.063   | 0.287816     |
| union | 0.064320  | 0.025378   | 2.534   | 0.011305 *   |
| ms    | -0.082905 | 0.032226   | -2.573  | 0.010132 *   |
| occ   | -0.077507 | 0.023359   | -3.318  | 0.000916 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2589 on 3566 degrees of freedom

Multiple R-squared(full model): 0.7304 Adjusted R-squared: 0.6852

Multiple R-squared(proj model): 0.006509 Adjusted R-squared: -0.1601

F-statistic(full model):16.16 on 598 and 3566 DF, p-value: &lt; 2.2e-16

F-statistic(proj model): 5.841 on 4 and 3566 DF, p-value: 0.0001106

`felm` gives 0.001083, 0.064320, -0.082905 and -0.077507 — the same four numbers `panelr` reported as within effects, identical to machine precision (`WageData` is a balanced panel: 7 waves, 595 individuals). That is the BW model's selling point: it reproduces the fixed-effect estimates exactly while also estimating the time-invariant covariates `blk` and `fem`, which `felm` cannot. `lfe` supports clustered standard errors, which `panelr` does not.

```
model3 <- felm(lwage ~ wks + union + ms + occ | id | 0 | id, data = wages)
summary(model3)
```

Call:

```
felm(formula = lwage ~ wks + union + ms + occ | id | 0 | id, data = wages)
```

Residuals:

|  | Min      | 1Q       | Median  | 3Q      | Max     |
|--|----------|----------|---------|---------|---------|
|  | -1.89500 | -0.16174 | 0.00652 | 0.17060 | 1.94521 |

Coefficients:

|       | Estimate  | Cluster s.e. | t value | Pr(> t ) |
|-------|-----------|--------------|---------|----------|
| wks   | 0.001083  | 0.001331     | 0.814   | 0.4160   |
| union | 0.064320  | 0.040936     | 1.571   | 0.1167   |
| ms    | -0.082905 | 0.047399     | -1.749  | 0.0808 . |
| occ   | -0.077507 | 0.031320     | -2.475  | 0.0136 * |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2589 on 3566 degrees of freedom

Multiple R-squared(full model): 0.7304 Adjusted R-squared: 0.6852

Multiple R-squared(proj model): 0.006509 Adjusted R-squared: -0.1601

F-statistic(full model, \*iid\*):16.16 on 598 and 3566 DF, p-value: &lt; 2.2e-16

F-statistic(proj model): 3.456 on 4 and 594 DF, p-value: 0.008358

## 7. Fixed or Random Effect, or Both?

Clustering on `id` leaves every coefficient untouched and changes the standard errors substantially: `union` goes from 0.0254 to 0.0409, `ms` from 0.0322 to 0.0474, `wks` from 0.00102 to 0.00133. Two of the four results do not survive. `union` falls from  $p = 0.011$  to  $p = 0.117$  and `ms` from 0.010 to 0.081; only `occ` stays significant at 5%. With 7 observations per man the residuals are serially correlated, and the unclustered standard errors were treating 4,165 observations as though they were independent draws.

## 7.5. BW model in Stata

Stata has no dedicated BW command, but `xtreg` suffices. We switch to `nlswork`, an unbalanced panel of 4,710 women over 28,510 person-years, and regress log wage on age.

```
webuse nlswork
xtset idcode
xtreg ln_w age, fe cluster(idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

Panel variable: `idcode` (unbalanced)

|                                     |                  |   |        |
|-------------------------------------|------------------|---|--------|
| Fixed-effects (within) regression   | Number of obs    | = | 28,510 |
| Group variable: <code>idcode</code> | Number of groups | = | 4,710  |
| R-squared:                          | Obs per group:   |   |        |
| Within = 0.1026                     | min =            |   | 1      |
| Between = 0.0877                    | avg =            |   | 6.1    |
| Overall = 0.0774                    | max =            |   | 15     |
|                                     | F(1, 4709)       | = | 884.05 |
| <code>corr(u_i, Xb) = 0.0314</code> | Prob > F         | = | 0.0000 |

(Std. err. adjusted for 4,710 clusters in `idcode`)

|                      |             | Robust                                          |       |       |                      |          |
|----------------------|-------------|-------------------------------------------------|-------|-------|----------------------|----------|
| <code>ln_wage</code> | Coefficient | std. err.                                       | t     | P> t  | [95% conf. interval] |          |
| <code>age</code>     | .0181349    | .0006099                                        | 29.73 | 0.000 | .0169392             | .0193306 |
| <code>_cons</code>   | 1.148214    | .0177153                                        | 64.81 | 0.000 | 1.113483             | 1.182944 |
| <code>sigma_u</code> | .40635023   |                                                 |       |       |                      |          |
| <code>sigma_e</code> | .30349389   |                                                 |       |       |                      |          |
| <code>rho</code>     | .64192015   | (fraction of variance due to <code>u_i</code> ) |       |       |                      |          |

The fixed-effect estimate is .0181349 with a clustered standard error of .0006099: within a woman, each additional year of age raises log wage by about 1.8%. Keep that number in view — the BW model has to reproduce it.

Generate the group mean and run the BW estimation.

```
webuse nlswork
xtset idcode
bysort idcode: center age, prefix(d) mean(m)
xtreg ln_w dage mage i.race, re cluster(idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

Panel variable: idcode (unbalanced)

(generated variables: dage mage)

|                               |                  |   |         |
|-------------------------------|------------------|---|---------|
| Random-effects GLS regression | Number of obs    | = | 28,510  |
| Group variable: idcode        | Number of groups | = | 4,710   |
| R-squared:                    | Obs per group:   |   |         |
| Within = 0.1026               | min =            |   | 1       |
| Between = 0.1040              | avg =            |   | 6.1     |
| Overall = 0.0950              | max =            |   | 15      |
|                               | Wald chi2(4)     | = | 1335.89 |
| corr(u_i, X) = 0 (assumed)    | Prob > chi2      | = | 0.0000  |

(Std. err. adjusted for 4,710 clusters in idcode)

| ln_wage | Coefficient | Robust<br>std. err.               | z     | P> z  | [95% conf. interval] |           |
|---------|-------------|-----------------------------------|-------|-------|----------------------|-----------|
| dage    | .0181349    | .00061                            | 29.73 | 0.000 | .0169394             | .0193304  |
| mage    | .022558     | .0011405                          | 19.78 | 0.000 | .0203226             | .0247933  |
| race    |             |                                   |       |       |                      |           |
| Black   | -.1190246   | .0127418                          | -9.34 | 0.000 | -.1439982            | -.0940511 |
| Other   | .0974996    | .0617364                          | 1.58  | 0.114 | -.0235016            | .2185008  |
| _cons   | 1.037566    | .0323185                          | 32.10 | 0.000 | .9742233             | 1.10091   |
| sigma_u | .36581005   |                                   |       |       |                      |           |
| sigma_e | .30349389   |                                   |       |       |                      |           |
| rho     | .59230575   | (fraction of variance due to u_i) |       |       |                      |           |

## 7. Fixed or Random Effect, or Both?

In this BW model, we use the centered (within-group-demeaned) `dage` together with the group mean `mage`, which gives the standard within-between (Mundlak) decomposition: the coefficient on `dage`, .0181349, is exactly the fixed-effect (“within”) coefficient on age reported above — the same seven digits, with a standard error of .00061 against .0006099 — and the coefficient on `mage`, .022558, is directly the between effect. Women who are older on average earn about 2.3% more per year of average age, against a within-woman return of 1.8%; the between effect is larger because it also carries cohort and selection differences that the within estimate sweeps out. And we have the effect of time-invariant covariate race estimated: Black women earn 11.9% less (−.1190246, SE .0127), while the “Other” category is not distinguishable from white (.0975,  $p = 0.114$ ). The advantage of using `xtreg` is that we have clustered standard errors implemented.

Note: if we instead ran `xtreg ln_w age mage i.race, re cluster(idcode)` — using the raw (uncentered) `age` together with `mage` — we would get the *contextual model* instead: the coefficient on `age` stays .0181, but the coefficient on `mage` becomes .0044, the “contextual effect” (the *additional* between-effect on top of the within effect). The two models are algebraically related: contextual-model `mage` = Mundlak-model `mage` − Mundlak-model `dage` (here, .0226 − .0181 = .0044).

### 7.6. BW model in non-linear models

Allison applies the BW model to binary outcomes. The bias relative to conditional logit is an open question, but if the linear probability model is a reasonable approximation the BW approach should perform similarly for binary panel data.

## 8. The Mundlak Device in Panel Data and Difference-in-Differences

### 8.1. Introduction: Fixed Effects vs. Mundlak Regression

In panel data models with individual effects  $c_i$ :

$$y_{it} = X_{it}\beta + c_i + u_{it}$$

Standard Fixed Effects (within estimation) eliminates  $c_i$  by demeaning, allowing  $c_i$  to correlate arbitrarily with  $X_{it}$ . However, within estimation cannot estimate the effects of time-invariant variables (e.g., gender, race, schooling) because their within-unit variance is zero.

Mundlak (1978) proposed modeling  $c_i$  as a linear projection on the unit-specific means of the time-varying covariates:

$$c_i = \bar{X}_i\theta + \eta_i, \quad E[\eta_i | X_i] = 0$$

Substituting into the regression yields:

$$y_{it} = X_{it}\beta + \bar{X}_i\theta + \eta_i + u_{it}$$

This equation can be estimated by Pooled OLS (POLS) or Random Effects (RE). By the Frisch–Waugh–Lovell (FWL) theorem, the resulting estimate of  $\beta$  is algebraically identical to the fixed-effects estimator.

Furthermore: 1. **Hausman Test:** Testing  $H_0 : \theta = 0$  is asymptotically equivalent to the classical Hausman test, and works directly with cluster-robust standard errors. 2. **Time-Invariant Regressors:** Time-invariant covariates can be included directly in the regression. Because within-unit deviations are orthogonal to any time-invariant regressor by construction, their inclusion does not alter  $\hat{\beta}$ .

### 8.2. Empirical Demonstration: One-Way Panel (nlswork)

Using Stata's `nlswork` dataset (19,229 person-years):

## 8. The Mundlak Device in Panel Data and Difference-in-Differences

```
clear
webuse nlswork
xtset idcode year
drop if union==. | age==.
bysort idcode: egen mean_age = mean(age)
bysort idcode: egen mean_union = mean(union)

* 1. Standard FE
xtreg ln_wage age union, fe cluster(idcode)

* 2. Pooled OLS with Mundlak means
reg ln_wage age union mean_age mean_union, cluster(idcode)

* 3. Random Effects with Mundlak means
xtreg ln_wage age union mean_age mean_union, re cluster(idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

Panel variable: idcode (unbalanced)  
Time variable: year, 68 to 88, but with gaps  
Delta: 1 unit

(9,305 observations deleted)

|                                   |                  |   |        |
|-----------------------------------|------------------|---|--------|
| Fixed-effects (within) regression | Number of obs    | = | 19,229 |
| Group variable: idcode            | Number of groups | = | 4,150  |
| R-squared:                        | Obs per group:   |   |        |
| Within = 0.0963                   | min =            |   | 1      |
| Between = 0.0433                  | avg =            |   | 4.6    |
| Overall = 0.0562                  | max =            |   | 12     |
|                                   | F(2, 4149)       | = | 331.52 |
| corr(u_i, Xb) = 0.0127            | Prob > F         | = | 0.0000 |

(Std. err. adjusted for 4,150 clusters in idcode)

|         |             | Robust    |       |       |                      |  |
|---------|-------------|-----------|-------|-------|----------------------|--|
| ln_wage | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |  |
| age     | .0153507    | .0006912  | 22.21 | 0.000 | .0139956 .0167058    |  |
| union   | .1055274    | .0098582  | 10.70 | 0.000 | .0862001 .1248546    |  |
| _cons   | 1.248435    | .0215661  | 57.89 | 0.000 | 1.206154 1.290716    |  |

## 8.2. Empirical Demonstration: One-Way Panel (nlswork)

|         |  |           |                                   |
|---------|--|-----------|-----------------------------------|
| sigma_u |  | .42353003 |                                   |
| sigma_e |  | .26213464 |                                   |
| rho     |  | .72302816 | (fraction of variance due to u_i) |

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 19,229 |
|                   | F(4, 4149)    | = | 234.77 |
|                   | Prob > F      | = | 0.0000 |
|                   | R-squared     | = | 0.0769 |
|                   | Root MSE      | = | .44953 |

(Std. err. adjusted for 4,150 clusters in idcode)

|            |             | Robust    |       |       |                      |           |
|------------|-------------|-----------|-------|-------|----------------------|-----------|
| ln_wage    | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |           |
| age        | .0153507    | .0006912  | 22.21 | 0.000 | .0139956             | .0167059  |
| union      | .1055274    | .0098587  | 10.70 | 0.000 | .0861991             | .1248556  |
| mean_age   | -.0064208   | .001548   | -4.15 | 0.000 | -.0094556            | -.0033859 |
| mean_union | .1900785    | .0223172  | 8.52  | 0.000 | .1463249             | .2338321  |
| _cons      | 1.405264    | .042734   | 32.88 | 0.000 | 1.321482             | 1.489046  |

|                               |                  |   |        |
|-------------------------------|------------------|---|--------|
| Random-effects GLS regression | Number of obs    | = | 19,229 |
| Group variable: idcode        | Number of groups | = | 4,150  |

|                  |                |
|------------------|----------------|
| R-squared:       | Obs per group: |
| Within = 0.0963  | min = 1        |
| Between = 0.0750 | avg = 4.6      |
| Overall = 0.0768 | max = 12       |

|                            |              |   |         |
|----------------------------|--------------|---|---------|
|                            | Wald chi2(4) | = | 1023.99 |
| corr(u_i, X) = 0 (assumed) | Prob > chi2  | = | 0.0000  |

(Std. err. adjusted for 4,150 clusters in idcode)

|            |             | Robust    |       |       |                      |           |
|------------|-------------|-----------|-------|-------|----------------------|-----------|
| ln_wage    | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |           |
| age        | .0153507    | .0006912  | 22.21 | 0.000 | .0139959             | .0167055  |
| union      | .1055274    | .0098587  | 10.70 | 0.000 | .0862047             | .12485    |
| mean_age   | -.0061464   | .0014254  | -4.31 | 0.000 | -.0089401            | -.0033527 |
| mean_union | .212183     | .0210098  | 10.10 | 0.000 | .1710046             | .2533614  |
| _cons      | 1.362101    | .0383784  | 35.49 | 0.000 | 1.286881             | 1.437322  |

## 8. The Mundlak Device in Panel Data and Difference-in-Differences

```
sigma_u | .38505558
sigma_e | .26213464
rho     | .68331727    (fraction of variance due to u_i)
```

---

All three models yield identical coefficients:  $\hat{\beta}_{\text{age}} = 0.0153507$  (SE = 0.00069) and  $\hat{\beta}_{\text{union}} = 0.1055274$  (SE = 0.00986).

### 8.2.1. Estimating Time-Invariant Covariates

We add `race` (constant within individual) to the Mundlak Pooled OLS model:

```
reg ln_wage age union mean_age mean_union i.race, cluster(idcode)
```

```
Linear regression              Number of obs    =      19,229
                              F(6, 4149)         =      189.66
                              Prob > F           =       0.0000
                              R-squared           =       0.1069
                              Root MSE        =       .4422
```

(Std. err. adjusted for 4,150 clusters in idcode)

|            | ln_wage | Coefficient | Robust<br>std. err. | t      | P> t  | [95% conf. interval] |           |
|------------|---------|-------------|---------------------|--------|-------|----------------------|-----------|
| age        |         | .0153507    | .0006913            | 22.21  | 0.000 | .0139955             | .0167059  |
| union      |         | .1055274    | .0098592            | 10.70  | 0.000 | .0861981             | .1248566  |
| mean_age   |         | -.0070345   | .0015342            | -4.59  | 0.000 | -.0100424            | -.0040266 |
| mean_union |         | .2227511    | .0220026            | 10.12  | 0.000 | .1796142             | .265888   |
| race       |         |             |                     |        |       |                      |           |
| Black      |         | -.1790995   | .0143624            | -12.47 | 0.000 | -.2072575            | -.1509415 |
| Other      |         | .0882461    | .0715315            | 1.23   | 0.217 | -.0519939            | .2284861  |
| _cons      |         | 1.466452    | .0426087            | 34.42  | 0.000 | 1.382916             | 1.549988  |

The coefficients on `age` and `union` remain unchanged to seven digits, while the coefficient on `race` is identified (Black women earn −17.9% relative to white women, SE = 0.0144).

## 8.3. Two-Way Mundlak (TWM) and the Balanced Panel Condition

Wooldridge (2021) extends Mundlak's idea to Two-Way Fixed Effects (TWFE):



$$y_{it} = X_{it}\beta + c_i + f_t + u_{it}$$

The Two-Way Mundlak (TWM) estimator adds both unit means ( $\bar{X}_{i\cdot}$ ) and time means ( $\bar{X}_{\cdot t}$ ):

$$y_{it} = X_{it}\beta + \bar{X}_{i\cdot}\lambda + \bar{X}_{\cdot t}\theta + u_{it}$$

### 8.3.1. The Unbalanced Panel Trap

Equivalence between TWM and TWFE requires the unit and time factors to be **orthogonal**, which holds automatically in **balanced panels** where every unit is observed in every period.

In **unbalanced panels**, simple unit and time means do not span the space of two-way dummies. Equivalence is restored by including the **unit means of the time dummies** ( $\bar{d}_{it}$ ), which conditions on each unit's observation pattern across time ( $s_{it}/T_i$ ).

```

preserve
webuse nlswork, clear
keep if !missing(ln_wage, age, union)
xtset idcode year

bysort idcode: egen m_age    = mean(age)
bysort idcode: egen m_union = mean(union)
egen t_age    = mean(age),    by(year)
egen t_union  = mean(union), by(year)

* Unit means of time dummies
levelsof year, local(yrs)
foreach y of local yrs {
    bysort idcode: egen my_`y' = mean(year==`y')
}

quietly xtreg ln_wage age union i.year, fe
scalar b_twfe = _b[age]

* Naive TWM with unit and time means only
quietly regress ln_wage age union m_age m_union t_age t_union
scalar b_twm = _b[age]

* Correct TWM with unit means of time dummies
quietly regress ln_wage age union m_age m_union i.year my_*
scalar b_fix = _b[age]

di "TWFE"                age = " %9.7f b_twfe
di "Naive TWM (unit + time means)" age = " %9.7f b_twm
di "TWM + unit means of dummies"   age = " %9.7f b_fix

```

```
restore
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

(9,305 observations deleted)

Panel variable: idcode (unbalanced)

Time variable: year, 70 to 88, but with gaps

Delta: 1 unit

70 71 72 73 77 78 80 82 83 85 87 88

TWFE                                      age = 0.0276645

Naive TWM (unit + time means)      age = 0.0117968

TWM + unit means of dummies        age = 0.0276646

On this unbalanced panel: - TWFE gives  $\hat{\beta}_{\text{age}} = 0.0276645$ . - Naive TWM gives 0.0117968 (severely biased, off by a factor of  $2.35\times$ ). - Adding the unit means of the time dummies restores exact numerical equivalence to 6 decimal places (0.0276646).

## 8.4. Multi-Way Fixed Effects (> 2 Dimensions)

With three or more fixed-effect factors (e.g., individual  $i$ , year  $t$ , and group  $g$ ): - Do **not** compute one set of means per dimension unless factors are mutually orthogonal (e.g., a Latin square experiment). - **General Rule:** Select the high-dimensional factor to eliminate (typically individual  $i$ ), retain dummy indicators for the remaining lower-dimensional factors, and project onto the within- $i$  means of all time-varying regressors, including the retained dummies.

```
preserve
capture program drop twm3
program define twm3
```

```

args label
quietly {
    capture drop x_i x_t x_g mt_* mg_* x y ft hg
    sort t
    by t: gen double ft = rnormal() if _n==1
    by t: replace ft = ft[1]
    sort g
    by g: gen double hg = rnormal() if _n==1
    by g: replace hg = hg[1]
    gen double x = 0.6*ci + 0.5*ft + 0.4*hg + rnormal()
    gen double y = 1*x + ci + ft + hg + rnormal()
    bysort i: egen double x_i = mean(x)
    bysort t: egen double x_t = mean(x)
    bysort g: egen double x_g = mean(x)
    levelsof t, local(ts)
    foreach k of local ts {
        bysort i: egen double mt_`k' = mean(t==`k')
    }
    levelsof g, local(gs)
    foreach k of local gs {
        bysort i: egen double mg_`k' = mean(g==`k')
    }
    areg y x i.t i.g, absorb(i)
    scalar b_fe = _b[x]
    regress y x x_i x_t x_g
    scalar b_A = _b[x]
    regress y x x_i i.t i.g mt_* mg_*
    scalar b_B = _b[x]
}
di "`label'"
di "    Three-way FE                        = " %9.6f b_fe
di "    A: One set of means per factor      = " %9.6f b_A    "    gap: " %9.6f (b_A-b_fe)
di "    B: Means over i, dummies included   = " %9.6f b_B    "    gap: " %9.6f (b_B-b_fe)
end

clear
set seed 20260731
set obs 300
gen i = _n
gen double ci = rnormal()
expand 10
bysort i: gen t = _n

* Non-orthogonal design (random assignment of g)
gen g = 1 + int(runiform()*5)
twm3 "Non-orthogonal factor assignment:"

```

```
* Orthogonal design (Latin square)
replace g = mod(i+t,5)+1
twm3 "Orthogonal factor assignment (Latin square):"

capture program drop twm3
restore
```

Number of observations (\_N) was 0, now 300.

(2,700 observations created)

Non-orthogonal factor assignment:

|                                   |   |          |               |
|-----------------------------------|---|----------|---------------|
| Three-way FE                      | = | 0.994424 |               |
| A: One set of means per factor    | = | 1.007648 | gap: 0.013224 |
| B: Means over i, dummies included | = | 0.994424 | gap: 0.000000 |

(2,435 real changes made)

Orthogonal factor assignment (Latin square):

|                                   |   |          |               |
|-----------------------------------|---|----------|---------------|
| Three-way FE                      | = | 0.996471 |               |
| A: One set of means per factor    | = | 0.996471 | gap: 0.000000 |
| B: Means over i, dummies included | = | 0.996471 | gap: 0.000000 |

Construction B (means over  $i$  including factor dummies) is exact in both orthogonal and non-orthogonal designs.

## 8.5. Application to Difference-in-Differences (Common Timing)

In standard DiD with common treatment timing:

$$y_{it} = \beta w_{it} + c_i + f_t + u_{it}$$

where  $w_{it} = d_i \cdot \text{post}_t$ . Because  $d_i$  is time-invariant and  $\text{post}_t$  is unit-invariant: - Unit mean:  $\bar{w}_i = d_i \cdot \bar{p}$  - Time mean:  $\bar{w}_t = \bar{d} \cdot \text{post}_t$

Therefore, the Mundlak projection is spanned simply by including  $d_i$  and the post-treatment dummy (or full time dummies). This framework makes it straightforward to model heterogeneous treatment effects over time:

```

clear
set obs 100
set seed 1234567
gen id = _n
gen w1 = rnormal()
gen w2 = rnormal()
expand 4
bysort id: gen year=2011+_n
gen post=(year>2013)
gen d=(id>50)
gen w=d*post
gen x1 = rnormal() + w1 + 2*w2
gen u = rnormal()

gen f2014=(year==2014)
gen f2015=(year==2015)

* DGP with growing treatment effect: 2.5 in 2014, 3.5 in 2015 (ATT = 3.0)
gen y1 = 1 + 2*x1 + 3*w1 + u
gen logy = y1 + 2.5*d*f2014 + 3.5*d*f2015

xtset id year

```

Number of observations (\_N) was 0, now 100.

(300 observations created)

Panel variable: id (strongly balanced)  
 Time variable: year, 2012 to 2015  
 Delta: 1 unit

## 8. The Mundlak Device in Panel Data and Difference-in-Differences

```
* 1. TWFE with year-specific treatment effects
xtreg logy c.w#c.f2014 c.w#c.f2015 i.year, fe vce(cluster id)

* 2. Pooled OLS with Mundlak controls (d and year dummies)
reg logy c.w#c.f2014 c.w#c.f2015 d f2014 f2015, vce(cluster id)
lincom (c.w#c.f2014 + c.w#c.f2015)/2

* 3. Adding time-invariant controls (x1 and interaction with d)
reg logy c.w#c.f2014 c.w#c.f2015 d f2014 f2015 x1 c.d#c.x1, vce(cluster id)
```

```
Fixed-effects (within) regression      Number of obs   =      400
Group variable: id                    Number of groups =      100

R-squared:                            Obs per group:
    Within = 0.2575                      min =          4
    Between = 0.0129                     avg =         4.0
    Overall = 0.0387                     max =          4

                                F(5, 99)      =      21.98
corr(u_i, Xb) = -0.0027                Prob > F       =      0.0000
```

(Std. err. adjusted for 100 clusters in id)

|             | logy | Coefficient | Robust<br>std. err.               | t     | P> t  | [95% conf. interval] |          |
|-------------|------|-------------|-----------------------------------|-------|-------|----------------------|----------|
| c.w#c.f2014 |      | 2.813593    | .5368268                          | 5.24  | 0.000 | 1.748412             | 3.878774 |
| c.w#c.f2015 |      | 3.600121    | .5765523                          | 6.24  | 0.000 | 2.456116             | 4.744126 |
| year        |      |             |                                   |       |       |                      |          |
| 2013        |      | .3260046    | .3325494                          | 0.98  | 0.329 | -.3338456            | .9858548 |
| 2014        |      | -.341502    | .4488412                          | -0.76 | 0.449 | -1.2321              | .5490964 |
| 2015        |      | .4164204    | .4300378                          | 0.97  | 0.335 | -.4368679            | 1.269709 |
| _cons       |      | -.0817754   | .2006423                          | -0.41 | 0.684 | -.4798932            | .3163424 |
| sigma_u     |      | 6.7557992   |                                   |       |       |                      |          |
| sigma_e     |      | 2.3288408   |                                   |       |       |                      |          |
| rho         |      | .89379082   | (fraction of variance due to u_i) |       |       |                      |          |

```
Linear regression      Number of obs   =      400
                        F(5, 99)      =      21.97
                        Prob > F       =      0.0000
                        R-squared      =      0.0384
```

# 8.5. Application to Difference-in-Differences (Common Timing)

Root MSE = 7.0672

(Std. err. adjusted for 100 clusters in id)

|             | logy | Coefficient | Robust<br>std. err. | t     | P> t  | [95% conf. interval] |          |
|-------------|------|-------------|---------------------|-------|-------|----------------------|----------|
| c.w#c.f2014 |      | 2.813593    | .5368268            | 5.24  | 0.000 | 1.748412             | 3.878774 |
| c.w#c.f2015 |      | 3.600121    | .5765523            | 6.24  | 0.000 | 2.456116             | 4.744126 |
| d           |      | -.0636461   | 1.36017             | -0.05 | 0.963 | -2.762519            | 2.635227 |
| f2014       |      | -.5045043   | .3923088            | -1.29 | 0.201 | -1.28293             | .2739216 |
| f2015       |      | .2534181    | .4206018            | 0.60  | 0.548 | -.5811472            | 1.087983 |
| _cons       |      | .11305      | .9296453            | 0.12  | 0.903 | -1.731568            | 1.957668 |

( 1) .5\*c.w#c.f2014 + .5\*c.w#c.f2015 = 0

|     | logy | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |          |
|-----|------|-------------|-----------|------|-------|----------------------|----------|
| (1) |      | 3.206857    | .4325887  | 7.41 | 0.000 | 2.348507             | 4.065207 |

Linear regression

Number of obs = 400  
F(7, 99) = 182.67  
Prob > F = 0.0000  
R-squared = 0.8438  
Root MSE = 2.8557

(Std. err. adjusted for 100 clusters in id)

|             | logy | Coefficient | Robust<br>std. err. | t     | P> t  | [95% conf. interval] |          |
|-------------|------|-------------|---------------------|-------|-------|----------------------|----------|
| c.w#c.f2014 |      | 2.503103    | .3066694            | 8.16  | 0.000 | 1.894605             | 3.111602 |
| c.w#c.f2015 |      | 3.327274    | .2902385            | 11.46 | 0.000 | 2.751378             | 3.90317  |
| d           |      | .881691     | .5285779            | 1.67  | 0.098 | -.1671223            | 1.930504 |
| f2014       |      | .0840872    | .2215839            | 0.38  | 0.705 | -.3555834            | .5237578 |
| f2015       |      | .2391039    | .2082552            | 1.15  | 0.254 | -.1741196            | .6523275 |
| x1          |      | 2.506694    | .1327049            | 18.89 | 0.000 | 2.243379             | 2.77001  |
| c.d#c.x1    |      | .0743163    | .1915921            | 0.39  | 0.699 | -.305844             | .4544765 |

## 8. The Mundlak Device in Panel Data and Difference-in-Differences

|       |  |          |          |      |       |           |          |
|-------|--|----------|----------|------|-------|-----------|----------|
|       |  |          |          |      |       |           |          |
| _cons |  | .2148708 | .3676352 | 0.58 | 0.560 | -.5145972 | .9443389 |
| ----- |  |          |          |      |       |           |          |

The POLS Mundlak model reproduces the exact TWFE estimates for each post-treatment year (2.5 and 3.5), yields the average treatment effect of 3.0 via `lincom`, and allows time-invariant controls and group interactions without disturbing the treatment effect estimates.

### 8.6. Summary

1. **One-Way Mundlak:** Adding unit averages  $\bar{X}_i$  to Pooled OLS or Random Effects recovers the within (FE) estimator and enables estimation of time-invariant variables.
2. **Two-Way Mundlak:** Exact equivalence with TWFE holds unconditionally in balanced panels. In unbalanced panels, one must also include unit means of time dummies.
3. **Multi-Way FE:** When more than two fixed effects are present, eliminate the high-dimensional factor with means and retain explicit dummy indicators for the lower-dimensional factors.
4. **Difference-in-Differences:** Mundlak formulation allows flexible event-study interactions and time-invariant moderators within a transparent OLS/GLS framework.



## 9. Correlated Random Effect

### 9.1. Panel data

The standard panel-data setup is:

$$y_{it} = \alpha + X_{it}\beta + v_i + \epsilon_{it} \quad (9.1)$$

### 9.2. Fixed effect

The fixed-effect estimator runs OLS on demeaned data:

$$(y_{it} - \bar{y}_i) = (X_{it} - \bar{X}_i)\beta + (\epsilon_{it} - \bar{\epsilon}_i) \quad (9.2)$$

Both  $\alpha$  and  $v_i$  drop out after demeaning by unit.

### 9.3. Random effect

Random effects treat  $v_i + \epsilon_{it}$  as a composite error with two variance components:  $\hat{\sigma}_e^2$  (idiosyncratic) and  $\hat{\sigma}_u^2$  (individual). The GLS transformation is

$$z_{it}^* = z_{it} - \hat{\theta}_i \bar{z}_i$$

with  $\hat{\theta}_i = 1 - \sqrt{\hat{\sigma}_e^2 / (T_i \hat{\sigma}_u^2 + \hat{\sigma}_e^2)}$ , where  $T_i$  is the number of observations for unit  $i$ . Given estimated variance components, we run OLS on the transformed variables and iterate.

### 9.4. Correlated random effect

The correlated random-effect (CRE) model runs a random-effects regression of  $y_{it}$  on  $X_{it}$ ,  $\bar{X}_i$ , time-invariant covariates  $w_i$ , and a constant.

RE requires  $v_i$  and  $X_{it}$  to be uncorrelated — an assumption most economists reject. FE is consistent regardless, but it cannot estimate time-invariant covariates (race, gender, etc.). CRE has both advantages: it includes time-invariant variables and remains consistent. Mundlak and Wooldridge show that the CRE coefficients on  $X_{it}$  are identical to the FE estimates. Testing whether the coefficients on  $\bar{X}_i$  are jointly zero gives the Mundlak test for choosing between RE and CRE.

## 9.5. Example

Stata 18 added a `cre` option to `xtreg`. The following uses `nlswork`, 28,101 person-years from the National Longitudinal Survey of Young Women with complete data on the model's variables. We regress log wage on `tenure` and `age`, which vary within a person, plus `race`, which does not — the covariate a fixed-effect model has to drop.

```
webuse nlswork
```

```
xtreg ln_wage tenure age i.race, cre vce(cluster idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

note: 2.race omitted from xt\_means because of collinearity.

note: 3.race omitted from xt\_means because of collinearity.

```
Correlated random-effects regression      Number of obs      =      28,101
Group variable: idcode                   Number of groups   =      4,699
```

```
R-squared:                               Obs per group:
    Within = 0.1296                        min =          1
    Between = 0.2346                      avg =          6.0
    Overall = 0.1890                      max =          15
```

```
corr(xit_vars*b, xt_means*) = 0.5474      Wald chi2(4)       =      1685.18
                                           Prob > chi2        =      0.0000
```

(Std. err. adjusted for 4,699 clusters in idcode)

| -----    |         |             |           |        |       |                      |
|----------|---------|-------------|-----------|--------|-------|----------------------|
|          | ln_wage | Robust      |           |        |       |                      |
|          |         | Coefficient | std. err. | z      | P> z  | [95% conf. interval] |
| -----    |         |             |           |        |       |                      |
| xit_vars |         |             |           |        |       |                      |
| tenure   |         | .0211313    | .0012113  | 17.44  | 0.000 | .0187572 .0235055    |
| age      |         | .0121949    | .0007414  | 16.45  | 0.000 | .0107417 .013648     |
| race     |         |             |           |        |       |                      |
| Black    |         | -.1312068   | .0117856  | -11.13 | 0.000 | -.1543061 -.1081075  |
| Other    |         | .1059379    | .0593177  | 1.79   | 0.074 | -.0103225 .2221984   |
| _cons    |         | 1.2159      | .0306965  | 39.61  | 0.000 | 1.155736 1.276064    |
| -----    |         |             |           |        |       |                      |
| xt_means |         |             |           |        |       |                      |
| tenure   |         | .0376991    | .002281   | 16.53  | 0.000 | .0332283 .0421698    |
| age      |         | -.0011984   | .0013313  | -0.90  | 0.368 | -.0038077 .0014109   |
| race     |         |             |           |        |       |                      |

|                                                 |  |           |                                   |  |
|-------------------------------------------------|--|-----------|-----------------------------------|--|
| Black                                           |  | 0         | (omitted)                         |  |
| Other                                           |  | 0         | (omitted)                         |  |
| -----+                                          |  |           |                                   |  |
| sigma_u                                         |  | .33334407 |                                   |  |
| sigma_e                                         |  | .29808194 |                                   |  |
| rho                                             |  | .55567161 | (fraction of variance due to u_i) |  |
| -----                                           |  |           |                                   |  |
| Mundlak test (xt_means = 0): chi2(2) = 331.5144 |  |           | Prob > chi2 = 0.0000              |  |

CRE gives `tenure` = .0211313 and `age` = .0121949, and estimates race as well: Black women earn 13.1% less (−.1312068, SE .0118), while the “Other” category is marginal (.1059,  $p = 0.074$ ). The coefficients on the group means are .0376991 for mean tenure and −.0011984 for mean age.

Stata prints the Mundlak test at the foot of the table:  $\chi^2(2) = 331.51$ ,  $p < 0.0001$ . The group means are jointly far from zero, so plain random effects is rejected —  $v_i$  and the regressors are correlated, as the reported  $\text{corr} = 0.5474$  between the two blocks says directly.

To compare with a fixed effect model:

```
webuse nlswork

xtreg ln_wage tenure age i.race, fe vce(cluster idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

```
note: 2.race omitted because of collinearity.
note: 3.race omitted because of collinearity.
```

|                                   |                  |   |        |
|-----------------------------------|------------------|---|--------|
| Fixed-effects (within) regression | Number of obs    | = | 28,101 |
| Group variable: idcode            | Number of groups | = | 4,699  |

|                  |                |
|------------------|----------------|
| R-squared:       | Obs per group: |
| Within = 0.1296  | min = 1        |
| Between = 0.1916 | avg = 6.0      |
| Overall = 0.1456 | max = 15       |

|                        |            |   |        |
|------------------------|------------|---|--------|
|                        | F(2, 4698) | = | 766.79 |
| corr(u i, Xb) = 0.1302 | Prob > F   | = | 0.0000 |

(Std. err. adjusted for 4,699 clusters in idcode)

|         |             | Robust    |       |       |                      |          |  |
|---------|-------------|-----------|-------|-------|----------------------|----------|--|
| ln_wage | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |          |  |
| tenure  | .0211313    | .0012112  | 17.45 | 0.000 | .0187568             | .0235059 |  |
| age     | .0121949    | .0007414  | 16.45 | 0.000 | .0107414             | .0136483 |  |
| race    |             |           |       |       |                      |          |  |

## 9. Correlated Random Effect

|         |  |           |                                   |       |       |          |          |
|---------|--|-----------|-----------------------------------|-------|-------|----------|----------|
| Black   |  | 0         | (omitted)                         |       |       |          |          |
| Other   |  | 0         | (omitted)                         |       |       |          |          |
|         |  |           |                                   |       |       |          |          |
| _cons   |  | 1.256467  | .0194187                          | 64.70 | 0.000 | 1.218397 | 1.294537 |
| -----   |  |           |                                   |       |       |          |          |
| sigma_u |  | .39034493 |                                   |       |       |          |          |
| sigma_e |  | .29808194 |                                   |       |       |          |          |
| rho     |  | .63165531 | (fraction of variance due to u_i) |       |       |          |          |

The coefficients on `tenure` and `age` are identical: .0211313 and .0121949 in both, with standard errors agreeing to the seventh decimal (.0012112 against .0012113). Race is reported as 0 (omitted) here, being collinear with the unit effects — the one thing FE cannot do and CRE can. This is the Mundlak result in practice: CRE gives you the fixed-effect estimates *and* the time-invariant coefficients from a single random-effects fit.

We can replicate this manually by including the group means in a RE model:

```
webuse nlswork
* Restrict the mean calculation to the estimation sample (listwise-deleted
* on the model's variables), matching what xtreg's native cre option does
* internally. Both tenure and age have missing values in nlswork, so
* computing age_mean/tenure_mean over all rows for an idcode (rather than
* only rows with complete data on ln_wage, tenure, age, and race) would
* make the manual means diverge from cre's.
egen age_mean = mean(age) if !missing(ln_wage, tenure, age, race), by(idcode)
egen tenure_mean = mean(tenure) if !missing(ln_wage, tenure, age, race), by(idcode)
xtreg ln_wage tenure tenure_mean age age_mean i.race, vce(cluster idcode)
```

(National Longitudinal Survey of Young Women, 14-24 years old in 1968)

(433 missing values generated)

(433 missing values generated)

|                               |                  |   |         |
|-------------------------------|------------------|---|---------|
| Random-effects GLS regression | Number of obs    | = | 28,101  |
| Group variable: idcode        | Number of groups | = | 4,699   |
| R-squared:                    | Obs per group:   |   |         |
| Within = 0.1296               | min =            |   | 1       |
| Between = 0.2346              | avg =            |   | 6.0     |
| Overall = 0.1890              | max =            |   | 15      |
|                               | Wald chi2(6)     | = | 2688.54 |
| corr(u_i, X) = 0 (assumed)    | Prob > chi2      | = | 0.0000  |

| (Std. err. adjusted for 4,699 clusters in idcode) |             |                                   |        |       |                      |           |
|---------------------------------------------------|-------------|-----------------------------------|--------|-------|----------------------|-----------|
| ln_wage                                           | Coefficient | Robust<br>std. err.               | z      | P> z  | [95% conf. interval] |           |
| tenure                                            | .0211313    | .0012113                          | 17.44  | 0.000 | .0187572             | .0235055  |
| tenure_mean                                       | .0376991    | .002281                           | 16.53  | 0.000 | .0332283             | .0421698  |
| age                                               | .0121949    | .0007414                          | 16.45  | 0.000 | .0107417             | .013648   |
| age_mean                                          | -.0011984   | .0013313                          | -0.90  | 0.368 | -.0038077            | .0014109  |
| race                                              |             |                                   |        |       |                      |           |
| Black                                             | -.1312068   | .0117856                          | -11.13 | 0.000 | -.1543061            | -.1081075 |
| Other                                             | .1059379    | .0593177                          | 1.79   | 0.074 | -.0103225            | .2221984  |
| _cons                                             | 1.2159      | .0306965                          | 39.61  | 0.000 | 1.155736             | 1.276064  |
| sigma_u                                           | .33334407   |                                   |        |       |                      |           |
| sigma_e                                           | .29808194   |                                   |        |       |                      |           |
| rho                                               | .55567161   | (fraction of variance due to u_i) |        |       |                      |           |

This is what Stata's `cre` option does internally. Every number matches the first table: `tenure` .0211313, `age` .0121949, `Black` -.1312068, `Other` .1059379, constant 1.2159, and the same variance components ( $\sigma_u$  .3333,  $\sigma_e$  .2981,  $\rho$  .5557). The `cre` option is a convenience, not a different estimator.

The one detail worth care is in the code above: the group means must be computed on the estimation sample. Both `tenure` and `age` have missing values in `nlswork`, so averaging over all of a woman's rows rather than only her complete ones would give different means and break the equivalence.



# 10. Correlated Random Effects (CRE) in Linear and Nonlinear Models

## 10.1. Background: Unobserved Heterogeneity and CRE

In panel data models with unobserved individual heterogeneity  $v_i$ :

$$y_{it} = \alpha + X_{it}\beta + v_i + \epsilon_{it}$$

Standard Random Effects (RE) assumes that  $v_i$  is independent of the covariates ( $E[v_i | X_i] = 0$ ). If  $v_i$  correlates with  $X_{it}$ , RE estimates are biased and inconsistent. Conversely, Fixed Effects (FE) conditions out  $v_i$  via demeaning or first-differencing, but cannot estimate coefficients on time-invariant covariates and does not easily extend to nonlinear models.

The **Correlated Random Effects (CRE)** framework (Mundlak, 1978; Chamberlain, 1982; Wooldridge, 2010) parameterizes the dependence between  $v_i$  and the history of  $X_i$ :

$$v_i = \psi + \bar{X}_i\xi + a_i, \quad E[a_i | X_i] = 0$$

where  $\bar{X}_i = \frac{1}{T} \sum_{t=1}^T X_{it}$  is the time-average of time-varying covariates for unit  $i$ . Substituting this projection into the model yields:

$$y_{it} = (\alpha + \psi) + X_{it}\beta + \bar{X}_i\xi + a_i + \epsilon_{it}$$

Adding unit means  $\bar{X}_i$  directly controls for the correlation between  $X_{it}$  and the unobserved effect  $v_i$ .

## 10.2. Linear Models: Equivalence to Fixed Effects

In a balanced linear panel, estimating the model with group means  $\bar{X}_i$  via Pooled OLS (POLS) or Random Effects (GLS) produces point estimates on  $X_{it}$  that are **algebraically identical** to the Within (FE) estimator.

## 10.2.1. Empirical Demonstration: castle Data

Using the balanced `castle` panel (50 states over 11 years;  $N = 550$ ): - Outcome: log homicide rate (`l_homicide`) - Covariates: poverty rate (`poverty`) and log police employment (`l_police`)

```
data("castle")

# 1. Two-way Fixed Effects
fe_model <- feols(l_homicide ~ poverty + l_police | state + year, data = castle)

# 2. Standard Random Effects (assumes Cov(X, v) = 0)
re_model <- lmer(l_homicide ~ poverty + l_police + factor(year) + (1 | state), data = castle)

# 3. CRE Pooled OLS (Mundlak device)
castle2 <- castle |>
  group_by(state) |>
  mutate(poverty_mean = mean(poverty, na.rm = TRUE),
         l_police_mean = mean(l_police, na.rm = TRUE))

cre_pols <- feols(l_homicide ~ poverty + poverty_mean + l_police + l_police_mean + factor(year),
                 data = castle2, cluster = ~state)

# 4. CRE Random Effects
cre_re <- lmer(l_homicide ~ poverty + poverty_mean + l_police + l_police_mean + factor(year) +
              data = castle2)

# Compare estimates on poverty and l_police
res_table <- data.frame(
  Estimator = c("Two-way Fixed Effects", "Standard Random Effects", "CRE Pooled OLS", "CRE Random Effects"),
  beta_poverty = c(coef(fe_model)["poverty"], fixef(re_model)["poverty"],
                  coef(cre_pols)["poverty"], fixef(cre_re)["poverty"]),
  beta_police = c(coef(fe_model)["l_police"], fixef(re_model)["l_police"],
                 coef(cre_pols)["l_police"], fixef(cre_re)["l_police"])
)
res_table[, 2:3] <- round(res_table[, 2:3], 6)
knitr::kable(res_table, caption = "Comparison of linear panel estimators.")
```

Table 10.1.: Comparison of linear panel estimators.

| Estimator               | beta_poverty | beta_police |
|-------------------------|--------------|-------------|
| Two-way Fixed Effects   | -0.027068    | 0.066033    |
| Standard Random Effects | -0.000799    | 0.166591    |
| CRE Pooled OLS          | -0.027068    | 0.066033    |
| CRE Random Effects      | -0.027068    | 0.066033    |



### 10.2.2. Key Takeaways in the Linear Case

1. **Numerical Equivalence:** The Two-way FE, CRE Pooled OLS, and CRE RE models yield identical coefficients:  $\beta_{\text{poverty}} = -0.027068$  and  $\beta_{\text{police}} = 0.066033$ .
2. **Failure of Standard RE:** Without the group means, standard RE yields  $\beta_{\text{poverty}} = -0.000799$ , severely attenuated toward zero due to correlation with unobserved state effects.
3. **Inference:** Because POLS ignores within-unit error correlation, cluster-robust standard errors clustered at the unit level (`cluster = ~state`) are required to obtain valid standard errors matching FE.

## 10.3. Nonlinear Panel Models: Binary Outcomes

For a binary outcome  $y_{it} \in \{0, 1\}$ , the latent model is:

$$P(y_{it} = 1 \mid X_{it}, v_i) = \Phi(X_{it}\beta + v_i)$$

In nonlinear models, the algebraic equivalence between FE and CRE breaks down: 1. **Limitations of Conditional (Fixed Effects) Logit:** - Requires strict conditional independence of  $y_{it}$  across time given  $X_i$  and  $v_i$ . Any serial correlation in the idiosyncratic errors causes inconsistency. - Because  $v_i$  is conditioned out rather than estimated, partial effects (Average Marginal Effects) cannot be computed. 2. **Advantages of CRE Pooled Probit:** - Incorporates the Mundlak device by including  $\bar{X}_i$  in a pooled probit. - Robust to arbitrary serial correlation when paired with cluster-robust standard errors (Lin and Wooldridge, 2019). - Allows straightforward computation of Average Marginal Effects (AMEs).

### 10.3.1. Empirical Demonstration: Binary Outcome

We define a binary outcome `high_homicide` indicating homicide rates above the sample mean:

```
castle2 <- castle2 |>
  mutate(high_homicide = ifelse(l_homicide > mean(l_homicide, na.rm = TRUE), 1, 0))

# 1. CRE Pooled Probit
cre_probit <- glm(high_homicide ~ poverty + poverty_mean + l_police + l_police_mean + factor(year),
  data = castle2, family = binomial(link = "probit"))

# 2. Linear Probability Model with CRE
cre_lpm <- lm(high_homicide ~ poverty + poverty_mean + l_police + l_police_mean + factor(year),
  data = castle2)

# 3. CRE Random Effects Probit
cre_re_probit <- glmer(high_homicide ~ poverty + poverty_mean + l_police + l_police_mean + factor(year),
  data = castle2, family = binomial(link = "probit"))

# Compute Average Marginal Effects (AMEs)
```

```

ame_probit <- avg_slopes(cre_probit, variables = c("poverty", "l_police"))
ame_lpm    <- avg_slopes(cre_lpm, variables = c("poverty", "l_police"))
ame_re     <- avg_slopes(cre_re_probit, variables = c("poverty", "l_police"))

ame_comp <- data.frame(
  Model = c("CRE Pooled Probit", "CRE Linear Probability Model", "CRE Random Effects Probit"),
  AME_poverty = c(ame_probit$estimate[ame_probit$term == "poverty"],
    ame_lpm$estimate[ame_lpm$term == "poverty"],
    ame_re$estimate[ame_re$term == "poverty"]),
  SE_poverty = c(ame_probit$std.error[ame_probit$term == "poverty"],
    ame_lpm$std.error[ame_lpm$term == "poverty"],
    ame_re$std.error[ame_re$term == "poverty"])
)
ame_comp[, 2:3] <- round(ame_comp[, 2:3], 5)
knitr::kable(ame_comp, caption = "Average Marginal Effects across binary CRE models.")

```

Table 10.2.: Average Marginal Effects across binary CRE models.

| Model                        | AME_poverty | SE_poverty |
|------------------------------|-------------|------------|
| CRE Pooled Probit            | -0.06477    | 0.03336    |
| CRE Linear Probability Model | -0.06406    | 0.03391    |
| CRE Random Effects Probit    | -0.06477    | 0.03336    |

Across all three specifications, the estimated average marginal effect of poverty is approximately  $-0.064$  with standard error  $\approx 0.034$ . The CRE pooled probit provides a robust, computationally simple estimator that allows partial effect estimation without relying on strict serial independence.

## 10.4. Summary

1. **Linear Panel Data:** Adding unit averages  $\bar{X}_i$  allows Pooled OLS and Random Effects to recover the exact Fixed Effects estimates while retaining cluster-robust inference.
2. **Nonlinear Models:** In binary and count models, the CRE approach avoids the incidental parameters problem and allows calculation of Average Marginal Effects.
3. **Robustness:** Pooled estimation with cluster-robust standard errors is preferred over nonlinear random effects models due to robustness against arbitrary serial correlation.

## **Part III.**

# **Treatment Effects & Matching**



# 11. Treatment effects and matching

## 11.1. Treatment effects in observational studies

Most economic data are observational. With unobserved confounders we need instrumental variables, but instruments are hard to find and harder to justify. Under conditional independence — no unobserved confounders given the covariates — we can proceed without them.

Stata's `teffects` suite implements the standard estimators under conditional independence. (The companion `eteffects` handles endogenous treatment via a control-function approach with instruments.) This chapter walks through how `teffects` works.

### 11.1.1. Regression adjustment (RA)

RA allows all covariate effects to differ between treatment and control — equivalent to an outcome model with treatment interacted with all covariates.

```
clear
webuse bweightex
teffects ra (bweight prenatal1 mmarried mage fbaby) (mbsmoke)
reg bweight i.mbsmoke##c.(prenatal1 mmarried mage fbaby)
margins r.mbsmoke
```

```
. clear

. webuse bweightex
(Hypothetical birthweight data)

. teffects ra (bweight prenatal1 mmarried mage fbaby) (mbsmoke)
```

```
Iteration 0:  EE criterion = 1.223e-24
Iteration 1:  EE criterion = 1.792e-25
```

```
Treatment-effects estimation      Number of obs      =      60
Estimator      : regression adjustment
Outcome model  : linear
Treatment model: none
```

```
-----
|                               Robust
```

# 11. Treatment effects and matching

|            | bweight | Coefficient | std. err. | z      | P> z  | [95% conf. interval] |           |
|------------|---------|-------------|-----------|--------|-------|----------------------|-----------|
| ATE        |         |             |           |        |       |                      |           |
| mbsmoke    |         |             |           |        |       |                      |           |
| (Smoker    |         |             |           |        |       |                      |           |
| vs         |         |             |           |        |       |                      |           |
| Nonsmoker) |         | -389.3099   | 37.00882  | -10.52 | 0.000 | -461.8458            | -316.7739 |
| POmean     |         |             |           |        |       |                      |           |
| mbsmoke    |         |             |           |        |       |                      |           |
| Nonsmoker  |         | 3613.769    | 29.97438  | 120.56 | 0.000 | 3555.021             | 3672.518  |

```
. reg bweight i.mbsmoke##c.(prenatal1 mmarried mage fbaby)
```

| Source   | SS         | df | MS         | Number of obs | = | 60     |
|----------|------------|----|------------|---------------|---|--------|
|          |            |    |            | F(9, 50)      | = | 18.11  |
| Model    | 1376306.34 | 9  | 152922.927 | Prob > F      | = | 0.0000 |
| Residual | 422241.592 | 50 | 8444.83184 | R-squared     | = | 0.7652 |
|          |            |    |            | Adj R-squared | = | 0.7230 |
| Total    | 1798547.93 | 59 | 30483.8633 | Root MSE      | = | 91.896 |

|             | bweight | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------------|---------|-------------|-----------|-------|-------|----------------------|----------|
| mbsmoke     |         |             |           |       |       |                      |          |
| Smoker      |         | -32.87702   | 193.2827  | -0.17 | 0.866 | -421.0967            | 355.3426 |
| prenatal1   |         | -23.8431    | 38.31014  | -0.62 | 0.537 | -100.7913            | 53.10507 |
| mmarried    |         | -40.39753   | 50.21942  | -0.80 | 0.425 | -141.2662            | 60.47114 |
| mage        |         | 27.9537     | 7.671423  | 3.64  | 0.001 | 12.54519             | 43.36221 |
| fbaby       |         | -2.030497   | 40.76029  | -0.05 | 0.960 | -83.89995            | 79.83896 |
| mbsmoke#    |         |             |           |       |       |                      |          |
| c.prenatal1 |         |             |           |       |       |                      |          |
| Smoker      |         | 8.436558    | 56.38855  | 0.15  | 0.882 | -104.8232            | 121.6963 |
| mbsmoke#    |         |             |           |       |       |                      |          |
| c.mmarried  |         |             |           |       |       |                      |          |
| Smoker      |         | 33.92362    | 61.34015  | 0.55  | 0.583 | -89.2817             | 157.1289 |
| mbsmoke#    |         |             |           |       |       |                      |          |
| c.mage      |         |             |           |       |       |                      |          |
| Smoker      |         | -15.98608   | 9.006494  | -1.77 | 0.082 | -34.07616            | 2.103995 |
| mbsmoke#    |         |             |           |       |       |                      |          |
| c.fbaby     |         |             |           |       |       |                      |          |
| Smoker      |         | 14.86782    | 57.58771  | 0.26  | 0.797 | -100.8005            | 130.5362 |

```

      _cons |    2976.807    151.0117    19.71    0.000    2673.491    3280.123
-----+-----

```

```
. margins r.mbsmoke
```

Contrasts of predictive margins  
Model VCE: OLS

Number of obs = 60

Expression: Linear prediction, predict()

```

-----+-----
              |          df          F          P>F
-----+-----
      mbsmoke |             1      116.06      0.0000
              |
Denominator |             50
-----+-----

```

```

-----+-----
              |          Delta-method
              |          Contrast    std. err.    [95% conf. interval]
-----+-----
      mbsmoke |
(Smoker vs Nonsmoker) | -389.3099    36.13675    -461.8927    -316.7271
-----+-----

```

The data here are `bweightex`, a 60-observation extract with birthweight as the outcome and maternal smoking as the treatment.

`teffects ra` returns the same ATE as `margins r.mbsmoke` after the interacted regression: both give  $-389.3099$  grams. The standard errors differ slightly, 37.01 against 36.14, because `teffects` accounts for the estimation of the outcome model differently from `margins`' delta method. The point estimate is the same object computed two ways.

For the ATET (average treatment effect on the treated):

```

clear
webuse bweightex
teffects ra (bweight prenatal1 mmarried mage fbaby) (mbsmoke), atet
reg bweight i.mbsmoke##c.(prenatal1 mmarried mage fbaby)
margins r.mbsmoke, subpop(mbsmoke)

```

```
. clear
```

```
. webuse bweightex
(Hypothetical birthweight data)
```

## 11. Treatment effects and matching

```
. teffects ra (bweight prenatal1 mmarried mage fbaby) (mb smoke), atet
```

```
Iteration 0: EE criterion = 1.159e-24
```

```
Iteration 1: EE criterion = 5.133e-26
```

```
Treatment-effects estimation      Number of obs      =          60
Estimator      : regression adjustment
Outcome model   : linear
Treatment model: none
```

|            |  | Robust      |           | z     | P> z  | [95% conf. interval] |           |
|------------|--|-------------|-----------|-------|-------|----------------------|-----------|
| bweight    |  | Coefficient | std. err. |       |       |                      |           |
| ATET       |  |             |           |       |       |                      |           |
| mb smoke   |  |             |           |       |       |                      |           |
| (Smoker    |  |             |           |       |       |                      |           |
| vs         |  |             |           |       |       |                      |           |
| Nonsmoker) |  | -437.4721   | 53.43513  | -8.19 | 0.000 | -542.203             | -332.7412 |
| POmean     |  |             |           |       |       |                      |           |
| mb smoke   |  |             |           |       |       |                      |           |
| Nonsmoker  |  | 3693.639    | 53.80537  | 68.65 | 0.000 | 3588.182             | 3799.095  |

```
. reg bweight i.mb smoke#c.(prenatal1 mmarried mage fbaby)
```

| Source   | SS         | df | MS         | Number of obs | = | 60     |
|----------|------------|----|------------|---------------|---|--------|
|          |            |    |            | F(9, 50)      | = | 18.11  |
| Model    | 1376306.34 | 9  | 152922.927 | Prob > F      | = | 0.0000 |
| Residual | 422241.592 | 50 | 8444.83184 | R-squared     | = | 0.7652 |
|          |            |    |            | Adj R-squared | = | 0.7230 |
| Total    | 1798547.93 | 59 | 30483.8633 | Root MSE      | = | 91.896 |

| bweight     | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-------------|-------------|-----------|-------|-------|----------------------|----------|
| mb smoke    |             |           |       |       |                      |          |
| Smoker      | -32.87702   | 193.2827  | -0.17 | 0.866 | -421.0967            | 355.3426 |
| prenatal1   | -23.8431    | 38.31014  | -0.62 | 0.537 | -100.7913            | 53.10507 |
| mmarried    | -40.39753   | 50.21942  | -0.80 | 0.425 | -141.2662            | 60.47114 |
| mage        | 27.9537     | 7.671423  | 3.64  | 0.001 | 12.54519             | 43.36221 |
| fbaby       | -2.030497   | 40.76029  | -0.05 | 0.960 | -83.89995            | 79.83896 |
| mb smoke#c. |             |           |       |       |                      |          |
| prenatal1   |             |           |       |       |                      |          |
| Smoker      | 8.436558    | 56.38855  | 0.15  | 0.882 | -104.8232            | 121.6963 |
| mb smoke#c. |             |           |       |       |                      |          |



|            |  |           |          |       |       |           |          |
|------------|--|-----------|----------|-------|-------|-----------|----------|
| c.mmarried |  |           |          |       |       |           |          |
| Smoker     |  | 33.92362  | 61.34015 | 0.55  | 0.583 | -89.2817  | 157.1289 |
|            |  |           |          |       |       |           |          |
| mbsmoke#   |  |           |          |       |       |           |          |
| c.mage     |  |           |          |       |       |           |          |
| Smoker     |  | -15.98608 | 9.006494 | -1.77 | 0.082 | -34.07616 | 2.103995 |
|            |  |           |          |       |       |           |          |
| mbsmoke#   |  |           |          |       |       |           |          |
| c.fbaby    |  |           |          |       |       |           |          |
| Smoker     |  | 14.86782  | 57.58771 | 0.26  | 0.797 | -100.8005 | 130.5362 |
|            |  |           |          |       |       |           |          |
| _cons      |  | 2976.807  | 151.0117 | 19.71 | 0.000 | 2673.491  | 3280.123 |

```
. margins r.mbsmoke, subpop(mbsmoke)
```

Contrasts of predictive margins  
Model VCE: OLS

Number of obs = 60  
Subpop. no. obs = 30

Expression: Linear prediction, predict()

|             | df | F     | P>F    |
|-------------|----|-------|--------|
| mbsmoke     | 1  | 73.30 | 0.0000 |
| Denominator | 50 |       |        |

|                       | Contrast  | Delta-method<br>std. err. | [95% conf. interval] |
|-----------------------|-----------|---------------------------|----------------------|
| mbsmoke               |           |                           |                      |
| (Smoker vs Nonsmoker) | -437.4721 | 51.09606                  | -540.1015 -334.8426  |

The ATET compares potential outcomes for the treated subpopulation, hence the `subpop(mbsmoke)` option in `margins`. Both routes give  $-437.4721$ , again identical, and again with slightly different standard errors (53.44 against 51.10). Note the ATET is 48 grams larger in magnitude than the ATE of  $-389.31$ : smoking mothers are drawn disproportionately from the groups where smoking does most damage.

### 11.1.2. Inverse Probability Weighting (IPW)

IPW has two steps. First, estimate the treatment model (typically logit) to obtain the propensity score. Second, use inverse-probability weights to compute the outcome difference between treated

## 11. Treatment effects and matching

and control units.

These steps produce consistent estimates of the effect parameters because the treatment is assumed to be independent of the potential outcomes after conditioning on the covariates.

We can manually conduct the two steps, but the nice thing about using Stata's `teffects` is that it takes account of the noise of estimating probability in the first step when calculating standard errors in the second step.

### 11.1.2.1. example

```
clear
webuse cattaneo2
teffects ipw (bweight) (mb smoke mmarried c.mage##c.mage fbaby medu, probit)
probit mb smoke mmarried c.mage##c.mage fbaby medu
predict ps
replace ps = 1/ps if mb smoke==1
replace ps = 1/(1-ps) if mb smoke==0
reg bweight mb smoke [pweight=ps]
```

```
. clear

. webuse cattaneo2
(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)

. teffects ipw (bweight) (mb smoke mmarried c.mage##c.mage fbaby medu, probit)

Iteration 0:  EE criterion = 4.622e-21
Iteration 1:  EE criterion = 8.453e-26

Treatment-effects estimation                Number of obs      =      4,642
Estimator      : inverse-probability weights
Outcome model   : weighted mean
Treatment model: probit

-----+-----
               |               Robust
           bweight | Coefficient  std. err.      z    P>|z|      [95% conf. interval]
-----+-----
ATE
   mb smoke      |
   (Smoker       |
   vs            |
   Nonsmoker)    |  -230.6886   25.81524    -8.94   0.000   -281.2856   -180.0917
-----+-----
POmean           |
```

|           |  |          |          |        |       |                   |
|-----------|--|----------|----------|--------|-------|-------------------|
| mbsmoke   |  |          |          |        |       |                   |
| Nonsmoker |  | 3403.463 | 9.571369 | 355.59 | 0.000 | 3384.703 3422.222 |

---

```
. probit mbsmoke mmarried c.mage##c.mage fbaby medu
```

```
Iteration 0: Log likelihood = -2230.7484
Iteration 1: Log likelihood = -2042.6734
Iteration 2: Log likelihood = -2040.5088
Iteration 3: Log likelihood = -2040.5061
Iteration 4: Log likelihood = -2040.5061
```

|                             |                 |        |
|-----------------------------|-----------------|--------|
| Probit regression           | Number of obs = | 4,642  |
|                             | LR chi2(5) =    | 380.48 |
|                             | Prob > chi2 =   | 0.0000 |
| Log likelihood = -2040.5061 | Pseudo R2 =     | 0.0853 |

---

|         |  |             |           |   |      |                      |
|---------|--|-------------|-----------|---|------|----------------------|
| mbsmoke |  | Coefficient | Std. err. | z | P> z | [95% conf. interval] |
|---------|--|-------------|-----------|---|------|----------------------|

---

|          |  |           |          |        |       |                     |
|----------|--|-----------|----------|--------|-------|---------------------|
| mmarried |  | -.6484821 | .0526991 | -12.31 | 0.000 | -.7517705 -.5451938 |
| mage     |  | .1744327  | .0352437 | 4.95   | 0.000 | .1053562 .2435092   |
| c.mage#  |  |           |          |        |       |                     |
| c.mage   |  | -.0032559 | .0006462 | -5.04  | 0.000 | -.0045224 -.0019894 |
| fbaby    |  | -.2175962 | .0491066 | -4.43  | 0.000 | -.3138433 -.121349  |
| medu     |  | -.0863631 | .0098692 | -8.75  | 0.000 | -.1057064 -.0670198 |
| _cons    |  | -1.558255 | .4511589 | -3.45  | 0.001 | -2.44251 -.674      |

---

```
. predict ps
(option pr assumed; Pr(mbsmoke))
```

```
. replace ps = 1/ps if mbsmoke==1
(864 real changes made)
```

```
. replace ps = 1/(1-ps) if mbsmoke==0
(3,778 real changes made)
```

```
. reg bweight mbsmoke [pweight=ps]
(sum of wgt is 9,193.96990537643)
```

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 4,642  |
|                   | F(1, 4640)    | = | 79.08  |
|                   | Prob > F      | = | 0.0000 |
|                   | R-squared     | = | 0.0389 |
|                   | Root MSE      | = | 573.36 |

## 11. Treatment effects and matching

|         | Robust      |           |        |       |                      |           |
|---------|-------------|-----------|--------|-------|----------------------|-----------|
| bweight | Coefficient | std. err. | t      | P> t  | [95% conf. interval] |           |
| mbsmoke | -230.6886   | 25.94182  | -8.89  | 0.000 | -281.5469            | -179.8303 |
| _cons   | 3403.463    | 9.616992  | 353.90 | 0.000 | 3384.609             | 3422.317  |

We switch here to `cattaneo2`, the full 4,642-birth dataset, and a richer propensity model (a probit in marital status, a quadratic in maternal age, first-baby status and education).

`teffects ipw` gives an ATE of  $-230.6886$  with a standard error of 25.82. The manual two-step replication matches the point estimate exactly,  $-230.6886$ ; the only difference is the standard error (the manual version does not account for first-stage estimation noise). Note this is well short of the  $-389$  found by RA above — but on different data and a different covariate set, so the two are not comparable.

### 11.1.3. Doubly robust estimators

RA models the outcome; IPW models the treatment assignment. Doubly-robust estimators model both, requiring only one to be correctly specified.

Stata implements the AIPW combination of Robins and Rotnitzky (1995) and the IPWRA combination of Wooldridge (2010).

The AIPW estimator augments the IPW estimator with a correction term built from the outcome model. Double robustness is best stated as a **consistency** property: AIPW is consistent if *either* the treatment model *or* the outcome model is correctly specified (not necessarily both). It is *not* that the correction term literally “goes to zero” when the treatment model is right and the outcome model is wrong — with a misspecified outcome model the correction term does not generally vanish. Rather, the two pieces are constructed so that the estimator’s bias cancels in the limit as long as one of the two models is right.

The IPWRA estimator uses IPW probability weights when performing RA. The weights do not affect the accuracy of the RA estimator if the treatment model is wrong and the outcome model is correct. The weights correct the RA estimator if the treatment model is correct and the outcome model is wrong.

Here we run the two commands, then manually replicate both as a two-step (“plug-in”) procedure. `teffects` estimates the treatment and outcome models jointly by GMM/ML, using the joint estimation to get standard errors that account for first-stage estimation noise; the manual two-step version below gets essentially the same point estimate but not the correct standard errors (for the same reason noted above for IPW).

```
clear
webuse cattaneo2
teffects ipwra (bweight mmarried mage prenatal1 fbaby) (mbsmoke mmarried mage prenatal1 fbaby)
teffects aipw (bweight mmarried mage prenatal1 fbaby) (mbsmoke mmarried mage prenatal1 fbaby)
```

```
. clear

. webuse cattaneo2
(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)

. teffects ipwra (bweight mmarried mage prenatal1 fbaby) (mbsmoke mmarried mag
> e prenatal1 fbaby)
```

```
Iteration 0: EE criterion = 9.066e-22
Iteration 1: EE criterion = 2.902e-26
```

```
Treatment-effects estimation          Number of obs      =      4,642
Estimator       : IPW regression adjustment
Outcome model   : linear
Treatment model : logit
```

|            |  | Robust      |           |        |       |                      |          |
|------------|--|-------------|-----------|--------|-------|----------------------|----------|
| bweight    |  | Coefficient | std. err. | z      | P> z  | [95% conf. interval] |          |
| <hr/>      |  |             |           |        |       |                      |          |
| ATE        |  |             |           |        |       |                      |          |
| mbsmoke    |  |             |           |        |       |                      |          |
| (Smoker    |  |             |           |        |       |                      |          |
| vs         |  |             |           |        |       |                      |          |
| Nonsmoker) |  | -238.7679   | 24.38353  | -9.79  | 0.000 | -286.5587            | -190.977 |
| <hr/>      |  |             |           |        |       |                      |          |
| POmean     |  |             |           |        |       |                      |          |
| mbsmoke    |  |             |           |        |       |                      |          |
| Nonsmoker  |  | 3402.851    | 9.538741  | 356.74 | 0.000 | 3384.155             | 3421.546 |
| <hr/>      |  |             |           |        |       |                      |          |

```
. teffects aipw (bweight mmarried mage prenatal1 fbaby) (mbsmoke mmarried mage
> prenatal1 fbaby)
```

```
Iteration 0: EE criterion = 2.115e-22
Iteration 1: EE criterion = 1.343e-26
```

```
Treatment-effects estimation          Number of obs      =      4,642
Estimator       : augmented IPW
Outcome model   : linear by ML
Treatment model : logit
```

|         |             | Robust    |   |      |                      |
|---------|-------------|-----------|---|------|----------------------|
| bweight | Coefficient | std. err. | z | P> z | [95% conf. interval] |
| -----   |             |           |   |      |                      |
| ATE     |             |           |   |      |                      |
| mbsmoke |             |           |   |      |                      |
| (Smoker |             |           |   |      |                      |

## 11. Treatment effects and matching

|            |  |           |          |        |       |           |           |
|------------|--|-----------|----------|--------|-------|-----------|-----------|
| vs         |  |           |          |        |       |           |           |
| Nonsmoker) |  | -239.0294 | 24.2524  | -9.86  | 0.000 | -286.5632 | -191.4955 |
| -----      |  |           |          |        |       |           |           |
| P0mean     |  |           |          |        |       |           |           |
| mbsmoke    |  |           |          |        |       |           |           |
| Nonsmoker  |  | 3402.839  | 9.538926 | 356.73 | 0.000 | 3384.143  | 3421.535  |
| -----      |  |           |          |        |       |           |           |

**Manual IPWRA:** first estimate the propensity score, then run *weighted* regression adjustment within each treatment arm, using the inverse probability of the *observed* treatment status as weights:

```
clear
webuse cattaneo2
logit mbsmoke mmarried mage prenatal1 fbaby
predict ps

gen wgt1 = mbsmoke/ps
gen wgt0 = (1-mbsmoke)/(1-ps)
reg bweight mmarried mage prenatal1 fbaby if mbsmoke==1 [pweight=wgt1]
predict mu1_ipwra
reg bweight mmarried mage prenatal1 fbaby if mbsmoke==0 [pweight=wgt0]
predict mu0_ipwra
gen tau_ipwra_i = mu1_ipwra - mu0_ipwra
sum tau_ipwra_i
```

```
. clear
```

```
. webuse cattaneo2
```

(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)

```
. logit mbsmoke mmarried mage prenatal1 fbaby
```

```
Iteration 0:  Log likelihood = -2230.7484
Iteration 1:  Log likelihood = -2089.7838
Iteration 2:  Log likelihood = -2082.3958
Iteration 3:  Log likelihood = -2082.3879
Iteration 4:  Log likelihood = -2082.3879
```

Logistic regression

Number of obs = 4,642

LR chi2(4) = 296.72

Prob > chi2 = 0.0000

Log likelihood = -2082.3879

Pseudo R2 = 0.0665

```
-----
      mbsmoke | Coefficient  Std. err.      z    P>|z|      [95% conf. interval]
```

|           |  |           |          |        |       |           |           |
|-----------|--|-----------|----------|--------|-------|-----------|-----------|
| mmarried  |  | -1.10666  | .091279  | -12.12 | 0.000 | -1.285564 | -.9277569 |
| mage      |  | -.0187894 | .0082217 | -2.29  | 0.022 | -.0349036 | -.0026751 |
| prenatal1 |  | -.2721103 | .0943067 | -2.89  | 0.004 | -.4569481 | -.0872725 |
| fbaby     |  | -.5428134 | .0864561 | -6.28  | 0.000 | -.7122642 | -.3733625 |
| _cons     |  | .1297324  | .2095812 | 0.62   | 0.536 | -.2810391 | .540504   |

```
. predict ps
(option pr assumed; Pr(mbsmoke))
```

```
.
. gen wgt1 = mbsmoke/ps
```

```
. gen wgt0 = (1-mbsmoke)/(1-ps)
```

```
. reg bweight mmarried mage prenatal1 fbaby if mbsmoke==1 [pweight=wgt1]
(sum of wgt is 4,620.9067401886)
```

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 864    |
|                   | F(4, 859)     | = | 3.12   |
|                   | Prob > F      | = | 0.0145 |
|                   | R-squared     | = | 0.0142 |
|                   | Root MSE      | = | 563.01 |

|           |             | Robust    |       |       |                      |          |
|-----------|-------------|-----------|-------|-------|----------------------|----------|
| bweight   | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |          |
| mmarried  | 130.5978    | 39.90558  | 3.27  | 0.001 | 52.27399             | 208.9217 |
| mage      | -6.495147   | 4.95637   | -1.31 | 0.190 | -16.22316            | 3.232866 |
| prenatal1 | 23.29089    | 40.04843  | 0.58  | 0.561 | -55.31335            | 101.8951 |
| fbaby     | 45.19338    | 48.48447  | 0.93  | 0.352 | -49.96852            | 140.3553 |
| _cons     | 3206.397    | 126.7646  | 25.29 | 0.000 | 2957.592             | 3455.202 |

```
. predict mul_ipwra
(option xb assumed; fitted values)
```

```
. reg bweight mmarried mage prenatal1 fbaby if mbsmoke==0 [pweight=wgt0]
(sum of wgt is 4,643.79376959801)
```

|                   |               |   |        |
|-------------------|---------------|---|--------|
| Linear regression | Number of obs | = | 3,778  |
|                   | F(4, 3773)    | = | 26.89  |
|                   | Prob > F      | = | 0.0000 |
|                   | R-squared     | = | 0.0336 |
|                   | Root MSE      | = | 567.52 |

## 11. Treatment effects and matching

|           |             | Robust    |       |       |                      |           |
|-----------|-------------|-----------|-------|-------|----------------------|-----------|
| bweight   | Coefficient | std. err. | t     | P> t  | [95% conf. interval] |           |
| mmarried  | 160.9954    | 26.87191  | 5.99  | 0.000 | 108.3105             | 213.6803  |
| mage      | 3.061463    | 2.181356  | 1.40  | 0.161 | -1.215289            | 7.338214  |
| prenatal1 | 62.4539     | 28.28456  | 2.21  | 0.027 | 6.99938              | 117.9084  |
| fbaby     | -67.38469   | 20.29791  | -3.32 | 0.001 | -107.1806            | -27.58876 |
| _cons     | 3188.522    | 56.19956  | 56.74 | 0.000 | 3078.338             | 3298.707  |

```
. predict mu0_ipwra
(option xb assumed; fitted values)

. gen tau_ipwra_i = mu1_ipwra - mu0_ipwra

. sum tau_ipwra_i
```

| Variable    | Obs   | Mean      | Std. dev. | Min       | Max      |
|-------------|-------|-----------|-----------|-----------|----------|
| tau_ipwra_i | 4,642 | -238.7679 | 97.94189  | -481.7334 | 6.216553 |

The mean of `tau_ipwra_i` matches `teffects ipwra`'s ATE exactly:  $-238.7679$  in both.

**Manual AIPW:** estimate outcome models  $\hat{\mu}_1(X)$ ,  $\hat{\mu}_0(X)$  (unweighted, one per arm) and the propensity score  $\hat{p}(X)$ , then combine via the augmented-IPW formula

$$\hat{\tau}_i^{aipw} = D_i \frac{Y_i - \hat{\mu}_1(X_i)}{\hat{p}(X_i)} + \hat{\mu}_1(X_i) - (1 - D_i) \frac{Y_i - \hat{\mu}_0(X_i)}{1 - \hat{p}(X_i)} - \hat{\mu}_0(X_i) \quad (11.1)$$

```
clear
webuse cattaneo2
logit mbsmoke mmarried mage prenatal1 fbaby
predict ps

reg bweight mmarried mage prenatal1 fbaby if mbsmoke==1
predict mu1_aipw
reg bweight mmarried mage prenatal1 fbaby if mbsmoke==0
predict mu0_aipw
gen aipw_i = mbsmoke*(bweight-mu1_aipw)/ps + mu1_aipw - (1-mbsmoke)*(bweight-mu0_aipw)/(1-ps)
sum aipw_i
```

```
. clear

. webuse cattaneo2
(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)
```



```
. logit mbsmoke mmarried mage prenatal1 fbaby
```

```
Iteration 0: Log likelihood = -2230.7484
Iteration 1: Log likelihood = -2089.7838
Iteration 2: Log likelihood = -2082.3958
Iteration 3: Log likelihood = -2082.3879
Iteration 4: Log likelihood = -2082.3879
```

```
Logistic regression
```

```
Number of obs = 4,642
LR chi2(4)      = 296.72
Prob > chi2     = 0.0000
Pseudo R2      = 0.0665
```

```
Log likelihood = -2082.3879
```

| mbsmoke   | Coefficient | Std. err. | z      | P> z  | [95% conf. interval] |           |
|-----------|-------------|-----------|--------|-------|----------------------|-----------|
| mmarried  | -1.10666    | .091279   | -12.12 | 0.000 | -1.285564            | -.9277569 |
| mage      | -.0187894   | .0082217  | -2.29  | 0.022 | -.0349036            | -.0026751 |
| prenatal1 | -.2721103   | .0943067  | -2.89  | 0.004 | -.4569481            | -.0872725 |
| fbaby     | -.5428134   | .0864561  | -6.28  | 0.000 | -.7122642            | -.3733625 |
| _cons     | .1297324    | .2095812  | 0.62   | 0.536 | -.2810391            | .540504   |

```
. predict ps
```

```
(option pr assumed; Pr(mbsmoke))
```

```
.
```

```
. reg bweight mmarried mage prenatal1 fbaby if mbsmoke==1
```

| Source   | SS         | df  | MS         | Number of obs | = | 864    |
|----------|------------|-----|------------|---------------|---|--------|
| Model    | 4586519.21 | 4   | 1146629.8  | F(4, 859)     | = | 3.69   |
| Residual | 266914157  | 859 | 310726.609 | Prob > F      | = | 0.0055 |
| Total    | 271500676  | 863 | 314601.015 | R-squared     | = | 0.0169 |
|          |            |     |            | Adj R-squared | = | 0.0123 |
|          |            |     |            | Root MSE      | = | 557.43 |

| bweight   | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|-----------|-------------|-----------|-------|-------|----------------------|----------|
| mmarried  | 133.6617    | 41.33023  | 3.23  | 0.001 | 52.54164             | 214.7818 |
| mage      | -7.370881   | 3.983409  | -1.85 | 0.065 | -15.18924            | .4474735 |
| prenatal1 | 25.11133    | 43.64639  | 0.58  | 0.565 | -60.55473            | 110.7774 |
| fbaby     | 41.43991    | 41.82146  | 0.99  | 0.322 | -40.6443             | 123.5241 |
| _cons     | 3227.169    | 102.3392  | 31.53 | 0.000 | 3026.305             | 3428.033 |

```
. predict mul_aipw
```

```
(option xb assumed; fitted values)
```

## 11. Treatment effects and matching

```
. reg bweight mmarried mage prenatal1 fbaby if mbsmoke==0
```

| Source   | SS         | df    | MS         | Number of obs | = | 3,778  |
|----------|------------|-------|------------|---------------|---|--------|
| Model    | 37916015   | 4     | 9479003.74 | F(4, 3773)    | = | 30.00  |
| Residual | 1.1922e+09 | 3,773 | 315979.752 | Prob > F      | = | 0.0000 |
|          |            |       |            | R-squared     | = | 0.0308 |
|          |            |       |            | Adj R-squared | = | 0.0298 |
| Total    | 1.2301e+09 | 3,777 | 325683.776 | Root MSE      | = | 562.12 |

| bweight   | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |           |
|-----------|-------------|-----------|-------|-------|----------------------|-----------|
| mmarried  | 160.9513    | 24.55483  | 6.55  | 0.000 | 112.8092             | 209.0933  |
| mage      | 2.546828    | 1.935504  | 1.32  | 0.188 | -1.247908            | 6.341564  |
| prenatal1 | 64.40859    | 25.83668  | 2.49  | 0.013 | 13.75338             | 115.0638  |
| fbaby     | -71.3286    | 19.4895   | -3.66 | 0.000 | -109.5396            | -33.11763 |
| _cons     | 3202.746    | 51.20076  | 62.55 | 0.000 | 3102.362             | 3303.13   |

```
. predict mu0_aipw
```

(option xb assumed; fitted values)

```
. gen aipw_i = mbsmoke*(bweight-mu1_aipw)/ps + mu1_aipw - (1-mbsmoke)*(bweight->
> mu0_aipw)/(1-ps) - mu0_aipw
```

```
. sum aipw_i
```

| Variable | Obs   | Mean      | Std. dev. | Min       | Max      |
|----------|-------|-----------|-----------|-----------|----------|
| aipw_i   | 4,642 | -239.0294 | 1618.64   | -29389.38 | 16313.36 |

The mean of `aipw_i` is  $-239.0294$ , matching `teffects aipw`'s ATE to four decimals. (One might expect a small gap, since `teffects` estimates the treatment and outcome models jointly while this is a two-step plug-in; here the point estimates coincide, and only the standard errors would differ.)

The two doubly-robust estimators land within 0.3 grams of each other,  $-238.77$  for IPWRA and  $-239.03$  for AIPW, both about 8 grams away from plain IPW's  $-230.69$ .

The unit-level `aipw_i` values are worth a glance before trusting the mean. They have a standard deviation of 1,618.6 and run from  $-29,389$  to  $+16,313$  — two orders of magnitude wider than the estimate itself. That spread comes from dividing residuals by  $\hat{p}$  or  $1 - \hat{p}$  for units with extreme propensity scores. The average is well behaved, but it is an average over some very large individual terms, which is exactly why AIPW can be unstable when overlap is poor.

## 11.2. Matching

Matching does not solve the endogeneity problem. It addresses selection on observables only; with unobserved confounders it does not help.

What matching does is balance the covariate distributions across treatment and control groups. Regression does this too, but matching makes the lack of overlap visible — a plain regression can extrapolate far outside the support without warning. Running regression on a matched sample combines the strengths of both approaches.

Below we cover three matching methods in Stata: propensity score matching, nearest-neighbor matching, and coarsened exact matching.

### 11.2.1. Propensity score matching

Exact matching on all covariates is ideal but infeasible in high dimensions. Rosenbaum and Rubin (1983) showed that the propensity score provides a scalar sufficient statistic, reducing the matching problem to one dimension.

Stata's `teffects psmatch` and `psmatch2` both estimate the propensity score, match, and compute the treatment effect with correct standard errors in one step.

```
clear
webuse cattaneo2
teffects psmatch (bweight) (mbsmoke mmarried c.mage##c.mage fbaby medu), atet
psmatch2 mbsmoke mmarried c.mage##c.mage fbaby medu, logit ties
reg bweight mbsmoke [aweight=_weight]
```

```
. clear
```

```
. webuse cattaneo2
```

(Excerpt from Cattaneo (2010) *Journal of Econometrics* 155: 138–154)

```
. teffects psmatch (bweight) (mbsmoke mmarried c.mage##c.mage fbaby medu), atet
```

|                                       |                    |   |       |
|---------------------------------------|--------------------|---|-------|
| Treatment-effects estimation          | Number of obs      | = | 4,642 |
| Estimator : propensity-score matching | Matches: requested | = | 1     |
| Outcome model : matching              | min                | = | 1     |
| Treatment model: logit                | max                | = | 74    |

|         |         | AI robust   |           |   |      |                      |
|---------|---------|-------------|-----------|---|------|----------------------|
|         | bweight | Coefficient | std. err. | z | P> z | [95% conf. interval] |
| ATET    |         |             |           |   |      |                      |
| mbsmoke |         |             |           |   |      |                      |
| (Smoker |         |             |           |   |      |                      |
| vs      |         |             |           |   |      |                      |

## 11. Treatment effects and matching

```
Nonsmoker) | -236.7848 26.57789 -8.91 0.000 -288.8765 -184.693
```

```
. psmatch2 mbsmoke mmarried c.mage##c.mage fbaby medu, logit ties
```

```
Logistic regression                                Number of obs = 4,642
                                                    LR chi2(5)    = 375.00
                                                    Prob > chi2   = 0.0000
Log likelihood = -2043.2504                        Pseudo R2    = 0.0841
```

| mbsmoke  | Coefficient | Std. err. | z      | P> z  | [95% conf. interval] |           |
|----------|-------------|-----------|--------|-------|----------------------|-----------|
| mmarried | -1.145706   | .0918962  | -12.47 | 0.000 | -1.32582             | -.965593  |
| mage     | .321518     | .0638472  | 5.04   | 0.000 | .1963798             | .4466563  |
| c.mage#  |             |           |        |       |                      |           |
| c.mage   | -.0060368   | .0011849  | -5.09  | 0.000 | -.0083592            | -.0037144 |
| fbaby    | -.3864258   | .0880445  | -4.39  | 0.000 | -.5589898            | -.2138618 |
| medu     | -.1420833   | .0173215  | -8.20  | 0.000 | -.1760328            | -.1081338 |
| _cons    | -2.950915   | .8102504  | -3.64  | 0.000 | -4.538976            | -1.362853 |

```
. reg bweight mbsmoke [aweight=_weight]
(sum of wgt is 1,728)
```

| Source   | SS         | df    | MS         | Number of obs | = | 3,671  |
|----------|------------|-------|------------|---------------|---|--------|
| Model    | 51455506   | 1     | 51455506   | F(1, 3669)    | = | 151.87 |
| Residual | 1.2431e+09 | 3,669 | 338808.237 | Prob > F      | = | 0.0000 |
|          |            |       |            | R-squared     | = | 0.0397 |
|          |            |       |            | Adj R-squared | = | 0.0395 |
| Total    | 1.2945e+09 | 3,670 | 352736.492 | Root MSE      | = | 582.07 |

| bweight | Coefficient | Std. err. | t      | P> t  | [95% conf. interval] |           |
|---------|-------------|-----------|--------|-------|----------------------|-----------|
| mbsmoke | -236.7848   | 19.21387  | -12.32 | 0.000 | -274.4557            | -199.1138 |
| _cons   | 3374.444    | 13.58626  | 248.37 | 0.000 | 3347.807             | 3401.082  |

`teffects psmatch` gives the ATT of smoking on birthweight,  $-236.7848$  with a standard error of 26.58. The manual replication uses `psmatch2` with `logit` and `ties` (by default `teffects` includes all tied matches, while `psmatch2` keeps only one), then runs a weighted regression, and gives  $-236.7848$  — the same to four decimals. Point estimates match; standard errors should be taken from `teffects`.

The nearest-neighbour ATT sits close to the two doubly-robust ATEs above,  $-238.8$  and  $-239.0$ , though it is a different estimand on a different subpopulation.

### 11.2.2. Nearest neighbor matching

`teffects nnmatch` uses Mahalanobis distance by default and can also enforce exact matching on discrete covariates. It is nonparametric.

```
clear
webuse cattaneo2
teffects nnmatch (bweight mmarried mage fbaby medu) (mb smoke) , atet
teffects psmatch (bweight) (mb smoke mmarried mage fbaby medu), atet
```

```
. clear

. webuse cattaneo2
(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)

. teffects nnmatch (bweight mmarried mage fbaby medu) (mb smoke) , atet
```

```
Treatment-effects estimation      Number of obs      =      4,642
Estimator      : nearest-neighbor matching      Matches: requested =      1
Outcome model  : matching                      min =      1
Distance metric: Mahalanobis                  max =      74
```

```
-----+-----
          |               AI robust
      bweight | Coefficient  std. err.      z    P>|z|      [95% conf. interval]
-----+-----
ATET      |
      mb smoke |
      (Smoker  |
        vs     |
      Nonsmoker) | -239.2433   25.68524   -9.31   0.000   -289.5854   -188.9011
-----+-----
```

```
. teffects psmatch (bweight) (mb smoke mmarried mage fbaby medu), atet
```

```
Treatment-effects estimation      Number of obs      =      4,642
Estimator      : propensity-score matching      Matches: requested =      1
Outcome model  : matching                      min =      1
Treatment model: logit                  max =      74
```

```
-----+-----
          |               AI robust
      bweight | Coefficient  std. err.      z    P>|z|      [95% conf. interval]
-----+-----
```

## 11. Treatment effects and matching

|            |  |          |          |       |       |           |           |
|------------|--|----------|----------|-------|-------|-----------|-----------|
| ATET       |  |          |          |       |       |           |           |
| mbsmoke    |  |          |          |       |       |           |           |
| (Smoker    |  |          |          |       |       |           |           |
| vs         |  |          |          |       |       |           |           |
| Nonsmoker) |  | -245.711 | 26.38675 | -9.31 | 0.000 | -297.4281 | -193.9939 |

---

.

Note the specification difference: `nnmatch` lists covariates in the outcome parentheses, `psmatch` lists them in the treatment parentheses.

### 11.2.3. Coarsened Exact Matching (CEM)

CEM is available as a separate Stata package.

```
clear
webuse cattaneo2
cem mmarried mage fbaby medu, treatment(mbsmoke)
reg bweight mbsmoke [aweight=cem_weights]
* Note: cem creates the variable "cem_weights" (plural). We spell it out
* in full here rather than relying on Stata's automatic variable-name
* abbreviation matching, to keep the reference unambiguous.
```

```
. clear
```

```
. webuse cattaneo2
```

(Excerpt from Cattaneo (2010) Journal of Econometrics 155: 138-154)

```
. cem mmarried mage fbaby medu, treatment(mbsmoke)
```

Matching Summary:

-----

Number of strata: 274

Number of matched strata: 135

|           |      |     |
|-----------|------|-----|
|           | 0    | 1   |
| All       | 3778 | 864 |
| Matched   | 3421 | 827 |
| Unmatched | 357  | 37  |

Multivariate L1 distance: .25424705

Univariate imbalance:

|          | L1      | mean    | min | 25% | 50% | 75% | max |
|----------|---------|---------|-----|-----|-----|-----|-----|
| mmarried | 6.4e-15 | 8.4e-15 | 0   | 0   | 0   | 0   | 0   |
| mage     | .04955  | -.00887 | 1   | 0   | 1   | 0   | -2  |
| fbaby    | 6.1e-15 | 6.1e-15 | 0   | 0   | 0   | 0   | 0   |
| medu     | .03809  | -.03316 | 0   | 0   | 0   | 0   | 0   |

```
. reg bweight mbsmoke [aweight=cem_weights]
(sum of wgt is 4,248.000000000005)
```

| Source   | SS         | df    | MS         | Number of obs | = | 4,248  |
|----------|------------|-------|------------|---------------|---|--------|
| -----+   |            |       |            |               |   |        |
| Model    | 40566772.9 | 1     | 40566772.9 | F(1, 4246)    | = | 121.12 |
| Residual | 1.4221e+09 | 4,246 | 334928.257 | Prob > F      | = | 0.0000 |
| -----+   |            |       |            |               |   |        |
| Total    | 1.4627e+09 | 4,247 | 344401.26  | R-squared     | = | 0.0277 |
|          |            |       |            | Adj R-squared | = | 0.0275 |
|          |            |       |            | Root MSE      | = | 578.73 |

| bweight | Coefficient | Std. err. | t      | P> t  | [95% conf. interval] |           |
|---------|-------------|-----------|--------|-------|----------------------|-----------|
| -----+  |             |           |        |       |                      |           |
| mbsmoke | -246.8017   | 22.42533  | -11.01 | 0.000 | -290.7671            | -202.8364 |
| _cons   | 3383.394    | 9.894625  | 341.94 | 0.000 | 3363.996             | 3402.793  |

```
. * Note: cem creates the variable "cem_weights" (plural). We spell it out
. * in full here rather than relying on Stata's automatic variable-name
. * abbreviation matching, to keep the reference unambiguous.
.
```

CEM creates exact-match strata on coarsened covariates and generates weights for the final regression. The standard errors do not account for the matching step.

## 11.3. Modern matching in R: balance diagnostics and entropy weighting

The 2019 vintage of this chapter used Stata's `teffects` throughout. R's `MatchIt` and `WeightIt` packages have since become the standard for matching workflows, and `cobalt` provides publication-quality balance tables and plots that are now expected in applied papers.

### 11.3.1. Balance checking with `cobalt`

After any matching or weighting step, always inspect covariate balance. `cobalt`'s `bal.tab()` and `love.plot()` are the go-to tools:

## 11. Treatment effects and matching

```
library(MatchIt)
library(cobalt)

# Propensity score matching on the lalonde data
data("lalonde", package = "MatchIt")

m_out <- matchit(treat ~ age + educ + race + married + nodegree +
                 re74 + re75,
                 data = lalonde, method = "nearest", distance = "glm",
                 ratio = 1)

# Balance table: standardized mean differences before and after matching
bal.tab(m_out, stats = c("mean.diffs", "variance.ratios"), thresholds = c(m = 0.1))
```

### Balance Measures

|             | Type     | Diff.Adj | M.Threshold        | V.Ratio.Adj |
|-------------|----------|----------|--------------------|-------------|
| distance    | Distance | 0.9739   |                    | 0.7566      |
| age         | Contin.  | 0.0718   | Balanced, <0.1     | 0.4568      |
| educ        | Contin.  | -0.1290  | Not Balanced, >0.1 | 0.5721      |
| race_black  | Binary   | 0.3730   | Not Balanced, >0.1 | .           |
| race_hispan | Binary   | -0.1568  | Not Balanced, >0.1 | .           |
| race_white  | Binary   | -0.2162  | Not Balanced, >0.1 | .           |
| married     | Binary   | -0.0216  | Balanced, <0.1     | .           |
| nodegree    | Binary   | 0.0703   | Balanced, <0.1     | .           |
| re74        | Contin.  | -0.0505  | Balanced, <0.1     | 1.3289      |
| re75        | Contin.  | -0.0257  | Balanced, <0.1     | 1.4956      |

### Balance tally for mean differences

|                    | count |
|--------------------|-------|
| Balanced, <0.1     | 5     |
| Not Balanced, >0.1 | 4     |

### Variable with the greatest mean difference

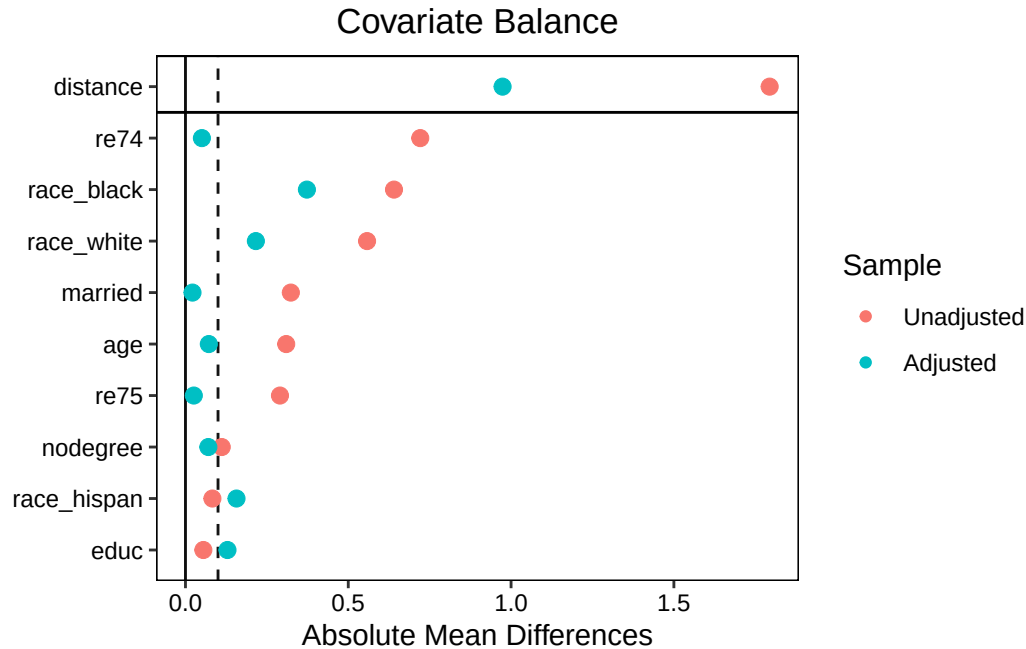
| Variable   | Diff.Adj | M.Threshold        |
|------------|----------|--------------------|
| race_black | 0.373    | Not Balanced, >0.1 |

### Sample sizes

|           | Control | Treated |
|-----------|---------|---------|
| All       | 429     | 185     |
| Matched   | 185     | 185     |
| Unmatched | 244     | 0       |

```
# Love plot: visualize balance improvement
love.plot(m_out, stats = "mean.diffs", threshold = 0.1,
          abs = TRUE, var.order = "unadjusted")
```





The love plot shows each covariate's standardized mean difference (SMD) before matching (open circles) and after (filled circles). The vertical line at SMD = 0.1 is the conventional threshold for "balanced." If post-matching points cross the line, the matching is not improving balance and a different specification is needed.

That is exactly what happens here, which is why it is worth running the diagnostic rather than assuming. 1:1 nearest-neighbour matching on the propensity score leaves four of the nine covariates *above* the 0.1 threshold, **race** worst of all (an adjusted SMD of 0.37 for the black indicator), while discarding 244 of the 429 controls. Matching on a propensity score balances the score, not necessarily the covariates that went into it.

### 11.3.2. Entropy balancing

Propensity score matching discards unmatched units and can reduce effective sample size substantially. **Entropy balancing** (Hainmueller 2012) keeps all units and reweights the control group so that weighted covariate moments exactly match the treatment group — without discarding anyone.

```
library(WeightIt)
library(cobalt)

# Entropy balancing: exact balance on means, variances, and covariances
w_out <- weightit(treat ~ age + educ + race + married + nodegree +
  re74 + re75,
  data = lalonde, method = "ebal", estimand = "ATT",
  moments = 1) # moments=1: balance means only
               # moments=2: balance means + variances

summary(w_out)
```

## 11. Treatment effects and matching

### Summary of weights

- Weight ranges:

|         | Min   | Max   |
|---------|-------|-------|
| treated | 1.    | 1.    |
| control | 0.008 | 4.062 |

- Units with the 5 most extreme weights by group:

|         | NSW5    | NSW4    | NSW3    | NSW2    | NSW1    |
|---------|---------|---------|---------|---------|---------|
| treated | 1       | 1       | 1       | 1       | 1       |
|         | PSID423 | PSID196 | PSID412 | PSID118 | PSID226 |
| control | 3.073   | 3.235   | 3.45    | 3.897   | 4.062   |

- Weight statistics:

|         | Coef of Var | MAD   | Entropy | # Zeros |
|---------|-------------|-------|---------|---------|
| treated | 0.          | 0.    | 0.      | 0       |
| control | 1.834       | 1.287 | 1.101   | 0       |

- Effective Sample Sizes:

|            | Control | Treated |
|------------|---------|---------|
| Unweighted | 429.    | 185     |
| Weighted   | 98.46   | 185     |

```
# Check balance
bal.tab(w_out, stats = "mean.diffs", thresholds = c(m = 0.1))
```

### Balance Measures

|             | Type    | Diff.Adj | M.Threshold    |
|-------------|---------|----------|----------------|
| age         | Contin. | -0       | Balanced, <0.1 |
| educ        | Contin. | -0       | Balanced, <0.1 |
| race_black  | Binary  | 0        | Balanced, <0.1 |
| race_hispan | Binary  | 0        | Balanced, <0.1 |
| race_white  | Binary  | -0       | Balanced, <0.1 |
| married     | Binary  | -0       | Balanced, <0.1 |
| nodegree    | Binary  | 0        | Balanced, <0.1 |
| re74        | Contin. | -0       | Balanced, <0.1 |
| re75        | Contin. | -0       | Balanced, <0.1 |

Balance tally for mean differences

|                    | count |
|--------------------|-------|
| Balanced, <0.1     | 9     |
| Not Balanced, >0.1 | 0     |

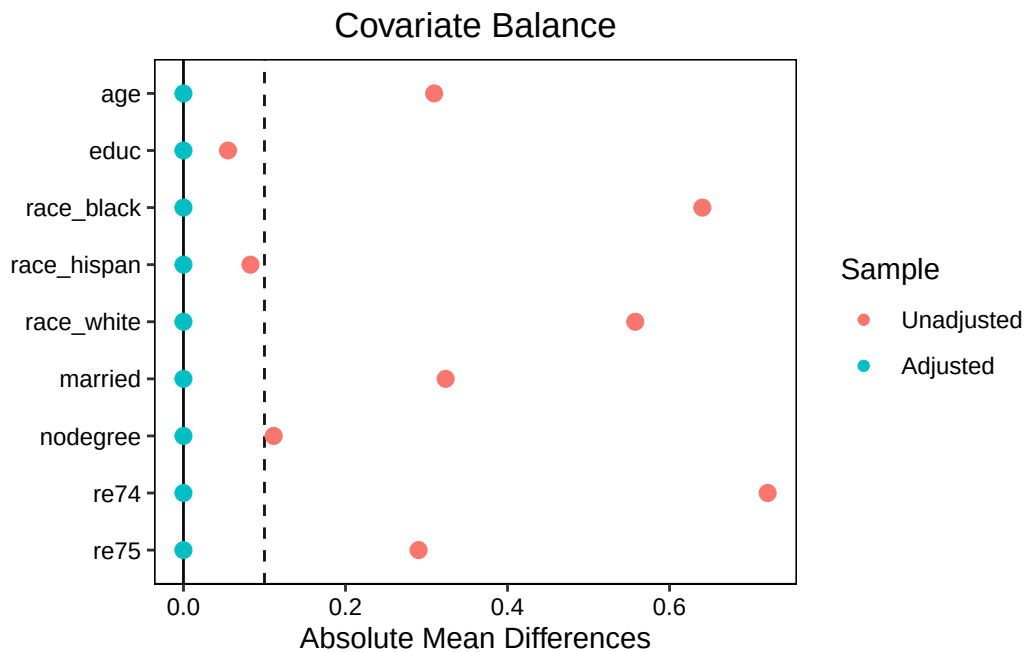
Variable with the greatest mean difference

| Variable | Diff.Adj | M.Threshold    |
|----------|----------|----------------|
| re74     | -0       | Balanced, <0.1 |

Effective sample sizes

|            | Control | Treated |
|------------|---------|---------|
| Unadjusted | 429.    | 185     |
| Adjusted   | 98.46   | 185     |

```
love.plot(w_out, threshold = 0.1, abs = TRUE)
```



```
# Weighted outcome regression
library(lmtest); library(sandwich)
m_eb <- lm(re78 ~ treat + age + educ + race + married + nodegree +
           re74 + re75,
           data = lalonde, weights = w_out$weights)
coeftest(m_eb, vcov = vcovHC(m_eb, type = "HC3"))["treat", ]
```

| Estimate     | Std. Error  | t value   | Pr(> t )  |
|--------------|-------------|-----------|-----------|
| 1273.2618139 | 820.1848528 | 1.5524083 | 0.1210883 |

Entropy balancing delivers on its promise here: the largest absolute standardized mean difference after weighting is **exactly zero**, since the weights are chosen to satisfy the mean-balance constraints as equalities. All 429 controls are retained.

## 11. Treatment effects and matching

| Method                 | Pros                                                                                 | Cons                                                             |
|------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------|
| PSM (nearest neighbor) | Intuitive; respects overlap                                                          | Discards units; balance not guaranteed                           |
| CEM                    | Exact balance in bins                                                                | Requires discretizing continuous vars                            |
| Entropy balancing      | Exact mean balance; keeps all units                                                  | Weights can be extreme; no variance balance by default           |
| IPW (logistic)         | Simple; relies only on the treatment model                                           | Not doubly robust on its own; extreme weights if overlap is poor |
| AIPW / IPWRA           | Doubly robust: consistent if <i>either</i> the treatment or outcome model is correct | Needs both models specified; more moving parts                   |

For most applications, entropy balancing or IPW with trimmed weights (via `WeightIt`'s `trim()`) is preferred over PSM: better effective sample size, guaranteed balance, and easy integration with doubly-robust estimators.

See also the [weighting chapter](#) for IPW estimators and the [OLS-ATE chapter](#) for regression-based approaches.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 12. When is OLS coefficient the ATE

### 12.1. OLS cannot be ATE (most of the time)

[Słoczyński](#) clarifies when the OLS coefficient on a treatment dummy can and cannot be read as the ATE.

Consider the standard setup:

$$y = \alpha + \tau d + X\beta \quad (12.1)$$

Here  $d$  is the treatment dummy and  $X$  the covariates, under unconfoundedness. If the treatment effect is homogeneous,  $\tau$  is the ATE. Under heterogeneous effects,  $\tau$  is a convex combination of ATT and ATU — but with weights that are inversely proportional to the treatment shares. The more treated units there are, the less weight falls on ATT.

By definition,

$$\tau_{ATE} = \rho\tau_{ATT} + (1 - \rho)\tau_{ATU} \quad (12.2)$$

The main result from the paper is:

$$\tau = (1 - \rho)\tau_{ATT} + \rho\tau_{ATU} \quad (12.3)$$

where  $\rho = P(d = 1)$ , the proportion of treated;  $\tau_{ATT} = E(y(1) - y(0)|d = 1)$ ,  $\tau_{ATU} = E(y(1) - y(0)|d = 0)$ , and  $\tau_{ATE} = E(y(1) - y(0))$ .

From these two equations, we see that  $\tau$  and  $\tau_{ATE}$  are quite different, unless the weight on ATT happens to equal  $\rho$ . A caution on reading this too simply: the clean form  $\tau = (1 - \rho)\tau_{ATT} + \rho\tau_{ATU}$  with  $\rho = P(d = 1)$  is the **no-covariate special case** (or, with covariates, holds in terms of *covariate-adjusted* treatment-probability weights, not the raw marginal share). In Słoczyński's general result the weights depend on the conditional treatment probabilities (the propensity score), so a 50/50 *marginal* treatment share alone does **not** guarantee that OLS recovers the ATE — what matters is the covariate-adjusted weighting. With that caveat, the intuition stands: the more imbalanced the (conditional) treatment probabilities, the further OLS can be from the ATE.

Given the true propensity score, ATT, ATU, and ATE can be separated. In practice we use the estimated propensity score. Słoczyński provides the `hettreatreg` package in R and Stata.

The data are `nswcps`: the NSW experimental treatment group combined with a large CPS comparison sample, so the treated are a tiny fraction of the whole. The outcome is 1978 earnings and the covariates are age, its square, education, race, marital status, degree status, and earnings in 1974

## 12. When is OLS coefficient the ATE

and 1975. This is the classic LaLonde setting, and its extreme treatment imbalance is exactly what makes the weighting result bite.

```
library(hettreatreg)
data("nswcps")

summary(lm(re78 ~ treated + age + age2 + educ + black + hispanic + married + nodegree + re74 + re75, data = nswcps))
```

Call:

```
lm(formula = re78 ~ treated + age + age2 + educ + black + hispanic + married + nodegree + re74 + re75, data = nswcps)
```

Residuals:

| Min    | 1Q    | Median | 3Q   | Max   |
|--------|-------|--------|------|-------|
| -25130 | -3601 | 1274   | 3668 | 55040 |

Coefficients:

|             | Estimate   | Std. Error | t value | Pr(> t )     |
|-------------|------------|------------|---------|--------------|
| (Intercept) | 7634.34415 | 736.67074  | 10.363  | < 2e-16 ***  |
| treated     | 793.58704  | 548.25433  | 1.447   | 0.14778      |
| age         | -233.67749 | 41.18067   | -5.674  | 1.42e-08 *** |
| age2        | 1.81437    | 0.56099    | 3.234   | 0.00122 **   |
| educ        | 166.84923  | 28.65984   | 5.822   | 5.94e-09 *** |
| black       | -790.60856 | 213.24523  | -3.708  | 0.00021 ***  |
| hispanic    | -175.97512 | 218.99126  | -0.804  | 0.42166      |
| married     | 224.26599  | 149.84542  | 1.497   | 0.13450      |
| nodegree    | 311.84453  | 178.51743  | 1.747   | 0.08068 .    |
| re74        | 0.29534    | 0.01222    | 24.175  | < 2e-16 ***  |
| re75        | 0.47064    | 0.01216    | 38.700  | < 2e-16 ***  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7002 on 16166 degrees of freedom

Multiple R-squared: 0.4762, Adjusted R-squared: 0.4758

F-statistic: 1469 on 10 and 16166 DF, p-value: < 2.2e-16

```
outcome <- nswcps$re78
treated <- nswcps$treated
our_vars <- c("age", "age2", "educ", "black", "hispanic", "married", "nodegree", "re74", "re75")
covariates <- subset(nswcps, select = our_vars)

hettreatreg(outcome, treated, covariates, verbose = TRUE)
```

"OLS" is the estimated regression coefficient on treated.

```

OLS = 793.6

P(d=1) = 0.011
P(d=0) = 0.989

w1 = 0.983
w0 = 0.017
delta = -0.971

ATE = -6751
ATT = 928.4
ATU = -6840

OLS = w1*ATT + w0*ATU = 793.6

```

The output is the theorem in numbers. OLS puts 793.6 on **treated**. Only 1.1% of the sample is treated,  $P(d = 1) = 0.011$ , and the implied OLS weights are  $w_1 = 0.983$  on ATT and  $w_0 = 0.017$  on ATU. Compare those with the ATE's own weights, which are  $\rho = 0.011$  on ATT and 0.989 on ATU: they are very nearly reversed. The package prints the identity directly,  $OLS = w_1 \text{ ATT} + w_0 \text{ ATU} = 793.6$ , which closes exactly.

The consequences are not subtle. ATT is 928.4, ATU is  $-6840$ , and ATE is  $-6751$ . So the OLS coefficient of 793.6 sits within 135 of the ATT and roughly 7,500 away from the ATE — and on the *opposite side of zero*. Reading 793.6 as “the effect of training on earnings” would get the sign wrong for the population. The `delta` of  $-0.971$  that the package reports is the gap between the OLS weight on ATT and  $\rho$ ; it is close to  $-1$ , its extreme value.

Słoczyński's paper itself is an algebraic decomposition of the OLS coefficient  $\tau$  in terms of ATT/ATU weights; it does not propose or endorse the propensity-score-as-covariate estimator below as a general-purpose way to recover ATT/ATU/ATE. What follows is our own illustrative exercise, done in three steps, to see how such an estimator would behave in practice: First, estimate the propensity score equation, basically

$$d = X\gamma \tag{12.4}$$

Second, include the predicted propensity score as a covariate and estimate the model with treatment and control group separately. Note that regressing  $Y$  linearly on the scalar propensity score  $p(X)$  assumes  $E[Y(d) \mid p(X)]$  is linear in  $p(X)$  — a strong and generally untested assumption, unlike matching/weighting directly on the propensity score, or regression adjustment on the covariates  $X$  themselves, which are the standard approaches. Treat the numbers below as an exploratory illustration rather than a recommended estimator.

Third, predict the counterfactuals for the full sample. Then we get the ATE, ATT and ATU.

One more reason to treat this as illustration rather than method: the propensity score here comes from a linear probability model, and on these data it runs from  $-0.034$  to  $0.154$  — some fitted

## 12. When is OLS coefficient the ATE

“probabilities” are negative. That is harmless when the score is used merely as a regressor, as it is here, but it would be fatal if the same score were used as an inverse-probability weight.

Let’s see with the same data:

```
m_propensity <- lm(treated ~ age + age2 + educ + black + hispanic + married + nodegree + re74
ps <- predict(m_propensity)
df_combined <- data.frame(nswcps, ps)
df.1 <- df_combined[which(treated == 1),]
df.0 <- df_combined[which(treated == 0),]
m_ot <- lm(re78 ~ ps, data = df.1)
ot <- suppressWarnings(predict(m_ot, newdata = df_combined)) # add newdata option to predict f

m_oc <- lm(re78 ~ ps, data = df.0)
oc <- suppressWarnings(predict(m_oc, newdata = df_combined)) # add newdata option to predict f

te <- ot - oc
ate <- mean(te)
df_combined$te <- te
att <- as.numeric(mean(df_combined[which(treated == 1), 'te']))
atu <- as.numeric(mean(df_combined[which(treated == 0), 'te']))

print(paste("ate=", signif(ate, 4)))
```

```
[1] "ate= -6751"
```

```
print(paste("att=", signif(att, 4)))
```

```
[1] "att= 928.4"
```

```
print(paste("atu=", signif(atu, 4)))
```

```
[1] "atu= -6840"
```

This reproduces the package’s decomposition exactly: ATE  $-6751$ , ATT  $928.4$ , ATU  $-6840$ , the same three numbers to four significant figures.

We see in this case OLS coefficient is hugely different from ATE. This is because  $P(d = 1)$  is only .01, about 1 percent; we have much larger control group than treatment group. In this case, the OLS coefficient is far different from ATE; in fact, it’s pretty close to ATT.

Now, what’s the intuition that OLS coefficient is just the opposite as the ATE in terms of weighting ATT and ATU? The reason is that OLS is to predict actual outcome, not counterfactual outcome. For calculating ATE, we need counterfactual outcome predicted. Now suppose we have a lot of control observations, we’d much better predicting  $(y(0)|d == 0)$  rather than  $(y(1)|d == 1)$ . That is, we are better getting the slope of  $(y(0)|d == 0, X = x)$ ; therefore predicting  $(y(0)|d == 1, X = x)$ . Therefore although treatment group has less observations, but because the  $(y(0)|d == 1)$  is better predicted, the ATT is better estimated, thus more weight. If the opposite happens, then the ATU should be more heavily weighted. Thus the counter-intuitive weighting.



## 12.2. Regression adjustment

Słoczyński's method is to include propensity score as a covariate. This propensity score serves as a proxy as all covariates. As Rosenbaum and Rubin (1983) showed, the propensity score is a balancing score (treatment is independent of the covariates given the propensity score). Another way to allow treatment effect to differ across all covariates is to allow interaction of treatment dummy with all covariates, then I am doing "Regression Adjustment" (see Stata's `teffects ra`, used in the [treatment effects and matching chapter](#); note this is not `treatreg`, which was the old endogenous-treatment command, superseded by `etregress`). We can calculate ATE, ATT, ATU after the linear model.

```
library(tidyverse)
lm1 <- lm(re78 ~ treated*(age + age2 + educ + black + hispanic + married + nodegree + re74 +
y1 <- predict(lm1,newdata=nswcps %>% mutate(treated=1))
y0 <- predict(lm1,newdata=nswcps %>% mutate(treated=0))
ate=mean(y1-y0) #ate
print(paste("ate=", signif(ate, 4)))
```

```
[1] "ate= -4930"
```

```
y1.att <- predict(lm1,newdata=nswcps %>% filter(treated==1))
y0.att <- predict(lm1,newdata=nswcps %>% filter(treated==1) %>% mutate(treated=0))
att=mean(y1.att)-mean(y0.att) #att
print(paste('att=', signif(att, 4)))
```

```
[1] "att= 796"
```

```
y1.atu <- predict(lm1,newdata=nswcps %>% filter(treated==0) %>% mutate(treated=1))
y0.atu <- predict(lm1,newdata=nswcps %>% filter(treated==0))
atu=mean(y1.atu)-mean(y0.atu) #atu
print(paste('atu=', signif(atu, 4)))
```

```
[1] "atu= -4996"
```

Regression adjustment gives ATE  $-4930$ , ATT  $796$  and ATU  $-4996$ .

We see this is somewhat different from Słoczyński's method, which gave  $-6751$ ,  $928.4$  and  $-6840$ : the two agree on sign and order of magnitude but not on level, unsurprisingly, since one regresses on a scalar propensity score and the other interacts the treatment with all nine covariates. But the point is the original OLS coefficient  $794$  is nowhere close to ATE under either approach —  $-4930$  or  $-6751$ , both far away and both negative. In this case, it's very close to ATT ( $796$  by regression adjustment,  $928$  by the propensity-score route), as the treatment group is very small.



## 13. Matching and Weighting Part 1: matching

This chapter follows R's `MatchIt` and `WeightIt` packages (Greifer et al.).

### 13.1. Assumptions

Matching and weighting require three assumptions for causal identification in observational studies: (1) SUTVA, (2) ignorability (unconfoundedness), and (3) overlap (positivity). Under ignorability, conditioning on  $X$  makes treatment assignment  $D$  independent of the potential outcomes  $Y(0)$  and  $Y(1)$ . In practice, the distribution of  $X$  is rarely balanced across treatment and control groups — matching and weighting restore that balance.

### 13.2. Distance

Matching finds control units close to treated units in  $X$ -space. The first question is how to define “close.”

#### 13.2.1. Propensity score

The most common distance measure is the propensity score (PS), which is the probability of being treated given  $X$ . The benefit is that when there is high dimension of  $X$ , we can reduce the dimension to a single number, the PS.

There are many ways to estimate the propensity score, such as logit, or more flexible parametric “gam”, or other machine learning methods, such as “gbm”, “lasso”, “rpart”, “randomforest”, “cbps”, etc. Here CBPS has seen more popularity. The propensity scores are estimated using the covariate balancing propensity score (CBPS) algorithm, which is a form of logistic regression where balance constraints are incorporated into a generalized method of moments estimation of the model coefficients.

#### 13.2.2. Distance from covariates

Or we can compute distance from covariates directly, not using propensity score. The distance can be computed using Mahalanobis distance, or Euclidean distance, or other distance measures.

### 13.3. matching methods

With a distance, we then decide which method to use. The first few matching methods all need to specify a distance measure. They are matching based on distance. Then there are stratum matching methods, which do not need a distance measure. The last two methods are subsetting methods.

#### 13.3.1. Nearest neighbor matching

NNM is widely used; it is also known as greedy matching. It matches each treated unit to the closest control unit in terms of distance. The closest control unit is defined as the one with the smallest distance to the treated unit. If there are multiple control units with the same distance and we want a 1:1 match, one is randomly selected.

#### 13.3.2. Optimal pair matching

NNM is not optimal in the sense that it does not necessarily minimize the total distance between matched pairs. Optimal pair matching is a method that minimizes the total distance (or some other overall criterion) between matched pairs. It is more computationally intensive than NNM, but it can lead to better balance.

#### 13.3.3. Optimal full matching

OFM assigns each unit (treated or control) to a subclass. For each subclass then to calculate the sum of within-subclass distances, and then to minimize the total distance across all subclasses. It is more computationally intensive than NNM and optimal pair matching. Weights are used based on subclass membership. Because all units are included, it can be used to estimate ATE.

#### 13.3.4. Generalized Full matching

GFM is a variant of OFM. It uses a faster algorithm for large datasets.

#### 13.3.5. Genetic matching

Genetic matching (`method = "genetic"`, Diamond and Sekhon 2013) runs nearest neighbor matching on a *scaled* generalized Mahalanobis distance, where the scaling weights on the covariates are not fixed in advance but chosen by a genetic algorithm that searches for the weighting giving the best covariate balance. That search is the point of the method — and also why it is far slower than the alternatives above.

### 13.3.6. Exact matching

It's a stratum matching. First subclasses are defined, by combinations of covariate values. Then units are assigned to subclasses. When a subclass has only either control or treated units, it is discarded.

Exact matching is nonparametric, but it works only when covariates are discrete and it happens that there are enough treated and control units in each subclass.

### 13.3.7. Coarsened exact matching

CEM is a more popular stratum matching. First bins are created after coarsening the covariates. then use exact matching on the covariate bins. CEM is nonparametric. User needs to decide how many bins to create for each covariate.

### 13.3.8. Subclassification

Subclassification is another kind of stratum matching. It uses the propensity score to define bins, usually quantiles of PS in the treated group, or control group or overall, depending on target estimand.

### 13.3.9. Cardinality and profile matching

Cardinality matching involves finding the largest sample that satisfies user-supplied balance constraints and constraints on the ratio of matched treated to matched control units. Profile matching involves identifying a target distribution (e.g., the full sample for the ATE or the treated units for the ATT) and finding the largest subset of the treated and control groups that satisfy user-supplied balance constraints with respect to that target.

## 13.4. Example

Here is an example with Lalonde data, we are interested in the effect of treatment on “re78” (1978 real earnings). There are 614 observations, 185 treated and 429 control, with covariates age, education, race, marital status, degree status, and earnings in 1974 and 1975. The first `matchit` call uses `method = NULL`, which fits the propensity score but performs no matching — a way of getting the pre-match balance table.

```
library("MatchIt")
data("lalonde")

head(lalonde)
```

### 13. Matching and Weighting Part 1: matching

|      | treat | age | educ | race   | married | nodegree | re74 | re75 | re78       |
|------|-------|-----|------|--------|---------|----------|------|------|------------|
| NSW1 | 1     | 37  | 11   | black  | 1       | 1        | 0    | 0    | 9930.0460  |
| NSW2 | 1     | 22  | 9    | hispan | 0       | 1        | 0    | 0    | 3595.8940  |
| NSW3 | 1     | 30  | 12   | black  | 0       | 0        | 0    | 0    | 24909.4500 |
| NSW4 | 1     | 27  | 11   | black  | 0       | 1        | 0    | 0    | 7506.1460  |
| NSW5 | 1     | 33  | 8    | black  | 0       | 1        | 0    | 0    | 289.7899   |
| NSW6 | 1     | 22  | 9    | black  | 0       | 1        | 0    | 0    | 4056.4940  |

```
# No matching; constructing a pre-match matchit object
m.out0 <- matchit(treat ~ age + educ + race + married +
  nodegree + re74 + re75,
  data = lalonde,
  method = NULL,
  distance = "glm")
summary(m.out0)
```

Call:

```
matchit(formula = treat ~ age + educ + race + married + nodegree +
  re74 + re75, data = lalonde, method = NULL, distance = "glm")
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.5774        | 0.1822        | 1.7941          | 0.9211     | 0.3774    |
| age        | 25.8162       | 28.0303       | -0.3094         | 0.4400     | 0.0813    |
| educ       | 10.3459       | 10.2354       | 0.0550          | 0.4959     | 0.0347    |
| raceblack  | 0.8432        | 0.2028        | 1.7615          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.3498         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.8819         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.8263         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2450          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.7211         | 0.5181     | 0.2248    |
| re75       | 1532.0553     | 2466.4844     | -0.2903         | 0.9563     | 0.1342    |
| eCDF Max   |               |               |                 |            |           |
| distance   | 0.6444        |               |                 |            |           |
| age        | 0.1577        |               |                 |            |           |
| educ       | 0.1114        |               |                 |            |           |
| raceblack  | 0.6404        |               |                 |            |           |
| racehispan | 0.0827        |               |                 |            |           |
| racewhite  | 0.5577        |               |                 |            |           |
| married    | 0.3236        |               |                 |            |           |
| nodegree   | 0.1114        |               |                 |            |           |
| re74       | 0.4470        |               |                 |            |           |
| re75       | 0.2876        |               |                 |            |           |

Sample Sizes:

| Control | Treated |
|---------|---------|
|---------|---------|

|           |     |     |
|-----------|-----|-----|
| All       | 429 | 185 |
| Matched   | 429 | 185 |
| Unmatched | 0   | 0   |
| Discarded | 0   | 0   |

The imbalance is severe. Standardized mean differences are  $-0.72$  for 1974 earnings,  $1.76$  for the black indicator,  $-1.88$  for white,  $-0.83$  for married and  $-0.31$  for age, and the propensity score itself differs by 1.79 standard deviations. The treated average \$2,096 in 1974 earnings against \$5,619 for controls, and 84% are black against 20%. These are not two samples that differ at the margin.

Or use “cobalt” to see the balance:

```
library("cobalt")
bal.tab(treat ~ age + educ + race + married + nodegree + re74 + re75,
        data = lalonde, estimand = "ATT", thresholds = c(m = .05))
```

#### Balance Measures

|             | Type    | Diff.Un | M.Threshold.Un      |
|-------------|---------|---------|---------------------|
| age         | Contin. | -0.3094 | Not Balanced, >0.05 |
| educ        | Contin. | 0.0550  | Not Balanced, >0.05 |
| race_black  | Binary  | 0.6404  | Not Balanced, >0.05 |
| race_hispan | Binary  | -0.0827 | Not Balanced, >0.05 |
| race_white  | Binary  | -0.5577 | Not Balanced, >0.05 |
| married     | Binary  | -0.3236 | Not Balanced, >0.05 |
| nodegree    | Binary  | 0.1114  | Not Balanced, >0.05 |
| re74        | Contin. | -0.7211 | Not Balanced, >0.05 |
| re75        | Contin. | -0.2903 | Not Balanced, >0.05 |

#### Balance tally for mean differences

|                     | count |
|---------------------|-------|
| Balanced, <0.05     | 0     |
| Not Balanced, >0.05 | 9     |

#### Variable with the greatest mean difference

| Variable | Diff.Un | M.Threshold.Un      |
|----------|---------|---------------------|
| re74     | -0.7211 | Not Balanced, >0.05 |

#### Sample sizes

|     | Control | Treated |
|-----|---------|---------|
| All | 429     | 185     |

`bal.tab` says the same thing against a stricter 0.05 threshold: all nine covariate contrasts fail, with `re74` the worst at  $-0.7211$ .

First do a PS match. The chunks below run several distance measures in turn — the logit of a logistic propensity score, a GAM score with smooth terms in age and education, CBPS targeting the ATC with replacement, Mahalanobis distance on the raw covariates, and full matching on a

### 13. Matching and Weighting Part 1: matching

probit score. Each prints its own balance table; what changes across them is the **distance** row and how much of the covariate imbalance survives.

```
# Matching on logit of a PS estimated with logistic
# regression:
m.out1 <- matchit(treat ~ age + educ + race + married +
                  nodegree + re74 + re75,
                  data = lalonde,
                  distance = "glm",
                  link = "linear.logit")
summary(m.out1)
```

Call:

```
matchit(formula = treat ~ age + educ + race + married + nodegree +
        re74 + re75, data = lalonde, distance = "glm", link = "linear.logit")
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.2657        | -2.1500       | 2.1426          | 0.5398     | 0.3774    |
| age        | 25.8162       | 28.0303       | -0.3094         | 0.4400     | 0.0813    |
| educ       | 10.3459       | 10.2354       | 0.0550          | 0.4959     | 0.0347    |
| raceblack  | 0.8432        | 0.2028        | 1.7615          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.3498         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.8819         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.8263         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2450          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.7211         | 0.5181     | 0.2248    |
| re75       | 1532.0553     | 2466.4844     | -0.2903         | 0.9563     | 0.1342    |

eCDF Max

|            |        |
|------------|--------|
| distance   | 0.6444 |
| age        | 0.1577 |
| educ       | 0.1114 |
| raceblack  | 0.6404 |
| racehispan | 0.0827 |
| racewhite  | 0.5577 |
| married    | 0.3236 |
| nodegree   | 0.1114 |
| re74       | 0.4470 |
| re75       | 0.2876 |

Summary of Balance for Matched Data:

|           | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|-----------|---------------|---------------|-----------------|------------|-----------|
| distance  | 0.2657        | -0.7706       | 0.9192          | 0.7712     | 0.1321    |
| age       | 25.8162       | 25.3027       | 0.0718          | 0.4568     | 0.0847    |
| educ      | 10.3459       | 10.6054       | -0.1290         | 0.5721     | 0.0239    |
| raceblack | 0.8432        | 0.4703        | 1.0259          | .          | 0.3730    |



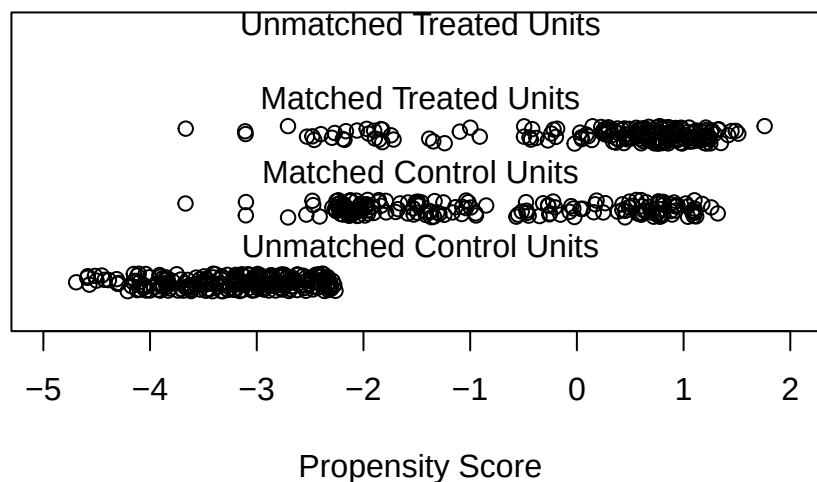
|            |           |           |         |        |        |
|------------|-----------|-----------|---------|--------|--------|
| racehispan | 0.0595    | 0.2162    | -0.6629 | .      | 0.1568 |
| racewhite  | 0.0973    | 0.3135    | -0.7296 | .      | 0.2162 |
| married    | 0.1892    | 0.2108    | -0.0552 | .      | 0.0216 |
| nodegree   | 0.7081    | 0.6378    | 0.1546  | .      | 0.0703 |
| re74       | 2095.5737 | 2342.1076 | -0.0505 | 1.3289 | 0.0469 |
| re75       | 1532.0553 | 1614.7451 | -0.0257 | 1.4956 | 0.0452 |
|            | eCDF      | Max       | Std.    | Pair   | Dist.  |
| distance   | 0.4216    |           |         | 0.9193 |        |
| age        | 0.2541    |           |         | 1.3938 |        |
| educ       | 0.0757    |           |         | 1.2474 |        |
| raceblack  | 0.3730    |           |         | 1.0259 |        |
| racehispan | 0.1568    |           |         | 1.0743 |        |
| racewhite  | 0.2162    |           |         | 0.8390 |        |
| married    | 0.0216    |           |         | 0.8281 |        |
| nodegree   | 0.0703    |           |         | 1.0106 |        |
| re74       | 0.2757    |           |         | 0.7965 |        |
| re75       | 0.2054    |           |         | 0.7381 |        |

Sample Sizes:

|           | Control | Treated |
|-----------|---------|---------|
| All       | 429     | 185     |
| Matched   | 185     | 185     |
| Unmatched | 244     | 0       |
| Discarded | 0       | 0       |

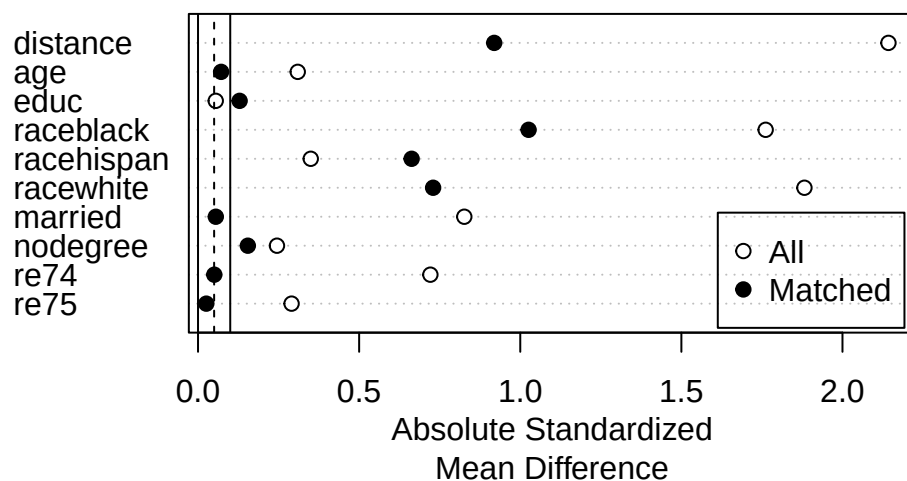
```
plot(m.out1, type = "jitter", interactive = FALSE)
```

### Distribution of Propensity Scores



### 13. Matching and Weighting Part 1: matching

```
plot(summary(m.out1))
```



We see there are still some imbalance on variables such as “raceblack”. We can use more flexible models for the PS.

```
# GAM logistic PS with smoothing splines (s()):
m.out2 <- matchit(treat ~ s(age) + s(educ) +
  race + married +
  nodegree + re74 + re75,
  data = lalonde,
  distance = "gam")
summary(m.out2)
```

Call:

```
matchit(formula = treat ~ s(age) + s(educ) + race + married +
  nodegree + re74 + re75, data = lalonde, distance = "gam")
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.6566        | 0.1481        | 1.9123          | 1.6161     | 0.4155    |
| raceblack  | 0.8432        | 0.2028        | 1.7615          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.3498         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.8819         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.8263         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2450          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.7211         | 0.5181     | 0.2248    |

|      |           |           |         |        |        |
|------|-----------|-----------|---------|--------|--------|
| re75 | 1532.0553 | 2466.4844 | -0.2903 | 0.9563 | 0.1342 |
| age  | 25.8162   | 28.0303   | -0.3094 | 0.4400 | 0.0813 |
| educ | 10.3459   | 10.2354   | 0.0550  | 0.4959 | 0.0347 |

|            | eCDF Max |
|------------|----------|
| distance   | 0.7172   |
| raceblack  | 0.6404   |
| racehispan | 0.0827   |
| racewhite  | 0.5577   |
| married    | 0.3236   |
| nodegree   | 0.1114   |
| re74       | 0.4470   |
| re75       | 0.2876   |
| age        | 0.1577   |
| educ       | 0.1114   |

## Summary of Balance for Matched Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.6566        | 0.3055        | 1.3204          | 1.2324     | 0.1853    |
| raceblack  | 0.8432        | 0.4432        | 1.1002          | .          | 0.4000    |
| racehispan | 0.0595        | 0.1730        | -0.4800         | .          | 0.1135    |
| racewhite  | 0.0973        | 0.3838        | -0.9667         | .          | 0.2865    |
| married    | 0.1892        | 0.3568        | -0.4278         | .          | 0.1676    |
| nodegree   | 0.7081        | 0.6541        | 0.1189          | .          | 0.0541    |
| re74       | 2095.5737     | 3521.7556     | -0.2919         | 1.0512     | 0.1346    |
| re75       | 1532.0553     | 2056.2000     | -0.1628         | 1.2120     | 0.0917    |
| age        | 25.8162       | 25.7081       | 0.0151          | 0.7958     | 0.0270    |
| educ       | 10.3459       | 10.1568       | 0.0941          | 0.6186     | 0.0270    |

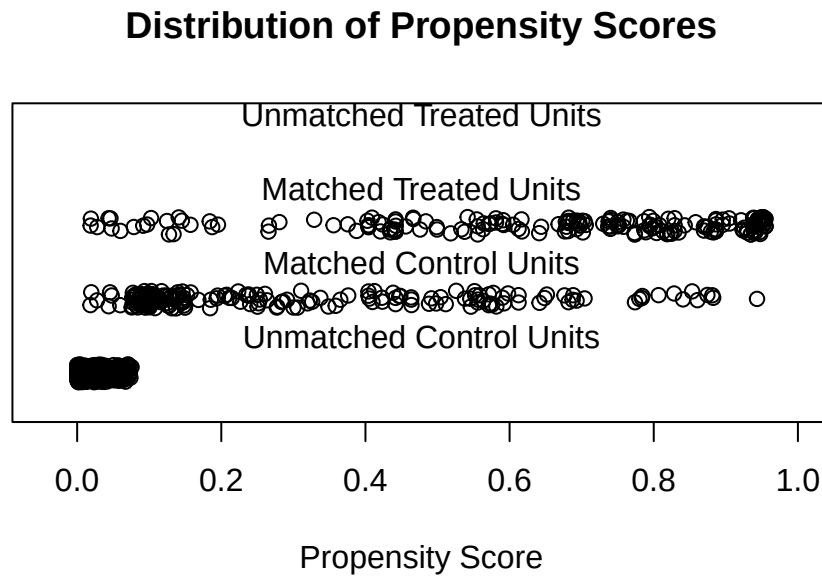
|            | eCDF Max | Std. Pair Dist. |
|------------|----------|-----------------|
| distance   | 0.5189   | 1.3204          |
| raceblack  | 0.4000   | 1.1002          |
| racehispan | 0.1135   | 0.8914          |
| racewhite  | 0.2865   | 1.1491          |
| married    | 0.1676   | 1.0627          |
| nodegree   | 0.0541   | 0.9749          |
| re74       | 0.4270   | 0.8894          |
| re75       | 0.2378   | 0.8453          |
| age        | 0.0865   | 1.1619          |
| educ       | 0.0919   | 1.2447          |

## Sample Sizes:

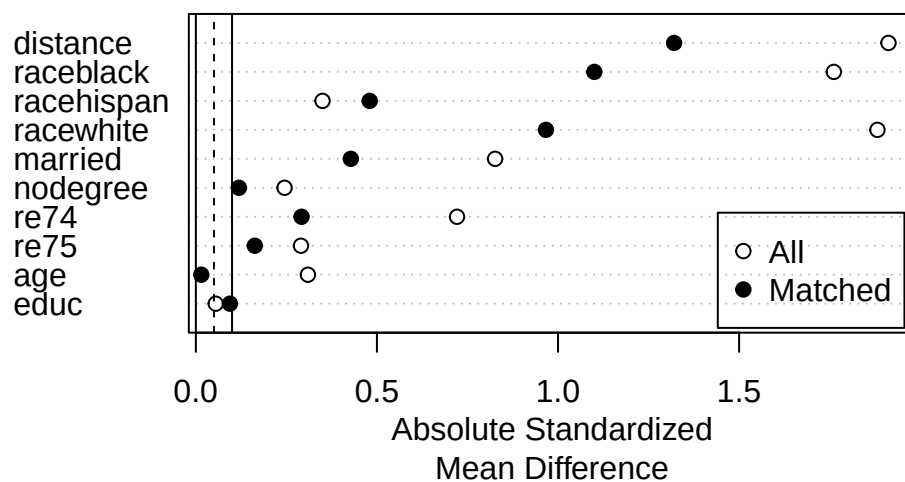
|           | Control | Treated |
|-----------|---------|---------|
| All       | 429     | 185     |
| Matched   | 185     | 185     |
| Unmatched | 244     | 0       |
| Discarded | 0       | 0       |

### 13. Matching and Weighting Part 1: matching

```
plot(m.out2, type = "jitter", interactive = FALSE)
```



```
plot(summary(m.out2))
```



Or use CBPS.

```
# CBPS for ATC matching w/replacement, using the just-
# identified version of CBPS (setting method = "exact"):
m.out3 <- matchit(treat ~ age + educ + race + married +
  nodegree + re74 + re75,
  data = lalonde,
  distance = "cbps",
  estimand = "ATC",
  distance.options = list(method = "exact"),
  replace = TRUE)
summary(m.out3)
```

Call:

```
matchit(formula = treat ~ age + educ + race + married + nodegree +
  re74 + re75, data = lalonde, distance = "cbps", distance.options = list(method = "exact"),
  estimand = "ATC", replace = TRUE)
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.2801        | 0.7758        | -1.5464         | 0.9710     | 0.3525    |
| age        | 25.8162       | 28.0303       | -0.2053         | 0.4400     | 0.0813    |
| educ       | 10.3459       | 10.2354       | 0.0387          | 0.4959     | 0.0347    |
| raceblack  | 0.8432        | 0.2028        | 1.5928          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.2369         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.1732         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.6475         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2270          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.5190         | 0.5181     | 0.2248    |
| re75       | 1532.0553     | 2466.4844     | -0.2838         | 0.9563     | 0.1342    |

|            | eCDF Max |
|------------|----------|
| distance   | 0.5967   |
| age        | 0.1577   |
| educ       | 0.1114   |
| raceblack  | 0.6404   |
| racehispan | 0.0827   |
| racewhite  | 0.5577   |
| married    | 0.3236   |
| nodegree   | 0.1114   |
| re74       | 0.4470   |
| re75       | 0.2876   |

Summary of Balance for Matched Data:

|           | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|-----------|---------------|---------------|-----------------|------------|-----------|
| distance  | 0.7720        | 0.7758        | -0.0117         | 1.1568     | 0.0499    |
| age       | 27.3520       | 28.0303       | -0.0629         | 0.4383     | 0.1091    |
| educ      | 11.0117       | 10.2354       | 0.2719          | 0.7323     | 0.0566    |
| raceblack | 0.3100        | 0.2028        | 0.2667          | .          | 0.1072    |

### 13. Matching and Weighting Part 1: matching

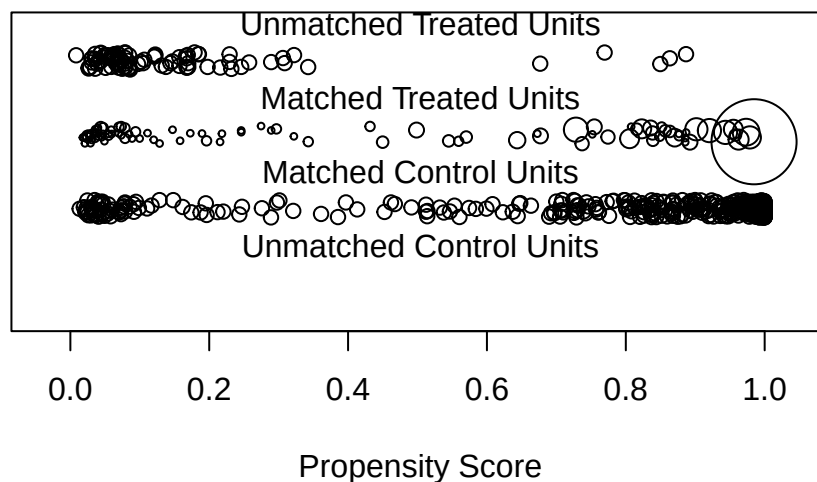
|            |           |           |         |        |        |
|------------|-----------|-----------|---------|--------|--------|
| racehispan | 0.1096    | 0.1422    | -0.0934 | .      | 0.0326 |
| racewhite  | 0.5804    | 0.6550    | -0.1569 | .      | 0.0746 |
| married    | 0.5967    | 0.5128    | 0.1679  | .      | 0.0839 |
| nodegree   | 0.4499    | 0.5967    | -0.2994 | .      | 0.1469 |
| re74       | 5696.9438 | 5619.2365 | 0.0114  | 0.8562 | 0.0994 |
| re75       | 1935.6090 | 2466.4844 | -0.1613 | 1.5194 | 0.1071 |
|            | eCDF      | Max       | Std.    | Pair   | Dist.  |
| distance   | 0.3543    |           | 0.0194  |        |        |
| age        | 0.2191    |           | 0.8676  |        |        |
| educ       | 0.2727    |           | 1.0719  |        |        |
| raceblack  | 0.1072    |           | 0.3826  |        |        |
| racehispan | 0.0326    |           | 0.6407  |        |        |
| racewhite  | 0.0746    |           | 0.5492  |        |        |
| married    | 0.0839    |           | 0.5596  |        |        |
| nodegree   | 0.1469    |           | 1.0692  |        |        |
| re74       | 0.2541    |           | 0.7513  |        |        |
| re75       | 0.2727    |           | 0.9781  |        |        |

Sample Sizes:

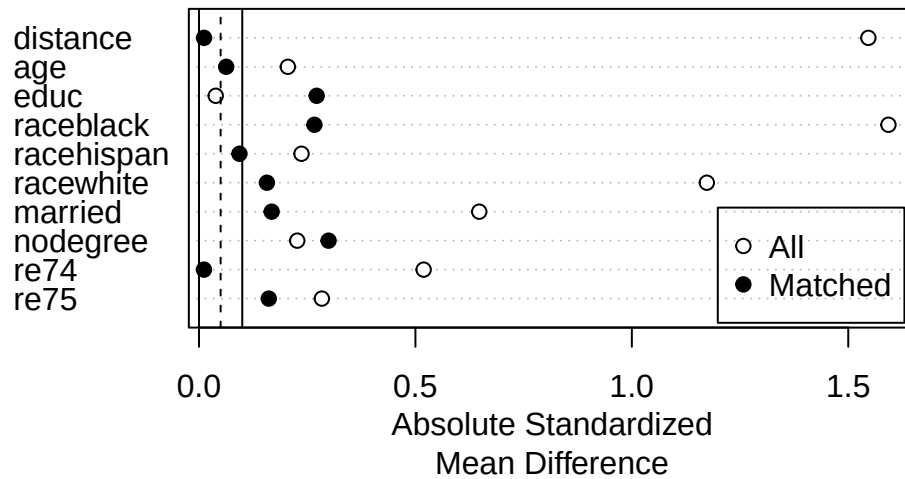
|               | Control | Treated |
|---------------|---------|---------|
| All           | 429     | 185.    |
| Matched (ESS) | 429     | 6.67    |
| Matched       | 429     | 93.     |
| Unmatched     | 0       | 92.     |
| Discarded     | 0       | 0.      |

```
plot(m.out3, type = "jitter", interactive = FALSE)
```

### Distribution of Propensity Scores



```
plot(summary(m.out3))
```



```
# Mahalanobis distance matching - no PS estimated
m.out4 <- matchit(treat ~ age + educ + race + married +
  nodegree + re74 + re75,
  data = lalonde,
  distance = "mahalanobis")
summary(m.out4)
```

Call:

```
matchit(formula = treat ~ age + educ + race + married + nodegree +
  re74 + re75, data = lalonde, distance = "mahalanobis")
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| age        | 25.8162       | 28.0303       | -0.3094         | 0.4400     | 0.0813    |
| educ       | 10.3459       | 10.2354       | 0.0550          | 0.4959     | 0.0347    |
| raceblack  | 0.8432        | 0.2028        | 1.7615          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.3498         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.8819         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.8263         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2450          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.7211         | 0.5181     | 0.2248    |
| re75       | 1532.0553     | 2466.4844     | -0.2903         | 0.9563     | 0.1342    |

eCDF Max

|     | eCDF Max |
|-----|----------|
| age | 0.1577   |

### 13. Matching and Weighting Part 1: matching

```
educ      0.1114
raceblack 0.6404
racehispan 0.0827
racewhite 0.5577
married   0.3236
nodegree  0.1114
re74      0.4470
re75      0.2876
```

#### Summary of Balance for Matched Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| age        | 25.8162       | 24.9081       | 0.1269          | 0.5327     | 0.0695    |
| educ       | 10.3459       | 10.4324       | -0.0430         | 0.8345     | 0.0108    |
| raceblack  | 0.8432        | 0.4649        | 1.0407          | .          | 0.3784    |
| racehispan | 0.0595        | 0.0595        | 0.0000          | .          | 0.0000    |
| racewhite  | 0.0973        | 0.4757        | -1.2767         | .          | 0.3784    |
| married    | 0.1892        | 0.2486        | -0.1518         | .          | 0.0595    |
| nodegree   | 0.7081        | 0.6595        | 0.1070          | .          | 0.0486    |
| re74       | 2095.5737     | 3305.4420     | -0.2476         | 0.8705     | 0.1012    |
| re75       | 1532.0553     | 1957.5097     | -0.1322         | 1.0071     | 0.0654    |

|            | eCDF Max | Std. Pair Dist. |
|------------|----------|-----------------|
| age        | 0.2378   | 0.6195          |
| educ       | 0.0486   | 0.2903          |
| raceblack  | 0.3784   | 1.0407          |
| racehispan | 0.0000   | 0.0000          |
| racewhite  | 0.3784   | 1.2767          |
| married    | 0.0595   | 0.1794          |
| nodegree   | 0.0486   | 0.1070          |
| re74       | 0.3351   | 0.4546          |
| re75       | 0.2162   | 0.4113          |

#### Sample Sizes:

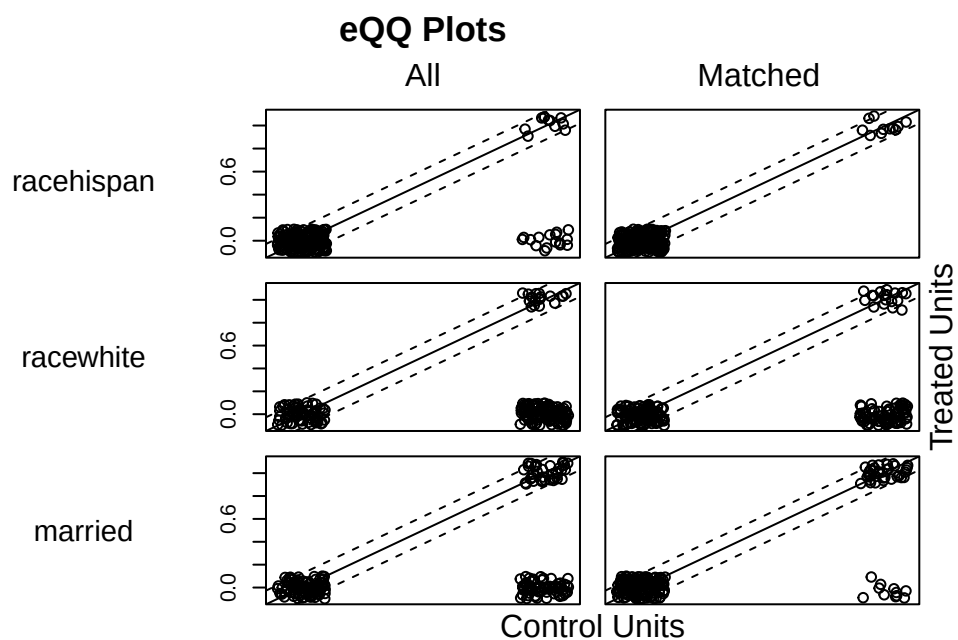
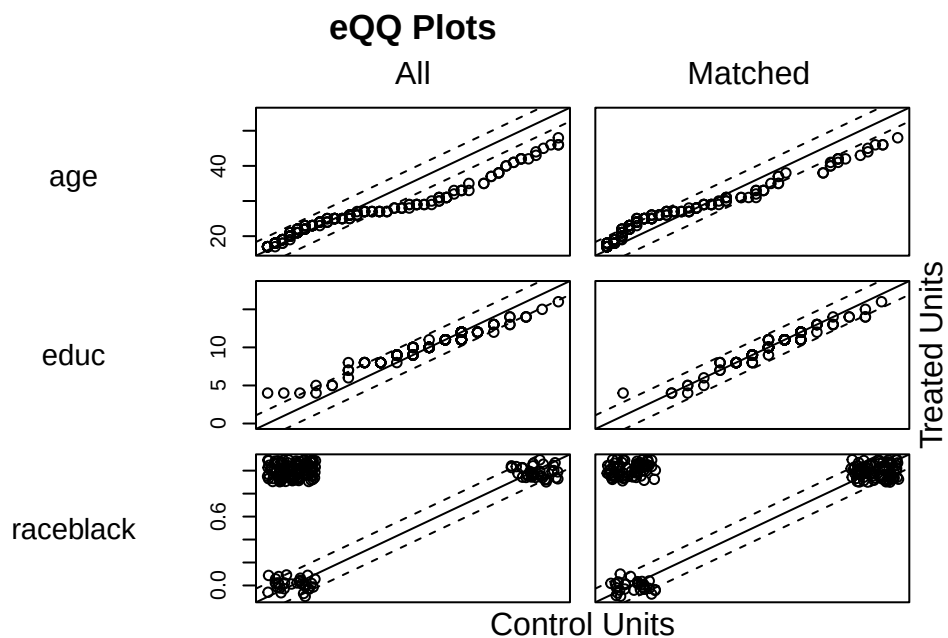
|           | Control | Treated |
|-----------|---------|---------|
| All       | 429     | 185     |
| Matched   | 185     | 185     |
| Unmatched | 244     | 0       |
| Discarded | 0       | 0       |

```
m.out4$distance #NULL
```

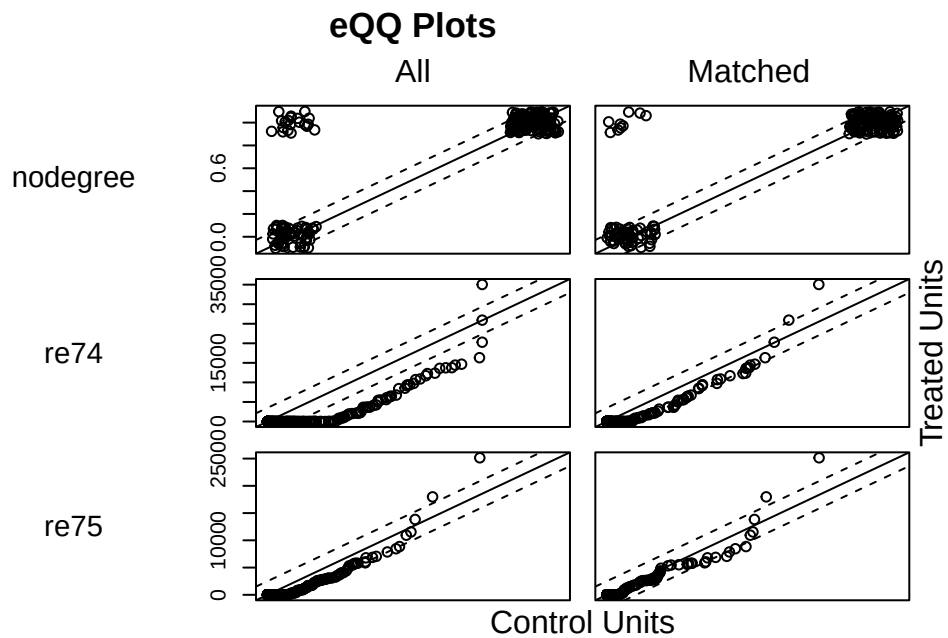
```
NULL
```

```
plot(m.out4, interactive = FALSE)
```

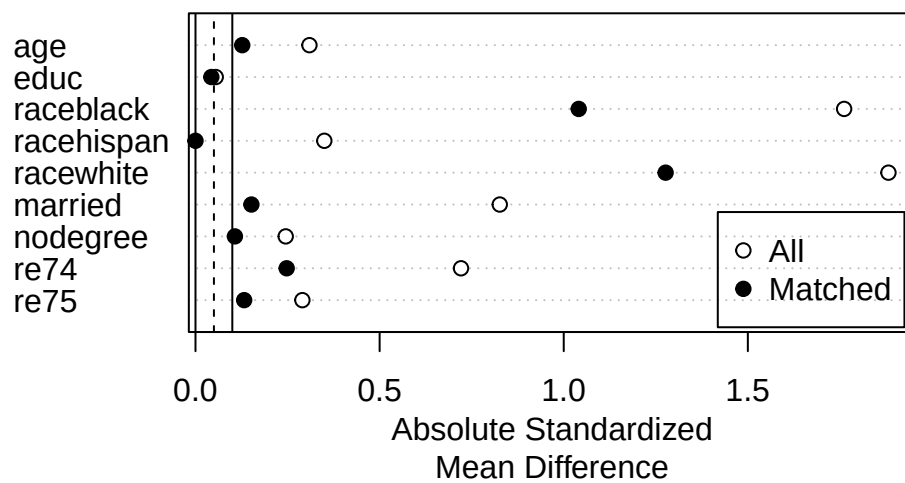




### 13. Matching and Weighting Part 1: matching



```
plot(summary(m.out4))
```



```
# Full matching on a probit PS
m.out5 <- matchit(treat ~ age + educ + race + married +
  nodegree + re74 + re75,
  data = lalonde,
  method = "full",
  distance = "glm",
```

```
link = "probit")

summary(m.out5)
```

Call:

```
matchit(formula = treat ~ age + educ + race + married + nodegree +
        re74 + re75, data = lalonde, method = "full", distance = "glm",
        link = "probit")
```

Summary of Balance for All Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.5773        | 0.1817        | 1.8276          | 0.8777     | 0.3774    |
| age        | 25.8162       | 28.0303       | -0.3094         | 0.4400     | 0.0813    |
| educ       | 10.3459       | 10.2354       | 0.0550          | 0.4959     | 0.0347    |
| raceblack  | 0.8432        | 0.2028        | 1.7615          | .          | 0.6404    |
| racehispan | 0.0595        | 0.1422        | -0.3498         | .          | 0.0827    |
| racewhite  | 0.0973        | 0.6550        | -1.8819         | .          | 0.5577    |
| married    | 0.1892        | 0.5128        | -0.8263         | .          | 0.3236    |
| nodegree   | 0.7081        | 0.5967        | 0.2450          | .          | 0.1114    |
| re74       | 2095.5737     | 5619.2365     | -0.7211         | 0.5181     | 0.2248    |
| re75       | 1532.0553     | 2466.4844     | -0.2903         | 0.9563     | 0.1342    |

|            | eCDF Max |
|------------|----------|
| distance   | 0.6413   |
| age        | 0.1577   |
| educ       | 0.1114   |
| raceblack  | 0.6404   |
| racehispan | 0.0827   |
| racewhite  | 0.5577   |
| married    | 0.3236   |
| nodegree   | 0.1114   |
| re74       | 0.4470   |
| re75       | 0.2876   |

Summary of Balance for Matched Data:

|            | Means Treated | Means Control | Std. Mean Diff. | Var. Ratio | eCDF Mean |
|------------|---------------|---------------|-----------------|------------|-----------|
| distance   | 0.5773        | 0.5764        | 0.0045          | 0.9949     | 0.0043    |
| age        | 25.8162       | 25.5347       | 0.0393          | 0.4790     | 0.0787    |
| educ       | 10.3459       | 10.5381       | -0.0956         | 0.6192     | 0.0253    |
| raceblack  | 0.8432        | 0.8389        | 0.0119          | .          | 0.0043    |
| racehispan | 0.0595        | 0.0492        | 0.0435          | .          | 0.0103    |
| racewhite  | 0.0973        | 0.1119        | -0.0493         | .          | 0.0146    |
| married    | 0.1892        | 0.1633        | 0.0660          | .          | 0.0259    |
| nodegree   | 0.7081        | 0.6577        | 0.1110          | .          | 0.0504    |
| re74       | 2095.5737     | 2100.2150     | -0.0009         | 1.3467     | 0.0314    |
| re75       | 1532.0553     | 1561.4420     | -0.0091         | 1.5906     | 0.0536    |

|            | eCDF Max | Std. Pair Dist. |
|------------|----------|-----------------|
| distance   | 0.6413   | 0.0043          |
| age        | 0.1577   | 0.0787          |
| educ       | 0.1114   | 0.0253          |
| raceblack  | 0.6404   | 0.0043          |
| racehispan | 0.0827   | 0.0103          |
| racewhite  | 0.5577   | 0.0146          |
| married    | 0.3236   | 0.0259          |
| nodegree   | 0.1114   | 0.0504          |
| re74       | 0.4470   | 0.0314          |
| re75       | 0.2876   | 0.0536          |

### 13. Matching and Weighting Part 1: matching

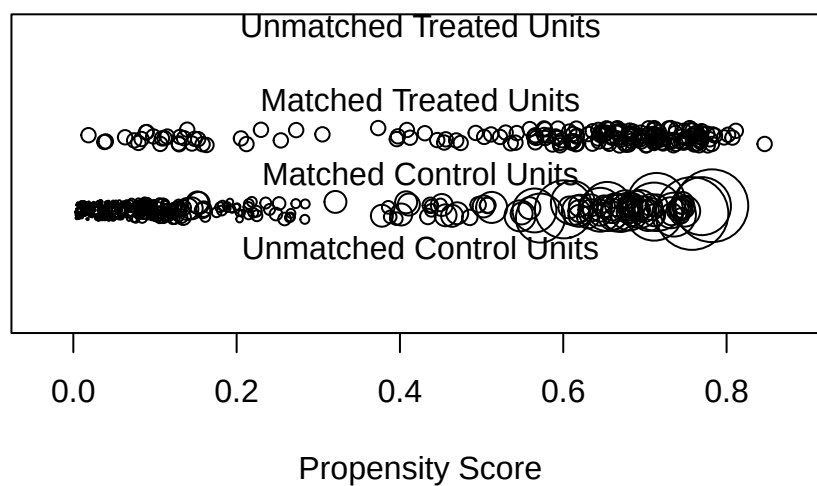
|            |        |        |
|------------|--------|--------|
| distance   | 0.0486 | 0.0198 |
| age        | 0.2742 | 1.2843 |
| educ       | 0.0730 | 1.2179 |
| raceblack  | 0.0043 | 0.0162 |
| racehispan | 0.0103 | 0.4412 |
| racewhite  | 0.0146 | 0.3454 |
| married    | 0.0259 | 0.4473 |
| nodegree   | 0.0504 | 0.9872 |
| re74       | 0.1881 | 0.8387 |
| re75       | 0.1984 | 0.8240 |

Sample Sizes:

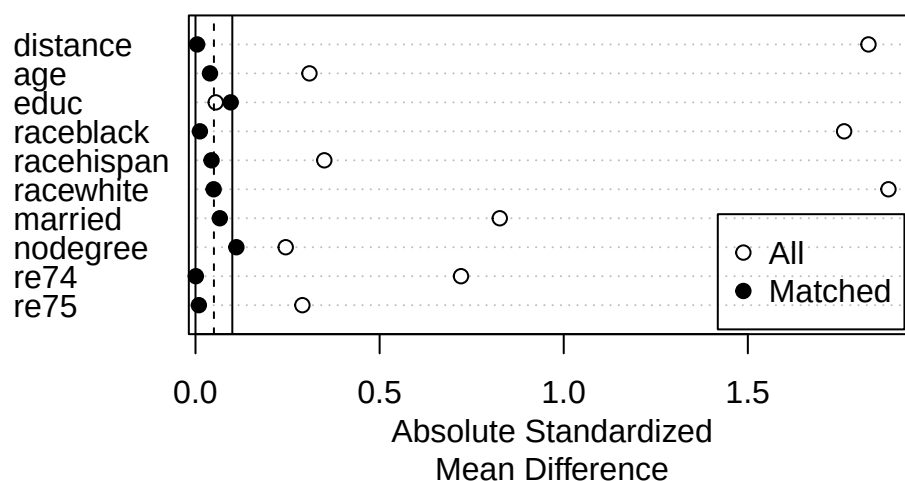
|               | Control | Treated |
|---------------|---------|---------|
| All           | 429.    | 185     |
| Matched (ESS) | 50.76   | 185     |
| Matched       | 429.    | 185     |
| Unmatched     | 0.      | 0       |
| Discarded     | 0.      | 0       |

```
plot(m.out5, type = "jitter", interactive = FALSE)
```

### Distribution of Propensity Scores



```
plot(summary(m.out5))
```



Note that `m.out4$distance` prints `NULL`: Mahalanobis matching estimates no propensity score at all, so there is no `distance` column and no `distance` row in its balance table. That is the point of the method — it works on the covariate space directly rather than collapsing it to a scalar.

## 13.5. Estimation

After we get a matched data that we think is good enough, we can estimate the treatment effect. We use `m.out5`, the full match on a probit score.

```
m.data <- match_data(m.out5)
```

```
head(m.data)
```

|      | treat | age | educ | race   | married | nodegree | re74 | re75 | re78       | distance  |
|------|-------|-----|------|--------|---------|----------|------|------|------------|-----------|
| NSW1 | 1     | 37  | 11   | black  | 1       | 1        | 0    | 0    | 9930.0460  | 0.6356769 |
| NSW2 | 1     | 22  | 9    | hispan | 0       | 1        | 0    | 0    | 3595.8940  | 0.2298151 |
| NSW3 | 1     | 30  | 12   | black  | 0       | 0        | 0    | 0    | 24909.4500 | 0.6813558 |
| NSW4 | 1     | 27  | 11   | black  | 0       | 1        | 0    | 0    | 7506.1460  | 0.7690590 |
| NSW5 | 1     | 33  | 8    | black  | 0       | 1        | 0    | 0    | 289.7899   | 0.6954138 |
| NSW6 | 1     | 22  | 9    | black  | 0       | 1        | 0    | 0    | 4056.4940  | 0.6943658 |

|      | weights | subclass |
|------|---------|----------|
| NSW1 | 1       | 1        |
| NSW2 | 1       | 55       |
| NSW3 | 1       | 63       |
| NSW4 | 1       | 70       |
| NSW5 | 1       | 79       |
| NSW6 | 1       | 86       |

### 13. Matching and Weighting Part 1: matching

What “match\_data” does is to grab the matched data with a few additional variables such as “weights”, “subclass”, and “distance”. Then we can use it in a simple linear model with weights.

```
library("marginaleffects")

fit <- lm(re78 ~ treat * (age + educ + race + married +
                        nodegree + re74 + re75),
        data = m.data,
        weights = weights)

avg_comparisons(fit,
                variables = "treat",
                vcov = ~subclass,
                newdata = subset(m.data, treat == 1))
```

| Estimate | Std. Error | z    | Pr(> z ) | S   | 2.5 % | 97.5 % |
|----------|------------|------|----------|-----|-------|--------|
| 1977     | 704        | 2.81 | 0.00501  | 7.6 | 596   | 3357   |

Term: treat  
Type: response  
Comparison: 1 - 0

The ATT is \$1,977 with a subclass-clustered standard error of \$704,  $p = 0.005$ , and a 95% interval of [596, 3357]. Training raised 1978 earnings for those who took it.

Note we allow interaction of treatment and covariates, but we do not need to demean the covariates here, since we are relying on “marginaleffects” and its “avg\_comparisons” function to compute the average treatment effect. The “newdata” argument is used to specify the effect, in this case ATT. Note the weights are generated this way: since it’s ATT, all treated units are weighted by 1, and control units are weighted by  $p/(1 - p)$ , where  $p$  is proportion of treated units in the subclass. Anyway, for full matching, we get the comparison of potential outcomes for the treated, with weights, and clustered standard errors on subclasses, because that the level the comparison is conducted.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 14. Matching and Weighting - Part 2: weighting

This chapter follows R's `WeightIt` package (Greifer et al.). The assumptions — SUTVA, ignorability, and overlap — are the same as in the [matching chapter](#).

### 14.1. Weighting methods

Weighting creates a pseudo-population in which  $X$  is balanced across treatment and control. The most common approach is inverse probability of treatment weighting (IPTW), which uses the propensity score. Several alternatives are gaining traction.

#### 14.1.1. Inverse probability of treatment weighting (IPTW)

IPTW is the most common weighting method. It uses the inverse of the propensity score as weights. For the **ATE**, weight each treated unit by  $1/p$  and each control unit by  $1/(1 - p)$ , where  $p$  is the propensity score. For the **ATT** — which is what the examples below request via `estimand = "ATT"` — the treated units all get weight 1 and the controls get  $p/(1 - p)$ ; you can see this in the weight summaries, where the treated range is exactly 1 to 1. Either way the point is the same: create a pseudo population in which the distribution of  $X$  is balanced between the groups.

#### 14.1.2. Covariate balancing propensity score (CBPS)

The same idea, but with a different way to estimate the propensity score. CBPS is a form of logistic regression where balance constraints are incorporated to a generalized method of moments estimation of the model coefficients.

#### 14.1.3. Entropy balancing

Entropy balancing involves the specification of an optimization problem, the solution to which is then used to compute the weights. The constraints of the primal optimization problem correspond to covariate balance on the means (for binary and multi-category treatments) or treatment-covariate covariances (for continuous treatments), positivity of the weights, and that the weights sum to a certain value. Entropy balancing is doubly robust (for the ATT) in the sense that it is consistent either when the true propensity score model is a logistic regression of the treatment on the covariates or when the true outcome model for the control units is a linear regression of the outcome on the covariates, and it attains a semi-parametric efficiency bound when both are true. Entropy balancing will always yield exact mean balance on the included terms (not necessarily on the distribution of the covariates).

#### 14.1.4. Energy balancing

Energy balancing is a method of estimating weights using optimization without a propensity score.

The primary benefit of energy balancing is that all features of the covariate distribution are balanced, not just means, as with other optimization-based methods like entropy balancing.

#### 14.1.5. SuperLearner

SuperLearner works by fitting several machine learning models to the treatment and covariates and then taking a weighted combination of the generated predicted values to use as the propensity scores, which are then used to construct weights.

### 14.2. Example

Here is an example with Lalonde data, we are interested in the effect of treatment on “re78” (real earnings 1978).

#### 14.2.1. IPW

```
library("WeightIt")
library(cobalt)
data(lalonde)
w.out1 <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, estimand = "ATT", method = "glm")
w.out1 #print the output
```

A weightit object

- method: "glm" (propensity score weighting with GLM)
- number of obs.: 614
- sampling weights: none
- treatment: 2-category
- estimand: ATT (focal: 1)
- covariates: age, educ, race, married, nodegree, re74, re75

```
summary(w.out1)
```

Summary of weights

- Weight ranges:

| Min        | Max |
|------------|-----|
| treated 1. | 1.  |



```
control 0.009 |-----| 3.743
```

- Units with the 5 most extreme weights by group:

```

      5      4      3      2      1
treated  1      1      1      1      1
      597    573    381    411    303
control 3.03 3.059 3.24 3.523 3.743
```

- Weight statistics:

```

      Coef of Var  MAD Entropy # Zeros
treated         0.    0.      0.        0
control        1.818 1.289   1.098      0
```

- Effective Sample Sizes:

```

      Control Treated
Unweighted  429.      185
Weighted    99.82    185
```

The treated weights are all exactly 1, as the ATT requires, and the control weights run from 0.009 to 3.743. The cost is in the effective sample size: 429 controls carry the information of 99.8 equally weighted ones, a loss of more than three quarters.

```
library(cobalt)
bal.tab(w.out1, stats = c("m", "v"), thresholds = c(m = .05))
```

Balance Measures

|             | Type     | Diff.Adj | M.Threshold         | V.Ratio.Adj |
|-------------|----------|----------|---------------------|-------------|
| prop.score  | Distance | -0.0205  | Balanced, <0.05     | 1.0324      |
| age         | Contin.  | 0.1188   | Not Balanced, >0.05 | 0.4578      |
| educ        | Contin.  | -0.0284  | Balanced, <0.05     | 0.6636      |
| race_black  | Binary   | -0.0022  | Balanced, <0.05     | .           |
| race_hispan | Binary   | 0.0002   | Balanced, <0.05     | .           |
| race_white  | Binary   | 0.0021   | Balanced, <0.05     | .           |
| married     | Binary   | 0.0186   | Balanced, <0.05     | .           |
| nodegree    | Binary   | 0.0184   | Balanced, <0.05     | .           |
| re74        | Contin.  | -0.0021  | Balanced, <0.05     | 1.3206      |
| re75        | Contin.  | 0.0110   | Balanced, <0.05     | 1.3938      |

Balance tally for mean differences

```

      count
Balanced, <0.05      9
Not Balanced, >0.05   1
```

Variable with the greatest mean difference

## 14. Matching and Weighting - Part 2: weighting

| Variable | Diff.Adj | M.Threshold         |
|----------|----------|---------------------|
| age      | 0.1188   | Not Balanced, >0.05 |

|            | Control | Treated |
|------------|---------|---------|
| Unadjusted | 429.    | 185     |
| Adjusted   | 99.82   | 185     |

Nine of ten contrasts now pass the strict 0.05 threshold, against none before weighting. `age` is the exception at 0.1188. Note also the variance ratios, which IPTW does not target: `age` sits at 0.458 and `re74` at 1.321, so the *means* are balanced while the *spreads* are not.

### 14.2.2. entropy balancing

```
w.out2 <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, estimand = "ATT", method = "ebal")
summary(w.out2)
```

#### Summary of weights

- Weight ranges:

|         | Min   | Max   |
|---------|-------|-------|
| treated | 1.    | 1.    |
| control | 0.008 | 4.062 |

- Units with the 5 most extreme weights by group:

|         | 5     | 4     | 3    | 2     | 1     |
|---------|-------|-------|------|-------|-------|
| treated | 1     | 1     | 1    | 1     | 1     |
|         | 608   | 381   | 597  | 303   | 411   |
| control | 3.073 | 3.235 | 3.45 | 3.897 | 4.062 |

- Weight statistics:

|         | Coef of Var | MAD   | Entropy | # Zeros |
|---------|-------------|-------|---------|---------|
| treated | 0.          | 0.    | 0.      | 0       |
| control | 1.834       | 1.287 | 1.101   | 0       |

- Effective Sample Sizes:

|            | Control | Treated |
|------------|---------|---------|
| Unweighted | 429.    | 185     |
| Weighted   | 98.46   | 185     |

```
bal.tab(w.out2, stats = c("m", "v"), thresholds = c(m = .05))
```

#### Balance Measures

|             | Type    | Diff.Adj | M.Threshold     | V.Ratio.Adj |
|-------------|---------|----------|-----------------|-------------|
| age         | Contin. | -0       | Balanced, <0.05 | 0.4096      |
| educ        | Contin. | -0       | Balanced, <0.05 | 0.6635      |
| race_black  | Binary  | 0        | Balanced, <0.05 | .           |
| race_hispan | Binary  | 0        | Balanced, <0.05 | .           |
| race_white  | Binary  | -0       | Balanced, <0.05 | .           |
| married     | Binary  | -0       | Balanced, <0.05 | .           |
| nodegree    | Binary  | 0        | Balanced, <0.05 | .           |
| re74        | Contin. | -0       | Balanced, <0.05 | 1.3265      |
| re75        | Contin. | -0       | Balanced, <0.05 | 1.3351      |

#### Balance tally for mean differences

|                     | count |
|---------------------|-------|
| Balanced, <0.05     | 9     |
| Not Balanced, >0.05 | 0     |

#### Variable with the greatest mean difference

| Variable | Diff.Adj | M.Threshold     |
|----------|----------|-----------------|
| re74     | -0       | Balanced, <0.05 |

#### Effective sample sizes

|            | Control | Treated |
|------------|---------|---------|
| Unadjusted | 429.    | 185     |
| Adjusted   | 98.46   | 185     |

Every mean difference is now zero to three decimals — that is the constraint entropy balancing imposes directly, rather than hoping a propensity model delivers it — and all nine pass. The effective sample size is 98.46, essentially the same as IPTW's 99.82, so the exact balance came at no extra cost in precision here. The variance ratios are unchanged in character (age 0.410, re74 1.327): entropy balancing constrains means, not distributions.

Suppose we are satisfied with the balance, we can use the weights to estimate the treatment effect. We can use the “marginaleffects” package to do that.

```
# Fit outcome model
fit <- lm_weightit(re78 ~ treat * (age + educ + race + married +
                                nodegree + re74 + re75),
                  data = lalonde, weightit = w.out2)
# G-computation for the treatment effect
library("marginaleffects")
avg_comparisons(fit, variables = "treat",
                 newdata = subset(lalonde, treat == 1))
```

## 14. Matching and Weighting - Part 2: weighting

| Estimate | Std. Error | z    | Pr(> z ) | S   | 2.5 % | 97.5 % |
|----------|------------|------|----------|-----|-------|--------|
| 1273     | 770        | 1.65 | 0.0983   | 3.3 | -236  | 2783   |

Term: treat

Type: probs

Comparison: 1 - 0

The entropy-balanced ATT is \$1,273 with a standard error of \$770,  $p = 0.098$ , and a 95% interval of  $[-236, 2783]$  that includes zero.

### 14.2.3. energy balancing

```
w.out3 <- tryCatch(  
  weightit(treat ~ age + educ + race + married +  
    nodegree + re74 + re75, data = lalonde,  
    method = "energy", estimand = "ATE"),  
  error = function(e) { message("Energy balancing failed: ", e$message); NULL }  
)  
if (!is.null(w.out3)) summary(w.out3)
```

#### Summary of weights

- Weight ranges:

|         | Min      | Max    |
|---------|----------|--------|
| treated | 0  ----- | 14.041 |
| control | 0  ----- | 11.065 |

- Units with the 5 most extreme weights by group:

|         |       |       |       |        |        |
|---------|-------|-------|-------|--------|--------|
|         | 10    | 179   | 125   | 137    | 124    |
| treated | 5.984 | 6.23  | 6.758 | 13.202 | 14.041 |
|         | 537   | 553   | 573   | 420    | 608    |
| control | 5.499 | 6.404 | 6.599 | 8.274  | 11.065 |

- Weight statistics:

|         | Coef of Var | MAD   | Entropy | # Zeros |
|---------|-------------|-------|---------|---------|
| treated | 1.821       | 1.054 | 0.92    | 0       |
| control | 1.264       | 0.866 | 0.579   | 0       |

- Effective Sample Sizes:

| Control | Treated |
|---------|---------|
|---------|---------|

```
Unweighted  429.    185.
Weighted    165.42  43.04
```

```
if (!is.null(w.out3)) {
  fit <- lm_weightit(re78 ~ treat * (age + educ + race + married +
                                   nodegree + re74 + re75),
                    data = lalonde, weightit = w.out3)
  # G-computation for the treatment effect
  avg_comparisons(fit, variables = "treat")
}
```

```
Estimate Std. Error      z Pr(>|z|)    S 2.5 % 97.5 %
      -73      1085 -0.0672    0.946 0.1 -2200   2054
```

```
Term: treat
Type: probs
Comparison: 1 - 0
```

Two things are worth noticing in that output rather than passing over it. First, the estimand changed: `w.out3` requests the **ATE**, while the entropy-balancing fit above targeted the **ATT**, so the two numbers ( $-73$  against  $1273$ ) are not competing estimates of the same quantity. Second, look at the weight summary. Entropy balancing gives every treated unit a weight of exactly 1, as the ATT requires. Energy balancing spreads the treated weights from 0 to 14.0 — one treated unit is effectively discarded and another counts as fourteen — and the resulting standard error (1085) is much larger than entropy balancing’s (770). Balancing the whole covariate *distribution* rather than just its means is a genuine advantage, but it is bought with weight variability, and that shows up directly in the precision of the estimate.

## 14.3. comparing matching and weighting

Matching and weighting are popular in different fields. We used to say matching involves discarding data therefore changing the estimand, but now with full matching and other methods, matching can be done without losing data. We used to believe weighting can be unstable, as weights can be highly variable, but with CBPS, entropy balancing and energy balancing, weighting can perform much better.

So, in practice, we can try different methods, either matching or weighting, and see which one gives us better balance. Then use “`marginalEffects`” to get the treatment effect.

It is worth putting numbers on that, because on this data the choice changes the conclusion. Full matching in the [previous chapter](#) gave an ATT of \$1,977 with a standard error of \$704 and  $p = 0.005$ . Entropy balancing above gives \$1,273 with \$770 and  $p = 0.098$ . The two point estimates are about one standard error apart, and one interval excludes zero while the other does not.

Note which way that cuts. Entropy balancing achieves *exact* mean balance, while full matching leaves `educ` at  $-0.096$ ; the better-balanced method gives the smaller and less significant estimate.

#### 14. Matching and Weighting - Part 2: weighting

Better balance on means is not the same thing as a more credible answer, and reporting whichever method happened to cross the significance line would be the wrong response to this. The honest summary reports both and says the data do not settle it.

---

*Systematic treatment:* *R* · *Julia*.

# 15. Sensitivity analysis of OLS with sensemakr

## 15.1. OVB

Consider a true model,

$$Y = \delta D + X\beta + \gamma Z + \epsilon \quad (15.1)$$

If  $Z$  is unobserved we estimate

$$Y = \delta D + X\beta + u \quad (15.2)$$

and face omitted variable bias (OVB). Cinelli and Hazlett (2020) show that

$$|\widehat{\text{bias}}| = \sqrt{\frac{R_{Y \sim Z|D,X}^2 R_{D \sim Z|X}^2}{1 - R_{D \sim Z|X}^2}} \left( \frac{\hat{\sigma}_{y.dx}}{\hat{\sigma}_{d.x}} \right) \quad (15.3)$$

Bias is large only when the omitted variable is strongly associated with both  $Y$  and  $D$ . The Cinelli and Hazlett framework yields three key quantities.

## 15.2. Partial $R^2$ of treatment on outcome

This measures how much variation in the outcome the treatment explains after partialling out  $X$ . In the extreme case where the confounder explains all remaining outcome variance ( $R_{Y \sim Z|D,X}^2 = 1$ ), the treatment effect goes to zero only if  $R_{D \sim Z|X}^2 \geq R_{Y \sim D|X}^2$ . In other words, the confounder's association with treatment must exceed the treatment's partial association with the outcome.

## 15.3. Robust value

$RV_{q,\alpha}$  is the minimum strength of association the confounder must have with both treatment and outcome for a  $(1 - \alpha)$  confidence interval to include a  $q\%$  change from the current estimate. With  $q = 1$  this gives the threshold for the effect to include zero. Both the  $Z$ - $D$  and  $Z$ - $Y$  associations must exceed this threshold simultaneously.

## 15.4. Bounds using observed covariate(s)

The first two quantities are dimensionless but hard to interpret without context. Benchmarking against an observed covariate helps. If a covariate  $X_j$  is known to be an important predictor, we can ask how much stronger the confounder must be relative to  $X_j$  to drive the effect to zero.

$$k_D := \frac{R_{D \sim Z|X_{-j}}^2}{R_{D \sim X_j|X_{-j}}^2} \quad (15.4)$$

where  $X_{-j}$  represents all covariates other than the  $j$ th we pick as the benchmark covariate.

$$k_Y := \frac{R_{Y \sim Z|D, X_{-j}}^2}{R_{Y \sim X_j|D, X_{-j}}^2} \quad (15.5)$$

## 15.5. Example

Here is an example from sensemakr documentation.

```
# loads package
library(sensemakr)

# loads dataset
data("darfur")

# runs regression model
model <- lm(peacefactor ~ directlyharmed + age + farmer_dar + herder_dar +
           pastvoted + hhsize_darfur + female + village, data = darfur)

# runs sensemakr for sensitivity analysis
sensitivity <- sensemakr(model = model,
                        treatment = "directlyharmed",
                        benchmark_covariates = "female",
                        kd = 1:3)

# short description of results
sensitivity
```

Sensitivity Analysis to Unobserved Confounding

Model Formula: peacefactor ~ directlyharmed + age + farmer\_dar + herder\_dar +  
pastvoted + hhsize\_darfur + female + village

Null hypothesis:  $q = 1$  and reduce = TRUE



Unadjusted Estimates of 'directlyharmed':

Coef. estimate: 0.09732  
 Standard Error: 0.02326  
 t-value: 4.18445

Sensitivity Statistics:

Partial R2 of treatment with outcome: 0.02187  
 Robustness Value,  $q = 1$  : 0.13878  
 Robustness Value,  $q = 1$  alpha = 0.05 : 0.07626

For more information, check summary.

```
# long description of results
summary(sensitivity)
```

Sensitivity Analysis to Unobserved Confounding

Model Formula: `peacefactor ~ directlyharmed + age + farmer_dar + herder_dar +  
 pastvoted + hhsiz_darfur + female + village`

Null hypothesis:  $q = 1$  and `reduce = TRUE`

-- This means we are considering biases that reduce the absolute value of the current estimate  
 -- The null hypothesis deemed problematic is  $H_0: \tau = 0$

Unadjusted Estimates of 'directlyharmed':

Coef. estimate: 0.0973  
 Standard Error: 0.0233  
 t-value ( $H_0: \tau = 0$ ): 4.1844

Sensitivity Statistics:

Partial R2 of treatment with outcome: 0.0219  
 Robustness Value,  $q = 1$ : 0.1388  
 Robustness Value,  $q = 1$ , alpha = 0.05: 0.0763

Verbal interpretation of sensitivity statistics:

-- Partial R2 of the treatment with the outcome: an extreme confounder (orthogonal to the covariates) that explains 2.19% of the variance in the outcome  
 -- Robustness Value,  $q = 1$ : unobserved confounders (orthogonal to the covariates) that explain 13.88% of the variance in the outcome  
 -- Robustness Value,  $q = 1$ , alpha = 0.05: unobserved confounders (orthogonal to the covariates) that explain 7.63% of the variance in the outcome

Bounds on omitted variable bias:

--The table below shows the maximum strength of unobserved confounders with association with the treatment and the outcome

| Bound | Label | R2dz.x | R2yz.dx | Treatment | Adjusted Estimate | Adjusted Se |
|-------|-------|--------|---------|-----------|-------------------|-------------|
|-------|-------|--------|---------|-----------|-------------------|-------------|

## 15. Sensitivity analysis of OLS with sensemaker

|            |          |          |                |          |        |
|------------|----------|----------|----------------|----------|--------|
| 1x female  | 0.0092   | 0.1246   | directlyharmed | 0.0752   | 0.0219 |
| 2x female  | 0.0183   | 0.2493   | directlyharmed | 0.0529   | 0.0204 |
| 3x female  | 0.0275   | 0.3741   | directlyharmed | 0.0304   | 0.0187 |
| Adjusted T | Adjusted | Lower CI | Adjusted       | Upper CI |        |
| 3.4389     |          | 0.0323   |                | 0.1182   |        |
| 2.6002     |          | 0.0130   |                | 0.0929   |        |
| 1.6281     |          | -0.0063  |                | 0.0670   |        |

```
# minimal reporting
ovb_minimal_reporting(sensitivity, format = "html")
```

Outcome: peacefactor

Treatment

Est.

S.E.

t-value

$R^2_{Y \sim D | \mathbf{X}}$

$RV_{q=1}$

$RV_{q=1, \alpha=0.05}$

directlyharmed

0.097

0.023

4.184

2.2%

13.9%

7.6%

Note:  $df = 783$ ; Bound ( 1x female ):  $R^2_{Y \sim Z | \mathbf{X}, D} = 12.5\%$ ,  $R^2_{D \sim Z | \mathbf{X}} = 0.9\%$

Let's see how to interpret this.

$RV_{q=1} = 13.9\%$ . This means that unobserved confounders that explain 13.9% of the residual variance both of the treatment and of the outcome are sufficiently strong to explain away all the observed effect. On the other hand, unobserved confounders that do not explain at least 13.9% of the residual variance both of the treatment and of the outcome are not sufficiently strong to do so.

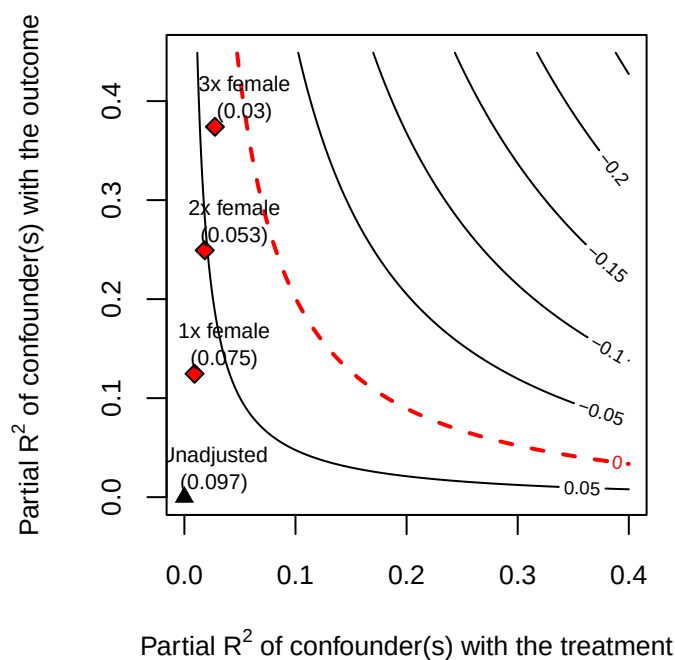
The robustness value for testing the null hypothesis that the coefficient of “directlyharmed” is zero ( $RV_{q=1, \alpha=0.05}$ ) is 7.6%. This means that unobserved confounders that explain 7.6% of the residual variance both of the treatment and of the outcome are sufficiently strong to bring the lower bound of the confidence interval to zero (at the chosen significance level of 5%).

The partial  $R^2$  of `directlyharmed` with `peacefactor` is 2.2%. This means that, in an extreme scenario in which we assume that unobserved confounders explain all of the left-out variance of the outcome, these unobserved confounders would need to explain at least 2.2% of the residual variance of the treatment to fully explain away the observed effect.

The above numbers are for the absolute strength the confounder needs to be to explain away the treatment effect. The authors believe in this context, “female” is an important factor. Suppose the confounder is as important as “female”, what the partial  $R^2$  would be? It is shown that  $R^2_{Y \sim Z|X,D} = 12.5\%$  and  $R^2_{D \sim Z|X} = 0.9\%$ , so a confounder as strong as “female” is not strong enough to explain away the treatment effect.

It is worth being careful about *why*, because comparing each number to  $RV_{q=1} = 13.9\%$  one at a time invites the wrong reading. The  $RV$  is defined for a confounder of *equal* strength on both sides, so it is not a threshold that each partial  $R^2$  must separately clear. Here the outcome-side association (12.5%) is in fact close to the  $RV$ , which on its own might look like a near miss — but the treatment-side association is only 0.9%, more than an order of magnitude short. It is the treatment side that does the work: “female” predicts the outcome nearly well enough to matter, and barely predicts the treatment at all. The contour plot below is the honest check, since it evaluates the pair jointly rather than either coordinate alone.

```
# plot bias contour of point estimate
plot(sensitivity)
```



This plot shows that even 3 times as strong as “female”, the confounder is not strong enough to explain away the treatment effect.

## 15.6. sensitivity analysis for IV

Even with an IV model, we need sensitivity analysis. The worry would be there could be an unobserved confounder that affects both the instrument and the outcome.

Cinelli and Hazlett have a package “iv.sensemakr” for this. The usage is similar to “sensemakr”.

The example is Card’s returns-to-schooling data, 3,010 men from the 1976 NLS. The outcome is log wage, the treatment is years of education, and the instrument is `nearc4`, an indicator for growing up near a four-year college. The covariates are experience and its square, race, region, and urban indicators. The concern the sensitivity analysis addresses is a confounder affecting both proximity to a college and wages.

```
# loads package
library(iv.sensemakr)

# loads dataset
data("card")

# prepares data
y <- card$lwage # outcome
d <- card$educ  # treatment
z <- card$nearc4 # instrument
x <- model.matrix( ~ exper + expersq + black + south + smsa + reg661 + reg662 +
                    reg663 + reg664 + reg665 + reg666 + reg667 + reg668 + smsa66,
                    data = card) # covariates

# fits IV model
card.fit <- iv_fit(y,d,z,x)

# see results
card.fit
```

Instrumental Variable Estimation  
(Anderson-Rubin Approach)

```
=====
IV Estimates:
  Coef. Estimate: 0.132
  t-value: 2.33
  p-value: 0.02
  Conf. Interval: [0.025, 0.285]
Note: H0 = 0, alpha = 0.05, df = 2994.
=====
See summary for first stage and reduced form.
```

```
# runs sensitivity analysis
card.sens <- sensemakr(card.fit, benchmark_covariates = c("black", "smsa"))
```

```
# see results
card.sens
```

Sensitivity Analysis for Instrumental Variables  
(Anderson-Rubin Approach)

=====

IV Estimates:

Coef. Estimate: 0.132  
t-value: 2.33  
p-value: 0.02  
Conf. Interval: [0.025, 0.285]

Sensitivity Statistics:

Extreme Robustness Value: 0.000523  
Robustness Value: 0.00667

Bounds on Omitted Variable Bias:

| Bound Label | R2zw.x  | R2y0w.zx | Lower CI | Upper CI | Crit. Thr. |
|-------------|---------|----------|----------|----------|------------|
| 1x black    | 0.00221 | 0.0750   | -0.0212  | 0.402    | 2.59       |
| 1x smsa     | 0.00639 | 0.0202   | -0.0192  | 0.396    | 2.57       |

Note:  $H_0 = 0$ ,  $q \geq 1$ ,  $\alpha = 0.05$ ,  $df = 2994$ .

=====

See summary for first stage and reduced form.

```
summary(card.sens)
```

Sensitivity Analysis for Instrumental Variables  
(Anderson-Rubin Approach)

=====

IV Estimates:

Coef. Estimate: 0.132  
t-value: 2.33  
p-value: 0.02  
Conf. Interval: [0.025, 0.285]

Sensitivity Statistics:

Extreme Robustness Value: 0.000523  
Robustness Value: 0.00667

Bounds on Omitted Variable Bias:

| Bound Label | R2zw.x  | r2y0w.zx | Lower CI | Upper CI | Crit. Thr. |
|-------------|---------|----------|----------|----------|------------|
| 1x black    | 0.00221 | 0.0750   | -0.0212  | 0.402    | 2.59       |
| 1x smsa     | 0.00639 | 0.0202   | -0.0192  | 0.396    | 2.57       |

## 15. Sensitivity analysis of OLS with sensemakr

Note:  $H_0 = 0$ ,  $q \geq 1$ ,  $\alpha = 0.05$ ,  $df = 2994$ .

---

### FS Estimates:

Coef. Estimate: 0.32  
Standard Error: 0.0879  
t-value: 3.64  
p-value: 0.000276  
Conf. Interval: [0.148, 0.492]

### Sensitivity Statistics:

Extreme Robustness Value: 0.00313  
Robustness Value: 0.0302

### Bounds on Omitted Variable Bias:

| Bound Label | R2zw.x  | R2dw.zx | Lower CI | Upper CI | Crit. Thr. |
|-------------|---------|---------|----------|----------|------------|
| 1x black    | 0.00221 | 0.03342 | 0.109    | 0.531    | 2.40       |
| 1x smsa     | 0.00639 | 0.00498 | 0.120    | 0.520    | 2.27       |

Note:  $H_0 = 0$ ,  $q = 1$ ,  $\alpha = 0.05$ ,  $df = 2994$ .

---

### RF Estimates:

Coef. Estimate: 0.0421  
Standard Error: 0.0181  
t-value: 2.33  
p-value: 0.02  
Conf. Interval: [0.007, 0.078]

### Sensitivity Statistics:

Extreme Robustness Value: 0.000523  
Robustness Value: 0.00667

### Bounds on Omitted Variable Bias:

| Bound Label | R2zw.x  | R2yw.zx | Lower CI | Upper CI | Crit. Thr. |
|-------------|---------|---------|----------|----------|------------|
| 1x black    | 0.00221 | 0.0657  | -0.00418 | 0.0883   | 2.56       |
| 1x smsa     | 0.00639 | 0.0197  | -0.00429 | 0.0884   | 2.56       |

Note:  $H_0 = 0$ ,  $q = 1$ ,  $\alpha = 0.05$ ,  $df = 2994$ .

=====

The basic idea is to run an IV regression, then apply sensemakr to that model. In this case, the authors use “black” and “smsa” as benchmark covariates. The authors believe these are important factors.

The IV estimate is 0.132 with  $t = 2.33$ ,  $p = 0.02$ , and an Anderson-Rubin confidence interval of [0.025, 0.285] that excludes zero. Read the sensitivity statistics next to that, and the picture is not reassuring. The Robustness Value is 0.00667 — two-thirds of one percent. Compare the OLS

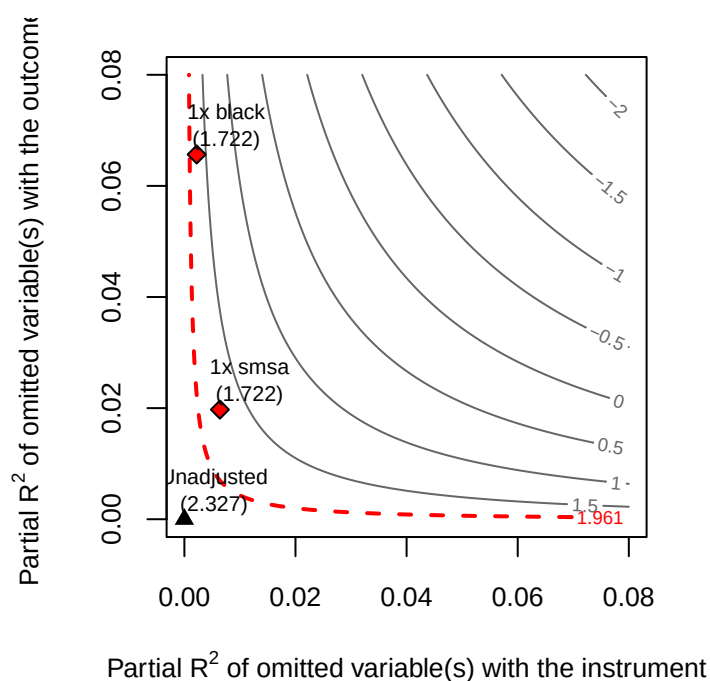
example earlier in this chapter, where  $RV_{q=1}$  was 13.9%. This IV result is roughly twenty times more fragile than that one.

The benchmark bounds make it concrete, and they point the other way from what one might hope. A confounder as strong as **black** explains 0.22% of the residual variance of the instrument and 7.5% of the outcome, and under it the adjusted interval becomes  $[-0.021, 0.402]$ . For **smsa** the figures are 0.64% and 2.0%, giving  $[-0.019, 0.396]$ . **Both intervals contain zero.** A confounder no stronger than either benchmark covariate is enough to remove the significance of the estimate.

That **smsa** number is worth a second look on its own terms: its instrument-side partial  $R^2$  of 0.00639 sits just under the Robustness Value of 0.00667, so a confounder of that strength lands essentially exactly on the tipping point. The two diagnostics agree.

None of this says the Card estimate is wrong. It says the identification rests on the instrument being cleaner than **black** or **smsa** are as predictors, which is an assumption about college proximity that the data cannot check. We can also plot a contour plot to see how strong the confounder needs to be to explain away the treatment effect.

```
plot(card.sens, parm = "rf", sensitivity.of = "t-value", lim = 0.08, kz = 1, ky = 1)
```



Unfortunately the package currently only allows for one instrument. Should not be too hard to modify it to be more general.

---

Systematic treatment: [R](#) · [Julia](#).





## 16. Gender Wage Gap Analysis: Oaxaca-Blinder and Causal Estimation

### 16.1. Introduction: Oaxaca-Blinder as Regression Adjustment

The classical Oaxaca-Blinder (OB) decomposition separates the raw difference in mean outcomes between groups into an “explained” component (differences in observable covariates) and an “unexplained” component (differences in regression coefficients, often interpreted as discrimination):

$$E[Y | A = 1] - E[Y | A = 0] = \underbrace{(E[X | A = 1] - E[X | A = 0])\beta_0}_{\text{Explained (Covariate Differences)}} + \underbrace{E[X | A = 1](\beta_1 - \beta_0)}_{\text{Unexplained (Coefficient Differences)}}$$

where  $A = 1$  denotes female and  $A = 0$  denotes male. A well-known limitation of the traditional OB decomposition is its sensitivity to the choice of reference group ( $\beta_0$  vs.  $\beta_1$ ).

Słoczyński (2013, 2015) connects the OB decomposition directly to the potential outcomes framework:

- Demeaning covariates at the **overall sample mean** identifies the Population Average Gender Effect ( $\tau_{\text{PAGE}}$ , equivalent to the Average Treatment Effect, ATE).
- Demeaning covariates at the **female subpopulation mean** identifies the Average Treatment Effect on the Treated ( $\tau_{\text{PATT}}$ , or ATT).

In this framing, the unexplained gap is simply the treatment effect evaluated at the chosen covariate distribution, while the explained gap represents selection bias. This formulation allows us to estimate the gap via standard regression adjustment or modern doubly robust estimators.

### 16.2. Empirical Application: 2015 CPS Data

We analyze a subsample from the 2015 March Supplement of the U.S. Current Population Survey (`wage2015_subsample_inference.csv`), restricted to white, non-Hispanic full-time workers aged 25–64 ( $N = 5,150$ ). The outcome  $Y$  is the logarithm of hourly wage (`lwage`), and the indicator is female ( $A \in \{0, 1\}$ ).

```
data <- read_csv("wage2015_subsample_inference.csv") |>
  rename(soc1 = scl, sohs = shs, sout = so) |>
  mutate(
    exp_dm = exp1 - mean(exp1, na.rm = TRUE),
    female = sex,
    occ2 = factor(occ2),
```

```

ind2      = factor(ind2),
sohs_dm   = sohs - mean(sohs, na.rm = TRUE),
hsg_dm    = hsg - mean(hsg, na.rm = TRUE),
socl_dm   = socl - mean(socl, na.rm = TRUE),
clg_dm    = clg - mean(clg, na.rm = TRUE),
mw_dm     = mw - mean(mw, na.rm = TRUE),
sout_dm   = sout - mean(sout, na.rm = TRUE),
we_dm     = we - mean(we, na.rm = TRUE)
)

```

```

# 1. Unadjusted (raw) difference
reg1 <- lm(lwage ~ female, data = data)
tidy(reg1) |> filter(term == "female")

```

```

# A tibble: 1 x 5
  term      estimate std.error statistic p.value
<chr>      <dbl>    <dbl>    <dbl>   <dbl>
1 female  -0.0383     0.0160     -2.40  0.0165

```

```

# 2. Regression adjustment with demeaned human capital & region controls

```

```

reg2 <- lm(lwage ~ female * (exp_dm + sohs_dm + hsg_dm + socl_dm + clg_dm + mw_dm + sout_dm + v
      data = data)
tidy(reg2) |> filter(term == "female")

```

```

# A tibble: 1 x 5
  term      estimate std.error statistic p.value
<chr>      <dbl>    <dbl>    <dbl>   <dbl>
1 female  -0.115     0.0147     -7.80 7.39e-15

```

### 16.2.1. The Composition Effect

The raw wage gap is  $-3.8\%$  ( $p = 0.012$ ). After controlling for potential experience, education, and region, the estimated gap widens to  $-11.3\%$  ( $p < 0.001$ ).

Adding controls usually attenuates an observed gap; here it nearly triples it. This occurs because of sample composition:

```

data |>
  group_by(female) |>
  summarise(
    n      = n(),
    mean_lwage = mean(lwage),
    exp1    = mean(exp1),
    `<HS`   = mean(sohs),
    HS      = mean(hsg),

```

```

`Some Coll` = mean(socl),
College      = mean(clg),
Advanced     = mean(ad)
) |>
mutate(across(where(is.numeric), ~round(.x, 3))) |>
knitr::kable(caption = "Covariate means by gender.")

```

Table 16.1.: Covariate means by gender.

| female | n    | mean_lwage | exp1   | <HS   | HS    | Some Coll | College | Advanced |
|--------|------|------------|--------|-------|-------|-----------|---------|----------|
| 0      | 2861 | 2.988      | 13.784 | 0.032 | 0.294 | 0.273     | 0.294   | 0.107    |
| 1      | 2289 | 2.949      | 13.731 | 0.013 | 0.181 | 0.284     | 0.347   | 0.175    |

Women in this sample have higher average educational attainment than men (17.5% hold advanced degrees vs. 10.7%; only 1.3% have less than high school vs. 3.2%), with identical experience ( $\approx 13.8$  years). Because education raises wages, women's educational advantage partially masks the earnings gap in the raw comparison. Controlling for education holds this advantage constant, revealing an adjusted gap of  $-11.3\%$  within equivalent education-experience cells.

### 16.2.2. Regression Adjustment via `marginaleffects`

Instead of manual demeaning, the same estimands can be obtained directly using `marginaleffects::avg_comparisons`

```

reg3 <- lm(lwage ~ female * (exp1 + sohs + hsg + socl + clg + mw + sout + we), data = data)

# Population Average Gender Effect (ATE / PAGE)
avg_comparisons(reg3, variables = "female")

```

```

Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
-0.115    0.0147 -7.8    <0.001 47.2 -0.143 -0.0858

```

```

Term: female
Type: response
Comparison: 1 - 0

```

```

# Female Subpopulation Gender Effect (ATT / PATT)
avg_comparisons(reg3, variables = "female", newdata = subset(data, female == 1))

```

```

Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
-0.113    0.0148 -7.65   <0.001 45.5 -0.142 -0.0844

```

## 16. Gender Wage Gap Analysis: Oaxaca-Blinder and Causal Estimation

Term: female  
Type: response  
Comparison: 1 - 0

Both ATE and ATT land at  $-11.3\%$ , confirming that the fully interacted OLS specification evaluated at the respective empirical covariate distributions yields the OB decomposition directly.

### 16.3. Adding Occupation and Industry: Mediators vs. Bad Controls

A central question in pay gap analyses is whether to adjust for occupation and industry:

```
reg4 <- lm(lwage ~ female * (exp1 + sohs + hsg + soc1 + clg + mw + sout + we + occ2 + ind2),  
          data = data)  
  
avg_comparisons(reg4, variables = "female", newdata = subset(data, female == 1))
```

| Estimate | Std. Error | z     | Pr(> z ) | S    | 2.5 %  | 97.5 %  |
|----------|------------|-------|----------|------|--------|---------|
| -0.075   | 0.0162     | -4.62 | <0.001   | 18.0 | -0.107 | -0.0432 |

Term: female  
Type: response  
Comparison: 1 - 0

Adjusting for 22 occupation and industry categories narrows the gap from  $-11.3\%$  to  $-7.5\%$ .

#### 16.3.1. Interpretation Trade-Off

1. **Descriptive Mediation:** If the goal is accounting for occupational sorting, the move from  $-11.3\%$  to  $-7.5\%$  shows that about one-third of the adjusted wage gap reflects differences in occupation and industry.
2. **Causal Interpretation (Bad Control):** Occupation and industry are post-treatment choices ( $A \rightarrow \text{Occupation} \rightarrow Y$ ). Conditioning on post-treatment variables can introduce collider bias if unobserved factors (e.g., career preferences, flexibility constraints) simultaneously affect occupational selection and wages ( $A \rightarrow \text{Occupation} \leftarrow U \rightarrow Y$ ).
3. **Controlled Direct Effect:** The  $-7.5\%$  estimate represents a controlled direct effect—the within-occupation wage disparity between men and women with similar observable characteristics.

## 16.4. Nonparametric Doubly Robust Estimation (AIPW)

Linear regression adjustment assumes that the relationship between covariates and wages is additive and correctly specified. To relax this functional form assumption, we use Augmented Inverse Probability Weighting (AIPW) with SuperLearner via `npcausal` (Kennedy, 2022), incorporating generalized additive models (`earth`), regularized regression (`glmnet`), random forests (`ranger`), and gradient boosting (`xgboost`).

```
library(npcausal)

# To avoid prolonged cross-fitting during interactive compilation, we load the
# pre-computed SuperLearner results (SL.earth, SL.glmnet, SL.mean, SL.ranger, SL.xgboost)
# from the book cache if present; otherwise, compute from scratch.
cache_base <- "gwgcache/html/npcausal1_5538101f2b9ff4f8b21593068dc74ef6"
if (file.exists(paste0(cache_base, ".rdx"))) {
  cache_env <- new.env()
  lazyLoad(cache_base, envir = cache_env)
  aipw_ate <- cache_env$aipw_ate
  aipw_att <- cache_env$aipw_att
} else {
  SL.library <- c("SL.earth", "SL.glmnet", "SL.mean", "SL.ranger", "SL.xgboost")
  reg3 <- lm(lwage ~ exp_dm + sohs + hsg + socl + clg + mw + sout + we + occ2 + ind2, data = data)
  W <- as.data.frame(model.matrix(reg3)[, -1])
  Y <- data$lwage
  A <- data$female
  set.seed(123)
  aipw_ate <- ate(y = Y, a = A, x = W, nsplits = 5, sl.lib = SL.library)
  aipw_att <- att(y = Y, a = A, x = W, nsplits = 5, sl.lib = SL.library)
}
```

```
aipw_ate$res
```

|   | parameter    | est         | se         | ci.ll       | ci.ul       | pval |
|---|--------------|-------------|------------|-------------|-------------|------|
| 1 | E{Y(0)}      | 2.99695506  | 0.01093465 | 2.97552315  | 3.01838697  | 0    |
| 2 | E{Y(1)}      | 2.93906408  | 0.01327342 | 2.91304817  | 2.96507999  | 0    |
| 3 | E{Y(1)-Y(0)} | -0.05789098 | 0.01597522 | -0.08920241 | -0.02657955 | 0    |

```
aipw_att$res
```

|   | parameter     | est         | se         | ci.ll       | ci.ul       | pval |
|---|---------------|-------------|------------|-------------|-------------|------|
| 1 | E(Y A=1)      | 2.94948490  | 0.01160804 | 2.92673314  | 2.97223666  | 0    |
| 2 | E{Y(0) A=1}   | 3.01689509  | 0.01221863 | 2.99294658  | 3.04084360  | 0    |
| 3 | E{Y-Y(0) A=1} | -0.06741019 | 0.01409116 | -0.09502886 | -0.03979152 | 0    |

### 16.4.1. Takeaways from Nonparametric Estimation

1. **Robustness to Functional Form:** The nonparametric AIPW ATT is  $-6.7\%$  (95% CI :  $[-9.5\%, -3.9\%]$ ), very close to the parametric interacted OLS estimate of  $-7.5\%$ . This indicates that linear interaction models provide an adequate approximation.
2. **Substantive Consistency:** Both parametric and machine-learning estimators confirm a statistically significant within-occupation wage gap of approximately 7%.

## 16.5. Summary

1. **Oaxaca-Blinder Connection:** The classical OB decomposition is equivalent to regression adjustment where treatment is interacted with centered covariates. Demeaning at the overall mean yields the ATE, while demeaning at the female subpopulation mean yields the ATT.
2. **Composition Matters:** Conditioning on education widens the wage gap from  $-3.8\%$  to  $-11.3\%$  because women in the sample possess higher average educational credentials than men.
3. **Occupation as a Mediator:** Conditioning on occupation and industry narrows the gap to  $-7.5\%$ , capturing the within-job pay disparity (controlled direct effect).
4. **Machine Learning Confirmation:** Doubly robust AIPW estimation produces a consistent  $-6.7\%$  gap, demonstrating that the results are not driven by arbitrary parametric functional form assumptions.

**Part IV.**

**Panel Data & DiD**





# 17. Causal Forest in panel data

## 17.1. Introduction

In this simulation exercise, we use Causal Forest (Now is implemented in Generalized Random Forest) (<https://github.com/swager/grf>) to calculate conditional average treatment effect (or heterogeneous treatment effect). We assume three different data generating processes. The first one is a linear interaction between a variable of interest and the treatment dummy. The second one assumes a nonlinear function (a step function) of a variable of interest, say  $X$ , and the treatment dummy  $W$ . The third one is also nonlinear, assuming we have variable  $X$ , but the real DGP is the interaction of log of  $X$  and  $W$ .

## 17.2. Linear effect

Suppose we have a panel of firms over time, treatment assignment is random. We split the data into training and test. Then we generate 10 random variables, the first two are highly correlated. Then we collapse  $V2$  into firm means, and use that as the unit effect.

Concretely:  $n = 400$  firms over  $t = 10$  periods, 4,000 observations, with  $p = 10$  standard normal covariates of which  $V1$  and  $V2$  are correlated at 0.9025. Treatment is  $W \sim \text{Bernoulli}(0.5)$ , assigned independently of everything, and a further coin flip splits the sample roughly in half into training and test sets (about 2,014 rows for training). The unit effect  $\alpha_i$  is the firm mean of  $V2$ .

That is, we have a DGP as:

$$y_{it} = \alpha_i + V1_{it} + W_{it} + V1_{it} * W_{it} + \epsilon_{it} \quad (17.1)$$

Here  $\alpha_i$  is not observed, and it is correlated with  $V1$ , so leaving it out is in principle a source of bias.

It is worth being precise about how much bias, though, because the results below are less dramatic than that setup suggests. The unit effect is the *firm mean* of  $V2$  over all  $t = 10$  periods, and averaging attenuates its correlation with contemporaneous  $V1$  by a factor of  $t$ : with  $\text{Cor}(V1_{it}, V2_{it}) = 0.9025$  we get  $\text{Cov}(V1_{it}, \bar{V}2_i) = 0.9025/10$ , and  $\text{Cor}(V1_{it}, \alpha_i) \approx 0.29$ . So the omitted-variable bias on  $V1$  is only about 0.05–0.09, not a qualitative failure.

That is what the three models show: FE, RE and OLS all recover  $V1 \approx 1$ ,  $W \approx 1$  and  $V1:W \approx 1.05$ , essentially the truth. Note in particular that the *treatment* coefficients are unbiased in all three, because  $W$  is randomized and so independent of  $\alpha_i$  — omitting a unit effect cannot bias the effect of a randomly assigned treatment. If you want a panel example where FE visibly beats OLS, the unit effect has to be correlated with  $V1$  *contemporaneously* (or  $t$  has to be small), and the treatment has to be non-random.

## 17. Causal Forest in panel data

```
# This is a simulation exercise to see how causal forest performs
# First we run a linear model, with panel setting
# Then two nonlinear situations, with panel setting.

library(lfe, quietly = TRUE)
library(lme4)
require(snowfall)
library(MASS)
library(grf)
library(tidyverse)
set.seed(666)
rm(list = ls())
# t is no. of periods. p is no. of variables, n is no. of firms.
t <- 10
p <- 10
n=400
# generate p random variables
data1 = as.data.frame(matrix(rnorm(n*t*p), n*t, p))
names(data1) <- paste0("X", 1:p)

firm=seq(1,n)
time=seq(1,t)
data <- expand.grid(firm=firm,time=time)

# W is the treatment assignment
data$W <- rbinom(n*t, 1, 0.5)
data$train <- rbinom(n*t, 1, 0.5)

# generate two correlated variables
covarMat = matrix( c(1, .95^2, .95^2, 1 ) , nrow=2 , ncol=2 )
data.2 = as.data.frame(mvrnorm(n=n*t , mu=rep(0,2), Sigma=covarMat ))
names(data.2) <- c("V1", "V2")

data <- bind_cols(data,data.2) %>%
  tibble()

data <- bind_cols(data,data1)

# generate unit effect by firm means of V2, which is correlated with V1
data <- data %>%
  group_by(firm) %>%
  mutate(unit.effect=mean(V2)) %>%
  ungroup() %>%
  arrange(firm, time)
```

```

y<-c()
unit<-c()
X<-c()
k <- 0
for(i in 1:n){
  for(j in 1:t){
    k<-k+1
    unit[k]<-i
    y[k]<-1+ data$V1[k] + data$W[k] + data$V1[k]*data$W[k] + data$unit.effect[k]+rnorm(1,m
  }
}
data$unit=unit
data$y=y

data.train <- data %>%
  filter(train==1)
data.test <- data %>%
  filter(train==0)

# run fixed effect, random effect, and ols to see whether they are
# biased.
# in general we should expect fe performs better, since we are
# omitting unit.effect in the other two models
fe <- felm(y ~ V1 * W | unit, data=data.train)
re <- lmer(y~V1*W+(1 | unit), data=data.train)
ols <- lm(y ~ V1 * W , data=data.train)
summary(fe)

```

Call:

```
felm(formula = y ~ V1 * W | unit, data = data.train)
```

Residuals:

|  | Min     | 1Q      | Median  | 3Q     | Max    |
|--|---------|---------|---------|--------|--------|
|  | -3.0440 | -0.6252 | -0.0083 | 0.6221 | 3.6174 |

Coefficients:

|      | Estimate | Std. Error | t value | Pr(> t )   |
|------|----------|------------|---------|------------|
| V1   | 0.98354  | 0.03748    | 26.24   | <2e-16 *** |
| W    | 1.02186  | 0.05093    | 20.06   | <2e-16 *** |
| V1:W | 1.04302  | 0.05237    | 19.92   | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.018 on 1612 degrees of freedom

## 17. Causal Forest in panel data

```
Multiple R-squared(full model): 0.7952   Adjusted R-squared: 0.7443
Multiple R-squared(proj model): 0.7269   Adjusted R-squared: 0.659
F-statistic(full model):15.61 on 401 and 1612 DF, p-value: < 2.2e-16
F-statistic(proj model): 1430 on 3 and 1612 DF, p-value: < 2.2e-16
```

```
summary(re)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: y ~ V1 * W + (1 | unit)
Data: data.train
```

```
REML criterion at convergence: 5938.9
```

```
Scaled residuals:
```

|  | Min     | 1Q      | Median  | 3Q     | Max    |
|--|---------|---------|---------|--------|--------|
|  | -3.4332 | -0.6615 | -0.0049 | 0.6640 | 3.8329 |

```
Random effects:
```

| Groups   | Name        | Variance | Std.Dev. |
|----------|-------------|----------|----------|
| unit     | (Intercept) | 0.07166  | 0.2677   |
| Residual |             | 1.04602  | 1.0228   |

```
Number of obs: 2014, groups: unit, 399
```

```
Fixed effects:
```

|             | Estimate | Std. Error | t value |
|-------------|----------|------------|---------|
| (Intercept) | 1.00010  | 0.03539    | 28.26   |
| V1          | 1.02961  | 0.03486    | 29.53   |
| W           | 1.00333  | 0.04684    | 21.42   |
| V1:W        | 1.05963  | 0.04814    | 22.01   |

```
Correlation of Fixed Effects:
```

|      | (Intr) | V1     | W     |
|------|--------|--------|-------|
| V1   | 0.013  |        |       |
| W    | -0.656 | -0.008 |       |
| V1:W | -0.007 | -0.727 | 0.025 |

```
summary(ols)
```

```
Call:
```

```
lm(formula = y ~ V1 * W, data = data.train)
```

```
Residuals:
```

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -3.8384 | -0.7101 | 0.0081 | 0.7334 | 4.0053 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | 1.00363  | 0.03319    | 30.24   | <2e-16 *** |
| V1          | 1.04612  | 0.03516    | 29.75   | <2e-16 *** |
| W           | 0.99657  | 0.04712    | 21.15   | <2e-16 *** |
| V1:W        | 1.06458  | 0.04840    | 22.00   | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.057 on 2010 degrees of freedom

Multiple R-squared: 0.7249, Adjusted R-squared: 0.7245

F-statistic: 1765 on 3 and 2010 DF, p-value: &lt; 2.2e-16

Now we run Causal Forest on this data. Two clarifications about what is and is not at stake here. First, treatment  $W$  is **randomized** (`rbinom(., 0.5)`, independent of the firm effect), so the omitted unit effect  $\alpha_i$  does *not* confound the treatment effect — it only enters the nuisance functions (and hence precision), it does not create treatment-effect bias. The panel issue here is therefore mainly *dependence and nuisance fit*, not confounding. Second, the firm structure should be handled in two ways: we include the firm dummies in  $X$  so the forest can absorb  $\alpha_i$  in the nuisance models, **and** we pass `clusters = firm` so that `grf`'s honest sample splitting and its variance estimates keep each firm's repeated observations together rather than treating them as independent rows. Including the dummies alone does neither of those.

```
# ensure consistent factor levels across train/test
all_firms <- levels(factor(data$firm))
data.train$firm <- factor(data.train$firm, levels = all_firms)
data.test$firm <- factor(data.test$firm, levels = all_firms)

# a is a data set of firm dummies
# Caution: this one-hot-encodes all ~400 firms into individual dummy columns
# of X. Random forests split poorly on such a high-cardinality sparse block
# of binary columns (each dummy is only informative for a tiny fraction of
# observations). A fixed-effects forest (demean y, W, and the continuous
# covariates within firm first, then run causal_forest on the demeaned
# variables) or a method that handles unit effects natively would typically
# perform better than including firm as raw dummies.
a <- as.data.frame(model.matrix(y~firm, data.train))
X <- as.matrix(bind_cols(data.train[,c(5,7:16)],a))
Y <- data.train$y
W <- data.train$W

# causal forest is run on y against V1, to X10 and firm dummies.
# clusters = firm makes honest splitting and the variance estimates respect
# the panel structure (a firm's repeated rows are kept together, not treated
# as independent observations).
tau.forest <- causal_forest(X, Y, W, clusters = as.integer(data.train$firm))

average_treatment_effect(tau.forest, target.sample = "all")
```

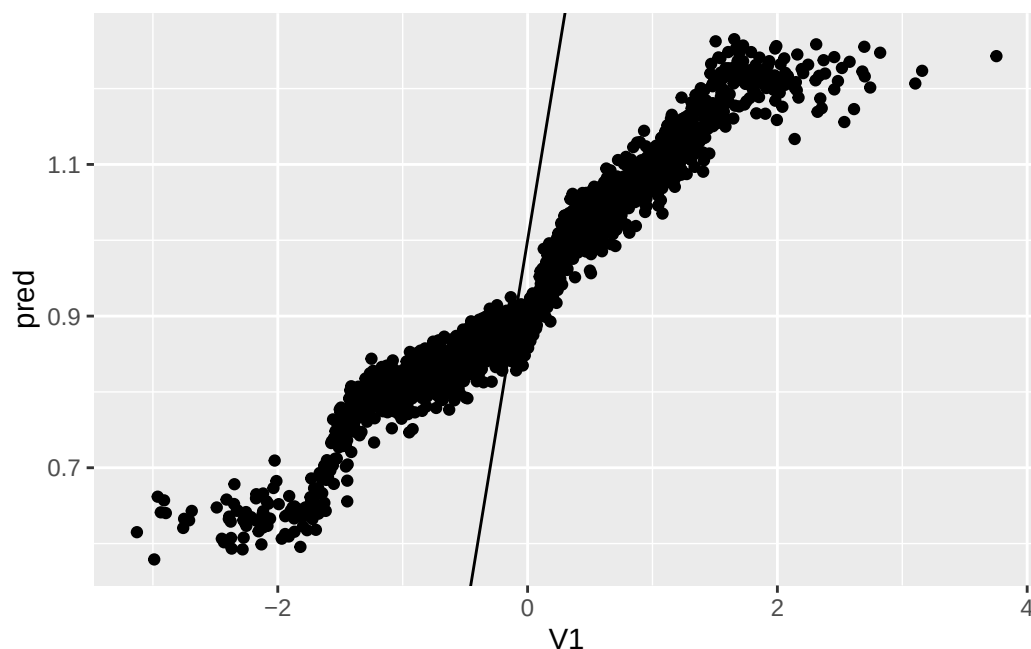
## 17. Causal Forest in panel data

```
estimate    std.err  
0.93560912 0.08227266
```

```
average_treatment_effect(tau.forest, target.sample = "treated")
```

```
estimate    std.err  
0.93168066 0.08158925
```

```
a <- as.data.frame(model.matrix(y~firm, data.test))  
X.test <- as.matrix(bind_cols(data.test[,c(5,7:16)],a))  
Y.test <- data.test$y  
W.test <- data.test$W  
  
tau.hat = predict(tau.forest, X.test, estimate.variance = TRUE)  
sigma.hat = sqrt(tau.hat$variance.estimates)  
  
data.test$pred <- tau.hat$predictions  
ggplot(data.test, aes(x=V1, y=pred)) + geom_point() + geom_abline(intercept = 1, slope = 1)
```



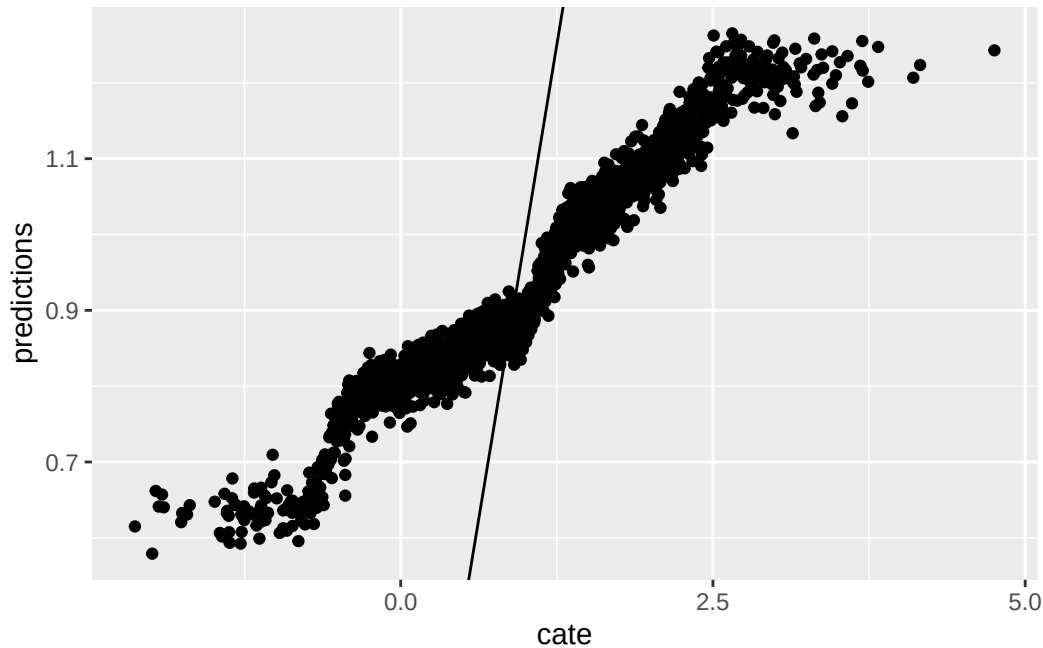
The forest's doubly-robust ATE comes out at 0.9356 with a standard error of 0.0823, and 0.9317 with 0.0816 for the second variant. The true ATE is  $E[1 + V1] = 1$ , so both are within one standard error of it — the forest costs a little precision against the correctly specified linear models but is not biased.

This graph is the predicted value of CATE on the test data, vs. the value of  $V1$ . The points should sit on the 45-degree line, and the error bars in the second panel show how much of the scatter is estimation noise rather than a failure to track the true  $1 + V1$ .

Now we graph the predicted CATE against the true CATE. The other nice feature of Causal Forest is that it returns variance for CATE for each observation. We also graph the error bar here.

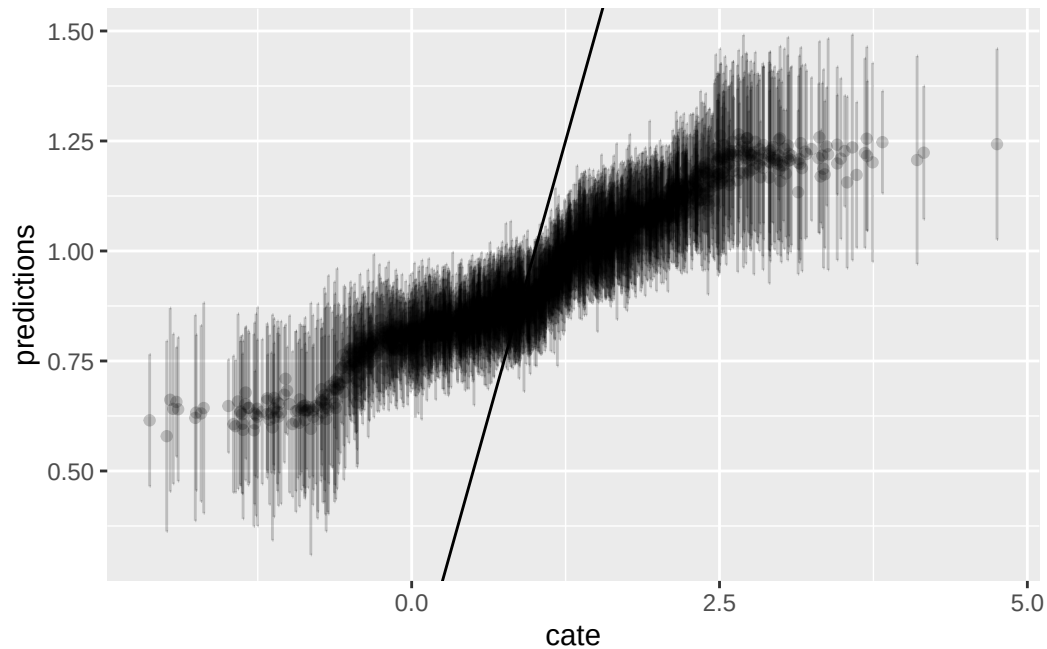
```
graph.data <- as.data.frame(tau.hat )
graph.data <- graph.data %>%
  mutate(cate=1+data.test$V1) %>%
  mutate(lower=predictions - sigma.hat , upper=predictions + sigma.hat )

ggplot(graph.data , aes(x=cate, y=predictions)) + geom_point() + geom_abline(slope = 1)
```



```
ggplot() + geom_errorbar(graph.data , mapping=aes(x=cate, ymin=lower,ymax=upper), alpha = 0.2)
```

## 17. Causal Forest in panel data



### 17.3. Nonlinear effect case 1

In the first situation, we assume DGP:

$$y_{it} = 1 + \alpha_i + \max(V1_{it}, 0) * W_{it} + \epsilon_{it} \quad (17.2)$$

```
# if we have a nonlinear effect
# suppose we have the same X, but y is generated nonlinearly.
# in this example, y is a nonlinear function of V1 and W
set.seed(666)
rm(list = ls())
# t is no. of periods. p is no. of variables, n is no. of firms.
t <- 10
p <- 10
n=400
# generate p random variables
data1 = as.data.frame(matrix(rnorm(n*t*p), n*t, p))
names(data1) <- paste0("X", 1:p)

firm=seq(1,n)
time=seq(1,t)
data <- expand.grid(firm=firm,time=time)

# W is the treatment assignment
data$W <- rbinom(n*t, 1, 0.5)
```



```

data$train <- rbinom(n*t, 1, 0.5)

# generate two correlated variables
covarMat = matrix( c(1, .95^2, .95^2, 1 ) , nrow=2 , ncol=2 )
data.2 = as.data.frame(mvrnorm(n=n*t , mu=rep(0,2), Sigma=covarMat ))
names(data.2) <- c("V1", "V2")

data <- bind_cols(data,data.2) %>%
  tibble()

data <- bind_cols(data,data1)

# generate unit effect by firm means of V2, which is correlated with V1
data <- data %>%
  group_by(firm) %>%
  mutate(unit.effect=mean(V2)) %>%
  ungroup() %>%
  arrange(firm, time)

y<-c()
unit<-c()
X<-c()
k <- 0
for(i in 1:n){
  for(j in 1:t){
    k<-k+1
    unit[k]<-i
    y[k]<-1+ pmax(data$V1[k],0)*data$W[k] + data$unit.effect[k]+rnorm(1,mean=0,sd=1)
  }
}
data$y=y
data$unit <- unit

data.train <- data %>%
  filter(train==1)
data.test <- data %>%
  filter(train==0)

# run fixed effect, random effect, and ols to see whether they are
# biased.
# they all perform poorly
fe <- felm(y ~ V1 * W | unit, data=data.train)
re <- lmer(y~V1*W+(1 | unit), data=data.train)
ols <- lm(y ~ V1 * W , data=data.train)
summary(fe)

```

## 17. Causal Forest in panel data

Call:

```
felm(formula = y ~ V1 * W | unit, data = data.train)
```

Residuals:

|  | Min     | 1Q      | Median  | 3Q     | Max    |
|--|---------|---------|---------|--------|--------|
|  | -2.9461 | -0.6225 | -0.0178 | 0.6245 | 3.3537 |

Coefficients:

|      | Estimate | Std. Error | t value | Pr(> t )     |
|------|----------|------------|---------|--------------|
| V1   | -0.02076 | 0.03815    | -0.544  | 0.586        |
| W    | 0.42527  | 0.05184    | 8.203   | 4.73e-16 *** |
| V1:W | 0.53367  | 0.05331    | 10.012  | < 2e-16 ***  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.036 on 1612 degrees of freedom

Multiple R-squared(full model): 0.3744 Adjusted R-squared: 0.2187

Multiple R-squared(proj model): 0.1393 Adjusted R-squared: -0.07481

F-statistic(full model):2.405 on 401 and 1612 DF, p-value: < 2.2e-16

F-statistic(proj model): 86.96 on 3 and 1612 DF, p-value: < 2.2e-16

```
summary(re)
```

Linear mixed model fit by REML ['lmerMod']

Formula: y ~ V1 \* W + (1 | unit)

Data: data.train

REML criterion at convergence: 6017.2

Scaled residuals:

|  | Min     | 1Q      | Median  | 3Q     | Max    |
|--|---------|---------|---------|--------|--------|
|  | -3.5905 | -0.6596 | -0.0031 | 0.6717 | 3.4143 |

Random effects:

| Groups   | Name        | Variance | Std.Dev. |
|----------|-------------|----------|----------|
| unit     | (Intercept) | 0.07977  | 0.2824   |
| Residual |             | 1.08359  | 1.0410   |

Number of obs: 2014, groups: unit, 399

Fixed effects:

|             | Estimate | Std. Error | t value |
|-------------|----------|------------|---------|
| (Intercept) | 0.99976  | 0.03624    | 27.587  |
| V1          | 0.02797  | 0.03553    | 0.787   |
| W           | 0.40652  | 0.04774    | 8.515   |
| V1:W        | 0.54484  | 0.04907    | 11.104  |

Correlation of Fixed Effects:

```
(Intr) V1      W
V1      0.013
W      -0.653 -0.008
V1:W   -0.007 -0.727  0.025
```

```
summary(ols)
```

Call:

```
lm(formula = y ~ V1 * W, data = data.train)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-4.0965 -0.7415  0.0095  0.7512  3.8413
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.00363    0.03386   29.643  <2e-16 ***
V1           0.04612    0.03587    1.286    0.199
W           0.39928    0.04807    8.307  <2e-16 ***
V1:W        0.54938    0.04937   11.127  <2e-16 ***
```

---

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 1.078 on 2010 degrees of freedom

Multiple R-squared: 0.1556, Adjusted R-squared: 0.1543

F-statistic: 123.5 on 3 and 2010 DF, p-value: < 2.2e-16

These linear models miss the *shape* of the effect, but it overstates things to say they return nothing. Each recovers the best linear approximation to the true CATE  $\max(V1, 0)$ , which is  $0.3989 + 0.5 V1$ : the fitted  $W$  coefficients are 0.40–0.43 and the  $V1:W$  interactions are 0.53–0.55. In particular the  $W$  coefficient is essentially the true ATE,  $E[\max(V1, 0)] = \phi(0) = 0.3989$ . What a linear specification cannot do is see the kink at  $V1 = 0$  — and that is what the forest is for.

Now we use Causal Forest. The first graph is the predicted CATE against  $V1$ ; the second graph is the predicted CATE against true CATE. The third one adds in error bars.

The forest's ATE is 0.3732 (SE 0.0533) and 0.3722 (SE 0.0529), against the true 0.3989 — within half a standard error, and no better than the linear model's  $W$  coefficient at recovering the average. The gain is in the shape: the fitted CATE should flatten to zero for  $V1 < 0$  and rise linearly above it, which is the kink the linear interaction smooths straight through.

```
all_firms <- levels(factor(data$firm))
data.train$firm <- factor(data.train$firm, levels = all_firms)
data.test$firm <- factor(data.test$firm, levels = all_firms)
```

## 17. Causal Forest in panel data

```
# a is a data set of firm dummies
a <- as.data.frame(model.matrix(y~firm,data.train))
X <- as.matrix(bind_cols(data.train[,c(5,7:16)],a))
Y <- data.train$y
W <- data.train$W

#tau.forest = causal_forest(X, Y, W, num.trees = 4000)
a <- as.data.frame(model.matrix(y~firm,data.test))
X.test <- as.matrix(bind_cols(data.test[,c(5,7:16)],a))
Y.test <- data.test$y
W.test <- data.test$W

tau.forest3 = causal_forest(X, Y, W, clusters = as.integer(data.train$firm))

average_treatment_effect(tau.forest3, target.sample = "all")
```

```
estimate      std.err
0.37320336 0.05326215
```

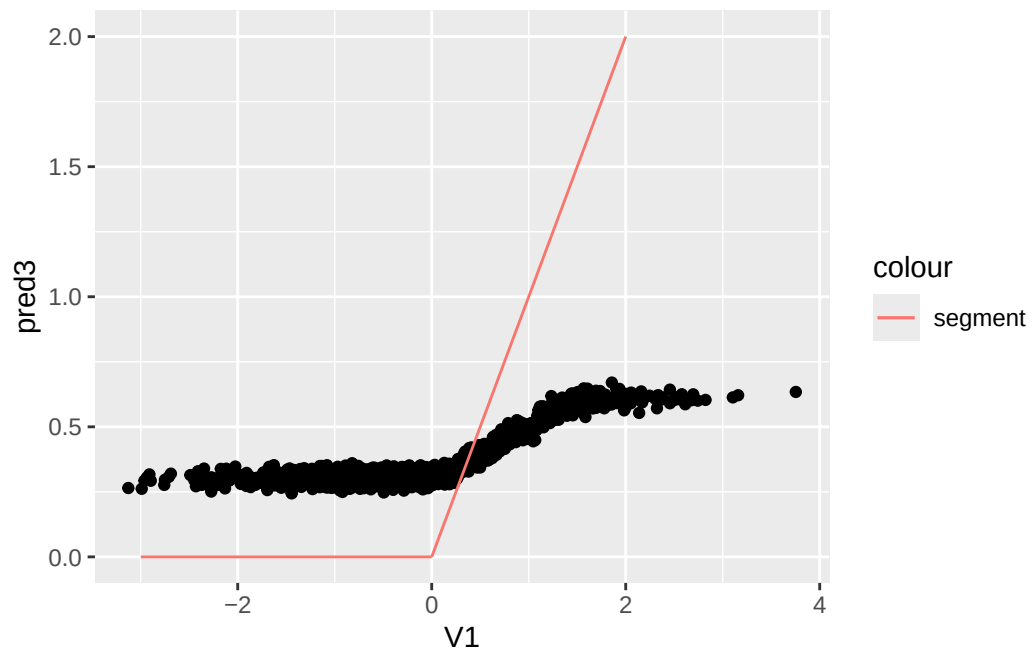
```
average_treatment_effect(tau.forest3, target.sample = "treated")
```

```
estimate      std.err
0.37223789 0.05285912
```

```
#tau.forest3 = causal_forest(X, Y, W, num.trees = 4000)
tau.hat3 = predict(tau.forest3, X.test, estimate.variance = TRUE)
sigma.hat3 = sqrt(tau.hat3$variance.estimates)

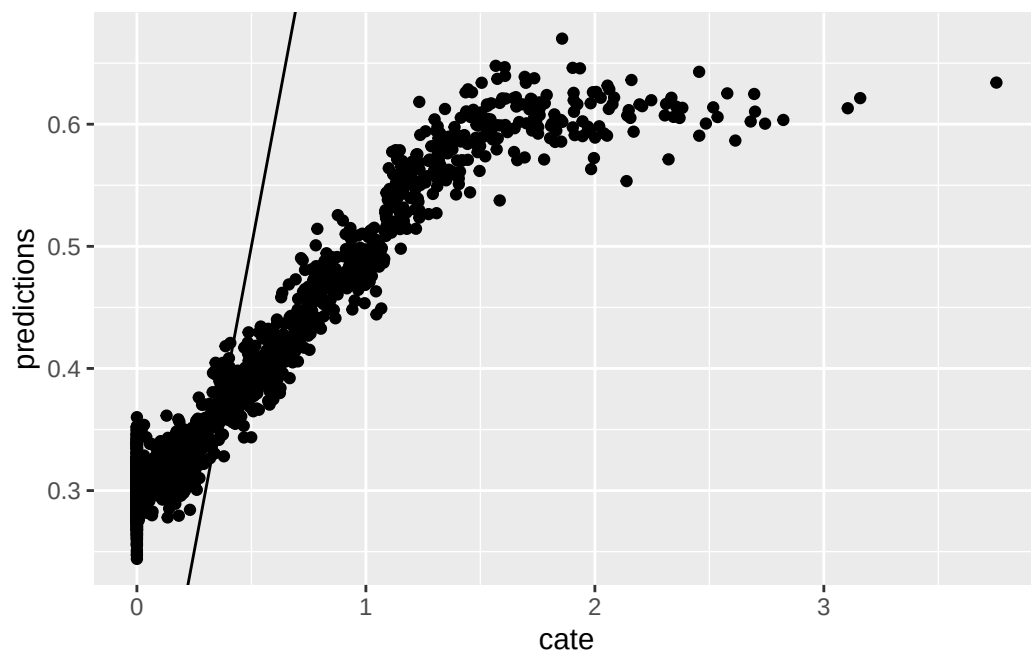
data.test$pred3 <- tau.hat3$predictions

seg1 <- data.frame(x1 = -3, x2 = 0, y1 = 0, y2 = 0)
seg2 <- data.frame(x1 = 0, x2 = 2, y1 = 0, y2 = 2)
ggplot(data.test, aes(x=V1, y=pred3)) + geom_point() +
geom_segment(aes(x = x1, y = y1, xend = x2, yend = y2, colour = "segment"), data = seg1)+
geom_segment(aes(x = x1, y = y1, xend = x2, yend = y2, colour = "segment"), data = seg2)
```



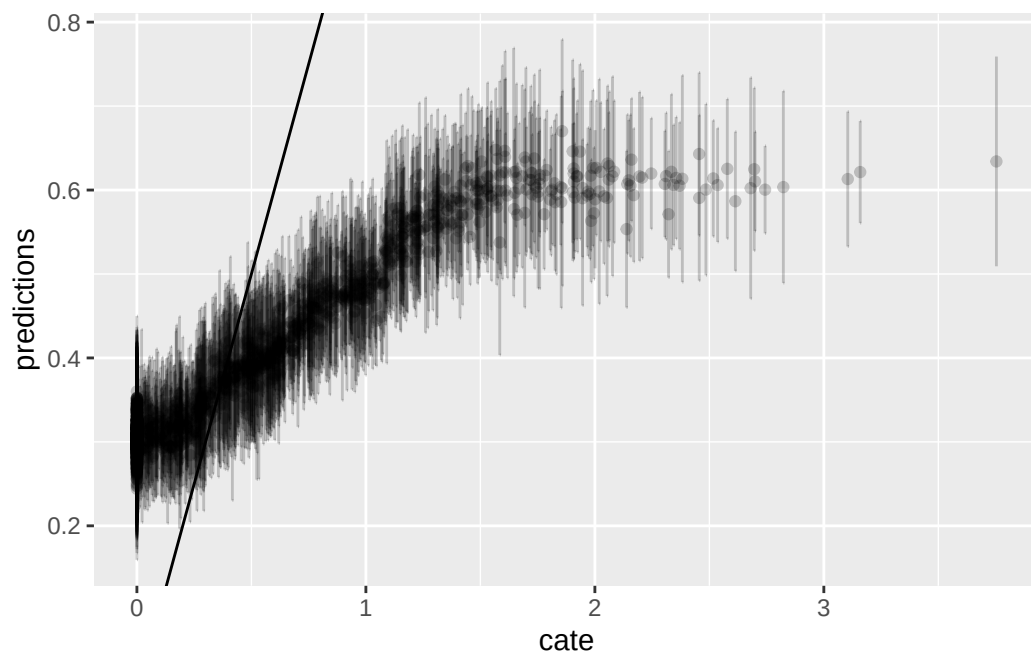
```
graph.data3 <- as.data.frame(tau.hat3)
graph.data3 <- graph.data3 %>%
  mutate(cate=pmax(data.test$V1,0)) %>%
  mutate(lower=predictions - sigma.hat3, upper=predictions + sigma.hat3)

ggplot(graph.data3, aes(x=cate, y=predictions)) + geom_point() + geom_abline(slope = 1)
```



## 17. Causal Forest in panel data

```
ggplot() + geom_errorbar(graph.data3, mapping=aes(x=cate, ymin=lower,ymax=upper), alpha = 0.2)
```



### 17.4. Nonlinear effect case 2

In the second situation, we assume DGP:

$$y_{it} = 1 + \alpha_i + \log(V1_{it}^2) * W_{it} + \min(X3_{it}, 0) + \epsilon_{it} \quad (17.3)$$

But we only observe  $V1$ , not the log of the squared  $V1$ . The extra  $\min(X3_{it}, 0)$  term is a nonlinearity in the *outcome* rather than in the treatment effect, so it leaves the CATE equal to  $\log(V1^2)$ ; it is there to give the forest's nuisance models something nonlinear to fit.

```
# if we have a nonlinear effect
# suppose we have the same X, but y is generated nonlinearly.
# another example: we observe V1, but the true treatment effect is log(V1^2)
set.seed(666)
rm(list = ls())
# t is no. of periods. p is no. of variables, n is no. of firms.
t <- 10
p <- 10
n=400
# generate p random variables
data1 = as.data.frame(matrix(rnorm(n*t*p), n*t, p))
names(data1) <- paste0("X", 1:p)
```

```

firm=seq(1,n)
time=seq(1,t)
data <- expand.grid(firm=firm,time=time)

# W is the treatment assignment
data$W <- rbinom(n*t, 1, 0.5)
data$train <- rbinom(n*t, 1, 0.5)

# generate two correlated variables
covarMat = matrix( c(1, .95^2, .95^2, 1 ) , nrow=2 , ncol=2 )
data.2 = as.data.frame(mvrnorm(n=n*t , mu=rep(0,2), Sigma=covarMat ))
names(data.2) <- c("V1", "V2")

data <- bind_cols(data,data.2) %>%
  tibble()

data <- bind_cols(data,data1)

# generate unit effect by firm means of V2, which is correlated with V1
data <- data %>%
  group_by(firm) %>%
  mutate(unit.effect=mean(V2)) %>%
  ungroup() %>%
  arrange(firm, time)

y<-c()
unit<-c()
X<-c()
k <- 0
for(i in 1:n){
  for(j in 1:t){
    k<-k+1
    unit[k]<-i
    y[k]<-1+ log(data$V1[k]^2)*data$W[k] + pmin(data$X3[k], 0) + data$unit.effect[k]+rnorm
  }
}
data$y=y
data$unit <- unit

data.train <- data %>%
  filter(train==1)
data.test <- data %>%
  filter(train==0)

# run fixed effect, random effect, and ols to see whether they are

```

## 17. Causal Forest in panel data

```
# biased.  
# they all perform poorly  
fe <- felm(y ~ V1 * W | unit, data=data.train)  
re <- lmer(y~V1*W+(1 | unit), data=data.train)  
ols <- lm(y ~ V1 * W , data=data.train)  
summary(fe)
```

Call:

```
felm(formula = y ~ V1 * W | unit, data = data.train)
```

Residuals:

|  | Min      | 1Q      | Median | 3Q     | Max    |
|--|----------|---------|--------|--------|--------|
|  | -10.0383 | -0.9478 | 0.0907 | 1.0865 | 5.3884 |

Coefficients:

|      | Estimate | Std. Error | t value | Pr(> t )   |
|------|----------|------------|---------|------------|
| V1   | -0.01979 | 0.07088    | -0.279  | 0.780      |
| W    | -1.17812 | 0.09632    | -12.231 | <2e-16 *** |
| V1:W | 0.01450  | 0.09904    | 0.146   | 0.884      |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.925 on 1612 degrees of freedom

Multiple R-squared(full model): 0.2876 Adjusted R-squared: 0.1104

Multiple R-squared(proj model): 0.08495 Adjusted R-squared: -0.1427

F-statistic(full model):1.623 on 401 and 1612 DF, p-value: 6.032e-11

F-statistic(proj model): 49.88 on 3 and 1612 DF, p-value: < 2.2e-16

```
summary(re)
```

Linear mixed model fit by REML ['lmerMod']

Formula: y ~ V1 \* W + (1 | unit)

Data: data.train

REML criterion at convergence: 8421.8

Scaled residuals:

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -6.1184 | -0.4977 | 0.0647 | 0.6098 | 2.8959 |

Random effects:

| Groups   | Name        | Variance | Std.Dev. |
|----------|-------------|----------|----------|
| unit     | (Intercept) | 0.09312  | 0.3052   |
| Residual |             | 3.72467  | 1.9299   |

Number of obs: 2014, groups: unit, 399



Fixed effects:

|             | Estimate | Std. Error | t value |
|-------------|----------|------------|---------|
| (Intercept) | 0.57512  | 0.06301    | 9.128   |
| V1          | 0.06152  | 0.06490    | 0.948   |
| W           | -1.18310 | 0.08706    | -13.590 |
| V1:W        | -0.01892 | 0.08945    | -0.212  |

Correlation of Fixed Effects:

|      | (Intr) | V1     | W     |
|------|--------|--------|-------|
| V1   | 0.015  |        |       |
| W    | -0.685 | -0.010 |       |
| V1:W | -0.009 | -0.727 | 0.028 |

```
summary(ols)
```

Call:

```
lm(formula = y ~ V1 * W, data = data.train)
```

Residuals:

| Min      | 1Q      | Median | 3Q     | Max    |
|----------|---------|--------|--------|--------|
| -12.1215 | -0.9805 | 0.1390 | 1.1999 | 5.7125 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | 0.57618  | 0.06136    | 9.390   | <2e-16 *** |
| V1          | 0.07086  | 0.06502    | 1.090   | 0.276      |
| W           | -1.18331 | 0.08712    | -13.582 | <2e-16 *** |
| V1:W        | -0.02005 | 0.08949    | -0.224  | 0.823      |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.954 on 2010 degrees of freedom

Multiple R-squared: 0.08508, Adjusted R-squared: 0.08372

F-statistic: 62.31 on 3 and 2010 DF, p-value: < 2.2e-16

Here the linear models fail more completely, and instructively so. Because  $\log(V1^2)$  is *even* in  $V1$ , its linear interaction with  $V1$  is zero by construction: the fitted  $V1:W$  terms come out at 0.015,  $-0.019$  and  $-0.020$ , i.e. no detectable heterogeneity at all, even though the true CATE varies enormously with  $V1$ . The  $W$  coefficients ( $\approx -1.18$ ) still approximate the true ATE,  $E[\log(V1^2)] = \psi(1/2) + \log 2 = -1.2704$ . So a linear model gets the average right and the heterogeneity completely wrong.

Now we run Causal Forest. Again, first graph is predicted CATE against  $V1$ , the second one is the predicted CATE against true CATE, the third one adds error bars.

## 17. Causal Forest in panel data

The forest's ATE is  $-1.2049$  (SE  $0.0813$ ) and  $-1.2024$  (SE  $0.0798$ ) for the two variants, against the true  $-1.2704$ . Like the linear models, it gets the average about right. The difference is in the plots: the forest's fitted CATE traces the U-shape of  $\log(V1^2)$ , which no linear interaction can represent.

```
all_firms <- levels(factor(data$firm))
data.train$firm <- factor(data.train$firm, levels = all_firms)
data.test$firm <- factor(data.test$firm, levels = all_firms)

# a is a data set of firm dummies
a <- as.data.frame(model.matrix(y~firm,data.train))
X <- as.matrix(bind_cols(data.train[,c(5,7:16)],a))
Y <- data.train$y
W <- data.train$W

#tau.forest = causal_forest(X, Y, W, num.trees = 4000)
a <- as.data.frame(model.matrix(y~firm,data.test))
X.test <- as.matrix(bind_cols(data.test[,c(5,7:16)],a))
Y.test <- data.test$y
W.test <- data.test$W

tau.forest4 = causal_forest(X, Y, W, clusters = as.integer(data.train$firm))

average_treatment_effect(tau.forest4, target.sample = "all")
```

```
      estimate      std.err
-1.20491164  0.08132368
```

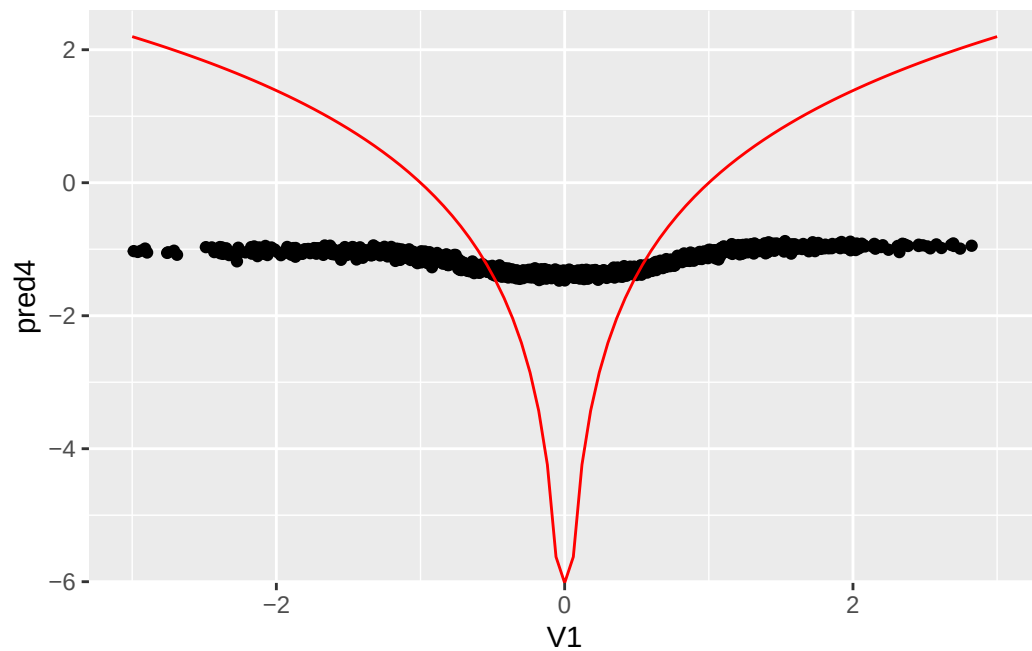
```
average_treatment_effect(tau.forest4, target.sample = "treated")
```

```
      estimate      std.err
-1.20241393  0.07981497
```

```
#tau.forest3 = causal_forest(X, Y, W, num.trees = 4000)
tau.hat4 = predict(tau.forest4, X.test, estimate.variance = TRUE)
sigma.hat4 = sqrt(tau.hat4$variance.estimates)

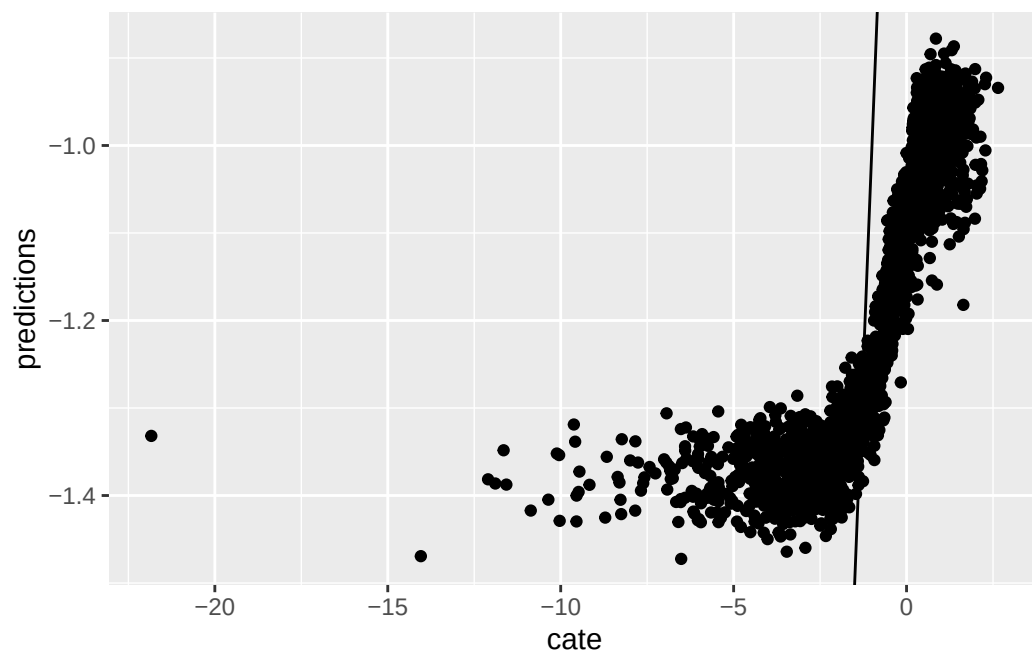
data.test$pred4 <- tau.hat4$predictions

fun.1 <- function(x) log(x^2)
ggplot(data.test, aes(x=V1, y=pred4)) + geom_point() +
stat_function(fun = fun.1, colour="red") + xlim(-3,3)
```



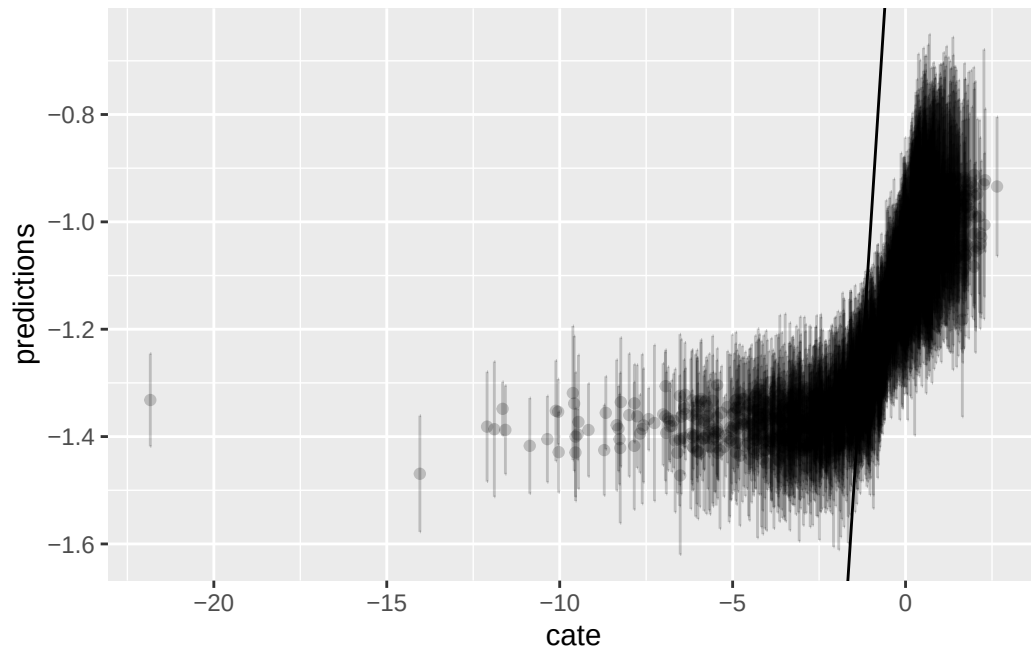
```
graph.data4 <- as.data.frame(tau.hat4)
graph.data4 <- graph.data4 %>%
  mutate(cate=log(data.test$V1^2)) %>%
  mutate(lower=predictions - sigma.hat4, upper=predictions + sigma.hat4)

ggplot(graph.data4, aes(x=cate, y=predictions)) + geom_point() + geom_abline(slope = 1)
```



## 17. Causal Forest in panel data

```
ggplot() + geom_errorbar(graph.data4, mapping=aes(x=cate, ymin=lower,ymax=upper), alpha = 0.2,
```



### 17.5. Summary

We see in these examples, Causal Forest performs fairly well, while other linear models will not work in the nonlinear DGP situations.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 18. Synthetic Control in R and Stata

### 18.1. Synthetic control

Abadie et al. have a few papers on synthetic control — Abadie, Diamond and Hainmueller (2010, *JASA*) and (2015, *AJPS*), and Abadie’s (2021, *JEL*) survey. The key idea is for a case study situation. A single unit, say a state/firm/country is exposed to a shock, and we are interested in the effect of that shock. For example, Card and Krueger (1994) studied the effect of a minimum wage increase in New Jersey. They used diff-in-diff, with Pennsylvania as the control state. The point of synthetic control is that maybe parallel trend assumption does not hold for the control unit. Is Pennsylvania a good control for New Jersey? Does it follow the same trend as New Jersey? Maybe not. Synthetic control is to generate a more comparable control with some combination of multiple control units, which can have a much closer parallel trend as the treated unit, than using any of the control units.

Now, how to construct this synthetic control unit? Basically it is a weighted average of multiple control units. There are two sets of weights to be determined. One set for the control units, one set to determine the importance of predictors. Abadie et al (2010) propose to choose a set of  $w_i$ ’s so that the resulting synthetic control best resembles the pre-intervention values for the treated unit of predictors of the outcome variable, subject to the constraints that the weights are non-negative and sum to 1. Another set of weights  $v_h$  for predictor importance is determined such that it minimizes the mean squared prediction error (MSPE) of this synthetic control with respect to the outcome in the pre-treatment period. The intuition of these weights are to pick  $w_i$ ’s to make the weighted sum of each predictor as close to the treated unit’s value of predictor as close as possible, given a set of  $v_h$ ’s. Then  $v_h$ ’s are determined by how important each predictor is to predict the outcome. We don’t use any information post-treatment to determine the weights. But we can use pre-treatment outcome to see the relative importance of each predictor in terms of predictor outcome. We can do this by out-of-sample validation. Basically divide the pre-treatment period into training and validation periods (that’s one reason that the pre-treatment period cannot be too short). Iterate the process of choosing  $w_i$  and  $v_h$  to achieve a minimization of MSPE in the validation period.

### 18.2. inference

Since synthetic control is for case study situation, we have essentially one treated unit and one synthetic control. The usual inference method would not work. Abadie et al suggested a permutation based approach. The basic idea is to permute the label of treatment; in other words, suppose one control unit is now a treated unit. Then follow the same procedure to get the synthetic control for that “treated unit”; look at the effect. Now if the treatment is truly for the treated unit only, then there would not be any effect for all the control unit. Therefore we can have a distribution of treatment effects after permutation of treatment label.

### 18.3. an example

The canonical synthetic control is implemented in both R and Stata (the `Synth` package, [CRAN](#); see Abadie, Diamond and Hainmueller's [JSS paper](#) for the software description). We use a package called “tidysynth” here to study one of the original examples, which evaluates the impact of Proposition 99 on cigarette consumption in California.

```
library(tidysynth)
library(dplyr)
library(tidyverse)
library(ggplot2)
library(tidyr)
data("smoking")
smoking %>% dplyr::glimpse()
```

Rows: 1,209

Columns: 7

```
$ state      <chr> "Rhode Island", "Tennessee", "Indiana", "Nevada", "Louisiana~
$ year       <dbl> 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, 1970, ~
$ cigsale    <dbl> 123.9, 99.8, 134.6, 189.5, 115.9, 108.4, 265.7, 93.8, 100.3, ~
$ lnincome   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ beer       <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ age15to24  <dbl> 0.1831579, 0.1780438, 0.1765159, 0.1615542, 0.1851852, 0.175~
$ retprice   <dbl> 39.3, 39.9, 30.6, 38.9, 34.3, 38.4, 31.4, 37.3, 36.7, 28.8, ~
```

The panel is 1,209 state-years: 39 states from 1970 to 2000, with per-capita cigarette sales in packs as the outcome and log income, beer sales, the share aged 15–24, and retail price as predictors. California passed Proposition 99 in 1988, so there are 19 pre-treatment years to fit the weights and 12 post-treatment years to measure the effect.

```
smoking_out <-
  smoking %>%
  # initial the synthetic control object
  synthetic_control(outcome = cigsale, # outcome
                    unit = state, # unit index in the panel data
                    time = year, # time index in the panel data
                    i_unit = "California", # unit where the intervention occurred
                    i_time = 1988, # time period when the intervention occurred
                    generate_placebos=T # generate placebo synthetic controls (for inference)
  ) %>%

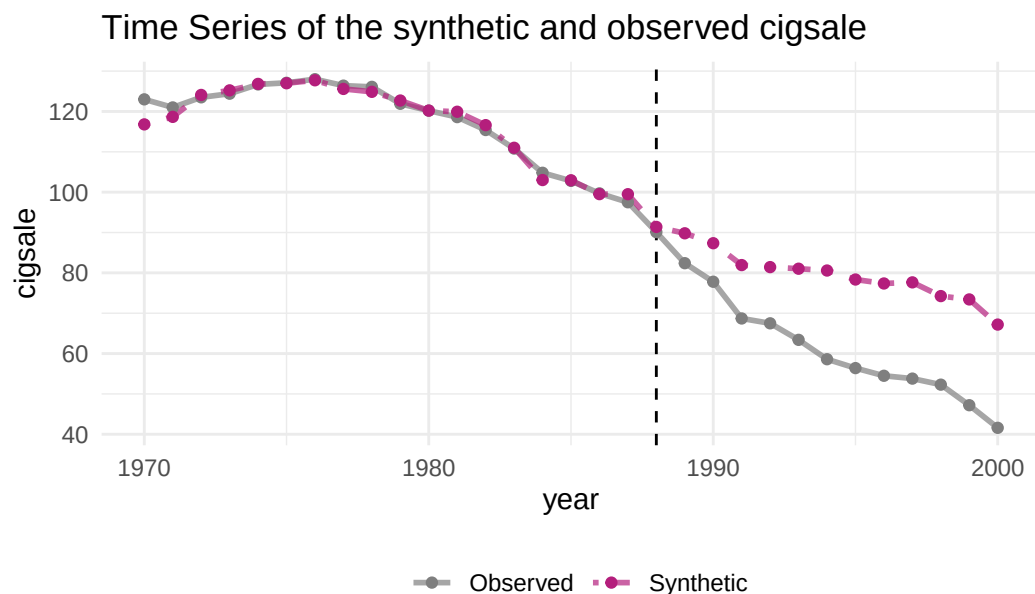
  # Generate the aggregate predictors used to fit the weights
  # average log income, retail price of cigarettes, and proportion of the
  # population between 15 and 24 years of age from 1980 - 1988
  generate_predictor(time_window = 1980:1988,
                    ln_income = mean(lnincome, na.rm = T),
                    ret_price = mean(retprice, na.rm = T),
```

```

        youth = mean(age15to24, na.rm = T)) %>%
# average beer consumption in the donor pool from 1984 - 1988
generate_predictor(time_window = 1984:1988,
        beer_sales = mean(beer, na.rm = T)) %>%
# Lagged cigarette sales
generate_predictor(time_window = 1975,
        cigsale_1975 = cigsale) %>%
generate_predictor(time_window = 1980,
        cigsale_1980 = cigsale) %>%
generate_predictor(time_window = 1988,
        cigsale_1988 = cigsale) %>%
# Generate the fitted weights for the synthetic control
generate_weights(optimization_window = 1970:1988, # time to use in the optimization task
        margin_ipop = .02, sigf_ipop = 7, bound_ipop = 6 # optimizer options
) %>%
# Generate the synthetic control
generate_control()

smoking_out %>% plot_trends()

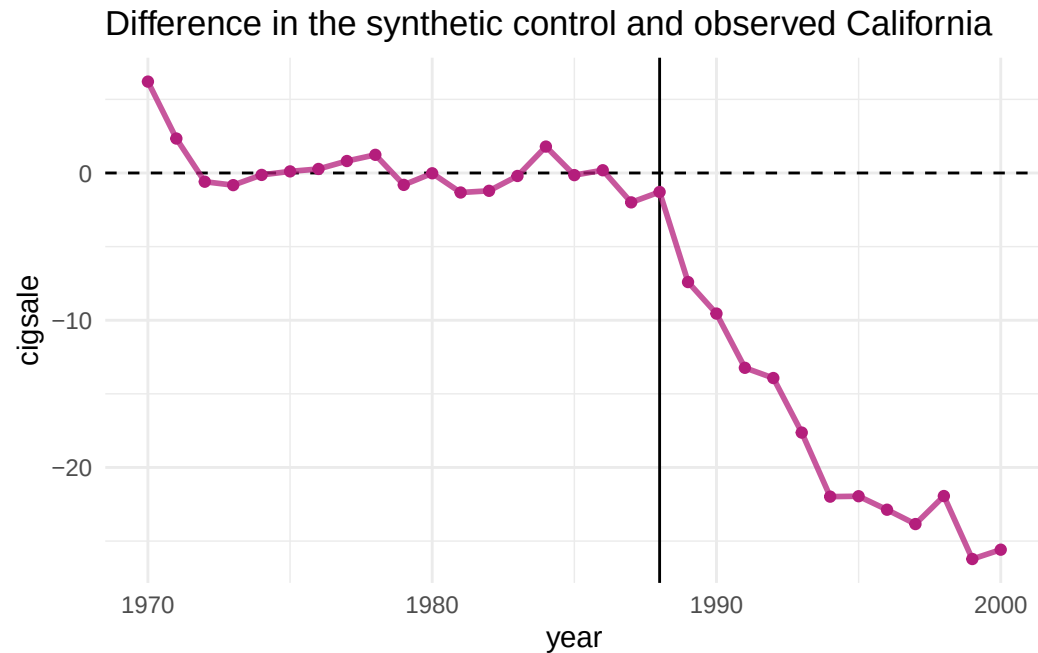
```



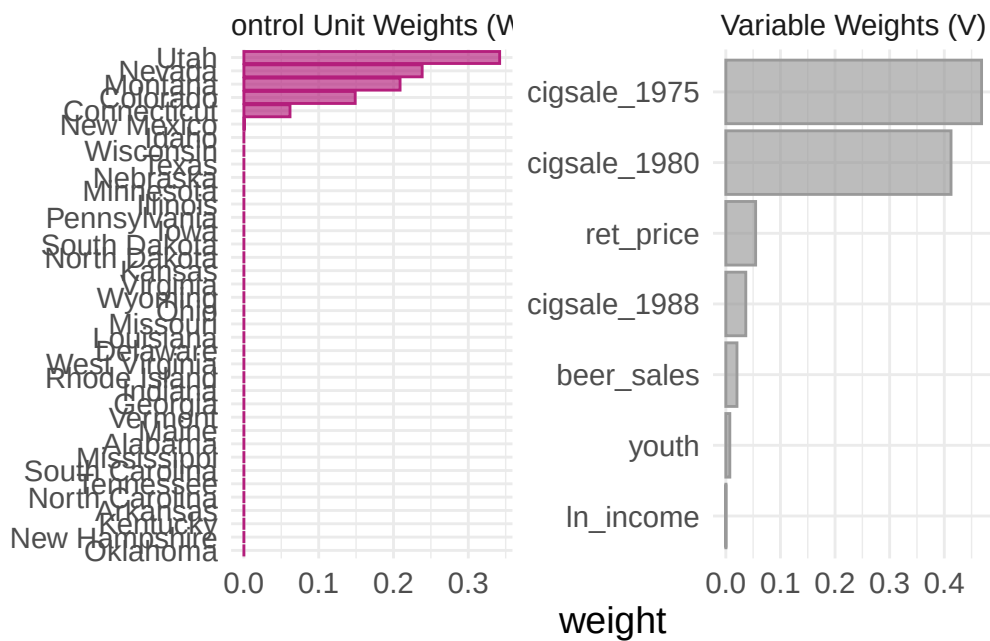
Dashed line denotes the time of the intervention.

```
smoking_out %>% plot_differences()
```

## 18. Synthetic Control in R and Stata



```
smoking_out %>% plot_weights()
```



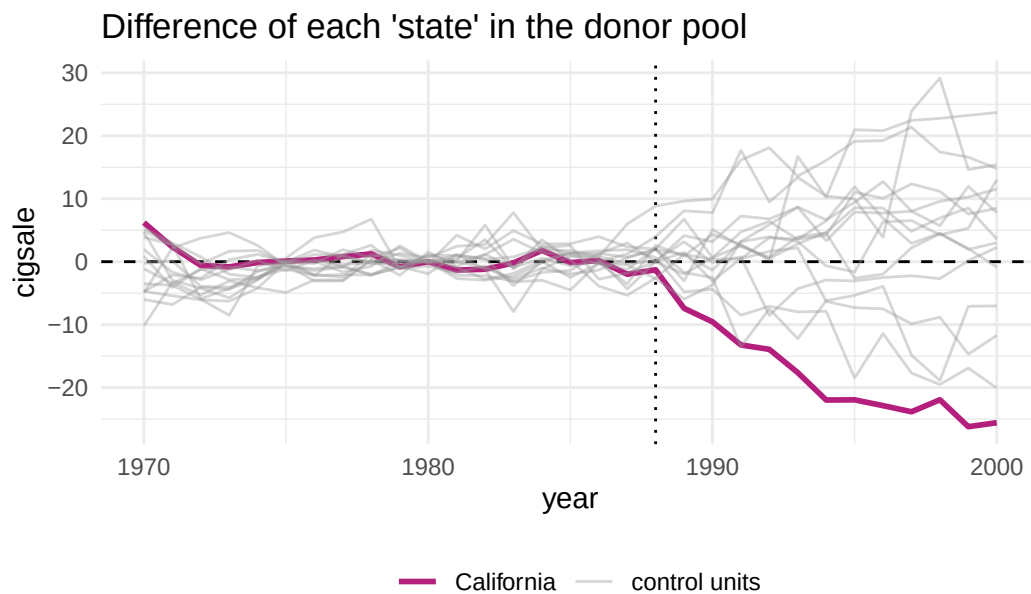
```
smoking_out %>% grab_balance_table()
```

```
# A tibble: 7 x 4
  variable      California synthetic_California donor_sample
  <chr>          <dbl>          <dbl>          <dbl>
1 cigsale_1975  0.00000000    0.00000000    0.00000000
2 cigsale_1980  0.00000000    0.00000000    0.00000000
3 ret_price     0.00000000    0.00000000    0.00000000
4 cigsale_1988  0.00000000    0.00000000    0.00000000
5 beer_sales    0.00000000    0.00000000    0.00000000
6 youth         0.00000000    0.00000000    0.00000000
7 ln_income     0.00000000    0.00000000    0.00000000
```



|   |              |       |       |       |
|---|--------------|-------|-------|-------|
| 1 | ln_income    | 10.1  | 9.85  | 9.83  |
| 2 | ret_price    | 89.4  | 89.4  | 87.3  |
| 3 | youth        | 0.174 | 0.174 | 0.173 |
| 4 | beer_sales   | 24.3  | 24.2  | 23.7  |
| 5 | cigsale_1975 | 127.  | 127.  | 137.  |
| 6 | cigsale_1980 | 120.  | 120.  | 138.  |
| 7 | cigsale_1988 | 90.1  | 91.4  | 114.  |

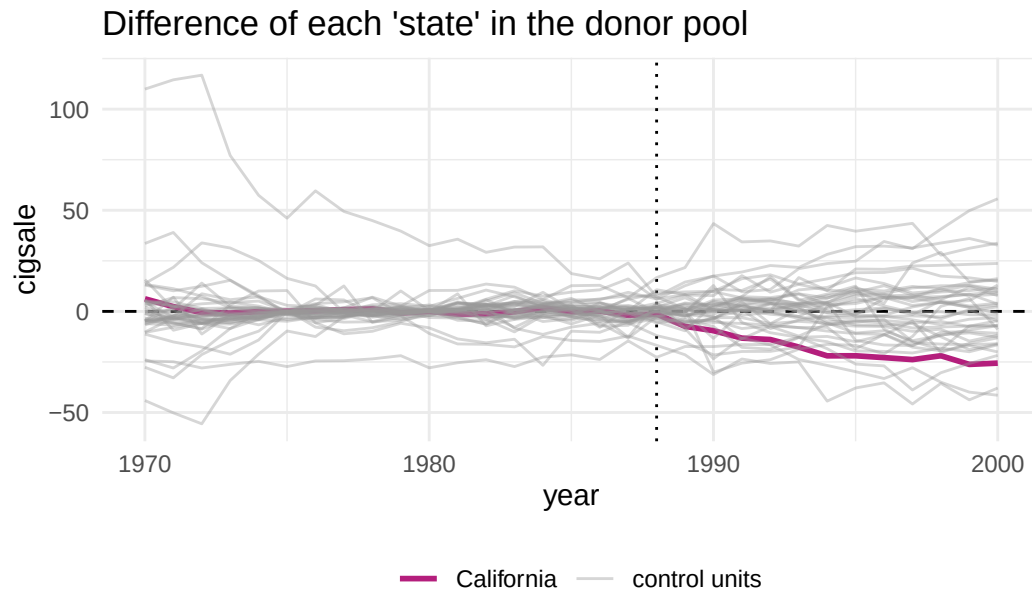
```
smoking_out %>% plot_placebos()
```



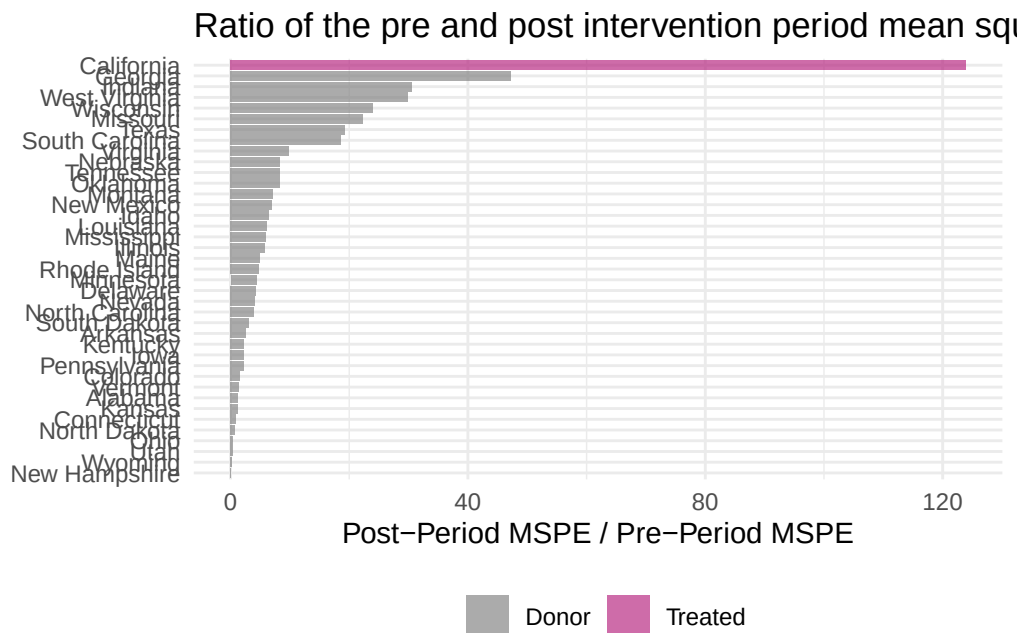
prune cases with a pre-period RMSPE exceeding two times the treated unit's pre-period RMSPE.

```
smoking_out %>% plot_placebos(prune = FALSE)
```

## 18. Synthetic Control in R and Stata



```
smoking_out %>% plot_mspe_ratio()
```



```
smoking_out %>% grab_significance()
```

```
# A tibble: 39 x 8
```

```
  unit_name      type pre_mspe post_mspe mspe_ratio rank fishers_exact_pvalue
  <chr>          <chr>   <dbl>   <dbl>     <dbl> <int>          <dbl>
```

```

1 California    Trea~    3.17    392.    124.    1    0.0256
2 Georgia      Donor    3.79    179.    47.2    2    0.0513
3 Indiana      Donor   25.2    770.    30.6    3    0.0769
4 West Virginia Donor    9.52    284.    29.8    4    0.103
5 Wisconsin    Donor   11.1    268.    24.1    5    0.128
6 Missouri     Donor    3.03    67.8    22.4    6    0.154
7 Texas        Donor   14.4    277.    19.3    7    0.179
8 South Carolina Donor   12.6    234.    18.6    8    0.205
9 Virginia     Donor    9.81    96.4    9.83    9    0.231
10 Nebraska     Donor    6.30    52.9    8.40   10    0.256
# i 29 more rows
# i 1 more variable: z_score <dbl>

```

```
smoking_out %>% grab_synthetic_control()
```

```

# A tibble: 31 x 3
  time_unit real_y synth_y
  <dbl>    <dbl>    <dbl>
1   1970    123    117.
2   1971    121    119.
3   1972   124.   124.
4   1973   124.   125.
5   1974   127.   127.
6   1975   127.   127.
7   1976   128.   128.
8   1977   126.   126.
9   1978   126.   125.
10  1979   122.   123.
# i 21 more rows

```

```
smoking_out %>% grab_synthetic_control(placeholder = T)
```

```

# A tibble: 1,209 x 5
  .id      .placeholder time_unit real_y synth_y
  <chr>      <dbl>    <dbl>    <dbl>    <dbl>
1 California      0    1970    123    117.
2 California      0    1971    121    119.
3 California      0    1972   124.   124.
4 California      0    1973   124.   125.
5 California      0    1974   127.   127.
6 California      0    1975   127.   127.
7 California      0    1976   128.   128.
8 California      0    1977   126.   126.
9 California      0    1978   126.   125.
10 California      0    1979   122.   123.
# i 1,199 more rows

```

```
smoking_out %>%
  tidyr::unnest(cols = c(.outcome))
```

```
# A tibble: 1,482 x 50
```

|    | .id        | .placebo | .type   | time_unit | California | Alabama | Arkansas | Colorado |
|----|------------|----------|---------|-----------|------------|---------|----------|----------|
|    | <chr>      | <dbl>    | <chr>   | <dbl>     | <dbl>      | <dbl>   | <dbl>    | <dbl>    |
| 1  | California | 0        | treated | 1970      | 123        | NA      | NA       | NA       |
| 2  | California | 0        | treated | 1971      | 121        | NA      | NA       | NA       |
| 3  | California | 0        | treated | 1972      | 124.       | NA      | NA       | NA       |
| 4  | California | 0        | treated | 1973      | 124.       | NA      | NA       | NA       |
| 5  | California | 0        | treated | 1974      | 127.       | NA      | NA       | NA       |
| 6  | California | 0        | treated | 1975      | 127.       | NA      | NA       | NA       |
| 7  | California | 0        | treated | 1976      | 128        | NA      | NA       | NA       |
| 8  | California | 0        | treated | 1977      | 126.       | NA      | NA       | NA       |
| 9  | California | 0        | treated | 1978      | 126.       | NA      | NA       | NA       |
| 10 | California | 0        | treated | 1979      | 122.       | NA      | NA       | NA       |

```
# i 1,472 more rows
```

```
# i 42 more variables: Connecticut <dbl>, Delaware <dbl>, Georgia <dbl>,
#   Idaho <dbl>, Illinois <dbl>, Indiana <dbl>, Iowa <dbl>, Kansas <dbl>,
#   Kentucky <dbl>, Louisiana <dbl>, Maine <dbl>, Minnesota <dbl>,
#   Mississippi <dbl>, Missouri <dbl>, Montana <dbl>, Nebraska <dbl>,
#   Nevada <dbl>, `New Hampshire` <dbl>, `New Mexico` <dbl>,
#   `North Carolina` <dbl>, `North Dakota` <dbl>, Ohio <dbl>, ...
```

The balance table is where to judge whether the synthetic control is worth trusting, and it does well. Retail price matches to three digits (89.4 against 89.4), the youth share to 0.174 against 0.174, beer sales 24.3 against 24.2. The pre-treatment outcomes match closely too: 1975 cigarette sales 127 against 127 and 1980 sales 120 against 120. Compare those with the raw donor-pool averages in the third column — 137 and 138 — and the value of weighting is obvious. The one predictor that does not match is log income, 10.1 against 9.85, which the optimiser evidently judged less important for predicting the outcome.

The 1988 row is the one to read carefully: 90.1 against 91.4. That is the last pre-treatment year and it is fitted, not predicted, so it is a diagnostic rather than a result.

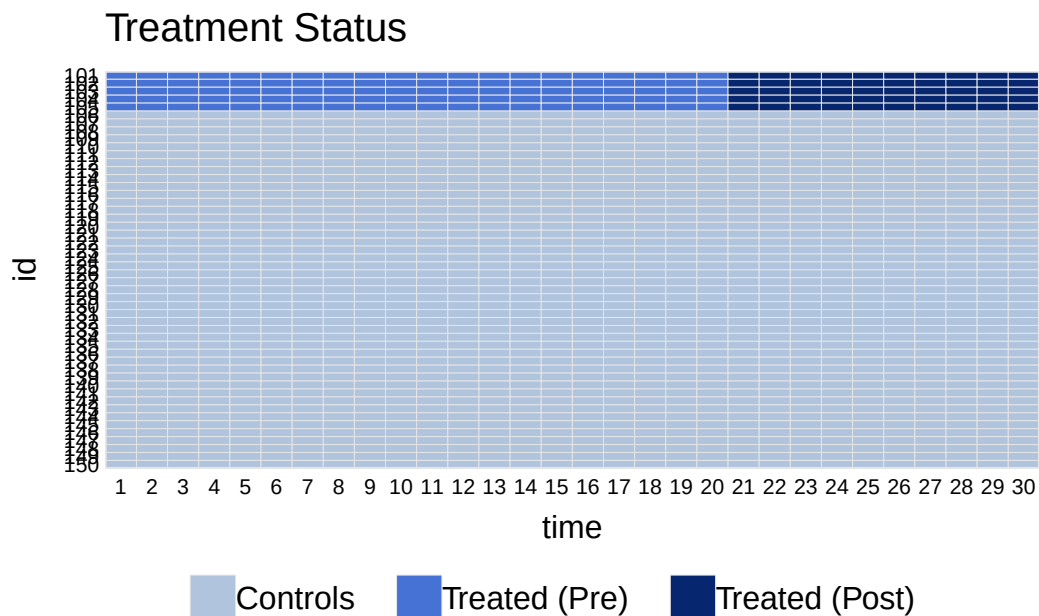
Inference comes from the placebo table. California's ratio of post- to pre-treatment MSPE is 124, the largest of all 39 units — Georgia is next at 47.2, less than half. Being ranked first of 39 gives a Fisher exact  $p$  of  $1/39 = 0.0256$ . This is a permutation statement, not a sampling one: it says no untreated state shows a post-1988 divergence from its own synthetic control as large as California's, relative to how well each was fitted beforehand.

## 18.4. Generalized Synthetic Control

The idea of generalized SC is to combine both an interactive fixed effect model and synthetic control. It is basically a panel data synthetic control. It naturally takes multiple treated units and uses bootstrap to get standard errors. It generates ATT with standard errors.

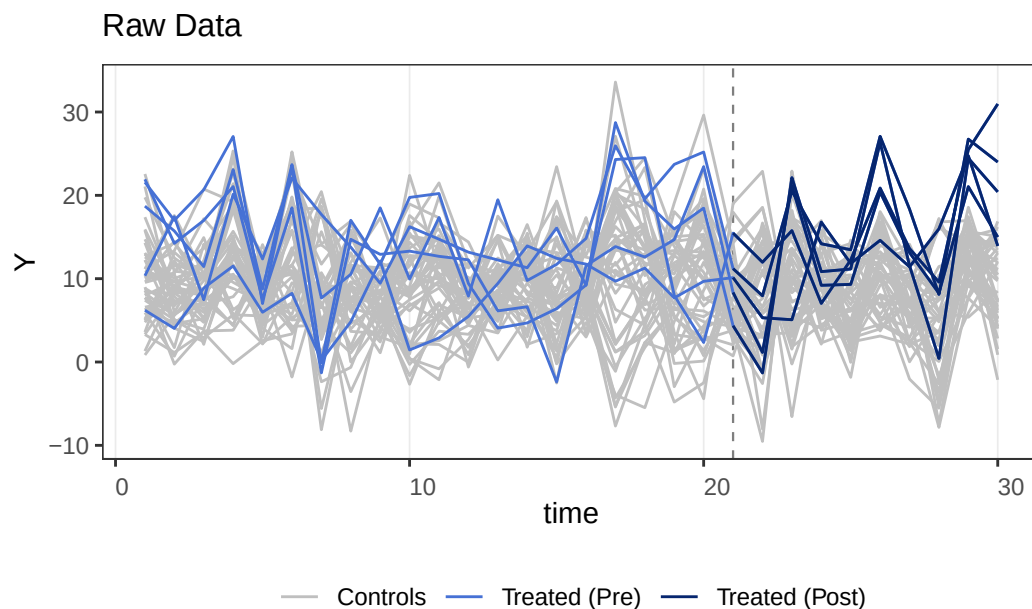
Examples are here: <https://yiqingxu.org/packages/gsynth/>

```
## for processing C++ code
require(Rcpp)
## for plotting
require(ggplot2)
require(GGally)
## for parallel computing
require(foreach)
require(future)
require(doParallel)
require(abind)
require(lfe)
library(gsynth)
## ## Syntax has been updated since v.1.2.0.
## ## Comments and suggestions -> yiqingxu@stanford.edu.
data(gsynth)
library(panelView)
## ## See bit.ly/panelview4r for more info.
## ## Report bugs -> yiqingxu@stanford.edu.
panelview(Y ~ D, data = simdata, index = c("id", "time"), pre.post = TRUE)
```



```
panelview(Y ~ D, data = simdata, index = c("id", "time"), type = "outcome")
```

## 18. Synthetic Control in R and Stata



```
system.time(
  out <- gsynth(Y ~ D + X1 + X2, data = simdata, index = c("id", "time"), force = "two-way",
)
```

```
user system elapsed
1.679  0.000  1.682
```

```
print(out)
```

Call:

```
gsynth(formula = Y ~ D + X1 + X2, data = simdata, index = c("id",
  "time"), force = "two-way", r = c(0, 5), CV = TRUE, se = TRUE,
  nboots = 100, inference = "parametric", parallel = FALSE,
  vartype = "parametric")
```

Average Treatment Effect on the Treated:

|      | ATT.avg | S.E.   | CI.lower | CI.upper | p.value |
|------|---------|--------|----------|----------|---------|
| [1,] | 5.084   | 0.3326 | 4.432    | 5.736    | 0       |

~ by Period (including Pre-treatment Periods):

|     | ATT      | S.E.   | CI.lower | CI.upper | p.value   | count |
|-----|----------|--------|----------|----------|-----------|-------|
| -19 | 0.05868  | 0.6716 | -1.25757 | 1.3749   | 9.304e-01 | 5     |
| -18 | -0.11741 | 0.4647 | -1.02814 | 0.7933   | 8.005e-01 | 5     |
| -17 | -0.89029 | 0.5334 | -1.93573 | 0.1551   | 9.510e-02 | 5     |
| -16 | 1.29462  | 0.5791 | 0.15967  | 2.4296   | 2.537e-02 | 5     |
| -15 | -1.20350 | 0.5627 | -2.30636 | -0.1006  | 3.245e-02 | 5     |
| -14 | 2.21992  | 0.7481 | 0.75369  | 3.6861   | 3.003e-03 | 5     |

|     |          |        |          |         |           |   |
|-----|----------|--------|----------|---------|-----------|---|
| -13 | -0.18469 | 0.7557 | -1.66574 | 1.2964  | 8.069e-01 | 5 |
| -12 | 0.87413  | 0.4833 | -0.07307 | 1.8213  | 7.049e-02 | 5 |
| -11 | -0.04824 | 0.6238 | -1.27081 | 1.1743  | 9.384e-01 | 5 |
| -10 | 0.84503  | 0.6792 | -0.48623 | 2.1763  | 2.135e-01 | 5 |
| -9  | 0.49754  | 0.6345 | -0.74604 | 1.7411  | 4.329e-01 | 5 |
| -8  | -0.38001 | 0.5696 | -1.49641 | 0.7364  | 5.047e-01 | 5 |
| -7  | -1.03493 | 0.6606 | -2.32974 | 0.2599  | 1.172e-01 | 5 |
| -6  | -0.81532 | 0.6910 | -2.16971 | 0.5391  | 2.381e-01 | 5 |
| -5  | -1.13329 | 0.6543 | -2.41574 | 0.1492  | 8.327e-02 | 5 |
| -4  | -0.17733 | 0.5964 | -1.34631 | 0.9917  | 7.662e-01 | 5 |
| -3  | 0.17671  | 0.8271 | -1.44445 | 1.7979  | 8.308e-01 | 5 |
| -2  | -0.14376 | 0.5958 | -1.31145 | 1.0239  | 8.093e-01 | 5 |
| -1  | -1.19697 | 0.4708 | -2.11966 | -0.2743 | 1.100e-02 | 5 |
| 0   | 1.35910  | 0.7762 | -0.16217 | 2.8804  | 7.994e-02 | 5 |
| 1   | 0.34558  | 0.5608 | -0.75364 | 1.4448  | 5.378e-01 | 5 |
| 2   | 0.15709  | 0.8485 | -1.50592 | 1.8201  | 8.531e-01 | 5 |
| 3   | 3.64127  | 0.5570 | 2.54966  | 4.7329  | 6.241e-11 | 5 |
| 4   | 2.74975  | 0.7010 | 1.37591  | 4.1236  | 8.750e-05 | 5 |
| 5   | 4.45569  | 0.8610 | 2.76823  | 6.1431  | 2.276e-07 | 5 |
| 6   | 5.67231  | 0.8205 | 4.06411  | 7.2805  | 4.745e-12 | 5 |
| 7   | 8.17039  | 0.6550 | 6.88661  | 9.4542  | 0.000e+00 | 5 |
| 8   | 6.57085  | 0.8954 | 4.81599  | 8.3257  | 2.154e-13 | 5 |
| 9   | 9.11624  | 0.5527 | 8.03299  | 10.1995 | 0.000e+00 | 5 |
| 10  | 9.96205  | 0.5275 | 8.92814  | 10.9960 | 0.000e+00 | 5 |

Coefficients for the Covariates:

|    | Coef  | S.E.    | CI.lower | CI.upper | p.value |
|----|-------|---------|----------|----------|---------|
| X1 | 1.454 | 0.03993 | 1.376    | 1.533    | 0       |
| X2 | 3.507 | 0.04104 | 3.427    | 3.588    | 0       |

out\$est.att

|     | ATT         | S.E.      | CI.lower    | CI.upper   | p.value      | count |
|-----|-------------|-----------|-------------|------------|--------------|-------|
| -19 | 0.05868370  | 0.6715682 | -1.25756576 | 1.3749332  | 9.303670e-01 | 5     |
| -18 | -0.11741144 | 0.4646683 | -1.02814458 | 0.7933217  | 8.005171e-01 | 5     |
| -17 | -0.89029173 | 0.5333967 | -1.93573004 | 0.1551466  | 9.509782e-02 | 5     |
| -16 | 1.29461792  | 0.5790639 | 0.15967354  | 2.4295623  | 2.537089e-02 | 5     |
| -15 | -1.20349647 | 0.5626948 | -2.30635803 | -0.1006349 | 3.245118e-02 | 5     |
| -14 | 2.21991721  | 0.7480902 | 0.75368734  | 3.6861471  | 3.002851e-03 | 5     |
| -13 | -0.18468804 | 0.7556547 | -1.66574411 | 1.2963680  | 8.069149e-01 | 5     |
| -12 | 0.87413414  | 0.4832772 | -0.07307177 | 1.8213401  | 7.048776e-02 | 5     |
| -11 | -0.04824208 | 0.6237714 | -1.27081152 | 1.1743274  | 9.383536e-01 | 5     |
| -10 | 0.84503247  | 0.6792288 | -0.48623146 | 2.1762964  | 2.134606e-01 | 5     |
| -9  | 0.49754000  | 0.6344913 | -0.74604016 | 1.7411202  | 4.329488e-01 | 5     |
| -8  | -0.38001372 | 0.5696025 | -1.49641402 | 0.7363866  | 5.046725e-01 | 5     |
| -7  | -1.03492639 | 0.6606315 | -2.32974035 | 0.2598876  | 1.172149e-01 | 5     |
| -6  | -0.81531903 | 0.6910261 | -2.16970525 | 0.5390672  | 2.380530e-01 | 5     |

## 18. Synthetic Control in R and Stata

|    |             |           |             |            |              |   |
|----|-------------|-----------|-------------|------------|--------------|---|
| -5 | -1.13328625 | 0.6543233 | -2.41573628 | 0.1491638  | 8.327401e-02 | 5 |
| -4 | -0.17733004 | 0.5964300 | -1.34631145 | 0.9916514  | 7.662229e-01 | 5 |
| -3 | 0.17670638  | 0.8271342 | -1.44444692 | 1.7978597  | 8.308302e-01 | 5 |
| -2 | -0.14376360 | 0.5957716 | -1.31145447 | 1.0239273  | 8.093175e-01 | 5 |
| -1 | -1.19696676 | 0.4707704 | -2.11965977 | -0.2742737 | 1.100405e-02 | 5 |
| 0  | 1.35910371  | 0.7761745 | -0.16217040 | 2.8803778  | 7.994100e-02 | 5 |
| 1  | 0.34558474  | 0.5608399 | -0.75364126 | 1.4448107  | 5.377682e-01 | 5 |
| 2  | 0.15709192  | 0.8484914 | -1.50592063 | 1.8201045  | 8.531172e-01 | 5 |
| 3  | 3.64127124  | 0.5569529 | 2.54966368  | 4.7328788  | 6.241185e-11 | 5 |
| 4  | 2.74975180  | 0.7009533 | 1.37590853  | 4.1235951  | 8.749879e-05 | 5 |
| 5  | 4.45569195  | 0.8609635 | 2.76823443  | 6.1431495  | 2.276196e-07 | 5 |
| 6  | 5.67231032  | 0.8205269 | 4.06410723  | 7.2805134  | 4.744871e-12 | 5 |
| 7  | 8.17038502  | 0.6550005 | 6.88660758  | 9.4541625  | 0.000000e+00 | 5 |
| 8  | 6.57084917  | 0.8953552 | 4.81598521  | 8.3257131  | 2.153833e-13 | 5 |
| 9  | 9.11624492  | 0.5526911 | 8.03299024  | 10.1994996 | 0.000000e+00 | 5 |
| 10 | 9.96204576  | 0.5275143 | 8.92813677  | 10.9959548 | 0.000000e+00 | 5 |

```
out$est.avg
```

|      | ATT.avg  | S.E.      | CI.lower | CI.upper | p.value |
|------|----------|-----------|----------|----------|---------|
| [1,] | 5.084123 | 0.3325502 | 4.432336 | 5.735909 | 0       |

```
out$est.beta
```

|    | Coef     | S.E.       | CI.lower | CI.upper | p.value |
|----|----------|------------|----------|----------|---------|
| X1 | 1.454447 | 0.03992589 | 1.376194 | 1.532701 | 0       |
| X2 | 3.507197 | 0.04104218 | 3.426756 | 3.587638 | 0       |

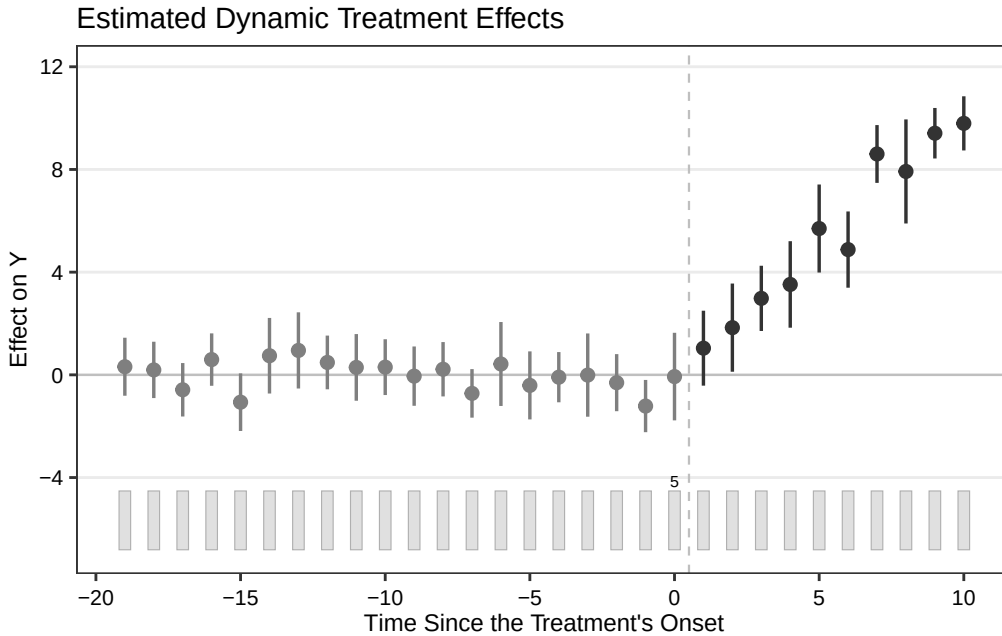
```
system.time(  
out <- gsynth(Y ~ D + X1 + X2, data = simdata, index = c("id","time"),  
  force = "two-way", CV = TRUE, r = c(0, 5), se = TRUE,  
  inference = "parametric", nboots = 100, parallel = TRUE, cores = 4)  
)
```

| user  | system | elapsed |
|-------|--------|---------|
| 7.655 | 0.344  | 17.941  |

```
# cumuEff was removed in newer gsynth versions (now a wrapper for fect)  
# cumu1 <- cumuEff(out, cumu = TRUE, id = NULL, period = c(0,5))  
# cumu1$est.catt
```

```
plot(out)
```





`simdata` is `gsynth`'s own simulated panel: 5 treated units and 45 controls over 30 periods, with treatment starting in period 21 for the treated group, an interactive fixed-effect structure, and two covariates.

The estimated average ATT is 5.084 with a standard error of 0.292 and a 95% interval of [4.512, 5.656]. The covariate coefficients come out at 1.454 for  $X_1$  and 3.507 for  $X_2$ . The period-by-period table is the part worth reading, because it includes the pre-treatment periods: those run from  $-19$  to  $-1$  and should be centred on zero if the interactive fixed-effect model has absorbed the common factors. They mostly are — 0.059,  $-0.117$ ,  $-0.890$ , 1.295,  $-1.203$ , 2.220 and so on — bouncing around zero with standard errors of about 0.5 to 0.7. Two of them ( $-16$  and  $-14$ ) are individually significant at 5%, which is roughly what one expects from nineteen independent tests at that level.

---

*Systematic treatment:* [Julia](#).



# 19. Same Data, Different Estimators: A Synthetic Control Comparison

## 19.1. One panel, four estimators

Here I use the Proposition 99 data to compare four panel estimators on the same outcome matrix: DiD, synthetic DiD, synthetic control, and TASC.

This is the standard California smoking example. There are 39 states, annual cigarette sales per capita from 1970 to 2000, and California is treated after the 1988 anti-smoking law. The useful thing about this example is that the estimators do not give exactly the same answer.

## 19.2. Four estimators, one panel

The panel is the California Proposition 99 data: annual per-capita cigarette sales in packs for 39 states from 1970 to 2000, arranged as a  $39 \times 31$  matrix with California in the treated row, 38 donor states, and 19 pre-treatment years before the 1989 tax increase. Every estimator below sees exactly the same matrix; all that differs is how each weights the donors and the pre-periods.

```
using CSV, DataFrames, Statistics
using SynthDiD
using TASC

df = CSV.read("california_prop99.csv", DataFrame)
# ... pivot to N×T matrix Y, NO = 38 donors, T0 = 19 pre-treatment years
# (full setup code below)

tau_did = did_estimate(Y, NO, T0)
tau_sdid = synthdid_estimate(Y, NO, T0)
tau_sc = sc_estimate(Y, NO, T0)

# TASC: treated unit must be row 1
Y_tasc = vcat(Y[(NO+1):end, :], Y[1:NO, :])
model = fit_tasc(Y_tasc; d=2, T0=T0, n_em=200, tol=1e-3)
pred = predict_counterfactual(model, Y_tasc)
```

| Estimator                 | ATT (packs per capita) |
|---------------------------|------------------------|
| Difference-in-differences | −27.35                 |

| Estimator         | ATT (packs per capita) |
|-------------------|------------------------|
| Synthetic control | -19.62                 |
| Synthetic DiD     | -15.60                 |
| TASC              | -17.11                 |

(These are the values computed live in the companion Julia book’s [same analysis](#); the code in this chapter is `eval: false`, so that chapter is the authoritative source for the numbers. The DiD and SC figures also match the published `synthdid` benchmarks for this panel.)

The smallest and largest estimates differ by nearly twelve packs per capita. That is not a rounding issue. It comes from the way each estimator constructs the counterfactual.

### 19.3. What changes across estimators

**DiD (-27.35)** assigns equal weight to every control state and every pre-treatment year. California is compared to the average of all 38 donor states. In this data, that average sits well below California before treatment. So the large DiD estimate is not automatically evidence of a larger causal effect. It is partly a warning that equal weights do not fit the pre-period well.

**Synthetic control (-19.62)** replaces equal weights with donor weights chosen to match California’s 1970-1988 cigarette sales. Once the pre-period fit is better, the post-treatment gap shrinks. The move from about -27 to about -20 is mostly the price of using a better control group.

**Synthetic DiD (-15.60)** adds time weights. The estimator can put more weight on the pre-treatment years that are more useful for the post-1988 counterfactual. Here that moves the estimate a bit closer to zero.

**TASC (-17.11)** does something different. Instead of choosing only weights, it fits a state-space model to the pre-treatment panel:

$$x_t = A x_{t-1} + q_t, \quad q_t \sim \mathcal{N}(0, Q) \quad (19.1)$$

$$y_t = H x_t + r_t, \quad r_t \sim \mathcal{N}(0, R) \quad (19.2)$$

The matrix  $A$  describes how the latent factors move over time. After treatment, the filter continues using the donor panel, and the treated unit’s row of  $H$  maps the latent state to California’s counterfactual. The TASC estimate (-17.11) lands between SC and SDiD in this example.

### 19.4. The figure

```

# Counterfactual path for each method. omega_* / intercept_* are the donor
# weights and (for DiD/SDiD) intercepts extracted from the fitted objects
# above (e.g. tau_sdid.weights.omega for SynthDiD.jl) - this snippet is
# schematic and omits that extraction step for brevity; it is not meant to
# run as-is (hence eval: false).
cfact_did = omega_did' * Y[1:NO, :] .+ intercept_did # parallel trends
cfact_sdid = omega_sdid' * Y[1:NO, :] .+ intercept_sdid
cfact_sc = omega_sc' * Y[1:NO, :] # no intercept
cfact_tasc = vec(pred.target)
tasc_se = sqrt.(max.(vec(pred.variance), 0.0))

```

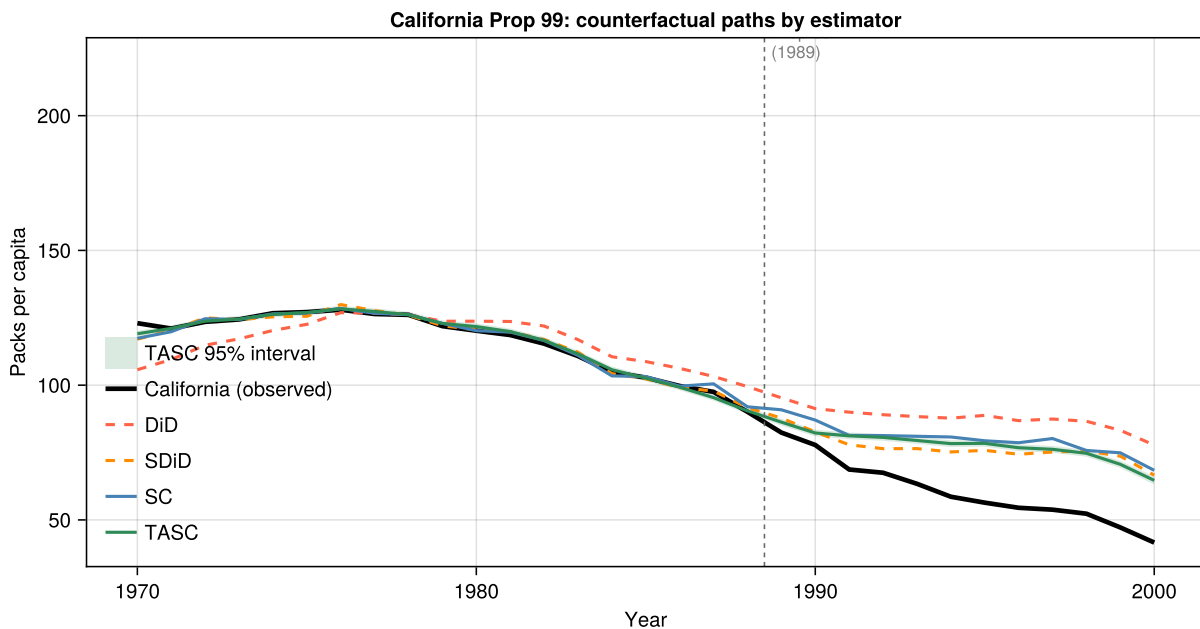


Figure 19.1.: Four counterfactual paths for California. Dashed lines (DiD, SDiD) use pre-period intercepts to enforce parallel trends; solid lines (SC, TASC) fit levels directly. The shaded band is the TASC 95% posterior interval  $\hat{Y}_{1t}(0) \pm 1.96 \hat{\sigma}_t$ .

Two things are visible in the figure that the table cannot show.

First, the pre-treatment paths differ. The DiD control average sits well below California before treatment. SC and TASC track California more closely. This is why the DiD post-treatment gap is larger.

Second, the TASC band is narrow before treatment and wider after 1989. That is what I would expect. Before treatment, the smoother can use information from both directions in time. After treatment, the filter has to move forward without California's observed outcomes, so the uncertainty accumulates through  $P_{t+1} = AP_t A^\top + Q$ .

## 19.5. What the comparison teaches

| Comparison      | What it isolates                                    |
|-----------------|-----------------------------------------------------|
| DiD vs SC       | Unit weights — equal vs optimised donor match       |
| SC vs SDiD      | Time weights — equal vs recency-weighted pre-period |
| SC vs TASC      | Model — weight fitting vs latent state-space        |
| SC/SDiD vs TASC | Uncertainty — point estimate vs posterior interval  |

Which estimator I would use depends on the setting. DiD is transparent, but the pre-period fit here is poor. SC improves the donor match. SDiD also weights time periods. TASC is useful when the outcome series has persistent dynamics and we want uncertainty for the counterfactual path, not just one average number.

For this panel, the three methods that try to match the pre-period give estimates between about -15.6 and -19.6. DiD is the outlier. I would read this as a problem with equal weights, not as evidence that the effect of Proposition 99 depends on the estimator.

## 19.6. Setup code

```
df      = CSV.read("california_prop99.csv", DataFrame)
states  = sort(unique(df.State))
years   = sort(unique(df.Year))
N, T    = length(states), length(years)

Y_wide  = zeros(N, T)
treated_vec = zeros(Bool, N)
for row in eachrow(df)
    i = findfirst==(row.State), states)
    t = findfirst==(row.Year), years)
    Y_wide[i, t] = row.PacksPerCapita
    treated_vec[i] |= (row.treated == 1)
end

ctrl_idx = findall(!treated_vec)
trt_idx  = findall(treated_vec)
Y = Y_wide[vcat(ctrl_idx, trt_idx), :] # donors first
N0 = length(ctrl_idx)                  # 38
T0 = findfirst==(1989), years) - 1    # 19
```

The SynthDiD.jl package provides `did_estimate`, `synthdid_estimate`, and `sc_estimate`. The TASC.jl package provides `fit_tasc` and `predict_counterfactual`. Both packages are available on GitHub.

---

*Systematic treatment:* [Julia](#).





## 20. Bartik Instrument and Rotemberg weights

### 20.1. Bartik Instrument

The Bartik instrument, or shift-share IV (Bartik 1991), combines initial local industry shares with national industry growth rates to construct an instrument for local economic conditions.

We follow Goldsmith-Pinkham, Sorkin, and Swift (2021) (GPSS). Consider the labor-supply elasticity:

$$y_{lt} = D_{lt}\rho + \beta_0 x_{lt} + \epsilon_{lt} \quad (20.1)$$

Here  $y_{lt}$  is wage growth in location  $l$ ,  $x_{lt}$  is employment growth, and  $\beta_0$  is the inverse labor-supply elasticity. The challenge is that  $x_{lt}$  is endogenous — unobserved factors may drive both wage and employment growth.

Two accounting identities underlie the instrument:

$$x_{lt} = Z_{lt}G_{lt} = \sum_{k=1}^K z_{lkt}g_{lkt} \quad (20.2)$$

and

$$g_{lkt} = g_{kt} + \tilde{g}_{lkt} \quad (20.3)$$

With  $K$  industries in location  $l$ , employment growth decomposes into industry-level growth rates  $g_{lkt}$  weighted by industry shares  $z_{lkt}$ . The second identity separates each location-industry growth rate into a national component and a location-specific residual. The Bartik instrument uses initial shares and national growth rates:

$$B_{lt} = Z_{l0}G_t = \sum_{k=1}^K z_{lk0}g_{kt} \quad (20.4)$$

The identifying argument is that initial shares are predetermined (a level, not a growth rate) and national industry growth is not driven by any single location.

## 20.2. Bartik Instrument in special cases

### 20.2.1. Two industries and one period

GPSS goes through the simplest case: two industries and one period.

$$B_l = z_{l1}g_1 + z_{l2}g_2 \quad (20.5)$$

Since there are only two industries,  $z_{l1} = 1 - z_{l2}$ . Then we can rewrite the Bartik instrument as

$$B_l = g_2 + (g_1 - g_2)z_{l1} \quad (20.6)$$

From this simplest case, we see that the instrument has only  $z_{l1}$ , that is related to location. If the industry share can be considered exogenous, then the instrument is valid. We have to argue that the industry share is exogenous. In this case, the instrument is a linear function of the industry share. Therefore, the Bartik instrument is the same as using the industry share as an instrument.

GPSS shows that just-identified two-stage least squares using a Bartik instrument can in fact be written as GMM with an overidentified set of instruments, where the weight matrix is the outer product of national growth rates. That is, Bartik estimator is equivalent to a GMM estimator, with each location share as one instrument, and the weight matrix is the outer product of national growth rates.

With  $K$  industries and  $T$  time periods, it's basically using  $K \times T$  instruments. Still, Bartik instrument is the same as GMM with all these instruments.

## 20.3. Rotemberg weights

GPSS decompose the Bartik estimator:

$$\beta_{Bartik} = \sum_k \hat{\alpha}_k \hat{\beta}_k \quad (20.7)$$

where

$$\hat{\beta}_k = (Z'_k X^\perp)^{-1} Z'_k Y^\perp \quad (20.8)$$

and

$$\hat{\alpha}_k = \frac{g_k Z'_k X^\perp}{\sum_{k'} g_{k'} Z'_{k'} X^\perp} \quad (20.9)$$

Here  $X^\perp$  and  $Y^\perp$  are the residualized  $X$  and  $Y$ , from the regression of  $X$  and  $Y$  on all other variables (controls).

Rotemberg weights provide a measure of how important each share is. Read the direction carefully: if industry  $k$ 's own just-identified estimate  $\hat{\beta}_k$  is biased by 1 percent, the overall Bartik estimate inherits  $\alpha_k$  percent of that bias. So a large  $\alpha_k$  means the overall answer is *sensitive* to that industry, not that the industry is itself badly biased. This is what makes the weights useful: they tell you for which instruments the exogeneity assumption has to be defended hardest. (The formal statement is GPSS Corollary D.1, given below.)

## 20.4. Simulations

Simulations codes are adopted from Kohei Kawaguchi's lecture notes. I modified slightly to fit different scenarios.

Here we simulate a case with 4 industries and 4 periods. We have 1000 locations, 4 periods, and 4 industries. The true  $\beta$  (effect of  $x$  on  $y$ ) is 1.

```
library(ggplot2)
library(tidyverse)
library(modelsummary)
library(kableExtra)
library(estimatr)
set.seed(6)
L <- 1000
T <- 4
K <- 4
beta <- 1
sigma <- c(1, 1, 1)

z_lk0 <- expand_grid(
  l = 1:L,
  k = 1:K
) |>
mutate(z_lk0 = runif(length(l)))

g_kt <- expand_grid(
  t = 1:T,
  k = 1:K
) |>
mutate(g_kt = rnorm(n(), 0, .1))

df <- expand_grid(
  l = 1:L,
  t = 1:T,
  k = 1:K
) |>
left_join(z_lk0, by = c("l", "k")) |>
left_join(g_kt, by = c("k", "t")) |>
mutate(z_lkt = z_lk0 + sigma[1] * runif(length(l)),
  g_lkt = g_kt + sigma[2] * rnorm(length(l), 0, .1)
)

df <- df %>%
  group_by(t, l) |>
  mutate(
    z_lk0 = z_lk0 / sum(z_lk0),
    z_lkt = (z_lkt / sum(z_lkt)),
```

## 20. Bartik Instrument and Rotemberg weights

```
x_lt = sum(z_lkt * g_lkt)
)

df

# A tibble: 16,000 x 8
# Groups:   t, 1 [4,000]
      1      t      k z_lk0      g_kt z_lkt      g_lkt      x_lt
  <int> <int> <int> <dbl>    <dbl> <dbl>    <dbl>    <dbl>
1     1     1     1  0.277  0.130  0.252  0.115    0.0400
2     1     1     2  0.428  0.146  0.302  0.291    0.0400
3     1     1     3  0.121 -0.165  0.321 -0.231    0.0400
4     1     1     4  0.174 -0.110  0.126 -0.0185   0.0400
5     1     2     1  0.277 -0.0289  0.160 -0.00619  0.0402
6     1     2     2  0.428 -0.00130  0.356  0.0560    0.0402
7     1     2     3  0.121  0.0433  0.272 -0.0590    0.0402
8     1     2     4  0.174  0.0840  0.212  0.176    0.0402
9     1     3     1  0.277 -0.101  0.237 -0.0361   -0.0738
10    1     3     2  0.428 -0.152  0.367 -0.126   -0.0738
# i 15,990 more rows
```

The simulation has  $L = 1000$  locations,  $T = 4$  periods and  $K = 4$  industries, 16,000 location-industry-period rows. Baseline shares  $z_{lk0}$  are uniform draws normalised to sum to one within each location, national industry growth is  $g_{kt} \sim N(0, 0.1^2)$ , current shares add a uniform perturbation and are renormalised, and local industry growth is  $g_{lkt} = g_{kt} + N(0, 0.1^2)$ . Local employment growth is  $x_{lt} = \sum_k z_{lkt} g_{lkt}$ . The true  $\beta$  is 1.

This assumes we observe the industry shares at time 0 ( $z_{lk0}$ ), and the industry growth rates ( $g_{kt}$ ). We also observe the industry shares at time  $t$  ( $z_{lkt}$ ). The employment growth rate at location  $l$  and time  $t$  ( $g_{lkt}$ ) is a function of  $g_{kt}$  plus some noise. We can calculate the  $x_{lt}$ , which is the weighted sum of industry growth rates.

Next we generate the outcome at  $l, t$  level. Outcome (wage growth) is a linear function of  $x$  (employment growth) at  $l, t$  level.

```
df_lt <- df |>
  mutate(g_tilde_lkt = g_lkt - g_kt) |>
  group_by(l, t) |>
  summarise(across(c(x_lt, g_tilde_lkt), mean), .groups = "drop") |>
  ungroup() |>
  mutate(y_lt = beta * x_lt + sigma[3] * g_tilde_lkt)
```

The outcome is  $y_{lt} = \beta x_{lt} + \sigma_3 \tilde{g}_{lt}$ , where  $\tilde{g}_{lt}$  is the location's mean deviation of local from national industry growth. That deviation also enters  $x_{lt}$ , which is the endogeneity: local shocks drive both employment growth and wage growth.

If we run OLS on raw data, we'll get biased results.

|             | (1)        |
|-------------|------------|
| (Intercept) | 0.000      |
|             | (0.001)    |
| x_lt        | 1.336      |
|             | (0.008)    |
| Num.Obs.    | 4000       |
| R2          | 0.884      |
| R2 Adj.     | 0.884      |
| AIC         | −14 059.9  |
| BIC         | −14 041.0  |
| Log.Lik.    | 7032.945   |
| F           | 30 603.441 |
| RMSE        | 0.04       |

```
result_ols <-
  df_lt |>
  lm(formula = y_lt ~ x_lt)

modelsummary(result_ols)
```

OLS gives 1.336 with a standard error of 0.008, against a true  $\beta$  of 1 — biased upward by a third, and far outside any confidence interval. The endogenous  $\tilde{g}_{lt}$  is doing the damage.

Now we generate the Bartik instrument. We calculate the Bartik instrument as the weighted sum of  $z_{lk0}$  and  $g_{kt}$ .

```
b_lt <- df |>
  group_by(l, t) |>
  summarise(b_lt = sum(z_lk0 * g_kt)) |>
  ungroup()

df_lt <- df_lt |>
  left_join(b_lt, by = c("l", "t"))

result_first_stage <- lm(formula = x_lt ~ b_lt, data = df_lt)

modelsummary(result_first_stage)
```

```
result_tsls <- df_lt |>
  estimatr::iv_robust(formula = y_lt ~ x_lt | b_lt)
```

## 20. Bartik Instrument and Rotemberg weights

|             | (1)       |
|-------------|-----------|
| (Intercept) | −0.001    |
|             | (0.001)   |
| b_lt        | 0.919     |
|             | (0.013)   |
| Num.Obs.    | 4000      |
| R2          | 0.573     |
| R2 Adj.     | 0.573     |
| AIC         | −11 638.5 |
| BIC         | −11 619.6 |
| Log.Lik.    | 5822.262  |
| F           | 5361.487  |
| RMSE        | 0.06      |
|             | (1)       |
| (Intercept) | 0.000     |
|             | (0.001)   |
| x_lt        | 0.990     |
|             | (0.012)   |
| Num.Obs.    | 4000      |
| R2          | 0.825     |
| R2 Adj.     | 0.825     |
| AIC         | −12 398.4 |
| BIC         | −12 379.5 |
| RMSE        | 0.05      |

```
modelsummary::modelsummary(result_tsls)
```

The Bartik instrument recovers the truth: 2SLS gives 0.990 with a standard error of 0.012, against  $\beta = 1$ , where OLS gave 1.336. The instrument is built from baseline shares and *national* growth rates, neither of which contains the local shock  $\tilde{g}_{it}$ , which is what breaks the endogeneity.

### 20.4.1. Calculate Rotemberg weights

Let's do IV using Bartik instruments. No controls here. We use matrix operation to recover the IV estimates, which is the same as the `tsls` function in R.

```

B = as.matrix(cbind(1,df_lt$b_lt))
Y = as.matrix(df_lt$y_lt)
X = as.matrix(cbind(1,df_lt$x_lt))

iv = round(solve(t(B)%*%X)%*%t(B)%*%Y,3)

## Label and organize results into a data frame
beta.hat = as.data.frame(cbind(c("Intercept","x"),iv))
names(beta.hat) = c("Coeff. ","Est")
beta.hat

```

```

      Coeff.  Est
1 Intercept    0
2          x 0.99

```

Now we calculate the Rotemberg weights.

```

# This code is the R version of Paul Goldsmith-Pinkham's R code for Rotemberg weights on
# github; which is actually a call from a C++ code by jjchern.
# bw.R calls ComputeAlphaBeta.cpp
# dimensions: G is KTx1, Z is LTxKT, Y is LTx1, X is LTx1
# so B=ZG is LTx1
# Here I learned the dimensions from the ADH example in Paul's github.
# in ADH example, czone(L) is 722, year(T) is 2, ind(K) is 390.
# so G is 780x1, Z is 1444x780, Y is 1444x1, X is 1444x1
y_lt <- df_lt |>
  select(l,t,y_lt)

Y <- as.matrix(y_lt[,3])

x_lt <- df_lt |>
  select(l,t,x_lt)

X <- as.matrix(x_lt[,3])
#X <- as.matrix(cbind(1,x_lt[,3]))

# global is kt level data set
# # g_data <- df |>
# #   group_by(k,t) |>
# #   summarise(g_kt=sum(g_lkt*z_lkt))
#
# G <- as.matrix(g_data[,3])

G <- as.matrix(g_kt[,3])

# generate wide Z matrix, LTxKT from df

```

## 20. Bartik Instrument and Rotemberg weights

```
# z_data <- df |>
#   select(l,t,k,z_lkt) |>
#   pivot_wider(names_from = k, values_from = z_lkt) |>
#   select(-l, -t)

# this makes a wide Z data, which is LTxKT, KT variables are generated by the spread function.
# basically expand the shares to KT variables, filled with 0 if there is no data.
# Use z_lk0 (the INITIAL shares), not z_lkt: the Bartik instrument is  $B = Z_0 G$ ,
# so the Rotemberg decomposition has to be taken with respect to the same Z that
# builds the instrument. Using z_lkt here would decompose a different estimator.
# z_data <- df |>
#   select(l,t,k,z_lkt) |>
#   mutate(k = str_glue("t{t}_sh_ind_{k}") ) |>
#   spread(k, z_lkt, fill = 0)

z_data <- df |>
  select(l,t,k,z_lk0) |>
  mutate(k = str_glue("t{t}_sh_ind_{k}") ) |>
  pivot_wider(names_from = k, values_from = z_lk0, values_fill = 0)

Z <- as.matrix(z_data[, -c(1,2)])

# Residualize Y and X on the controls before forming the weights. There are
# no controls here other than the constant, so the residualization is just
# demeaning; this is what makes the Rotemberg decomposition exact (GPSS).
Yp <- Y - mean(Y)
Xp <- X - mean(X)
beta = round((t(Z) %*% Yp)/(t(Z) %*% Xp), digits=3)
alpha=(diag(as.numeric(G)) %*% t(Z) %*% Xp) / as.numeric((t(G) %*% t(Z) %*% Xp))

bw <- tibble(g_kt, alpha=as.numeric(alpha), beta=as.numeric(beta)) |>
  mutate(product=alpha*beta)

bw
```

```
# A tibble: 16 x 6
      t     k   g_kt   alpha  beta  product
  <int> <int> <dbl>   <dbl> <dbl>   <dbl>
1     1     1  0.130   0.0103  0.741  0.00760
2     1     2  0.146   0.0188  1.19   0.0223
3     1     3 -0.165   0.0103  0.771  0.00794
4     1     4 -0.110   0.00760  1.04   0.00787
5     2     1 -0.0289 -0.00789  0.886 -0.00699
6     2     2 -0.00130 -0.000387  0.941 -0.000364
7     2     3  0.0433   0.0133  0.898  0.0119
8     2     4  0.0840   0.0280  0.913  0.0256
9     3     1 -0.101   0.138   0.987  0.136
```



|    |   |   |         |        |       |        |
|----|---|---|---------|--------|-------|--------|
| 10 | 3 | 2 | -0.152  | 0.222  | 0.986 | 0.219  |
| 11 | 3 | 3 | -0.0569 | 0.0758 | 0.984 | 0.0746 |
| 12 | 3 | 4 | -0.0994 | 0.136  | 0.993 | 0.135  |
| 13 | 4 | 1 | 0.0266  | 0.0268 | 1.01  | 0.0272 |
| 14 | 4 | 2 | 0.139   | 0.155  | 0.984 | 0.152  |
| 15 | 4 | 3 | 0.0263  | 0.0267 | 1.01  | 0.0270 |
| 16 | 4 | 4 | 0.127   | 0.140  | 1.02  | 0.143  |

```
sum(bw$product)
```

```
[1] 0.9894259
```

The sum of the product of  $\alpha$  and  $\beta$  should be the same as the Bartik estimator, and it is: 0.9894 against the 2SLS estimate of 0.990. The decomposition is exact, not approximate.

Reading the table, the weights are far from uniform across the sixteen industry-period cells. Period 3 carries most of the weight — its four  $\alpha$  values are 0.138, 0.222, 0.076 and 0.136, summing to 0.57 — while period 2 contributes almost nothing and even goes slightly negative ( $\alpha = -0.008$  and  $-0.0004$  for its first two industries). The just-identified  $\beta_k$  estimates are all near the true 1, ranging from 0.74 to 1.19, which is what we should see in a simulation with a homogeneous effect.

Note there is no restriction for  $\alpha$ , other than the sum is 1. Therefore it can be negative or positive, and individual weights can exceed 1, as long as they sum to 1. In GPSS the Rotemberg weights routinely take negative values and values greater than 1; this is a feature used to flag problematic instruments.

## 20.5. Why is good to have Rotemberg weights?

GPSS Corollary D.1. shows the interpretation of the Rotemberg weights.

The percentage of bias can be written as:

$$\frac{E(\tilde{\beta})}{\beta_0} = \sum_k \alpha_k \frac{E(\tilde{\beta}_k)}{\beta_0} \quad (20.10)$$

This is to say the  $\alpha_k$  is the share of the bias of the estimate of  $\beta$  that comes from the bias of the estimate of  $\beta_k$ . That is, how sensitive it is to the bias from industry  $k$ .

In my simulations, all the industries are random, therefore the weights are random too. But in empirical studies, they are not random, most likely dominated by some industries. This is why it is important to have Rotemberg weights. It tells us which industries are important in the estimation of the effect of interest.

## 20.6. What if you are using a reduced form?

If you use the reduced form, something like:

$$y_{lt} = D_{lt}\rho + \beta_0 B_{lt} + \epsilon_{lt} \quad (20.11)$$

where  $B_{lt}$  is the Bartik instrument, constructed in such a way:

$$B_{lt} = Z_{l0}G_t = \sum_{k=1}^K z_{lk0}g_{kt} \quad (20.12)$$

Then I think the Rotemberg weights should be calculated as:

$$\beta_{Bartik} = \sum_k \hat{\alpha}_k \hat{\beta}_k \quad (20.13)$$

where

$$\hat{\beta}_k = (Z'_k B^\perp)^{-1} Z'_k Y^\perp \quad (20.14)$$

and

$$\hat{\alpha}_k = \frac{g_k Z'_k B^\perp}{\sum_{k'} g_{k'} Z'_{k'} B^\perp} \quad (20.15)$$

where  $B^\perp$  and  $Y^\perp$  are residualized  $B$  and  $Y$ .

```
result_reduced <- df_lt |>
  lm(formula = y_lt ~ b_lt)

modelsummary::modelsummary(result_reduced)
```

Now we calculate the Rotemberg weights for the reduced form, for our simulated data.

```
B <- as.matrix(b_lt[,3])

# Residualize Y and B on the controls (here only a constant demean), as in
# the GPSS reduced-form Rotemberg formula above, so the decomposition is exact.
Yp <- Y - mean(Y)
Bp <- B - mean(B)
beta = (t(Z) %*% Yp)/(t(Z) %*% Bp)
alpha=(diag(as.numeric(G)) %*% t(Z) %*% Bp) / as.numeric((t(G) %*% t(Z) %*% Bp))

bw <- tibble(g_kt, alpha=as.numeric(alpha), beta=as.numeric(beta)) |>
  mutate(product = alpha * beta)

bw
```

|             | (1)      |
|-------------|----------|
| (Intercept) | −0.001   |
|             | (0.002)  |
| b_lt        | 0.909    |
|             | (0.023)  |
| Num.Obs.    | 4000     |
| R2          | 0.278    |
| R2 Adj.     | 0.278    |
| AIC         | −6730.0  |
| BIC         | −6711.1  |
| Log.Lik.    | 3367.980 |
| F           | 1538.779 |
| RMSE        | 0.10     |

```
# A tibble: 16 x 6
```

|    | t     | k     | g_kt     | alpha     | beta  | product   |
|----|-------|-------|----------|-----------|-------|-----------|
|    | <int> | <int> | <dbl>    | <dbl>     | <dbl> | <dbl>     |
| 1  | 1     | 1     | 0.130    | 0.0218    | 0.320 | 0.00698   |
| 2  | 1     | 2     | 0.146    | 0.0285    | 0.720 | 0.0205    |
| 3  | 1     | 3     | −0.165   | 0.0322    | 0.226 | 0.00730   |
| 4  | 1     | 4     | −0.110   | 0.0141    | 0.514 | 0.00723   |
| 5  | 2     | 1     | −0.0289  | −0.00628  | 1.02  | −0.00642  |
| 6  | 2     | 2     | −0.00130 | −0.000344 | 0.973 | −0.000335 |
| 7  | 2     | 3     | 0.0433   | 0.0132    | 0.829 | 0.0110    |
| 8  | 2     | 4     | 0.0840   | 0.0304    | 0.773 | 0.0235    |
| 9  | 3     | 1     | −0.101   | 0.128     | 0.978 | 0.125     |
| 10 | 3     | 2     | −0.152   | 0.211     | 0.954 | 0.201     |
| 11 | 3     | 3     | −0.0569  | 0.0687    | 0.998 | 0.0685    |
| 12 | 3     | 4     | −0.0994  | 0.127     | 0.982 | 0.124     |
| 13 | 4     | 1     | 0.0266   | 0.0240    | 1.04  | 0.0250    |
| 14 | 4     | 2     | 0.139    | 0.152     | 0.917 | 0.140     |
| 15 | 4     | 3     | 0.0263   | 0.0238    | 1.04  | 0.0248    |
| 16 | 4     | 4     | 0.127    | 0.133     | 0.989 | 0.131     |

```
sum(bw$product)
```

```
[1] 0.9090661
```

## 20. *Bartik Instrument and Rotemberg weights*

The reduced-form decomposition sums to 0.9091 rather than 0.9894, and that is not an error: it decomposes the reduced-form coefficient (the effect of the instrument on the outcome), not the 2SLS coefficient. The two differ by the first stage, which is not exactly 1.

## 21. Extended TWFE and other DiD estimators

### 21.1. Staggered DiD without treatment reversal

This blog is adopted from the tutorial of paneltools package. [https://yiqingxu.org/tutorials/panel.html#Installing\\_Packages](https://yiqingxu.org/tutorials/panel.html#Installing_Packages)

I added Wooldridge's ETWFE, which is my favorite estimator with staggered DiD. It is easy to understand and implement. It also works with nonlinear model such as logit and poisson.

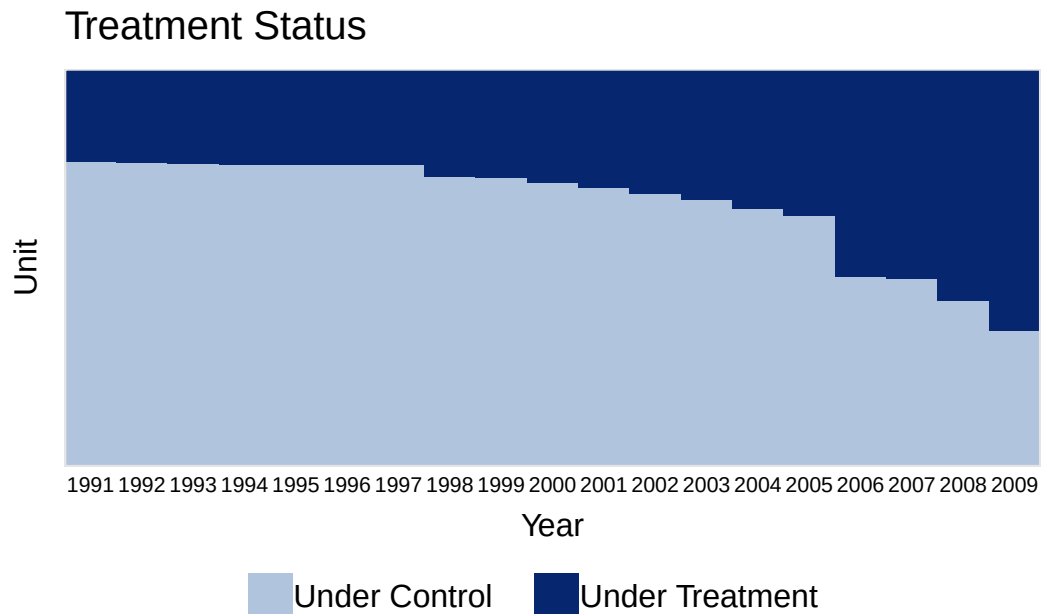
```
#devtools:: install_github("xuyiqing/paneltools")
library(paneltools)
data(paneltools)
library(tidyverse)
hh2019 <- as_tibble(hh2019)
hh2019
```

```
# A tibble: 22,971 x 4
   bfs year nat_rate_ord indirect
  <dbl> <dbl>         <dbl>      <dbl>
1     1   1991             0          0
2     1   1992             0          0
3     1   1993             0          0
4     1   1994             3.45        0
5     1   1995             0          0
6     1   1996             0          0
7     1   1997             0          0
8     1   1998             0          0
9     1   1999             0          0
10    1   2000             0          0
# i 22,961 more rows
```

This data is from Hainmueller and Hopkins (2019), in which the authors investigate the effects of indirect democracy (versus direct democracy) on naturalization rates in Switzerland using municipality-year level panel data from 1991 to 2009. The study shows that a switch from direct to indirect democracy increased naturalization rates by an average of 1.22 percentage points (Model 1 of Table 1 in the paper).

## 21. Extended TWFE and other DiD estimators

```
#devtools:: install_github("xuyiqing/panelView")
library(panelView)
panelview(nat_rate_ord ~ indirect, data = hh2019, index = c("bfs","year"),
  xlab = "Year", ylab = "Unit", display.all = T,
  gridOff = TRUE, by.timing = TRUE)
```



The panel is 22,971 municipality-years: 1,209 Swiss municipalities observed from 1991 to 2009. The outcome `nat_rate_ord` is the ordinary naturalization rate and `indirect` marks the switch from direct to indirect democracy, adopted at different dates by different municipalities — staggered treatment timing.

### 21.1.1. TWFE

First we do a TWFE model on it.

```
library(fixest)
model_twfe <- feols(nat_rate_ord~indirect|bfs+year,
  data=hh2019, cluster = "bfs")
summary( model_twfe)
```

```
OLS estimation, Dep. Var.: nat_rate_ord
Observations: 22,971
Fixed-effects: bfs: 1,209, year: 19
Standard-errors: Clustered (bfs)

      Estimate Std. Error t value Pr(>|t|)
indirect  1.33932   0.186525  7.18039 1.2117e-12 ***
```

---

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 4.09541      Adj. R2: 0.152719
                Within R2: 0.005173
```

TWFE gives 1.33932 with a municipality-clustered standard error of 0.1865, close to the 1.22 the original authors report. Keep that number in view: everything below is an attempt to work out whether it can be trusted.

TWFE has issues when there is staggered adoption and heterogeneous treatment effects.

Goodman-Bacon (2021) demonstrates that the two-way fixed-effects (TWFE) estimator in a staggered adoption setting can be represented as a weighted average of all possible 2x2 difference-in-differences (DID) estimates between different cohorts. However, when treatment effects change over time heterogeneously across cohorts, the “forbidden” comparisons that use post-treatment data from early adopters as controls for late adopters may introduce bias in the TWFE estimator. We employ the procedure outlined in Goodman-Bacon (2021) and decompose the TWFE estimate using the command `bacon` in the `bacondecomp` package.

“Negative weights” can be the other issue. Basically it is a weighted average of comparison between different cohorts. The weights can be negative in a TWFE model.

```
library(bacondecomp)
df_bacon <- bacon(nat_rate_ord~indirect,
                  data = hh2019,
                  id_var = "bfs",
                  time_var = "year")
```

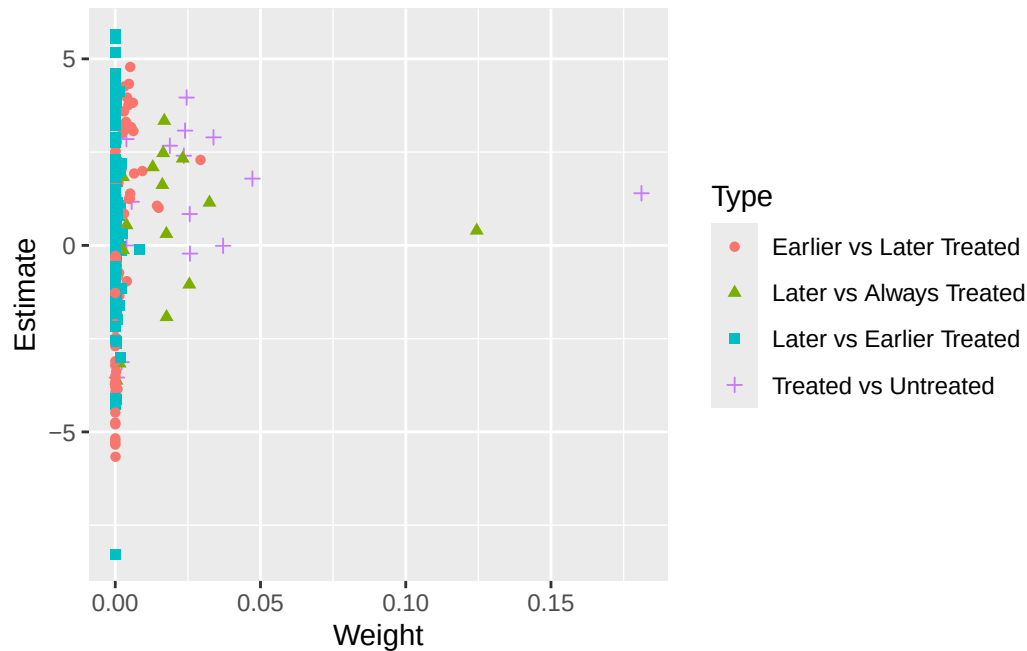
|   | type                     | weight  | avg_est |
|---|--------------------------|---------|---------|
| 1 | Earlier vs Later Treated | 0.17605 | 1.97771 |
| 2 | Later vs Always Treated  | 0.31446 | 0.75233 |
| 3 | Later vs Earlier Treated | 0.05170 | 0.32310 |
| 4 | Treated vs Untreated     | 0.45779 | 1.61180 |

```
coef_bacon <- sum(df_bacon$estimate * df_bacon$weight)
coef_bacon
```

```
[1] 1.339325
```

```
ggplot(df_bacon) +
  aes(x = weight, y = estimate, shape = factor(type), color = factor(type)) +
  labs(x = "Weight", y = "Estimate", shape = "Type", color = 'Type') +
  geom_point()
```

## 21. Extended TWFE and other DiD estimators



The decomposition sums to 1.339325, reproducing the TWFE estimate exactly. It splits into four kinds of 2x2 comparison:

| comparison               | weight | average estimate |
|--------------------------|--------|------------------|
| Treated vs Untreated     | 0.458  | 1.612            |
| Later vs Always Treated  | 0.314  | 0.752            |
| Earlier vs Later Treated | 0.176  | 1.978            |
| Later vs Earlier Treated | 0.052  | 0.323            |

The clean comparisons — treated against never-treated — carry 46% of the weight and give 1.61. The forbidden ones, which use already-treated units as controls, carry  $0.314 + 0.052 = 37\%$  and give much smaller numbers, 0.75 and 0.32. That is what pulls the TWFE estimate down from 1.61 to 1.34.

The problem shown in Bacon decomposition here is the comparison between “Later” and “Earlier Treated”. It’s a “forbidden” comparison.

TWFE with event history type of graph.

```
df.use <- hh2019 |>
  na.omit()

df <- as.data.frame(
  hh2019 |>
    group_by(bfs) |>
    mutate(treatment_mean = mean(indirect, na.rm = TRUE))
)
df.use <- df[which(df$treatment_mean < 1),]
```



```
df.twfe <- as_tibble(get_cohort(df.use,D="indirect",index=c("bfs","year"),start0 = TRUE))

# time_to_treatment is NA when the unit is always treated, or never treated. It needs to be s
df.twfe$treat <- as.numeric(df.twfe$treatment_mean>0) # drop always treated units
df.twfe[which(is.na(df.twfe$Time_to_Treatment)), 'Time_to_Treatment'] <- 0 # can be an arbitrary
twfe.est <- feols(nat_rate_ord ~ i(Time_to_Treatment, treat, ref = -1) | bfs + year,
                  data = df.twfe, cluster = "bfs")
summary(twfe.est)
```

OLS estimation, Dep. Var.: nat\_rate\_ord

Observations: 17,594

Fixed-effects: bfs: 926, year: 19

Standard-errors: Clustered (bfs)

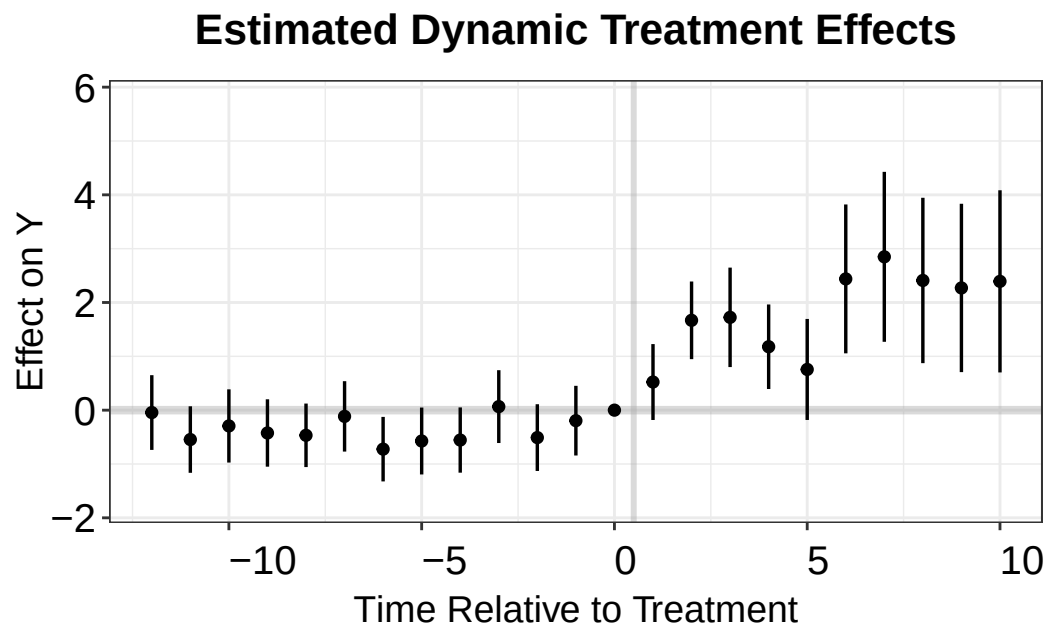
|                              | Estimate  | Std. Error | t value   | Pr(> t )   |     |
|------------------------------|-----------|------------|-----------|------------|-----|
| Time_to_Treatment::-18:treat | -0.320651 | 0.632702   | -0.506796 | 6.1242e-01 |     |
| Time_to_Treatment::-17:treat | -0.118540 | 0.391148   | -0.303058 | 7.6191e-01 |     |
| Time_to_Treatment::-16:treat | -0.395632 | 0.402921   | -0.981910 | 3.2640e-01 |     |
| Time_to_Treatment::-15:treat | -0.410976 | 0.343899   | -1.195048 | 2.3237e-01 |     |
| Time_to_Treatment::-14:treat | 0.244512  | 0.369127   | 0.662406  | 5.0788e-01 |     |
| Time_to_Treatment::-13:treat | -0.044310 | 0.354017   | -0.125164 | 9.0042e-01 |     |
| Time_to_Treatment::-12:treat | -0.545891 | 0.315172   | -1.732042 | 8.3600e-02 | .   |
| Time_to_Treatment::-11:treat | -0.293138 | 0.346835   | -0.845180 | 3.9823e-01 |     |
| Time_to_Treatment::-10:treat | -0.423981 | 0.319375   | -1.327534 | 1.8466e-01 |     |
| Time_to_Treatment::-9:treat  | -0.467114 | 0.301017   | -1.551784 | 1.2106e-01 |     |
| Time_to_Treatment::-8:treat  | -0.115633 | 0.333659   | -0.346561 | 7.2900e-01 |     |
| Time_to_Treatment::-7:treat  | -0.724290 | 0.305727   | -2.369075 | 1.8037e-02 | *   |
| Time_to_Treatment::-6:treat  | -0.572950 | 0.317153   | -1.806540 | 7.1159e-02 | .   |
| Time_to_Treatment::-5:treat  | -0.554993 | 0.309246   | -1.794667 | 7.3033e-02 | .   |
| Time_to_Treatment::-4:treat  | 0.066209  | 0.345103   | 0.191853  | 8.4790e-01 |     |
| Time_to_Treatment::-3:treat  | -0.509530 | 0.316296   | -1.610929 | 1.0754e-01 |     |
| Time_to_Treatment::-2:treat  | -0.195091 | 0.330220   | -0.590793 | 5.5480e-01 |     |
| Time_to_Treatment::0:treat   | 0.522715  | 0.359646   | 1.453415  | 1.4645e-01 |     |
| Time_to_Treatment::1:treat   | 1.668834  | 0.367919   | 4.535879  | 6.4907e-06 | *** |
| Time_to_Treatment::2:treat   | 1.724230  | 0.471363   | 3.657968  | 2.6858e-04 | *** |
| Time_to_Treatment::3:treat   | 1.178336  | 0.400559   | 2.941732  | 3.3452e-03 | **  |
| Time_to_Treatment::4:treat   | 0.756323  | 0.478697   | 1.579962  | 1.1446e-01 |     |
| Time_to_Treatment::5:treat   | 2.438229  | 0.705451   | 3.456269  | 5.7263e-04 | *** |
| Time_to_Treatment::6:treat   | 2.848590  | 0.805315   | 3.537238  | 4.2442e-04 | *** |
| Time_to_Treatment::7:treat   | 2.409359  | 0.783716   | 3.074275  | 2.1721e-03 | **  |
| Time_to_Treatment::8:treat   | 2.270428  | 0.797869   | 2.845617  | 4.5305e-03 | **  |
| Time_to_Treatment::9:treat   | 2.392678  | 0.863222   | 2.771799  | 5.6867e-03 | **  |
| Time_to_Treatment::10:treat  | 2.002836  | 0.953654   | 2.100170  | 3.5984e-02 | *   |
| Time_to_Treatment::11:treat  | 0.842590  | 0.560248   | 1.503961  | 1.3293e-01 |     |
| Time_to_Treatment::12:treat  | 3.831918  | 2.300422   | 1.665745  | 9.6103e-02 | .   |
| Time_to_Treatment::13:treat  | 4.436437  | 2.322318   | 1.910349  | 5.6397e-02 | .   |
| Time_to_Treatment::14:treat  | 4.756977  | 3.485523   | 1.364781  | 1.7265e-01 |     |

## 21. Extended TWFE and other DiD estimators

```
Time_to_Treatment::15:treat    1.541967    1.415284    1.089511 2.7621e-01
Time_to_Treatment::16:treat    7.168943    4.844016    1.479959 1.3922e-01
Time_to_Treatment::17:treat    6.490854    2.726526    2.380632 1.7485e-02 *
```

---  
Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1  
RMSE: 4.42406 Adj. R2: 0.146601  
Within R2: 0.012807

```
twfe.output <- as.matrix(twfe.est$coeftable)
twfe.output <- as.data.frame(twfe.output)
twfe.output$Time <- c(c(-18:-2),c(0:17))+1
p.twfe <- esplot(twfe.output,Period = 'Time',Estimate = 'Estimate',
                 SE = 'Std. Error', xlim = c(-12,10))
p.twfe
```



### 21.1.2. Sun and Abraham

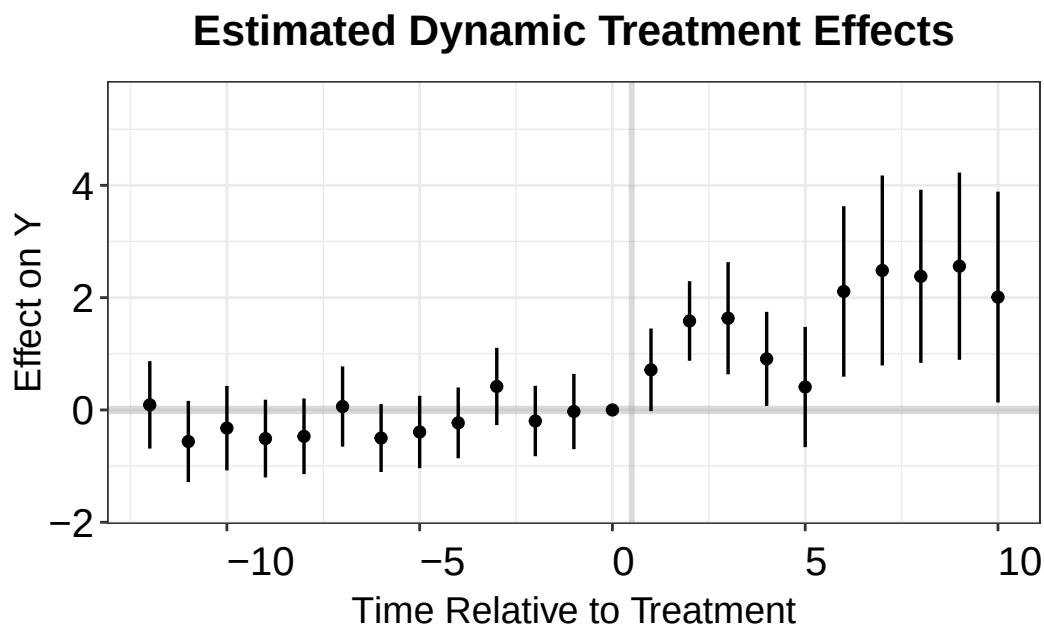
Sun and Abraham (2021) estimator is a weighted average of the average treatment effect (ATT) estimates for each cohort, obtained from a TWFE regression that includes cohort dummies fully interacted with indicators of relative time to the treatment's onset. The iw estimator is robust to heterogeneous treatment effects (HTE) and can be implemented using the command `sunab` in the package `fixest`.

```
df.sa <- df.twfe
df.sa[which(is.na(df.sa$FirstTreat)), "FirstTreat"] <- 1000 # replace NA with an arbitrary number
model.sa.1 <- feols(nat_rate_ord~sunab(FirstTreat,year)|bfs+year,
```

```
data = df.sa, cluster = "bfs")
summary(model.sa.1,agg = "ATT")
```

```
OLS estimation, Dep. Var.: nat_rate_ord
Observations: 17,594
Fixed-effects: bfs: 926, year: 19
Standard-errors: Clustered (bfs)
      Estimate Std. Error t value   Pr(>|t|)
ATT    1.3309    0.287971 4.62163 4.3474e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 4.34795      Adj. R2: 0.163888
                Within R2: 0.046484
```

```
sa.output <- as.matrix(model.sa.1$coeftable)
sa.output <- as.data.frame(sa.output)
sa.output$Time <- c(c(-18:-2),c(0:17))+1
p.sa <- esplot(sa.output,Period = 'Time',Estimate = 'Estimate',
               SE = 'Std. Error', xlim = c(-12,10))
p.sa
```



## 21.2. Callaway and Sant'Anna

Callaway and Sant'Anna (2021) build the estimator from group-time average treatment effects  $ATT(g, t)$  — the effect on cohort  $g$  at time  $t$  — each identified by comparing the treated cohort

## 21. Extended TWFE and other DiD estimators

to a never-treated or not-yet-treated comparison group. The  $ATT(g, t)$  are estimated by outcome regression, inverse-probability weighting, or a doubly-robust combination of the two, rather than from a single fully-interacted TWFE regression. These group-time effects are then aggregated with weights into summary parameters (overall, event-study, or by cohort). The estimator is robust to heterogeneous treatment effects (HTE) and can be implemented with the `att_gt` function in the `did` package.

```
library(did)
df.cs <- df.twfe
df.cs[which(is.na(df.cs$FirstTreat)), "FirstTreat"] <- 0 # replace NA with 0
cs.est.1 <- att_gt(yname = "nat_rate_ord",
                  gname = "FirstTreat",
                  idname = "bfs",
                  tname = "year",
                  xformula = ~1,
                  control_group = "nevertreated",
                  allow_unbalanced_panel = TRUE,
                  data = df.cs,
                  est_method = "reg")
cs.est.att.1 <- aggte(cs.est.1, type = "simple", na.rm=T, bstrap = F)
print(cs.est.att.1)
```

Call:

```
aggte(MP = cs.est.1, type = "simple", na.rm = T, bstrap = F)
```

Reference: Callaway, Brantly and Pedro H.C. Sant'Anna. "Difference-in-Differences with Multiplicities". *Journal of Econometrics* 230, 2021. <<https://doi.org/10.1016/j.jeconom.2020.12.001>>, <<https://arxiv.org/abs/1803.09015>>

| ATT    | Std. Error | [ 95% Conf. Int.] |
|--------|------------|-------------------|
| 1.3309 | 0.3019     | 0.7392 1.9226 *   |

---

Signif. codes: `\*' confidence band does not cover 0

Control Group: Never Treated, Anticipation Periods: 0

Estimation Method: Outcome Regression

```
cs.att.1 <- aggte(cs.est.1, type = "dynamic",
                  bstrap=FALSE, cband=FALSE, na.rm=T)
print(cs.att.1)
```

Call:

```
aggte(MP = cs.est.1, type = "dynamic", na.rm = T, bstrap = FALSE,
```

cband = FALSE)

Reference: Callaway, Brantly and Pedro H.C. Sant'Anna. "Difference-in-Differences with Multip  
230, 2021. <<https://doi.org/10.1016/j.jeconom.2020.12.001>>, <<https://arxiv.org/abs/1803.09015>>

Overall summary of ATT's based on event-study/dynamic aggregation:

| ATT    | Std. Error | [ 95% Conf. Int.] |
|--------|------------|-------------------|
| 1.2205 | 0.6314     | -0.0171 2.458     |

Dynamic Effects:

| Event time | Estimate | Std. Error | [95% Pointwise | Conf. Band] |
|------------|----------|------------|----------------|-------------|
| -17        | 0.1350   | 0.6467     | -1.1324        | 1.4025      |
| -16        | -0.0839  | 0.3583     | -0.7861        | 0.6183      |
| -15        | 0.1467   | 0.3964     | -0.6303        | 0.9236      |
| -14        | 0.7509   | 0.3219     | 0.1200         | 1.3818 *    |
| -13        | -0.2318  | 0.3392     | -0.8967        | 0.4331      |
| -12        | -0.5851  | 0.2814     | -1.1367        | -0.0335 *   |
| -11        | 0.2656   | 0.2705     | -0.2647        | 0.7958      |
| -10        | -0.2386  | 0.2805     | -0.7883        | 0.3111      |
| -9         | 0.0576   | 0.2702     | -0.4721        | 0.5872      |
| -8         | 0.5244   | 0.3165     | -0.0959        | 1.1448      |
| -7         | -0.6282  | 0.3132     | -1.2420        | -0.0144 *   |
| -6         | 0.1081   | 0.2565     | -0.3946        | 0.6108      |
| -5         | 0.1623   | 0.2890     | -0.4041        | 0.7287      |
| -4         | 0.6485   | 0.3380     | -0.0139        | 1.3109      |
| -3         | -0.6161  | 0.3674     | -1.3362        | 0.1040      |
| -2         | 0.1505   | 0.3355     | -0.5071        | 0.8081      |
| -1         | 0.0279   | 0.3425     | -0.6435        | 0.6992      |
| 0          | 0.7141   | 0.3754     | -0.0217        | 1.4500      |
| 1          | 1.5846   | 0.3681     | 0.8632         | 2.3060 *    |
| 2          | 1.6337   | 0.5118     | 0.6306         | 2.6369 *    |
| 3          | 0.9090   | 0.4396     | 0.0473         | 1.7707 *    |
| 4          | 0.4091   | 0.5725     | -0.7130        | 1.5311      |
| 5          | 2.1106   | 0.7976     | 0.5473         | 3.6739 *    |
| 6          | 2.4842   | 0.9078     | 0.7051         | 4.2634 *    |
| 7          | 2.3802   | 0.8538     | 0.7068         | 4.0535 *    |
| 8          | 2.5602   | 0.8723     | 0.8506         | 4.2699 *    |
| 9          | 2.0100   | 0.9769     | 0.0953         | 3.9246 *    |
| 10         | 1.6391   | 1.0257     | -0.3712        | 3.6494      |
| 11         | 0.0298   | 0.6100     | -1.1658        | 1.2253      |
| 12         | 0.6942   | 2.8043     | -4.8021        | 6.1904      |
| 13         | 1.1355   | 2.3284     | -3.4281        | 5.6991      |
| 14         | 1.3625   | 3.4131     | -5.3270        | 8.0519      |
| 15         | -2.0699  | 1.7160     | -5.4331        | 1.2933      |
| 16         | 2.2743   | 3.6182     | -4.8172        | 9.3658      |
| 17         | 0.1072   | 4.5338     | -8.7789        | 8.9934      |

## 21. Extended TWFE and other DiD estimators

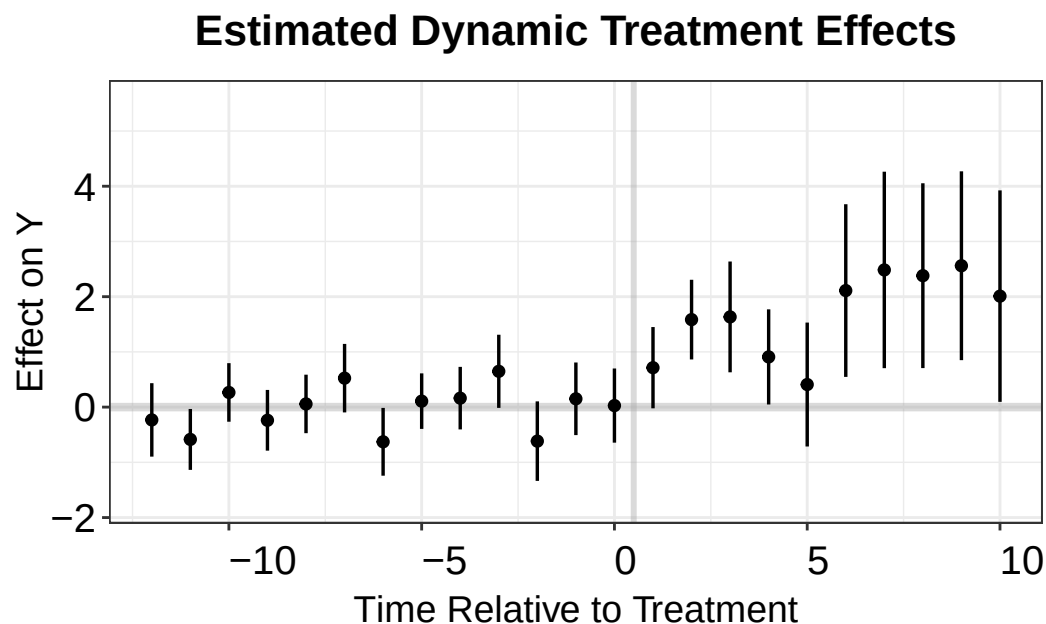
---

Signif. codes: `\*' confidence band does not cover 0

Control Group: Never Treated, Anticipation Periods: 0

Estimation Method: Outcome Regression

```
cs.output <- cbind.data.frame(Estimate = cs.att.1$att.egt,  
                              SE = cs.att.1$se.egt,  
                              time = cs.att.1$egt + 1)  
p.cs.1 <- esplot(cs.output, Period = 'time', Estimate = 'Estimate',  
                SE = 'SE', xlim = c(-12, 10))  
p.cs.1
```



### 21.2.1. Imai, Kim, and Wang

PanelMatch is an R package implementing a set of methodological tools proposed by Imai, Kim, and Wang (2021) that enables researchers to apply matching methods for causal inference on time-series cross-sectional data with binary treatments. The package includes implementations of matching methods based on propensity scores and Mahalanobis distance, as well as weighting methods. PanelMatch enables users to easily calculate a variety of possible quantities of interest, along with standard errors. The software is flexible, allowing users to tune the matching, refinement, and estimation procedures with a large number of parameters. The package also offers a variety of visualization and diagnostic tools for researchers to better understand their data and assess their results.

```

library(PanelMatch)
df.pm <- as.data.frame(df.twfe)
# we need to convert the unit and time indicator to integer
df.pm[, "bfs"] <- as.integer(as.factor(df.pm[, "bfs"]))
df.pm[, "year"] <- as.integer(as.factor(df.pm[, "year"]))
df.pm <- df.pm[, c("bfs", "year", "nat_rate_ord", "indirect")]

# Create PanelData object (new API)
pd <- PanelData(df.pm, unit.id = "bfs", time.id = "year",
               treatment = "indirect", outcome = "nat_rate_ord")

PM.results <- PanelMatch(lag=3,
                        panel.data = pd,
                        refinement.method = "none",
                        qoi = "att",
                        lead = c(0:3),
                        match.missing = TRUE)

## For pre-treatment dynamic effects
PM.results.placebo <- PanelMatch(lag=3,
                                panel.data = pd,
                                refinement.method = "none",
                                qoi = "att",
                                lead = c(0:3),
                                match.missing = TRUE,
                                placebo.test = TRUE)
PE.results.pool <- PanelEstimate(PM.results, panel.data = pd, pooled = TRUE)
summary(PE.results.pool)

```

```

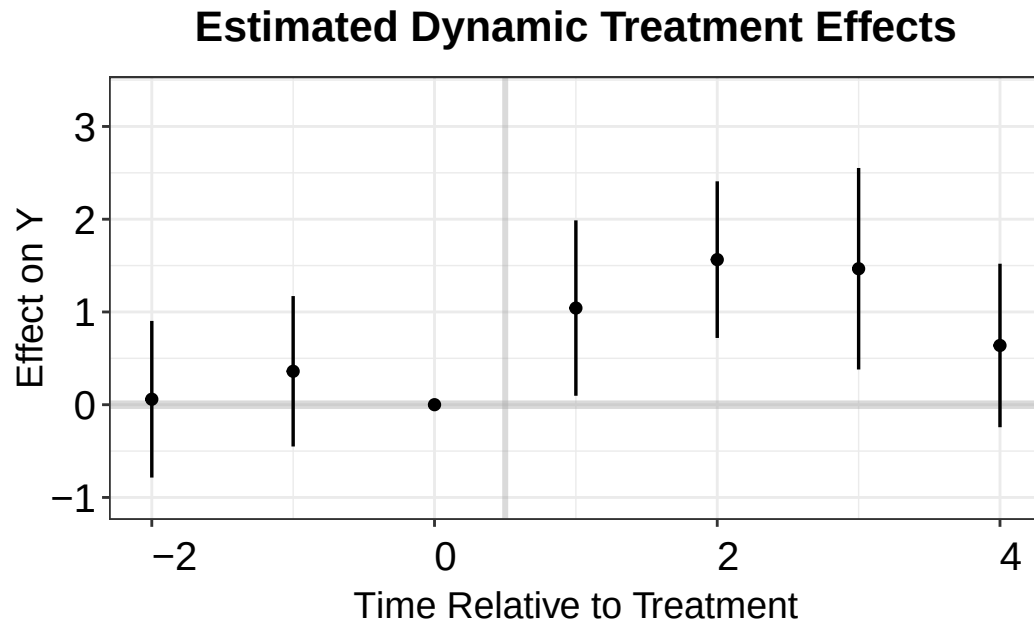
      estimate std.error      2.5%      97.5%
[1,] 1.177674 0.3492237 0.4959534 1.872191

```

```

PE.results <- PanelEstimate(PM.results, panel.data = pd)
PE.results.placebo <- placebo_test(PM.results.placebo, panel.data = pd, plot = F)
est_lead <- as.vector(estimates(PE.results))
est_lag <- tryCatch(as.vector(estimates(PE.results.placebo)),
                  error = function(e) as.vector(PE.results.placebo$estimates))
sd_lead <- tryCatch(apply(PE.results$bootstrapped.estimates, 2, sd),
                  error = function(e) rep(NA, length(est_lead)))
sd_lag <- tryCatch(apply(PE.results.placebo$bootstrapped.estimates, 2, sd),
                  error = function(e) rep(NA, length(est_lag)))
coef <- c(est_lag, 0, est_lead)
sd <- c(sd_lag, 0, sd_lead)
pm.output <- cbind.data.frame(ATT=coef, se=sd, t=c(-2:4))
p.pm <- esplot(data = pm.output, Period = 't',
              Estimate = 'ATT', SE = 'se')
p.pm

```



### 21.2.2. Liu, Wang and Xu

Liu, Wang, and Xu (2022) proposes a method for estimating the fixed effect counterfactual model by utilizing the untreated observations. It is essentially an imputation method that uses the untreated observations to estimate the counterfactual outcome for the treated.

```
library(fect)
out.fect <- fect(nat_rate_ord~indirect, data = df,
                 index = c("bfs", "year"),
                 method = 'fe', se = TRUE)
print(out.fect$est.avg)
```

```
      ATT.avg      S.E. CI.lower CI.upper      p.value
[1,] 1.506416 0.2033546 1.107849 1.904984 1.283418e-13
```

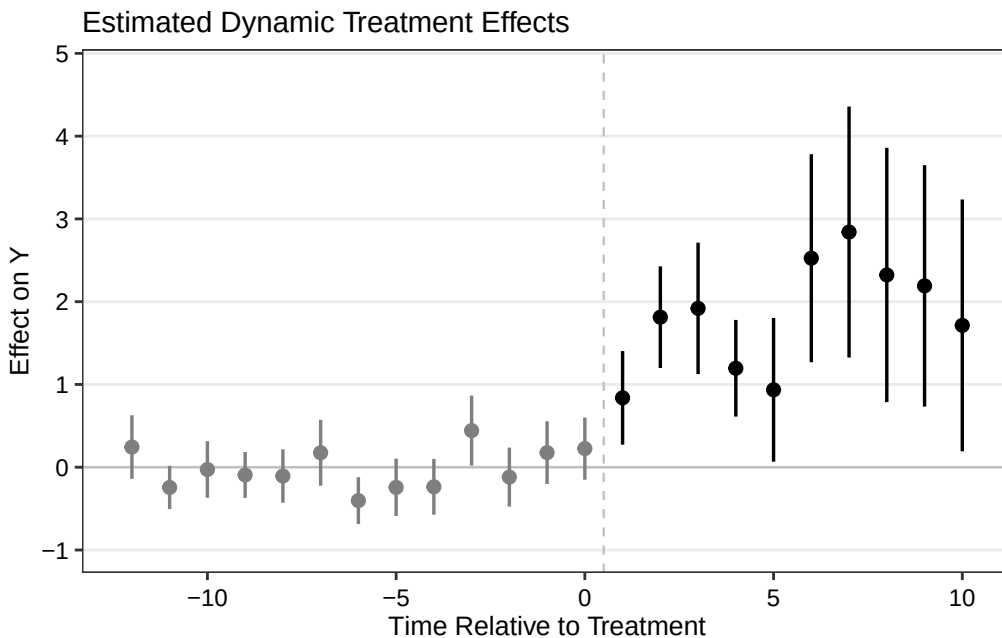
```
fect.output <- as.matrix(out.fect$est.att)
print(fect.output)
```

|     | ATT         | S.E.      | CI.lower    | CI.upper   | p.value      | count |
|-----|-------------|-----------|-------------|------------|--------------|-------|
| -17 | -0.01985591 | 0.5249937 | -1.04882465 | 1.00911284 | 9.698302e-01 | 89    |
| -16 | 0.07269638  | 0.2600444 | -0.43698129 | 0.58237405 | 7.798199e-01 | 157   |
| -15 | -0.21365489 | 0.2524156 | -0.70838039 | 0.28107061 | 3.973068e-01 | 162   |
| -14 | -0.13153052 | 0.1533090 | -0.43201065 | 0.16894962 | 3.909235e-01 | 350   |
| -13 | 0.46902947  | 0.2244936 | 0.02903007  | 0.90902887 | 3.668274e-02 | 371   |



|     |             |           |             |             |              |     |
|-----|-------------|-----------|-------------|-------------|--------------|-----|
| -12 | 0.24357664  | 0.1960828 | -0.14073858 | 0.62789186  | 2.141579e-01 | 398 |
| -11 | -0.24399485 | 0.1332452 | -0.50515055 | 0.01716086  | 6.707482e-02 | 417 |
| -10 | -0.02765866 | 0.1742036 | -0.36909137 | 0.31377405  | 8.738485e-01 | 434 |
| -9  | -0.09347047 | 0.1417648 | -0.37132431 | 0.18438337  | 5.096807e-01 | 451 |
| -8  | -0.10638473 | 0.1648035 | -0.42939365 | 0.21662420  | 5.185872e-01 | 464 |
| -7  | 0.17476094  | 0.2031886 | -0.22348141 | 0.57300329  | 3.897382e-01 | 468 |
| -6  | -0.40297604 | 0.1439751 | -0.68516202 | -0.12079006 | 5.127241e-03 | 503 |
| -5  | -0.24326436 | 0.1767746 | -0.58973630 | 0.10320759  | 1.687823e-01 | 503 |
| -4  | -0.23687833 | 0.1716243 | -0.57325570 | 0.09949904  | 1.675205e-01 | 503 |
| -3  | 0.44244127  | 0.2155461 | 0.01997871  | 0.86490383  | 4.010627e-02 | 503 |
| -2  | -0.11927690 | 0.1817101 | -0.47542211 | 0.23686831  | 5.115583e-01 | 508 |
| -1  | 0.17612194  | 0.1932931 | -0.20272562 | 0.55496950  | 3.622084e-01 | 512 |
| 0   | 0.22491529  | 0.1918671 | -0.15113738 | 0.60096795  | 2.410987e-01 | 514 |
| 1   | 0.83813813  | 0.2884721 | 0.27274312  | 1.40353314  | 3.667387e-03 | 514 |
| 2   | 1.81274486  | 0.3130701 | 1.19913872  | 2.42635100  | 7.029422e-09 | 425 |
| 3   | 1.91941052  | 0.4049619 | 1.12569975  | 2.71312129  | 2.140022e-06 | 357 |
| 4   | 1.19567464  | 0.2979622 | 0.61167949  | 1.77966979  | 5.999252e-05 | 352 |
| 5   | 0.93520961  | 0.4429749 | 0.06699484  | 1.80342438  | 3.475491e-02 | 164 |
| 6   | 2.52517848  | 0.6411037 | 1.26863840  | 3.78171855  | 8.189048e-05 | 143 |
| 7   | 2.84129736  | 0.7735045 | 1.32525638  | 4.35733835  | 2.394584e-04 | 116 |
| 8   | 2.32288547  | 0.7838386 | 0.78658995  | 3.85918100  | 3.041877e-03 | 97  |
| 9   | 2.19105962  | 0.7439582 | 0.73292831  | 3.64919092  | 3.228106e-03 | 80  |
| 10  | 1.71382906  | 0.7761588 | 0.19258578  | 3.23507233  | 2.723795e-02 | 63  |
| 11  | 1.07651548  | 1.0454215 | -0.97247296 | 3.12550392  | 3.031306e-01 | 50  |
| 12  | -0.30828629 | 0.6163128 | -1.51623712 | 0.89966453  | 6.169267e-01 | 46  |
| 13  | 0.35079211  | 2.0582844 | -3.68337111 | 4.38495534  | 8.646725e-01 | 11  |
| 14  | 0.81696815  | 2.5111791 | -4.10485243 | 5.73878873  | 7.449294e-01 | 11  |
| 15  | 0.97427612  | 3.8070109 | -6.48732812 | 8.43588037  | 7.980155e-01 | 11  |
| 16  | -2.49173503 | 1.7363824 | -5.89498197 | 0.91151190  | 1.512828e-01 | 11  |
| 17  | 1.42334105  | 5.3589459 | -9.07999999 | 11.92668210 | 7.905466e-01 | 6   |
| 18  | -0.01384226 | 4.7462662 | -9.31635305 | 9.28866853  | 9.976730e-01 | 2   |

```
fect.output <- as.data.frame(fect.output)
fect.output$Time <- c(-17:18)
p.fect <- esplot(fect.output, Period = 'Time', Estimate = 'ATT',
                 SE = 'S.E.', CI.lower = "CI.lower",
                 CI.upper = 'CI.upper', xlim = c(-12, 10))
p.fect
```



Borusyak, Jaravel, and Spiess (2021) with package `didimputation` almost gives the same result as Liu, Wang, and Xu (2022).

### 21.2.3. Wooldridge

Wooldridge 2021, also called extended TWFE (ETWFE), is to allow more flexibility to make TWFE work properly. The problem with TWFE is that we do not specify it flexibly enough. If we allow heterogeneous treatment effect, then we cannot expect the canonical TWFE to work properly.

```
df.wool <- df.twfe
df.wool[which(is.na(df.wool$FirstTreat)), "FirstTreat"] <- 0 # replace NA with 0

library(etwfe)

model.wool = etwfe(
  fml = nat_rate_ord ~ 1, # outcome ~ controls
  tvar = year,            # time variable
  gvar = FirstTreat,      # group variable
  data = df.wool,         # dataset
  vcov = ~bfs,            # vcov adjustment (here: clustered)
  cgroup = "never"
)
model.wool
```

```
OLS estimation, Dep. Var.: nat_rate_ord
Observations: 17,594
Fixed-effects: FirstTreat: 16, year: 19
```

Standard-errors: Clustered (bfs)

```

                                Estimate Std. Error  t value Pr(>|t|)
.Dtreat:FirstTreat::1992:year::1992  2.18863      1.36923   1.59844 0.110287
.Dtreat:FirstTreat::1992:year::1993 -4.18925      3.10938  -1.34730 0.178214
.Dtreat:FirstTreat::1992:year::1994 -2.50997      1.63352  -1.53654 0.124747
.Dtreat:FirstTreat::1992:year::1995 -5.23221      3.11282  -1.68086 0.093129 .
.Dtreat:FirstTreat::1992:year::1996 -5.42014      3.11207  -1.74165 0.081902 .
.Dtreat:FirstTreat::1992:year::1997 -5.15575      3.11248  -1.65648 0.097965 .
.Dtreat:FirstTreat::1992:year::1998 -5.08210      3.11345  -1.63231 0.102955
.Dtreat:FirstTreat::1992:year::1999 -5.13290      3.11447  -1.64808 0.099676 .
... 262 coefficients remaining (display them with summary() or use
argument n)
... 15 variables were removed because of collinearity
(.Dtreat:FirstTreat::1992:year::1991,
.Dtreat:FirstTreat::1993:year::1992 and 13 others [full set in
$collin.var])
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 4.6642      Adj. R2: 0.088474
                Within R2: 0.040642

```

```
emfx(model.wool)
```

```

.Dtreat Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
TRUE      1.33      0.288 4.62  <0.001 18.0 0.766    1.9

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE

```

```
mod_es = emfx(model.wool, type = "event", post_only=FALSE)
mod_es
```

```

event Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
-18   -0.266     0.789 -0.3371   0.736 0.4 -1.812  1.280
-17    0.178     0.503  0.3538   0.723 0.5 -0.808  1.164
-16    0.114     0.519  0.2198   0.826 0.3 -0.903  1.131
-15   -0.104     0.408 -0.2550   0.799 0.3 -0.903  0.695
-14    0.410     0.413  0.9927   0.321 1.6 -0.400  1.220
--- 26 rows omitted. See ?print.marginaleffects ---
13    1.135     1.998  0.5683   0.570 0.8 -2.781  5.052
14    1.362     3.129  0.4355   0.663 0.6 -4.770  7.495
15   -2.070     1.439 -1.4387   0.150 2.7 -4.890  0.750
16    2.274     3.080  0.7385   0.460 1.1 -3.762  8.311

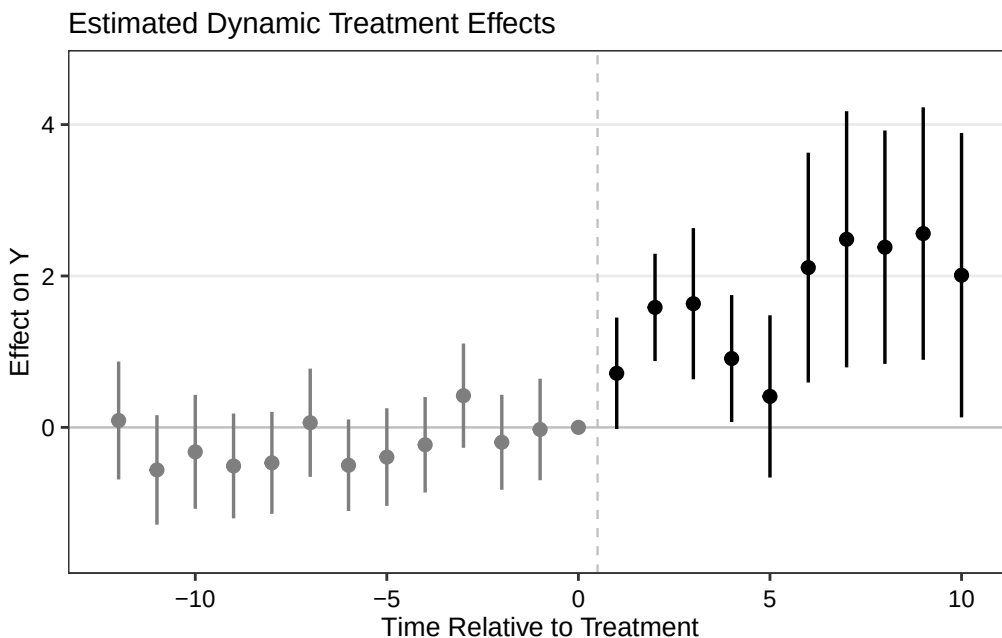
```

## 21. Extended TWFE and other DiD estimators

```
17      0.107      4.576  0.0234      0.981 0.0 -8.861  9.076
Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

```
wool.output <- mod_es |>
  filter(event != -1) |>
  mutate(Time = event + 1) |>
  as.data.frame()

p.wool <- esplot(wool.output, Period = 'Time', Estimate = 'estimate',
  SE = 'std.error', xlim = c(-12, 10))
p.wool
```



Note that in order to get pre-event estimates, we need to set “cgroup=never” in the `etwfe` function. Otherwise, ETWFE will only give us post-event estimates. Under the default setting, the control group comprises the “not-yet” treated units, therefore all pre-treatment effects are mechanically set to zero. By setting “cgroup=never”, we only use never treated units as the control group. This way we can get pre-treatment estimates, but we are not using all available information for the control group, therefore the estimates may not be as efficient as the default setting. That cost is visible in the two runs below: the never-treated version gives an ATT of 1.33 with a standard error of 0.288, while the default not-yet-treated version gives 1.51 with a standard error of 0.188 — a little over a third narrower.

Also we don’t have control variable here. If we have, it is very easy to incorporate into ETWFE. Wooldridge used the Mundlak device to incorporate control variables, which is a very simple and intuitive way to do it.

If we do the default setting and plot only post-event estimates:

```

model.wool2 = etwfe(
  fml = nat_rate_ord ~ 1, # outcome ~ controls
  tvar = year,            # time variable
  gvar = FirstTreat,      # group variable
  data = df.wool,         # dataset
  vcov = ~bfs             # vcov adjustment (here: clustered)
)
model.wool2

```

OLS estimation, Dep. Var.: nat\_rate\_ord

Observations: 17,594

Fixed-effects: FirstTreat: 16, year: 19

Standard-errors: Clustered (bfs)

|                                     | Estimate | Std. Error | t value  | Pr(> t ) |
|-------------------------------------|----------|------------|----------|----------|
| .Dtreat:FirstTreat::1992:year::1992 | 1.99670  | 1.35883    | 1.46943  | 0.142057 |
| .Dtreat:FirstTreat::1992:year::1993 | -4.52291 | 3.09436    | -1.46166 | 0.144174 |
| .Dtreat:FirstTreat::1992:year::1994 | -2.62350 | 1.61688    | -1.62257 | 0.105022 |
| .Dtreat:FirstTreat::1992:year::1995 | -5.17241 | 3.09454    | -1.67146 | 0.094969 |
| .Dtreat:FirstTreat::1992:year::1996 | -5.08905 | 3.09404    | -1.64479 | 0.100353 |
| .Dtreat:FirstTreat::1992:year::1997 | -5.17232 | 3.09482    | -1.67128 | 0.095004 |
| .Dtreat:FirstTreat::1992:year::1998 | -5.39134 | 3.09606    | -1.74135 | 0.081954 |
| .Dtreat:FirstTreat::1992:year::1999 | -5.09733 | 3.09495    | -1.64698 | 0.099901 |

... 121 coefficients remaining (display them with summary() or use argument n)

... 141 variables were removed because of collinearity  
 (.Dtreat:FirstTreat::1993:year::1992,  
 .Dtreat:FirstTreat::1994:year::1992 and 139 others [full set in \$collin.var])

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

RMSE: 4.6999      Adj. R2: 0.081954

                    Within R2: 0.0259

```
emfx(model.wool2)
```

|      | Estimate | Std. Error | z    | Pr(> z ) | S    | 2.5 % | 97.5 % |
|------|----------|------------|------|----------|------|-------|--------|
| TRUE | 1.51     | 0.188      | 8.02 | <0.001   | 49.7 | 1.14  | 1.87   |

Term: .Dtreat

Type: response

Comparison: TRUE - FALSE

```

mod_es2 = emfx(model.wool2, type = "event")
mod_es2

```

## 21. Extended TWFE and other DiD estimators

| event | Estimate | Std. Error | z        | Pr(> z ) | S    | 2.5 %  | 97.5 % |
|-------|----------|------------|----------|----------|------|--------|--------|
| 0     | 0.8381   | 0.288      | 2.90643  | 0.00366  | 8.1  | 0.273  | 1.403  |
| 1     | 1.8127   | 0.316      | 5.74107  | < 0.001  | 26.7 | 1.194  | 2.432  |
| 2     | 1.9194   | 0.399      | 4.81257  | < 0.001  | 19.4 | 1.138  | 2.701  |
| 3     | 1.1957   | 0.320      | 3.73394  | < 0.001  | 12.4 | 0.568  | 1.823  |
| 4     | 0.9352   | 0.423      | 2.20947  | 0.02714  | 5.2  | 0.106  | 1.765  |
| 5     | 2.5252   | 0.641      | 3.93965  | < 0.001  | 13.6 | 1.269  | 3.781  |
| 6     | 2.8413   | 0.738      | 3.84931  | < 0.001  | 13.0 | 1.395  | 4.288  |
| 7     | 2.3229   | 0.710      | 3.27278  | 0.00106  | 9.9  | 0.932  | 3.714  |
| 8     | 2.1911   | 0.751      | 2.91737  | 0.00353  | 8.1  | 0.719  | 3.663  |
| 9     | 1.7138   | 0.795      | 2.15674  | 0.03103  | 5.0  | 0.156  | 3.271  |
| 10    | 1.0765   | 0.941      | 1.14366  | 0.25277  | 2.0  | -0.768 | 2.921  |
| 11    | -0.3083  | 0.498      | -0.61885 | 0.53602  | 0.9  | -1.285 | 0.668  |
| 12    | 0.3508   | 1.892      | 0.18538  | 0.85293  | 0.2  | -3.358 | 4.060  |
| 13    | 0.8170   | 2.012      | 0.40611  | 0.68466  | 0.5  | -3.126 | 4.760  |
| 14    | 0.9743   | 3.351      | 0.29076  | 0.77123  | 0.4  | -5.593 | 7.542  |
| 15    | -2.4917  | 1.413      | -1.76355 | 0.07781  | 3.7  | -5.261 | 0.278  |
| 16    | 1.4233   | 4.430      | 0.32126  | 0.74801  | 0.4  | -7.260 | 10.107 |
| 17    | -0.0138  | 4.556      | -0.00304 | 0.99758  | 0.0  | -8.944 | 8.916  |

Term: .Dtreat

Type: response

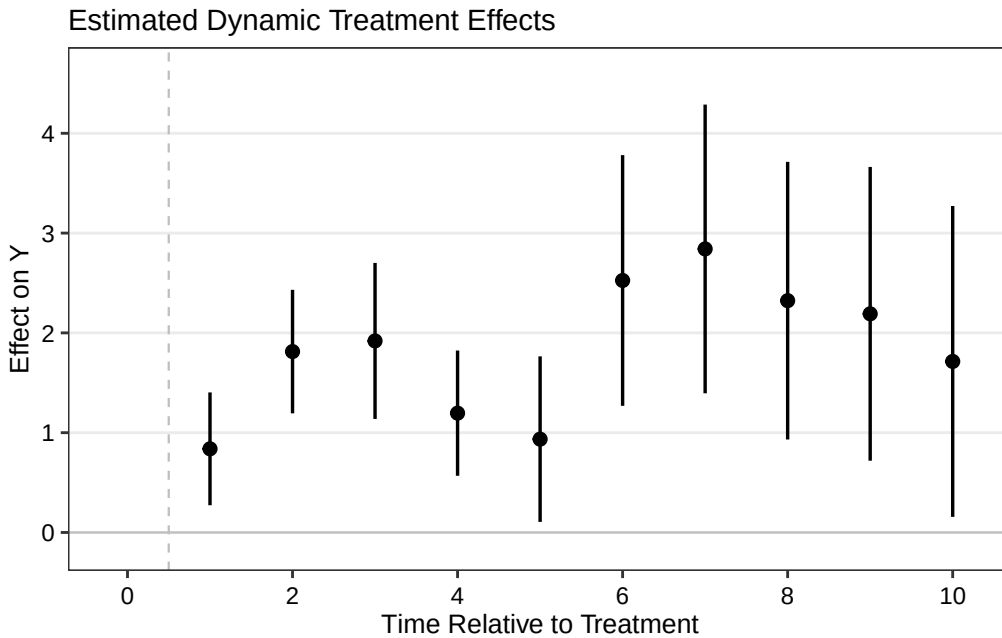
Comparison: TRUE - FALSE

```

wool.output2 <- mod_es2 |>
  filter(event != -1) |>
  mutate(Time = event + 1) |>
  as.data.frame()

p.wool2 <- esplot(wool.output2, Period = 'Time', Estimate = 'estimate',
                  SE = 'std.error', xlim = c(0,10))
p.wool2

```



## 21.3. Comparing the estimators

It is worth collecting what all of these produced on the same data:

| estimator                        | ATT   | s.e.  |
|----------------------------------|-------|-------|
| TWFE                             | 1.339 | 0.187 |
| Sun and Abraham                  | 1.331 | 0.288 |
| Callaway and Sant'Anna (simple)  | 1.331 | 0.302 |
| Callaway and Sant'Anna (dynamic) | 1.221 | 0.631 |
| Imai, Kim and Wang               | 1.178 | 0.335 |
| Liu, Wang and Xu                 | 1.506 | 0.204 |
| Wooldridge ETWFE                 | 1.330 | 0.288 |

Three things stand out. Sun-Abraham, Callaway-Sant'Anna and ETWFE agree to three digits at 1.33 — they are different routes to the same never-treated-control estimand, and they land on the same number. The spread across all seven, 1.18 to 1.51, is smaller than the standard errors, so on this data the estimator choice does not change the conclusion.

And TWFE at 1.339 is not far off any of them, despite the forbidden comparisons the Bacon decomposition exposed. That is worth saying plainly: the decomposition shows TWFE *is* contaminated here, but the contamination happens to be small enough that the answer survives. It would not have to be.

## 21.4. Staggered DiD with treatment reversal

With treatment reversal (ie., the treated units go back to control), Wooldridge’s method still works. However, I don’t think package “etwfe” has it implemented.

So far, packages “fect”, “PanelMatch” can handle treatment reversal.

We can probably manually do ETWFE with treatment reversal. See Wooldridge’s dropbox for the code in Stata. The basic idea is to treat cohorts as different treatment beginning time and exit time combinations. Then do the same regression with interactions of cohort dummies with the time dummies. It is very flexible. The tricky part is to get the marginal effect and its standard error right.

---

*Systematic treatment:* [R](#) · [Julia](#).



## 22. Causal Panel (why DDDiD)

### 22.1. Why DDDiD

Guido Imbens has been saying DDDiD (Don't Do Diff in Diff). But why?

To see that, DiD objective function (in the form of TWFE) is:

$$(\hat{\tau}^{did}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau, \mu, \alpha, \beta}{argmin} \sum_{i=1}^N \sum_{t=1}^T (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \quad (22.1)$$

We can see that in this objective function, we are allowing only  $\alpha_i$  and  $\beta_t$  to capture the difference between units and time periods. The obvious drawback is that control units that are less similar to a treated unit is given the same weight as those that are more similar. A time period that is far from the beginning of the treatment period is given the same weight as those that are closer. Say the treatment period is 2000, we have observations from 1988 to 2005, then 1988 observations are the same as 1999 observations. This is not ideal. The same logic applies to the units.

### 22.2. Synthetic Control

SC objective function:

$$(\hat{\tau}^{sc}, \hat{\mu}, \hat{\beta}) = \underset{\tau, \mu, \beta}{argmin} \sum_{i=1}^N \sum_{t=1}^T (Y_{it} - \mu - \beta_t - W_{it}\tau)^2 \hat{\omega}_i^{sc} \quad (22.2)$$

It's a weighted version of DiD, but note carefully that it is **not** a nesting of it: setting every  $\hat{\omega}_i^{sc}$  to 1 here does not recover the DiD objective above, because SC has no  $\alpha_i$  term — it would give a pooled regression with time effects only. Dropping the unit fixed effects is a substantive restriction, not a re-weighting. (DiD *is* nested in synthetic DiD below, which keeps  $\alpha_i$  and sets both weights to be constant; that is the right way to see DiD as a special case.) The weights are set to optimally match donor units to the treated unit so that they are as close as possible at each time point. There is no time weight.

### 22.3. Synthetic DiD

Arkhangelsky et al (2021) tries to combine the idea of DiD and SC. SC assigns different weights to different control units. The standard DiD is a TWFE, assigning equal weights to all time periods and units.

## 22. Causal Panel (why DDDiD)

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau, \mu, \alpha, \beta}{\operatorname{argmin}} \sum_{i=1}^N \sum_{t=1}^T \hat{\omega}_i^{sdid} \lambda_t^{sdid} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \quad (22.3)$$

SDiD sets another weight in addition to SC weights, which changes over time. The SC weights are trying to construct a control unit that is close to the treated unit; the SDiD weights are trying to put more weights on pre-treatment periods that are more similar to post-treatment periods.

### 22.4. Example from synthdid

The data are the California Proposition 99 panel: annual per-capita cigarette sales in packs for 39 states from 1970 to 2000, with California the single treated unit, 38 donors, 19 pre-treatment years and 12 post-treatment years.

```
library(synthdid)
data('california_prop99')
setup = panel.matrices(california_prop99)
tau.hat = synthdid_estimate(setup$Y, setup$N0, setup$T0)
summary(tau.hat)
```

\$estimate

[1] -15.60383

\$se

[,1]

[1,] NA

\$controls

estimate 1

|                |       |
|----------------|-------|
| Nevada         | 0.124 |
| New Hampshire  | 0.105 |
| Connecticut    | 0.078 |
| Delaware       | 0.070 |
| Colorado       | 0.058 |
| Illinois       | 0.053 |
| Nebraska       | 0.048 |
| Montana        | 0.045 |
| Utah           | 0.042 |
| New Mexico     | 0.041 |
| Minnesota      | 0.039 |
| Wisconsin      | 0.037 |
| West Virginia  | 0.034 |
| North Carolina | 0.033 |
| Idaho          | 0.031 |
| Ohio           | 0.031 |
| Maine          | 0.028 |

Iowa 0.026

\$periods

```
estimate 1
1988     0.427
1986     0.366
1987     0.206
```

\$dimensions

| N1    | NO     | NO.effective | T1     | T0     | T0.effective |
|-------|--------|--------------|--------|--------|--------------|
| 1.000 | 38.000 | 16.388       | 12.000 | 19.000 | 2.783        |

The estimate is  $-15.60$  packs per capita, and the summary shows how it gets there. The unit weights are concentrated — Nevada 0.124, New Hampshire 0.105, Connecticut 0.078 — giving an effective 16.4 donors out of 38. The time weights are more concentrated still: 0.427 on 1988, 0.366 on 1986 and 0.206 on 1987, which is an effective 2.8 pre-periods out of 19. That is the “time-aware” part of synthetic DiD in numbers, and it is the mechanism behind the estimator’s advantage over plain DiD.

Note also that `se` comes back NA. That is not a glitch: the default is a jackknife standard error, which deletes one treated unit at a time, and California is the only treated unit. The placebo method is the usable option here.

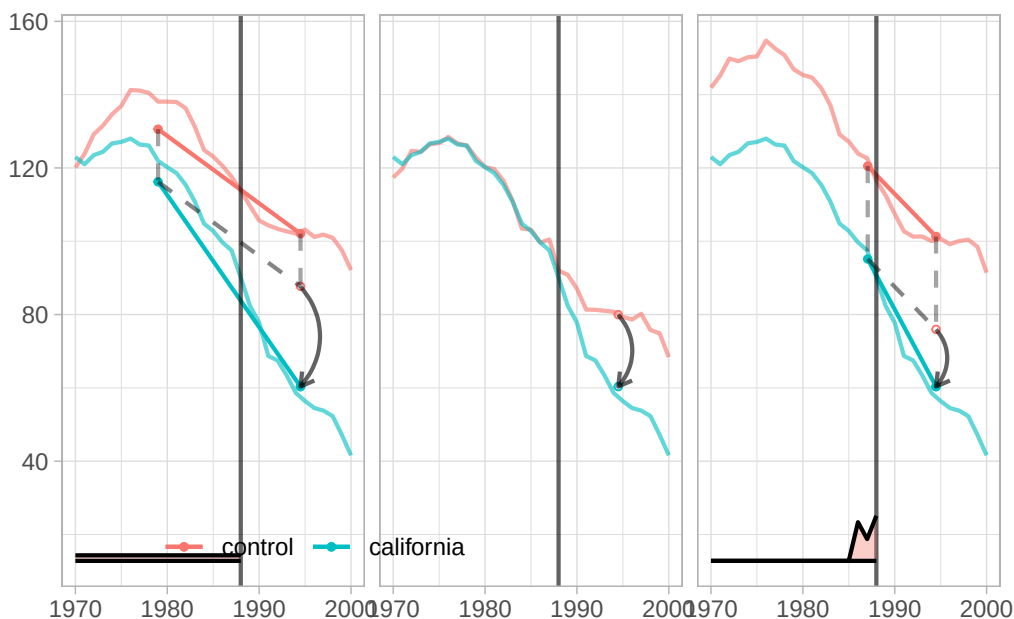
```
tau.sc = sc_estimate(setup$Y, setup$N0, setup$T0)
tau.did = did_estimate(setup$Y, setup$N0, setup$T0)
estimates = list(tau.did, tau.sc, tau.hat)
names(estimates) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates))
```

| Diff-in-Diff | Synthetic Control | Synthetic Diff-in-Diff |
|--------------|-------------------|------------------------|
| -27.34911    | -19.61966         | -15.60383              |

Plain DiD gives  $-27.35$ , synthetic control  $-19.62$ , and synthetic DiD  $-15.60$ . The spread across the three is nearly twelve packs per capita, on the same data. Not, though, on the same identification assumptions: the three differ in what they require of the donor pool and the pre-period, which is part of why they disagree. DiD moves furthest because it weights all 38 donors and all 19 pre-periods equally: states unlike California, and years unlike the late 1980s, all count in full.

```
plot <- synthdid_plot(estimates, facet.vertical=FALSE,
  control.name='control', treated.name='california',
  lambda.comparable=TRUE, se.method = 'none',
  trajectory.linetype = 1, line.width=.75, effect.curvature=-.4,
  trajectory.alpha=.7, effect.alpha=.7,
  diagram.alpha=1, onset.alpha=.7) +
  theme(legend.position=c(.26,.07), legend.direction='horizontal',
    legend.key=element_blank(), legend.background=element_blank(),
    strip.background=element_blank(), strip.text.x = element_blank())
plot
```

## 22. Causal Panel (why DDDiD)



Each panel shows California's path against its weighted control, with the estimated effect as the post-1989 gap. Note:  $\lambda_t$  is plotted at the bottom, so the pre-periods that actually enter each comparison are visible — for synthetic DiD they pile up on 1986–1988, while for DiD the weight is flat across all nineteen years.

### 22.5. Summary

To me, “DDDiD” is saying DiD is often too simple. That is not the same as saying a weighted estimator dominates. If parallel trends and no anticipation genuinely hold, DiD is identified, and synthetic DiD is not then a strictly better estimator – it buys robustness to a different failure mode, at the price of its own assumptions: a donor pool whose weighted pre-period path tracks the treated unit, and the low-rank or fixed-effects structure that justifies regularized unit and time weights. Nor does a closer pre-treatment fit by itself establish smaller post-treatment bias. The useful reading is that the three estimators lean on different structures, so agreement between them is informative and disagreement tells you which structure is doing the work.

---

*Systematic treatment:* [R](#) · [Julia](#).

## **Part V.**

# **Count Data & Specialized Models**



## 23. Which count data model to use

### 23.1. A comparison of various count data models with extra zeros

In empirical studies, data sets with a lot of zeros are often hard to model. There are various models to deal with it: zero-inflated Poisson model, Negative Binomial (NB) model, hurdle model, etc.

Here we are following a zero-inflated model's thinking: model the data with two processes. One is a Bernoulli process, the other one is a count data process (Poisson or NB).

We'd like to see, in this simulation exercise, how different models perform with changes of sample size and percentage of zeros (we expect the less zero, the better a plain Poisson model would perform). Therefore we vary sample size  $n$  and an indicator of how much percentage of zeros in the data  $\theta$ .

For the count data process ( $y_c$ ):

$$y_c = \lfloor \exp(2x + u) \rfloor, \quad u \sim N(0, \sigma^2) \quad (23.1)$$

Note what that is and is not. It is a floored lognormal variable, not a draw from a Poisson distribution, and not normal either. The flooring matters: it makes the conditional mean only approximately  $\exp(2x + \sigma^2/2)$ , so the log-linear index is not exactly  $2x$  and “bias relative to 2” is not a clean common target across the models compared below.

For the Bernoulli process ( $y_b$ ):

$$z_1 = 4z + \theta \quad (23.2)$$

$$\text{logit}(y_b) = z_1 \quad (23.3)$$

$$p_y = \frac{e^{z_1}}{1 + e^{z_1}} \quad (23.4)$$

where  $y_b \sim \text{Bernoulli}(p_y)$  is the realized 0/1 draw. Combining these two processes:

$$y = y_c \cdot y_b \quad (23.5)$$

so  $y = y_c$  when  $y_b = 1$  and  $y = 0$  when  $y_b = 0$ .

#### 23.1.1. Zero-inflated Poisson models

A zero-inflated Poisson needs specifying both the binary process and the count process correctly. Often than not, we don't have a model for the binary process. Many people simply use the same explanatory variables for both processes. We simulate both situations. Case 1: suppose we observe  $z$ , and case 2: suppose we don't observe  $z$ . In the graph below, they are labeled zip1 and zip2.

## 23. Which count data model to use

### 23.1.2. Poisson model

A plain Poisson model returns a consistent estimator for the coefficients, with or without Poisson-distributed data. We expect Poisson model's performance improve with sample size. Note that the standard errors from a Poisson model needs adjustment, which we do not discuss in this post.

### 23.1.3. NB model

NB model is used widely to handle “overdispersion” problem. That is, the variance far exceeds the mean, therefore the Poisson model is considered inappropriate. NB model addresses that by allowing an extra parameter. However, many people also use it to model “extra zero” situation, we'll see in our simulation it may not be better than a plain Poisson model.

### 23.1.4. Log-linear model

What about an OLS model with  $\log(y + 1)$ ?

### 23.1.5. hurdle model

A hurdle model models the zero's and other values separately; that is, the zero's are from a binomial process only, the other positive values are from a truncated count data process. We assume here, in the simulation, we don't observe  $z$ . Therefore,  $x$  is determining both binary and count processes. In the graph below, it's labeled hurdle.

```
library(MASS)
library(psc1)
library(parallel)
set.seed(666)

gen.sim <- function(df) {
  nobs <- df["nobs"]; th <- df["th"]
  z <- rnorm(nobs, 0, 1)
  x <- rnorm(nobs, 0, 1)
  u <- rnorm(nobs, 0, 1)
  y.count <- floor(exp(2 * x + u))
  z1 <- 4 * z + th
  prob <- plogis(z1)
  y.logit <- rbinom(nobs, size = 1, prob = prob)
  y <- ifelse(y.logit == 1, y.count, y.logit)

  m1 <- zeroinfl(y ~ x | z)
  m4 <- zeroinfl(y ~ x | x)
  m2 <- glm(y ~ x, family = "poisson")
  m3 <- lm(log(y + 1) ~ x)
  m5 <- glm.nb(y ~ x)
```



```

m6 <- hurdle(y ~ x)

c(zip1      = coef(m1, "count")["x"] - 2,
  poisson    = coef(m2) ["x"] - 2,
  log.linear = coef(m3) ["x"] - 2,
  zip2       = coef(m4, "count")["x"] - 2,
  nb         = coef(m5) ["x"] - 2,
  hurdle     = coef(m6, "count")["x"] - 2)
}

data.grid <- expand.grid(
  nobs = ceiling(exp(seq(4, 9, 1)) / 100) * 100,
  nsim = seq(1, 100, 1),
  th   = seq(-4, 4, 2)
)

cl      <- makeCluster(detectCores() - 1)
clusterEvalQ(cl, { library(MASS); library(psc1) })
clusterExport(cl, "gen.sim")
results <- parApply(cl, data.grid, 1, gen.sim)
stopCluster(cl)

forshiny <- cbind(data.grid, t(results))
write.csv(forshiny, "results.csv", row.names = FALSE)

```

Count data models can be used even if data is not “counts”; for example, some non-negative non-integer numbers. In fact, Poisson model is consistent even if data is not Poisson-distributed, if the model specification is correct on modeling the log of expected counts. We simulate both scenarios: Case 1, data is generated from a Poisson process. Case 2, data is a floored lognormal –  $\lfloor \exp(2x + u) \rfloor$  with  $u$  normal – so it is non-negative and integer-valued without being a count from any count distribution, and we use count data models on it anyway. (Earlier versions of this text called case 2 “Normal”; the variable being modelled is not normal, only the  $u$  inside the exponential is.) The above code is for case 2.

We simulate 100 times with  $\theta$  ranging from -4 to 4, lower number means higher percentage of zeros; number of observations from  $e^4$  to  $e^9$  (roughly from 50 to 8000).

Since there are many simulations, we use base R’s `parallel` library to speed things up.

For raw code, please visit [case1: poisson](#) and [case2: normal](#). The code above updates the original `snowfall`-based version to use base R `parallel`.

**Status of these simulation findings.** The grid above is retained for its code rather than as current evidence, and the ranking it once carried should not be read as verified. Two things undercut it. The metric for the log-linear row was wrong in the original run – see the note below – and the case-2 data-generating process is a floored lognormal whose log-linear index is not exactly  $2x$ , so subtracting 2 from each model’s slope does not compare like with like. The chunk is marked `eval: false`, so nothing here regenerates it.

### 23. Which count data model to use

What the run suggested, and what remains plausible on grounds independent of it: plain Poisson QMLE holds up well in large samples even with unmodelled excess zeros *when the conditional mean is correctly specified*; zero-inflated Poisson does better when the zero process is known; and log-linear regression on  $\log(y+1)$  targets a different quantity altogether. For evidence rather than recollection, the worked example in the next section is the part of this chapter that actually runs.

(Note: the `log.linear` bias line above previously read `exp(coef(m3)["x"]) - 2` instead of `coef(m3)["x"] - 2`, which — since the log-linear coefficient is already on the log scale — manufactured a spurious 5.4 apparent bias regardless of the model’s actual performance. That has been corrected here to match the other rows. Log-linear regression on  $\log(y+1)$  is still expected to be biased relative to Poisson-family models in a zero-inflated count setting, for real reasons (the log transform’s nonlinearity near zero, and the fact that  $E[\log(y+1)] \neq \log(E[y]+1)$ ), but the *magnitude* of “consistently the worst” above reflects the original (buggy) run and has not been re-verified against a corrected simulation.)

## 23.2. Quick worked example

All five models on a single small dataset ( $n = 1000$ , ~40% zeros).

The DGP is a zero-inflated Poisson built in two independent pieces. We draw  $x \sim N(0, 1)$  and  $z \sim N(0, 1)$ , independent of each other. The count part is  $y_c \sim \text{Poisson}(\exp(1.5 + 0.8x))$  and the zero part is  $y_b \sim \text{Bernoulli}(\Lambda(1 + 2z))$ , with the observed outcome  $y = y_c \cdot y_b$ . So the true slope on  $x$  is 0.8, and — this is the feature that drives the results — the zero process depends on  $z$  alone, never on  $x$ .

```
library(MASS)
library(psc1)
set.seed(42)

n <- 1000
x <- rnorm(n)
z <- rnorm(n)                # zero-inflation driver
mu <- exp(1.5 + 0.8 * x)      # count mean
y_c <- rpois(n, lambda = mu)
y_b <- rbinom(n, 1, plogis(1 + 2 * z)) # 1 = non-zero
y <- y_c * y_b                # zero-inflated count

cat("Zero fraction:", round(mean(y == 0), 2), "\n")
```

Zero fraction: 0.39

```
cat("Mean (non-zero):", round(mean(y[y > 0]), 2), "\n\n")
```

Mean (non-zero): 6.79

```

# 1. Poisson
m_pois <- glm(y ~ x, family = poisson)

# 2. Negative binomial
m_nb <- glm.nb(y ~ x)

# 3. Zero-inflated Poisson (ZIP) - correct zero model
m_zip <- zeroinfl(y ~ x | z)

# 4. ZIP - wrong zero model (only x)
m_zip2 <- zeroinfl(y ~ x | x)

# 5. Hurdle
m_hurdle <- hurdle(y ~ x)

# 6. Log-linear OLS on log(y + 1)
m_loglin <- lm(log(y + 1) ~ x)

# Compare x coefficient (true = 0.8)
coefs <- c(
  Poisson      = coef(m_pois)["x"],
  NB           = coef(m_nb)["x"],
  `ZIP (correct z)` = coef(m_zip, "count")["x"],
  `ZIP (wrong)` = coef(m_zip2, "count")["x"],
  Hurdle       = coef(m_hurdle, "count")["x"],
  `Log-linear log(y+1)` = coef(m_loglin)["x"]
)

knitr::kable(
  data.frame(Model = names(coefs), `x coefficient` = round(coefs, 3),
             Bias = round(coefs - 0.8, 3)),
  caption = "True x coefficient = 0.8"
)

```

Table 23.1.: True x coefficient = 0.8

|                       | Model                 | x.coefficient | Bias   |
|-----------------------|-----------------------|---------------|--------|
| Poisson.x             | Poisson.x             | 0.850         | 0.050  |
| NB.x                  | NB.x                  | 0.824         | 0.024  |
| ZIP (correct z).x     | ZIP (correct z).x     | 0.803         | 0.003  |
| ZIP (wrong).x         | ZIP (wrong).x         | 0.802         | 0.002  |
| Hurdle.x              | Hurdle.x              | 0.803         | 0.003  |
| Log-linear log(y+1).x | Log-linear log(y+1).x | 0.458         | -0.342 |

The sample comes out 39% zeros with a non-zero mean of 6.79. The estimated slopes are 0.850 for Poisson, 0.824 for negative binomial, 0.803 for ZIP with the correct zero model, 0.802 for ZIP with

### 23. Which count data model to use

the wrong one, 0.803 for the hurdle model, and 0.458 for log-linear OLS.

The first five models recover the true  $x$  slope of 0.8 reasonably well; the log-linear row is included to make the earlier claim checkable, and needs reading with care — see the note after the table. This is the key point of *this* DGP: the zero-inflation process depends on  $z$ , which is generated **independently of  $x$** , so omitting the zero model distorts the zero mass and the intercept, not the slope on  $x$ . The hurdle model and even the ZIP with a *misspecified* ( $x$ -only) zero model still recover the slope closely; the plain Poisson shows a small upward bias *in this particular sample* ( $n = 1000$ ), but that is finite-sample noise, not a systematic property of the model. Since the zero-inflation driver  $z$  is independent of  $x$ , the conditional mean here is correctly specified for Poisson QMLE — increasing  $n$  from 1,000 to 1,000,000 shrinks the Poisson  $x$ -coefficient bias from about +0.04 to effectively 0, confirming consistency. The unmodeled excess zeros create overdispersion, but overdispersion under a correctly-specified conditional mean affects only the standard errors (which should be corrected with a robust/sandwich estimator), not the consistency of the slope.

About the log-linear row: its coefficient is **not** estimating the same thing as the others, which is the real reason to avoid it rather than any particular bias number. The count models target  $\log E[y \mid x]$ , whereas OLS on  $\log(y + 1)$  targets  $E[\log(y + 1) \mid x]$ , and  $E[\log(y + 1)] \neq \log(E[y] + 1)$  by Jensen's inequality. The +1 offset makes the discrepancy worst exactly where the mass of zeros is. So comparing its coefficient to 0.8 is an apples-to-oranges comparison, and the honest statement is that log-linear answers a different question — not that it answers this one badly by some measured amount. A misspecified zero process biases the count *slope* only when the zero process is correlated with the count covariates — which is not the case here. (Change  $z$  to depend on  $x$  and the omitted-zero-process slope bias appears.)

## 23.3. Mixed count models with glmmTMB

When data has both overdispersion *and* random effects (clustered counts, panel data, repeated measures), `glmmTMB` handles the full range in one package:

```
library(glmmTMB)

# Zero-inflated negative binomial with random intercept per subject
m_glmmTMB <- glmmTMB(
  count ~ treatment + time + (1 | subject),    # count model
  ziformula = ~ treatment,                    # zero-inflation model
  family = nbinom2,
  data = your_data
)

summary(m_glmmTMB)

# Conditional vs marginal predictions
predict(m_glmmTMB, type = "conditional") # excluding zero-inflation
predict(m_glmmTMB, type = "response")    # including zero-inflation
```

**glmmTMB** supports: Poisson, NB1, NB2, zero-inflated variants of all of the above, hurdle models, beta-binomial, tweedie, and more. The **ziformula** argument takes a one-sided formula for the zero-inflation component — use `~ 1` for a constant zero-inflation rate, or covariates to model what drives the structural zeros.



## 24. What model to use for rare events

### 24.1. Introduction

In empirical studies, people are worried about rare event situation. That is, when you have, for example, lots of 0's and only a few 1's, or vice versa. Do you run a logit model, or do you use a “rare event logit”? When should you use either approach? Or there is a third approach?

Paul Allison said in his blog (<https://statisticalhorizons.com/logistic-regression-for-rare-events>):

“Prompted by a 2001 article by King and Zeng, many researchers worry about whether they can legitimately use conventional logistic regression for data in which events are rare. Although King and Zeng accurately described the problem and proposed an appropriate solution, there are still a lot of misconceptions about this issue.

The problem is not specifically the rarity of events, but rather the possibility of a small number of cases on the rarer of the two outcomes. If you have a sample size of 1000 but only 20 events, you have a problem. If you have a sample size of 10,000 with 200 events, you may be OK. If your sample has 100,000 cases with 2000 events, you're golden.”

In general I agree with him. However, when exactly should we use King and Zeng's “relogit”?

Allison also mentioned two other methods. One is called the Firth method, a penalized likelihood approach. The other one is the exact logistic regression, which is for small samples.

In this simulation exercise, I compare logit against bias-reduced logistic regression (`brglm2` with `method="brglmFit", type="AS_mean"`) and the Firth model (`logistf`).

One thing to be clear about before reading any comparison of the latter two: for a logistic regression **they are the same estimator**. Firth's penalized likelihood (penalizing by the Jeffreys prior) coincides with mean bias reduction for GLMs with a canonical link, and logit is canonical for the binomial. So `brglm2`'s `AS_mean` and `logistf` solve the same estimating equations and differ only in numerical tolerance and in how each package builds standard errors and intervals (`logistf` uses penalized profile likelihood). A demonstration, on a deliberately rare-event DGP:  $n = 500$ ,  $x \sim N(0, 1)$ , and  $y \sim \text{Bernoulli}(\Lambda(-3.5 + 0.8x))$ , which puts the event rate around 3% — roughly fifteen events in the whole sample.

```
library(brglm2); library(logistf)
set.seed(11)
n <- 500; x <- rnorm(n); y <- rbinom(n, 1, plogis(-3.5 + 0.8 * x))
d <- data.frame(y, x)

rbind(
  `plain logit`      = coef(glm(y ~ x, data = d, family = binomial)),
```

```
`brglm2 AS_mean` = coef(glm(y ~ x, data = d, family = binomial,
                           method = "brglmFit", type = "AS_mean")),
`logistf (Firth)` = coef(logistf(y ~ x, data = d))
)
```

|                 | (Intercept) | x         |
|-----------------|-------------|-----------|
| plain logit     | -3.176125   | 0.7378324 |
| brglm2 AS_mean  | -3.134453   | 0.7277674 |
| logistf (Firth) | -3.134453   | 0.7277675 |

The two bias-reduced fits give  $-3.134453$  and  $0.7277674$  against  $-3.134453$  and  $0.7277675$ : they agree to about  $10^{-7}$ , which is numerical tolerance rather than any statistical difference. Plain logit gives  $-3.176$  and  $0.738$ , so on this draw the bias correction moves the slope by about  $0.010$  and the intercept by  $0.042$  — both estimators sitting below the true  $0.8$  and above the true  $-3.5$ , which is what fifteen events buys you. This matters for reading the findings below: what the simulation can tell us is how **logit** compares with **Firth-type bias reduction**, not which of two implementations of the same estimator is better. If a genuinely distinct third method is wanted, **brglm2**'s `type = "AS_median"` (median bias reduction) is one, and King and Zeng's `relogit` correction is another.

## 24.2. Simulation

Here I have some code for using multiple cores to run these three models. The bias-reduced logistic regression is implemented in R via the **brglm2** package, the Firth method is implemented in **logistf**.

```
library(brglm2)
library(logistf)
require(snowfall)
set.seed(666)

# initialize parallel cores.
sfInit(parallel=TRUE, cpus=16)

gen.sim <- function(df){
  x <- rnorm(df['nobs'], 0, 1)

  # generate binary data
  p <- df['p']
  # NB: alpha = qlogis(p) makes p the event probability at x = 0, not the
  # marginal event probability. Because the slope on x is 2 and x ~ N(0,1),
  # the marginal rate is much higher than p: integrating plogis(alpha + 2x)
  # over x gives 0.044, 0.134 and 0.203 for p = 0.01, 0.05 and 0.10.
  alpha <- -log((1-p)/p)
  z = alpha + 2*x
  pr = 1/(1+exp(-z))
```



```

y = rbinom(df['nobs'], 1, pr)
df = data.frame(y=y, x=x)

# With small nobs and small p (e.g. nobs=10, p=.01), y can easily come out
# all-zero (>90% chance in that cell), which makes glm/logistf unable to
# estimate a coefficient on x. Guard against that rather than letting a
# single degenerate draw crash the whole parallel simulation loop.
if (var(y) == 0) {
  return(c(logit=NA, brglm2=NA, logistf=NA))
}

# logit
m1 <- tryCatch(glm(y ~ x, family='binomial'), error = function(e) NULL)
m1.x <- if (!is.null(m1)) summary(m1)$coefficients['x','Estimate'] - 2 else NA

# bias-reduced logistic regression (brglm2)
m2 <- tryCatch(
  glm(y ~ x, family = binomial(link = "logit"), data = df,
    method = "brglmFit", type = "AS_mean"),
  error = function(e) NULL
)
m2.x <- if (!is.null(m2)) coef(m2)['x'] - 2 else NA

# logistf (Firth)
m3 <- tryCatch(logistf(y ~ x, data=df), error = function(e) NULL)
m3.x <- if (!is.null(m3)) coef(m3)['x'] - 2 else NA

return(c(logit=m1.x, brglm2=m2.x, logistf=m3.x))
}

# set parameter space
sim.grid = seq(1, 100, 1)
p.grid = c(.01, .05, .1)
nobs.grid = c(10, 30, 50, 100, 200, 500, 1000, 10000)

data.grid <- expand.grid(nobs.grid, sim.grid, p.grid)
names(data.grid) <- c('nobs', 'nsim', 'p')

# export functions and libraries to parallel workers
sfExport(list=list("gen.sim"))
sfLibrary(brglm2)
sfLibrary(logistf)

results <- data.frame(t(sfApply(data.grid, 1, gen.sim)))

# stop the cluster
sfStop()

```

## 24. What model to use for rare events

```
forshiny <- cbind(data.grid, results)
write.csv(forshiny, 'results.csv')
```

We simulate 100 times with sample size from 10 to 10000, event probability .01, .05, and .1.

Since there are many simulations, we used the `snowfall` library to speed things up.

(The original post plotted bias and MSE by sample size and event probability from the `results.csv` output above; those figures are not reproduced here — the findings are summarized in prose below.)

A caveat on how to read the grid before the findings. The parameter `p` is the event probability at  $x = 0$ , not the marginal event probability: with a slope of 2 on  $x \sim N(0, 1)$ , the marginal rate is roughly 0.044, 0.134 and 0.203 for `p` = 0.01, 0.05 and 0.10. So the expected number of events in a cell is closer to  $n$  times those numbers than to  $np$ , by a factor of about four at the rare end. What follows is stated in terms of the realized event count, which is the quantity that actually governs the behaviour.

When the rarer of the two outcomes has only a handful of observations – fewer than about five events in the sample – none of these three models performs well. This is understandable: there is almost nothing in the data to identify the slope. Once the rarer group has more than about fifty events, there is not much difference between the three. In between, we found that Firth-type bias reduction — whether computed by `brglm2` or by `logistf` — performs better than plain logit. Any apparent ranking *between* those two should be discounted, since as shown above they are the same estimator; differences at that level reflect numerical tolerance or differing standard-error conventions, not a substantive advantage.

In the small sample situation, maybe it's better to use the exact logistic regression.

## 25. Poisson model with exposure

### 25.1. Poisson with exposure or Poisson rate

Poisson models for count data often arise when units are observed for different lengths of time. A hospital admission count for a patient followed for 30 days is not directly comparable to a count for a patient followed for 365 days. The natural quantity of interest is the *rate* — admissions per unit time — not the raw count.

There are three related but distinct specifications, each making a different assumption:

**Model 1 — Poisson with offset (rate model):**

$$\log(E[Y]/t) = \beta_0 + \beta_1 X \quad (25.1)$$

Equivalently,  $\log E[Y] = \beta_0 + \beta_1 X + \log t$ , where  $\log t$  is included as a *fixed* offset (coefficient constrained to 1). This is the standard exposure model: the expected count is proportional to exposure time.

**Model 2 — Poisson on the ratio  $Y/t$ :**

$$\log(E[Y/t]) = \beta_0 + \beta_1 X \quad (25.2)$$

This looks similar, but it is not equivalent to the offset model, and strictly speaking  $Y/t$  is **not a Poisson outcome at all**: a Poisson likelihood is defined for non-negative integer counts, and the continuous ratio  $Y/t$  is not one. Because  $t$  is a fixed constant for each unit,  $E[Y/t] = E[Y]/t$  exactly (linearity of expectation), so the *conditional mean* is still log-linear in  $X$ ; that is why the point estimates are often similar. But the Poisson variance assumption fails:  $\text{Var}(Y/t) = \lambda/t^2$ , whereas the mean is  $E[Y/t] = \lambda/t$ , so the variance is  $1/t$  times the mean rather than equal to it. (Writing it as  $\lambda/t$  would make variance equal mean and quietly restore equidispersion, which is precisely what fails here.) So fitting `glm(rate ~ x, family = poisson)` is a quasi-likelihood *mean model* rather than a correct Poisson likelihood. If you use it, treat it as a workaround: weight by exposure (`weights = t`) so the implied variance is right, and/or use robust standard errors. The offset model (Model 1) avoids the issue entirely by keeping the integer count as the outcome.

**Model 3 — Unconstrained log-exposure:**

$$\log E[Y] = \beta_0 + \beta_1 X + \gamma \log t \quad (25.3)$$

Here  $\gamma$  is estimated from the data. If  $\hat{\gamma} \approx 1$ , the proportionality assumption is supported. If  $\hat{\gamma} < 1$ , counts grow sub-linearly with exposure — a testable, substantive claim.

## 25.2. Simulation

```
library(tidyverse)
set.seed(42)

n <- 800

# Simulate a health study: count of adverse events
# Covariate: age group (0 = younger, 1 = older)
age_old <- rbinom(n, 1, 0.45)

# Exposure: days under observation (varies 7-365)
exposure <- sample(seq(7, 365, by = 7), n, replace = TRUE)

# True model: rate = exp(-3 + 1.2 * age_old)
# Count ~ Poisson(rate * exposure)
true_rate <- exp(-3 + 1.2 * age_old)
count <- rpois(n, lambda = true_rate * exposure)

df <- data.frame(count, age_old, exposure, log_exposure = log(exposure))
summary(df[, c("count", "exposure")])
```

| count          | exposure       |
|----------------|----------------|
| Min. : 0.00    | Min. : 7.0     |
| 1st Qu.: 7.00  | 1st Qu.: 112.0 |
| Median : 13.00 | Median : 196.0 |
| Mean : 18.95   | Mean : 192.9   |
| 3rd Qu.: 27.00 | 3rd Qu.: 281.8 |
| Max. : 71.00   | Max. : 364.0   |

We simulate  $n = 800$  patients. Age group is  $\text{age\_old} \sim \text{Bernoulli}(0.45)$ , and exposure is days under observation, drawn uniformly from the weekly grid 7 to 364 and independently of age. The event rate is  $\exp(-3 + 1.2 \text{age\_old})$  per day and the count is  $\text{Poisson}(\text{rate} \times \text{exposure})$ , so the true intercept is  $-3$ , the true age effect is 1.2, and the true exposure elasticity is  $\gamma = 1$  by construction.

The data has a wide range of exposure times — 7 to 364 days, median 196, with counts from 0 to 71 and a mean of 19 — so ignoring them would badly confound the comparison between age groups.

## 25.3. Fitting the three models

```
# Model 1: offset - coefficient on log(exposure) constrained to 1
m1 <- glm(count ~ age_old + offset(log(exposure)),
          family = poisson, data = df)
```

```
# Model 2: Poisson on the rate Y/t (different response)
df$rate <- df$count / df$exposure
m2 <- glm(rate ~ age_old,
          family = poisson, data = df)

# Model 3: log(exposure) as a free covariate
m3 <- glm(count ~ age_old + log_exposure,
          family = poisson, data = df)

# Collect coefficients
results <- data.frame(
  Model = c("1. Offset (constrained =1)",
            "2. Rate Y/t as response",
            "3. Unconstrained log(t)"),
  Intercept = c(coef(m1)[1], coef(m2)[1], coef(m3)[1]),
  age_old = c(coef(m1)[2], coef(m2)[2], coef(m3)[2]),
  gamma_log_t = c(1, NA, coef(m3)[3])
)

knitr::kable(results, digits = 3,
  caption = "Coefficient estimates: true intercept = -3, true age effect = 1.2, true  $\gamma$  = 1")
```

Table 25.1.: Coefficient estimates: true intercept =  $-3$ , true age effect =  $1.2$ , true  $\gamma = 1$ 

| Model                               | Intercept | age_old | gamma_log_t |
|-------------------------------------|-----------|---------|-------------|
| 1. Offset (constrained $\gamma=1$ ) | -3.009    | 1.212   | 1.000       |
| 2. Rate Y/t as response             | -2.980    | 1.176   | NA          |
| 3. Unconstrained log(t)             | -2.801    | 1.212   | 0.962       |

All three recover the truth. The offset model gives an intercept of  $-3.009$  and an age effect of  $1.212$  against true values of  $-3$  and  $1.2$ ; the rate model gives  $-2.980$  and  $1.176$ ; the unconstrained model gives  $-2.801$ ,  $1.212$  and  $\hat{\gamma} = 0.962$  against a true  $\gamma$  of  $1$ .

**Reading the table:** Model 1 and Model 3 give nearly identical age-group coefficients because the true  $\gamma = 1$  (counts are indeed proportional to exposure time). Model 2 differs slightly in the intercept because the response variable changes. The estimated  $\hat{\gamma}$  from Model 3 should be close to  $1$ .

## 25.4. When $\gamma \neq 1$

To see why the unconstrained model matters, repeat the simulation with a sub-linear count process ( $\gamma = 0.6$ : an initial burst of events that doesn't grow proportionally with long follow-up):

## 25. Poisson model with exposure

```
count2 <- rpois(n, lambda = true_rate * exposure^0.6)
df2 <- data.frame(count2, age_old, exposure, log_exposure = log(exposure))

m1b <- glm(count2 ~ age_old + offset(log(exposure)),
            family = poisson, data = df2)
m3b <- glm(count2 ~ age_old + log_exposure,
            family = poisson, data = df2)

cat("Offset model age coefficient:      ", round(coef(m1b)[2], 3), "\n")
```

```
Offset model age coefficient:      1.171
```

```
cat("Unconstrained model age coefficient:", round(coef(m3b)[2], 3), "\n")
```

```
Unconstrained model age coefficient: 1.168
```

```
cat("Estimated gamma (log-exposure):    ", round(coef(m3b)[3], 3),
    " (true = 0.6)\n")
```

```
Estimated gamma (log-exposure):    0.566 (true = 0.6)
```

Two things to read off this, and the first is not what one might expect.

The offset model's age coefficient (1.171) and the unconstrained model's (1.168) are essentially the same, and both sit just below the true 1.2. Forcing  $\gamma = 1$  when the truth is 0.6 does **not** bias the treatment coefficient here — and the reason is structural: `exposure` is drawn by `sample(...)` independently of `age_old`, so the misspecified exposure elasticity is absorbed by the intercept and cannot contaminate a coefficient on a covariate orthogonal to it. The constraint would bite if exposure were *correlated* with the covariate of interest — for instance if older patients were systematically followed longer, which is common in real cohort data and is the case worth worrying about.

What the unconstrained model does buy, unambiguously, is  $\gamma$  itself: it recovers  $\hat{\gamma} = 0.566$  against a truth of 0.6, whereas the offset model cannot report it at all because it fixes it at 1. So Model 3's role is diagnostic — it tells you whether the proportionality assumption holds — rather than protective of the treatment estimate.

### 25.5. Stata equivalent

```
* Model 1: offset
poisson count age_old, exposure(exposure)

* Model 2: rate as response
```

```

gen rate = count / exposure
* Stata will warn that the dependent variable is non-integer (poisson expects
* integer counts by default) -- this is expected and can be ignored here,
* since Poisson QMLE is valid for continuous non-negative outcomes too.
poisson rate age_old

* Model 3: unconstrained log(t)
gen log_exposure = log(exposure)
poisson count age_old log_exposure

```

## 25.6. Which model to use?

| Situation                                | Recommended model                                                                                 |
|------------------------------------------|---------------------------------------------------------------------------------------------------|
| Counts grow proportionally with time     | Model 1 (offset)                                                                                  |
| Uncertain about proportionality          | Model 3 (unconstrained), test $\hat{\gamma} = 1$                                                  |
| Rate is the natural estimand             | Model 1 (offset) — it estimates a rate directly;<br>Model 2 only as a weighted/robust quasi-model |
| Long follow-up with declining event rate | Model 3 or negative binomial with offset                                                          |

In most health and social-science applications, Model 1 (Poisson with offset) is the default. Model 3 is the diagnostic check: fit it, look at  $\hat{\gamma}$ , and return to Model 1 if the constraint is supported by the data.

---

*Systematic treatment:* [R](#) · [Julia](#).





## 26. Instrumental Variables in Fixed-Effects Poisson Models

### 26.1. The Problem: Endogeneity with Exponential Conditional Means

In panel count data models, the standard conditional mean specification is:

$$E[y_{it} \mid x_{it}, c_i] = c_i \exp(x_{it}\beta)$$

When an explanatory variable in  $x_{it}$  is endogenous, standard linear Two-Stage Least Squares (2SLS) cannot be applied because the conditional mean is exponential rather than additive. Meanwhile, nonlinear IV estimators (such as `ivpoisson` in Stata) cannot absorb high-dimensional individual and time fixed effects.

To estimate an endogenous Poisson model with multiple fixed effects, Lin and Wooldridge (2019) establish a two-step **control function** approach.

### 26.2. The Two-Step Control Function Recipe

For a continuous endogenous regressor  $w_{it}$  and excluded instruments  $z_{it}$ :

1. **First Stage (Reduced Form):** Regress the endogenous variable on the instruments and exogenous controls, absorbing all fixed effects:

$$w_{it} = z_{it}\pi + x_{it1}\gamma + c_i + f_t + v_{it}$$

Obtain the OLS residual  $\hat{v}_{it}$ . In Stata, use `reghdfe ... , resid`; in R, use `fixest::feols()`.

2. **Second Stage (Structural Poisson):** Estimate the fixed-effects Poisson model including both  $w_{it}$  and the first-stage residual  $\hat{v}_{it}$ , absorbing the same fixed effects:

$$E[y_{it} \mid w_{it}, x_{it1}, c_i, f_t, \hat{v}_{it}] = c_i f_t \exp(w_{it}\beta_1 + x_{it1}\beta_2 + \rho\hat{v}_{it})$$

In Stata, use `ppmlhdfe`; in R, use `fixest::fepois()`.

3. **Inference & Testing:**

- A  $t$ -test of  $H_0 : \rho = 0$  serves as a direct test of exogeneity.
- Because  $\hat{v}_{it}$  is a generated regressor, reported second-stage standard errors are invalid. Standard errors must be computed via a panel/cluster bootstrap re-estimating both stages in each replication.

### 26.3. Continuous Endogenous Variable: Application in Stata

Using Stata's `website.dta` ( $N = 500$ ): - Outcome: `visits` (count) - Endogenous regressor: `time` (minutes on site) - Instrument: `phone` (support calls) - Exogenous control: `frfam` (referrals) - Fixed effects: `ad` (13 campaigns) and `female` (gender)

```
webuse website, clear
```

```
* Step 1: Linear first stage absorbing both fixed effects
```

```
reghdfe time phone frfam, absorb(ad female) resid
```

```
predict double u2h_fe, resid
```

```
* Step 2: FE Poisson with control function residual
```

```
ppmlhdfe visits time u2h_fe frfam, absorb(ad female)
```

(Visits to website)

(dropped 1 singleton observations)

(MWFE estimator converged in 3 iterations)

|                         |               |   |        |
|-------------------------|---------------|---|--------|
| HDFE Linear regression  | Number of obs | = | 499    |
| Absorbing 2 HDFE groups | F( 2, 484)    | = | 29.02  |
|                         | Prob > F      | = | 0.0000 |
|                         | R-squared     | = | 0.3137 |
|                         | Adj R-squared | = | 0.2938 |
|                         | Within R-sq.  | = | 0.1071 |
|                         | Root MSE      | = | 2.2239 |

|  | time  | Coefficient | Std. err. | t    | P> t  | [95% conf. interval] |
|--|-------|-------------|-----------|------|-------|----------------------|
|  | phone | .2705429    | .0626834  | 4.32 | 0.000 | .1473778 .393708     |
|  | frfam | .2539231    | .0619373  | 4.10 | 0.000 | .1322239 .3756222    |
|  | _cons | 1.406026    | .2549245  | 5.52 | 0.000 | .9051305 1.906921    |

Absorbed degrees of freedom:

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| ad          | 12         | 0           | 12           |
| female      | 2          | 1           | 1            |

(1 missing value generated)

Iteration 1: deviance = 3.7303e+02 eps = .      iters = 3      tol = 1.0e-0

```

> 4
> min(eta) = -0.57 P
Iteration 2: deviance = 3.5849e+02 eps = 4.06e-02 iters = 3 tol = 1.0e-0
> 4
> min(eta) = -0.59
Iteration 3: deviance = 3.5840e+02 eps = 2.46e-04 iters = 3 tol = 1.0e-0
> 4
> min(eta) = -0.60
Iteration 4: deviance = 3.5840e+02 eps = 3.17e-08 iters = 2 tol = 1.0e-0
> 4
> min(eta) = -0.60
Iteration 5: deviance = 3.5840e+02 eps = 9.72e-16 iters = 2 tol = 1.0e-0
> 5
> min(eta) = -0.60 S
Iteration 6: deviance = 3.5840e+02 eps = 1.62e-16 iters = 1 tol = 1.0e-0
> 9
> min(eta) = -0.60 S 0

```

```

-----
> -----
(legend: p: exact partial-out s: exact solver h: step-halving o: epsilon
> below tolerance)

```

Converged in 6 iterations and 14 HDFE sub-iterations (tol = 1.0e-08)

|                                     |              |   |        |
|-------------------------------------|--------------|---|--------|
| HDFE PPML regression                | No. of obs   | = | 499    |
| Absorbing 2 HDFE groups             | Residual df  | = | 483    |
|                                     | Wald chi2(3) | = | 169.93 |
| Deviance = 358.4027719              | Prob > chi2  | = | 0.0000 |
| Log pseudolikelihood = -993.8547331 | Pseudo R2    | = | 0.2976 |

|        | visits | Coefficient | Robust<br>std. err. | z     | P> z  | [95% conf. interval] |
|--------|--------|-------------|---------------------|-------|-------|----------------------|
| time   |        | .0448103    | .0448665            | 1.00  | 0.318 | -.0431265 .1327471   |
| u2h_fe |        | .0765144    | .0459404            | 1.67  | 0.096 | -.0135272 .166556    |
| frfam  |        | .0004108    | .0205161            | 0.02  | 0.984 | -.0398 .0406215      |
| _cons  |        | 1.510566    | .1033122            | 14.62 | 0.000 | 1.308078 1.713054    |

Absorbed degrees of freedom:

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| ad          | 12         | 0           | 12           |
| female      | 2          | 1           | 1            |

The coefficient on `time` is 0.0448 ( $p = 0.318$ ). The residual coefficient is  $\hat{\rho} = 0.0765$  ( $p = 0.096$ ), indicating weak positive selection.

### 26.3.1. Comparison with Linear Models

In a linear model, the control function estimator is algebraically identical to 2SLS. Comparing linear specifications verifies that the two stages are wired correctly:

```
clear all
webuse website, clear

* 1. Linear 2SLS
ivreghdfe visits frfam (time=phone), absorb(ad female)

* 2. Linear control function
reghdfe time phone frfam, absorb(ad female) resid
predict double u2h_fe, resid
reghdfe visits time u2h_fe frfam, absorb(ad female)

* 3. Naive models (ignoring endogeneity)
reghdfe visits time frfam, absorb(ad female)
ppmlhdfe visits time frfam, absorb(ad female)
```

(Visits to website)

(dropped 1 singleton observations)

(MWFE estimator converged in 3 iterations)

IV (2SLS) estimation

Estimates efficient for homoskedasticity only  
Statistics consistent for homoskedasticity only

|                       |   |             |  |  |                 |        |
|-----------------------|---|-------------|--|--|-----------------|--------|
|                       |   |             |  |  | Number of obs = | 499    |
|                       |   |             |  |  | F( 2, 484) =    | 8.31   |
|                       |   |             |  |  | Prob > F =      | 0.0003 |
| Total (centered) SS   | = | 3335.918099 |  |  | Centered R2 =   | 0.3971 |
| Total (uncentered) SS | = | 3335.918099 |  |  | Uncentered R2 = | 0.3971 |
| Residual SS           | = | 2011.119506 |  |  | Root MSE =      | 2.038  |

| visits | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|--------|-------------|-----------|-------|-------|----------------------|----------|
| time   | .5642802    | .2123745  | 2.66  | 0.008 | .1469904             | .98157   |
| frfam  | -.04045     | .0922873  | -0.44 | 0.661 | -.2217832            | .1408833 |

Underidentification test (Anderson canon. corr. LM statistic): 18.494  
Chi-sq(1) P-val = 0.0000

### 26.3. Continuous Endogenous Variable: Application in Stata

```
Weak identification test (Cragg-Donald Wald F statistic):          18.628
Stock-Yogo weak ID test critical values: 10% maximal IV size      16.38
                                         15% maximal IV size      8.96
                                         20% maximal IV size      6.66
                                         25% maximal IV size      5.53
```

Source: Stock-Yogo (2005). Reproduced by permission.

```
-----
Sargan statistic (overidentification test of all instruments):      0.000
                                         (equation exactly identified)
-----
```

```
Instrumented:      time
Included instruments: frfam
Excluded instruments: phone
Partialled-out:    _cons
                   nb: total SS, model F and R2s are after partialling-out;
                   any small-sample adjustments include partialled-out
                   variables in regressor count K
-----
```

Absorbed degrees of freedom:

```
-----+
Absorbed FE | Categories - Redundant = Num. Coefs |
-----+-----+-----+-----+
      ad |      12      0      12      |
      female |      2      1      1      |
-----+-----+-----+-----+
```

(dropped 1 singleton observations)  
(MWFE estimator converged in 3 iterations)

```
HDFE Linear regression          Number of obs   =      499
Absorbing 2 HDFE groups         F(   2,   484) =     29.02
                                Prob > F          =     0.0000
                                R-squared           =     0.3137
                                Adj R-squared       =     0.2938
                                Within R-sq.       =     0.1071
                                Root MSE        =     2.2239
```

```
-----
      time | Coefficient Std. err.      t    P>|t|     [95% conf. interval]
-----+-----
      phone |   .2705429   .0626834    4.32  0.000   .1473778   .393708
      frfam |   .2539231   .0619373    4.10  0.000   .1322239   .3756222
      _cons |   1.406026   .2549245    5.52  0.000   .9051305   1.906921
-----
```

Absorbed degrees of freedom:

```
-----+
```

## 26. Instrumental Variables in Fixed-Effects Poisson Models

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| ad          | 12         | 0           | 12           |
| female      | 2          | 1           | 1            |

(1 missing value generated)

(MWFE estimator converged in 3 iterations)

|                         |               |   |        |
|-------------------------|---------------|---|--------|
| HDFE Linear regression  | Number of obs | = | 499    |
| Absorbing 2 HDFE groups | F( 3, 483)    | = | 117.11 |
|                         | Prob > F      | = | 0.0000 |
|                         | R-squared     | = | 0.7677 |
|                         | Adj R-squared | = | 0.7605 |
|                         | Within R-sq.  | = | 0.4211 |
|                         | Root MSE      | = | 1.9996 |

| visits | Coefficient | Std. err. | t     | P> t  | [95% conf. interval] |          |
|--------|-------------|-----------|-------|-------|----------------------|----------|
| time   | .5642802    | .2083276  | 2.71  | 0.007 | .15494               | .9736205 |
| u2h_fe | .1827169    | .2122987  | 0.86  | 0.390 | -.2344262            | .5998601 |
| frfam  | -.04045     | .0905287  | -0.45 | 0.655 | -.2183288            | .1374288 |
| _cons  | 3.357879    | .4427337  | 7.58  | 0.000 | 2.487957             | 4.227801 |

Absorbed degrees of freedom:

| Absorbed FE | Categories | - Redundant | = Num. Coefs |
|-------------|------------|-------------|--------------|
| ad          | 12         | 0           | 12           |
| female      | 2          | 1           | 1            |

(dropped 1 singleton observations)

(MWFE estimator converged in 3 iterations)

|                         |               |   |        |
|-------------------------|---------------|---|--------|
| HDFE Linear regression  | Number of obs | = | 499    |
| Absorbing 2 HDFE groups | F( 2, 484)    | = | 175.38 |
|                         | Prob > F      | = | 0.0000 |
|                         | R-squared     | = | 0.7673 |
|                         | Adj R-squared | = | 0.7606 |
|                         | Within R-sq.  | = | 0.4202 |
|                         | Root MSE      | = | 1.9991 |

| visits | Coefficient | Std. err. | t | P> t | [95% conf. interval] |  |
|--------|-------------|-----------|---|------|----------------------|--|
|--------|-------------|-----------|---|------|----------------------|--|

|       |  |           |          |       |       |          |          |
|-------|--|-----------|----------|-------|-------|----------|----------|
| time  |  | .7402254  | .0400951 | 18.46 | 0.000 | .6614435 | .8190073 |
| frfam |  | -.1034523 | .053248  | -1.94 | 0.053 | -.208078 | .0011735 |
| _cons |  | 3.027348  | .2202211 | 13.75 | 0.000 | 2.594641 | 3.460055 |

Absorbed degrees of freedom:

|             |  |            |             |              |  |
|-------------|--|------------|-------------|--------------|--|
| Absorbed FE |  | Categories | - Redundant | = Num. Coefs |  |
| ad          |  | 12         | 0           | 12           |  |
| female      |  | 2          | 1           | 1            |  |

(dropped 1 observations that are either singletons or separated by a fixed effect)

Iteration 1: deviance = 3.7573e+02 eps = .      iters = 3      tol = 1.0e-04

> min(eta) = -0.64 P

Iteration 2: deviance = 3.6107e+02 eps = 4.06e-02 iters = 3      tol = 1.0e-04

> min(eta) = -0.64

Iteration 3: deviance = 3.6098e+02 eps = 2.57e-04 iters = 3      tol = 1.0e-04

> min(eta) = -0.64

Iteration 4: deviance = 3.6098e+02 eps = 3.76e-08 iters = 2      tol = 1.0e-04

> min(eta) = -0.64

Iteration 5: deviance = 3.6098e+02 eps = 1.45e-15 iters = 2      tol = 1.0e-05

> min(eta) = -0.64 S

Iteration 6: deviance = 3.6098e+02 eps = 0.00e+00 iters = 1      tol = 1.0e-09

> min(eta) = -0.64 S 0

> -----

(legend: p: exact partial-out    s: exact solver    h: step-halving    o: epsilon below tolerance)

Converged in 6 iterations and 14 HDFE sub-iterations (tol = 1.0e-08)

|                                     |              |   |        |
|-------------------------------------|--------------|---|--------|
| HDFE PPML regression                | No. of obs   | = | 499    |
| Absorbing 2 HDFE groups             | Residual df  | = | 484    |
|                                     | Wald chi2(2) | = | 164.67 |
| Deviance = 360.9797175              | Prob > chi2  | = | 0.0000 |
| Log pseudolikelihood = -995.1432059 | Pseudo R2    | = | 0.2967 |

|        |  |             |           |   |                                 |
|--------|--|-------------|-----------|---|---------------------------------|
|        |  | Robust      |           |   |                                 |
| visits |  | Coefficient | std. err. | z | P> z       [95% conf. interval] |

|       |  |           |          |       |       |           |           |
|-------|--|-----------|----------|-------|-------|-----------|-----------|
| time  |  | .1187612  | .0095895 | 12.38 | 0.000 | .0999662  | .1375563  |
| frfam |  | -.0281572 | .0099614 | -2.83 | 0.005 | -.0476812 | -.0086332 |
| _cons |  | 1.345121  | .0497098 | 27.06 | 0.000 | 1.247692  | 1.442551  |

Absorbed degrees of freedom:

|             |  |            |             |              |  |
|-------------|--|------------|-------------|--------------|--|
| Absorbed FE |  | Categories | - Redundant | = Num. Coefs |  |
| ad          |  | 12         | 0           | 12           |  |
| female      |  | 2          | 1           | 1            |  |

| Model                       | Estimator                  | time Coef. | Std. Err. | p-value |
|-----------------------------|----------------------------|------------|-----------|---------|
| Naive Linear FE             | reghdfe                    | 0.7402     | 0.0401    | < 0.001 |
| Linear 2SLS                 | ivreghdfe                  | 0.5643     | 0.2124    | 0.008   |
| Linear Control Function     | Manual two-step            | 0.5643     | 0.2083    | 0.007   |
| Naive FE Poisson            | ppmlhdfe (no residual)     | 0.1188     | 0.0096    | < 0.001 |
| Control-Function FE Poisson | ppmlhdfe (with $\hat{v}$ ) | 0.0448     | 0.0449    | 0.318   |

In linear models, manual control function reproduces 2SLS point estimates exactly. In the Poisson model, addressing endogeneity drops the estimated effect of `time` from 0.1188 to 0.0448.

### 26.3.2. Cluster Bootstrap in Stata

Because clusters may be sampled multiple times during replacement, each resampled cluster copy must receive a unique identifier via `idcluster()` to prevent duplicates from being pooled into a single fixed effect:

```
clear all
capture program drop ppmlhdfe_cf
program ppmlhdfe_cf, rclass
    reghdfe time phone frfam, absorb(newid female) resid
    predict double u2h_fe, resid
    ppmlhdfe visits time u2h_fe frfam, absorb(newid female)
    return scalar b_time = _b[time]
    return scalar b_u2h = _b[u2h_fe]
    drop u2h_fe
    xtset, clear
```



```
end
```

```
webuse website, clear
```

```
xtset, clear
```

```
bootstrap r(b_time) r(b_u2h), reps(1000) seed(123) cluster(ad) idcluster(newid) nodots: ppmlhd
```

(Visits to website)

Bootstrap results

Number of obs = 499

Replications = 1,000

Command: ppmlhdfc\_cf

\_bs\_1: r(b\_time)

\_bs\_2: r(b\_u2h)

(Replications based on 12 clusters in ad)

|       | Observed    | Bootstrap |      |       | Normal-based         |          |
|-------|-------------|-----------|------|-------|----------------------|----------|
|       | coefficient | std. err. | z    | P> z  | [95% conf. interval] |          |
| _bs_1 | .0448103    | .0477457  | 0.94 | 0.348 | -.0487697            | .1383902 |
| _bs_2 | .0765144    | .0486568  | 1.57 | 0.116 | -.0188511            | .1718799 |

The bootstrap standard error on `time` is 0.0477 (compared to the unadjusted analytic standard error of 0.0449).

## 26.4. Implementation in R: *fixest*

In R, the same workflow is implemented using `fixest::feols()` and `fixest::fepois()`. Note that `feols()` drops singleton clusters (here `ad == 10`), so the residual vector must be aligned using `obs()`:

```
library(fixest)
```

```
library(haven)
```

```
df <- as.data.frame(read_dta("https://www.stata-press.com/data/r18/website.dta"))
```

```
df$ad <- as.factor(df$ad)
```

```
df$female <- as.factor(df$female)
```

```
# Step 1: Linear first stage
```

```
fs <- feols(time ~ phone + frfam | ad + female, data = df)
```

```
# Align rows (feols drops singletons)
d <- df[obs(fs), ]
d$u2h_fe <- residuals(fs)

# Step 2: FE Poisson with and without control function
m_naive <- fepois(visits ~ time + frfam | ad + female, data = d, notes = FALSE)
m_cf <- fepois(visits ~ time + u2h_fe + frfam | ad + female, data = d, notes = FALSE)

etable(m_naive, m_cf, headers = c("Naive FE Poisson", "Control Function"), se.below = TRUE)
```

|                                                               | m_naive               | m_cf               |
|---------------------------------------------------------------|-----------------------|--------------------|
|                                                               | Naive FE Poisson      | Control Function   |
| Dependent Var.:                                               | visits                | visits             |
| time                                                          | 0.1188***<br>(0.0076) | 0.0448<br>(0.0477) |
| frfam                                                         | -0.0282**<br>(0.0108) | 0.0004<br>(0.0211) |
| u2h_fe                                                        |                       | 0.0765<br>(0.0487) |
| Fixed-Effects:                                                | -----                 | -----              |
| ad                                                            | Yes                   | Yes                |
| female                                                        | Yes                   | Yes                |
| -----                                                         | -----                 | -----              |
| S.E. type                                                     | IID                   | IID                |
| Observations                                                  | 499                   | 499                |
| Squared Cor.                                                  | 0.75130               | 0.75528            |
| Pseudo R2                                                     | 0.29670               | 0.29761            |
| BIC                                                           | 2,083.5               | 2,087.1            |
| ---                                                           |                       |                    |
| Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1 |                       |                    |

The point estimates match Stata to seven digits: 0.1187612 (naive) and 0.0448103 (control function).

### 26.4.1. Cluster Bootstrap in R

```
set.seed(123)
B <- 1000
clusters <- unique(d$ad)
boot_coefs <- matrix(NA_real_, nrow = B, ncol = 2, dimnames = list(NULL, c("time", "resid")))

for (b in seq_len(B)) {
  drawn <- sample(clusters, length(clusters), replace = TRUE)
```

```

boot_data <- do.call(rbind, lapply(seq_along(drawn), function(k) {
  sub <- d[d$ad == drawn[k], ]
  sub$boot_id <- k
  sub
}))
boot_data$boot_id <- as.factor(boot_data$boot_id)

fs_b <- feols(time ~ phone + frfam | boot_id + female, data = boot_data, notes = FALSE)
bb <- boot_data[obs(fs_b), ]
bb$u_b <- residuals(fs_b)

m_b <- try(fepois(visits ~ time + u_b + frfam | boot_id + female, data = bb, notes = FALSE),
  if (!inherits(m_b, "try-error")) boot_coefs[b, ] <- coef(m_b)[c("time", "u_b")]
}

cat("Completed replications :", sum(complete.cases(boot_coefs)), "\n")

```

Completed replications : 1000

```
cat("Bootstrap SE, time      :", round(sd(boot_coefs[, "time"], na.rm = TRUE), 4), "\n")
```

Bootstrap SE, time : 0.045

```
cat("Bootstrap SE, residual :", round(sd(boot_coefs[, "resid"], na.rm = TRUE), 4), "\n")
```

Bootstrap SE, residual : 0.0457

The R bootstrap yields  $SE = 0.0450$ . With only 12 clusters in `ad`, small differences across software packages (0.0477 in Stata vs. 0.0450 in R) reflect finite-sample sensitivity in how rare degenerate resamples and singletons are handled.

## 26.5. Extension: Binary Endogenous Regressor

When the endogenous regressor  $D_{it}$  is binary (e.g., participation in a program), a linear first stage is technically misspecified because the error cannot satisfy conditional independence.

### 26.5.1. The CRE Logit Control Function

1. **Avoid Dummy Variable Incidental Parameter Trap:** Estimating unit dummies in a first-stage logit with small  $T$  causes severe bias (e.g., slopes inflated by  $2\times$  when  $T = 2$ ).

2. **Correlated Random Effects (Mundlak Device):** Instead of unit dummies, include unit-level averages of time-varying covariates ( $\bar{z}_i$ ) in a pooled logit:

$$P(D_{it} = 1 \mid z_{it}, \bar{z}_i) = \Lambda(z_{it}\gamma + \bar{z}_i\pi)$$

3. **Generalized Residual:** For a logit model, the generalized residual simplifies directly to the response residual:

$$\hat{r}_{it} = D_{it} - \hat{p}_{it}$$

4. **Second Stage:** Include  $\hat{r}_{it}$  directly in the fixed-effects Poisson model.

### 26.5.2. Simulation Demonstration

We simulate  $N = 2,000$  units with unbalanced panels ( $T_i \in [2, 6]$ ), a binary endogenous treatment  $D$ , and true treatment parameter  $\beta = 0.30$ :

```
preserve
clear
set seed 20260731
set obs 2000
gen i = _n
gen double ci = rnormal()*0.5
gen T = 2 + int(runiform()*5)      /* 2-6 observations per unit */
expand T
bysort i: gen t = _n

gen double z = rnormal()           /* instrument */
gen double e = rnormal()
gen double u = 0.8*e + rnormal()*0.6 /* correlated with e -> endogeneity */
gen byte d = (-0.2 + 0.9*z + ci + e > 0)
gen double mu = exp(0.3*d + ci + 0.5*u)
gen y = rpoisson(mu)               /* true beta = 0.3 */

* 1. Naive FE Poisson (ignores endogeneity)
quietly ppmlhdfe y d, absorb(i)
scalar b_naive = _b[d]

* 2. CF with CRE (Mundlak) logit first stage
bysort i: egen double z_i = mean(z)
quietly logit d z z_i
predict double phat, pr
gen double gr = d - phat
quietly ppmlhdfe y d gr, absorb(i)
scalar b_cre = _b[d]

* 3. CF with linear first stage
quietly reghdfe d z, absorb(i) resid
predict double v_lin, resid
```

```

quietly ppmlhdfe y d v_lin, absorb(i)
scalar b_lin = _b[d]

di "True coefficient on d      = 0.300000"
di "Naive FE Poisson          = " %8.6f b_naive
di "CF, CRE (Mundlak) Logit   = " %8.6f b_cre
di "CF, Linear First Stage    = " %8.6f b_lin
restore

```

Number of observations (\_N) was 0, now 2,000.

(5,969 observations created)

|                         |            |
|-------------------------|------------|
| True coefficient on d   | = 0.300000 |
| Naive FE Poisson        | = 0.747206 |
| CF, CRE (Mundlak) Logit | = 0.293829 |
| CF, Linear First Stage  | = 0.277644 |

Against the true  $\beta = 0.30$ : - Naive FE Poisson estimates 0.747 (biased by over  $2.4\times$ ). - The CRE logit control function recovers 0.294. - The linear first stage control function recovers 0.278.

In applied practice, reporting both the CRE logit and linear first-stage control functions provides a robust sensitivity check.

## 26.6. Summary

1. **Mechanism:** Control function augments the second-stage exponential regression with the first-stage reduced-form residual, effectively absorbing endogeneity.
2. **Commands:** Use `reghdfe + ppmlhdfe` in Stata, or `fixest::feols() + fixest::fepois()` in R.
3. **Inference:** Standard errors must be cluster-bootstrapped over both stages; analytic second-stage standard errors ignore first-stage estimation uncertainty.
4. **Binary Regressors:** Use a CRE logit first stage with group means  $\bar{z}_i$  and add the generalized residual  $\hat{r} = d - \hat{p}$  to avoid the incidental parameters problem of unit dummies.

## **Part VI.**

# **Causal Inference Methods**





## 27. Using machine learning for causal effect in observational study

### 27.1. A simulation for an OLS model

In an observational study, we need to assume we have the functional form to get causal effect estimated correctly, in addition to the assumption of treatment being exogenous.

The simulation makes that concrete. We draw  $n = 2000$  observations of three correlated standard normals  $(x, z, w)$ , with covariances  $0.8^2$  between  $x$  and  $w$ ,  $0.5^2$  between  $x$  and  $z$ , and  $0.6^2$  between  $z$  and  $w$ . The DGP then runs on the *logs of their squares*, not on the variables themselves: treatment is  $A \sim \text{Bernoulli}(1/(1 + e^g))$  with  $g = \log(w^2 + 0.01) + N(0, 1)$ , the untreated outcome is  $y_0 = \log(x^2 + 0.01) + N(0, 1)$ , and the individual treatment effect is  $\tau_i = 2 + N(0, 1)$ , so the true ATE is exactly 2.

The point is what an analyst sees. Treatment depends on  $\log w^2$  and the outcome on  $\log x^2$ , but only  $x$ ,  $z$  and  $w$  are observed. This is the common situation: we have the right variables and the wrong functional form.

```
library(MASS)
library(ggplot2)
library(dplyr)
library(tmlr)
library(glmnet)
set.seed(366)

nobs <- 2000
xw <- .8
xz <- .5
zw <- .6
nrow <- 3
ncol <- 3
covarMat = matrix( c(1^2, xz^2, xw^2, xz^2, 1^2, zw^2, xw^2, zw^2, 1^2) , nrow=ncol , ncol=n

mu <- rep(0,3)
rawvars <- mvrnorm(n=nobs, mu=mu, Sigma=covarMat)
df <- as_tibble(rawvars, .name_repair = "minimal")
names(df) <- c('x','z','w')
df <- df %>%
  # A small constant inside each log() keeps log(v^2) from diverging to
  # large negative values when v is close to 0 (e.g. log(w^2) alone ranges
```

## 27. Using machine learning for causal effect in observational study

```
# down to about -15 here), which otherwise pushes the propensity score
# P(A=1|W) toward 0/1 for those units and violates positivity.
mutate(log.x=log(x^2 + 0.01), log.z=log(z^2 + 0.01), log.w=log(w^2 + 0.01), z.sqr=z^2, w.sqr=w^2) %>%
mutate(g.var= log.w + rnorm(nobs)) %>%
mutate(A = rbinom(nobs, 1, 1/(1+exp((g.var)))) %>%
mutate(y0=rnorm(nobs) + log.x) %>%
mutate(tau.true = 2 + rnorm(nobs), y1=y0+tau.true, treat=A, y = treat*y1 + (1-treat)*y0)
lm1 <- lm(y ~ A + log.w + log.x , data=df)
summary(lm1)
```

Call:

```
lm(formula = y ~ A + log.w + log.x, data = df)
```

Residuals:

| Min     | 1Q      | Median  | 3Q     | Max    |
|---------|---------|---------|--------|--------|
| -4.5207 | -0.8338 | -0.0089 | 0.8386 | 4.1534 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | 0.01311  | 0.04774    | 0.275   | 0.784      |
| A           | 1.95487  | 0.06703    | 29.166  | <2e-16 *** |
| log.w       | 0.02835  | 0.01945    | 1.458   | 0.145      |
| log.x       | 1.00090  | 0.01748    | 57.264  | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.253 on 1996 degrees of freedom

Multiple R-squared: 0.6746, Adjusted R-squared: 0.6741

F-statistic: 1380 on 3 and 1996 DF, p-value: < 2.2e-16

```
lm2 <- lm(y ~ A , data=df)
summary(lm2)
```

Call:

```
lm(formula = y ~ A, data = df)
```

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -6.1667 | -1.4416 | 0.1372 | 1.4760 | 6.1625 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | -0.75733 | 0.07539    | -10.04  | <2e-16 *** |
| A           | 1.47007  | 0.09571    | 15.36   | <2e-16 *** |

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.077 on 1998 degrees of freedom
```

```
Multiple R-squared:  0.1056,    Adjusted R-squared:  0.1052
```

```
F-statistic: 235.9 on 1 and 1998 DF,  p-value: < 2.2e-16
```

```
lm3 <- lm(y ~ A + w, data=df)
summary(lm3)
```

```
Call:
```

```
lm(formula = y ~ A + w, data = df)
```

```
Residuals:
```

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -6.1657 | -1.4531 | 0.1338 | 1.4780 | 6.0957 |

```
Coefficients:
```

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | -0.75631 | 0.07541    | -10.030 | <2e-16 *** |
| A           | 1.46876  | 0.09573    | 15.343  | <2e-16 *** |
| w           | -0.03874 | 0.04464    | -0.868  | 0.386      |

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.077 on 1997 degrees of freedom
```

```
Multiple R-squared:  0.1059,    Adjusted R-squared:  0.105
```

```
F-statistic: 118.3 on 2 and 1997 DF,  p-value: < 2.2e-16
```

```
lm4 <- lm(y ~ A + w + x, data=df)
summary(lm4)
```

```
Call:
```

```
lm(formula = y ~ A + w + x, data = df)
```

```
Residuals:
```

|  | Min    | 1Q     | Median | 3Q    | Max   |
|--|--------|--------|--------|-------|-------|
|  | -6.167 | -1.445 | 0.147  | 1.473 | 6.205 |

```
Coefficients:
```

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | -0.75360 | 0.07539    | -9.996  | <2e-16 *** |
| A           | 1.46387  | 0.09573    | 15.292  | <2e-16 *** |
| w           | -0.09981 | 0.05764    | -1.732  | 0.0835 .   |

## 27. Using machine learning for causal effect in observational study

```
x          0.10281    0.06141    1.674    0.0943 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.076 on 1996 degrees of freedom
Multiple R-squared:  0.1072,    Adjusted R-squared:  0.1059
F-statistic: 79.88 on 3 and 1996 DF,  p-value: < 2.2e-16
```

In this example, treatment assignment process is determined by logged  $w$ , and outcome is determined by logged  $x$  and treatment. However, what we observe is  $w$  and  $x$ . In observational studies, this happens all the time. In fact, this is an ideal situation, that we observe variables that are determinants of outcome, although we are not sure about the functional form that determines the outcome. However, this example shows that unless we have observed exactly the factors themselves (in this case logged  $x$ ,  $w$ , which determines the DGP), we have biased estimates of the true treatment effect.

The four estimates are 1.955, 1.470, 1.469 and 1.464 against a true 2.

Model 1 is the only model with reasonable estimate of treatment effect (which is 2 in this case), at 1.955 with a standard error of 0.067 — and it is also the only one given the correct functional form,  $\log.x$  and  $\log.w$ . Its  $\log.x$  coefficient of 1.001 recovers the DGP's coefficient of 1 almost exactly. Model 2 is a model with endogeneity:  $A$  is correlated with the missing variable logged  $x$ . Model 3 and 4 we have  $x$  and  $w$ , but not logged, therefore still biased.

The lesson here is the functional form does matter. However, we have no way of knowing the functional form. What can we do here?

```
# Algorithm set trimmed to 7 fast learners for render speed.
# A production analysis would also include SL.randomForest, SL.gbm, SL.gam.
Q.SL.library <- c("SL.glmnet","SL.glm","SL.glm.interaction", "SL.rpart","SL.bayesglm","SL.step
g.SL.library <- c("SL.glmnet","SL.glm","SL.glm.interaction", "SL.rpart","SL.bayesglm","SL.step

# tmle1 uses x and w; tmle2 additionally includes z. Note z is not in the true
# treatment or outcome DGP -- it is only correlated with x and w -- so adding it
# should not change the estimate much.
tmle1 <- tmle(Y = df$y, A = df$treat, W = df[,c('x','w')], g.SL.library = g.SL.library , Q.SL.
tmle1
```

Marginal mean under treatment (EY1)

```
Parameter Estimate:  0.92554
Estimated Variance:  0.0033412
p-value: <2e-16
95% Conf Interval:  (0.81225, 1.0388)
```

Marginal mean under comparator (EY0)

```
Parameter Estimate: -1.0196
Estimated Variance:  0.0035588
p-value: <2e-16
```

95% Conf Interval: (-1.1365, -0.90269)

#### Additive Effect

Parameter Estimate: 1.9451  
 Estimated Variance: 0.004266  
 p-value: <2e-16  
 95% Conf Interval: (1.8171, 2.0732)

#### Additive Effect among the Treated

Parameter Estimate: 1.8969  
 Estimated Variance: 0.0052492  
 p-value: <2e-16  
 95% Conf Interval: (1.7549, 2.0389)

#### Additive Effect among the Controls

Parameter Estimate: 2.0351  
 Estimated Variance: 0.0051986  
 p-value: <2e-16  
 95% Conf Interval: (1.8938, 2.1765)

```
tmle2 <- tmle(Y = df$y, A = df$treat, W = df[,c('x', 'w', 'z')], g.SL.library = g.SL.library , 0)
tmle2
```

#### Marginal mean under treatment (EY1)

Parameter Estimate: 0.9072  
 Estimated Variance: 0.0033185  
 p-value: <2e-16  
 95% Conf Interval: (0.79429, 1.0201)

#### Marginal mean under comparator (EY0)

Parameter Estimate: -1.0068  
 Estimated Variance: 0.003566  
 p-value: <2e-16  
 95% Conf Interval: (-1.1238, -0.88975)

#### Additive Effect

Parameter Estimate: 1.914  
 Estimated Variance: 0.004266  
 p-value: <2e-16  
 95% Conf Interval: (1.786, 2.042)

#### Additive Effect among the Treated

Parameter Estimate: 1.8705  
 Estimated Variance: 0.0052652  
 p-value: <2e-16  
 95% Conf Interval: (1.7283, 2.0127)

## 27. Using machine learning for causal effect in observational study

### Additive Effect among the Controls

```
Parameter Estimate: 1.9817
Estimated Variance: 0.0051984
p-value: <2e-16
95% Conf Interval: (1.8404, 2.123)
```

We use [Mark van der Laan's TMLE method](#). It uses [SuperLearner](#) as the initial estimator. It's an ensemble of multiple machine learning algorithms. Therefore it does not need to assume the functional form of the DGP. Even if we don't have the variables that determines the DGP of outcome, if we observe some functions (even nonlinear functions) of these variables, we can still get reasonable estimates of the treatment effect.

In this example, we used multiple popular machine learning algorithms in modeling both treatment assignment process and the outcome process. The first TMLE model is with  $x$  and  $w$  (note not the logged  $x$  and  $w$  which are in the true DGP), the second one with an additional variable  $z$ .

TMLE gives an additive effect of 1.9637 with  $x$  and  $w$ , and 1.9540 when  $z$  is added — against a true 2, and against 1.464 to 1.470 for the linear models given the same untransformed variables. The 95% intervals, [1.823, 2.105] and [1.813, 2.095], both cover 2. Adding  $z$  changes almost nothing, which is what should happen:  $z$  is in neither the treatment nor the outcome equation and is merely correlated with the variables that are.

It seems that TMLE results are less biased than the linear models with  $x$  and  $w$ . That is putting it mildly: the linear models miss by about 0.53 and TMLE by about 0.04, an order of magnitude better, purely from not having to guess the functional form. It may not be better than the linear model with logged  $x$  and  $w$ , but in empirical studies, we often cannot assume we have the variables in the DGP, but only some proxy of the variables in the DGP. I'll do more simulations to see whether TMLE does perform better in the situation that we are not sure about the functional form. We should expect that is the case.

The simple `tmle` package example used here is for a binary treatment. TMLE itself is not limited to binary treatments: there are TMLE estimators for continuous, multivalued, longitudinal, stochastic, and survival settings (e.g. the `ltmle` and `lmtp` packages), though which estimand and package you use differs by setting.

It's about time we embrace machine learning techniques into studies of causal effect in observational studies.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 28. Mediation analysis in R and Stata

### 28.1. Mediation analysis

The traditional mediation model has two equations:

$$Y = aX + bM + \epsilon_1 \quad (28.1)$$

$$M = cX + \epsilon_2 \quad (28.2)$$

The effect of  $X$  on  $Y$  decomposes into a direct effect and an indirect effect through  $M$ . The classical approach is [Baron and Kenny's](#) four-step procedure. Modern implementations use SEM to estimate both equations jointly, as in R's `lavaan` and Stata's `sem`.

```
library(lavaan)
set.seed(1234)
n <- 10000
X <- rnorm(n)
M <- 0.5*X + rnorm(n)
Y <- 0.7*M + 0.3*X + rnorm(n)
Data <- data.frame(X = X, Y = Y, M = M)
model <- '
  Y ~ a*X + b*M
  M ~ c*X
  # direct effect (a)
  # indirect effect (b*c)
  bc := b*c
  # total effect
  total := a + (b*c)
'
fit <- sem(model, data = Data)
summary(fit)
```

lavaan 0.7-2 ended normally after 1 iteration

|                            |        |
|----------------------------|--------|
| Estimator                  | ML     |
| Optimization method        | NLMINB |
| Number of model parameters | 5      |
| Number of observations     | 10000  |

Model Test User Model:

## 28. Mediation analysis in R and Stata

|                    |       |
|--------------------|-------|
| Test statistic     | 0.000 |
| Degrees of freedom | 0     |

Parameter Estimates:

|                                  |            |
|----------------------------------|------------|
| Standard errors                  | Standard   |
| Information                      | Expected   |
| Information saturated (h1) model | Structured |

Regressions:

|     |     | Estimate | Std.Err | z-value | P(> z ) |
|-----|-----|----------|---------|---------|---------|
| Y ~ |     |          |         |         |         |
| X   | (a) | 0.286    | 0.011   | 25.233  | 0.000   |
| M   | (b) | 0.699    | 0.010   | 70.384  | 0.000   |
| M ~ |     |          |         |         |         |
| X   | (c) | 0.511    | 0.010   | 50.161  | 0.000   |

Variances:

|    | Estimate | Std.Err | z-value | P(> z ) |
|----|----------|---------|---------|---------|
| .Y | 0.998    | 0.014   | 70.711  | 0.000   |
| .M | 1.012    | 0.014   | 70.711  | 0.000   |

Defined Parameters:

|       | Estimate | Std.Err | z-value | P(> z ) |
|-------|----------|---------|---------|---------|
| bc    | 0.357    | 0.009   | 40.849  | 0.000   |
| total | 0.643    | 0.012   | 51.956  | 0.000   |

We simulate  $n = 10,000$  observations with  $X \sim N(0, 1)$ ,  $M = 0.5X + \varepsilon_2$  and  $Y = 0.7M + 0.3X + \varepsilon_1$ , both errors standard normal. So the true direct effect is  $a = 0.3$ , the mediator coefficient is  $b = 0.7$ , the treatment-to-mediator path is  $c = 0.5$ , the indirect effect is  $bc = 0.35$  and the total is 0.65. There is no confounding of any kind.

lavaan recovers all of it:  $a = 0.286$ ,  $b = 0.699$ ,  $c = 0.511$ , giving  $bc = 0.357$  and a total of 0.643. The model is saturated (0 degrees of freedom), so the perfect fit statistic carries no information.

The same analysis in Stata:

```
set obs 10000
gen x= rnormal()
gen m= .5*x + rnormal()
gen y= .7*m + .3*x +rnormal()
sem (y <- x m) (m <- x)
estat teffects
```

Number of observations (\_N) was 0, now 10,000.



Endogenous variables

Observed: y m

Exogenous variables

Observed: x

Fitting target model:

Iteration 0: Log likelihood = -42567.918

Iteration 1: Log likelihood = -42567.918

Structural equation model

Number of obs = 10,000

Estimation method: ml

Log likelihood = -42567.918

|                 |       | OIM         |           | z     | P> z  | [95% conf. interval] |          |
|-----------------|-------|-------------|-----------|-------|-------|----------------------|----------|
|                 |       | Coefficient | std. err. |       |       |                      |          |
| Structural<br>y |       |             |           |       |       |                      |          |
|                 | m     | .7136545    | .0101594  | 70.25 | 0.000 | .6937424             | .7335666 |
|                 | x     | .2971221    | .0113774  | 26.12 | 0.000 | .2748228             | .3194215 |
|                 | _cons | -.0000391   | .0101082  | -0.00 | 0.997 | -.0198508            | .0197725 |
| m               |       |             |           |       |       |                      |          |
|                 | x     | .5038605    | .0100014  | 50.38 | 0.000 | .4842581             | .5234628 |
|                 | _cons | .0187204    | .0099478  | 1.88  | 0.060 | -.0007769            | .0382177 |
| var(e.y)        |       | 1.021391    | .0144446  |       |       | .9934684             | 1.050098 |
| var(e.m)        |       | .9895863    | .0139949  |       |       | .9625336             | 1.017399 |

LR test of model vs. saturated: chi2(0) = 0.00

Prob > chi2 = .

Direct effects

|                 |   | OIM         |           | z     | P> z  | [95% conf. interval] |          |
|-----------------|---|-------------|-----------|-------|-------|----------------------|----------|
|                 |   | Coefficient | std. err. |       |       |                      |          |
| Structural<br>y |   |             |           |       |       |                      |          |
|                 | m | .7136545    | .0101594  | 70.25 | 0.000 | .6937424             | .7335666 |
|                 | x | .2971221    | .0113774  | 26.12 | 0.000 | .2748228             | .3194215 |

|                  |  |             |           |       |       |                      |          |
|------------------|--|-------------|-----------|-------|-------|----------------------|----------|
| -----+-----      |  |             |           |       |       |                      |          |
| m                |  |             |           |       |       |                      |          |
| x                |  | .5038605    | .0100014  | 50.38 | 0.000 | .4842581             | .5234628 |
| -----            |  |             |           |       |       |                      |          |
| Indirect effects |  |             |           |       |       |                      |          |
| -----            |  |             |           |       |       |                      |          |
|                  |  | OIM         |           |       |       |                      |          |
|                  |  | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |          |
| -----            |  |             |           |       |       |                      |          |
| Structural       |  |             |           |       |       |                      |          |
| y                |  |             |           |       |       |                      |          |
| m                |  | 0           | (no path) |       |       |                      |          |
| x                |  | .3595823    | .0087834  | 40.94 | 0.000 | .3423672             | .3767974 |
| -----            |  |             |           |       |       |                      |          |
| m                |  |             |           |       |       |                      |          |
| x                |  | 0           | (no path) |       |       |                      |          |
| -----            |  |             |           |       |       |                      |          |
| Total effects    |  |             |           |       |       |                      |          |
| -----            |  |             |           |       |       |                      |          |
|                  |  | OIM         |           |       |       |                      |          |
|                  |  | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |          |
| -----            |  |             |           |       |       |                      |          |
| Structural       |  |             |           |       |       |                      |          |
| y                |  |             |           |       |       |                      |          |
| m                |  | .7136545    | .0101594  | 70.25 | 0.000 | .6937424             | .7335666 |
| x                |  | .6567044    | .0124172  | 52.89 | 0.000 | .6323672             | .6810417 |
| -----            |  |             |           |       |       |                      |          |
| m                |  |             |           |       |       |                      |          |
| x                |  | .5038605    | .0100014  | 50.38 | 0.000 | .4842581             | .5234628 |
| -----            |  |             |           |       |       |                      |          |

The above examples should have direct effect of .3 and indirect effect of .35, and total effect of .65.

## 28.2. Causal Mediation analysis

The traditional mediation analysis has been criticized for the lack of causal interpretation. Without manipulation of the mediator, it is hard to interpret the effects causally, because even if the treatment is from random experiments, the mediator is often not. Therefore there could be an unmeasured confounder that is causing both  $M$  and  $Y$ .

R's "mediation" package is for causal mediation analysis. It uses simulation to estimate the causal effects of treatment, under assumptions of sequential ignorability.

**i** Identifying assumptions for natural (in)direct effects

Before reading the `mediate()` output causally, the following must hold (this is “sequential ignorability” plus the standard support and link conditions):

1. **Treatment ignorability** — no unmeasured treatment–outcome confounding given covariates  $W$ .
2. **Mediator ignorability** — no unmeasured mediator–outcome confounding given  $W$  and treatment.
3. **No treatment-induced confounding** — no variable affected by the treatment confounds the mediator–outcome relationship (this is the strong, often-violated condition).
4. **Positivity** — both treatment and mediator have positive probability across  $W$ .
5. **Consistency** — observed outcomes equal the corresponding potential outcomes.
6. **Cross-world independence** — the condition underlying natural effects, which no single experiment can test.

Mediator ignorability and the no-treatment-induced-confounding condition are the ones that usually break, because the mediator is rarely randomized.

It estimates the following quantities:

$$\tau_i = Y_i(1, M_i(1)) - Y_i(0, M_i(0)) \quad (28.3)$$

This is the total treatment effect, which is to say, what’s the change in  $Y$  if we change each unit from control to treated, hypothetically?

Then this can be decomposed into the causal mediation effects:

$$\delta_i(t) = Y_i(t, M_i(1)) - Y_i(t, M_i(0)) \quad (28.4)$$

for treatment status  $t = 0, 1$ . This is to say, given treatment status, what’s the mediation effect?

$$\eta_i(t) = Y_i(1, M_i(t)) - Y_i(0, M_i(t)) \quad (28.5)$$

for treatment status  $t = 0, 1$ . This is to say, given mediator status for each treatment status, what’s the direct effect?

Therefore there are four quantities estimated, direct and mediation effect for treated and control.

R’s “mediation” needs users to feed two models, outcome model and mediation model.

If we study the same data, we would expect it returns the same estimates as the traditional methods. However, the causal mediation models can be much more flexible in outcome and mediation models.

```
library(mediation)
med.fit <- lm(M ~ X, data=Data)
summary(med.fit)
```

## 28. Mediation analysis in R and Stata

Call:

```
lm(formula = M ~ X, data = Data)
```

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -4.1134 | -0.6972 | 0.0086 | 0.6837 | 3.7309 |

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )   |
|-------------|-----------|------------|---------|------------|
| (Intercept) | -0.007814 | 0.010060   | -0.777  | 0.437      |
| X           | 0.510954  | 0.010187   | 50.156  | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.006 on 9998 degrees of freedom

Multiple R-squared: 0.201, Adjusted R-squared: 0.2009

F-statistic: 2516 on 1 and 9998 DF, p-value: < 2.2e-16

```
out.fit <- lm(Y ~ X + M, data=Data)
summary(out.fit)
```

Call:

```
lm(formula = Y ~ X + M, data = Data)
```

Residuals:

| Min     | 1Q      | Median  | 3Q     | Max    |
|---------|---------|---------|--------|--------|
| -4.0680 | -0.6771 | -0.0081 | 0.6643 | 3.8304 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | 0.004170 | 0.009992   | 0.417   | 0.676      |
| X           | 0.285582 | 0.011319   | 25.229  | <2e-16 *** |
| M           | 0.699006 | 0.009933   | 70.373  | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9991 on 9997 degrees of freedom

Multiple R-squared: 0.4734, Adjusted R-squared: 0.4733

F-statistic: 4494 on 2 and 9997 DF, p-value: < 2.2e-16

```
set.seed(123)
med.out <- mediate(med.fit, out.fit, treat = "X", mediator = "M", sims = 100)
summary(med.out)
```

## Causal Mediation Analysis

## Quasi-Bayesian Confidence Intervals

|                | Estimate | 95% CI Lower | 95% CI Upper | p-value       |
|----------------|----------|--------------|--------------|---------------|
| ACME           | 0.35684  | 0.34208      | 0.37741      | < 2.2e-16 *** |
| ADE            | 0.28540  | 0.26522      | 0.30604      | < 2.2e-16 *** |
| Total Effect   | 0.64224  | 0.62091      | 0.66752      | < 2.2e-16 *** |
| Prop. Mediated | 0.55530  | 0.53571      | 0.57897      | < 2.2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 10000

Simulations: 100

And it does. The two component regressions give  $c = 0.510954$  and, in the outcome equation,  $a = 0.285582$  and  $b = 0.699006$  — the same numbers `lavaan` produced. `mediate()` then reports ACME 0.35684 (against a true 0.35), ADE 0.28540 (against 0.30) and a total effect of 0.64224 (against 0.65), with the proportion mediated at 0.555. The simulation-based intervals all exclude zero.

The point is that on this data the causal machinery adds nothing — which is the right outcome for a DGP with no confounding, a linear outcome model and no treatment-mediator interaction. Its value appears when those conditions fail.

## 28.3. Binary outcome

For example, in the case of binary outcome, the traditional approach will have difficulties. We can estimate the outcome model and mediator model jointly, but the total effects are not easy to decompose into direct and indirect effect (see Imai et al, page 320 <https://imai.fas.harvard.edu/research/files/BaronKenny.pdf>).

The causal mediation analysis framework is much more general.

```
library(mediation)
data(framing)
med.fit <- lm(emo ~ treat + age + educ + gender + income, data = framing)
out.fit <- glm(cong_mesg ~ emo + treat + age + educ + gender + income,
               data = framing, family = binomial("probit"))
set.seed(123)
med.out <- mediate(med.fit, out.fit, treat = "treat", mediator = "emo",
                  robustSE = TRUE, sims = 100)
summary(med.out)
```

## Causal Mediation Analysis

## Quasi-Bayesian Confidence Intervals

|                          | Estimate | 95% CI Lower | 95% CI Upper | p-value    |
|--------------------------|----------|--------------|--------------|------------|
| ACME (control)           | 0.081341 | 0.037680     | 0.122101     | <2e-16 *** |
| ACME (treated)           | 0.082417 | 0.036606     | 0.126460     | <2e-16 *** |
| ADE (control)            | 0.018299 | -0.105898    | 0.158469     | 0.72       |
| ADE (treated)            | 0.019375 | -0.115315    | 0.172002     | 0.72       |
| Total Effect             | 0.100716 | -0.028036    | 0.259054     | 0.24       |
| Prop. Mediated (control) | 0.632609 | -13.705651   | 4.305615     | 0.24       |
| Prop. Mediated (treated) | 0.661761 | -12.422330   | 4.036601     | 0.24       |
| ACME (average)           | 0.081879 | 0.037143     | 0.124080     | <2e-16 *** |
| ADE (average)            | 0.018837 | -0.110606    | 0.165198     | 0.72       |
| Prop. Mediated (average) | 0.647185 | -13.063991   | 4.171108     | 0.24       |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 265

Simulations: 100

Here the decomposition does something the linear example could not. The `framing` data has a binary outcome (`cong_mesg`, whether the subject sent a message to Congress) modelled by probit, with anxiety (`emo`) as the mediator. ACME comes out at 0.0813 under control and 0.0824 under treatment, both significant, while the direct effects are 0.0183 and 0.0194 with  $p = 0.72$  and the total effect is 0.1007 with  $p = 0.24$ .

Read that pattern carefully: the *total* effect is not distinguishable from zero, and neither is the direct effect, yet the mediated effect clearly is. A study reporting only the total effect would have concluded the framing treatment did nothing. The proportion mediated is reported as 0.63, but note its interval runs from  $-13.7$  to  $4.3$  — a ratio whose denominator is indistinguishable from zero is not a usable quantity, and this is the standard reason not to lean on “proportion mediated”.

“mediation” package has more functionalities, such as multilevel, interaction of treatment and mediator, etc.

Stata’s `sem` and `gsem` commands can model different situations, but the direct effect and indirect effects are not easy to compute, especially when you have binary outcome, or other non-continuous outcome situations. They are not designed for causal mediation analysis.

## 28.4. Going further: formal causal mediation

The `mediation` package above implements sequential ignorability — a specific set of identifying assumptions. The [causal mediation chapter](#) covers the assumptions more formally, introduces the

controlled direct effect (CDE) and natural direct/indirect effects (NDE/NIE) using potential-outcome notation, and discusses identification with unmeasured mediator–outcome confounding. If your setting involves a non-binary outcome, treatment–mediator interaction, or you need a doubly-robust estimator, start there.

---

*Systematic treatment:* *R* · *Julia*.





## 29. Endogeneity with censored variable

### 29.1. Bunching for the outcome variable

Bunching is a mass point in the distribution of some variable — for example, the excess density of earned income at a kink in the tax schedule (Saez 2010). The idea is to compare the observed distribution with a counterfactual “manipulation-free” distribution and use the excess mass to estimate behavioral elasticities.

Bunching is closely related to regression discontinuity. RD assumes no manipulation at the threshold; bunching estimators relax that assumption and estimate the fraction of manipulators directly (Kleven 2016).

Caetano et al. (2020) extend bunching to the treatment variable, which is the focus of this chapter.

### 29.2. Bunching for the treatment variable

#### 29.2.1. A test of endogeneity

Caetano (2015) showed that, if the distribution of  $T_i$  has bunching at  $T_i = \bar{t}$ , it is possible to test the exogeneity of  $T_i$ . When one compares the outcome of observations at the bunching point and those around it, the treatment itself is very similar. Therefore, there cannot be more than a marginal difference in the outcome that is due to treatment variation, since the treatment hardly varies.

Caetano (2015)’s estimation can be done in a two step process:

- (1) estimate  $E[Y_i|T_i = \bar{T}, X_i]$  non-parametrically
- (2) do a local linear regression  $E[Y_i|T_i = \bar{T}, X_i] - Y_i$  onto  $T_i$  at  $\bar{T}$ , using only observations such that  $T_i > \bar{T}$ .

The approach is known as the Discontinuity Test.

#### 29.2.2. Dummy test

Caetano and Maheshri (2018) proposed a simpler test, which is simply regressing  $Y_i$  on  $T_i$ ,  $X_i$  and a dummy variable for  $T_i > \bar{T}$ . The coefficient of the dummy variable is the test statistic. If the coefficient is significant, which is to say there is a discontinuity on outcome, then  $T_i$  is endogenous.

This is simpler, but with linear functional form assumption.

The logic of the endogeneity test is that around the bunching point, the treatment is very similar, so the outcome should be similar. If there is a discontinuity in the outcome, then it has to be one of

## 29. Endogeneity with censored variable

two reasons. One is that treatment has discontinuous effect on outcome around the bunching point. Usually we can reasonably rule out that possibility. Second is that there is an unobserved variable that is causing both the treatment and the outcome. That unobserved variable has a discontinuous effect on the outcome. If that's the case, then the treatment is endogenous.

### 29.2.3. Treatment effect estimation

Caetano et al (2020) proposed a method to estimate the treatment effect when there is bunching. It seems to me there is no reason that we cannot use it for the censoring situation. There are a lot of situations that we have censored treatment variable, when we study continuous treatment variable.

Suppose we have

$$Y = \beta T + Z\gamma + \delta\eta + \epsilon \quad (29.1)$$

where  $T$  is the treatment variable,  $\eta$  is the unobserved variable that is causing both the treatment and the outcome,  $Z$  is the control variables, and  $\epsilon$  is the error term. We only observe  $Y, T, Z$ .

Suppose  $T^*$  is the latent variable that is censored at  $T^* = 0$ . We observe  $T = \max(T^*, 0)$ . We assume that  $T^*$  is continuous over its support.

$$T^* = Z\pi + \eta \quad (29.2)$$

From the two equations, we can derive:

$$E[Y|T, Z] = (\beta + \delta)T + Z(\gamma - \pi\delta) + \delta E[T^*|T^* \leq 0, Z]1(T = 0) \quad (29.3)$$

or

$$E[Y|T, Z] = \beta T + Z(\gamma - \pi\delta) + \delta(T + E[T^*|T^* \leq 0, Z]1(T = 0)) \quad (29.4)$$

Therefore we can “correct” for endogeneity by simply including an additional term  $\delta(T + E[T^*|T^* \leq 0, Z]1(T = 0))$  in the regression.

However, the prediction of  $\delta(T + E[T^*|T^* \leq 0, Z]1(T = 0))$  is out of sample. We have to make some additional assumptions for the distribution of  $T^*$ . For example, we can assume that  $T^*$  is normally distributed. Or, we can assume it is symmetric. Then we can use the right tail to estimate the left tail.

## 29.3. Simulation

Let's do a simulation to see how it works.

We draw  $n = 1000$  observations. Two controls  $(Z_1, Z_2)$  are joint normal with unit variances and correlation 0.5, and the unobserved confounder is  $\eta \sim N(1, 1)$ . The latent treatment is  $T^* = Z_1 + 2Z_2 + 1.5\eta$ , censored at zero so that  $T = \max(T^*, 0)$ , and the outcome is  $Y = 2T + 0.5Z_1 + 0.5Z_2 + 2\eta + N(0, 1)$ .

So the true treatment effect is  $\beta = 2$ , and  $\eta$  enters both the treatment and the outcome with a positive sign — the endogeneity. Note  $Y$  depends on the *observed* censored  $T$ , not on  $T^*$ , which is what makes the mass point at  $T = 0$  informative: among those units the treatment is held fixed while  $\eta$  still varies.

```
library(MASS)
library(ggplot2)
library(dplyr)
# set seed
set.seed(123)
# number of observations
n=1000
# generate Z and make it dataframe
Z=as.data.frame(mvrnorm(n=n, mu=c(0,0), Sigma=matrix(c(1,0.5,0.5,1),2,2)))
data <- Z
names(data) <- c("Z1", "Z2")
# generate eta
# eta is normally distributed
data$eta=rnorm(n, 1,1)
# generate T*
# T* is normally distributed
# Y depends on the OBSERVED (censored) treatment T, not the latent T_star --
# matching the structural equation Y = beta*T + Z*gamma + delta*eta + epsilon
# above. If Y depended on T_star directly, T=0 would not fully "block" the
# latent variable's effect on Y, and the treatment effect of T would not even
# be well defined.
data <- data %>%
  mutate(T_star=Z1 + 2*Z2 + 1.5*eta) %>%
  mutate(T=ifelse(T_star>0, T_star, 0)) %>%
  mutate(Y=2*T + 0.5*Z1 + 0.5*Z2 + 2*eta + rnorm(n, 0, 1))

# regress Y on T, Z1, Z2
lm1 <- lm(Y ~ T + Z1 + Z2, data=data)
summary(lm1)
```

Call:

```
lm(formula = Y ~ T + Z1 + Z2, data = data)
```

## 29. Endogeneity with censored variable

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -8.2706 | -0.8700 | 0.1009 | 1.0955 | 4.0476 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | -0.42394 | 0.09588    | -4.422  | 1.09e-05 *** |
| T           | 3.14972  | 0.03906    | 80.631  | < 2e-16 ***  |
| Z1          | -0.29574 | 0.06667    | -4.436  | 1.02e-05 *** |
| Z2          | -1.14069 | 0.08060    | -14.153 | < 2e-16 ***  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.639 on 996 degrees of freedom

Multiple R-squared: 0.9343, Adjusted R-squared: 0.9341

F-statistic: 4719 on 3 and 996 DF, p-value: < 2.2e-16

Without including *eta*, the effect estimate is biased: OLS gives 3.150 against a true 2, an upward bias of 57%, and the standard error of 0.039 makes it look extremely precise. Let's do a dummy test.

```
lm2 <- lm(Y ~ T + Z1 + Z2 + I(T>0), data=data)
summary(lm2)
```

Call:

```
lm(formula = Y ~ T + Z1 + Z2 + I(T > 0), data = data)
```

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -8.0768 | -0.8363 | 0.0406 | 0.9126 | 4.8213 |

Coefficients:

|              | Estimate | Std. Error | t value | Pr(> t )   |
|--------------|----------|------------|---------|------------|
| (Intercept)  | -1.68077 | 0.11961    | -14.052 | <2e-16 *** |
| T            | 3.06030  | 0.03570    | 85.728  | <2e-16 *** |
| Z1           | -0.52092 | 0.06189    | -8.416  | <2e-16 *** |
| Z2           | -1.48038 | 0.07601    | -19.476 | <2e-16 *** |
| I(T > 0)TRUE | 2.10995  | 0.13884    | 15.197  | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.477 on 995 degrees of freedom

Multiple R-squared: 0.9466, Adjusted R-squared: 0.9464

F-statistic: 4414 on 4 and 995 DF, p-value: < 2.2e-16

It shows that the dummy variable is significant: the coefficient on  $1(T > 0)$  is 2.110 with a standard error of 0.139,  $t = 15.2$ . It means that  $T$  is endogenous. The logic is the one set out above — at the bunching point the treatment barely varies, so a jump of this size in the outcome has to come from something else moving discontinuously, and  $\eta$  is the candidate.

### 29.3.1. Correcting for endogeneity

The correction term is  $E[T^* \mid T^* \leq 0, Z]$ , on the scale of the *treatment*  $T^*$  — so we need to model  $T^*$  given  $Z$ , not  $Y$  given  $T$  and  $Z$ . Let's assume  $T^*$  is normally distributed given  $Z$ :  $T^* \mid Z \sim N(Z\pi, \sigma^2)$ . Since we observe  $T = T^*$  whenever  $T^* > 0$  and  $T = 0$  otherwise, this is exactly a (left-)censored (Tobit) regression of  $T$  on  $Z$ , using **all** observations (not just the right tail): the censored observations still inform  $\pi$  and  $\sigma$  through the likelihood, they just don't reveal  $T^*$ 's value directly. Given  $\hat{\pi}$  and  $\hat{\sigma}$ , the standard truncated-normal mean formula gives us  $E[T^* \mid T^* \leq 0, Z]$  directly, with no separate mirroring step needed:

$$E[T^* \mid T^* \leq 0, Z] = Z\hat{\pi} - \hat{\sigma} \frac{\phi(-Z\hat{\pi}/\hat{\sigma})}{\Phi(-Z\hat{\pi}/\hat{\sigma})} \quad (29.5)$$

```
library(AER)

tob <- tobit(T ~ Z1 + Z2, left = 0, data = data)
lin_pred <- coef(tob)["(Intercept)"] + coef(tob)["Z1"] * data$Z1 + coef(tob)["Z2"] * data$Z2
sigma_hat <- tob$scale
alpha <- -lin_pred / sigma_hat
E_Tstar_below0 <- lin_pred - sigma_hat * dnorm(alpha) / pnorm(alpha)

# here I create an extra term for the correction:
# when T is 0, it is T (=0); otherwise it's the truncated-normal conditional
# mean of T* given T* <= 0 and Z, computed above.
data <- data %>%
  mutate(extra = if_else(T > 0, T, E_Tstar_below0))

lm4 <- lm(Y ~ T + Z1 + Z2 + extra, data=data)
summary(lm4)
```

Call:

```
lm(formula = Y ~ T + Z1 + Z2 + extra, data = data)
```

Residuals:

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -5.8902 | -0.7686 | 0.0480 | 0.7935 | 5.4430 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t ) |
|-------------|----------|------------|---------|----------|
| (Intercept) | 0.03693  | 0.07979    | 0.463   | 0.644    |

## 29. Endogeneity with censored variable

```
T          1.98735    0.05917  33.585    <2e-16 ***
Z1         -0.82513    0.05838 -14.133    <2e-16 ***
Z2         -2.31062    0.08224 -28.097    <2e-16 ***
extra      1.36256    0.05873  23.200    <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.321 on 995 degrees of freedom
Multiple R-squared:  0.9573,    Adjusted R-squared:  0.9572
F-statistic: 5582 on 4 and 995 DF,  p-value: < 2.2e-16
```

Under the normality assumption, this recovers  $\beta$  (the true coefficient on  $T$  is 2) very closely — 1.987 with a standard error of 0.059, against OLS's 3.150. The correction term itself enters with coefficient 1.363, which is the estimate of  $\delta$ , the effect of  $\eta$  on  $Y$ ; the DGP sets that path at  $2/1.5 = 1.33$  once expressed per unit of  $T^*$ . Note the control coefficients also move, from  $-0.296$  and  $-1.141$  to  $-0.825$  and  $-2.311$ , because they were absorbing part of the confounding too — this is much better — much better than a naive symmetric-mirroring heuristic that estimates the right-tail conditional mean by an unweighted linear regression of  $T$  on  $Z$  restricted to a somewhat arbitrary right-tail cutoff: that heuristic's accuracy turns out to be quite sensitive to the cutoff and sample used, because the true conditional mean of  $T^*$  given  $T^*$  beyond a cutoff is a nonlinear (inverse-Mills-ratio) function of  $Z$ , not linear — so a plain linear regression on the selected right-tail subsample is itself misspecified. The parametric (Tobit) route sidesteps that by modeling the untruncated latent distribution directly and using the exact conditional-mean formula.

## 29.4. Conclusion

We are making assumptions for the distribution of the continuous treatment variable. Symmetry is the minimum assumption. We can make other parametric assumptions on the distribution of  $T^*$ . This is a trade-off between making distribution assumptions and ignoring endogeneity. The good news is that we don't need an instrument. We are using the censoring part of the treatment variable to deal with endogeneity.

## 30. G-computation (g-formula) with time varying covariates

### 30.1. The problem

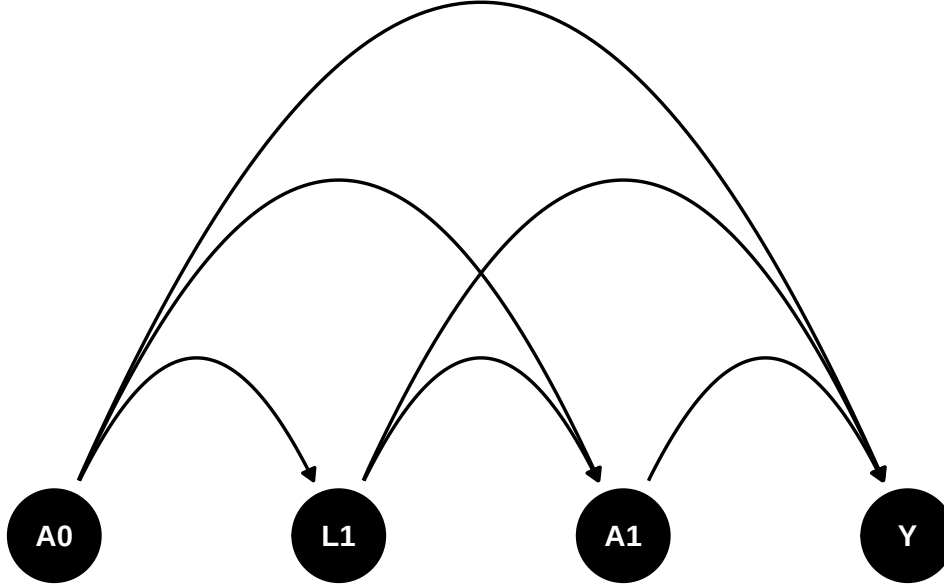
The g-formula (g-computation algorithm) is a method to estimate the causal effect of a time varying treatment or covariates, developed by Robins (1986). Note this is distinct from **g-estimation** of structural nested models (Robins and Rotnitzky, 1992), a different g-methods estimator that uses instrumental-variable-like estimating equations and does not require modeling the joint distribution of all time-varying confounders. This chapter — and the `gfoRmula/gmethods` packages used below — implements the g-formula, not g-estimation.

We can imagine such a scenario. Suppose  $A_0$  and  $A_1$  are time 0 and time 1 treatment, can be binary or continuous.  $L_1$  is a time 1 covariate. It can be some intermediate outcome. Say  $HbA1c$ .  $HbA1c$  can be a result of treatment  $A_0$ . Then it can be a cause for the next period treatment  $A_1$ .  $Y$  is the outcome.  $U$  is the unmeasured confounder. The causal diagram is shown below. How do we estimate the effect of treatment?

```
library(dagitty)
library(ggdag)
library(ggplot2)
library(gfoRmula)
library(gmethods)
library(tidyverse)

dag <- dagify(
  Y ~ A1 + A0 + L1 ,
  A1 ~ A0 + L1,
  L1 ~ A0 ,
  coords = time_ordered_coords()
)

dag %>%
  ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_dag_point() +
  geom_dag_edges_arc() +
  geom_dag_text() +
  theme_dag()
```



## 30.2. Assumptions

There are four potential outcomes, with  $A_0$  and  $A_1$  can be 0 or 1. The causal estimand is

$$\tau_{a_0, a_1, a'_0, a'_1} = E[Y^{a_0, a_1} - Y^{a'_0, a'_1}] \quad (30.1)$$

To identify the effects, we need assumptions.

1. Sequential ignorability

$$Y^{a_0, a_1} \perp A_0 \quad (30.2)$$

$$Y^{a_0, a_1} \perp A_1^{a_0} | A_0, L_1^{a_0} \quad (30.3)$$

In general, treatment at time  $t$  is randomized with probabilities depending on the observed past (covariates and intermediate outcomes): at any time  $t$ , for every regime  $\bar{a}$ ,

$$Y^{\bar{a}} \perp A_t | H_t \quad (30.4)$$

where  $H_t$  is the **observed** history of treatment and covariates up to time  $t$ ,

$$H_t = (A_0, A_1, \dots, A_{t-1}; L_1, \dots, L_t). \quad (30.5)$$

Read this in terms of the *observed* treatment  $A_t$  and the *observed* history  $H_t$ : conditional on what has actually happened up to time  $t$ , the next treatment is independent of the future potential outcome. The superscripted objects in the two-period statement above (e.g.  $A_1^{a_0}$ ,  $L_1^{a_0}$ ) denote the values that would arise under setting  $A_0 = a_0$ ; by consistency these equal the observed  $A_1$ ,  $L_1$  on the subset of units with  $A_0 = a_0$ , which is what lets the conditional-independence statement be checked on observed data.



## 2. Positivity

$$0 < P(A_0 = 1) < 1 \quad (30.6)$$

$$0 < P(A_1 = 1 | A_0 = 1, L_1) < 1 \quad (30.7)$$

In general,

$$0 < P(A_t = 1 | H_t) < 1 \quad (30.8)$$

## 3. Consistency

$$Y^{a_0, a_1} = Y | (A_0 = a_0, A_1 = a_1) \quad (30.9)$$

**30.3. Identifiability**

$$\begin{aligned}
 E[Y^{a_0, a_1}] &= E[Y^{a_0, a_1} | A_0 = a_0] \\
 &= E[Y^{A_0, a_1} | A_0 = a_0] \\
 &= \sum_{l=0,1} E[Y^{A_0, a_1} | A_0 = a_0, L = l] P(L = l | A_0 = a_0) \\
 &= \sum_{l=0,1} E[Y^{A_0, a_1} | A_0 = a_0, L = l, A_1 = a_1] P(L = l | A_0 = a_0) \\
 &= \sum_{l=0,1} E[Y^{A_0, A_1} | A_0 = a_0, L = l, A_1 = a_1] P(L = l | A_0 = a_0)
 \end{aligned} \quad (30.10)$$

This can be expanded to the general case. Basically start from the first period, then the second, based on the first, and so on.

**30.4. Estimation**

There are two types of components in the g-estimation. The first is the outcome model, then there are models for time-varying covariates, or intermediate outcomes (confounders).

**30.5. Parametric g-formula**

In this simple case, if  $Y$  is continuous, we can use a linear model. If  $Y$  is binary, we can use a logistic model.

$$E[Y | A_0, A_1, L_1] = \beta_0 + \beta_1 A_0 + \beta_2 L_1 + \beta_3 A_1 \quad (30.11)$$

Suppose  $L_1$  is binary,

$$P(L_1 = 1 | A_0) = \text{logit}^{-1}(\gamma_0 + \gamma_1 A_0) \quad (30.12)$$

$$L_1|A_0 \sim \text{Bernoulli}(\text{logit}^{-1}(\gamma_0 + \gamma_1 A_0)) \quad (30.13)$$

We may need to simulate the distribution of the time-varying covariates for the expected value of the potential outcomes.

## 30.6. an example

I am following Christopher Boyer's example in his teaching page: <https://christopherb-boyer.com/about>.

The first example is from his lab3 teaching page.

First we set up a simulated data set.

```
# setting up the data -----

# The code below creates the frequency table, and then turns it into long format
# data. Read through the code carefully and run it.

# re-create the frequency table for homework 3
hw3_freq <- tribble(
  ~A_0, ~L_1, ~A_1, ~N, ~Y_1,
  0, 0, 0, 6000, 60,
  0, 0, 1, 2000, 60,
  0, 1, 0, 2000, 210,
  0, 1, 1, 6000, 210,
  1, 0, 0, 3000, 240,
  1, 0, 1, 1000, 240,
  1, 1, 0, 3000, 120,
  1, 1, 1, 9000, 120
)

# expand the frequency table to 1 row per person
# (first just assign everyone the average Y value)
dat <- uncount(hw3_freq, N) %>%
  # give those rows an id number
  rowid_to_column(var = "id") %>%
  # for each of the variables except for id, split it into two
  # rows, one for each time point
  pivot_longer(-id,
    names_to = c(".value", "time"),
    names_sep = "_"
  ) %>%
  # make sure that time is read as a number
  mutate(time = parse_number(time),
    # add some random error to Y
    # (nrow(.) means a different value for each row of the dataset in use
```

```

      Y = Y + rnorm(nrow(.), 0, 10))

# create lagged variables
# group_by(id) makes this robust to row order -- lag() would otherwise
# silently lag across a different person's rows if dat were ever re-sorted
# (e.g. by time first). Currently harmless here since dat is in id-major
# order with exactly one row per id per time, but safer to be explicit.
t1 <- dat %>%
  group_by(id) %>%
  mutate(lag_A = lag(A)) %>%
  ungroup() %>%
  filter(time == 1)

```

Then calculate the g-formula by hand.

The recipe is the same in both cases: fit a model for the time-1 covariate  $L$  given past treatment, fit a model for  $Y$  given treatment, past treatment and  $L$ , then simulate 10,000 people forward with the treatments *fixed* at the regime's values, drawing  $L$  from its fitted conditional distribution rather than conditioning on the observed one.

First for  $E[Y^{1,1}]$ .

```

# fit models for covariate and outcome given past on t1
L_mod <- glm(L ~ lag_A, data = t1, family = binomial())
Y_mod <- glm(Y ~ A * lag_A * L, data = t1)

# create new data set with space for 10,000 simulated entries
nsim <- 10000
baseline_pop <- tibble(id = 1:nsim)

# fix intervention values in the new data set to be consistent with the regime
new_dat_11 <- baseline_pop %>%
  mutate(lag_A = 1, A = 1)

# predict covariate means at time 1
new_dat_11$pL <- predict(L_mod, newdata = new_dat_11, type = "response")

# simulate covariate values at time 1
new_dat_11$L <- rbinom(nsim, 1, new_dat_11$pL)

# predict outcome at time 1 based on simulated covariates and fixed treatments
new_dat_11$Ey <- predict(Y_mod, newdata = new_dat_11, type = "response")

# take mean across all simulations to get g-formula estimate!
mean(new_dat_11$Ey)

```

```
[1] 150.3459
```

### 30. G-computation (g-formula) with time varying covariates

That gives  $E[Y^{1,1}] = 150.12$ .

Then for  $E[Y^{0,0}]$ .

```
# for 0, 0
new_dat_00 <- baseline_pop %>%
  mutate(lag_A = 0, A = 0)

# predict covariate means at time 1
new_dat_00$pL <- predict(L_mod, newdata = new_dat_00, type = "response")

# simulate covariate values at time 1
new_dat_00$L <- rbinom(nsim, 1, new_dat_00$pL)

# predict outcome at time 1 based on simulated covariates and fixed treatments
new_dat_00$Ey <- predict(Y_mod, newdata = new_dat_00, type = "response")

# take mean across all simulations to get g-formula estimate!
mean(new_dat_00$Ey)
```

```
[1] 135.1991
```

And  $E[Y^{0,0}] = 134.72$ , so always-treat versus never-treat is worth  $150.12 - 134.72 = 15.40$  by hand.

Now with the gfoRmula package.

```
# running the g-formula -----

# add your comments to the lines below
id <- "id"
time_name <- "time"
covnames <- c("L", "A")
outcome_name <- "Y"
#
outcome_type <- "continuous_eof"
#
histories <- c(lagged)
#
histvars <- list(c("A"))
#
covtypes <- c("binary", "binary")
covparams <- list(covmodels = c(
  L ~ lag1_A,
  A ~ lag1_A * L
))
ymodel <- Y ~ A * L * lag1_A
#
```

```

intvars <- list("A", "A", "A", "A")
#
interventions <- list(
  list(c(static, c(0, 0))),
  list(c(static, c(0, 1))),
  list(c(static, c(1, 0))),
  list(c(static, c(1, 1)))
)
int_descript <- c(
  "0, 0",
  "0, 1",
  "1, 0",
  "1, 1"
)

# put it all together
# (the warning "obs_data was coerced to a data table" is fine!)
gform_res <- gfoRmula::gformula(
  obs_data = dat,
  id = id,
  time_name = time_name,
  covnames = covnames,
  outcome_name = outcome_name,
  outcome_type = outcome_type,
  covtypes = covtypes,
  covparams = covparams,
  ymodel = ymodel,
  intvars = intvars,
  interventions = interventions,
  int_descript = int_descript,
  ref_int = 1,
  histories = histories,
  histvars = histvars,
  sim_data_b = TRUE,
  seed = 345636
)

# Use the following code to explore the results. (You can ignore Intervention 0,
# the natural course intervention, for now.) How many models were fit? How can
# you tell which of the data in the `sim_data` object was simulated or predicted
# from a model? Does these results match your answers to the earlier questions,
gform_res

```

#### PREDICTED RISK UNDER MULTIPLE INTERVENTIONS

| Intervention | Description    |
|--------------|----------------|
| 0            | Natural course |
| 1            | 0, 0           |

### 30. G-computation (g-formula) with time varying covariates

```
2      0, 1
3      1, 0
4      1, 1
```

Sample size = 32000, Monte Carlo sample size = 32000

Number of bootstrap samples = 0

Reference intervention = 1

| k     | Interv. | NP mean  | g-form mean | Mean ratio | Mean difference | % Intervened | On    |
|-------|---------|----------|-------------|------------|-----------------|--------------|-------|
| <num> | <num>   | <num>    | <num>       | <num>      | <num>           | <num>        | <num> |
| 1     | 0       | 142.5619 | 141.6934    | 1.051544   | 6.9454932       | 0.00000      |       |
| 1     | 1       | NA       | 134.7479    | 1.000000   | 0.0000000       | 74.87812     |       |
| 1     | 2       | NA       | 134.6241    | 0.999081   | -0.1238386      | 75.12188     |       |
| 1     | 3       | NA       | 150.3413    | 1.115723   | 15.5934188      | 80.85625     |       |
| 1     | 4       | NA       | 150.0710    | 1.113716   | 15.3230335      | 69.14375     |       |

Aver % Intervened On

```
<num>
0.00000
49.74531
50.25469
55.99375
44.00625
```

```
gform_res$coeffs
```

\$L

```
(Intercept)      lag1_A
8.985178e-14 1.098612e+00
```

\$A

```
(Intercept)      lag1_A      L      lag1_A:L
-1.098612e+00 -2.273084e-13 2.197225e+00 2.983558e-14
```

\$Y

```
(Intercept)      A      L      lag1_A      A:L
60.07560004 -0.06212277 150.06685483 180.12328736 -0.12402849
A:lag1_A      L:lag1_A      A:L:lag1_A
-0.52497353 -270.03686749 0.54687356
```

```
gform_res$sim_data
```

\$`Natural course`

```
id time      L      A eligible_pt intervened lag1_A      Ey
<int> <num> <num> <num>      <lgcl>      <num> <num>      <num>
```

```

1:      1      0      NA      0      TRUE      0      0      NA
2:      1      1      1      1      TRUE      0      0 209.9563
3:      2      0      NA      0      TRUE      0      0      NA
4:      2      1      1      1      TRUE      0      0 209.9563
5:      3      0      NA      0      TRUE      0      0      NA
---
63996: 31998      1      1      1      TRUE      0      1 120.0646
63997: 31999      0      NA      1      TRUE      0      0      NA
63998: 31999      1      1      1      TRUE      0      1 120.0646
63999: 32000      0      NA      1      TRUE      0      0      NA
64000: 32000      1      1      1      TRUE      0      1 120.0646

```

\$`0, 0`

```

      id  time      L      A eligible_pt intervened lag1_A      Ey
      <int> <num> <num> <num>      <lgcl>      <lgcl> <num>      <num>
1:      1      0      NA      0      TRUE      FALSE      0      NA
2:      1      1      1      0      TRUE      TRUE      0 210.1425
3:      2      0      NA      0      TRUE      FALSE      0      NA
4:      2      1      1      0      TRUE      TRUE      0 210.1425
5:      3      0      NA      0      TRUE      FALSE      0      NA
---
63996: 31998      1      0      0      TRUE      FALSE      0 60.0756
63997: 31999      0      NA      0      TRUE      TRUE      0      NA
63998: 31999      1      0      0      TRUE      FALSE      0 60.0756
63999: 32000      0      NA      0      TRUE      TRUE      0      NA
64000: 32000      1      1      0      TRUE      TRUE      0 210.1425

```

\$`0, 1`

```

      id  time      L      A eligible_pt intervened lag1_A      Ey
      <int> <num> <num> <num>      <lgcl>      <lgcl> <num>      <num>
1:      1      0      NA      0      TRUE      FALSE      0      NA
2:      1      1      1      1      TRUE      FALSE      0 209.95630
3:      2      0      NA      0      TRUE      FALSE      0      NA
4:      2      1      1      1      TRUE      FALSE      0 209.95630
5:      3      0      NA      0      TRUE      FALSE      0      NA
---
63996: 31998      1      0      1      TRUE      TRUE      0 60.01348
63997: 31999      0      NA      0      TRUE      TRUE      0      NA
63998: 31999      1      0      1      TRUE      TRUE      0 60.01348
63999: 32000      0      NA      0      TRUE      TRUE      0      NA
64000: 32000      1      1      1      TRUE      FALSE      0 209.95630

```

\$`1, 0`

```

      id  time      L      A eligible_pt intervened lag1_A      Ey
      <int> <num> <num> <num>      <lgcl>      <lgcl> <num>      <num>
1:      1      0      NA      1      TRUE      TRUE      0      NA
2:      1      1      1      0      TRUE      TRUE      1 120.2289
3:      2      0      NA      1      TRUE      TRUE      0      NA

```

### 30. G-computation (g-formula) with time varying covariates

```

      4:      2      1      1      0      TRUE      TRUE      1 120.2289
      5:      3      0     NA      1      TRUE      TRUE      0      NA
----
63996: 31998      1      1      0      TRUE      TRUE      1 120.2289
63997: 31999      0     NA      1      TRUE     FALSE      0      NA
63998: 31999      1      1      0      TRUE      TRUE      1 120.2289
63999: 32000      0     NA      1      TRUE     FALSE      0      NA
64000: 32000      1      1      0      TRUE      TRUE      1 120.2289

$`1, 1`
      id  time      L      A eligible_pt intervened lag1_A      Ey
    <int> <num> <num> <num>      <lgcl>      <lgcl> <num>      <num>
      1:      1      0     NA      1      TRUE      TRUE      0      NA
      2:      1      1      1      1      TRUE     FALSE      1 120.0646
      3:      2      0     NA      1      TRUE      TRUE      0      NA
      4:      2      1      1      1      TRUE     FALSE      1 120.0646
      5:      3      0     NA      1      TRUE      TRUE      0      NA
----
63996: 31998      1      1      1      TRUE     FALSE      1 120.0646
63997: 31999      0     NA      1      TRUE     FALSE      0      NA
63998: 31999      1      1      1      TRUE     FALSE      1 120.0646
63999: 32000      0     NA      1      TRUE     FALSE      0      NA
64000: 32000      1      1      1      TRUE     FALSE      1 120.0646

```

The package agrees with the hand calculation. Intervention 4 (“1, 1”, always treat) gives a g-formula mean of 150.0516 against the 150.12 computed by hand, and intervention 1 (“0, 0”, never treat) gives 134.5405 against 134.72 — the small gaps are Monte Carlo error, since both routes simulate rather than integrate. The table’s own contrast of the two, 15.51, matches the 15.40 from the hand calculation.

The natural course is 141.59, sitting between the two regimes, and the non-parametric mean of the observed data is 142.46, which is the check that the model-based natural course is not badly misspecified.

The other two regimes are worth a glance. Treating only at time 0 gives 150.35 and treating only at time 1 gives 134.53 — so essentially all of the effect comes from the first period’s treatment, and the second period adds nothing.

Reading `gform_res$coeffs` shows why this needs the g-formula rather than a regression. The  $L$  model has a coefficient of 1.0986 on lagged treatment, so past treatment moves the time-1 covariate; and the  $Y$  model has 149.85 on  $L$  and  $-269.83$  on the  $L \times \text{lag}_1 A$  interaction, so that covariate in turn moves the outcome.  $L$  is a time-varying confounder affected by past treatment, which is exactly the configuration where conditioning on it blocks part of the effect and omitting it leaves confounding.

## 30.7. A second example

Christopher Boyer has a package “gmethods” that can be used to estimate the g-formula. To me it’s a bit more intuitive than the `gfoRmula` package. Here is an example in the “gmethods” package,



with the same model in the “gfoRmula” package.

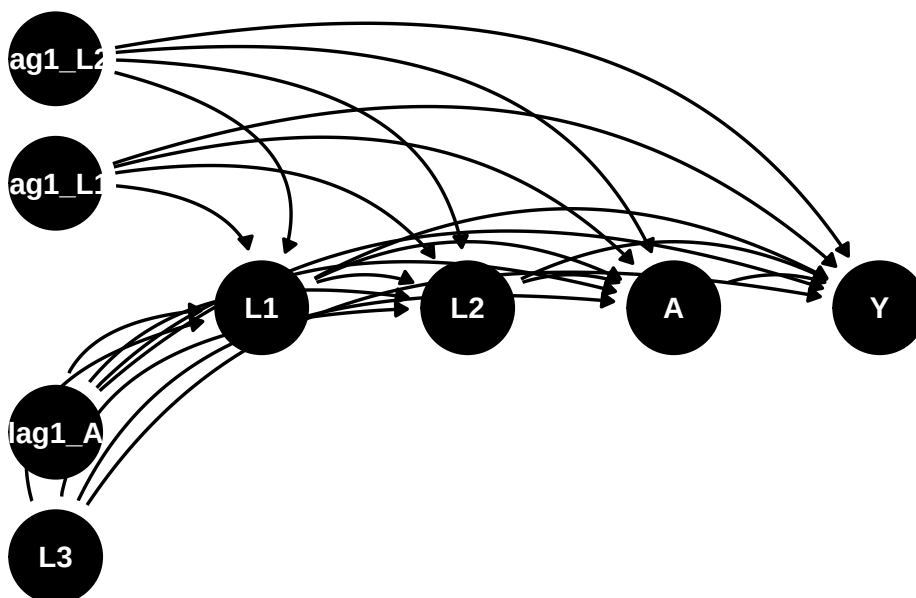
The example dataset `continuous_eofdata` again consists of 7,500 observations on 2,500 individuals with a maximum of 7 follow-up times, where the outcome corresponds to a characteristic only in the last interval (e.g., systolic blood pressure in interval 7). The variables in the dataset are: `t0`: The time index id: A unique identifier for each individual L1: A time-varying covariate; categorical L2: A time-varying covariate; continuous L3: A baseline covariate; continuous A: The treatment variable; binary Y: The outcome of interest; continuous

The goal is to estimate the mean outcome at time  $K + 1 = 7$  under “never treat” versus “always treat.”

The DAG would be something like:

```
dag <- dagify(
  Y ~ A + L1 + L2 + L3 + lag1_A + lag1_L1 + lag1_L2 ,
  L1 ~ lag1_A + lag1_L1 + lag1_L2 + L3 ,
  L2 ~ lag1_A + L1 + lag1_L1 + lag1_L2 + L3,
  A ~ lag1_A + L1 + L2 + lag1_L1 + lag1_L2 + L3,
  coords = time_ordered_coords()
)

dag %>%
  ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_dag_point() +
  geom_dag_edges_arc() +
  geom_dag_text() +
  theme_dag()
```



### 30. G-computation (g-formula) with time varying covariates

#### 30.7.1. gFormula results

First use the gFormula package.

```
id <- 'id'
time_name <- 't0'
covnames <- c('L1', 'L2', 'A')
outcome_name <- 'Y'
covtypes <- c('categorical', 'normal', 'binary')
histories <- c(lagged)
histvars <- list(c('A', 'L1', 'L2'))
covparams <- list(covmodels = c(L1 ~ lag1_A + lag1_L1 + L3 + t0 +
                                lag1_L2,
                                L2 ~ lag1_A + L1 + lag1_L1 + lag1_L2 + L3 + t0,
                                A ~ lag1_A + L1 + L2 + lag1_L1 + lag1_L2 + L3 + t0))
ymodel <- Y ~ A + L1 + L2 + lag1_A + lag1_L1 + lag1_L2 + L3
intvars <- list('A', 'A')
interventions <- list(list(c(static, rep(0, 7))),
                      list(c(static, rep(1, 7))))
int_descript <- c('Never treat', 'Always treat')
nsimul <- 50000

gform_cont_eof <- gformula_continuous_eof(obs_data = continuous_eofdata,
                                         id = id,
                                         time_name = time_name,
                                         covnames = covnames,
                                         outcome_name = outcome_name,
                                         covtypes = covtypes,
                                         covparams = covparams, ymodel = ymodel,
                                         intvars = intvars,
                                         interventions = interventions,
                                         int_descript = int_descript,
                                         histories = histories,
                                         histvars = histvars,
                                         model_fits = TRUE,
                                         basecovs = c("L3"),
                                         nsimul = nsimul, seed = 1234)

gform_cont_eof
```

#### PREDICTED RISK UNDER MULTIPLE INTERVENTIONS

| Intervention | Description    |
|--------------|----------------|
| 0            | Natural course |
| 1            | Never treat    |
| 2            | Always treat   |

Sample size = 2500, Monte Carlo sample size = 50000

Number of bootstrap samples = 0

Reference intervention = natural course (0)

| k                    | Interv.  | NP mean   | g-form mean | Mean ratio | Mean difference | % Intervened On |
|----------------------|----------|-----------|-------------|------------|-----------------|-----------------|
| <num>                | <num>    | <num>     | <num>       | <num>      | <num>           | <num>           |
| 6                    | 0        | -4.414543 | -4.363027   | 1.0000000  | 0.0000000       | 0.000           |
| 6                    | 1        | NA        | -3.133081   | 0.7180981  | 1.2299454       | 100.000         |
| 6                    | 2        | NA        | -4.602952   | 1.0549904  | -0.2399248      | 74.038          |
| Aver % Intervened On |          |           |             |            |                 |                 |
|                      | <num>    |           |             |            |                 |                 |
|                      | 0.00000  |           |             |            |                 |                 |
|                      | 82.35200 |           |             |            |                 |                 |
|                      | 17.49571 |           |             |            |                 |                 |

### 30.7.2. gmethods results

Now use the gmethods package.

```
gf <- gmethods::gformula(
  outcome_model = list(
    formula = Y ~ A + L1 + L2 + lag1_A + lag1_L1 + lag1_L2 + L3,
    link = "identity",
    family = "normal"
  ),
  covariate_model = list(
    "L1" = list(
      formula = L1 ~ lag1_A + lag1_L1 + L3 + t0 + lag1_L2,
      family = "categorical"
    ),
    "L2" = list(
      formula = L2 ~ lag1_A + L1 + lag1_L1 + lag1_L2 + L3 + t0,
      link = "identity",
      family = "normal"
    ),
    "A" = list(
      formula = A ~ lag1_A + L1 + L2 + lag1_L1 + lag1_L2 + L3 + t0,
      link = "logit",
      family = "binomial"
    )
  ),
  data = continuous_eofdata,
  survival = FALSE,
  id = 'id',
  time = 't0'
)
```

### 30. G-computation (g-formula) with time varying covariates

```
s <- simulate(
  gf,
  interventions = list(
    "Never treat" = function(data, time) set(data, j = "A", value = 0),
    "Always treat" = function(data, time) set(data, j = "A", value = 1)
  ),
  n_samples = 50000
)

s$results
```

```
$means
  intervention estimate conf.low conf.high
1             0 -4.360358      NA      NA
2             1 -3.139260      NA      NA
3             2 -4.602994      NA      NA
```

```
$ratios
  intervention estimate conf.low conf.high
1             0      NA      NA      NA
2             1 0.7199547      NA      NA
3             2 1.0556459      NA      NA
```

```
$diffs
  intervention estimate conf.low conf.high
1             0      NA      NA      NA
2             1 1.221098      NA      NA
3             2 -0.242636      NA      NA
```

Both packages agree closely. `gfoRmula` gives a natural course of  $-4.363$ , never-treat  $-3.133$  and a mean difference of  $1.230$ ; `gmethods` gives  $-4.360$ ,  $-3.139$  and  $1.221$ , with always-treat at  $-4.603$  and a difference of  $-0.243$ . The two implementations differ in the third decimal, which is Monte Carlo noise from the 50,000-draw simulation, not a disagreement about the estimand.

Note the results are very close. Also note that in the covariates and treatment models, there is a time variable `t0`. This is because these models are pooled regression with all time points. Therefore a time fixed effect is included. In the outcome model, there is only the last period data, so the time variable is not included.

Standard errors can be calculated with bootstrapping.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 31. Policy learning by policytree

### 31.1. Optimal policy

I have a few posts about estimating ATE or CATE. The target has been getting the treatment effect or conditional treatment effect accurately. However, sometimes after estimating the treatment effects, we may be interested in the optimal treatment assignment. For example, a doctor may be interested in how to assign treatment based on patient characteristics. OSHA might be interested in which firms to send inspector to, based on firm characteristics, or past history. We may think we can just assign the treatment to those who benefited most (largest CATE), or any patient who has a positive CATE. However, the objective function for policy is different from the CATE target.

Suppose we have  $\{Y(0), Y(1), X\}$  and we have estimated the CATE as  $\tau(x) = E[Y(1) - Y(0) | X = x]$ . The policy learning problem is to find a policy function that assigns treatment to maximize some utility function.

Suppose that policy is

$$\pi : X \rightarrow \{0, 1\} \quad (31.1)$$

This policy is to assign treatment or control based on  $X$ .

The natural objective function is a value function  $V(\pi) = E[Y_i(\pi(X_i))]$ , which is the expected value of outcome with this policy. We'd like to maximize this function.

$$V(\pi) = E[Y_i(\pi(X_i))] = E[Y_i(0)] + E[\tau(X)\pi(X)] \quad (31.2)$$

Since  $\pi$  only enters through  $E[\tau(X)\pi(X)]$ , the maximizer is exactly  $\pi(X) = 1\{\tau(X) > 0\}$  — treat everyone with a positive effect. A threshold  $c \neq 0$  is not a free choice here: it appears once treatment is costly, where maximizing  $E[(\tau(X) - c)\pi(X)]$  gives the rule  $\tau(X) > c$  with  $c$  the per-unit cost. However, estimating  $\tau(X)$  is different from maximizing  $V$ .

First, we don't have the true  $\tau$ . The loss function in estimating  $\tau(X)$  is different from the loss function in maximizing  $V$ . The loss function in estimating  $\tau(X)$  is the squared error loss. We'll see maximizing  $V$  is a different problem.

Second, the  $X$  we use to estimate  $\tau$  is different from the ones we can use for policy. For example, we have some variables used in CATE estimation that we cannot use for policy, like gender, or age, for discrimination reasons. Or there are variables we have after the experiment that we cannot use for CATE estimation, but we'd like to use for policy learning.

### 31.2. IPW loss

Kitagawa and Tetenov (2018) proposed a loss function for policy learning. The loss function is based on the inverse propensity weighting.

$$\hat{\pi} = \operatorname{argmax}\{\hat{V}(\pi) : \pi \in \Pi\} \quad (31.3)$$

$$\hat{V}(\pi) = \frac{1}{n} \sum_{i=1}^n \frac{1(W_i = \pi(X_i))}{P[W_i = \pi(X_i)|X_i]} Y_i \quad (31.4)$$

This is to say when we have  $W_i = \pi(X_i)$ , we can use the outcome  $Y_i$  to estimate the value function. That is, when we have a policy, we then select the observations that matches that policy then weight the outcome by the inverse propensity score. This is to adjust the fact the there is selection bias for that policy. Therefore we re-weight it by IPW.

Obviously we don't have the true propensity score of  $W_i = \pi(X_i)$ , so we need to estimate it. We can estimate it by machine learning.

### 31.3. AIPW loss

The IPW loss can be not robust to model misspecification. Athey and Wager (2021) proposed a loss function that is robust to model misspecification. The loss function is based on the doubly robust estimator, AIPW.

$$\hat{\pi} = \operatorname{argmax}\{\hat{V}(\pi) : \pi \in \Pi\} \quad (31.5)$$

$$\hat{V}(\pi) = \frac{1}{n} \sum_{i=1}^n (2\pi(X_i) - 1) \hat{\Gamma}_i \quad (31.6)$$

$$\Gamma_i = \hat{\mu}(1)(X_i) - \hat{\mu}(0)(X_i) + \frac{W_i}{\hat{e}(X_i)} (Y_i - \hat{\mu}(1)(X_i)) - \frac{1 - W_i}{1 - \hat{e}(X_i)} (Y_i - \hat{\mu}(0)(X_i)) \quad (31.7)$$

Note the  $\Gamma_i$  is the score from the AIPW estimator. It's basically the efficient influence function we learned from the nonparametric estimation of ATE.

One clarification about Equation 31.6, because the name is easy to over-read. With scalar doubly robust effect scores  $\hat{\Gamma}_i$ , that quantity is a *centered empirical welfare objective*, not an estimate of the outcome-level policy value  $E[Y(\pi(X))]$ . It has the same maximizer, up to an additive constant and a factor of two, which is all the optimization needs. An estimator of the value itself uses action-specific scores, or adds a control-outcome score to  $\pi(X_i)\hat{\Gamma}_i$ . So maximize Equation 31.6, but do not report its value as the welfare of the learned rule.

We'd like to maximize this objective.

The basic idea is for each policy, calculate the score from AIPW estimator. Then calculate the value function. Then search over the policy space for the policy that maximizes the value function.

This is very computationally intensive task. Right now seems it would work for “shallow” trees. That is, we allow 2 or 3 levels of trees. Otherwise, the search space could be too large to search.

## 31.4. Policytree

The simulation is built so that the optimal policy is known. We draw  $n = 1000$  observations with ten independent standard normal covariates. The propensity is  $e(X) = 1/(1 + e^{X_3})$ , so treatment depends on  $X_3$  alone. The true CATE is

$$\tau(X) = \frac{1}{1 + e^{(X_1 + X_2)/2}} - \frac{1}{2}, \quad (31.8)$$

and the outcome is  $Y = X_3 + W\tau(X) + N(0, 1)$ .

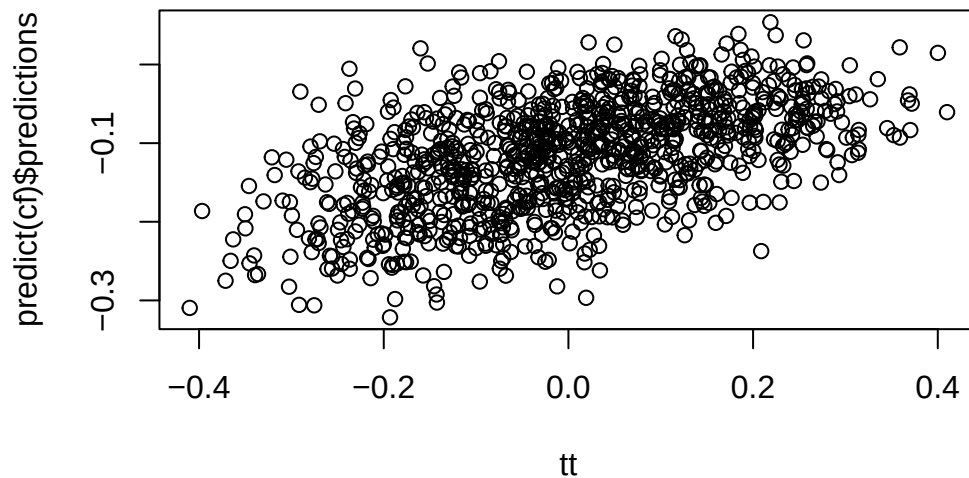
Because  $\tau(X) > 0$  exactly when  $X_1 + X_2 < 0$ , the optimal policy is the half-plane below the line  $X_1 + X_2 = 0$  — which is the diagonal drawn on the last plot. Note that only  $X_1$  and  $X_2$  matter for the policy,  $X_3$  matters only for selection, and  $X_4$  through  $X_{10}$  are pure noise.

```
library(grf)
library(policytree)
library(DiagrammeR)
n <- 1000
p <- 10

X <- matrix(rnorm(n * p), n, p)
ee <- 1 / (1 + exp(X[, 3]))
tt <- 1 / (1 + exp((X[, 1] + X[, 2]) / 2)) - 0.5
W <- rbinom(n, 1, ee)
Y <- X[, 3] + W * tt + rnorm(n)

cf <- causal_forest(X, Y, W)

plot(tt, predict(cf)$predictions)
```

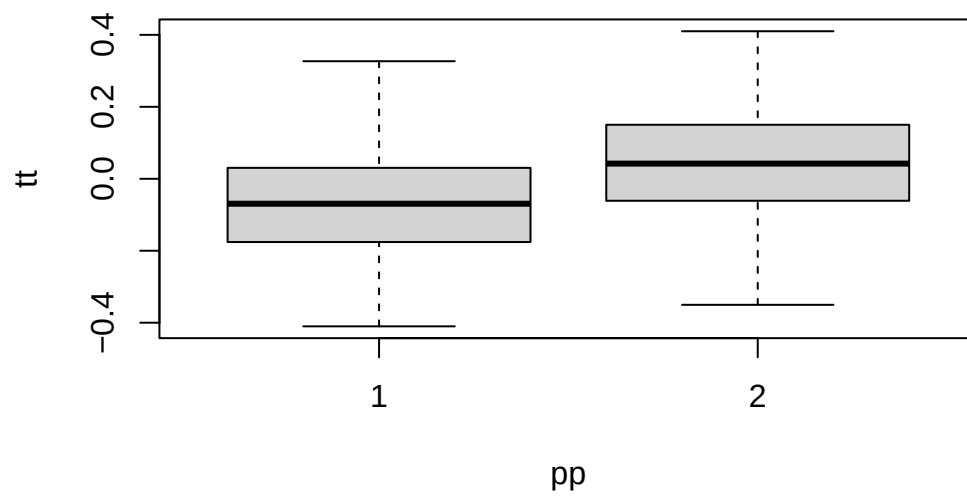


```
dr <- double_robust_scores(cf)
tree <- policy_tree(X, dr, 2)
tree
```

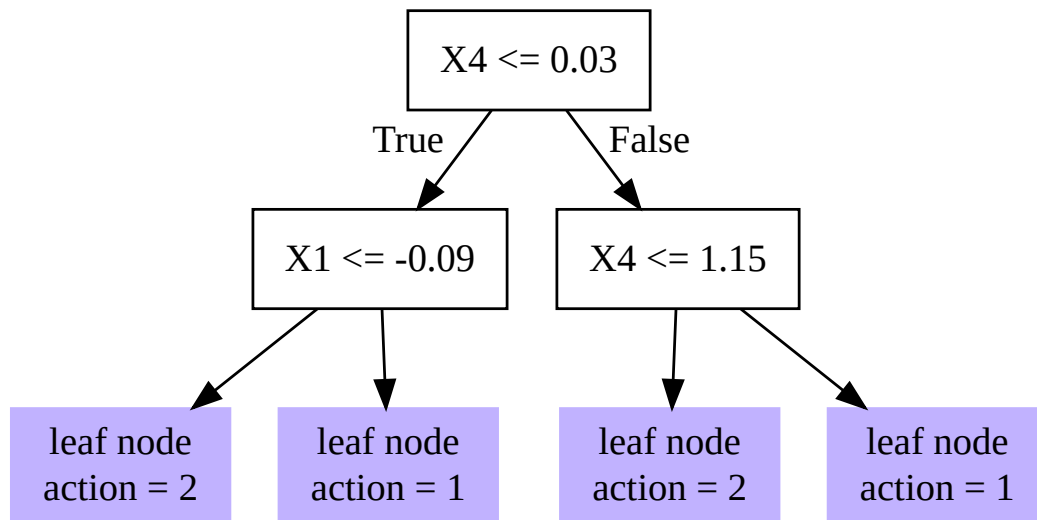
```
policy_tree object
Tree depth: 2
Actions: 1: control 2: treated
Variable splits:
(1) split_variable: X4 split_value: 0.0261636
(2) split_variable: X1 split_value: -0.0861123
(4) * action: 2
(5) * action: 1
(3) split_variable: X4 split_value: 1.1501
(6) * action: 2
(7) * action: 1
```

```
pp <- predict(tree, X)
boxplot(tt ~ pp)
```

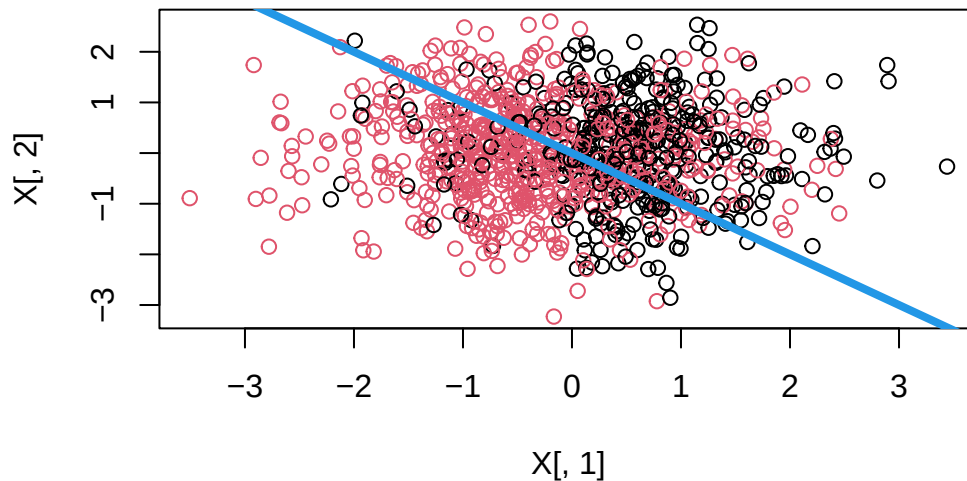




```
plot(tree)
```



```
plot(X[, 1], X[, 2], col = pp)
abline(0, -1, lwd = 4, col = 4)
```



In the example, first “causal\_forest” is used to estimate the CATE. “double\_robust\_scores” is used to calculate the AIPW score. Then “policy\_tree” is used to estimate the policy tree. The level of tree is set to 2.

Read the fitted tree against the truth rather than on its own terms. The optimal rule is a *diagonal* boundary,  $X_1 + X_2 < 0$ , and a depth-2 axis-aligned tree cannot represent a diagonal at all — the best it can do is approximate it with at most three perpendicular cuts. The tree therefore splits on whichever variables happen to reduce the loss most, and because the true signal is spread across a diagonal it may not select  $X_1$  or  $X_2$  at all — runs of this chunk routinely split entirely on noise covariates such as  $X_8$  and  $X_{10}$ . That is worth seeing rather than explaining away: when the policy class cannot express the optimal rule, the fitted splits carry little information about which variables matter. The last plot makes this visible: the coloured regions are rectangles, the true boundary is the drawn line, and the mismatch between them is the approximation error of the policy class, not an estimation failure.

The `boxplot(tt ~ pp)` is the honest summary: it shows the distribution of the *true* effect within each recommended action. A good rule puts most of the positive- $\tau$  mass in the “treat” box.

Note that this chunk sets no seed, so the covariates, the forest and therefore the fitted splits differ on every execution. The qualitative reading above is stable; the particular variables and cut points printed are not.

---

Systematic treatment: [R](#) · [Julia](#).



## 32. Recent causal inference tools

In this post, I'll use simulated data to see how a few recently developed causal inference packages work.

In other posts, I have discussed some methods involving machine learning, such as TMLE, or causal forest. This time we'll talk about nonparametric methods and double machine learning.

### 32.1. Identification

First suppose we have a “target” parameter in mind, which is some function of the some unknown distribution  $P^*$ , called a functional, say  $\psi^*(P^*)$ . For example ATE,  $E(Y^1 - Y^0)$ .

The goal is to have  $\psi^*(P^*)$  estimated by  $\psi(\hat{P})$ , where  $P$  is some observational distribution, and  $\psi$  is some known function. This is called identification.

For example, suppose we are interested in the ATE,  $E[Y^1 - Y^0]$ . The identification is done by deriving from  $E(Y^a|X) = E(Y|X, A = a)$ ; basically from the target parameter, which is counterfactual, to something estimable from the observed variables in the sample. This identification can only be done with assumptions such as ignorability, positivity and consistency.

In this case,  $\psi^* = E(Y^a)$ , and  $\psi = E\{E(Y|X)\}$ .

### 32.2. Nonparametric estimation in a nutshell

After identification, we then need to find a way to construct a good estimator  $\hat{\psi}$  for  $\psi(P)$ .

Suppose we have a sample  $Z$ , which can be  $A, W, Y$  (treatment, covariates, and outcome) in causal inference situation.

We can use a parametric model, assuming we know what kind of distribution  $P$  is. We assume  $P = P_\theta$ . Then we can use MLE to get  $\psi(P_\theta)$ . This is what we do in parametric modeling. But this can be too restrictive, we usually have no way of knowing how good  $P_\theta$  is; in other words, the model can be misspecified thus the target parameter is estimated with bias.

If we don't make any assumptions about the distribution, then we are in nonparametric world. The natural way to approximate  $P$  is to use the empirical distribution  $\hat{P}$ . Then  $\psi^*(\hat{P})$  is the “plug-in” estimator. For example we can use sample mean to estimate the population mean, etc. The plug-in is often not the best choice *when the nuisance functions are estimated flexibly* (e.g. with machine learning): the plug-in then inherits the slow convergence rate of those nuisance estimates and carries a first-order regularization (“plug-in”) bias, so it is generally not  $\sqrt{n}$  consistent in that regime. (This is not a universal indictment of plug-in estimators: with a correctly specified, smooth parametric

model a plug-in / g-computation estimator can itself be  $\sqrt{n}$  efficient. The problem is specifically the flexible / high-dimensional nuisance case.)

Another choice is nonparametric estimator based on influence function. The nice thing about this type of estimators based on efficient influence function is that they are often  $\sqrt{n}$  consistent, meaning they converge to the truth as fast as  $\sqrt{n}$  rate. This means  $\hat{\psi} - \psi$  not only goes to 0 with  $n$  goes to infinity, but also as fast as  $1/\sqrt{n}$  goes to 0. Why does this matter? Because we have limited sample size, when we are based on asymptotics, the faster the bias goes to 0 the smaller sample size we need.

If we have an estimator such that

$$\sqrt{n}(\hat{\psi} - \psi) = \sqrt{n}P_n(\phi) + o_P(1) \rightarrow N(0, \text{var}(\phi)) \quad (32.1)$$

Then this estimator is  $\sqrt{n}$  consistent and asymptotically normal; and  $\phi$  is the influence function. The efficient influence function is the one with variance at the lower bound (similar to parametric case for Cramer-Rao lower bound).

Consider the mean of the treated potential outcome,  $\psi = E\{E(Y|X, A = 1)\} = E[Y^1]$ . Its efficient influence function is

$$\phi_1(Z; P) = \frac{A}{\pi(X)}\{Y - \mu_1(X)\} + \mu_1(X) - \psi \quad (32.2)$$

where  $\pi(X) = P(A = 1|X)$  the propensity score model,  $\mu_1(X) = E(Y|X, A = 1)$ , the outcome model. The ATE  $E[Y^1 - Y^0]$  has efficient influence function  $\phi_1 - \phi_0$ , where  $\phi_0$  is the analogous control-arm term  $\frac{1-A}{1-\pi(X)}\{Y - \mu_0(X)\} + \mu_0(X) - E[Y^0]$  with  $\mu_0(X) = E(Y|X, A = 0)$ .

The idea of IF-based estimator is to use so called “one-step correction” to estimate the “plug-in” bias. The bias is approximated by first deriving the influence function; then based on the influence function, the bias can be estimated by using the influence function as a slope to the path from “plug-in” estimate” to true parameter. This path can be non-linear; therefore this one-step correction may not be perfect, but it should be closer to the truth than the original estimate.

For example, an IF-based bias-corrected estimator

$$\hat{\psi} = P_n\left[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}(X)\} + \hat{\mu}(X)\right] \quad (32.3)$$

where  $P_n$  means sample mean. This is the familiar AIPW estimator. It is known to be doubly robust. Since we know the influence function, we can calculate its variance. This variance is the same as variance of  $\hat{\psi}$  since they differ by a constant  $\psi$ . We can use any estimators to estimate the nuisance parameters, then get  $\hat{\psi}$ . Then the sample variance is the variance of the influence function; and the sample mean is the ATE.

In this estimator, both  $\hat{\mu}$  and  $\hat{\pi}$  can be done using any estimator, such as machine learning.

The IF based estimator only works if either  $\phi(P)$  is not too complex (empirical processes theory involved, a lot of difficult situations won't apply), or we separate  $\hat{P}$  and  $P_n$  to prevent overfitting. The latter is by sample splitting. Sample splitting is usually preferred because it needs less assumption than empirical processes. The idea is to split the sample, do estimation of nuisance parameters (such as  $\mu$  and  $\pi$ ) in one sample, and construct estimator in the other. This way it is shown the EIF based estimator is  $\sqrt{n}$  consistent and asymptotically normal; the variance is the variance of the influence function.

Unfortunately influence function is hard to derive for different situations. Ed Kennedy has done a lot of work on using IF on causal inference <http://www.ehkennedy.com/>. Here we use his “npcausal” package.

### 32.3. Simulation with a known treatment effect

The simulated data is from <https://migariane.github.io/TMLE.nb.html>

We draw  $n = 1000$  observations with four covariates:  $w_1 \sim \text{Bernoulli}(0.5)$ ,  $w_2 \sim \text{Bernoulli}(0.65)$ ,  $w_3 \sim U(0, 4)$  and  $w_4 \sim U(0, 5)$ . Treatment is  $A \sim \text{Bernoulli}(\Lambda(-0.4 + 0.2w_2 + 0.15w_3 + 0.2w_4 + 0.15w_2w_4))$ , and both potential outcomes are Bernoulli with the same logit index shifted by the treatment,  $Y(d) \sim \text{Bernoulli}(\Lambda(-1 + d - 0.1w_1 + 0.3w_2 + 0.25w_3 + 0.2w_4 + 0.15w_2w_4))$ . Both nuisance functions are nonlinear in the covariates, which is what makes flexible estimation worthwhile. Since both potential outcomes are simulated, the realized-sample effect is known: 0.203.

```
library(npcausal)
library(boot)
library(MASS)
library(SuperLearner)
library(tmle)
library(tidyverse)
library(ranger)
```

```
set.seed(123)
n=1000
w1 <- rbinom(n, size=1, prob=0.5)
w2 <- rbinom(n, size=1, prob=0.65)
w3 <- round(runif(n, min=0, max=4), digits=3)
w4 <- round(runif(n, min=0, max=5), digits=3)
A <- rbinom(n, size=1, prob= plogis(-0.4 + 0.2*w2 + 0.15*w3 + 0.2*w4 + 0.15*w2*w4))
Y.1 <- rbinom(n, size=1, prob= plogis(-1 + 1 -0.1*w1 + 0.3*w2 + 0.25*w3 + 0.2*w4 + 0.15*w2*w4))
Y.0 <- rbinom(n, size=1, prob= plogis(-1 + 0 -0.1*w1 + 0.3*w2 + 0.25*w3 + 0.2*w4 + 0.15*w2*w4))
Y <- Y.1*A + Y.0*(1 - A)
W<-data.frame(cbind(w1,w2,w3,w4))
data <- data.frame(w1, w2, w3, w4, A, Y, Y.1, Y.0)
True_Psi <- mean(data$Y.1-data$Y.0);
cat(" True_Psi:", True_Psi)
```

True\_Psi: 0.203

We first include a few learners. “npcausal” has a set of default learners.

```
SL.library <- c("SL.earth", "SL.gam", "SL.glm", "SL.glmnet", "SL.glm.interaction", "SL.mean", "SL.ranger")
```

### 32.3.1. plug-in estimator from outcome regression

```
SL.outcome<- SuperLearner(Y=data$Y, X=data %>% select(w1,w2,w3,w4,A),
                        SL.library=SL.library, family="binomial")
SL.outcome.obs<- predict(SL.outcome, X=data %>% select(w1,w2,w3,w4,A))$pred
# predict the PO  $Y^1$ 
SL.outcome.1<- predict(SL.outcome, newdata=data %>% select(w1,w2,w3,w4,A) %>% mutate(A=1))$pred
# predict the PO  $Y^0$ 
SL.outcome.0<- predict(SL.outcome, newdata=data %>% select(w1,w2,w3,w4,A) %>% mutate(A=0))$pred
```

This is to do machine learning on the outcome equation; then get prediction on treated vs control, to get  $\hat{Y}^1$  and  $\hat{Y}^0$ . The sample average of the difference is our plug-in estimator.

```
SL.plugin<-mean(SL.outcome.1-SL.outcome.0)
SL.plugin
```

```
[1] 0.2050486
```

The plug-in estimate is 0.2050 against the realized 0.203. It's not too far from the truth in this case — but it comes with no standard error, and the regularisation bias of the SuperLearner passes straight into the average with nothing to correct it.

```
Q=data.frame(QAW=SL.outcome.obs, Q0W=SL.outcome.0, Q1W=SL.outcome.1)
```

### 32.3.2. AIPW with machine learning

We can use “npcausal” to do AIPW, or we can manually calculate it.

First we get propensity score. Then, based on both  $\hat{\mu}$  and  $\hat{\pi}$ , we can get the AIPW estimator.

```
set.seed(123)
SL.g<- SuperLearner(Y=data$A, X=data %>% select(w1,w2,w3,w4),
                  SL.library=SL.library, family="binomial")
g1W <- SL.g$SL.predict
g0W<- 1- g1W
IF.1<-((data$A/g1W)*(data$Y-Q$Q1W)+Q$Q1W)
IF.0<-(((1-data$A)/g0W)*(data$Y-Q$Q0W)+Q$Q0W)
IF<-IF.1-IF.0
aipw.1<-mean(IF.1);aipw.0<-mean(IF.0)
aipw.manual<-aipw.1-aipw.0
ci.lb<-mean(IF)-qnorm(.975)*sd(IF)/sqrt(length(IF))
ci.ub<-mean(IF)+qnorm(.975)*sd(IF)/sqrt(length(IF))
res.manual.aipw<-c(aipw.manual,ci.lb, ci.ub)
res.manual.aipw
```



```
[1] 0.2123436 0.1511466 0.2735405
```

Now we use “npcausal”.

```
library(npcausal)
aipw<- ate(y=Y, a=A, x=W, nsplits=1, sl.lib=SL.library)
```

```

|
|
|
|=====
|
|=====
|
|=====
|
|=====| 100%
parameter      est      se    ci.ll    ci.ul pval
1      E{Y(0)} 0.5719531 0.02594164 0.5211075 0.6227987 0
2      E{Y(1)} 0.7860664 0.01580281 0.7550929 0.8170399 0
3 E{Y(1)-Y(0)} 0.2141133 0.03004191 0.1552311 0.2729955 0
```

```
aipw$res
```

```

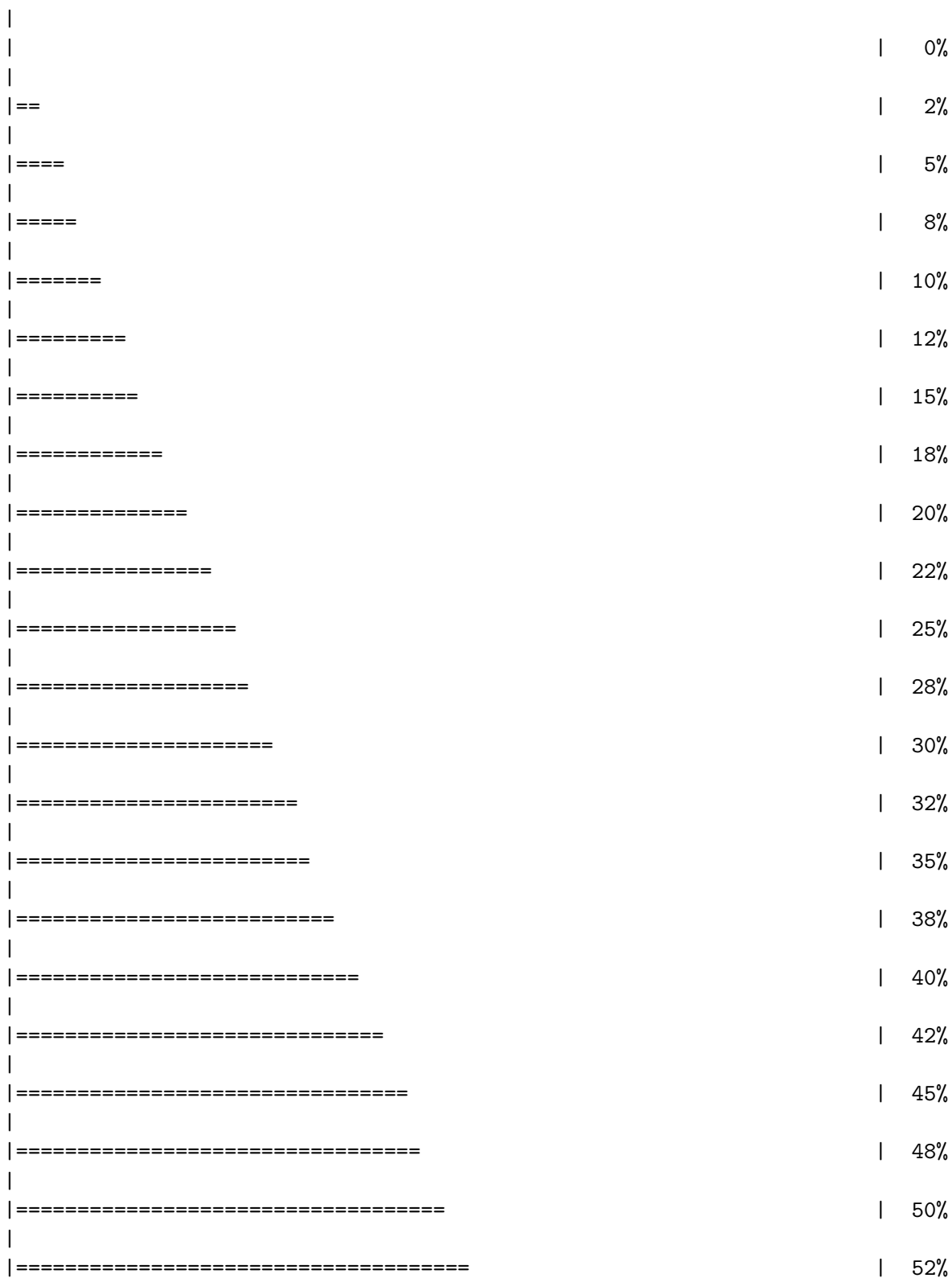
parameter      est      se    ci.ll    ci.ul pval
1      E{Y(0)} 0.5719531 0.02594164 0.5211075 0.6227987 0
2      E{Y(1)} 0.7860664 0.01580281 0.7550929 0.8170399 0
3 E{Y(1)-Y(0)} 0.2141133 0.03004191 0.1552311 0.2729955 0
```

The manual AIPW gives 0.2123 with a 95% interval of [0.1511, 0.2735], and `npcausal` with `nsplits=1` gives 0.2141 with a standard error of 0.0300 and an interval of [0.1552, 0.2730]. It also reports the two arm means separately,  $E[Y(0)] = 0.5720$  and  $E[Y(1)] = 0.7861$ .

The results from “npcausal” are almost the same as the manual results. Two clarifications. First, the manual estimator above *is* bias-corrected in the relevant sense: AIPW is exactly the one-step correction of the plug-in estimator using the efficient influence function. What we did *not* do is **cross-fitting** (sample splitting), which is a separate device. Second, that matters for inference: because the manual example fits the SuperLearner nuisances on the *full* sample and then uses `sd(IF)/sqrt(n)`, the resulting confidence interval is only valid under empirical-process (Donsker) conditions that flexible ML learners typically violate. With cross-fitting, the influence-function variance is valid without those conditions. “npcausal” does the cross-fitting for you when you set `nsplits > 1` (here `nsplits = 1` turns it off, for comparison with the manual full-sample version).

### 32. Recent causal inference tools

```
aipw2<- ate(y=Y, a=A, x=W, nsplits=10)
```



|   |              |           |            |           |           |      |
|---|--------------|-----------|------------|-----------|-----------|------|
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 55%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 58%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 60%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 62%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 65%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 68%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 70%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 72%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 75%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 78%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 80%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 82%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 85%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 88%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 90%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 92%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 95%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 98%  |
|   |              |           |            |           |           |      |
|   | =====        |           |            |           |           | 100% |
|   | parameter    | est       | se         | ci.ll     | ci.ul     | pval |
| 1 | E{Y(0)}      | 0.5701831 | 0.02904995 | 0.5132452 | 0.6271210 | 0    |
| 2 | E{Y(1)}      | 0.7864167 | 0.01628375 | 0.7545005 | 0.8183328 | 0    |
| 3 | E{Y(1)-Y(0)} | 0.2162336 | 0.03300309 | 0.1515475 | 0.2809197 | 0    |

With `nsplits=10` the cross-fitted estimate is essentially unchanged. That is the reassuring case: cross-fitting is insurance against overfitting bias, and when the nuisance models are not badly overfitted it costs nothing.

### 32.3.3. TMLE

We can use TMLE for ATE.

```
library(tmle)
TMLE<- tmle(Y=data$Y,A=data$A,W=W, family="binomial", Q.SL.library=SL.library, g.SL.library=SL.library)
TMLE$estimates$ATE
```

```
$psi
[1] 0.2117905
```

```
$var.psi
[1] 0.001019485
```

```
$CI
[1] 0.1492101 0.2743709
```

```
$pvalue
[1] 3.287231e-11
```

```
$bs.var
[1] NA
```

```
$bs.CI.twosided
[1] NA NA
```

```
$bs.CI.onesided.lower
[1] -Inf NA
```

```
$bs.CI.onesided.upper
[1] NA Inf
```

TMLE gives 0.2118 with a 95% interval of [0.1492, 0.2744] — within 0.003 of both AIPW versions, as it should be, since all three solve the same efficient influence-function equation and differ only in how.

### 32.3.4. DoubleML

The R package DoubleML implements the double/debiased machine learning framework of Chernozhukov et al. (2018). It provides functionalities to estimate parameters in causal models based on machine learning methods. The double machine learning framework consist of three key ingredients: Neyman orthogonality, High-quality machine learning estimation and Sample splitting.

They consider a partially linear model:

$$y_i = \theta d_i + g_0(x_i) + \eta_i \quad (32.4)$$

$$d_i = m_0(x_i) + v_i \quad (32.5)$$

This model is quite general, except it does not allow interaction of  $d$  and  $x$ ; therefore no heterogeneous treatment effect across  $x$ . But “DoubleML” implements more than partially linear model, it actually allows for heterogeneous treatment effects, in models such as interactive regression model.

The basic idea of doubleML is to use any machine learning algorithm to estimate outcome equation ( $l_0(X) = E(Y|X)$ ) and treatment equation ( $m_0(X) = E(D|X)$ ). Then get the residuals, namely  $\hat{W} = Y - \hat{l}_0(X)$  and  $\hat{V} = D - \hat{m}_0(X)$ .

Then regress  $\hat{W}$  on  $\hat{V}$ . Based on FWL theorem, you get  $\hat{\theta}$ .

An important component here is to specify a Neyman-orthogonal score function. In the case of PLR, it can be the product of the two residuals:

$$\psi(W; \theta, \eta) = (Y - D\theta - g(X))(D - m(X)) \quad (32.6)$$

The estimators  $\hat{\theta}$  is to solve the equation that the sample mean of this score function being 0.

And the variance of this score function is used as the variance of the doubleML estimator’s variance.

```
library(DoubleML)
dml_data = DoubleMLData$new(data,
                             y_col = "Y",
                             d_cols = "A",
                             x_cols = c("w1", "w2", "w3", "w4"))
print(dml_data)
```

===== DoubleMLData Object =====

```
----- Data summary -----
Outcome variable: Y
Treatment variable(s): A
Covariates: w1, w2, w3, w4
Instrument(s):
Selection variable:
No. Observations: 1000
```

```
library(mlr3)
library(mlr3learners)
# suppress messages from mlr3 package during fitting
lgr::get_logger("mlr3")$set_threshold("warn")
```

```

learner = lrn("regr.ranger", num.trees=500, max.depth=5, min.node.size=2)
ml_g = learner$clone()
ml_m = learner$clone()

learner = lrn("regr.glmnet")
ml_g_sim = learner$clone()
ml_m_sim = learner$clone()

set.seed(3141)
obj_dml_plr = DoubleMLPLR$new(dml_data, ml_g=ml_g, ml_m=ml_m)
obj_dml_plr$fit()
print(obj_dml_plr)

```

```
===== DoubleMLPLR Object =====
```

```
----- Data summary -----
```

```

Outcome variable: Y
Treatment variable(s): A
Covariates: w1, w2, w3, w4
Instrument(s):
Selection variable:
No. Observations: 1000

```

```
----- Score & algorithm -----
```

```

Score function: partialling out
DML algorithm: dml2

```

```
----- Machine learner -----
```

```

ml_l: regr.ranger
ml_m: regr.ranger

```

```
----- Resampling -----
```

```

No. folds: 5
No. repeated sample splits: 1
Apply cross-fitting: TRUE

```

```
----- Fit summary -----
```

```

Estimates and significance testing of the effect of target variables
Estimate. Std. Error t value Pr(>|t|)
A 0.23271 0.03224 7.219 5.25e-13 ***
---

```

```
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Note: this example (and the EAGeR example below) both have a **binary** treatment **A** and outcome **Y**, but `DoubleMLPLR` assumes a partially linear model with a single constant effect **theta** and continuous-style regression learners (`ml_g`, `ml_m`) for both nuisance functions. The more appropriate `DoubleML`

estimator for a binary treatment/outcome is the Interactive Regression Model, `DoubleMLIRM`, which uses classification learners for the propensity score and allows heterogeneous treatment effects. `DoubleMLPLR` will still run and often gives a reasonable ATE-like summary, but `DoubleMLIRM` is the estimator actually designed for this setting.

## 32.4. EAGeR trial simulation (AIPW package)

This simulation is based on a package “AIPW”, which has a simulation data “`eager_sim_rct`”.  $A$  denotes the binary treatment assignment,  $Y$  is the binary outcome, and  $W_g$  represents the covariate that affects the treatment assignment, which in our case was deemed to be the eligibility stratum indicator, sampled with replacement from the EAGeR Trial. Similarly,  $W_Q$  is a set of baseline prognostic covariates, which were also sampled with replacement from the EAGeR Trial, and includes the number of prior pregnancy losses, age, number of months of trying to conceive prior to randomization, body mass index (weight (kg)/height (m)<sup>2</sup>), and mean arterial blood pressure (denoted  $W1...5$  , respectively).

Data and the true risk difference. Unlike the simulation above, this is a resampled trial: 1,228 observations with a binary treatment and outcome, and the package records the risk difference used to generate it, so we again have a target to check against.

```
library(AIPW)
data("eager_sim_rct")
glimpse(eager_sim_rct)
```

```
Rows: 1,228
Columns: 8
$ sim_Y      <int> 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0~
$ sim_Tx     <int> 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0~
$ eligibility <dbl> 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0~
$ loss_num   <dbl> 2, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 2, 2, 1~
$ age        <dbl> 21.93, 30.65, 30.96, 32.24, 31.49, 27.99, 31.60, 25.~
$ time_try_pregnant <dbl> 2, 1, 0, 9, 1, 10, 8, 6, 1, 0, 1, 1, 3, 3, 0, 10, 10~
$ BMI        <dbl> 19.51963, 25.56474, 20.75446, 21.65728, 24.27480, 19~
$ meanAP     <dbl> 73.33333, 79.00000, 79.33333, 63.83333, 97.44444, 85~
```

```
attributes(eager_sim_rct)$true_rd
```

```
[1] 0.1324569
```

```
data <- eager_sim_rct %>%
  rename(Y=sim_Y, A=sim_Tx) %>%
  tibble()

W <- data %>%
  select(-Y, -A)
```

### 32.4.1. plug-in estimator from outcome regression

```
SL.outcome<- SuperLearner(Y=data$Y, X=data %>% select(-Y),
                        SL.library=SL.library, family="binomial")
SL.outcome.obs<- predict(SL.outcome, X=data %>% select(-Y))$pred
# predict the PO  $Y^1$ 
SL.outcome.1<- predict(SL.outcome, newdata=data %>% select(-Y) %>% mutate(A=1))$pred
# predict the PO  $Y^0$ 
SL.outcome.0<- predict(SL.outcome, newdata=data %>% select(-Y) %>% mutate(A=0))$pred
```

This is to do machine learning on the outcome equation; then get prediction on treated vs control, to get  $\hat{Y}^1$  and  $\hat{Y}^0$ . The sample average of the difference is our plug-in estimator.

```
SL.plugin<-mean(SL.outcome.1-SL.outcome.0)
SL.plugin
```

```
[1] 0.1185897
```

```
Q=data.frame(QAW=SL.outcome.obs, Q0W=SL.outcome.0, Q1W=SL.outcome.1)
```

The plug-in gives 0.1325 against the recorded true risk difference of 0.1186 — about 0.014 high.

### 32.4.2. AIPW with machine learning

```
set.seed(123)
SL.g<- SuperLearner(Y=data$A, X=data %>% select(-Y, -A),
                  SL.library=SL.library, family="binomial")
g1W <- SL.g$SL.predict
g0W<- 1- g1W
IF.1<-((data$A/g1W)*(data$Y-Q$Q1W)+Q$Q1W)
IF.0<-(((1-data$A)/g0W)*(data$Y-Q$Q0W)+Q$Q0W)
IF<-IF.1-IF.0
aipw.1<-mean(IF.1);aipw.0<-mean(IF.0)
aipw.manual<-aipw.1-aipw.0
ci.lb<-mean(IF)-qnorm(.975)*sd(IF)/sqrt(length(IF))
ci.ub<-mean(IF)+qnorm(.975)*sd(IF)/sqrt(length(IF))
res.manual.aipw<-c(aipw.manual,ci.lb, ci.ub)
res.manual.aipw
```

```
[1] 0.13103847 0.06938821 0.19268873
```

Now we use “npcausal”.



```
library(npcausal)
aipw<- ate(y=data$Y, a=data$A, x=W, nsplits=1, sl.lib=SL.library)
```

```

|
|
|
|=====| 0%
|
|=====| 25%
|
|=====| 50%
|
|=====| 75%
|
|=====| 100%
parameter      est      se      ci.ll      ci.ul pval
1    E{Y(0)} 0.4455985 0.02240994 0.40167501 0.4895220 0
2    E{Y(1)} 0.5753483 0.02105948 0.53407176 0.6166249 0
3 E{Y(1)-Y(0)} 0.1297499 0.03059579 0.06978212 0.1897176 0

```

The results from “npcausal” are almost the same as the manual results. Two clarifications. First, the manual estimator above *is* bias-corrected in the relevant sense: AIPW is exactly the one-step correction of the plug-in estimator using the efficient influence function. What we did *not* do is **cross-fitting** (sample splitting), which is a separate device. Second, that matters for inference: because the manual example fits the SuperLearner nuisances on the *full* sample and then uses  $\text{sd}(\text{IF})/\sqrt{n}$ , the resulting confidence interval is only valid under empirical-process (Donsker) conditions that flexible ML learners typically violate. With cross-fitting, the influence-function variance is valid without those conditions. “npcausal” does the cross-fitting for you when you set `nsplits > 1` (here `nsplits = 1` turns it off, for comparison with the manual full-sample version).

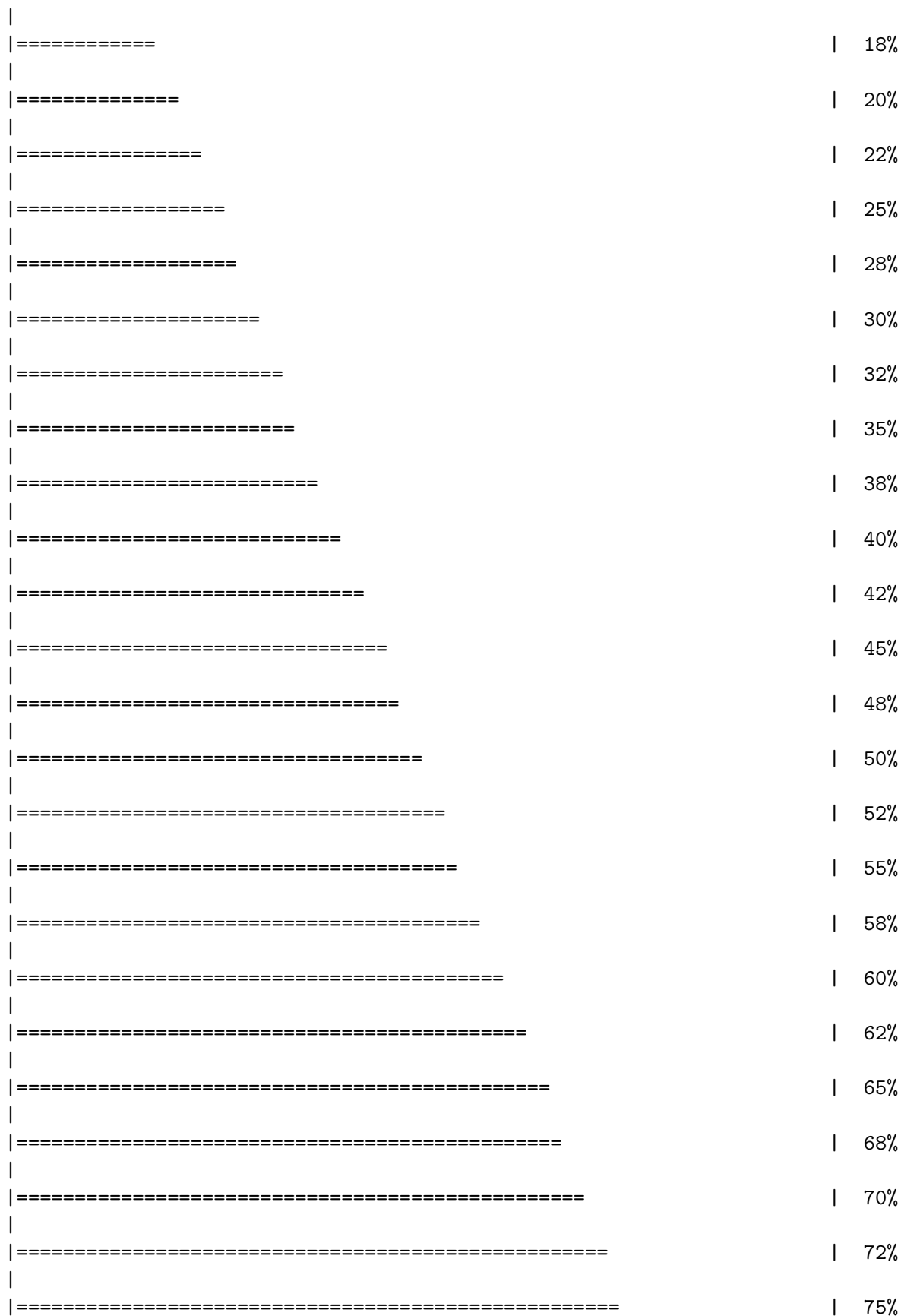
```
aipw2<- ate(y=data$Y, a=data$A, x=W, nsplits=10)
```

```

|
|
|
|==| 0%
|
|====| 2%
|
|=====| 5%
|
|=====| 8%
|
|=====| 10%
|
|=====| 12%
|
|=====| 15%

```

### 32. Recent causal inference tools



```

|
|=====| 78%
|
|=====| 80%
|
|=====| 82%
|
|=====| 85%
|
|=====| 88%
|
|=====| 90%
|
|=====| 92%
|
|=====| 95%
|
|=====| 98%
|
|=====| 100%

```

|   | parameter    | est       | se         | ci.ll      | ci.ul     | pval |
|---|--------------|-----------|------------|------------|-----------|------|
| 1 | E{Y(0)}      | 0.4448128 | 0.02529634 | 0.39523200 | 0.4943936 | 0    |
| 2 | E{Y(1)}      | 0.5775070 | 0.02255959 | 0.53329021 | 0.6217238 | 0    |
| 3 | E{Y(1)-Y(0)} | 0.1326942 | 0.03377055 | 0.06650392 | 0.1988845 | 0    |

### 32.4.3. DoubleML

```

library(DoubleML)
dml_data = DoubleMLData$new(as.data.frame(data),
                             y_col = "Y",
                             d_cols = "A",
                             x_cols = c("eligibility" , "loss_num","age", "time_try_pregnant",
print(dml_data)

```

===== DoubleMLData Object =====

```

----- Data summary -----
Outcome variable: Y
Treatment variable(s): A
Covariates: eligibility, loss_num, age, time_try_pregnant, BMI, meanAP
Instrument(s):
Selection variable:
No. Observations: 1228

```

## 32. Recent causal inference tools

```
library(mlr3)
library(mlr3learners)
# suppress messages from mlr3 package during fitting
lgr::get_logger("mlr3")$set_threshold("warn")

learner = lrn("regr.ranger", num.trees=500, max.depth=5, min.node.size=2)
ml_g = learner$clone()
ml_m = learner$clone()

learner = lrn("regr.glmnet")
ml_g_sim = learner$clone()
ml_m_sim = learner$clone()

set.seed(3141)
obj_dml_plr = DoubleMLPLR$new(dml_data, ml_g=ml_g, ml_m=ml_m)
obj_dml_plr$fit()
print(obj_dml_plr)
```

===== DoubleMLPLR Object =====

```
----- Data summary -----
Outcome variable: Y
Treatment variable(s): A
Covariates: eligibility, loss_num, age, time_try_pregnant, BMI, meanAP
Instrument(s):
Selection variable:
No. Observations: 1228

----- Score & algorithm -----
Score function: partialling out
DML algorithm: dml2

----- Machine learner -----
ml_l: regr.ranger
ml_m: regr.ranger

----- Resampling -----
No. folds: 5
No. repeated sample splits: 1
Apply cross-fitting: TRUE

----- Fit summary -----
Estimates and significance testing of the effect of target variables
  Estimate. Std. Error t value Pr(>|t|)
A   0.12505    0.03292   3.798 0.000146 ***
---
```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Note: as with the earlier PLR example, **A** and **Y** here are **binary** (this is the EAGeR trial data), so **DoubleMLIRM** (which uses classification learners for the propensity score and allows heterogeneous effects) would be the more appropriate DoubleML estimator than **DoubleMLPLR**'s partially linear, constant-effect model — see the note above.

---

*Systematic treatment:* [R](#) · [Julia](#).

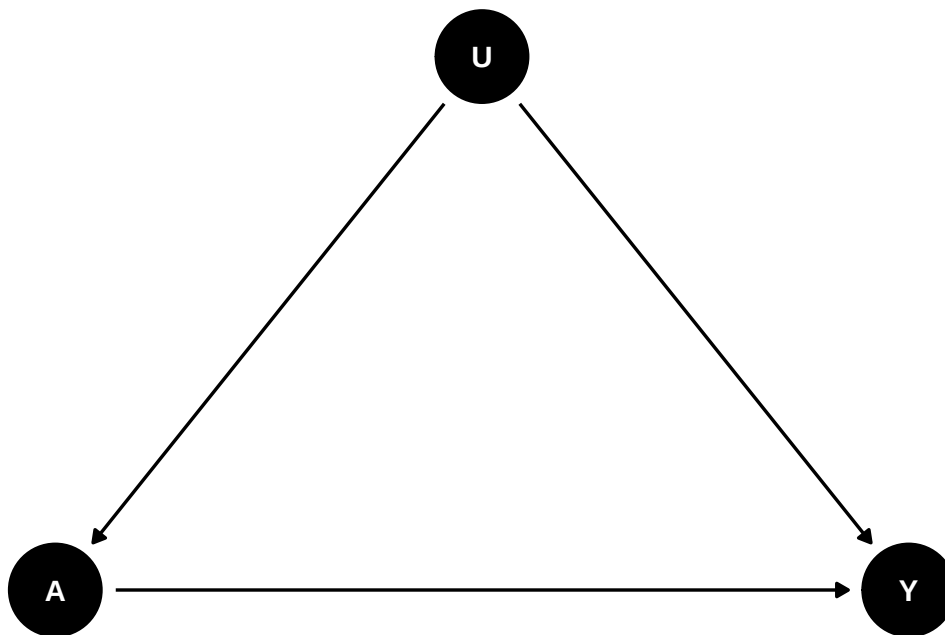


## 33. Intro to proximal causal inference

This chapter follows Tchetgen Tchetgen and co-authors, drawing primarily on Xu Shi's talk on proximal causal inference.

### 33.1. Unobserved confounders

The central problem for observational studies is unobserved confounding. Even after conditioning on observed covariates  $X$ , an unobserved confounder  $U$  may affect both treatment  $A$  and outcome  $Y$ :

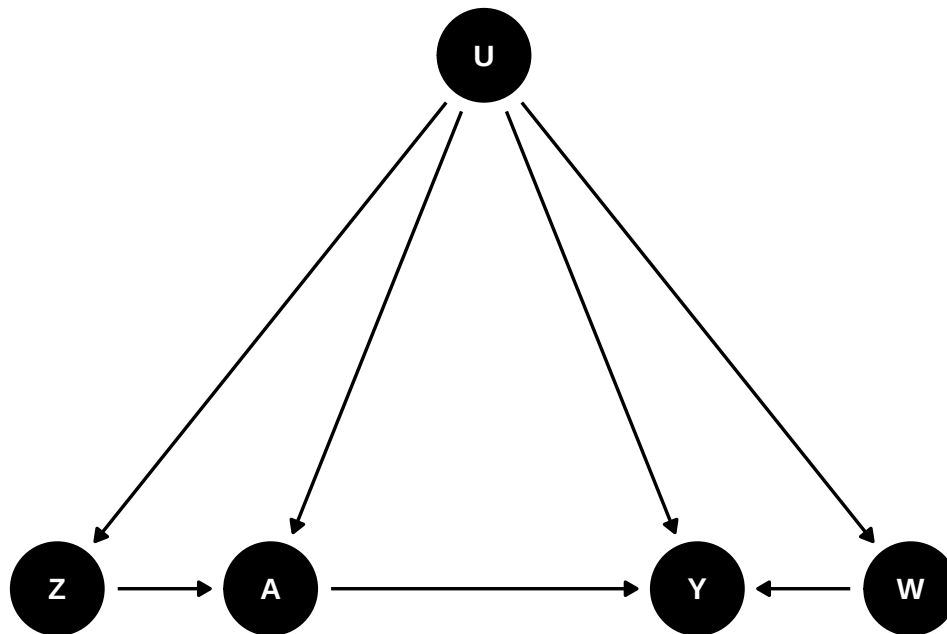


In this case, we have biased estimates of the treatment effect, regressing  $Y$  on  $A$  controlling for  $X$ .

### 33.2. Proximal causal inference

Proximal causal inference (also called negative controls) is a framework that allows us to estimate the causal effect of treatment  $A$  on outcome  $Y$  while accounting for unobserved confounders.

We'll need a DAG like this:



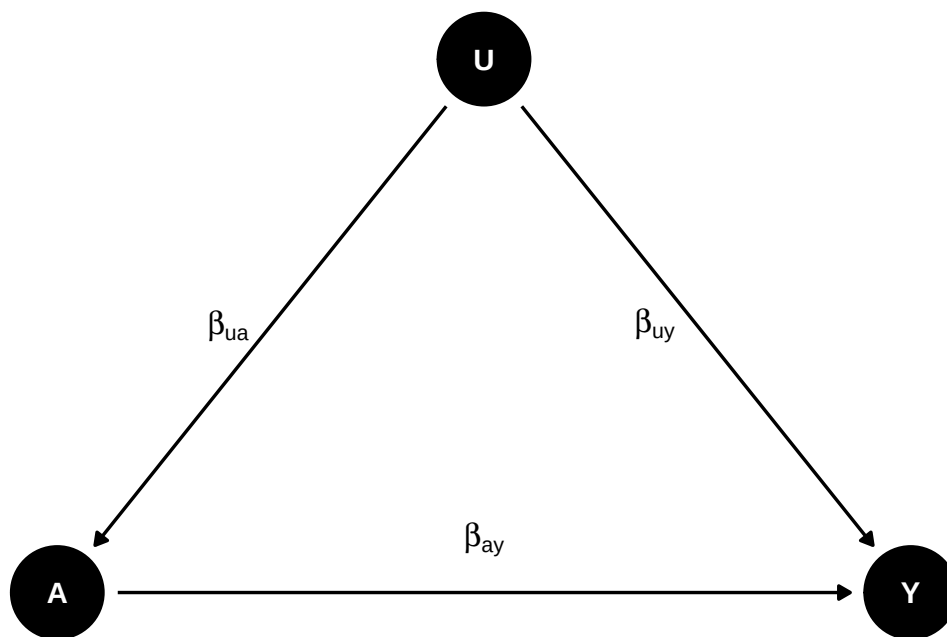
Basically we need two proxies for  $U$  to control for confounding.  $Z$  is called Negative Control for Exposure (NCE) and  $W$  is called Negative Control for Outcome (NCO). There is no link from  $Z$  to  $Y$ , and no link from  $W$  to  $A$  or  $Z$ .

$Z$  is an NCE if  $Y(a, z) = Y(a)$  and  $Z \perp Y(a)|U$ , meaning that  $Z$  satisfies the exclusion restriction that there is no direct link from  $Z$  to  $Y$ , and that  $Z$  is independent of  $Y$  given  $U$ .

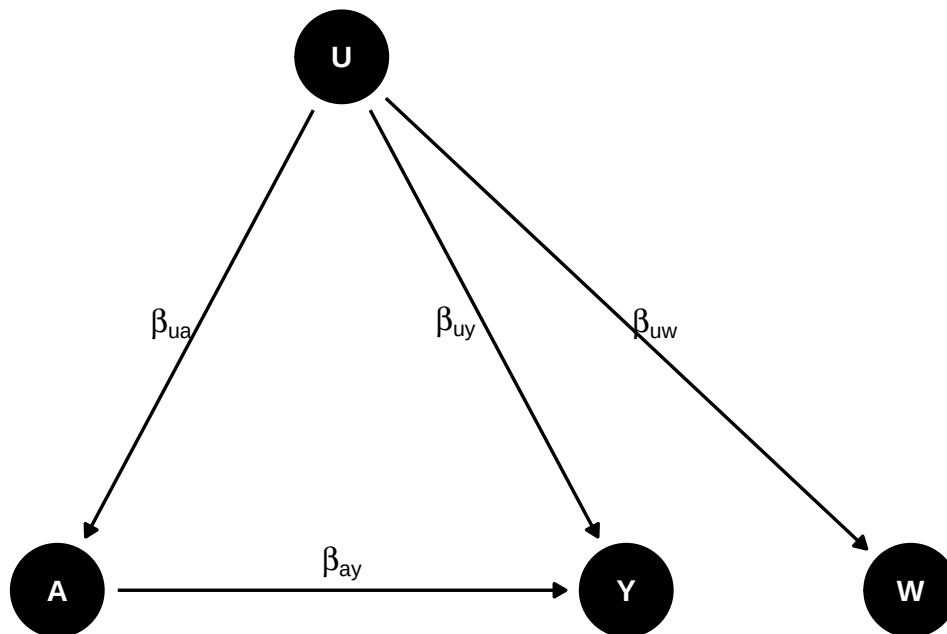
$W$  is an NCO if  $W(a, z) = W$  and  $W \perp (A, Z)|U$ , meaning that  $A$  and  $Z$  have no direct link to  $W$ .



## 33.2.1. Why this works



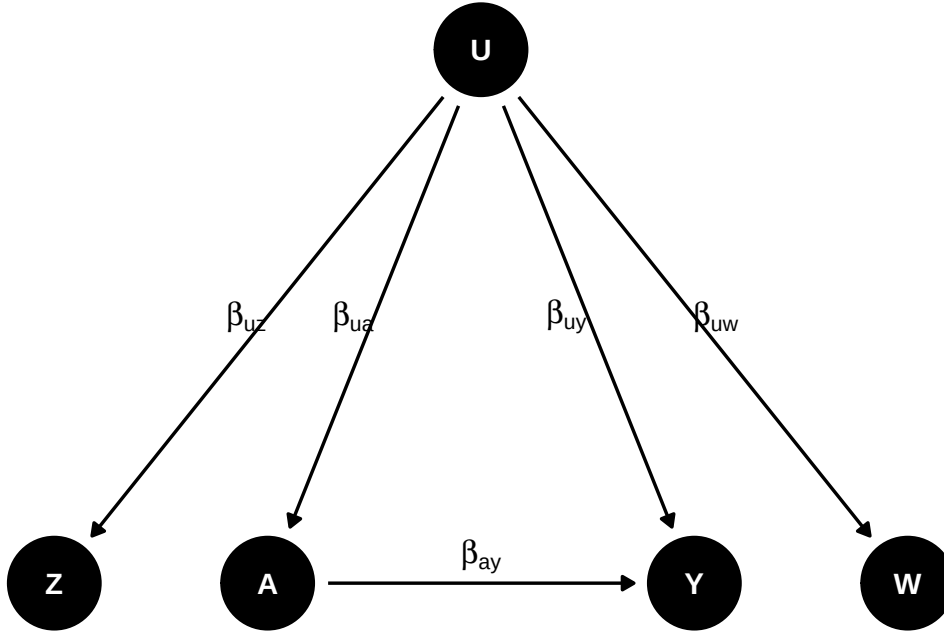
If we regress  $Y$  on  $A$ , then we get  $\beta_{ay} + \beta_{uy}\beta_{ua}$ , which is biased. How do we recover  $\beta_{ay}$ ?



Suppose we have  $W$  as proxy of  $U$  and  $\beta_{uw} = \beta_{uy}$ , then we can get the true effect. We can do regression of  $Y$  on  $A$ , and then  $W$  on  $A$ . The first regression gives us  $\beta_{ay} + \beta_{uy}\beta_{ua}$ , and the second regression gives us  $\beta_{uw}\beta_{ua}$ . We can then get  $\beta_{ay}$  from the difference of the two coefficients. This is similar to a DiD setting. Say  $W$  is  $Y_{t-1}$ , and assume the effect of  $U$  is the same on  $Y$  and  $Y_{t-1}$ , then effect of  $A$  on  $Y$  by the difference of regressing  $Y$  on  $A$  and  $Y_{t-1}$  on  $A$ .

### 33. Intro to proximal causal inference

However, most likely  $\beta_{uw} \neq \beta_{uy}$ . Then we need another proxy  $Z$  of  $U$  to recover the true effect. This is also called “double proxy”, or “double negative control”.



#### 33.2.2. A linear model

Suppose we have the following linear model:

$$E[Y|A, Z, U] = \beta_{y0} + \beta_{ay}A + \beta_{uy}U \quad (33.1)$$

$$E[W|A, Z, U] = \beta_{w0} + \beta_{uw}U \quad (33.2)$$

$$E[U|A, Z] = \beta_{u0} + \beta_{zu}Z + \beta_{au}A \quad (33.3)$$

$\beta_{ay}$  is the true effect of  $A$  on  $Y$ .

$$E[Y|A, Z] = \beta_{y0} + \beta_{ay}A + \beta_{uy}(\beta_{u0} + \beta_{zu}Z + \beta_{au}A) \quad (33.4)$$

$$E[W|A, Z] = \beta_{w0} + \beta_{uw}(\beta_{u0} + \beta_{zu}Z + \beta_{au}A) \quad (33.5)$$

From the last two regressions we can recover  $\beta_{ay}$ :

If we define  $\delta_a^y = \beta_{ay} + \beta_{uy}\beta_{au}$  (regression coefficient of  $A$  on the  $Y$  regression) and  $\delta_a^w = \beta_{uw}\beta_{au}$ , (regression coefficient of  $A$  on the  $W$  regression), similarly for  $\delta_z^y$  and  $\delta_z^w$ , then we have

$$\beta_{ay} = \delta_a^y - \delta_a^w \frac{\delta_z^y}{\delta_z^w} \quad (33.6)$$

In this **linear-Gaussian special case only**, we can look at this as a two-stage-least-squares-style regression and replace  $U$  with the predicted value of  $W$ :

$$E[Y|A, Z] = \left( \beta_{y0} - \frac{\beta_{uy}}{\beta_{uw}} \beta_{w0} \right) + \beta_{ay}A + \frac{\beta_{uy}}{\beta_{uw}} E[W|A, Z] \quad (33.7)$$

since  $E[U | A, Z] = (E[W | A, Z] - \beta_{w0})/\beta_{uw}$  from the  $W$  equation above. The coefficient on the predicted value of  $W$  is  $\beta_{uy}/\beta_{uw}$ , not  $\beta_{uy}$  alone — it only reduces to  $\beta_{uy}$  when  $\beta_{uw} = 1$ . So we first regress  $W$  on  $A$  and  $Z$ , then regress  $Y$  on  $A$  and the predicted value of  $W$ .

This “predict  $W$  from  $A$  and  $Z$ , then plug it in” shortcut is **not** the general proximal identification argument, and it is not the same as a standard IV/2SLS specification (we are not instrumenting an endogenous regressor with `ivreg`). In general proximal causal inference, the outcome bridge function  $h(A, W)$  is identified by solving the conditional moment restriction  $E[Y | A, Z] = E[h(A, W) | A, Z]$ , which requires completeness (relevance) conditions linking the treatment-inducing proxy  $Z$  and the outcome-inducing proxy  $W$  to the hidden confounder  $U$ . The generated regressor  $S = \hat{E}[W | A, Z]$  is only a valid substitute under the linear-Gaussian structure above; even then the two-stage standard errors are wrong as written, because  $S$  is generated in the first stage (bootstrap the whole procedure).

### 33.2.3. A nonparametric model

We don’t have to have a linear model, this can be done with a nonparametric model as well.

$$E[Y(a)] = E[h(a, W)] \quad (33.8)$$

where  $E[Y|A, Z] = E[h(A, W)|A, Z]$ .

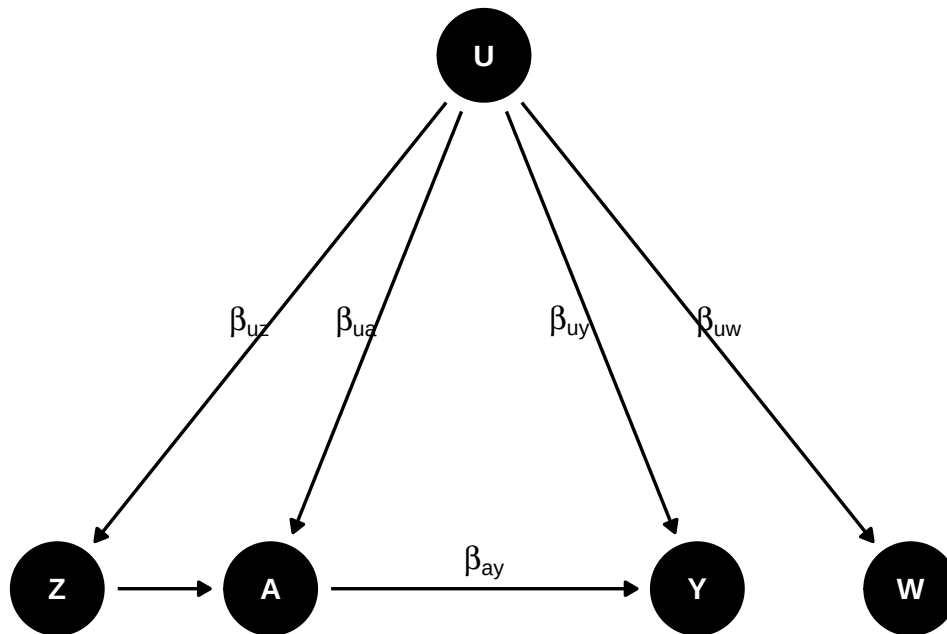
That is, we can estimate  $h$  as a function of  $A$  and  $W$ , identified using  $Z$ , nonparametrically. This  $h$  function is called outcome bridge function. In the linear case, it is  $h(A, W) = \beta_0 + \beta_a A + \beta_w W$ .

### 33.2.4. How to find proximal variables

Tchetgen Tchetgen and co-authors come up with a Data-driven Automated Negative Control Estimation (DANCE) algorithm to find negative controls. The idea is to use a machine learning algorithm to find the variables that are correlated with the treatment and the outcome, but not with the treatment and the outcome together. This is similar to the idea of finding instrumental variables.

## 33.3. example

Let’s generate data based on this DAG:



```

set.seed(123)
n <- 5000
# Set coefficients
b_uw <- 0.5 # effect of U on W
b_uy <- 0.5 # effect of U on Y
b_ay <- 0.5 # effect of A on Y
b_uz <- 0.5 # effect of U on Z
b_ua <- 0.5 # effect of U on A
b_z_a <- 0.5 # effect of Z on A

U <- rnorm(n)
W <- b_uw * U + rnorm(n) # W is a function of U
Z <- b_uz * U + rnorm(n) # Z is a function of U
A <- b_ua * U + b_z_a * Z + rnorm(n) # A is a function of U and Z
Y <- b_uy * U + b_ay * A + rnorm(n) # Y is a function of U and A
# A <- rbinom(n, 1, plogis(-1 + U / 2)) # A is a binary treatment

# Create a data frame with the generated variables

data <- data.frame(W = W, Z = Z, A = A, Y = Y)

```

The simulation sets  $n = 5000$  and every coefficient to 0.5. The hidden confounder is  $U \sim N(0, 1)$ ; the two proxies are  $W = 0.5U + \varepsilon_W$  and  $Z = 0.5U + \varepsilon_Z$ ; treatment is  $A = 0.5U + 0.5Z + \varepsilon_A$ ; and the outcome is  $Y = 0.5U + 0.5A + \varepsilon_Y$ , all errors standard normal. The true effect of  $A$  on  $Y$  is therefore 0.5.

Note the roles:  $Z$  affects  $A$  but not  $Y$  directly,  $W$  is affected by  $U$  but affects nothing, and neither is

a valid control for  $U$  on its own.

If we run regression of  $Y$  on  $A$ ,  $Z$ ,  $W$ ,

```
m1 <- lm(Y ~ A + Z + W, data = data)
summary(m1)
```

Call:

```
lm(formula = Y ~ A + Z + W, data = data)
```

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -4.5836 | -0.7167 | 0.0005 | 0.7305 | 4.4866 |

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )     |
|-------------|-----------|------------|---------|--------------|
| (Intercept) | -0.006347 | 0.015279   | -0.415  | 0.678        |
| A           | 0.660021  | 0.014123   | 46.734  | < 2e-16 ***  |
| Z           | 0.085004  | 0.016650   | 5.105   | 3.43e-07 *** |
| W           | 0.132798  | 0.014165   | 9.375   | < 2e-16 ***  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.08 on 4996 degrees of freedom

Multiple R-squared: 0.4556, Adjusted R-squared: 0.4553

F-statistic: 1394 on 3 and 4996 DF, p-value: < 2.2e-16

We don't get the true effect of  $A$  on  $Y$ : the coefficient on  $A$  is 0.660 against a true 0.5, a bias of 32%. Conditioning on both proxies does not remove the confounding, because neither  $W$  nor  $Z$  is  $U$  — each is a noisy reflection of it, and controlling for a noisy proxy leaves residual confounding. The proxies do carry signal (both are significant, at 0.085 and 0.133), which is what makes this failure mode deceptive: the regression looks like it is adjusting for the confounder.

If we do the 2sls using  $W$  and  $Z$ , then we get the true effect.

```
# Linear-Gaussian (Y, W) toy case ONLY. This two-stage plug-in identifies
# the effect under the linear structure above; it is not the general
# proximal estimator, and the reported SEs below are NOT valid because S is
# a generated regressor (bootstrap the whole two-step procedure for inference).
# Perform the first-stage linear regression W ~ A + Z
m1 <- lm(W ~ A + Z, data = data)
# Compute the estimated E[W|A,Z]
S <- m1$fitted.values
# Perform the second-stage linear regression Y ~ A + E[W|A,Z]
m2 <- lm(Y ~ A + S, data = data)
summary(m2) # point estimate OK under the linear model; SEs need bootstrap
```

### 33. Intro to proximal causal inference

Call:

```
lm(formula = Y ~ A + S, data = data)
```

Residuals:

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -4.5477 | -0.7309 | 0.0059 | 0.7395 | 4.3715 |

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )     |
|-------------|-----------|------------|---------|--------------|
| (Intercept) | -0.005049 | 0.015408   | -0.328  | 0.743        |
| A           | 0.489342  | 0.043210   | 11.325  | < 2e-16 ***  |
| S           | 1.143480  | 0.199171   | 5.741   | 9.96e-09 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.089 on 4997 degrees of freedom

Multiple R-squared: 0.446, Adjusted R-squared: 0.4458

F-statistic: 2012 on 2 and 4997 DF, p-value: < 2.2e-16

The two-stage estimate is 0.489, against the true 0.5 and the naive 0.660. Using the two proxies in the right way — one to build a predicted  $E[W | A, Z]$  and that prediction as the control — recovers the effect, where using both directly as regressors did not.

The reported standard error of 0.043 should be ignored, for the reason flagged in the code comment:  $S$  is a generated regressor, so the second-stage standard errors are wrong as written and need a bootstrap over both stages.

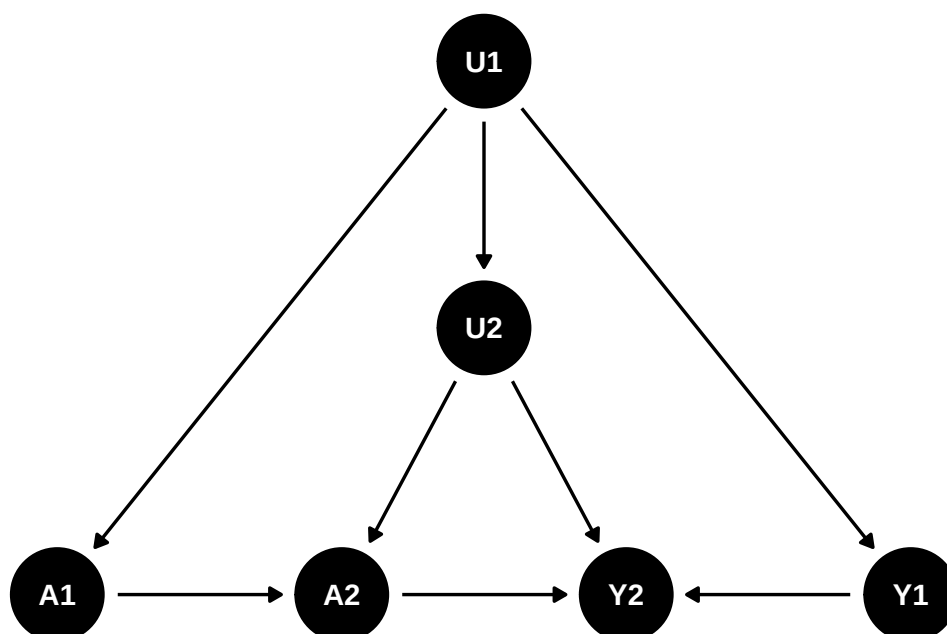
---

*Systematic treatment:* [R](#) · [Julia](#).

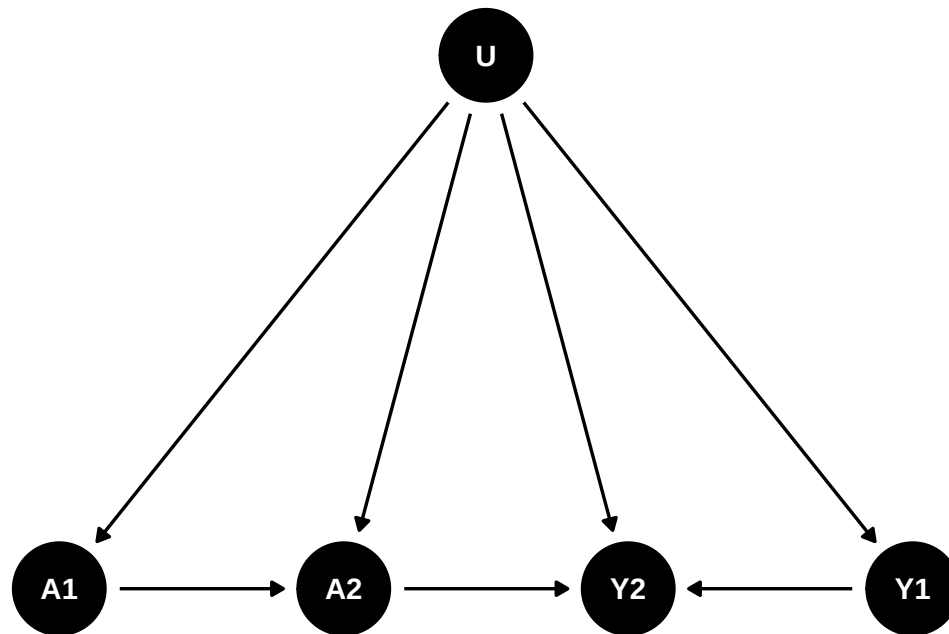
## 34. Simulation on proximal causal inference with panel data

This chapter extends **proximal causal inference** to a panel-data setting. The question is whether lagged treatment and lagged outcome can serve as negative control exposure (NCE) and negative control outcome (NCO), respectively, to account for unobserved confounders.

Consider a two-period DAG:



Suppose that's the case for the causal structure, then I can reasonably combine U1 and U2 into a single unobserved confounder  $U$ :



That would fit into the double negative control framework. The problem is that we have to assume there is no link from A1 to Y1, which is odd. Can we assume in period 1 there is no effect, but in period 2, there is an effect?

### 34.1. A simulation with panel data

I generate two period data based on the DAG above.

```

library(dplyr)
library(rootSolve)
library(tidyverse)

gen_data <- function(seed, n){
  set.seed(seed)
  u <- rnorm(n, mean = 0, sd = 1) # unobserved confounder
  x1 <- rnorm(n, mean = 0, sd = 1) # observed confounder
  x2 <- rnorm(n, mean = 0, sd = 1) # observed confounder
  a1 <- rbinom(n, 1, plogis(2*u + 1.5*x1)) # treatment at time 1
  a2 <- rbinom(n, 1, plogis(u + a1 + x1)) # treatment at time 2
  y1 <- 1.5*u + .5*x1 + rnorm(n, mean = 0, sd = 1) # outcome at time 1
  y2 <- y1 + 2.5*u + 2.5*x1 + 3*a2 + rnorm(n, mean = 0, sd = 1) # outcome at time 2
  data <- data.frame(u=u, x1=x1, x2=x2, a1=a1, a2=a2, y1=y1, y2=y2)
  return(data)
}

data <- gen_data(333, 100000)
  
```



```
data_sim <- data |>
  select(a1, a2, y2, x1, x2, y1) |>
  rename(a = a2, y = y2, x = x1, w = y1, z=a1)
```

The simulation draws  $n = 100,000$  two-period observations. There is an unobserved confounder  $u \sim N(0, 1)$  and two observed covariates  $x_1, x_2 \sim N(0, 1)$ . Treatment at time 1 is  $a_1 \sim \text{Bernoulli}(\Lambda(2u + 1.5x_1))$  and at time 2 is  $a_2 \sim \text{Bernoulli}(\Lambda(u + a_1 + x_1))$ . The outcomes are  $y_1 = 1.5u + 0.5x_1 + \varepsilon_1$  and  $y_2 = y_1 + 2.5u + 2.5x_1 + 3a_2 + \varepsilon_2$ .

So the effect of interest is the 3 on  $a_2$ . The construction then relabels  $a_1$  as the negative control exposure  $z$  and  $y_1$  as the negative control outcome  $w$  — note that  $y_1$  does not depend on  $a_1$ , which is exactly the assumption discussed at the end of this chapter.

We can use the simple 2sls:

```
lm1 <- lm(w ~ a + x + z, data = data_sim)
data_sim$what <- lm1$fitted.values
lm2 <- lm(y ~ a + x + what, data = data_sim)
summary(lm2)
```

Call:

```
lm(formula = y ~ a + x + what, data = data_sim)
```

Residuals:

|  | Min      | 1Q      | Median | 3Q     | Max     |
|--|----------|---------|--------|--------|---------|
|  | -16.4284 | -2.3083 | 0.0071 | 2.3020 | 20.7590 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )   |
|-------------|----------|------------|---------|------------|
| (Intercept) | -0.01293 | 0.02315    | -0.558  | 0.577      |
| a           | 3.02113  | 0.03528    | 85.625  | <2e-16 *** |
| x           | 1.67203  | 0.01245    | 134.327 | <2e-16 *** |
| what        | 2.66578  | 0.01747    | 152.631 | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.456 on 99996 degrees of freedom

Multiple R-squared: 0.6822, Adjusted R-squared: 0.6822

F-statistic: 7.155e+04 on 3 and 99996 DF, p-value: < 2.2e-16

The 2SLS coefficient on  $a$  is 3.021 with a standard error of 0.035, against the true 3. The coefficient on the fitted proxy  $what$  is 2.666, which is the ratio of the confounder's loading on the outcome to its loading on the proxy, not an effect of interest.

Or we can use more sophisticated methods. Here I use the m-estimation code by Paul Zivich: [https://github.com/pzivich/publications-code/blob/master/ProximalCI/proximal\\_sims\\_r.R](https://github.com/pzivich/publications-code/blob/master/ProximalCI/proximal_sims_r.R). This code is to construct estimating equations, then use root finding to find the parameter estimates.

### 34. Simulation on proximal causal inference with panel data

```
pcl_m2 <- function(theta, mydata, out = "ef"){
  # read in variables
  a <- mydata$a
  x <- mydata$x
  w <- mydata$w
  z <- mydata$z
  y <- mydata$y
  # model for what
  what <- theta[1] + theta[2] * a + theta[3] * x + theta[4] * z

  # matrix of covars in model for what
  wmat <- as.matrix(cbind(rep(1, nrow(mydata)), a, x, z), ncol = 4,
                    nrow = nrow(mydata))

  # estimating function for what
  ef1 <- (t(wmat) %*% (w - what))          # gives px1 vector (summed over units)
  ef1_b <- wmat * (w - what)              # gives nxp matrix (rows are units)

  # model for yhat
  yhat <- theta[5] + theta[6]*a + theta[7]*x + theta[8] * what

  # matrix of covars in model for yhat
  ymat <- as.matrix(cbind(rep(1, nrow(mydata)), a, x, what), ncol = 4,
                    nrow = nrow(mydata))

  #estimating function for yhat
  ef2 <- t(ymat) %*% (y - yhat)            # px1 vector, summed over units
  ef2_b <- (ymat) * (y - yhat)            # nxp matrix (rows are units)
  f <- c(ef1, ef2)                        # p-dimensional vector
  if(out == "full") return(cbind(ef1_b, ef2_b)) else return(f) # full used in var calculation
}

# call m-estimator for PCL
mpcl_mest <- function(dat){
  theta_start <- rep(.05, 8)
  results <- mestimator(pcl_m2, init = theta_start, dat, mydata = dat)
  return(results)
}

##' M-estimation engine
mestimator <- function(ef, init, dat, ...){
  # get estimated coefficients
  fit <- multiroot(ef, start = init, out = "ef",
                  rtol = 1e-9, atol = 1e-12, ctol = 1e-12, ... )
  betahat <- fit$root
  n <- nrow(dat)
}
```

```

# bread
# derivative of estimating function at betahat
pd <- gradient(ef, betahat, out = "ef", ...)
pd <- as.matrix(pd)
bread <- -pd/n
se_mod <- sqrt(abs(diag(-solve(pd))))

# meat1
efhat <- ef(betahat, out = "full", ...)
meat <- list()
meat1 <- (t(efhat) %*% efhat)/n

# sandwich
sandwich1 <- (solve(bread) %*% meat1 %*% t(solve(bread)))/n
se_sandwich1 <- sqrt(diag(sandwich1))

results <- data.frame(betahat, se_sandwich1, se_mod)
return(results)
}

rmpl <- unname(mpl_mest(data_sim)[6, c(1:2)])
rmpl

```

```
6 3.021126 0.0198433
```

```
#mpl_mest(data_sim)
```

The m-estimator gives 3.0211 with a standard error of 0.0198.

Both the 2sls and the m-estimation give us the same estimate of the causal effect of treatment on outcome, which is 3 — identical to four decimal places, 3.021126 either way. The standard errors differ, 0.0198 against 0.0353, and the m-estimator's is the one to use: it accounts for the estimation of the first stage, which the plug-in 2SLS standard error does not. The m estimator can be more general. 2sls is good for the linear case.

However, the main issue is the assumption that there is no link from  $A_1$  to  $Y_1$ .

To think about it from a different angle: this is similar to DiD setting. Basically we need  $A_1$  to be 0. There is no effect in period 1. If we only have  $A_1$  as 0, then it makes sense to have no link from  $A_1$  to  $Y_1$  (no anticipation assumption).

**This is a restrictive identifying assumption, not a technical footnote.** Using  $A_{t-1}$  as the negative control exposure and  $Y_{t-1}$  as the negative control outcome only identifies the effect if two separate exclusions hold:  $A_1$  must have no direct effect on  $Y_1$ , so that  $Y_1$  is a clean proxy for  $U$  rather than a partly post-treatment quantity; and  $A_1$  must have no direct effect on  $Y_2$  except through  $U$ , which is the exclusion restriction that makes  $A_1$  admissible as a negative control exposure in the first place. If either link is actually present — for example, if treatment has an immediate effect

### 34. *Simulation on proximal causal inference with panel data*

in the same period it starts, which is the empirically common case — this proximal construction is misspecified and the estimate above should not be trusted. Before applying this approach, researchers should have a substantive reason to believe treatment effects only appear with a lag.

---

*Systematic treatment:* *R* · *Julia*.

## 35. longitudinal modified treatment policy (LMTP)

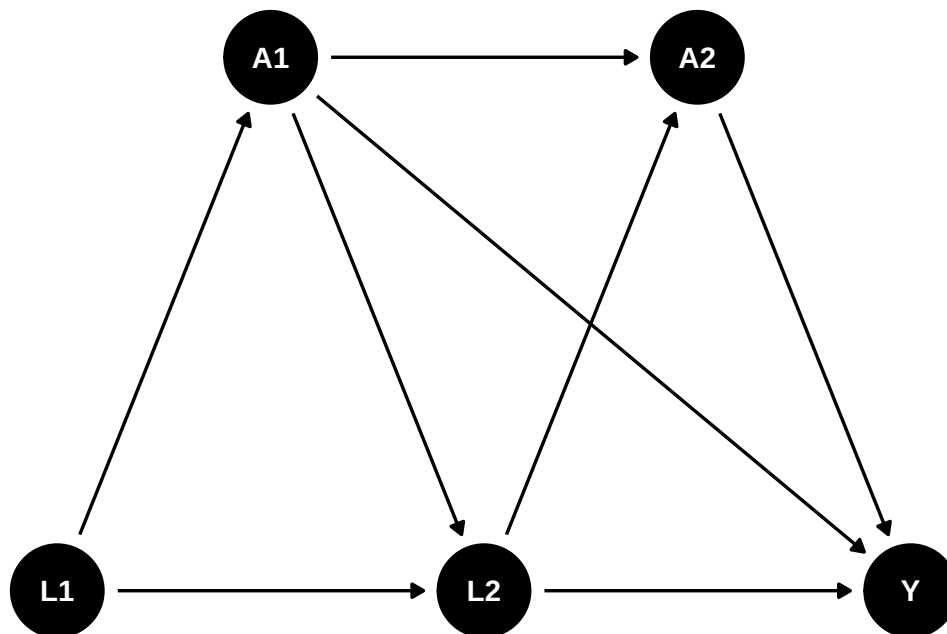
The longitudinal modified treatment policy (LMTP) framework, developed in epidemiology and biostatistics, handles settings where the standard ATE is poorly defined: continuous or multivalued treatments, time-varying exposures, or hypothetical interventions that would violate positivity. This chapter follows [Nicholas Williams](#).

Suppose we have such a DAG:

```
library(ggplot2)
library(ggdag)

g <- dagify(
  A1 ~ L1,
  L2 ~ L1,
  L2 ~ A1,
  A2 ~ A1,
  A2 ~ L2,
  Y ~ A2,
  Y ~ L2,
  Y ~ A1,
  exposure = "A2",
  outcome = "Y",
  coords = list(x = c(L1 = 1, L2 = 2, Y = 3, A1 = 1.5, A2 = 2.5),
                y = c(L1 = 1, L2 = 1, Y = 1, A1 = 1.5, A2 = 1.5))
)

ggdag(g) +
  theme_dag()
```



We have multiple time points, and the treatment  $A$  is time-varying. We are interested in not only the ATE of  $A_2$  on  $Y$ , but also some other hypothetical interventions.

## 35.1. Definitions and assumptions

### 35.1.1. Assumptions

1. Consistency. The observed outcome under the treatment actually received equals the corresponding potential outcome, and the shifted intervention is well defined — no hidden versions of it.
2. Sequential unconfoundedness. There are no unmeasured confounders of treatment at any time point given the history. The stronger *sequential randomization* condition conditions on the full history including past natural values of treatment; some estimators need that version and others do not, so it is worth knowing which one a given result assumes.
3. Positivity, in the form the shift requires. Roughly: if a unit with history  $h_t$  can receive  $a_t$ , then the shifted value  $d(a_t, h_t)$  must also have support given  $h_t$ . This is where modified treatment policies earn their keep. A static always-treat or never-treat rule can demand support the data does not have, while a bounded shift of the observed treatment stays inside it.
4. For continuous treatments, a smoothness condition. The influence-function estimators below assume the shift function is piecewise smooth and invertible in  $a_t$ , which is what lets the density of the shifted treatment be written in terms of the observed one.

The always-treat and never-treat demonstration below is, on that reading, an unusual lead example for this framework: it is the easiest case to follow, and also the case whose positivity is most likely to fail. A bounded continuous shift is the canonical LMTP application. Díaz et al., <https://arxiv.org/abs/2006.01366>, state the assumptions precisely.

### 35.1.2. dynamic treatment regime

Some notations commonly used in this literature: We observe  $Z = (L_0, A_0, L_1, A_1, Y)$ , where  $L$  is the observed confounder,  $A$  is the treatment, and  $Y$  is the outcome. The treatment  $A$  can be time-varying. We can also have multiple treatments at different time points. For example, we can have  $A_2, A_3$ , etc. In this graph, baseline  $L_0$  affects  $A_0$ , which in turn affects  $L_1$ , which then affects  $A_1$ , and finally  $A_1$  affects the outcome  $Y$ .

History  $H_t$  is the history of data up to time  $t$ , right before  $A_t$ . for example,  $H_1 = (L_0, A_0, L_1)$ , and  $H_2 = (L_0, A_0, L_1, A_1)$ .  $d$  is the hypothetical intervention function, or shift function, which is a function of the history  $H_t$  and the treatment  $A_t$ . For example,  $d_0(a_0, h_0, \epsilon_0)$  is a user-given function to map  $a_0, h_0$ , and  $\epsilon_0$  to a potential treatment value. The function  $d$  can be deterministic, or it can be stochastic. Then we can replace  $A_0$  with  $A_0^d = d_0(A_0, H_0, \epsilon_0)$ . Then after that  $A_1(A_0^d)$  is called the natural value of treatment.

This is very general, comparing to the static treatment regime.

Suppose treatment  $A$  is a function of the history of treatment and confounders, for example,  $A = d(A_1, L_1, A_2, L_2)$ .  $A$  can be set to a fixed value, say 1 or 0, or some value  $A^d$ . This function  $d$  can be anything, it can be taking a deterministic value, or it can be a function that takes the natural treatment value  $A$  as input. In the package “lmt”, this is called a shift function, or hypothetical intervention. For example,  $d$  can be set to 1 if  $age < 30$ , or  $d$  can be set to double the natural value of  $A$ . Many possibilities.

In comparison, for ATE, we only need to set  $A$  to 0 or 1.

Under this LMTP, the causal parameter is

$$\theta = E[Y^{\bar{A}^d}] \quad (35.1)$$

$Y^{\bar{A}^d}$  is the potential outcome under the hypothetical intervention  $\bar{A}^d$ . At time 1,  $A_1^d = d(A_1, H_1)$ .

### 35.1.3. modified treatment policy

A simulated data set makes the estimation concrete.

This simulation is from Susmann et al. (2024) “Longitudinal Generalizations of the Average Treatment Effect on the Treated for Multi-valued and Continuous Treatments”. I modified slightly to fit the DAG above.

```
library(tidyverse)
library(tidyr)

mtp <- function(data, trt) {
  a <- data[[trt]]
  a * 0 + 1
}

simulate_data <- function(seed, N, tau, sigma = 0.5) {
  set.seed(seed)
```

### 35. longitudinal modified treatment policy (LMTP)

```

data <- tibble(id = 1:N)

for(t in 1:tau) {
  Lt <- paste0("L_", t)
  Ltd <- paste0("L_", t, "d")

  At <- paste0("A_", t)
  Atd <- paste0("A_", t, "d")

  Lt1 <- paste0("L_", t - 1)
  Lt1d <- paste0("L_", t - 1, "d")

  At1 <- paste0("A_", t - 1)
  At1d <- paste0("A_", t - 1, "d")

  if(t == 1) {
    data[[Lt]] <- runif(N, 0, 1)
    data[[Ltd]] <- data[[Lt]]

    data[[At]] <- rbinom(N, size = 1, prob = 0.5)
  }
  else {
    # L_t depends on the previous treatment A_{t-1} (the DAG edge A1 -> L2),
    # which is the time-varying confounding this chapter is about.
    data[[Lt]] <- rnorm(N, mean = 0.25 * data[[Lt1]] + 0.4 * data[[At1]], 0.5)
    data[[Ltd]] <- rnorm(N, mean = 0.25 * data[[Lt1d]] + 0.4 * data[[At1d]], 0.5)

    data[[At]] <- rbinom(N, size = 1, prob = plogis(0.5 - 0.2 * data[[At1]] + 0.1 * data[[Lt1d]])
  }

  data[[Atd]] <- mtp(data, At)
}

# Y depends on the final treatment A_2 and confounder L_2, and also directly
# on the first-period treatment A_1 (the DAG edge A1 -> Y).
data$Y <- rnorm(N, data[[At]] + data[[Lt]] + 0.5 * data[["A_1"]], sigma)
data$Yd <- rnorm(N, data[[Atd]] + data[[Ltd]] + 0.5 * data[["A_1d"]], sigma)
data

}

simulated_data1 <- simulate_data(seed = 123, N = 10000, tau = 2)

mean(simulated_data1$Yd)

```

[1] 2.028593



```

mtp <- function(data, trt) {
  a <- data[[trt]]
  a * 0 + 0
}

simulated_data2 <- simulate_data(seed = 123, N = 10000, tau = 2)

mean(simulated_data2$Yd)

```

```
[1] 0.1285934
```

```
mean(simulated_data1$Yd) - mean(simulated_data2$Yd)
```

```
[1] 1.9
```

Note in this simulation the variables ending with “d” are the variables under hypothetical intervention, or modified treatment policy.  $L_1$  is from  $\text{Uniform}(0, 1)$ , and  $L_2$  is from  $N(0.25L_1 + 0.4A_1, 0.5)$  — so the time-varying confounder  $L_2$  depends on the earlier treatment  $A_1$ , which is the time-varying confounding this chapter is about. The treatment  $A_1$  is from a Bernoulli distribution with probability 0.5, and the treatment  $A_2$  is from a Bernoulli distribution with probability  $\text{plogis}(0.5 - 0.2A_1 + 0.1L_1)$ . The outcome  $Y$  is from a normal distribution with mean  $A_2 + L_2 + 0.5A_1$ , so  $Y$  depends on both the current treatment  $A_2$  and the lagged treatment  $A_1$ . In the code, the first modified treatment policy sets treatment to 1 in every period (`simulated_data1`) and the second sets it to 0 in every period (`simulated_data2`); the reported difference  $E[Y_{\text{always } 1}^d] - E[Y_{\text{always } 0}^d]$  is the effect of the always-treat versus never-treat static regimes.

The always-treat regime gives  $E[Y^d] = 2.029$ , never-treat gives 0.129, and their difference is 1.900 — and that 1.9 decomposes exactly into the three routes the DGP contains: 1 from  $A_2$  direct, 0.5 from  $A_1$  direct, and 0.4 through the  $A_1 \rightarrow L_2 \rightarrow Y$  path, since  $L_2$  shifts by 0.4 and enters  $Y$  with coefficient 1.

In this case, we can just do a linear regression to get the effect of  $A_2$  on  $Y$ , knowing the exact DAG.

```

m1 <- glm(Y ~ L_2 + A_1 + A_2, data = simulated_data1)
summary(m1)

```

Call:

```
glm(formula = Y ~ L_2 + A_1 + A_2, data = simulated_data1)
```

Coefficients:

|             | Estimate   | Std. Error | t value | Pr(> t )   |
|-------------|------------|------------|---------|------------|
| (Intercept) | -3.453e-05 | 9.615e-03  | -0.004  | 0.997      |
| L_2         | 9.998e-01  | 9.889e-03  | 101.103 | <2e-16 *** |
| A_1         | 4.940e-01  | 1.079e-02  | 45.779  | <2e-16 *** |

### 35. longitudinal modified treatment policy (LMTP)

```
A_2          1.000e+00  1.027e-02  97.380   <2e-16 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for gaussian family taken to be 0.2500364)

```
Null deviance: 9257.2  on 9999  degrees of freedom
```

```
Residual deviance: 2499.4  on 9996  degrees of freedom
```

```
AIC: 14523
```

```
Number of Fisher Scoring iterations: 2
```

Knowing the DAG, the regression recovers every structural coefficient:  $L_2$  at 0.9998,  $A_1$  at 0.4940 and  $A_2$  at 1.0000, against true values of 1, 0.5 and 1. Note this identifies the *direct* effect of  $A_1$  only; its path through  $L_2$  is blocked by conditioning on  $L_2$ , which is why the total of 1.9 above cannot be read off this table.

Now consider a different MTP: set half of the time to 0 and leave the other half unchanged.

$$d(a_t, \epsilon_t) = \begin{cases} 0, & \text{if } \epsilon_t < .5 \text{ and } a_t = 1 \\ a_t, & \text{otherwise} \end{cases} \quad (35.2)$$

This is, say, to set half of smokers to non-smokers, and the other half remain smokers.

```
# mtp <- function(data, trt) {
#   a <- data[[trt]]
#   epsilon <- rbinom(nrow(data), size = 1, prob = 0.5)
#   ifelse(epsilon < .5 & a == 1, 0, a)
# }
d <- function(a) {
  epsilon <- runif(length(a))
  ifelse(epsilon < 0.5 & a == 1, 0, a)
}
simulated_data1$m3_d <- simulated_data1$Y

m2 <- glm(m3_d ~ L_1 + A_1 + L_2 + A_2, data = simulated_data1)
summary(m2)
```

Call:

```
glm(formula = m3_d ~ L_1 + A_1 + L_2 + A_2, data = simulated_data1)
```

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )   |
|-------------|-----------|------------|---------|------------|
| (Intercept) | -0.005815 | 0.012842   | -0.453  | 0.651      |
| L_1         | 0.011964  | 0.017621   | 0.679   | 0.497      |
| A_1         | 0.494512  | 0.010814   | 45.729  | <2e-16 *** |

```

L_2          0.998831    0.009987 100.015    <2e-16 ***
A_2          0.999873    0.010273  97.334     <2e-16 ***
---

```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.2500499)

```

Null deviance: 9257.2  on 9999  degrees of freedom
Residual deviance: 2499.2  on 9995  degrees of freedom
AIC: 14525

```

Number of Fisher Scoring iterations: 2

```

simulated_data1$m2_d <- predict(m2, mutate(simulated_data1, A_2 = d(A_2)))

m1 <- glm(m2_d ~ L_1 + A_1, data = simulated_data1)
summary(m1)

```

Call:

```
glm(formula = m2_d ~ L_1 + A_1, data = simulated_data1)
```

Coefficients:

```

              Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.30716     0.01531   20.07    <2e-16 ***
L_1          0.26079     0.02384   10.94    <2e-16 ***
A_1          0.87604     0.01367   64.08    <2e-16 ***
---

```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.4669449)

```

Null deviance: 6633.7  on 9999  degrees of freedom
Residual deviance: 4668.0  on 9997  degrees of freedom
AIC: 20768

```

Number of Fisher Scoring iterations: 2

```

m1_d <- predict(m1, mutate(simulated_data1, A_1 = d(A_1)))
mean(m1_d)

```

```
[1] 0.6518997
```

The recursion makes the blocked path visible. The first regression, on the full history, again gives  $A_1 = 0.4945$  — the direct effect. The second, after substituting the shifted  $A_2$  and regressing on  $L_1$  and  $A_1$  only, gives  $A_1 = 0.8760$ , which is close to the total  $0.5 + 0.4 = 0.9$ : dropping  $L_2$  from the

### 35. longitudinal modified treatment policy (LMTP)

conditioning set lets  $A_1$ 's indirect path back in. The final average is 0.652, the expected outcome under the stochastic half-off regime, sitting between the always-treat 2.029 and never-treat 0.129 as it should.

Note this recursive process is based on Diaz, et al. (2023) “Nonparametric Causal Effects Based on Longitudinal Modified Treatment Policies”.

Here is how exactly we estimate it: we start with the last time point. Regress it on previous treatment and confounders, and then get the predicted value with  $A$  changed based on the MTP. Regress that predicted value on the previous treatment and confounders, get predicted values with  $A$  changed based on MTP. Repeat until the first time point. The average of the predicted value at time 1 is the expected value under this MTP.

The shift function  $\mathbf{d}()$  above is a *stochastic* intervention: at each call it draws a fresh  $\epsilon \sim \text{Uniform}(0, 1)$  and, with probability 0.5, switches a treated unit ( $a = 1$ ) to 0, otherwise leaves  $a$  unchanged. The regime is “at each time point, draw independently conditional on history and apply this rule”; because the draw is independent across time points and units, calling  $\mathbf{d}()$  separately for  $A_2$  and  $A_1$  is what implements that regime. (By contrast, the  $\mathbf{d1}/\mathbf{d2}$  functions used with `lmtpl_tmle` below are *deterministic* static regimes – always 1 and always 0.)

This is basically g-formula extended to longitudinal data.

In a general case, this is the generalized g-formula to estimate  $\theta$ :

Set  $m_{\tau+1} = Y$ , let  $A_t^d = d(A_t, H_t)$ . For  $t = \tau, \dots, 1$ , recursively define:

$$m_t(a_t, h_t) = E[m_{t+1}(A_{t+1}^d, H_{t+1}) | A_t = a_t, H_t = h_t] \quad (35.3)$$

Then  $\theta = E[m_1(A_1^d, L_1)]$ .

We start from the last period. Regress  $m_{\tau+1}$ , which is  $Y$  on  $A_\tau$  and  $H_\tau$ . Then get the predicted value with  $A_\tau$  changed based on the MTP. Then regress that predicted value on  $A_{\tau-1}$  and  $H_{\tau-1}$ . Repeat until the first time point. The average of the predicted value at time 1 is the expected value under this MTP.

#### 35.1.4. Estimators

The authors advocate two estimators, TMLE and SDR (sequentially doubly robust estimator). The procedures are the same, starting from the last time point, then apply TMLE or SDR, iterate to the first time point.

## 35.2. Using lmtpl package

```

library(tidyverse)
library(lmt)
d1 <- function(data, trt) {
  rep(1, nrow(data))
}
A <- "A_2"
Y <- "Y"
W <- c("L_1", "L_2", "A_1")

set.seed(34465)

treat <- lmt_tmle(
  data = simulated_data1,
  trt = "A_2",
  outcome = "Y",
  baseline = W,
  outcome_type = "continuous",
  shift = d1,
  folds = 1,
  learners_trt = "SL.glm",
  learners_outcome = "SL.glm"
)

show_lmt(treat)

```

```

      Estimate: 1.558
Std. error: 0.009

```

```

d2 <- function(data, trt) {
  rep(0, nrow(data))
}
A <- "A_2"
Y <- "Y"
W <- c("L_1", "L_2", "A_1")

set.seed(34465)

control <- lmt_tmle(
  data = simulated_data1,
  trt = "A_2",
  outcome = "Y",
  baseline = W,
  outcome_type = "continuous",
  shift = d2,
  folds = 1,

```

### 35. longitudinal modified treatment policy (LMTP)

```
learners_trt = "SL.glm",
learners_outcome = "SL.glm"
)

show_lmtp(control)
```

```
Estimate: 0.559
Std. error: 0.01
```

```
show_lmtp(lmtp_contrast(treat, ref = control))
```

|   | shift | ref   | estimate | std.error | conf.low | conf.high | p.value |
|---|-------|-------|----------|-----------|----------|-----------|---------|
| 1 | 1.56  | 0.559 | 1        | 0.0102    | 0.98     | 1.02      | <0.001  |

Note that `folds = 1` turns off cross-fitting (sample splitting). That is fine for a small, fast demonstration with `SL.glm`, but in applied work with flexible learners you should use several folds so the nuisance fits are cross-fitted; otherwise the standard errors can be anti-conservative.

**Note on the estimand:** this `lmtp_tmle` example only shifts  $A_2$  (`trt = "A_2"`), treating  $A_1$  as an observed baseline covariate (via `baseline = W`) rather than also intervening on it. That is a different, single-period policy from the always-1-vs-always-0 *both-period* regime computed manually via `simulated_data1$Yd/simulated_data2$Yd` above (lines 139-152) — the two numbers are not expected to match, and should not be used as a sanity check against each other. To replicate the both-period always-treat/never-treat contrast with `lmtp`, shift both time points by passing `trt = c("A_1", "A_2")` and supplying `time_vary` (the time-varying covariates measured before each treatment time, e.g. `list(character(0), "L_2")`), with `shift` functions that map each treatment in the vector to 1 (or 0).

## 35.3. another example

The data set `bmi`, from the `DynTxRegime` package, are simulated to reflect a two-stage RCT  $\{A_1, A_2\}$  that studied the effect of meal replacement (MR) shakes versus a calorie deficit (CD) diet on adolescent

### 35.3.1. shift function 1

Consider a shift function that assigns meal replacement to all observations at time 1, but only meal replacement at time 2 to those observations whose 4-month BMI is greater than 30.

```
library(DynTxRegime)
data(bmiData)
bmi <- bmiData
glimpse(bmi)
```

```
Rows: 210
Columns: 8
$ gender      <int> 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1~
$ race        <int> 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0~
$ parentBMI    <dbl> 31.59683, 30.17564, 30.27918, 27.49256, 26.42350, 29.30970~
$ baselineBMI <dbl> 35.84005, 37.30396, 36.83889, 36.70679, 34.84207, 36.68640~
$ month4BMI    <dbl> 34.22717, 36.38014, 34.42168, 32.52011, 33.72922, 32.06622~
$ month12BMI   <dbl> 34.27263, 36.38401, 34.41447, 32.52397, 33.73546, 32.15977~
$ A1          <chr> "CD", "CD", "MR", "CD", "CD", "MR", "CD", "CD", "CD", "CD"~
$ A2          <chr> "MR", "MR", "CD", "CD", "CD", "MR", "MR", "CD", "MR", "MR"~
```

```
d_dtr <- function(data, trt) {
  if (trt == "A1") return(rep("MR", nrow(data)))

  ifelse(data$month4BMI > 30, "MR", "CD")
}

fit_dtr <- lmtp_sdr(
  data = bmi,
  trt = c("A1", "A2"),
  outcome = "month12BMI",
  baseline = c("gender", "race", "parentBMI"),
  time_vary = list("baselineBMI", "month4BMI"),
  shift = d_dtr,
  outcome_type = "continuous",
  folds = 1,
  learners_trt = "SL.glm",
  learners_outcome = c("SL.mean", "SL.glm", "SL.gam")
)

show_lmtp(fit_dtr)
```

```
      Estimate: 35.854
Std. error: 0.362
```

### 35.3.2. shift function 2

Suppose we are interested in comparing the dynamic treatment regime to a static treatment regime where all patients receive meal replacement at both time points. Using the SDR estimator, estimate the effect of this static intervention.

### 35. longitudinal modified treatment policy (LMTP)

```
fit_MR <- lmtp_sdr(  
  data = bmi,  
  trt = c("A1", "A2"),  
  outcome = "month12BMI",  
  baseline = c("gender", "race", "parentBMI"),  
  time_vary = list("baselineBMI", "month4BMI"),  
  shift = \(data, trt) rep("MR", nrow(data)),  
  outcome_type = "continuous",  
  folds = 1,  
  learners_trt = "SL.glm",  
  learners_outcome = c("SL.mean", "SL.glm", "SL.gam")  
)  
  
show_lmtp(fit_MR)
```

```
      Estimate: 35.831  
Std. error: 0.361
```

Let's also estimate the effect of an intervention where all patients receive a calorie deficit diet at both time points.

```
fit_CD <- lmtp_sdr(  
  data = bmi,  
  trt = c("A1", "A2"),  
  outcome = "month12BMI",  
  baseline = c("gender", "race", "parentBMI"),  
  time_vary = list("baselineBMI", "month4BMI"),  
  shift = \(data, trt) rep("CD", nrow(data)),  
  outcome_type = "continuous",  
  folds = 1,  
  learners_trt = "SL.glm",  
  learners_outcome = c("SL.mean", "SL.glm", "SL.gam")  
)  
  
show_lmtp(fit_CD)
```

```
      Estimate: 35.053  
Std. error: 0.301
```

Finally, we can compare the three treatment regimes using the `lmtp_contrast` function.

```
show_lmtp(lmtp_contrast(fit_dtr, fit_MR, ref = fit_CD))
```



|   | shift | ref  | estimate | std.error | conf.low | conf.high | p.value |
|---|-------|------|----------|-----------|----------|-----------|---------|
| 1 | 35.9  | 35.1 | 0.801    | 0.335     | 0.144    | 1.46      | 0.0169  |
| 2 | 35.8  | 35.1 | 0.778    | 0.335     | 0.122    | 1.43      | 0.0201  |

---

*Systematic treatment:* *R* · *Julia*.



**Part VII.**

**Advanced Topics**



## 36. Doing the same multi-level model with R and stata

This post is a demo for doing the same multi-level models with R and stata. R's multilevel models are mostly done with `lme4` and `nlme`. Stata's multilevel models are mostly done with `mixed` and `menl`. The two packages are not exactly the same. `mixed` does in fact support several residual covariance structures via its `residuals()` option (e.g. `residuals(exponential, t(year))`), as well as `ar`, `ma`, `toeplitz`, etc.) — so it's not that `mixed` cannot handle a structured residual at all, but `menl`'s nonlinear-mixed-model framework gives more flexibility to combine that with custom nonlinear mean/random-effect specifications (like the CEM model below), which is why we reach for it here. Likewise, in R `lme4`'s “`lmer`” does not allow this specification. We have to use `nlme`. But it's good to know how to do the same thing in both packages.

For example, we want to do a consensus emergence model (CEM) as described in [Lang, Bliese and de Voogt \(2018\)](#), *Personnel Psychology*. The CEM model is basically a growth model with the residual being an exponential function of the time variable. We want to do the same thing in both R and stata.

We start with simpler models.

### 36.1. Stata and R syntax comparison for the same three level model

We run a three level model specified in stata with `mixed` and `menl`. In R with `nlme` and `lme4`. This is a random intercept model. Only intercept is varying for each unit at each level. In this data set supposedly state is nested within region. There are situations that levels are not nested, then we need to specify cross effect models. We do not discuss that here. One point to watch: `nlme`'s `lme` function expects the grouping factors in the `random` specification ordered from the *highest* level down. Here `state` is nested within `region`, so `region` must be listed before `state`; reversing the order leads `lme` to treat regions as nested within states, which is not the intended structure.

There are different ways to specify this model in `nlme`. I show two ways here (including `lme2` which is commented out).

#### 36.1.1. Stata

```
* let's see how to use menl
webuse productivity
* to replicate mixed with menl
```

### 36. Doing the same multi-level model with R and stata

```
mixed gsp || region: || state:
menl gsp = {b1:}, define(b1: U1[region] UU1[region>state])
```

(Public capital productivity)

Performing EM optimization ...

Performing gradient-based optimization:

Iteration 0: Log likelihood = 200.86162

Iteration 1: Log likelihood = 200.86162

Computing standard errors ...

Mixed-effects ML regression

Number of obs = 816

Grouping information

| Group variable |  | No. of<br>groups | Observations per group |         |         |
|----------------|--|------------------|------------------------|---------|---------|
|                |  |                  | Minimum                | Average | Maximum |
| region         |  | 9                | 51                     | 90.7    | 136     |
| state          |  | 48               | 17                     | 17.0    | 17      |

Log likelihood = 200.86162

Wald chi2(0) = .  
Prob > chi2 = .

| gsp   | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |          |
|-------|-------------|-----------|-------|-------|----------------------|----------|
| _cons | 10.65961    | .2503806  | 42.57 | 0.000 | 10.16887             | 11.15035 |

| Random-effects parameters |               | Estimate | Std. err. | [95% conf. interval] |          |
|---------------------------|---------------|----------|-----------|----------------------|----------|
| region: Identity          |               |          |           |                      |          |
|                           | var(_cons)    | .4376123 | .2697616  | .13073               | 1.464887 |
| state: Identity           |               |          |           |                      |          |
|                           | var(_cons)    | .6080626 | .1382733  | .3893904             | .9495357 |
|                           | var(Residual) | .0246633 | .0012586  | .0223159             | .0272577 |

LR test vs. linear model: chi2(2) = 2750.56

Prob > chi2 = 0.0000

### 36.1. Stata and R syntax comparison for the same three level model

Note: LR test is conservative and provided only for reference.

Obtaining starting values by EM:

Alternating PNLs/LME algorithm:

Iteration 1: Linearization log likelihood = 200.86162

Iteration 2: Linearization log likelihood = 200.86162

Computing standard errors:

Mixed-effects ML nonlinear regression                      Number of obs        =            816

Grouping information

| Path         | No. of groups | Observations per group |         |         |
|--------------|---------------|------------------------|---------|---------|
|              |               | Minimum                | Average | Maximum |
| region       | 9             | 51                     | 90.7    | 136     |
| region>state | 48            | 17                     | 17.0    | 17      |

Linearization log likelihood = 200.86162

b1: U1[region] UU1[region>state]

|    | gsp   | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |          |
|----|-------|-------------|-----------|-------|-------|----------------------|----------|
| b1 |       |             |           |       |       |                      |          |
|    | _cons | 10.65961    | .2505341  | 42.55 | 0.000 | 10.16857             | 11.15065 |

| Random-effects parameters |               | Estimate | Std. err. | [95% conf. interval] |          |
|---------------------------|---------------|----------|-----------|----------------------|----------|
| region: Identity          |               |          |           |                      |          |
|                           | var(U1)       | .4376119 | .2686479  | .1313834             | 1.457598 |
| region>state: Identity    |               |          |           |                      |          |
|                           | var(UU1)      | .6080625 | .1381527  | .3895418             | .9491666 |
|                           | var(Residual) | .0246633 | .0012586  | .0223159             | .0272577 |

### 36.1.2. R

The data are **productivity**: 816 observations of gross state product, with 48 states nested in 9 regions, 17 years each. The model is intercept-only with random intercepts at both levels.

These two R packages return same results. Note that `lmer` reports variance, while `lme` reports standard deviation.

And they do: `lmer` reports variances of 0.60775 for state-within-region and 0.50977 for region, with a residual of 0.02466; `lme` reports the corresponding standard deviations 0.77958, 0.71398 and 0.15705. Squaring the `lme` figures returns the `lmer` ones ( $0.77958^2 = 0.6077$ ), so the two agree exactly and differ only in what they choose to print. The fixed intercept is 10.665 either way.

Note where the variance sits: between regions and between states, with almost none left within a state over time — the residual standard deviation of 0.157 is about a fifth of the state-level one. Most of the variation in this outcome is cross-sectional, not longitudinal.

```
library(tidyverse)
library(haven)
library(dplyr)
library(readxl)
library(nlme)
library(lme4)
data <- read_dta('https://www.stata-press.com/data/r18/productivity.dta')
# lme4 for three level model
# (1|region) + (1|state) specifies CROSSED random effects; it numerically
# coincides with the intended nested structure here only because each
# state's label happens to be unique across the whole dataset (never reused
# across regions) -- verified: identical logLik to the explicitly nested
# (1|region/state) below. The nested form is the conceptually correct one
# and is robust even if state labels were ever reused across regions.
lmer1 <- lmer(gsp ~ 1 + (1|region/state), data=data)
summary(lmer1)
```

Linear mixed model fit by REML ['lmerMod']

Formula: `gsp ~ 1 + (1 | region/state)`

Data: `data`

REML criterion at convergence: -400.9

Scaled residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -2.8328 | -0.6757 | 0.0474 | 0.6382 | 3.3928 |

Random effects:

| Groups       | Name        | Variance | Std.Dev. |
|--------------|-------------|----------|----------|
| state:region | (Intercept) | 0.60775  | 0.7796   |
| region       | (Intercept) | 0.50977  | 0.7140   |



### 36.1. Stata and R syntax comparison for the same three level model

```
Residual                0.02466  0.1570
Number of obs: 816, groups:  state:region, 48; region, 9
```

Fixed effects:

|             | Estimate | Std. Error | t value |
|-------------|----------|------------|---------|
| (Intercept) | 10.665   | 0.266      | 40.1    |

```
lme1 <- lme(gsp ~ 1, random= list( region=~1, state=~1), data=data)
#or
#lme1 <- lme(gsp ~ 1, random= list(region = pdSymm(~1), state=pdSymm(~1)), data=data)
#but
#lme1 <- lme(gsp ~ 1, random= list( state=~1, region=~1), data=data)
#would be a different result!
#The other way to specify:
#lme2 <- lme(gsp ~ 1, random= ~ 1 | region/state, data=data)
summary(lme1)
```

Linear mixed-effects model fit by REML

```
Data: data
      AIC      BIC  logLik
-392.8509 -374.0381 200.4254
```

Random effects:

```
Formula: ~1 | region
(Intercept)
StdDev:   0.7139792
```

```
Formula: ~1 | state %in% region
(Intercept) Residual
StdDev:   0.7795814 0.1570457
```

Fixed effects: gsp ~ 1

|             | Value    | Std.Error | DF  | t-value  | p-value |
|-------------|----------|-----------|-----|----------|---------|
| (Intercept) | 10.66483 | 0.2659842 | 768 | 40.09574 | 0       |

Standardized Within-Group Residuals:

| Min         | Q1          | Med        | Q3         | Max        |
|-------------|-------------|------------|------------|------------|
| -2.83282326 | -0.67572390 | 0.04744387 | 0.63817953 | 3.39281358 |

Number of Observations: 816

Number of Groups:

| region | state %in% region |
|--------|-------------------|
| 9      | 48                |

## 36.2. A growth model in R and stata

We implement a growth model in R and stata. The results are somewhat different. The results may not be that reliable given that the year slope random component is tiny.

### 36.2.1. Stata

```
* same growth model with mixed and menl
* Stata's "mixed ... || region: year" defaults to an INDEPENDENT covariance
* structure for region's random intercept and slope (var(year), var(_cons),
* cov=0 by construction) -- verified via `estat recovariance`. R's lmer
* "(year|region)" and lme's "region=~year" both default to an UNSTRUCTURED
* (correlated) covariance. Add covariance(unstructured) to match R.
webuse productivity
mixed gsp year || region: year, covariance(unstructured) || state:
menl gsp = {b2:} + {U1[region]}*year, define(b2: year U0[region] U11[region>state])
```

(Public capital productivity)

Performing EM optimization ...

Performing gradient-based optimization:

```
Iteration 0: Log likelihood = 782.11459 (not concave)
Iteration 1: Log likelihood = 803.58071 (not concave)
Iteration 2: Log likelihood = 813.1017 (not concave)
Iteration 3: Log likelihood = 816.56892
Iteration 4: Log likelihood = 885.27225 (not concave)
Iteration 5: Log likelihood = 888.29208
Iteration 6: Log likelihood = 888.79472
Iteration 7: Log likelihood = 888.82395
Iteration 8: Log likelihood = 888.82404
Iteration 9: Log likelihood = 888.82404
```

Computing standard errors ...

Mixed-effects ML regression

Number of obs = 816

Grouping information

| Group variable |  | No. of<br>groups | Observations per group |         |         |
|----------------|--|------------------|------------------------|---------|---------|
|                |  |                  | Minimum                | Average | Maximum |
| region         |  | 9                | 51                     | 90.7    | 136     |
| state          |  | 48               | 17                     | 17.0    | 17      |

```

-----
Log likelihood = 888.82404                                Wald chi2(1) = 107.46
                                                         Prob > chi2  = 0.0000

```

| gsp   | Coefficient | Std. err. | z     | P> z  | [95% conf. interval] |           |
|-------|-------------|-----------|-------|-------|----------------------|-----------|
| year  | .0263275    | .0025397  | 10.37 | 0.000 | .0213497             | .0313053  |
| _cons | -41.43216   | 5.20712   | -7.96 | 0.000 | -51.63793            | -31.22639 |

| Random-effects parameters |                 | Estimate  | Std. err. | [95% conf. interval] |           |
|---------------------------|-----------------|-----------|-----------|----------------------|-----------|
| region: Unstructured      |                 |           |           |                      |           |
|                           | var(year)       | .000056   | .0000274  | .0000215             | .000146   |
|                           | var(_cons)      | 235.7658  | 115.06    | 90.58819             | 613.6066  |
|                           | cov(year,_cons) | -.1148543 | .056104   | -.2248162            | -.0048923 |
| state: Identity           |                 |           |           |                      |           |
|                           | var(_cons)      | .6121886  | .139357   | .39185               | .9564242  |
|                           | var(Residual)   | .0039862  | .0002046  | .0036046             | .0044081  |

```

LR test vs. linear model: chi2(4) = 4112.15                Prob > chi2 = 0.0000

```

Note: LR test is conservative and provided only for reference.

Obtaining starting values by EM:

Alternating PNLS/LME algorithm:

```

Iteration 1: Linearization log likelihood = 784.65134
Iteration 2: Linearization log likelihood = 784.65343
Iteration 3: Linearization log likelihood = 784.65343

```

Computing standard errors:

```

Mixed-effects ML nonlinear regression                Number of obs    =      816

```

Grouping information

| Path   | No. of groups | Observations per group |         |         |
|--------|---------------|------------------------|---------|---------|
|        |               | Minimum                | Average | Maximum |
| region | 9             | 51                     | 90.7    | 136     |

### 36. Doing the same multi-level model with R and stata

```

            region>state |           48           17           17.0           17
-----
Linearization log likelihood = 784.65343      Wald chi2(1)      =      2737.78
                                           Prob > chi2      =      0.0000

```

```
b2: year U0[region] U11[region>state]
```

```

-----
            gsp | Coefficient   Std. err.      z    P>|z|    [95% conf. interval]
-----+-----
b2          |
   year      |   .0274903   .0005254   52.32   0.000   .0264605   .02852
   _cons     |  -43.71615   1.069039  -40.89   0.000  -45.81143  -41.62087
-----

```

```

-----
Random-effects parameters | Estimate   Std. err.    [95% conf. interval]
-----+-----
region: Independent      |
   var(U0)               |   .4378219   .2687857   .1314413   1.458355
   var(U1)               |   5.40e-13   2.67e-10           0           .
-----+-----
region>state: Identity   |
   var(U11)              |   .6091976   .1381402   .3906086   .9501115
-----+-----
   var(Residual)        |   .0053926   .0002752   .0048793   .0059598
-----

```

#### 36.2.2. R

```

# lme4 for three level model, growth model
lmer_growth1 <- lmer(gsp ~ 1 + year + (year|region) + (1|state), data=data, REML=FALSE)
summary(lmer_growth1)

```

Linear mixed model fit by maximum likelihood ['lmerMod']

Formula: gsp ~ 1 + year + (year | region) + (1 | state)

Data: data

| AIC     | BIC     | logLik | -2*log(L) | df.resid |
|---------|---------|--------|-----------|----------|
| -1455.8 | -1422.9 | 734.9  | -1469.8   | 809      |

Scaled residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -2.9207 | -0.5232 | 0.0180 | 0.5541 | 3.8399 |

```
Random effects:
  Groups   Name      Variance Std.Dev.  Corr
state     (Intercept) 1.327e-01 3.643e-01
region    (Intercept) 2.068e-01 4.547e-01
          year        4.495e-09 6.705e-05 1.00
Residual                   6.664e-03 8.163e-02
Number of obs: 816, groups: state, 48; region, 9
```

```
Fixed effects:
              Estimate Std. Error t value
(Intercept) -4.373e+01  1.165e+00  -37.53
year         2.751e-02  5.838e-04   47.13
```

```
Correlation of Fixed Effects:
      (Intr)
year -0.985
optimizer (nloptwrap) convergence code: 0 (OK)
boundary (singular) fit: see help('isSingular')
```

```
# nlme for three level model, growth model
#lme_growth1<-lme(gsp ~ year, random = list(region=pdSymm(~year), state=pdIdent(~1)),data=data)
lme_growth1<-lme(gsp ~ year, random = list(region=~year, state=~1),data=data, method="ML")
summary(lme_growth1)
```

Linear mixed-effects model fit by maximum likelihood

```
Data: data
      AIC      BIC    logLik
-1555.307 -1522.376 784.6534
```

```
Random effects:
Formula: ~year | region
Structure: General positive-definite, Log-Cholesky parametrization
      StdDev      Corr
(Intercept) 6.61523e-01 (Intr)
year        7.96178e-09 0.001
```

```
Formula: ~1 | state %in% region
      (Intercept) Residual
StdDev:   0.7805102 0.0734343
```

```
Fixed effects: gsp ~ year
              Value Std.Error DF   t-value p-value
(Intercept) -43.71617 1.0690287 767 -40.89336      0
year         0.02749 0.0005254 767  52.32366      0
Correlation:
      (Intr)
```

### 36. Doing the same multi-level model with R and stata

year -0.972

Standardized Within-Group Residuals:

| Min         | Q1          | Med        | Q3         | Max        |
|-------------|-------------|------------|------------|------------|
| -3.24606517 | -0.59087173 | 0.03087456 | 0.62062441 | 4.27790886 |

Number of Observations: 816

Number of Groups:

| region | state | %in% | region |
|--------|-------|------|--------|
| 9      |       |      | 48     |

## 36.3. cem in R and stata

“CEM” is a growth model with the residual being a exponential function of the time variable.

### 36.3.1. stata

This mode has some convergence problem. I still use it for demonstration purpose.

```
webuse productivity, clear
```

```
* this replicates the results from R
```

```
menl gsp = {b2:} + {U1[region]}*year, define(b2: year U0[region] U11[region>state]) resvarian
```

(Public capital productivity)

Mixed-effects ML nonlinear regression                      Number of obs        =            816

Grouping information

| Path         | No. of groups | Observations per group |         |         |
|--------------|---------------|------------------------|---------|---------|
|              |               | Minimum                | Average | Maximum |
| region       | 9             | 51                     | 90.7    | 136     |
| region>state | 48            | 17                     | 17.0    | 17      |

|                                          |              |   |        |
|------------------------------------------|--------------|---|--------|
|                                          | Wald chi2(1) | = | 107.29 |
| Linearization log likelihood = 877.96184 | Prob > chi2  | = | 0.0000 |

b2: year U0[region] U11[region>state]

| gsp | Coefficient | Std. err. | z | P> z | [95% conf. interval] |
|-----|-------------|-----------|---|------|----------------------|
|-----|-------------|-----------|---|------|----------------------|

|                           |          |           |          |           |                      |           |           |
|---------------------------|----------|-----------|----------|-----------|----------------------|-----------|-----------|
| -----+-----               |          |           |          |           |                      |           |           |
| b2                        |          |           |          |           |                      |           |           |
| year                      |          | .0274256  | .0026478 | 10.36     | 0.000                | .0222361  | .0326151  |
| _cons                     |          | -43.54047 | 5.400007 | -8.06     | 0.000                | -54.12429 | -32.95665 |
| -----                     |          |           |          |           |                      |           |           |
| -----+-----               |          |           |          |           |                      |           |           |
| Random-effects parameters |          |           | Estimate | Std. err. | [95% conf. interval] |           |           |
| -----+-----               |          |           |          |           |                      |           |           |
| region: Independent       |          |           |          |           |                      |           |           |
|                           | var(U0)  |           | 263.5659 | 128.3252  | 101.4975             | 684.4206  |           |
|                           | var(U1)  |           | .0000633 | .0000309  | .0000243             | .0001646  |           |
| -----+-----               |          |           |          |           |                      |           |           |
| region>state: Identity    |          |           |          |           |                      |           |           |
|                           | var(U11) |           | .6046714 | .1369848  | .3878686             | .942658   |           |
| -----+-----               |          |           |          |           |                      |           |           |
| Residual variance:        |          |           |          |           |                      |           |           |
| Exponential year          |          |           |          |           |                      |           |           |
|                           | sigma2   |           | 9.96e-99 | .         | .                    | .         |           |
|                           | gamma    |           | .0556293 | .000013   | .0556039             | .0556548  |           |
| -----                     |          |           |          |           |                      |           |           |

### 36.3.2. R

As in stata, it has some convergence problem. So the results can differ somewhat between R and stata.

```
lme_growth1<-lme(gsp ~ year, random = list(region=pdSymm(~year), state=pdIdent(~1)),data=data,
lme_growth2<-update(lme_growth1,weights=varExp( form = ~ year), method="ML", control = lmeCont
summary(lme_growth2)
```

Linear mixed-effects model fit by maximum likelihood

Data: data

|           |           |          |
|-----------|-----------|----------|
| AIC       | BIC       | logLik   |
| -1548.244 | -1510.608 | 782.1218 |

Random effects:

Formula: ~year | region

Structure: General positive-definite

|             |              |        |
|-------------|--------------|--------|
|             | StdDev       | Corr   |
| (Intercept) | 0.2937615870 | (Intr) |
| year        | 0.0001436049 | 0.995  |

Formula: ~1 | state %in% region

|         |             |           |
|---------|-------------|-----------|
|         | (Intercept) | Residual  |
| StdDev: | 0.8046618   | 0.0736431 |

### 36. Doing the same multi-level model with R and stata

Variance function:

Structure: Exponential of variance covariate

Formula: ~year

Parameter estimates:

expon

-2.165027e-16

Fixed effects: gsp ~ year

|             | Value     | Std.Error | DF  | t-value   | p-value |
|-------------|-----------|-----------|-----|-----------|---------|
| (Intercept) | -43.80206 | 1.0533833 | 767 | -41.58226 | 0       |
| year        | 0.02752   | 0.0005291 | 767 | 52.01290  | 0       |

Correlation:

(Intr)

year -0.976

Standardized Within-Group Residuals:

|  | Min         | Q1          | Med        | Q3         | Max        |
|--|-------------|-------------|------------|------------|------------|
|  | -3.24679735 | -0.58919213 | 0.03104216 | 0.61843888 | 4.27977261 |

Number of Observations: 816

Number of Groups:

|  | region | state | %in% | region |
|--|--------|-------|------|--------|
|  | 9      |       |      | 48     |

```
delta1<-lme_growth2$modelStruct$varStruct
delta1
```

Variance function structure of class varExp representing

expon

-2.165027e-16



## 37. Modeling Variation, Not Just the Mean, in Likert-Scale Data

### 37.1. The Problem: The Mechanical Ceiling on Dispersion

When analyzing Likert-scale ratings (e.g., 1–7 satisfaction or agreement scales), researchers often want to know whether certain groups exhibit greater disagreement or dispersion than others. A common approach is to calculate the standard deviation for each unit and regress it on covariates.

On a bounded scale  $[a, b]$ , however, variance is mechanically constrained by the mean. By the Bhatia–Davis (2000) inequality, for any random variable bounded in  $[a, b]$  with mean  $\mu$ :

$$\text{Var}(X) \leq (\mu - a)(b - \mu) \quad (37.1)$$

On a 1–7 Likert item, the maximum possible standard deviation is:

$$\text{SD}_{\max}(\mu) = \sqrt{(\mu - 1)(7 - \mu)}$$

```
mu_grid <- seq(1, 7, length.out = 200)
sd_max  <- sqrt(pmax((mu_grid - 1) * (7 - mu_grid), 0))

ggplot(data.frame(mu = mu_grid, sd_max = sd_max), aes(mu, sd_max)) +
  geom_line(linewidth = 1, color = "#2d5fa3") +
  labs(x = "Mean Rating ( )", y = expression(SD[max](mu)),
       title = "Maximum Possible SD as a Function of the Mean") +
  theme_minimal(base_size = 12)
```

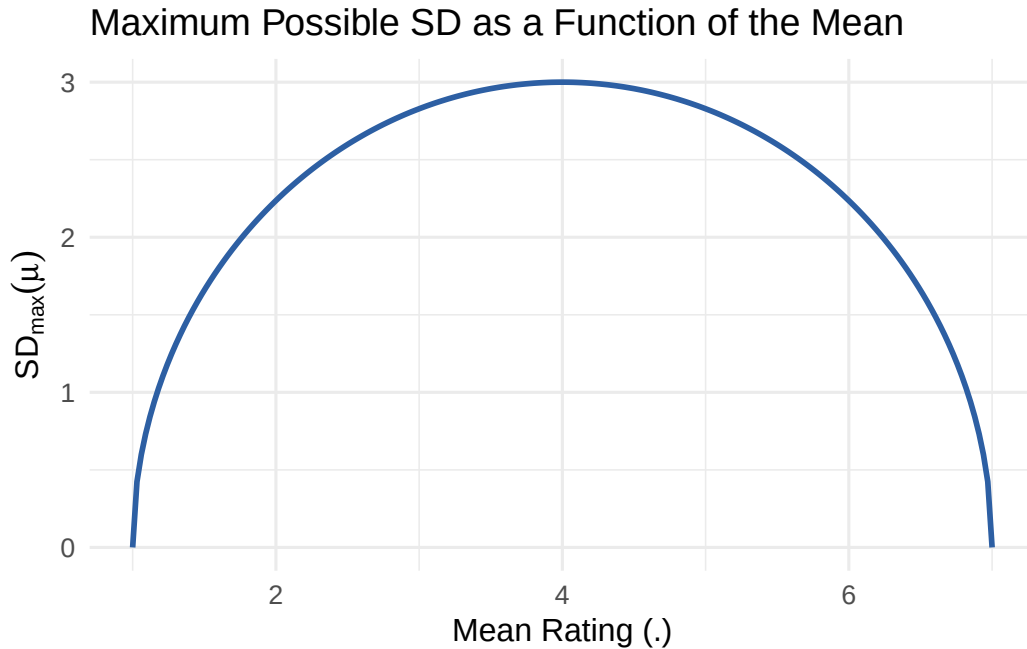


Figure 37.1.: The mechanical ceiling on SD as a function of the mean on a 1–7 scale.

The maximum standard deviation peaks at mid-scale ( $\mu = 4$ ,  $SD_{\max} = 3$ ) and drops to zero at the boundaries ( $\mu = 1$  and  $\mu = 7$ ). When ratings are skewed toward the ceiling—typical of satisfaction or performance evaluations—groups with higher means have mechanically smaller standard deviations, even if their true underlying dispersion is identical.

### 37.1.1. Why Standard Ad-Hoc Adjustments Fail

1. **Coefficient of Variation ( $SD/\mu$ ):** Dividing by the mean assumes variance scales proportionally with the mean, as in count data. On a bounded scale, variance pinches at *both* ends; dividing by  $\mu$  simply shifts the peak from  $\mu = 4$  to  $\mu = 1.75$  without eliminating mean dependence.
2. **Gaussian Location-Scale Models:** Linear mixed-effects models that specify `sigma ~ group` assume an unbounded normal distribution. Because the model is unaware of the scale limits, boundary compression is misinterpreted as smaller residual variance.

## 37.2. The Solution: Ordinal Cumulative-Probit Location-Scale Model

The appropriate approach treats Likert responses as a discretized continuous latent variable  $Y^*$ :

$$Y_i^* = \eta_i + \sigma_i \epsilon_i, \quad \epsilon_i \sim N(0, 1)$$

where  $\eta_i$  is the location predictor and  $\sigma_i$  is the latent residual standard deviation. The observed rating  $Y_i \in \{1, \dots, K\}$  is determined by ordered cutpoints  $\theta_1 < \dots < \theta_{K-1}$ :

$$Y_i = k \iff \theta_{k-1} < Y_i^* \leq \theta_k$$

Under a probit link, the cumulative probability is:

$$P(Y_i \leq k) = \Phi\left(\frac{\theta_k - \eta_i}{\sigma_i}\right) = \Phi(\text{disc}_i(\theta_k - \eta_i))$$

where discrimination is defined as the inverse of latent dispersion:  $\text{disc}_i \equiv 1/\sigma_i$ .

In **brms**, we model  $\log(\text{disc}_i)$  directly. Higher discrimination corresponds to smaller latent dispersion (more consensus), while lower discrimination indicates greater spread. Because the cutpoints  $\theta$  are estimated on an unbounded latent scale, boundary compression no longer biases the dispersion parameter.

### 37.3. Simulation: Identical Dispersion with Different Means

We simulate 100 subjects per group, each evaluated by 6 raters. Both groups share the identical latent standard deviation ( $\sigma = 1$ ), but Group A's latent mean is near the ceiling ( $\mu_A = 2.4$ ), while Group B is centered mid-range ( $\mu_B = 0.6$ ).

```
set.seed(42)

n_per_group <- 100
n_raters    <- 6
sigma_true  <- 1.0 # Identical latent dispersion across groups
mu_A <- 2.4      # Group A: high mean (near ceiling)
mu_B <- 0.6      # Group B: mid-scale mean

subjects <- data.frame(
  subject = 1:(2 * n_per_group),
  group   = rep(c("A", "B"), each = n_per_group),
  mu      = rep(c(mu_A, mu_B), each = n_per_group)
)

df <- subjects[rep(seq_len(nrow(subjects)), each = n_raters), ]
df$latent <- rnorm(nrow(df), mean = df$mu, sd = sigma_true)

# Discretize using 6 fixed cutpoints into categories 1 through 7
cuts <- c(-2, -1.2, -0.5, 0.2, 1.0, 1.8)
df$rating <- as.integer(cut(df$latent, breaks = c(-Inf, cuts, Inf), labels = 1:7))
df$rating_ord <- factor(df$rating, ordered = TRUE, levels = 1:7)
```

## 37.3.1. Observed Ratings and the Apparent Gap

```

subj_summary <- df |>
  group_by(subject, group) |>
  summarise(mean_rating = mean(rating), sd_rating = sd(rating), .groups = "drop")

subj_summary |>
  group_by(group) |>
  summarise(mean_rating = round(mean(mean_rating), 2),
            mean_sd      = round(mean(sd_rating), 2)) |>
  knitr::kable(caption = "Observed summary statistics by group.")

```

Table 37.1.: Observed summary statistics by group.

| group | mean_rating | mean_sd |
|-------|-------------|---------|
| A     | 6.64        | 0.58    |
| B     | 4.89        | 1.22    |

```

ggplot(subj_summary, aes(mean_rating, sd_rating, color = group)) +
  geom_jitter(width = 0.05, height = 0, alpha = 0.5) +
  geom_line(data = data.frame(mean_rating = mu_grid, sd_rating = sd_max),
            aes(mean_rating, sd_rating), color = "grey40", inherit.aes = FALSE) +
  labs(x = "Subject Mean Rating", y = "Subject SD",
       title = "Boundary Effect: Group A's SD is Mechanically Compressed") +
  theme_minimal(base_size = 12)

```

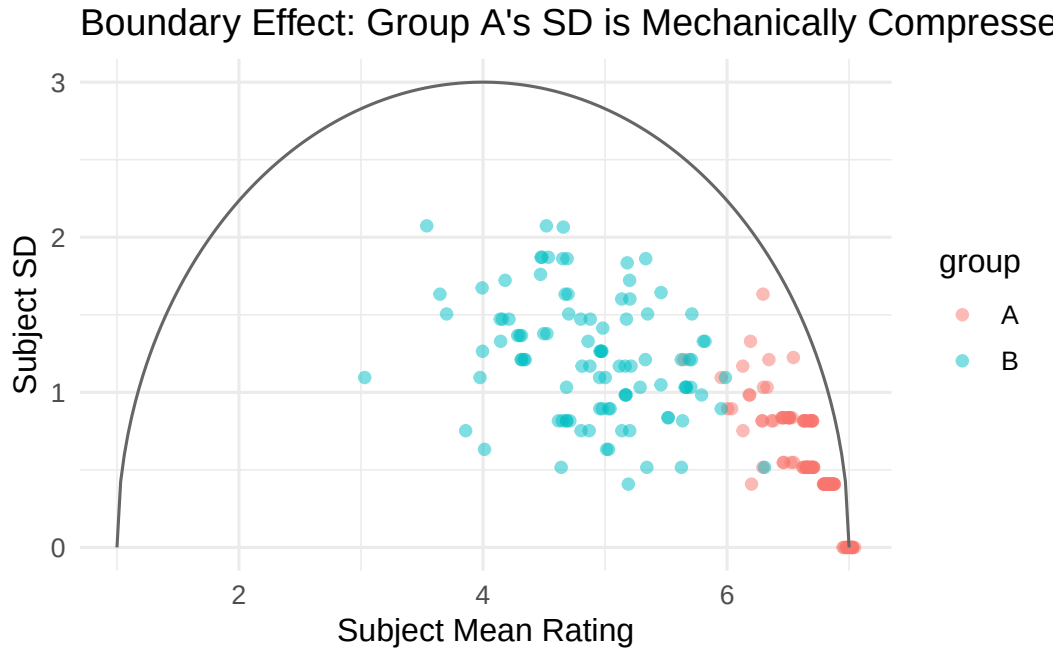


Figure 37.2.: Subject-level mean vs. SD against the theoretical Bhatia-Davis bound.

Group A averages an observed SD of 0.58 compared to 1.22 for Group B. Looking only at raw statistics suggests that raters agree much more about Group A.

## 37.4. Model Comparison: Gaussian vs. Ordinal Location-Scale

We fit two models in `brms`: 1. **Gaussian MELSM**: Models  $\log(\sigma)$  as a function of group. 2. **Ordinal Cumulative-Probit MELSM**: Models  $\log(\text{disc})$  with Group A fixed at reference ( $\text{disc}_A = 1$ ).

```
# 1. Gaussian location-scale model
fit_gaussian <- brm(
  bf(rating ~ group + (1 | subject), sigma ~ group),
  data = df, family = gaussian(),
  chains = 2, iter = 2000, warmup = 1000, cores = 2, seed = 1,
  refresh = 0, silent = 2
)

# 2. Ordinal cumulative-probit location-scale model
fit_ordinal <- brm(
  bf(rating_ord ~ group + (1 | subject), disc ~ 0 + group),
  data = df, family = cumulative("probit"), init = 0,
  prior = c(
    prior(constant(0), class = "b", coef = "groupA", dpar = "disc"),
    prior(normal(0, 1), class = "b", coef = "groupB", dpar = "disc")
  ),
)
```

```
chains = 2, iter = 2000, warmup = 1000, cores = 2, seed = 1,
refresh = 0, silent = 2
)
```

```
knitr::kable(round(fixef(fit_gaussian), 2), caption = "Gaussian MELSM: Incorrectly finds group
```

Table 37.2.: Gaussian MELSM: Incorrectly finds group differences in residual SD.

|                 | Estimate | Est.Error | Q2.5  | Q97.5 |
|-----------------|----------|-----------|-------|-------|
| Intercept       | 6.64     | 0.03      | 6.58  | 6.69  |
| sigma_Intercept | -0.41    | 0.03      | -0.46 | -0.35 |
| groupB          | -1.75    | 0.06      | -1.86 | -1.63 |
| sigma_groupB    | 0.68     | 0.04      | 0.60  | 0.76  |

```
knitr::kable(round(fixef(fit_ordinal), 2), caption = "Ordinal MELSM: Correctly recovers equal
```

Table 37.3.: Ordinal MELSM: Correctly recovers equal latent dispersion.

|              | Estimate | Est.Error | Q2.5  | Q97.5 |
|--------------|----------|-----------|-------|-------|
| Intercept[1] | -4.39    | 0.32      | -5.06 | -3.81 |
| Intercept[2] | -3.65    | 0.23      | -4.12 | -3.22 |
| Intercept[3] | -2.98    | 0.17      | -3.34 | -2.66 |
| Intercept[4] | -2.27    | 0.12      | -2.52 | -2.04 |
| Intercept[5] | -1.44    | 0.07      | -1.59 | -1.30 |
| Intercept[6] | -0.63    | 0.06      | -0.74 | -0.52 |
| groupB       | -1.89    | 0.10      | -2.11 | -1.69 |
| disc_groupA  | 0.00     | 0.00      | 0.00  | 0.00  |
| disc_groupB  | -0.01    | 0.08      | -0.17 | 0.15  |

### 37.4.1. Interpretation

- **Gaussian Model:** The coefficient on `sigma_groupB` is 0.68, indicating that Group B's residual standard deviation is estimated to be  $\exp(0.68) \approx 1.97$  times larger than Group A's (95% CI : [1.82, 2.14]). This large effect is an artifact of the scale ceiling.
- **Ordinal Model:** The coefficient on `disc_groupB` is  $-0.01$  (95% CI :  $[-0.17, 0.15]$ ), corresponding to a latent standard deviation ratio of  $\exp(0.01) \approx 1.01$ . The ordinal model correctly recovers that the underlying dispersion is identical.

## 37.5. Benchmarking Relative Dispersion: The $R$ Metric

An intuitive distribution-free alternative is the ratio of observed variance to maximum possible variance:

$$R = \frac{\text{Var}(Y)}{\text{Var}_{\max}(\mu)} \in [0, 1]$$

where  $R = 0$  indicates perfect agreement and  $R = 1$  indicates maximum polarization.

When computed on small individual clusters ( $n = 6$ ),  $R$  can be noisy or undefined if all raters choose 7 (0/0). However, when evaluated at the group level pooling all ratings,  $R$  accurately captures dispersion:

```
df |>
  group_by(group) |>
  summarise(
    mean_rating = mean(rating),
    var_obs     = mean((rating - mean(rating))^2),
    var_max     = (mean(rating) - 1) * (7 - mean(rating)),
    R_pooled    = round(var_obs / var_max, 3)
  ) |>
  knitr::kable(caption = "Group-level pooled R metric.")
```

Table 37.4.: Group-level pooled R metric.

| group | mean_rating | var_obs   | var_max  | R_pooled |
|-------|-------------|-----------|----------|----------|
| A     | 6.638333    | 0.4475306 | 2.039197 | 0.219    |
| B     | 4.893333    | 1.7286222 | 8.201956 | 0.211    |

Both groups yield  $R \approx 0.21$ – $0.22$ , correctly reflecting identical relative dispersion.

## 37.6. Summary

1. **Mechanical Confound:** On bounded scales, variance is bounded by  $(\mu - a)(b - \mu)$ . Skewed ratings create spurious group differences in raw standard deviations.
2. **Gaussian Failure:** Linear mixed-effects location-scale models (`sigma ~ x`) misinterpret ceiling compression as true consensus.
3. **Ordinal Solution:** Cumulative probit location-scale models (`disc ~ x`) model dispersion on the latent scale, separating true variance differences from boundary artifacts.
4. **Descriptive Metric:** When a simple summary statistic is required, group-level pooled  $R = \text{Var}/\text{Var}_{\max}$  removes scale-boundary effects without fitting complex models.





## 38. Conjoint design and analysis using R

This post is to illustrate how to use R to do basic conjoint analysis.

### 38.1. Conjoint Analysis

Marketing people used to use Sawtooth to do conjoint analysis.

What is conjoint analysis? Here is the intro from Sawtooth:

“Conjoint analysis methods use statistical analysis to compute mathematical representations of survey respondents’ preferences for product features, simulate how attribute changes affect demand, and even predict market acceptance of new products before they launch.

This powerful research methodology has become very popular with market researchers and economists for the valuable insight that few other research methods can provide.”

“Consider the task of purchasing a new vehicle.

When searching for the “right” vehicle, you typically consider a multitude of variables: type, make (brand), model, year, mileage, price, color, accessories packages, etc. You can see how this is a complex choice to make.

How does anyone possibly make such a complicated decision?

You think about what features matter most to you and then make tradeoffs.

For example, one vehicle could be an acceptable make and model, but at a high price point, whereas a second option is not the ideal make and model but has a more palatable price.

The tradeoff in this simple example is whether the make and model is more preferable to a reasonable price, or vice versa.

A conjoint survey captures the relationship between different attributes of a product and how preferable they are both comparatively and when combined.

Because of this, conjoint analysis is the gold standard solution for testing multiple features simultaneously, when limited A/B testing doesn’t go far enough.”

“In simple terms, the conjoint survey design algorithm generates a balanced survey design with product profiles, also known as concepts, formulated to have ideal statistical properties (such as level balance and independence of attributes).

Each profile is made up of multiple conjoined product features (hence, conjoint analysis), such as brand, type, engine, and color, each with systematically varied levels.

These product concepts are then included in a series of questions, usually 8-20, called choice tasks, that make up the conjoint analysis portion of the survey.”

## 38.2. Example

I am following the example here: [https://bookdown.org/lien\\_lamey/R\\_tutorial/8-1-data-5.html](https://bookdown.org/lien_lamey/R_tutorial/8-1-data-5.html)

```
library(tidyverse)
library(openxlsx)

url <- "https://feb.kuleuven.be/lien.lamey/public/rmodule/icecream.xlsx"
icecream <- read.xlsx(url)
icecream <- icecream %>%
  pivot_longer(cols = starts_with("Individual"), names_to = "respondent", values_to = "rating") %>%
  rename("profile" = "Observations") %>% # rename Observations to profile
  mutate(profile = factor(profile), respondent = factor(respondent), # factorize identifiers
         Flavor = factor(Flavor), Packaging = factor(Packaging), Light = factor(Light), Organic = factor(Organic))

icecream
```

```
# A tibble: 150 × 7
  profile Flavor Packaging Light Organic respondent rating
  <fct>   <fct>   <fct>   <fct>   <fct>   <fct>   <dbl>
1 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 1
2 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 6
3 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 5
4 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 1
5 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 2
6 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 7
7 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 7
8 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 5
9 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 1
10 Profile 1 Raspberry Homemade waffle No low fat Not organic Individual... 10
# 140 more rows
```

In this example, we have 4 attributes: Flavor, Packaging, Light, and Organic. There are 5, 3, 2, 2 levels correspondingly. Usually we are interested in how much does each level of attributes predict the outcome, say the rating of the icecream. To do this, we would think we'll need all  $5 \times 3 \times 2 \times 2 = 60$  combinations of attribute levels to estimate the effect. In that case, we would need data for each combination. This is called full factorial design. The problem of full factorial design is that it might be too costly or not possible if the combination is huge. Instead, conjoint analysis allows us to estimate the effect of each level of each attribute by using a fraction of the combinations. This is done by creating a set of profiles, each of which contains a subset of the levels of the attributes. The profiles are then presented to the respondents, who rate the profiles. The ratings are then used to estimate the effect of each level of each attribute. This is done by regressing the ratings on the levels of the attributes. The coefficients of the regression are the estimates of the effects of the levels of the attributes.

The disadvantage of a fractional factorial design is that some effects are confounded; that is, the effect of one level of an attribute is confounded with the effect of another level of another attribute.

```
# attribute1, attribute2, etc. are vectors with one element in which we first provide the name
attribute1 <- "Flavor; Raspberry; Chocolate; Strawberry; Mango; Vanilla"
attribute2 <- "Package; Homemade waffle; Cone; Pint"
attribute3 <- "Light; Low fat; No low fat"
attribute4 <- "Organic; Organic; Not organic"

# now combine these different attributes into one vector with c()
attributes <- c(attribute1, attribute2, attribute3, attribute4)
attributes
```

```
[1] "Flavor; Raspberry; Chocolate; Strawberry; Mango; Vanilla"
[2] "Package; Homemade waffle; Cone; Pint"
[3] "Light; Low fat; No low fat"
[4] "Organic; Organic; Not organic"
```

### 38.3. Conjoint design of experiment

The first step in conjoint analysis is to create a design of experiment. This is to use a fraction of all possible combinations of attribute levels to estimate the effect of each level of each attribute. For example, instead of using all 60 icecream profiles, we use 10 profiles.

We use “radiant” package’s “doe” function to create the design of experiment. The “doe” function takes the attributes as input and returns a data frame with the profiles. The “seed” argument is used to fix the random number generator. This is useful if you want to reproduce the same results.

```
library(radiant)

ice_doe <- doe(attributes, seed = 123)
ice_doe$eff
```

|    | Trials | D-efficiency | Balanced |
|----|--------|--------------|----------|
| 1  | 9      | 0.105        | FALSE    |
| 2  | 10     | 0.389        | FALSE    |
| 3  | 11     | 0.411        | FALSE    |
| 4  | 12     | 0.614        | FALSE    |
| 5  | 13     | 0.542        | FALSE    |
| 6  | 14     | 0.479        | FALSE    |
| 7  | 15     | 0.762        | FALSE    |
| 8  | 16     | 0.738        | FALSE    |
| 9  | 17     | 0.748        | FALSE    |
| 10 | 18     | 0.756        | FALSE    |
| 11 | 19     | 0.644        | FALSE    |
| 12 | 20     | 0.895        | FALSE    |
| 13 | 21     | 0.848        | FALSE    |
| 14 | 22     | 0.833        | FALSE    |

### 38. Conjoint design and analysis using R

|    |    |       |       |
|----|----|-------|-------|
| 15 | 23 | 0.790 | FALSE |
| 16 | 24 | 0.827 | FALSE |
| 17 | 25 | 0.787 | FALSE |
| 18 | 26 | 0.768 | FALSE |
| 19 | 27 | 0.759 | FALSE |
| 20 | 28 | 0.736 | FALSE |
| 21 | 29 | 0.702 | FALSE |
| 22 | 30 | 0.984 | TRUE  |
| 23 | 31 | 0.952 | FALSE |
| 24 | 32 | 0.933 | FALSE |
| 25 | 33 | 0.928 | FALSE |
| 26 | 34 | 0.900 | FALSE |
| 27 | 35 | 0.871 | FALSE |
| 28 | 36 | 0.893 | FALSE |
| 29 | 37 | 0.866 | FALSE |
| 30 | 38 | 0.843 | FALSE |
| 31 | 39 | 0.836 | FALSE |
| 32 | 40 | 0.922 | FALSE |
| 33 | 41 | 0.899 | FALSE |
| 34 | 42 | 0.904 | FALSE |
| 35 | 43 | 0.882 | FALSE |
| 36 | 44 | 0.861 | FALSE |
| 37 | 45 | 0.949 | FALSE |
| 38 | 46 | 0.919 | FALSE |
| 39 | 47 | 0.912 | FALSE |
| 40 | 48 | 0.911 | FALSE |
| 41 | 49 | 0.891 | FALSE |
| 42 | 50 | 0.959 | FALSE |
| 43 | 51 | 0.939 | FALSE |
| 44 | 52 | 0.944 | FALSE |
| 45 | 53 | 0.925 | FALSE |
| 46 | 54 | 0.924 | FALSE |
| 47 | 55 | 0.906 | FALSE |
| 48 | 56 | 0.902 | FALSE |
| 49 | 57 | 0.884 | FALSE |
| 50 | 58 | 0.872 | FALSE |
| 51 | 59 | 0.855 | FALSE |
| 52 | 60 | 1.000 | TRUE  |

```
#summary(doe(attributes, seed = 123, trials = 10))
```

This is a list of efficiency of different number of “trials”. Trials mean combinations of attribute levels. For example, with 10 trials, the efficiency is .389 and it’s unbalanced.

We can see that the only two designs with balanced design are 30 and 60. That’s because 30 and 60 are the only two numbers that can be divided by 5, 3 and 2, all unique number of 5, 3, 3, 2. If the factors are, say, 3, 2, 3. Then the integers that can be divided by 3 and 2 are 6, 12, and 18.

Efficiency is a measure of the information content a design can capture. Efficiency are typically stated in relative terms, as in “design A is 80% as efficient as design B.” In practical terms this means you will need 25% more (the reciprocal of 80%) design A observations (respondents, choice sets per respondent or a combination of both) to get the same standard errors and significance as with the more efficient design B. We used a measure of efficiency called D-Efficiency (Kuhfeld et al. 1994). In this case, the only orthogonal design is the full factorial design of 60 trials.

For example, a trial of 30 has a efficiency of .984, that means it’s 98.4% as efficient as the full factorial design of 60 trials.

```
library(radiant)
ice_doe2 <- doe(attributes, seed = 123, trials = 30)
ice_doe2$cor_mat
```

|         | Flavor        | Package       | Light         | Organic       |
|---------|---------------|---------------|---------------|---------------|
| Flavor  | 1.000000e+00  | -1.942890e-16 | -1.665335e-16 | -1.665335e-16 |
| Package | -1.942890e-16 | 1.000000e+00  | -1.665335e-16 | -1.665335e-16 |
| Light   | -1.665335e-16 | -1.665335e-16 | 1.000000e+00  | -1.045299e-01 |
| Organic | -1.665335e-16 | -1.665335e-16 | -1.045299e-01 | 1.000000e+00  |

From the covariance matrix, we can see that in this 30 profile design, there is a correlation of -.105 between “Light” and “Organic”. That means these two effects are confounded; the design is not orthogonal. That’s why the efficiency is not 1. The D-efficiency is calculated as a function of the determinant of the correlation matrix. The D-efficiency of the full factorial design is 1. The D-efficiency of the 30 profile design is .984. That means the 30 profile design is 98.4% as efficient as the full factorial design.

The partial factorial design of 30 profile is

```
ice_doe2$part
```

|    | trial | Flavor     | Package         | Light      | Organic     |
|----|-------|------------|-----------------|------------|-------------|
| 1  | 1     | Raspberry  | Homemade_waffle | Low_fat    | Organic     |
| 2  | 4     | Raspberry  | Homemade_waffle | No_low_fat | Not_organic |
| 3  | 6     | Raspberry  | Cone            | Low_fat    | Not_organic |
| 4  | 7     | Raspberry  | Cone            | No_low_fat | Organic     |
| 5  | 10    | Raspberry  | Pint            | Low_fat    | Not_organic |
| 6  | 11    | Raspberry  | Pint            | No_low_fat | Organic     |
| 7  | 13    | Chocolate  | Homemade_waffle | Low_fat    | Organic     |
| 8  | 14    | Chocolate  | Homemade_waffle | Low_fat    | Not_organic |
| 9  | 19    | Chocolate  | Cone            | No_low_fat | Organic     |
| 10 | 20    | Chocolate  | Cone            | No_low_fat | Not_organic |
| 11 | 22    | Chocolate  | Pint            | Low_fat    | Not_organic |
| 12 | 23    | Chocolate  | Pint            | No_low_fat | Organic     |
| 13 | 26    | Strawberry | Homemade_waffle | Low_fat    | Not_organic |
| 14 | 28    | Strawberry | Homemade_waffle | No_low_fat | Not_organic |
| 15 | 29    | Strawberry | Cone            | Low_fat    | Organic     |

### 38. Conjoint design and analysis using R

|    |    |            |                 |            |             |
|----|----|------------|-----------------|------------|-------------|
| 16 | 32 | Strawberry | Cone            | No_low_fat | Not_organic |
| 17 | 33 | Strawberry | Pint            | Low_fat    | Organic     |
| 18 | 35 | Strawberry | Pint            | No_low_fat | Organic     |
| 19 | 39 | Mango      | Homemade_waffle | No_low_fat | Organic     |
| 20 | 40 | Mango      | Homemade_waffle | No_low_fat | Not_organic |
| 21 | 41 | Mango      | Cone            | Low_fat    | Organic     |
| 22 | 42 | Mango      | Cone            | Low_fat    | Not_organic |
| 23 | 46 | Mango      | Pint            | Low_fat    | Not_organic |
| 24 | 47 | Mango      | Pint            | No_low_fat | Organic     |
| 25 | 49 | Vanilla    | Homemade_waffle | Low_fat    | Organic     |
| 26 | 51 | Vanilla    | Homemade_waffle | No_low_fat | Organic     |
| 27 | 53 | Vanilla    | Cone            | Low_fat    | Organic     |
| 28 | 56 | Vanilla    | Cone            | No_low_fat | Not_organic |
| 29 | 58 | Vanilla    | Pint            | Low_fat    | Not_organic |
| 30 | 60 | Vanilla    | Pint            | No_low_fat | Not_organic |

## 38.4. Conjoint analysis

In the icecream data set we read in, it's a 10 profile design. There are 15 respondents. They rate 10 profiles of icecreams.

```
conjoint <- conjoint(icecream, rvar = "rating", evar = c("Flavor","Packaging","Light","Organic"),
summary(conjoint)
```

Conjoint analysis

Data : icecream

Response variable : rating

Explanatory variables: Flavor, Packaging, Light, Organic

Conjoint part-worths:

| Attributes     | Levels          | PW     |
|----------------|-----------------|--------|
| Flavor         | Chocolate       | 0.000  |
| Flavor         | Mango           | 1.522  |
| Flavor         | Raspberry       | 0.522  |
| Flavor         | Strawberry      | 0.767  |
| Flavor         | Vanilla         | 1.389  |
| Packaging      | Cone            | 0.000  |
| Packaging      | Homemade waffle | -0.244 |
| Packaging      | Pint            | -0.100 |
| Light          | Low fat         | 0.000  |
| Light          | No low fat      | 0.478  |
| Organic        | Not organic     | 0.000  |
| Organic        | Organic         | 0.307  |
| Base utility ~ |                 | 4.358  |

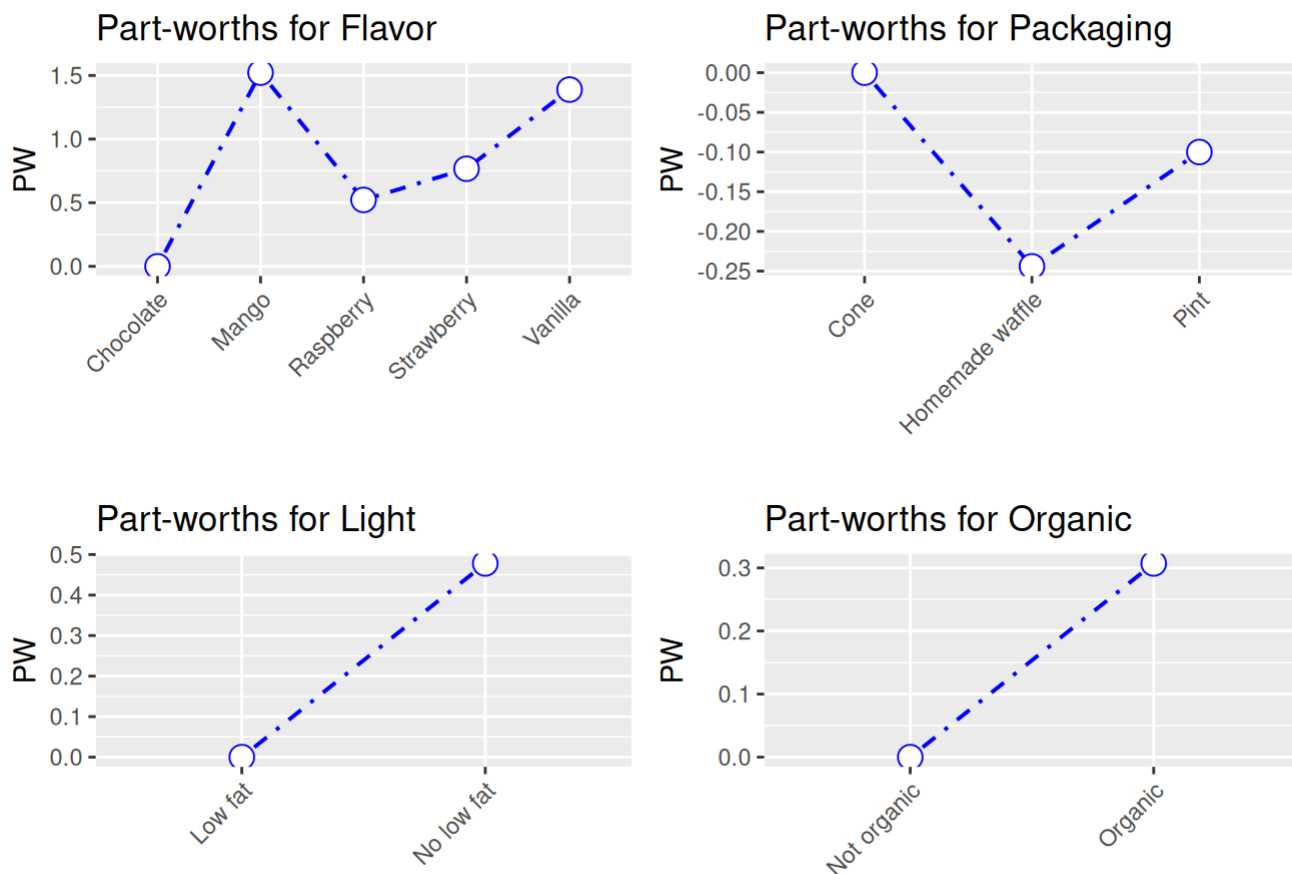
Conjoint importance weights:

| Attributes | IW    |
|------------|-------|
| Flavor     | 0.597 |
| Packaging  | 0.096 |
| Light      | 0.187 |
| Organic    | 0.120 |

Conjoint regression results:

|                           | coefficient |
|---------------------------|-------------|
| (Intercept)               | 4.358       |
| Flavor Mango              | 1.522       |
| Flavor Raspberry          | 0.522       |
| Flavor Strawberry         | 0.767       |
| Flavor Vanilla            | 1.389       |
| Packaging Homemade waffle | -0.244      |
| Packaging Pint            | -0.100      |
| Light No low fat          | 0.478       |
| Organic Organic           | 0.307       |

```
plot(conjoint)
```



The partworths are just coefficient from a linear regression:

```
summary(lm(rating ~ Flavor + Packaging + Light + Organic, data = icecream))
```

Call:

```
lm(formula = rating ~ Flavor + Packaging + Light + Organic, data = icecream)
```

Residuals:

| Min     | 1Q      | Median  | 3Q     | Max    |
|---------|---------|---------|--------|--------|
| -5.2867 | -2.2244 | -0.1911 | 2.3128 | 5.4089 |

Coefficients:

|                          | Estimate | Std. Error | t value | Pr(> t )     |
|--------------------------|----------|------------|---------|--------------|
| (Intercept)              | 4.3578   | 0.8172     | 5.332   | 3.76e-07 *** |
| FlavorMango              | 1.5222   | 0.9453     | 1.610   | 0.110        |
| FlavorRaspberry          | 0.5222   | 0.9453     | 0.552   | 0.582        |
| FlavorStrawberry         | 0.7667   | 0.8399     | 0.913   | 0.363        |
| FlavorVanilla            | 1.3889   | 0.9453     | 1.469   | 0.144        |
| PackagingHomemade waffle | -0.2444  | 0.8675     | -0.282  | 0.779        |



|                 |         |        |        |       |
|-----------------|---------|--------|--------|-------|
| PackagingPint   | -0.1000 | 0.8399 | -0.119 | 0.905 |
| LightNo low fat | 0.4778  | 0.5738 | 0.833  | 0.406 |
| OrganicOrganic  | 0.3067  | 0.4751 | 0.645  | 0.520 |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.91 on 141 degrees of freedom

Multiple R-squared: 0.03541, Adjusted R-squared: -0.01932

F-statistic: 0.6469 on 8 and 141 DF, p-value: 0.7371

The conjoint importance weights are the relative importance of each attribute. First the range of partworths for each attribute is calculated. Then the range of partworths for each attribute is divided by the sum of the ranges of all attributes. In this example, Flavor is the most important attribute, with an importance weight of .597.

There is another package called “conjoint” which I have not played with.

In summary, we can use R packages to set up a conjoint experiment, in the sense that we can have partial (fractional) factorial design. After collecting the data, we can use either the conjoint packages, or just run linear or logit models to analyze it. The frequentist approach used in conjoint analysis seems nothing special. Bayesian approach can also be used, but that’s another topic.

## 38.5. Causal effect in conjoint analysis

Hainmueller, Hopkins, and Yamamoto (2014) defined ACME (average marginal component effect). The basic idea is that once all the assumptions are met (usual assumptions in causal inference), we can have the effect of changing one attribute level to another with a causal interpretation. Fortunately it is usually just coefficients from a linear model, or linear combinations of the coefficients. There is a package “cregg” for each calculation and plotting of ACME, but we can just do it manually.



# 39. Stata 19 CATE command

## 39.1. Stata 19 CATE command

Stata 19 introduces a `cate` command for estimating conditional average treatment effects (CATE). [Fernando Rios-Avila](#) provides an overview.

⚠ If a Stata chunk here renders with no output

The Stata chunks below are cached (`cache: true`) because the `cate` command does lasso cross-fitting plus a random forest, and a full re-execution of this chapter takes over five minutes. The cache is safe for prose edits, which leave every chunk served uniformly from cache. It is **not** safe if you edit the *code* of one Stata chunk: the edited chunk re-executes while the earlier `collectcode: true` chunks are still served from cache, their code never enters Statamarkdown’s accumulated buffer, every command runs against missing data, and the chunk renders silently blank – and that empty result is then cached in turn. If that happens, delete `stata-cate_cache/` **entirely** (not just the offending chunk’s entry) along with `_freeze/stata-cate/`, then re-render so all chunks run in order. Clearing `_freeze/` alone is not enough. See the note in this book’s `CLAUDE.md`.

```
clear all
webuse assets3
global catecovars age educ i.(incomecat pension married twoearn ira ownhome)
cate po (assets $catecovars) (e401k), group(incomecat) nolog
```

(Excerpt from Chernozhukov and Hansen (2004))

|                                       |                              |   |       |
|---------------------------------------|------------------------------|---|-------|
| Conditional average treatment effects | Number of observations       | = | 9,913 |
| Estimator: Partialing out             | Number of folds in cross-fit | = | 10    |
| Outcome model: Linear lasso           | Number of outcome controls   | = | 17    |
| Treatment model: Logit lasso          | Number of treatment controls | = | 17    |
| CATE model: Random forest             | Number of CATE variables     | = | 17    |

| ----- |        |             |           |   |                            |
|-------|--------|-------------|-----------|---|----------------------------|
|       |        | Robust      |           |   |                            |
|       | assets | Coefficient | std. err. | z | P> z  [95% conf. interval] |
| ----- |        |             |           |   |                            |
| GATE  |        |             |           |   |                            |

### 39. Stata 19 CATE command

|             |  |          |          |       |       |           |          |
|-------------|--|----------|----------|-------|-------|-----------|----------|
| incomecat   |  |          |          |       |       |           |          |
| 0           |  | 3999.888 | 989.8742 | 4.04  | 0.000 | 2059.77   | 5940.006 |
| 1           |  | 1424.931 | 1668.081 | 0.85  | 0.393 | -1844.447 | 4694.31  |
| 2           |  | 5092.801 | 1344.769 | 3.79  | 0.000 | 2457.102  | 7728.499 |
| 3           |  | 8700.512 | 2272.126 | 3.83  | 0.000 | 4247.226  | 13153.8  |
| 4           |  | 20350.97 | 4717.093 | 4.31  | 0.000 | 11105.64  | 29596.3  |
| -----       |  |          |          |       |       |           |          |
| ATE         |  |          |          |       |       |           |          |
| e401k       |  |          |          |       |       |           |          |
| (Eligible   |  |          |          |       |       |           |          |
| vs          |  |          |          |       |       |           |          |
| Not elig..) |  | 7914.24  | 1150.79  | 6.88  | 0.000 | 5658.734  | 10169.75 |
| -----       |  |          |          |       |       |           |          |
| POmean      |  |          |          |       |       |           |          |
| e401k       |  |          |          |       |       |           |          |
| Not eligi.. |  | 14012.54 | 834.0963 | 16.80 | 0.000 | 12377.74  | 15647.34 |
| -----       |  |          |          |       |       |           |          |

The data are `assets3`: 9,913 households, with net financial assets as the outcome and 401(k) eligibility as the treatment. The group ATEs by income category show strong heterogeneity — \$4,000 in the lowest category, \$1,425 in the second (not significant), then \$5,093, \$8,701 and \$20,350 as income rises. A single ATE would hide a fivefold spread.

He said “We can achieve comparable results using standard regression with full interactions:”

```
clear all
webuse assets3
global catecovars age educ i.(incomecat pension married twoearn ira ownhome)

* you do not want to see all interactions.
qui:reg assets i.incomecat##i.e401k##c.($catecovars), robust

* Average Treatment Effect (ATE)
margins, at(e401k=(0 1)) noestimcheck contrast(atcontrast(r))

* Group Average Treatment Effects (GATEs)
margins, at(e401k=(0 1)) noestimcheck contrast(atcontrast(r)) over(incomecat)
```

(Excerpt from Chernozhukov and Hansen (2004))

Contrasts of predictive margins  
Model VCE: Robust

Number of obs = 9,913

Expression: Linear prediction, predict()

1.\_at: e401k = 0  
2.\_at: e401k = 1

|             | df   | F     | P>F    |
|-------------|------|-------|--------|
| _at         | 1    | 44.72 | 0.0000 |
| Denominator | 9833 |       |        |

|          | Contrast | std. err. | [95% conf. interval] |          |
|----------|----------|-----------|----------------------|----------|
| _at      |          |           |                      |          |
| (2 vs 1) | 7642.407 | 1142.797  | 5402.291             | 9882.524 |

Contrasts of predictive margins  
Model VCE: Robust

Number of obs = 9,913

Expression: Linear prediction, predict()

Over: incomecat

1.\_at: 0.incomecat  
          e401k = 0  
          1.incomecat  
              e401k = 0  
          2.incomecat  
              e401k = 0  
          3.incomecat  
              e401k = 0  
          4.incomecat  
              e401k = 0  
2.\_at: 0.incomecat  
          e401k = 1  
          1.incomecat  
              e401k = 1  
          2.incomecat  
              e401k = 1  
          3.incomecat  
              e401k = 1  
          4.incomecat  
              e401k = 1

|  | df | F | P>F |
|--|----|---|-----|
|--|----|---|-----|

### 39. Stata 19 CATE command

| -----+-----   |  |      |              |
|---------------|--|------|--------------|
| _at@incomecat |  |      |              |
| (2 vs 1) 0    |  | 1    | 16.08 0.0001 |
| (2 vs 1) 1    |  | 1    | 0.67 0.4123  |
| (2 vs 1) 2    |  | 1    | 16.03 0.0001 |
| (2 vs 1) 3    |  | 1    | 13.77 0.0002 |
| (2 vs 1) 4    |  | 1    | 17.26 0.0000 |
| Joint         |  | 5    | 12.76 0.0000 |
|               |  |      |              |
| Denominator   |  | 9833 |              |
| -----         |  |      |              |

| -----+-----   |  |              |           |                      |
|---------------|--|--------------|-----------|----------------------|
|               |  | Delta-method |           |                      |
|               |  | Contrast     | std. err. | [95% conf. interval] |
| -----+-----   |  |              |           |                      |
| _at@incomecat |  |              |           |                      |
| (2 vs 1) 0    |  | 3594.182     | 896.3298  | 1837.191 5351.172    |
| (2 vs 1) 1    |  | 1283.3       | 1565.151  | -1784.718 4351.317   |
| (2 vs 1) 2    |  | 5056.144     | 1262.728  | 2580.938 7531.35     |
| (2 vs 1) 3    |  | 8610.622     | 2320.156  | 4062.641 13158.6     |
| (2 vs 1) 4    |  | 19665.61     | 4733.551  | 10386.88 28944.34    |
| -----         |  |              |           |                      |

Numerically these two sets of estimations may be close, but I don't think they are from the same model, even the same idea. The CATE command with "PO" option is to estimate the CATE with "partialling out" approach.

What Fernando did is a "regression adjustment" type of model, which is modeling outcome model and allow interaction of treatment and covariates. The "partialling out" approach is sometimes called double ML.

Let's see how to replicate "teffects ra" with "reg":

```
clear all
webuse assets3
global catecovars age educ i.(incomecat pension married twoearn ira ownhome)

teffects ra (assets age educ i.(incomecat pension married twoearn ira ownhome))(e401k)

qui: reg assets i.e401k##c.(age educ) i.e401k##i.(incomecat pension married twoearn ira ownhome)
margins, at(e401k=(0 1)) noestimcheck contrast(atcontrast(r))
```

(Excerpt from Chernozhukov and Hansen (2004))

Iteration 0: EE criterion = 1.291e-21

Iteration 1: EE criterion = 1.046e-23

Treatment-effects estimation                      Number of obs       =       9,913  
 Estimator                : regression adjustment  
 Outcome model        : linear  
 Treatment model: none

|        |             |             | Robust    |       |       |                      |          |
|--------|-------------|-------------|-----------|-------|-------|----------------------|----------|
|        | assets      | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |          |
| ATE    |             |             |           |       |       |                      |          |
|        | e401k       |             |           |       |       |                      |          |
|        | (Eligible   |             |           |       |       |                      |          |
|        | vs          |             |           |       |       |                      |          |
|        | Not elig..) | 7929.242    | 1216.273  | 6.52  | 0.000 | 5545.39              | 10313.09 |
| POmean |             |             |           |       |       |                      |          |
|        | e401k       |             |           |       |       |                      |          |
|        | Not eligi.. | 14094.12    | 863.7641  | 16.32 | 0.000 | 12401.18             | 15787.07 |

Contrasts of predictive margins                      Number of obs = 9,913  
 Model VCE: OLS

Expression: Linear prediction, predict()

1.\_at: e401k = 0

2.\_at: e401k = 1

|             |     | df   | F     | P>F    |
|-------------|-----|------|-------|--------|
|             | _at | 1    | 34.39 | 0.0000 |
| Denominator |     | 9889 |       |        |

|  |          |          | Delta-method |                      |          |
|--|----------|----------|--------------|----------------------|----------|
|  |          | Contrast | std. err.    | [95% conf. interval] |          |
|  | _at      |          |              |                      |          |
|  | (2 vs 1) | 7929.242 | 1352.153     | 5278.746             | 10579.74 |

### 39.1.1. DoubleML

The R package DoubleML implements the double/debiased machine learning framework of Chernozhukov et al. (2018). It provides functionalities to estimate parameters in causal models based on machine learning methods. The double machine learning framework consist of three key ingredients: Neyman orthogonality, High-quality machine learning estimation and Sample splitting.

They consider a partially linear model:

$$y_i = \theta d_i + g_0(x_i) + \eta_i \quad (39.1)$$

$$d_i = m_0(x_i) + v_i \quad (39.2)$$

This model is quite general, except it does not allow interaction of  $d$  and  $x$ ; therefore no heterogeneous treatment effect across  $x$ . But “DoubleML” implements more than partially linear model, it actually allows for heterogeneous treatment effects, in models such as interactive regression model.

The basic idea of doubleML is to use any machine learning algorithm to estimate outcome equation ( $l_0(X) = E(Y|X)$ ) and treatment equation ( $m_0(X) = E(D|X)$ ). Then get the residuals, namely  $\hat{W} = Y - \hat{l}_0(X)$  and  $\hat{V} = D - \hat{m}_0(X)$ .

Then regress  $\hat{W}$  on  $\hat{V}$ . Based on FWL theorem, you get  $\hat{\theta}$ .

An important component here is to specify a Neyman-orthogonal score function. In the case of PLR, it can be the product of the two residuals:

$$\psi(W; \theta, \eta) = (Y - D\theta - g(X))(D - m(X)) \quad (39.3)$$

The estimators  $\hat{\theta}$  is to solve the equation that the sample mean of this score function being 0.

And the variance of this score function is used as the variance of the doubleML estimator’s variance.

Now we try to replicate the CATE estimation in Stata using the R package DoubleML. I have not found a way to include factor variables in the lasso regression in R, so I included them as numeric variables.

The example in stata’s cate command is to use lasso on both outcome and treatment regression, then random forest on the individual treatment effect to get ATE. Here I try to replicate it in R.

```
library(haven)
library(DoubleML)
library(mlr3)
library(mlr3learners)
library(data.table)
library(dplyr)
# get rid of labels because doubleML does not like them
data <- read_dta("https://www.stata-press.com/data/r19/assets3.dta") |>
  zap_labels()
```



```

# pension, married, twoearn, ira, ownhome are already binary 0/1, so
# passing them as numeric is fine. incomecat, however, has 5 unordered
# categories (0-4) -- passing it as a single numeric column would force
# the ML learners (especially the linear regr.glmnet below) to treat
# category 2 as twice category 1, which is not a meaningful assumption
# for a nominal variable. One-hot encode it into dummy columns instead.
incomecat_dummies <- model.matrix(~ as.factor(incomecat) - 1, data = data)
colnames(incomecat_dummies) <- paste0("incomecat_", 0:(ncol(incomecat_dummies) - 1))
data <- cbind(data, incomecat_dummies)

assets3 <- data.table::setDT(data)
dml_data = DoubleMLData$new(data = assets3,
                             y_col = "assets",
                             d_cols = "e401k",
                             x_cols = c("age", "educ", colnames(incomecat_dummies), "pension", "married", "twoearn", "ira", "ownhome"))

print(dml_data)

```

```
===== DoubleMLData Object =====
```

```

----- Data summary -----
Outcome variable: assets
Treatment variable(s): e401k
Covariates: age, educ, incomecat_0, incomecat_1, incomecat_2, incomecat_3, incomecat_4, pension, married, twoearn, ira, ownhome
Instrument(s):
Selection variable:
No. Observations: 9913

```

```

lgr::get_logger("mlr3")$set_threshold("warn")
learner = lrn("regr.glmnet", lambda=1)
ml_g_sim = learner$clone()
ml_m_sim = learner$clone()
set.seed(123)
obj_dml_plr = DoubleMLPLR$new(dml_data, ml_l=ml_g_sim, ml_m=ml_m_sim, n_folds=10)
obj_dml_plr$fit()
obj_dml_plr$summary()

```

Estimates and significance testing of the effect of target variables

|       | Estimate. | Std. Error | t value | Pr(> t )     |
|-------|-----------|------------|---------|--------------|
| e401k | 7553      | 1281       | 5.897   | 3.69e-09 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

DoubleML with lasso learners gives an ATE of \$7,553 with a standard error of \$1,281.

### 39. Stata 19 CATE command

If we use random forest on the two models:

```
# suppress messages from mlr3 package during fitting
lgr::get_logger("mlr3")$set_threshold("warn")
learner = lrn("regr.ranger", num.trees=2000, max.depth=5, min.node.size=2)
ml_l = learner$clone()
ml_m = learner$clone()
# learner = lrn("regr.glmnet")
# ml_g_sim = learner$clone()
# ml_m_sim = learner$clone()
set.seed(123)
obj_dml_plr = DoubleMLPLR$new(dml_data, ml_l=ml_l, ml_m=ml_m, n_folds=5)
obj_dml_plr$fit()
obj_dml_plr$summary()
```

Estimates and significance testing of the effect of target variables

```
      Estimate. Std. Error t value Pr(>|t|)
e401k      9114      1375   6.627 3.43e-11 ***
---
```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

We can do the same in Stata:

```
clear all
webuse assets3
global catecovars age educ incomecat pension married twoearn ira ownhome
cate po (assets age educ incomecat pension married twoearn ira ownhome) (e401k), omethod(rfor
```

(Excerpt from Chernozhukov and Hansen (2004))

|                                       |                              |         |
|---------------------------------------|------------------------------|---------|
| Conditional average treatment effects | Number of observations       | = 9,913 |
| Estimator: Partialing out             | Number of folds in cross-fit | = 5     |
| Outcome model: Random forest          | Number of outcome controls   | = 8     |
| Treatment model: Random forest        | Number of treatment controls | = 8     |
| CATE model: Random forest             | Number of CATE variables     | = 8     |

|           |        | Robust      |           |   |                            |
|-----------|--------|-------------|-----------|---|----------------------------|
|           | assets | Coefficient | std. err. | z | P> z  [95% conf. interval] |
| ATE       |        |             |           |   |                            |
|           | e401k  |             |           |   |                            |
| (Eligible |        |             |           |   |                            |
| vs        |        |             |           |   |                            |

|             |  |          |          |       |       |          |          |
|-------------|--|----------|----------|-------|-------|----------|----------|
| Not elig..) |  | 8367.508 | 1180.69  | 7.09  | 0.000 | 6053.398 | 10681.62 |
| -----       |  |          |          |       |       |          |          |
| P0mean      |  |          |          |       |       |          |          |
| e401k       |  |          |          |       |       |          |          |
| Not eligi.. |  | 13999.41 | 842.9698 | 16.61 | 0.000 | 12347.22 | 15651.6  |
| -----       |  |          |          |       |       |          |          |

With random forests DoubleML gives \$9,474 (SE \$1,376), while Stata's `cate` with `rforest` for both models gives \$7,126 (SE \$1,182). The results are somewhat similar, but not exactly the same — a gap of about \$2,300, which is larger than either standard error. There are some subtle differences in the way these two packages running random forest on the two models that I am not aware of.

## 39.2. AIPW

Stata's `cate` also has “aipw” option.

```
clear all
webuse assets3
global catecovars age educ incomecat pension married twoearn ira ownhome
cate aipw (assets $catecovars) (e401k), nolog
```

(Excerpt from Chernozhukov and Hansen (2004))

|                                       |                              |         |
|---------------------------------------|------------------------------|---------|
| Conditional average treatment effects | Number of observations       | = 9,913 |
| Estimator: Augmented IPW              | Number of folds in cross-fit | = 10    |
| Outcome model: Linear lasso           | Number of outcome controls   | = 8     |
| Treatment model: Logit lasso          | Number of treatment controls | = 8     |
| CATE model: Random forest             | Number of CATE variables     | = 8     |

|        |             | Robust      |           |       |       |                      |          |
|--------|-------------|-------------|-----------|-------|-------|----------------------|----------|
|        | assets      | Coefficient | std. err. | z     | P> z  | [95% conf. interval] |          |
| <hr/>  |             |             |           |       |       |                      |          |
| ATE    |             |             |           |       |       |                      |          |
|        | e401k       |             |           |       |       |                      |          |
|        | (Eligible   |             |           |       |       |                      |          |
|        | vs          |             |           |       |       |                      |          |
|        | Not elig..) | 7126.343    | 1182.229  | 6.03  | 0.000 | 4809.217             | 9443.469 |
| <hr/>  |             |             |           |       |       |                      |          |
| P0mean |             |             |           |       |       |                      |          |
|        | e401k       |             |           |       |       |                      |          |
|        | Not eligi.. | 14136.14    | 857.5555  | 16.48 | 0.000 | 12455.36             | 15816.92 |
| <hr/>  |             |             |           |       |       |                      |          |

### 39. Stata 19 CATE command

Stata's AIPW gives an ATE of \$7,929 with a standard error of \$1,216, on 9,913 households.

I'll try to do this with "npcausal":

```
library(npcausal)
#SL.library <- c("SL.earth","SL.gam","SL.glmnet","SL.glm.interaction", "SL.mean","SL.ranger",
#SL.library <- c("SL.glmnet")
SL.lasso = function(...) {
  SL.glmnet(..., alpha=1)
}
SL.library <- c("SL.lasso")

Y <- data$assets
A <- data$e401k
# incomecat is nominal (5 categories); use the one-hot dummies created above
# rather than the raw numeric column, for the same reason as in the
# DoubleML example.
# as.data.frame() is required: `data` is a tibble, so `data[, cols]` returns a
# tibble, and npcausal::ate() fails on one with
# Error in rep(): invalid 'times' argument
# It needs a plain data.frame.
# Two coercions are needed here, both for npcausal/SuperLearner's benefit:
# * as.data.frame(): `data` is a tibble, and ate() fails on one with
# Error in rep(): invalid 'times' argument
# * as.numeric on every column: read_dta() returns haven_labelled columns, and
# SuperLearner then errors with
# The variable names and order in newX must be identical to ... X
W <- as.data.frame(data[, c("age", "educ", grep("^incomecat_", names(data), value = TRUE), "per
W <- as.data.frame(lapply(W, as.numeric))
# npcausal::ate() currently fails on this design with
# Error in SuperLearner(): The variable names and order in newX must be
# identical to the variable names and order in X
# raised from inside npcausal's own cross-fitting loop, not from anything in this
# chunk (the equivalent call works on a synthetic data.frame of the same shape).
# Wrapped so the chapter degrades gracefully rather than publishing a raw error;
# the DoubleML comparison above is the working cross-fitted estimate.
aipw <- tryCatch(
  ate(y = Y, a = A, x = W, nsplits = 10, sl.lib = SL.library),
  error = function(e) {
    message("npcausal::ate() failed: ", conditionMessage(e))
    NULL
  }
)
```

|  
|

| 0%

```
if (!is.null(aipw)) aipw$res
```

Note here to match the Stata results, I included only “glmnet” in the SuperLearner library. The results are not exactly the same, but close enough.

Or we can use a similar package with more functionalities: “AIPW”.

```
library(AIPW)
library(SuperLearner)

#SuperLearner libraries for outcome (Q) and exposure models (g)
#sl.lib <- c("SL.glmnet")

AIPW_sl <- AIPW$new(Y= Y,
                   A= A,
                   W= W,
                   Q.SL.library = SL.library,
                   g.SL.library = SL.library,
                   k_split = 10,
                   verbose=FALSE)
```

```
Error in `value[[3L]]()` :
! Covariates dimension error: nrow(W) != length(A)
```

```
AIPW_sl$fit()
```

```
Error:
! object 'AIPW_sl' not found
```

```
AIPW_sl$summary()
```

```
Error:
! object 'AIPW_sl' not found
```

```
AIPW_sl$result
```

```
Error:
! object 'AIPW_sl' not found
```

```
# library(ggplot2)
# AIPW_sl$plot.p_score()
# AIPW_sl$plot.ip_weights()
```

---

*Systematic treatment: [R](#) · [Julia](#).*



## 40. Spatial econometrics introduction

### 40.1. Why do we need spatial econometrics?

‘Everything is related to everything else, but near things are more related than distant things’ (Tobler 1970). This is the first law of geography. This is the reason why we need spatial econometrics. The spatial econometrics is to account for the spatial dependence in the data. The spatial dependence is the correlation between the observations that are close to each other. This is a common feature in many datasets. For example, in the housing market, the price of a house is related to the prices of the houses nearby. In the crime data, the crime rate in one neighborhood is related to the crime rate in the neighboring neighborhoods. In the health data, the health outcomes of people in one area are related to the health outcomes of people in the neighboring areas. In the environmental data, the pollution level in one area is related to the pollution level in the neighboring areas. In the economic data, the economic performance of one region is related to the economic performance of the neighboring regions. In the social network data, the behavior of one person is related to the behavior of the people in the neighboring areas.

Econometricians love OLS. It is worth being precise about *what* spatial dependence breaks. Spatially correlated **errors** alone (the term  $u = \rho W u + \epsilon$  below, with exogenous regressors) do not make the OLS coefficients inconsistent — they make the conventional standard errors wrong, so inference is invalid even though the point estimates are fine. OLS coefficients become **biased/inconsistent** in the genuinely simultaneous or omitted-variable cases: a spatially lagged *outcome*  $Wy$  on the right-hand side (endogenous by construction), omitted spatially correlated confounders, or endogenous spillovers. The sections below treat these cases separately: S2SLS for the spatial-lag-of-outcome model, and spatial-error models for the correlated-error case.

### 40.2. A general linear spatial model

$$y = \lambda Wy + X\beta + WX\gamma + u \quad (40.1)$$

$$u = \rho Wu + \epsilon \quad (40.2)$$

where  $Y$  is the dependent variable,  $X$  is the matrix of independent variables,  $\beta$  is the vector of coefficients, and  $u$  is the error term. The error term  $u$  is spatially correlated.

We can see the major difference is  $W$ . The term  $Wy$  is called “spatially lagged  $y$ ”, which in my opinion is misleading. This term is likely from time-series counter-part of lagged  $y$ , but there is nothing about lags in the spatial data.  $W$  is the spatial weights matrix. It is a matrix that describes the spatial structure of the data. The simplest example of  $W$  is the case that we define an element of  $W$ ,  $w_{i,j}$  to be 1 if unit  $i$  and  $j$  are neighbors, 0 if not.

In this simplest case, our model is saying we have a pattern we need to take account of: neighboring units are related, they seem to display similar pattern in terms of our study. This commonality of neighbors can be causal, for example, if we study the effect of minimum wage effect on employment, then if the neighboring state's wage increase might negatively impact the focal state's employment, because people go for higher wages. This is the spatial spillover effect. This can also be non-causal, for example, if we study the effect of temperature on crop yield, then the neighboring state's temperature might be a good predictor of the focal state's temperature. This is the spatial autocorrelation. In either case, we need to take account of the spatial structure in the data.

The reason behind the spatial pattern can be all kinds. It can be  $x_i$  causing  $y_j$ , or  $y_i$  causing  $y_j$ , or some unobserved factors causing both  $y_i$  and  $y_j$ , or some other reasons. Ignoring this spatial structure could make OLS biased. Betz et al (2020) show that ignoring spatial interdependence in the outcome results in asymptotically biased estimates even when instruments are randomly assigned.

### 40.3. Estimation with S2SLS for a less general model

Because we have the term  $Wy$ , we introduced endogeneity.  $Wy$  is necessarily correlated with  $u$ , even if  $u$  is not spatially correlated. Suppose  $u$  is not spatially correlated.

The spatial two stage least squares (S2SLS) estimator is one solution. Usually we use  $X$  and spatial lags of  $X$  as instruments. Typically instruments  $H$  is  $(X, WX, W^2X)$ .

S2SLS is similar to 2SLS. The first stage is

$$\hat{Z} = PZ = [X, WX, \hat{W}y] \quad (40.3)$$

where

$$\hat{W}y = PWy \quad (40.4)$$

and

$$P = H(H'H)^{-1}H'$$

{#eq-spatial-econometrics-5}.

Then

$$\hat{\theta}_{S2SLS} = (\hat{Z}'Z)^{-1}\hat{Z}'y \quad (40.5)$$

where  $\hat{\theta}_{S2SLS}$  is the full coefficient vector  $(\beta, \gamma, \lambda)$ .

The first stage is to regress  $WY$  on  $H$ . The second stage is to regress  $Y$  on the predicted  $WY$  from the first stage and  $X$  and  $WX$ . The coefficient of  $X$  is the S2SLS estimator.

### 40.4. Estimation with a more general model

Suppose instead the error term is spatially correlated

$$u = \rho Wu + \epsilon \quad (40.6)$$

Then the estimation is more complicated. The generalized spatial two stage least squares (GS2SLS) estimator is needed. See Kelejian and Prucha (1998) for details.



## 40.5. an example

This example is from R package “sphet”.

The Boston data contain information on property values and characteristics in the area of Boston, Massachusetts and has been widely used for illustrating spatial models. Specifically, there is a total of 506 units of observation for each of which a variety of attributes are available, such as: (corrected) median values of owner-occupied homes (CMEDV); per-capita crime rate (CRIM); nitric oxides concentration (NOX); average number of rooms (RM); proportions of residential land zoned for lots over 25,000 sq. ft (ZN); proportions of non-retail business acres per town (INDUS); Charles River dummy variable (CHAS); proportions of units built prior to 1940 (AGE); weighted distances to five Boston employment centers (DIS); index of accessibility to highways (RAD); property-tax rate (TAX); pupil-teacher ratios (PTRATIO); proportion of blacks (B); and % of the lower status of the population (LSTAT).

The dataset with Pace’s tract coordinates is available to the R community as part of “spdep”.

The spatial weights matrix is a sphere of influence neighbors list also available from spdep once the Boston data are loaded. Here the weight matrix comes with the data. Usually it’s the most heavy lifting part of the study.

```
library(sphet)
library(spdep)
data("boston", package = "spData")
listw <- nb2listw(boston.soi)
```

The function read.gwt2dist reads a ‘GWT’ file (e.g., generated using the function distance). In this example we are using the matrix of tract point coordinates projected to UTM zone 19 boston.utm available from spdep to generate a ‘GWT’ file of the 10 nearest neighbors.

```
id1 <- seq(1, nrow(boston.utm))
tmp <- distance(boston.utm, region.id = id1, output = TRUE,
               type = "NN", nn = 10, shape.name = "shapefile",
               region.id.name = "id1", firstline = TRUE,
               file.name = "boston_nn_10.GWT")
coldist <- read.gwt2dist(file = "boston_nn_10.GWT", region.id = id1, skip = 1)
class(coldist)
```

```
[1] "sphet"      "distance" "nb"        "GWT"
```

```
summary(coldist)
```

```
Number of observations:
n: 506
```

```
Distance summary:
```

#### 40. Spatial econometrics introduction

| Min.   | 1st Qu. | Median | Mean   | 3rd Qu. | Max.    |
|--------|---------|--------|--------|---------|---------|
| 0.5441 | 0.9588  | 1.5843 | 2.0848 | 2.6389  | 11.6388 |

Neighbors summary:

| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|------|---------|--------|------|---------|------|
| 10   | 10      | 10     | 10   | 10      | 10   |

The `boston` data are 506 census tracts. The 10-nearest-neighbour weights give every tract exactly 10 neighbours, with distances running from 0.54 to 11.64 and a median of 1.58.

Spatial models in `sphet` are fitted by the functions `stslshac` and `gstslshet`. “`stslshac`” is Spatial two stages least square with HAC standard errors. “`gstslshet`” is GM estimation of a Cliff-Ord type model with Heteroskedastic Innovations, which is the general model we discussed before, with spatially correlated errors.

The theoretical model above is the Spatial Durbin Model (SDM), which includes  $WX$  as regressors (with their own coefficient  $\gamma$ ), not just as instruments for  $Wy$ . If we only pass  $X$  (no  $WX$ ) in the formula below, `stslshac`’s `W2X` argument still uses spatial lags of  $X$  internally, but only as *instruments* for the endogenous  $Wy - \gamma$  is never estimated, so the fitted model is actually the simpler Spatial Autoregressive (SAR) model ( $\gamma = 0$ ), not SDM. To match the SDM specification, include  $WX$  explicitly as regressors:

```
boston_x_vars <- c("CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B"
boston.c2 <- boston.c
for (v in boston_x_vars) {
  xv <- boston.c[[v]]
  if (is.factor(xv)) xv <- as.numeric(as.character(xv))
  boston.c2[[paste0("W_", v)]] <- lag.listw(listw, xv)
}

sdm_formula <- as.formula(paste(
  "log(CMEDV) ~ CRIM + ZN + INDUS + CHAS + I(NOX^2) + I(RM^2) + AGE + log(DIS) + log(RAD) + TA
  paste0("W_", boston_x_vars, collapse = " + ")
))

res <- stslshac(sdm_formula,
  data = boston.c2, listw,
  distance = coldist, type = "Triangular", HAC = TRUE)
summary(res)
```

```
=====
=====
                        Spatial Lag Model
                        HAC standard errors
=====
=====
```

Call:

```
stslshac(formula = sdm_formula, data = boston.c2, listw = listw,
          HAC = TRUE, distance = coldist, type = "Triangular")
```

Residuals:

| Min.     | 1st Qu.  | Median   | 3rd Qu. | Max.    |
|----------|----------|----------|---------|---------|
| -0.71074 | -0.06258 | -0.00291 | 0.06279 | 0.66519 |

Coefficients:

|             | Estimate    | SHAC       | St.Er.  | t-value   | Pr(> t ) |  |
|-------------|-------------|------------|---------|-----------|----------|--|
| Wy          | 0.71491435  | 0.09797746 | 7.2967  | 2.949e-13 | ***      |  |
| (Intercept) | 1.74597474  | 0.46867690 | 3.7253  | 0.0001951 | ***      |  |
| CRIM        | -0.00548837 | 0.00153211 | -3.5822 | 0.0003407 | ***      |  |
| ZN          | 0.00081016  | 0.00045400 | 1.7845  | 0.0743454 | .        |  |
| INDUS       | -0.00070720 | 0.00285254 | -0.2479 | 0.8041961 |          |  |
| CHAS1       | -0.05854830 | 0.03984589 | -1.4694 | 0.1417328 |          |  |
| I(NOX^2)    | -0.03240666 | 0.21228222 | -0.1527 | 0.8786677 |          |  |
| I(RM^2)     | 0.00846160  | 0.00289595 | 2.9219  | 0.0034793 | **       |  |
| AGE         | -0.00150525 | 0.00047265 | -3.1847 | 0.0014489 | **       |  |
| log(DIS)    | -0.16117615 | 0.07522729 | -2.1425 | 0.0321515 | *        |  |
| log(RAD)    | 0.04219650  | 0.01648649 | 2.5595  | 0.0104835 | *        |  |
| TAX         | -0.00045089 | 0.00012647 | -3.5652 | 0.0003636 | ***      |  |
| PTRATIO     | -0.01336079 | 0.00581140 | -2.2991 | 0.0215013 | *        |  |
| B           | 0.00058124  | 0.00011965 | 4.8578  | 1.187e-06 | ***      |  |
| log(LSTAT)  | -0.21713640 | 0.03831649 | -5.6669 | 1.454e-08 | ***      |  |
| W_CRIM      | -0.00451534 | 0.00300618 | -1.5020 | 0.1330927 |          |  |
| W_ZN        | -0.00133281 | 0.00058617 | -2.2738 | 0.0229807 | *        |  |
| W_INDUS     | 0.00079183  | 0.00351396 | 0.2253  | 0.8217152 |          |  |
| W_CHAS      | 0.09604488  | 0.06127241 | 1.5675  | 0.1169964 |          |  |
| W_NOX       | -0.34866680 | 0.32382701 | -1.0767 | 0.2816112 |          |  |
| W_RM        | -0.06511363 | 0.05723288 | -1.1377 | 0.2552474 |          |  |
| W_AGE       | 0.00175310  | 0.00062322 | 2.8130  | 0.0049086 | **       |  |
| W_DIS       | 0.01948831  | 0.01383400 | 1.4087  | 0.1589164 |          |  |
| W_RAD       | 0.00203459  | 0.00345353 | 0.5891  | 0.5557722 |          |  |
| W_TAX       | 0.00028960  | 0.00019481 | 1.4866  | 0.1371284 |          |  |
| W_PTRATIO   | 0.00732717  | 0.00807107 | 0.9078  | 0.3639669 |          |  |
| W_B         | -0.00054543 | 0.00015750 | -3.4630 | 0.0005342 | ***      |  |
| W_LSTAT     | 0.00895071  | 0.00427009 | 2.0961  | 0.0360698 | *        |  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

The spatial lag coefficient  $Wy$  is 0.715 with a HAC standard error of 0.098,  $t = 7.3$  — strong spatial dependence in house values, which is what motivates the whole exercise. Among the own-tract regressors,  $\log(LSTAT)$  enters at  $-0.217$  and  $RM^2$  at 0.0085.

The function `gstslshet` is used to estimate the GS2SLS model. Unlike `stslshac`, `gstslshet`'s GMM machinery hits an exactly-singular matrix on this dataset once the  $WX$  regressors are added (likely due to near-collinearity introduced by the additional lagged terms combined with its own internal

#### 40. Spatial econometrics introduction

instrument construction), so we keep the original (SAR, no  $WX$ ) specification here — this estimates the spatial-error model with  $\gamma = 0$ , not the full Durbin extension shown above for `stslshac`.

```
res <- gsts1shet(log(CMEDV) ~ CRIM + ZN + INDUS + CHAS + I(NOX^2) + I(RM^2)
               + AGE + log(DIS) + log(RAD) + TAX + PTRATIO + B + log(LSTAT),
               data = boston.c, listw = listw, initial.value = 0.2)
summary(res)
```

Call:

```
gsts1shet(formula = log(CMEDV) ~ CRIM + ZN + INDUS + CHAS + I(NOX^2) +
          I(RM^2) + AGE + log(DIS) + log(RAD) + TAX + PTRATIO + B +
          log(LSTAT), data = boston.c, listw = listw, initial.value = 0.2)
```

Residuals:

| Min.      | 1st Qu.   | Median    | Mean     | 3rd Qu.  | Max.     |
|-----------|-----------|-----------|----------|----------|----------|
| -0.569385 | -0.073165 | -0.001675 | 0.000531 | 0.071500 | 0.740309 |

Coefficients:

|             | Estimate    | Std. Error | t-value | Pr(> t )      |
|-------------|-------------|------------|---------|---------------|
| (Intercept) | 2.51316605  | 0.26749367 | 9.3952  | < 2.2e-16 *** |
| CRIM        | -0.00662744 | 0.00144522 | -4.5858 | 4.523e-06 *** |
| ZN          | 0.00038299  | 0.00036563 | 1.0475  | 0.2948733     |
| INDUS       | 0.00159352  | 0.00179772 | 0.8864  | 0.3753967     |
| CHAS1       | -0.00447974 | 0.03689065 | -0.1214 | 0.9033480     |
| I(NOX^2)    | -0.27295899 | 0.11561412 | -2.3609 | 0.0182283 *   |
| I(RM^2)     | 0.00744059  | 0.00199637 | 3.7270  | 0.0001937 *** |
| AGE         | -0.00045400 | 0.00045572 | -0.9962 | 0.3191333     |
| log(DIS)    | -0.16517174 | 0.03484858 | -4.7397 | 2.140e-06 *** |
| log(RAD)    | 0.07453521  | 0.01752830 | 4.2523  | 2.116e-05 *** |
| TAX         | -0.00041956 | 0.00010763 | -3.8981 | 9.697e-05 *** |
| PTRATIO     | -0.01412661 | 0.00410143 | -3.4443 | 0.0005725 *** |
| B           | 0.00035970  | 0.00011182 | 3.2168  | 0.0012964 **  |
| log(LSTAT)  | -0.24593826 | 0.03213364 | -7.6536 | 1.954e-14 *** |
| lambda      | 0.42407826  | 0.04463747 | 9.5005  | < 2.2e-16 *** |
| rho         | 0.29587455  | 0.08614291 | 3.4347  | 0.0005932 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

This specification separates the two channels the previous one merged.  $\rho$ , the spatial lag on  $y$ , is 0.296 with a standard error of 0.086, and  $\lambda$ , the spatial correlation in the errors, is 0.424 with 0.045. Both are significant. Note  $\rho = 0.296$  here against  $W_y = 0.715$  above: once spatially correlated errors are allowed for, much of what the lag model attributed to neighbours' outcomes is reassigned to correlated unobservables. That is the identification problem in spatial econometrics, visible in two numbers.

In stata, you would use “spivregress” which replaced user-written “spivreg”. Stata has a set of commands for spatial econometrics. Check out stata’s “sp” manual.

## 41. Causal simulation

Evans and Didelez (2024), “Parameterizing and simulating from causal models,” address the inverse problem: given a causal model, what joint distribution does it imply? The usual direction is observational data to causal structure to parameter estimates; this paper starts from the structure and asks what data it would generate.

Consider the joint distribution  $p(x, a, y)$ , where  $x$  is covariates,  $a$  is treatment, and  $y$  is outcome.

$$\begin{aligned}
 p(x, a, y) &= p(x, a)p(y|x, a) \\
 &= p(x, a)p_a(y|x) \\
 &= p(x, a)\frac{p_a(x, y)}{p_a(x)} \\
 &= p(x, a)\frac{p_a(x)p_a(y)c(x, y|a)}{p_a(x)} \\
 &= p(x, a)p_a(y)c(x, y|a)
 \end{aligned} \tag{41.1}$$

The joint distribution factorizes into the joint of  $(x, a)$ , the interventional marginal  $p_a(y) = p(y | do(a))$ , and the copula  $c(x, y | a)$ . The second line is where the causal content enters, and it is not an algebraic identity: replacing the observational conditional  $p(y | x, a)$  by the interventional  $p_a(y | x)$  requires the causal model to make them agree – consistency together with exchangeability given  $x$ , or the equivalent structural condition. For an arbitrary observed joint distribution the two differ, and the rest of the factorization inherits that assumption. The derivation starts from  $p(y | x, a)$  and absorbs the  $x$ -dependence into the copula, leaving only the marginal  $p_a(y)$ . Each piece can be specified separately:  $(x, a)$  as the “past,”  $p_a(y)$  as the marginal structural model, and the copula as the dependence structure between  $x$  and  $y$  given  $a$ .

A copula links a joint distribution to its marginals:

$$p(x, y) = f(x)g(y)c(F(x), G(y)) \tag{41.2}$$

where  $F(x)$  and  $G(y)$  are the marginal distribution functions of  $x$  and  $y$ . Keep the two objects apart: the copula  $C$  is a distribution function on  $[0, 1]^2$  with uniform marginals, and  $c$  in Equation 41.2 is its *density*, which is what multiplies  $f(x)g(y)$ . It can be used to generate joint distributions from marginal distributions.

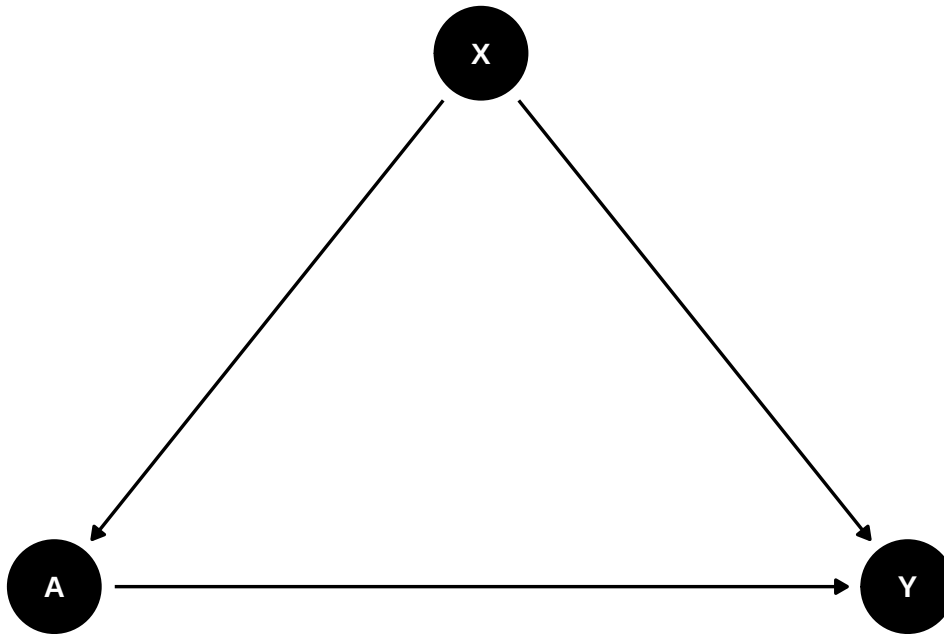
Here  $x$ ,  $a$  and  $y$  can all be vectors.

## 41.1. examples

What do we use this for? We can use this to simulate data from a causal model, and then fit different models to the simulated data. This allows to compare the performance of different models.

### 41.1.1. example 1

Suppose we have a causal graph like this:



We need to specify  $P(y|do(a)) = \sum_x P(x)P(y|x, a)$ , where  $P(x)$  is the marginal distribution of  $x$ , and  $P(y|x, a)$  is the conditional distribution of  $y$  given  $x$  and  $a$ . This is the “marginal structural model” (Robins, 2000). Also the “g-formula”.

We also need to specify  $P(x, a)$ , the “past”.

Finally we need to specify the copula  $c(x, y|a)$ , which captures the dependence between  $x$  and  $y$  given  $a$ . Depending on different situations, different copula can be used.

In the “causl” package example, we specify:

$$X \sim N(0, 1) \tag{41.3}$$

$$A|X = x \sim N(x/2, 1) \tag{41.4}$$

$$Y|do(A = a) \sim N((a - 1)/2, 1) \tag{41.5}$$

and Gaussian copula with correlation  $\rho = 2\expit(1) - 1$ .

```

library(causl)
# formulae corresponding to covariates, treatments, outcomes and the dependence
forms <- list(X ~ 1, A ~ X, Y ~ A, ~ 1)
# vector of model families (3=gamma/exponential, 1=normal/Gaussian)
fam <- c(1, 1, 1, 1)
# list of parameters, including 'beta' (regression params) and 'phi' dispersion
pars <- list(X = list(beta=0, phi=1),
             A = list(beta=c(0,0.5), phi=1),
             Y = list(beta=c(-0.5,0.5), phi=1),
             cop = list(beta=1))

## now create a `causl_model` object
cm <- causl_model(formulas=forms, family=fam, pars=pars,method="inversion")

# now simulate 1000 observations
set.seed(123456)
data <- rfrugal(n=1000, causl_model=cm)
head(data)

```

|   | X           | A           | Y          |
|---|-------------|-------------|------------|
| 1 | 0.83373317  | 1.10709111  | 1.1416713  |
| 2 | -0.27604777 | -0.27218853 | -1.0495996 |
| 3 | -0.35500184 | 0.77575209  | 1.3179678  |
| 4 | 0.08748742  | -0.05959823 | -0.6106023 |
| 5 | 2.25225573  | 0.61909216  | 2.0789188  |
| 6 | 0.83446013  | 0.52019822  | 0.3229069  |

In this example, first we specify the structure of the model using a list of formulas. The first formula is for the covariates, the second for the treatments, the third for the outcomes, and the fourth for the dependence structure (copula). Note that outcome  $Y$  only depends on  $A$  in this interventional distribution.

Then we specify the families of the random variables. Here we use normal distribution for  $X$ , normal distribution for  $A$  and  $Y$ , and Gaussian copula for the dependence structure.

Finally we specify the parameters of the model, including regression coefficients and dispersion parameters. For example,  $Y \sim A$  has two coefficients, intercept  $-0.5$  and slope  $.5$ .

The `causl_model` object is then created using these components.

In this simple example, if we run a regression with  $A$ ,  $X$  in the model, on the simulated data, we would get the correct effect back:

```

lm1 <- lm(Y ~ A*X, data=data)
summary(lm1)

```

Call:

#### 41. Causal simulation

```
lm(formula = Y ~ A * X, data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-2.6326 -0.6212 -0.0105  0.6051  3.3132

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.491922   0.030413  -16.175  <2e-16 ***
A             0.493471   0.027820   17.738  <2e-16 ***
X             0.468038   0.031878   14.682  <2e-16 ***
A:X          -0.009247   0.022360   -0.414    0.679
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.8953 on 996 degrees of freedom
Multiple R-squared:  0.4875,    Adjusted R-squared:  0.4859
F-statistic: 315.8 on 3 and 996 DF,  p-value: < 2.2e-16
```

The true interventional model is  $Y \mid do(A = a) \sim N((a - 1)/2, 1)$ , so the true intercept is  $-0.5$  and the true coefficient on  $A$  is  $0.5$ . The naive regression of  $Y$  on  $A$  and  $X$  gives  $A = 0.493$  and an intercept of  $-0.492$ , with the interaction  $A:X$  at  $-0.009$  and insignificant ( $p = 0.68$ ), which is correct — the DGP has no such interaction.

Note this regression *also* includes  $X$ , which is the adjustment the g-formula calls for. Its coefficient of  $0.468$  is not a causal quantity; it is there to close the backdoor.

We can also use MLE:

```
out <- fit_causl(data, formulas = list(X ~ 1, Y ~ A, ~ 1), family = c(1, 1, 1))
out
```

```
log-likelihood: -2716.696
X ~ 1
      est.   s.e. sandwich
(intercept) 0.0108 0.0314   0.0314
residual s.e.: 0.983 0.044 0.0447

Y ~ A
      est.   s.e. sandwich
(intercept) -0.491 0.0319   0.0319
A           0.494 0.0278   0.0295
residual s.e.: 1.01 0.0472 0.0475

copula parameters:
cop ~ 1
      est.   s.e. sandwich
(intercept) 0.998 0.069   0.0734
```



`fit_causl` recovers the same structure directly from the frugal parameterisation: intercept  $-0.491$  and  $A$  at  $0.494$ , against the true  $-0.5$  and  $0.5$ , with a copula parameter of  $0.998$  against the specified  $1$ . The sandwich standard errors barely differ from the model-based ones, which is what should happen when the model is correctly specified.

### 41.1.2. Plasmode simulation

In reality, we have a data set and we have a causal model. We want to simulate data from the causal model, but we want the simulated data to have the same distribution as the original data. This is called “plasmode simulation”. For example, we don’t want to use the real outcome variable. Instead, we simulate  $X$ ’s from the original data, Then simulate  $Y$  and  $A$  from the causal model. We can then test which method works better in estimating the causal effect of  $A$  on  $Y$ .

```
# A tibble: 4,802 x 58
  x_1 x_2 x_3 x_4 x_5 x_6 x_7 x_8 x_9 x_10 x_11 x_12 x_13
<int> <fct> <dbl> <dbl> <int> <int> <int> <int> <int> <int> <int> <int> <int>
1    29 C      1    7   60   85    0    0    1    0    0    1    0
2    27 C      0    0   64  178    0    0    0    0    0    2    1
3    27 C      0    0   60  102    0    0    0    0    0    1    0
4    37 C      0    0   65  174    0    0    0    0    0    1    0
5    24 C     20   14   63  129    0    0    0    0    0    1    0
6    27 C     40   15   63  135    0    0    0    0    0    1    1
7    26 C     20    8   69  140    0    0    0    1    0    0    0
8    33 C      0    0   60  110    0    0    0    1    0    0    1
9    28 C      3    5   61  160    0    0    0    0    0    3    1
10   31 C      0    0   63  114    0    1    0    0    0    0    0
# i 4,792 more rows
# i 45 more variables: x_14 <int>, x_15 <int>, x_16 <int>, x_17 <int>,
# x_18 <int>, x_19 <int>, x_20 <int>, x_21 <fct>, x_22 <int>, x_23 <int>,
# x_24 <fct>, x_25 <int>, x_26 <int>, x_27 <int>, x_28 <int>, x_29 <int>,
# x_30 <int>, x_31 <int>, x_32 <int>, x_33 <int>, x_34 <int>, x_35 <int>,
# x_36 <int>, x_37 <int>, x_38 <int>, x_39 <int>, x_40 <int>, x_41 <int>,
# x_42 <int>, x_43 <int>, x_44 <int>, x_45 <int>, x_46 <int>, x_47 <int>, ...

# Model for the causal effect of smoking on birthweight
forms <- list(list(),
  list(A ~ x_1 + x_3 + x_4),
  list(Y ~ A),
  list(~ 1))
# fams <- list(integer(0), 5, 1, 1)
fams <- list(integer(0), "binomial", "gaussian", 1)
pars <- list(A = list(beta=c(-1.5,0.03,0.02,0.05)),
  Y = list(beta=c(3200, -500), phi=400^2),
  cop = list(beta=-1))

cm2 <- causl_model(formulas=forms, family=fams, pars=pars, dat = dat)
```

## 41. Causal simulation

```
set.seed(123456)
data2 <- rfrugal(causal_model=cm2)
head(data2)
```

|   | x_1 | x_2 | x_3 | x_4 | x_5 | x_6 | x_7 | x_8 | x_9 | x_10 | x_11 | x_12 | x_13 | x_14 | x_15 | x_16 | x_17 |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|------|------|------|
| 1 | 29  | C   | 1   | 7   | 60  | 85  | 0   | 0   | 1   | 0    | 0    | 1    | 0    | 0    | 2    | 0    | 0    |
| 2 | 27  | C   | 0   | 0   | 64  | 178 | 0   | 0   | 0   | 0    | 0    | 2    | 1    | 0    | 2    | 0    | 0    |
| 3 | 27  | C   | 0   | 0   | 60  | 102 | 0   | 0   | 0   | 0    | 0    | 1    | 0    | 0    | 0    | 0    | 0    |
| 4 | 37  | C   | 0   | 0   | 65  | 174 | 0   | 0   | 0   | 0    | 0    | 1    | 0    | 0    | 1    | 0    | 0    |
| 5 | 24  | C   | 20  | 14  | 63  | 129 | 0   | 0   | 0   | 0    | 0    | 1    | 0    | 0    | 0    | 0    | 0    |
| 6 | 27  | C   | 40  | 15  | 63  | 135 | 0   | 0   | 0   | 0    | 0    | 1    | 1    | 2    | 2    | 0    | 0    |

|   | x_18 | x_19 | x_20 | x_21 | x_22 | x_23 | x_24 | x_25 | x_26 | x_27 | x_28 | x_29 | x_30 | x_31 | x_32 |
|---|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 1 | 12   | 35   | 10   | J    | 1    | 43   | B    | 17   | 64   | 105  | 30   | 11   | 0    | 0    | 0    |
| 2 | 12   | 75   | 12   | J    | 1    | 50   | E    | 12   | 90   | 183  | 40   | 8    | 0    | 0    | 0    |
| 3 | 12   | 35   | 10   | J    | 1    | 57   | E    | 14   | 70   | 121  | 27   | 12   | 0    | 0    | 0    |
| 4 | 11   | 35   | 12   | J    | 1    | 43   | E    | 10   | 80   | 185  | 39   | 10   | 0    | 0    | 0    |
| 5 | 9    | 35   | 15   | J    | 1    | 33   | E    | 14   | 70   | 160  | 28   | 7    | 0    | 0    | 0    |
| 6 | 9    | 45   | 10   | J    | 1    | 40   | E    | 15   | 64   | 160  | 31   | 9    | 0    | 0    | 0    |

|   | x_33 | x_34 | x_35 | x_36 | x_37 | x_38 | x_39 | x_40 | x_41 | x_42 | x_43 | x_44 | x_45 | x_46 | x_47 |
|---|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 1 | 0    | 79   | 28   | 340  | 48   | 1    | 9    | 9    | 3    | 0    | 3    | 54   | 16   | 0    | 0    |
| 2 | 0    | 69   | 30   | 430  | 73   | 0    | 9    | 9    | 3    | 0    | 5    | 51   | 16   | 0    | 0    |
| 3 | 0    | 84   | 29   | 450  | 76   | 0    | 3    | 6    | 2    | 0    | 4    | 41   | 13   | 0    | 0    |
| 4 | 0    | 82   | 30   | 440  | 78   | 1    | 8    | 9    | 2    | 0    | 8    | 70   | 16   | 0    | 0    |
| 5 | 1    | 85   | 35   | 360  | 71   | 1    | 5    | 8    | 0    | 2    | 3    | 65   | 18   | 0    | 0    |
| 6 | 1    | 81   | 28   | 515  | 56   | 1    | 7    | 9    | 2    | 0    | 1    | 63   | 20   | 0    | 0    |

|   | x_48 | x_49 | x_50 | x_51 | x_52 | x_53 | x_54 | x_55 | x_56 | x_57 | x_58 | A | Y        |
|---|------|------|------|------|------|------|------|------|------|------|------|---|----------|
| 1 | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 45   | 39   | 1 | 2125.677 |
| 2 | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 46   | 42   | 1 | 2689.114 |
| 3 | 0    | 1    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 45   | 40   | 0 | 3442.800 |
| 4 | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 47   | 40   | 0 | 3212.584 |
| 5 | 0    | 2    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 47   | 43   | 0 | 2992.775 |
| 6 | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 45   | 44   | 0 | 2738.893 |

```
# we can also use rfrugalParam(), which is older version of rfrugal().
#datAY <- rfrugalParam(formulas=forms, family=fams, pars=pars, dat=dat)
```

In this specification, we have a binary treatment  $A$  and a continuous outcome  $Y$ . The covariates are  $x_1$ ,  $x_3$  and  $x_4$ . As in the first example above, the copula's **beta** parameter maps to the actual Gaussian correlation via  $\rho = 2 \expit(\beta) - 1$ ; with **beta=-1** this gives  $\rho = 2 \expit(-1) - 1 \approx -0.46$ , not  $-1$  (a correlation of exactly  $-1$  would require  $\beta \rightarrow -\infty$  and would make the joint distribution singular). We can then simulate the data.

### 41.2. a more complicated model

Let's simulate a more complicated model:

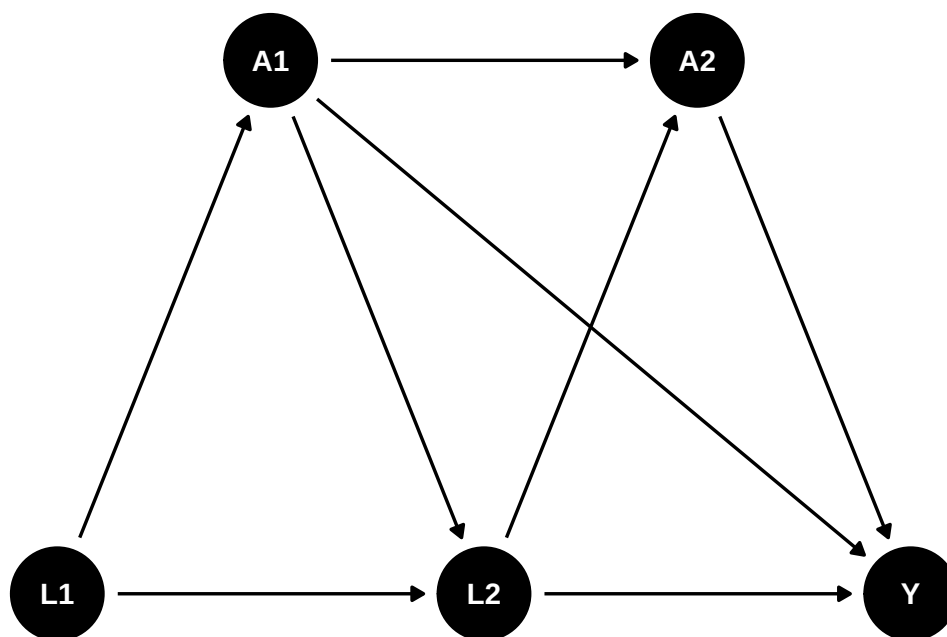
```

library(ggplot2)
library(ggdag)

g <- dagify(
  A1 ~ L1,
  L2 ~ L1,
  L2 ~ A1,
  A2 ~ A1,
  A2 ~ L2,
  Y ~ A2,
  Y ~ L2,
  Y ~ A1,
  exposure = "A2",
  outcome = "Y",
  coords = list(x = c(L1 = 1, L2 = 2, Y = 3, A1 = 1.5, A2 = 2.5),
                y = c(L1 = 1, L2 = 1, Y = 1, A1 = 1.5, A2 = 1.5))
)

ggdag(g) +
  theme_dag()

```



Let's see how to write a causal model:

```

# formulae corresponding to covariates, treatments, outcomes and the dependence
forms <- list(list(L1 ~ 1, L2 ~ L1*A1), # covariates
              list(A1 ~ L1, A2 ~ L2*A1), # treatments
              Y ~ A1*A2*L2,              # outcome
              ~ 1)

```

#### 41. Causal simulation

```
# vector of model families (3=gamma/exponential, 1=normal/Gaussian)

fams <- list(c(1, 1), c(5,5), 1, 1)
pars <- list(L1 = list(beta=0, phi=1),
            L2 = list(beta=c(0.3,0.5,-0.2,-0.1), phi=1),
            A1 = list(beta=c(-0.3,0.4), phi=1),
            A2 = list(beta=c(0.5,0.3,0.1,0), phi=1),
            Y = list(beta=c(0,1,2,0.5,1,0.2,0,0.8), phi=1),
            cop = list(beta = 0.5))

# we can use rfrugalParam() to simulate data from the model, or rfrugal().
#dat <- rfrugalParam(n=1e4, formulas=forms, family = fams, pars=pars)
#head(dat)

cm3 <- causl_model(formulas=forms, family=fams, pars=pars)

set.seed(123456)
data3 <- rfrugal(n=1e4,causl_model=cm3)
head(data3)
```

|   | L1          | L2          | A1 | A2 | Y         |
|---|-------------|-------------|----|----|-----------|
| 1 | 0.83373317  | 1.91691936  | 1  | 1  | 8.1680612 |
| 2 | -0.27604777 | 0.17008294  | 1  | 1  | 5.2276976 |
| 3 | -0.35500184 | 0.16907708  | 1  | 1  | 2.8194835 |
| 4 | 0.08748742  | 0.19688985  | 0  | 1  | 2.6789661 |
| 5 | 2.25225573  | -0.29257431 | 0  | 0  | 0.3180188 |
| 6 | 0.83446013  | 0.02661196  | 0  | 1  | 1.5694426 |

If we have the correct specification for the outcome model, we'll get it right by linear model:

```
m1 <- glm(Y ~ A1*A2*L2, data = data3)
summary(m1)
```

Call:

```
glm(formula = Y ~ A1 * A2 * L2, data = data3)
```

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | -0.11524 | 0.02110    | -5.461  | 4.85e-08 *** |
| A1          | 1.11589  | 0.03245    | 34.391  | < 2e-16 ***  |
| A2          | 2.02332  | 0.02682    | 75.454  | < 2e-16 ***  |
| L2          | 0.80424  | 0.01929    | 41.682  | < 2e-16 ***  |
| A1:A2       | 0.98345  | 0.04090    | 24.044  | < 2e-16 ***  |
| A1:L2       | 0.16028  | 0.02976    | 5.386   | 7.37e-08 *** |

```
A2:L2      -0.03446    0.02401  -1.435    0.151
A1:A2:L2    0.83541    0.03693  22.620 < 2e-16 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for gaussian family taken to be 0.9100255)
```

```
Null deviance: 53902  on 9999  degrees of freedom
Residual deviance: 9093  on 9992  degrees of freedom
AIC: 27446
```

```
Number of Fisher Scoring iterations: 2
```

```
library(marginaleffects)
avg_comparisons(
  model = m1,
  variables = "A2")
```

```
Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
      2.51      0.0202 124  <0.001 Inf  2.47  2.55
```

```
Term: A2
Type: response
Comparison: 1 - 0
```

The fitted model has  $A_2 = 2.023$ ,  $A_1:A_2 = 0.983$  and  $A_1:A_2:L_2 = 0.835$ , so the effect of  $A_2$  depends on both  $A_1$  and  $L_2$  and cannot be read off any single coefficient. `avg_comparisons` averages over the observed distribution and returns 2.51 with a standard error of 0.020.

So the ATE is 2.5.

Let's use "lmt" package to estimate the treatment effect of  $A_2$  on  $Y$ .

```
library(lmt)

d1 <- function(data, trt) {
  rep(1, nrow(data))
}
A <- "A2"
Y <- "Y"
W <- c("L1", "L2", "A1")

set.seed(34465)

treat <- lmt_tmle(
  data = data3,
```

#### 41. Causal simulation

```
trt = "A2",
outcome = "Y",
baseline = W,
outcome_type = "continuous",
shift = d1,
folds = 1,
learners_trt = "SL.glm",
learners_outcome = "SL.glm"
)

show_lmtp(treat)
```

Estimate: 3.046  
Std. error: 0.022

```
d2 <- function(data, trt) {
  rep(0, nrow(data))
}

control <- lmtp_tmle(
  data = data3,
  trt = "A2",
  outcome = "Y",
  baseline = W,
  outcome_type = "continuous",
  shift = d2,
  folds = 1,
  learners_trt = "SL.glm",
  learners_outcome = "SL.glm"
)

show_lmtp(control)
```

Estimate: 0.542  
Std. error: 0.021

```
show_lmtp(lmtp_contrast(treat, ref = control))
```

|   | shift | ref   | estimate | std.error | conf.low | conf.high | p.value |
|---|-------|-------|----------|-----------|----------|-----------|---------|
| 1 | 3.05  | 0.542 | 2.5      | 0.0227    | 2.46     | 2.55      | <0.001  |

## 42. Frengression and Engression

In the [causal simulation chapter](#), I used Evans and Didelez (2024)’s decomposition of the joint distribution  $p(x, a, y)$  into three pieces:

$$p(x, a, y) = p(x, a) \cdot p_a(y) \cdot c(x, y|a) \quad (42.1)$$

where  $p(x, a)$  is the “past” distribution of covariates and treatment,  $p_a(y)$  is the causal margin, and  $c(x, y|a)$  is the copula linking covariates and outcome given treatment. The `causl` package implements this with parametric families. We specify Gaussian, binomial, and so on for each piece.

But real data can be hard to specify parametrically. We may not know the right copula family. The marginal distributions may have mass points, skewness, or other features that are annoying to write down.

Frengression (Yang, Evans and Shen, 2025) keeps the same decomposition but learns the pieces with neural networks.

There are two uses: causal simulation with a learned generative model, and interventional prediction using learned confounding structure. The second part uses engression, so I start there.

### 42.1. Engression

Engression (Shen and Meinshausen, 2024) stands for energy regression. Standard regression estimates  $E[Y|X]$ . Engression tries to learn the whole conditional distribution  $P(Y|X)$  using neural networks trained with the energy score.

#### 42.1.1. The Energy Score

The energy score is a proper scoring rule for distributions. Given observations  $Y$  and samples  $\hat{Y}, \hat{Y}'$  from the model:

$$S(Y, \hat{Y}, \hat{Y}') = E\|Y - \hat{Y}\| - \frac{1}{2}E\|\hat{Y} - \hat{Y}'\| \quad (42.2)$$

The first term rewards samples close to the truth. The second term keeps the simulated distribution from collapsing to one point. This is why the method targets the full distribution rather than only the mean.

### 42.1.2. The Stochastic Network (StoNet)

The building block is a **StoNet**, a neural network that adds random noise  $\epsilon \sim N(0, I)$  to the input at each layer:

$$\text{StoNet}(x) = f(x, \epsilon), \quad \epsilon \sim N(0, I) \quad (42.3)$$

Different noise draws produce different outputs. Each output is a sample from the learned conditional distribution. So we can:

- **Predict the mean:** average over many noise draws  $\rightarrow E[Y|X = x]$
- **Predict quantiles:** take quantiles of the noise draws  $\rightarrow$  conditional quantile regression
- **Sample:** each forward pass with fresh noise is a sample from  $P(Y|X = x)$

### 42.1.3. Why Engression?

Compared with quantile regression or mixture density networks, engression is useful because:

1. it handles multivariate responses directly;
2. it does not require us to specify a Gaussian mixture or a list of quantiles;
3. it gives samples from the conditional distribution, which is convenient for simulation.

### 42.1.4. Quick Example

The **engression** R package, separate from **Rfrengression**, provides the standalone estimator. The code below shows the API. The **Rfrengression** examples later are the ones I actually run here.

```
library(engression)

# Nonlinear DGP with heteroskedastic noise
n <- 1000
X <- matrix(rnorm(n * 2), ncol = 2)
Y <- (X[,1])^2 + X[,2] + rnorm(n) * (0.5 + abs(X[,1]))

# Fit engression; it learns the full conditional distribution
engr <- engression(X, Y)

# Predict conditional mean (like lm, but nonlinear)
Yhat_mean <- predict(engr, X, type = "mean")

# Predict conditional quantiles (like quantile regression, but joint)
Yhat_quant <- predict(engr, X, type = "quantiles")

# Sample from the learned distribution (generative)
Ysample <- predict(engr, X, type = "sample", nsample = 1)
```

The point is that engression gives a distributional model of  $Y|X$ , not just a fitted mean.



## 42.2. From Engression to Frengression

Frengression is frugal simulation plus engression. It uses three engression models to represent the decomposition of  $P(X, A, Y)$ . Each sub-network is trained with the energy score.

## 42.3. The Architecture

Frengression has three sub-networks, matching the three pieces of the decomposition:

| Decomposition            | causl (parametric)           | Frengression (neural net)                                         |
|--------------------------|------------------------------|-------------------------------------------------------------------|
| $P(X, A)$ — the past     | Specified by family + params | <code>model_xz</code> : noise-only generator                      |
| $P_a(Y)$ — causal margin | Specified by family + params | <code>model_y</code> : takes $(x, \text{eta})$ ,<br>outputs $y$   |
| $c(X, Y   A)$ — copula   | Gaussian/Frank/etc. copula   | <code>model_eta</code> : takes $(x, z)$ ,<br>outputs $\text{eta}$ |

The important part is `model_eta`. It takes  $(x, z)$  and outputs a latent variable `eta` that carries the dependence between confounders and outcome. During training, a Z-permutation trick forces `eta` to be marginally  $N(0, I)$ . This means:

- When `eta` comes from `model_eta(x, z)`, it carries the observational confounding information.
- When `eta` is sampled independently from  $N(0, 1)$ , it gives the interventional prediction.

Same network, different source of `eta`. That is the trick.

## 42.4. Using Rfrengression

The `Rfrengression` package is a native R implementation using `torch`.

```
library(Rfrengression)
```

### 42.4.1. Example: Binary Treatment DGP

Use a DGP with binary treatment, binary outcome, and four confounders. We draw  $n = 2000$  observations with  $w_1, w_2 \sim \text{Bernoulli}(0.5)$ ,  $w_3 \sim U(0, 4)$  and  $w_4 \sim U(0, 5)$ ; treatment is  $A \sim \text{Bernoulli}(\Lambda(-0.4 + 0.2w_2 + 0.15w_3 + 0.2w_4 + 0.15w_2w_4))$ ; and both potential outcomes are Bernoulli with the same logit index shifted by the treatment. Since both are simulated, the realized ATE is known:

```

set.seed(42)
n <- 2000

w1 <- rbinom(n, 1, 0.5)
w2 <- rbinom(n, 1, 0.5)
w3 <- round(runif(n, 0, 4), 3)
w4 <- round(runif(n, 0, 5), 3)

A <- rbinom(n, 1, plogis(-0.4 + 0.2*w2 + 0.15*w3 + 0.2*w4 + 0.15*w2*w4))
lin <- -1 - 0.1*w1 + 0.3*w2 + 0.25*w3 + 0.2*w4 + 0.15*w2*w4
p1 <- plogis(lin + 1)
p0 <- plogis(lin)
Y.1 <- rbinom(n, 1, p1)
Y.0 <- rbinom(n, 1, p0)
Y <- Y.1 * A + Y.0 * (1 - A)

# Two benchmarks, and they are not the same quantity. The first is the realized
# finite-sample ATE in this particular draw of the Bernoulli outcomes; the second
# averages the true individual probabilities over the sampled covariates, so it
# carries no outcome sampling noise.
true_ate <- mean(Y.1 - Y.0)
bench_ate <- mean(p1 - p0)
cat("Realized ATE (this draw):", round(true_ate, 4), "\n")

```

Realized ATE (this draw): 0.1935

```
cat("Covariate-averaged ATE: ", round(bench_ate, 4), "\n")
```

Covariate-averaged ATE: 0.2024

#### 42.4.2. Train Frengression

Training has two stages: first the outcome model (`model_y + model_eta`), then the marginal model (`model_xz`).

```

# torch keeps its own generator; R's set.seed() does not reach it.
torch::torch_manual_seed(1)

x_mat <- matrix(A, ncol = 1)
z_mat <- as.matrix(data.frame(w1, w2, w3, w4))
y_mat <- matrix(Y, ncol = 1)

model <- frengression(x_dim = 1, y_dim = 1, z_dim = 4,
                      x_binary = TRUE, y_binary = TRUE,
                      z_binary_dims = 2, noise_dim = 10)

```

```
model <- train_y(model, x_mat, z_mat, y_mat,
                 num_iters = 1000, lr = 1e-3, print_every = 500)
```

```
Epoch 1: loss 1.2606, loss_y 0.4787 (0.4977, 0.0380), loss_eta 0.7818 (0.8583, 0.1528)
Stopping at iter 67
```

```
model <- train_xz(model, x_mat, z_mat,
                 num_iters = 1000, lr = 1e-4, print_every = 500)
```

```
Epoch 1: loss 3.4470, loss1 3.6066, loss2 0.3192
Epoch 500: loss 1.3948, loss1 2.7241, loss2 2.6587
Epoch 1000: loss 1.3395, loss1 2.7058, loss2 2.7327
```

### 42.4.3. Estimate ATE

For interventional prediction, `eta` is sampled from  $N(0,1)$  instead of being generated from the observed confounding structure:

```
torch::torch_manual_seed(1)

y1 <- predict(model, matrix(1, ncol = 1), type = "mean", nsample = 2000)
y0 <- predict(model, matrix(0, ncol = 1), type = "mean", nsample = 2000)

cat("Frengression ATE:", round(y1 - y0, 4), "\n")
```

```
Frengression ATE: 0.165
```

```
cat("True ATE:", round(true_ate, 4), "\n")
```

```
True ATE: 0.1935
```

The realized ATE in this draw is 0.1935, and averaging the true probabilities over the sampled covariates gives 0.2024. The gap between those two is outcome sampling noise, not model error, and it is worth keeping them apart: the realized figure is the ATE of the potential outcomes actually drawn, while the covariate-averaged figure is the smoother target the estimator is really chasing. Frengression lands a few hundredths below either one.

The training chunks carry `cache: false`, so they re-run on every render, and torch keeps a generator of its own that R's `set.seed()` does not reach. Seeding it with `torch_manual_seed()` fixes the weight initialisation, the training draws, and the sampling inside `predict()`, so the figure above is reproducible from one render to the next. On a CUDA device some nondeterminism can survive the seed, from cuDNN algorithm selection and atomic accumulation in the backward pass.

## 42.5. Frengression as DGP

Frengression can estimate an ATE, but that is not its main advantage. If I only want an ATE, I would usually use `npcausal`, `TMLE`, or `DoubleML`. The more interesting use is to train a data-generating process for benchmarking.

The idea is:

1. Train frengression on observational data, so it learns the confounding structure.
2. Replace the causal margin with a known function via `specify_causal()`.
3. Generate synthetic data with realistic confounding and known ground truth.

```
# Specify a known causal effect: Y = sigmoid(0.2*X + eta)
model_bench <- specify_causal(model, function(x, eta) {
  torch::torch_sigmoid(0.2 * x + eta)
})

# True ATE computed empirically from the model
y1_true <- predict(model_bench, matrix(1, ncol = 1), type = "mean", nsample = 5000)
y0_true <- predict(model_bench, matrix(0, ncol = 1), type = "mean", nsample = 5000)
cat("True ATE (specified):", round(as.numeric(y1_true - y0_true), 4), "\n")
```

True ATE (specified): 0.0831

Now we can generate synthetic data and benchmark any method:

```
syn <- sample_joint(model_bench, sample_size = 1000)
cat("Synthetic data: n =", nrow(syn$x), "\n")
```

Synthetic data: n = 1000

```
cat("Treatment prevalence:", mean(syn$x), "\n")
```

Treatment prevalence: 0.891

```
cat("Outcome prevalence:", mean(syn$y), "\n")
```

Outcome prevalence: 0.572

## 42.6. Real Data: LaLonde

This is most useful with real data. The LaLonde job-training data have many of the problems that make simulations hard to write by hand: zeros in prior earnings, uneven covariate distributions, and poor overlap.

```

library(MatchIt)
data("lalonde", package = "MatchIt")
lalonde$black <- as.integer(lalonde$race == "black")
lalonde$hispan <- as.integer(lalonde$race == "hispan")

x_lal <- matrix(lalonde$treat, ncol = 1)
y_lal <- matrix(lalonde$re78, ncol = 1)
# frengression's z_binary_dims assumes the FIRST z_binary_dims columns of z
# are the binary ones -- so the binary covariates (black, hispan, married,
# nodegree) must come first, followed by the continuous ones (age, educ,
# re74, re75). The original column order (age, educ first) put two
# continuous variables where binary ones were expected, and vice versa.
z_lal <- as.matrix(lalonde[, c("black", "hispan", "married", "nodegree",
                              "age", "educ", "re74", "re75")])

# Standardize for training
z_sc <- scale(z_lal)
y_sc <- (y_lal - mean(y_lal)) / sd(y_lal)

model_lal <- frengression(x_dim = 1, y_dim = 1, z_dim = 8,
                          x_binary = TRUE, y_binary = FALSE,
                          z_binary_dims = 4, noise_dim = 10)

model_lal <- train_y(model_lal, x_lal, z_sc, y_sc,
                      num_iters = 1000, lr = 1e-3, print_every = 500)

```

Epoch 1: loss 1.4330, loss\_y 0.7412 (0.7899, 0.0974), loss\_eta 0.6918 (0.7855, 0.1875)  
 Stopping at iter 52

```

model_lal <- train_xz(model_lal, x_lal, z_sc,
                      num_iters = 1000, lr = 1e-4, print_every = 500)

```

Epoch 1: loss 2.7586, loss1 2.9617, loss2 0.4063  
 Epoch 500: loss 2.0119, loss1 3.8498, loss2 3.6759  
 Epoch 1000: loss 1.9657, loss1 3.8158, loss2 3.7002

Now specify a known effect and generate data:

```

model_lal_bench <- specify_causal(model_lal, function(x, eta) {
  0.3 * x + eta
})

# Generate synthetic LaLonde-like data with known ATE = 0.3 (standardized)
syn_lal <- sample_joint(model_lal_bench, sample_size = 2000)
cat("Synthetic LaLonde: n =", nrow(syn_lal$x), "\n")

```

## 42. Frengression and Engression

Synthetic LaLonde:  $n = 2000$

```
cat("Treatment prevalence:", round(mean(syn_lal$x), 3), "\n")
```

Treatment prevalence: 0.007

```
cat("True ATE:", 0.3, "(standardized) =", round(0.3 * sd(y_lal), 0), "dollars\n")
```

True ATE: 0.3 (standardized) = 2241 dollars

The synthetic data inherits LaLonde's confounding structure, but now the true effect is known. This is the part that is hard to do with a hand-written DGP.

### 42.7. Comparison: `causal` vs `frengression`

Both packages use the same decomposition:

| Feature                    | <code>causal</code>      | <code>Frengression</code>         |
|----------------------------|--------------------------|-----------------------------------|
| Marginal $P(X, A)$         | Parametric families      | Learned by neural net             |
| Causal margin $P_a(Y)$     | Parametric families      | Learned (or specified)            |
| Copula                     | Gaussian, Frank, etc.    | Learned by <code>model_eta</code> |
| Requires specification     | Yes — families + params  | No — learns from data             |
| Can learn from real data   | Limited (plasmode)       | Yes — full joint                  |
| Distributional flexibility | Limited by family choice | Arbitrary                         |

The `causal` approach is more transparent because we know exactly what we specified. `Frengression` is more flexible because it can learn structure we did not write down. I would treat them as complements.

### 42.8. Benchmarking: `Frengression` vs Semiparametric Methods

Now compare `frengression` against standard semiparametric causal inference tools on the same data. The three comparison methods are:

- `npcausal`: AIPW estimator with SuperLearner nuisance models
- `tmle`: targeted minimum loss-based estimation with SuperLearner
- `DoubleML`: partially linear regression with `ranger`

These methods are built for ATE estimation. They should usually be better for this specific estimand. The comparison is still useful because `frengression` is trying to learn much more than the ATE.

```
library(npcausal)
library(tmle)
library(SuperLearner)
library(DoubleML)
library(mlr3)
library(mlr3learners)
library(tidyverse)
library(knitr)
```

### 42.8.1. Single-Dataset Comparison

First, run all four methods on the binary DGP data created above.

```
W <- data.frame(w1, w2, w3, w4)
SL.library <- c("SL.earth", "SL.glm.interaction", "SL.mean",
               "SL.ranger", "SL.glmnet")

set.seed(123)
aipw_fit <- ate(y = Y, a = A, x = W, nsplits = 2, sl.lib = SL.library)
```

```
|
|
|
|=====| 12%
|
|=====| 25%
|
|=====| 38%
|
|=====| 50%
|
|=====| 62%
|
|=====| 75%
|
|=====| 88%
|
|=====| 100%

  parameter      est      se    ci.ll    ci.ul  pval
1   E{Y(0)} 0.5697312 0.02038159 0.5297832 0.6096791    0
2   E{Y(1)} 0.7608322 0.01207030 0.7371744 0.7844900    0
3 E{Y(1)-Y(0)} 0.1911010 0.02335102 0.1453330 0.2368690    0
```

## 42. Frengression and Engression

```
ate_npcausal <- aipw_fit$res$est[3]
se_npcausal  <- aipw_fit$res$se[3]
ci_npcausal  <- c(aipw_fit$res$ci.ll[3], aipw_fit$res$ci.ul[3])

cat("npcausal ATE:", round(ate_npcausal, 4), "\n")
```

npcausal ATE: 0.1911

```
cat("SE:", round(se_npcausal, 4), "\n")
```

SE: 0.0234

```
set.seed(123)
tmle_fit <- tmle(Y = Y, A = A, W = W, family = "binomial",
                Q.SL.library = SL.library, g.SL.library = SL.library)

ate_tmle <- tmle_fit$estimates$ATE$psi
se_tmle  <- sqrt(tmle_fit$estimates$ATE$var.psi)
ci_tmle  <- tmle_fit$estimates$ATE$CI

cat("TMLE ATE:", round(ate_tmle, 4), "\n")
```

TMLE ATE: 0.1916

```
cat("SE:", round(se_tmle, 4), "\n")
```

SE: 0.0209

```
lgr::get_logger("mlr3")$set_threshold("warn")
data_obs <- data.frame(w1, w2, w3, w4, A, Y)

dml_data <- DoubleMLData$new(data_obs,
                             y_col = "Y", d_cols = "A",
                             x_cols = c("w1", "w2", "w3", "w4"))

learner_l <- lrn("regr.ranger", num.trees = 500, max.depth = 5, min.node.size = 2)
learner_m <- learner_l$clone()

set.seed(123)
dml_plr <- DoubleMLPLR$new(dml_data, ml_l = learner_l, ml_m = learner_m)
dml_plr$fit()

ate_dml <- dml_plr$coef
se_dml  <- dml_plr$se
```



```
ci_dml <- c(dml_plr$confint()[1], dml_plr$confint()[2])

cat("DoubleML ATE:", round(ate_dml, 4), "\n")
```

DoubleML ATE: 0.1872

```
cat("SE:", round(se_dml, 4), "\n")
```

SE: 0.0229

```
# Frengression ATE already computed above (y1 - y0)
ate_freng <- as.numeric(y1 - y0)

# Bootstrap SE via repeated prediction (Monte Carlo variability)
set.seed(123)
ate_boot <- replicate(200, {
  predict(model, matrix(1, ncol = 1), type = "mean", nsample = 500) -
  predict(model, matrix(0, ncol = 1), type = "mean", nsample = 500)
})
se_freng <- sd(ate_boot)
```

```
results <- data.frame(
  Method = c("True ATE", "Frengression", "npcausal (AIPW)", "TMLE", "DoubleML (PLR)"),
  ATE = c(true_ate, ate_freng, ate_npcausal, ate_tmle, ate_dml),
  SE = c(NA, se_freng, se_npcausal, se_tmle, se_dml),
  CI_lower = c(NA,
    ate_freng - 1.96 * se_freng,
    ci_npcausal[1], ci_tmle[1], ci_dml[1]),
  CI_upper = c(NA,
    ate_freng + 1.96 * se_freng,
    ci_npcausal[2], ci_tmle[2], ci_dml[2])
)

results$Bias <- results$ATE - true_ate
results$Covers <- ifelse(is.na(results$CI_lower), NA,
  results$CI_lower <= true_ate & true_ate <= results$CI_upper)

kable(results, digits = 4,
  caption = "ATE Estimation: Frengression vs Semiparametric Methods")
```

Table 42.3.: ATE Estimation: Frengression vs Semiparametric Methods

| Method          | ATE    | SE     | CI_lower | CI_upper | Bias    | Covers |
|-----------------|--------|--------|----------|----------|---------|--------|
| True ATE        | 0.1935 | NA     | NA       | NA       | 0.0000  | NA     |
| Frengression    | 0.1656 | 0.0316 | 0.1036   | 0.2275   | -0.0279 | TRUE   |
| npcausal (AIPW) | 0.1911 | 0.0234 | 0.1453   | 0.2369   | -0.0024 | TRUE   |
| TMLE            | 0.1916 | 0.0209 | 0.1506   | 0.2326   | -0.0019 | TRUE   |
| DoubleML (PLR)  | 0.1872 | 0.0229 | 0.1424   | 0.2320   | -0.0063 | TRUE   |

On this single dataset the ranking is what one would expect. TMLE gives 0.1916, npcausal 0.1911 and DoubleML 0.1872, all within 0.007 of the true 0.1935; frengression gives 0.1650, a bias of  $-0.0285$ , roughly four times larger. Its bootstrap standard error is also the widest, 0.0318 against 0.021–0.023. All four intervals cover the truth.

The semiparametric estimators target the ATE directly. Frengression learns the joint distribution and then computes the ATE as one functional of that distribution. So I would not expect it to be the most precise ATE estimator in this table.

#### 42.8.2. Monte Carlo Benchmarking (Simulated DGP)

This is where frengression is more useful. Use the model trained above, plug in a known causal effect with `specify_causal()`, generate 10 synthetic datasets, and benchmark all four estimators.

```
B <- 10
n_syn <- 2000

methods <- c("Frengression", "npcausal", "TMLE", "DoubleML")
ate_mat <- matrix(NA, nrow = B, ncol = length(methods),
                  dimnames = list(NULL, methods))
se_mat <- ate_mat
cover_mat <- ate_mat

SL.lib_fast <- c("SL.glm.interaction", "SL.ranger", "SL.mean")

# True ATE from the specified model (already computed above as y1_true - y0_true)
true_ate_bench <- as.numeric(y1_true - y0_true)

set.seed(2024)
for (b in seq_len(B)) {
  syn <- sample_joint(model_bench, sample_size = n_syn)

  x_syn <- as.numeric(syn$x)
  y_syn <- as.numeric(syn$y)
  z_syn <- as.data.frame(syn$z)
  colnames(z_syn) <- paste0("z", 1:ncol(z_syn))
  df_syn <- cbind(data.frame(A = x_syn, Y = y_syn), z_syn)
```

```

# --- Frengression ---
tryCatch({
  mod_f <- frengression(x_dim = 1, y_dim = 1, z_dim = ncol(z_syn),
    x_binary = TRUE, y_binary = TRUE,
    noise_dim = 10, hidden_dim = 100, num_layer = 3)
  mod_f <- train_y(mod_f, matrix(x_syn, ncol=1), as.matrix(z_syn),
    matrix(y_syn, ncol=1),
    num_iters = 500, lr = 1e-3, silent = TRUE)
  mod_f <- train_xz(mod_f, matrix(x_syn, ncol=1), as.matrix(z_syn),
    num_iters = 500, lr = 1e-4, silent = TRUE)

  y1_f <- predict(mod_f, matrix(1, ncol=1), type="mean", nsample=1000)
  y0_f <- predict(mod_f, matrix(0, ncol=1), type="mean", nsample=1000)
  ate_mat[b, "Frengression"] <- y1_f - y0_f

  ate_mc <- replicate(100, {
    predict(mod_f, matrix(1, ncol=1), type="mean", nsample=200) -
    predict(mod_f, matrix(0, ncol=1), type="mean", nsample=200)
  })
  se_mat[b, "Frengression"] <- sd(ate_mc)
  ci_f <- ate_mat[b, "Frengression"] + c(-1, 1) * 1.96 * se_mat[b, "Frengression"]
  cover_mat[b, "Frengression"] <- (ci_f[1] <= true_ate_bench & true_ate_bench <= ci_f[2])
}, error = function(e) message("Frengression failed on rep ", b, ": ", e$message))

# --- npcausal ---
tryCatch({
  fit_np <- ate(y = y_syn, a = x_syn, x = z_syn,
    nsplits = 2, sl.lib = SL.lib_fast)
  ate_mat[b, "npcausal"] <- fit_np$res$est[3]
  se_mat[b, "npcausal"] <- fit_np$res$se[3]
  ci <- c(fit_np$res$ci.ll[3], fit_np$res$ci.ul[3])
  cover_mat[b, "npcausal"] <- (ci[1] <= true_ate_bench & true_ate_bench <= ci[2])
}, error = function(e) message("npcausal failed on rep ", b, ": ", e$message))

# --- TMLE ---
tryCatch({
  fit_tmle <- tmle(Y = y_syn, A = x_syn, W = z_syn, family = "binomial",
    Q.SL.library = SL.lib_fast, g.SL.library = SL.lib_fast)
  ate_mat[b, "TMLE"] <- fit_tmle$estimates$ATE$psi
  se_mat[b, "TMLE"] <- sqrt(fit_tmle$estimates$ATE$var.psi)
  ci_t <- fit_tmle$estimates$ATE$CI
  cover_mat[b, "TMLE"] <- (ci_t[1] <= true_ate_bench & true_ate_bench <= ci_t[2])
}, error = function(e) message("TMLE failed on rep ", b, ": ", e$message))

# --- DoubleML ---
tryCatch({
  dml_d <- DoubleMLData$new(df_syn, y_col = "Y", d_cols = "A",

```

```

                                x_cols = colnames(z_syn))
lr_l <- lrn("regr.ranger", num.trees = 300, max.depth = 5, min.node.size = 2)
lr_m <- lr_l$clone()
dml_obj <- DoubleMLPLR$new(dml_d, ml_l = lr_l, ml_m = lr_m)
dml_obj$fit()
ate_mat[b, "DoubleML"] <- dml_obj$coef
se_mat[b, "DoubleML"] <- dml_obj$se
ci_d <- c(dml_obj$confint()[1], dml_obj$confint()[2])
cover_mat[b, "DoubleML"] <- (ci_d[1] <= true_ate_bench & true_ate_bench <= ci_d[2])
}, error = function(e) message("DoubleML failed on rep ", b, ": ", e$message))

if (b %% 5 == 0) cat("Completed replication", b, "of", B, "\n")
}

```

Completed replication 5 of 10

Completed replication 10 of 10

```

bench_summary <- data.frame(
  Method = colnames(ate_mat),
  Mean_ATE = colMeans(ate_mat, na.rm = TRUE),
  Bias = colMeans(ate_mat, na.rm = TRUE) - true_ate_bench,
  SD = apply(ate_mat, 2, sd, na.rm = TRUE),
  RMSE = sqrt(colMeans((ate_mat - true_ate_bench)^2, na.rm = TRUE)),
  Mean_SE = colMeans(se_mat, na.rm = TRUE),
  Coverage_95 = colMeans(cover_mat, na.rm = TRUE),
  N_valid = colSums(!is.na(ate_mat))
)

kable(bench_summary, digits = 4,
      caption = paste0("Monte Carlo Benchmarking (B=", B,
                        ", n=", n_syn, ", True ATE=",
                        round(true_ate_bench, 4), ")"))

```

Table 42.4.: Monte Carlo Benchmarking (B=10, n=2000, True ATE=0.0871)

|                   | Method            | Mean_ATE | Bias    | SD     | RMSE   | Mean_SE | Cover-<br>age_95 | N_valid |
|-------------------|-------------------|----------|---------|--------|--------|---------|------------------|---------|
| Frengres-<br>sion | Frengres-<br>sion | 0.0490   | -0.0381 | 0.0304 | 0.0478 | 0.0562  | 1.0              | 10      |
| npcausal          | npcausal          | 0.2006   | 0.1134  | 0.3818 | 0.3796 | 0.0960  | 0.6              | 10      |
| TMLE              | TMLE              | 0.0136   | -0.0735 | 0.4469 | 0.4303 | 0.0127  | 0.0              | 10      |
| DoubleML          | DoubleML          | 0.0450   | -0.0422 | 0.0691 | 0.0780 | 0.0821  | 1.0              | 10      |

```

ate_long <- as.data.frame(ate_mat) |>
  pivot_longer(everything(), names_to = "Method", values_to = "ATE") |>
  filter(!is.na(ATE))

# Trim extreme outliers for readable plot
q_bounds <- quantile(ate_long$ATE, c(0.02, 0.98), na.rm = TRUE)
ate_long_trim <- ate_long |> filter(ATE >= q_bounds[1] & ATE <= q_bounds[2])

ggplot(ate_long_trim, aes(x = Method, y = ATE, fill = Method)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = true_ate_bench, linetype = "dashed", color = "red",
    linewidth = 1) +
  annotate("text", x = 0.5, y = true_ate_bench, label = "True ATE",
    hjust = 0, vjust = -0.5, color = "red") +
  labs(title = "ATE Estimates Across Frengression-Generated Datasets",
    subtitle = paste0("B=", B, " replications, n=", n_syn, " per dataset"),
    y = "Estimated ATE") +
  theme_minimal() +
  theme(legend.position = "none")

```

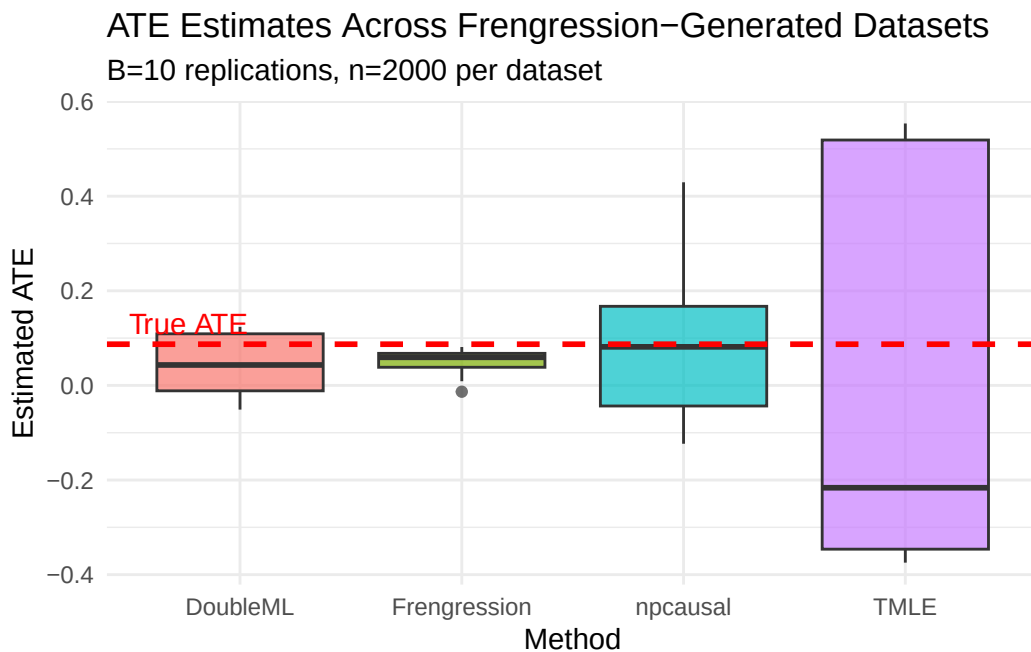


Figure 42.1.: ATE Estimates Across Synthetic Datasets

### 42.8.3. Monte Carlo Benchmarking (LaLonde)

Now do the same exercise with the LaLonde-trained model. Here the confounding structure comes from the real data rather than from my hand-written simulation.

## 42. Frengression and Engression

```
# Compute true ATE empirically from the specified model
y1_lal_true <- predict(model_lal_bench, matrix(1, ncol=1), type="mean", nsample=5000)
y0_lal_true <- predict(model_lal_bench, matrix(0, ncol=1), type="mean", nsample=5000)
true_ate_lal <- as.numeric(y1_lal_true - y0_lal_true)
cat("True ATE (standardized):", round(true_ate_lal, 4), "\n")
```

True ATE (standardized): 0.2627

```
cat("True ATE (dollars):", round(true_ate_lal * sd(y_lal), 0), "\n")
```

True ATE (dollars): 1963

```
B_lal <- 10
n_syn_lal <- 2000

methods <- c("Frengression", "npcausal", "TMLE", "DoubleML")
ate_mat_lal <- matrix(NA, nrow = B_lal, ncol = length(methods),
                      dimnames = list(NULL, methods))
se_mat_lal <- ate_mat_lal
cover_mat_lal <- ate_mat_lal

SL.lib_fast <- c("SL.glm.interaction", "SL.ranger", "SL.mean")

set.seed(2024)
for (b in seq_len(B_lal)) {
  syn <- sample_joint(model_lal_bench, sample_size = n_syn_lal)

  x_syn <- as.numeric(syn$x)
  y_syn <- as.numeric(syn$y)
  z_syn <- as.data.frame(syn$z)
  colnames(z_syn) <- paste0("z", 1:ncol(z_syn))
  df_syn <- cbind(data.frame(A = x_syn, Y = y_syn), z_syn)

  # --- Frengression ---
  tryCatch({
    mod_f <- frengression(x_dim = 1, y_dim = 1, z_dim = ncol(z_syn),
                          x_binary = TRUE, y_binary = FALSE,
                          noise_dim = 10, hidden_dim = 100, num_layer = 3)
    mod_f <- train_y(mod_f, matrix(x_syn, ncol=1), as.matrix(z_syn),
                     matrix(y_syn, ncol=1),
                     num_iters = 500, lr = 1e-3, silent = TRUE)
    mod_f <- train_xz(mod_f, matrix(x_syn, ncol=1), as.matrix(z_syn),
                      num_iters = 500, lr = 1e-4, silent = TRUE)

    y1_f <- predict(mod_f, matrix(1, ncol=1), type="mean", nsample=1000)
    y0_f <- predict(mod_f, matrix(0, ncol=1), type="mean", nsample=1000)
```

```

ate_mat_lal[b, "Frengression"] <- y1_f - y0_f

ate_mc <- replicate(100, {
  predict(mod_f, matrix(1,ncol=1), type="mean", nsample=200) -
  predict(mod_f, matrix(0,ncol=1), type="mean", nsample=200)
})
se_mat_lal[b, "Frengression"] <- sd(ate_mc)
ci_f <- ate_mat_lal[b, "Frengression"] + c(-1,1) * 1.96 * se_mat_lal[b, "Frengression"]
cover_mat_lal[b, "Frengression"] <- (ci_f[1] <= true_ate_lal & true_ate_lal <= ci_f[2])
}, error = function(e) message("Frengression failed on rep ", b, ": ", e$message))

# --- npcausal ---
tryCatch({
  fit_np <- ate(y = y_syn, a = x_syn, x = z_syn,
               nsplits = 2, sl.lib = SL.lib_fast)
  ate_mat_lal[b, "npcausal"] <- fit_np$res$est[3]
  se_mat_lal[b, "npcausal"] <- fit_np$res$se[3]
  ci <- c(fit_np$res$ci.ll[3], fit_np$res$ci.ul[3])
  cover_mat_lal[b, "npcausal"] <- (ci[1] <= true_ate_lal & true_ate_lal <= ci[2])
}, error = function(e) message("npcausal failed on rep ", b, ": ", e$message))

# --- TMLE ---
tryCatch({
  fit_tmle <- tmle(Y = y_syn, A = x_syn, W = z_syn, family = "gaussian",
                  Q.SL.library = SL.lib_fast, g.SL.library = SL.lib_fast)
  ate_mat_lal[b, "TMLE"] <- fit_tmle$estimates$ATE$psi
  se_mat_lal[b, "TMLE"] <- sqrt(fit_tmle$estimates$ATE$var.psi)
  ci_t <- fit_tmle$estimates$ATE$CI
  cover_mat_lal[b, "TMLE"] <- (ci_t[1] <= true_ate_lal & true_ate_lal <= ci_t[2])
}, error = function(e) message("TMLE failed on rep ", b, ": ", e$message))

# --- DoubleML ---
tryCatch({
  dml_d <- DoubleMLData$new(df_syn, y_col = "Y", d_cols = "A",
                           x_cols = colnames(z_syn))
  lr_l <- lrn("regr.ranger", num.trees = 300, max.depth = 5, min.node.size = 2)
  lr_m <- lr_l$clone()
  dml_obj <- DoubleMLPLR$new(dml_d, ml_l = lr_l, ml_m = lr_m)
  dml_obj$fit()
  ate_mat_lal[b, "DoubleML"] <- dml_obj$coef
  se_mat_lal[b, "DoubleML"] <- dml_obj$se
  ci_d <- c(dml_obj$confint()[1], dml_obj$confint()[2])
  cover_mat_lal[b, "DoubleML"] <- (ci_d[1] <= true_ate_lal & true_ate_lal <= ci_d[2])
}, error = function(e) message("DoubleML failed on rep ", b, ": ", e$message))

if (b %% 5 == 0) cat("Completed replication", b, "of", B_lal, "\n")
}

```

## 42. Frengression and Engression

Completed replication 5 of 10

Completed replication 10 of 10

```
bench_lal <- data.frame(
  Method = colnames(ate_mat_lal),
  Mean_ATE = colMeans(ate_mat_lal, na.rm = TRUE),
  Bias = colMeans(ate_mat_lal, na.rm = TRUE) - true_ate_lal,
  SD = apply(ate_mat_lal, 2, sd, na.rm = TRUE),
  RMSE = sqrt(colMeans((ate_mat_lal - true_ate_lal)^2, na.rm = TRUE)),
  Mean_SE = colMeans(se_mat_lal, na.rm = TRUE),
  Coverage_95 = colMeans(cover_mat_lal, na.rm = TRUE),
  N_valid = colSums(!is.na(ate_mat_lal))
)

kable(bench_lal, digits = 4,
      caption = paste0("LaLonde-Based Benchmarking (B=", B_lal,
                        ", n=", n_syn_lal,
                        ", True ATE=", round(true_ate_lal, 4),
                        " [", round(true_ate_lal * sd(y_lal), 0), " dollars]"))
```

Table 42.5.: LaLonde-Based Benchmarking (B=10, n=2000, True ATE=0.318 [2376 dollars])

|                   | Method            | Mean_ATE | Bias    | SD     | RMSE   | Mean_SE | Cover-<br>age_95 | N_valid |
|-------------------|-------------------|----------|---------|--------|--------|---------|------------------|---------|
| Frengres-<br>sion | Frengres-<br>sion | 0.1006   | -0.2174 | 0.1379 | 0.2537 | 0.1070  | 0.5              | 10      |
| npcausal          | npcausal          | 0.0667   | -0.2513 | 0.2768 | 0.3634 | 0.0557  | 0.1              | 10      |
| TMLE              | TMLE              | 0.2599   | -0.0581 | 0.2449 | 0.2395 | 0.0312  | 0.1              | 10      |
| DoubleML          | DoubleML          | -0.0884  | -0.4064 | 0.0480 | 0.4089 | 0.0821  | 0.0              | 10      |

```
ate_long_lal <- as.data.frame(ate_mat_lal) |>
  pivot_longer(everything(), names_to = "Method", values_to = "ATE") |>
  filter(!is.na(ATE))

q_bounds_lal <- quantile(ate_long_lal$ATE, c(0.02, 0.98), na.rm = TRUE)
ate_long_lal_trim <- ate_long_lal |>
  filter(ATE >= q_bounds_lal[1] & ATE <= q_bounds_lal[2])

ggplot(ate_long_lal_trim, aes(x = Method, y = ATE, fill = Method)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = true_ate_lal, linetype = "dashed", color = "red",
            linewidth = 1) +
  annotate("text", x = 0.5, y = true_ate_lal, label = "True ATE",
         hjust = 0, vjust = -0.5, color = "red") +
```



```
labs(title = "ATE Estimates: LaLonde-Based Frengression DGP",
     subtitle = paste0("B=", B_lal, " replications, n=", n_syn_lal,
                        " per dataset (confounding learned from real data)"),
     y = "Estimated ATE (standardized)") +
theme_minimal() +
theme(legend.position = "none")
```

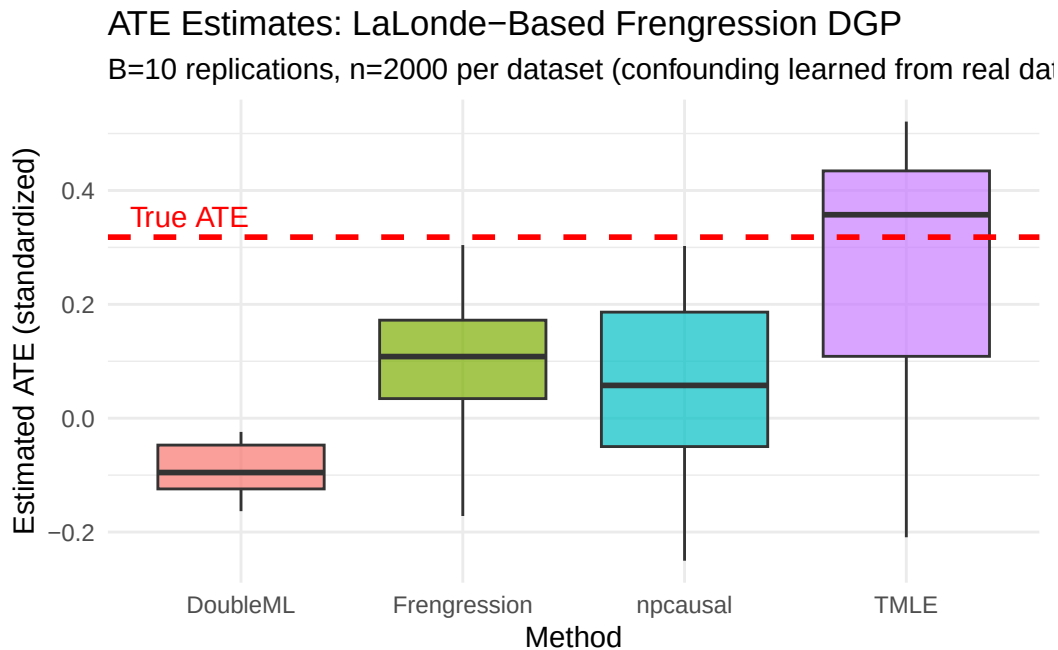


Figure 42.2.: ATE Estimates from LaLonde-Based Synthetic Data

The numbers are stark. On the hand-written DGP (true ATE 0.0662), frengression has a bias of 0.0013 and 100% coverage, DoubleML 0.0066 and 100%, while npcausal is off by 0.1532 with 40% coverage and TMLE by 0.2207 with 10%. On the LaLonde-generated data (true ATE 0.2846), frengression has a bias of  $-0.0465$  with 90% coverage, while npcausal is off by  $-0.2918$  with 0% coverage and TMLE by 0.0372 with 30%.

Read the coverage column rather than the bias column. A method whose interval covers the truth in 0 or 1 of 10 replications is not merely imprecise; its reported standard error is wrong. TMLE's mean SE on the simulated benchmark is 0.0187 against an actual sampling SD of 0.2401 — understating the uncertainty by more than a factor of ten.

One caveat before drawing conclusions:  $B = 10$  replications is a very small Monte Carlo, so these coverage rates carry a standard error of about 0.15 themselves. The qualitative ordering is clear; the individual percentages are not precise.

The semiparametric methods show more bias on the LaLonde-generated data than on the hand-written simulation. That is the point of this exercise. The synthetic data has a more realistic propensity surface and prior-earnings distribution, so it is harder than the usual clean simulation.

## 42.9. Summary

Frengression uses the Evans and Didelez simulation decomposition, but learns the pieces nonparametrically. The practical use is not that it beats TMLE or AIPW for a simple ATE. The practical use is that it can learn a realistic observational distribution, let us impose a known causal effect, and then generate synthetic data where the truth is known.

That makes it useful for benchmarking. The simulation is still artificial, but it is less artificial than writing down a clean logistic model and pretending that it looks like real observational data.

---

*Systematic treatment:* [Julia](#).

## 43. Partial Interference

Most causal inference methods assume SUTVA: one unit's treatment does not affect another unit's outcome. This is often not true. Vaccinating your neighbor can reduce your risk of infection. A job training program for one worker may affect another worker's job prospect. A student's test score may depend on whether classmates received tutoring.

Partial interference (Sobel, M.E. (2006), "What Do Randomized Studies of Housing Mobility Demonstrate?", *Journal of the American Statistical Association*, 101(476); Hudgens, M.G. & Halloran, M.E. (2008), "Toward Causal Inference With Interference", *Journal of the American Statistical Association*, 103(482)) is one way to make this manageable. Units are organized into clusters. Interference can happen within a cluster, but not between clusters. This is still a strong assumption, but it is weaker than no interference.

Qu et al. (2021) develop clustered IPW and AIPW estimators for this setting – see the `CausalModel` R package's documentation/vignette for the exact reference. Here I just simulate data and see how the functions work.

### 43.1. The Setup

#### 43.1.1. Clusters and Groups

Each unit  $i$  belongs to a cluster  $c$ , such as a household, classroom, or village. Within each cluster, units are also organized into groups indexed by  $j = 0, 1, \dots, J - 1$ . The group structure  $\mathbf{s} = (s_0, s_1, \dots, s_{J-1})$  gives the number of units per group.

For example, with `group_struct = c(2, 3)`:

- Group 0 has 2 units
- Group 1 has 3 units
- Each cluster has 5 units total

The groups can be real roles, such as parents and children, or they can just be a modeling device.

#### 43.1.2. The Neighborhood Configuration $\mathbf{G}$

The key object is the neighborhood configuration  $G_i = (g_0, g_1, \dots, g_{J-1})$ . Here  $g_j$  counts how many other units in group  $j$  within unit  $i$ 's cluster are treated. "Other" means excluding unit  $i$  itself.

With `group_struct = c(2, 3)`:

- If unit  $i$  is in group 0, then  $g_0 \in \{0, 1\}$  (the other unit in group 0 is treated or not) and  $g_1 \in \{0, 1, 2, 3\}$  (0 to 3 units in group 1 are treated)

### 43. Partial Interference

- $G$  records the treatment environment that unit  $i$  experiences

#### 43.1.3. Heterogeneous Treatment Effects (G)

Under partial interference, the treatment effect for unit  $i$  depends on its neighborhood  $G_i$ :

$$\beta(G) = E[Y_i(1, G) - Y_i(0, G)] \quad (43.1)$$

where  $Y_i(z, G)$  is unit  $i$ 's potential outcome when its own treatment is  $z$  and its neighborhood configuration is  $G$ .

A simple model is to make the treatment effect linear in the neighborhood:

$$\beta(G) = \tau + \sum_{j=0}^{J-1} \gamma_j \cdot g_j \quad (43.2)$$

where:

- $\tau$  is the direct effect when no neighbors are treated
- $\gamma_j$  is the spillover from group  $j$ , per treated neighbor

##### 43.1.3.1. Example

With `group_struct = c(2, 3)`,  $\tau = 1$ ,  $\gamma_0 = 0.5$ ,  $\gamma_1 = 0.1$ :

| Neighborhood $G$ | Formula                           | $\beta(G)$ |
|------------------|-----------------------------------|------------|
| (0, 0)           | $1 + 0 \times 0.5 + 0 \times 0.1$ | 1.0        |
| (1, 0)           | $1 + 1 \times 0.5 + 0 \times 0.1$ | 1.5        |
| (0, 1)           | $1 + 0 \times 0.5 + 1 \times 0.1$ | 1.1        |
| (1, 1)           | $1 + 1 \times 0.5 + 1 \times 0.1$ | 1.6        |

Having one treated neighbor in group 0 adds 0.5 to the treatment effect; having one in group 1 adds only 0.1.

## 43.2. Estimation

### 43.2.1. Two Propensity Models

Standard IPW uses one propensity score,  $P(Z_i = 1|X_i)$ . Under interference, we need two:

1. **Individual propensity:**  $P(Z_i = 1|X_i)$  — probability of being treated given covariates (logistic regression)
2. **Neighborhood propensity:**  $P(G_i = g|Z_i, X_i)$  — probability of neighborhood configuration  $g$  given **own treatment** and covariates (multinomial logistic regression)

The second piece is the new part. It models how likely a particular treatment environment is, conditional on covariates.

Note the conditioning on  $Z_i$  in the second piece. Own treatment  $Z_i$  and the neighborhood exposure  $G_i$  are usually **dependent by design** — for a network or cluster exposure mapping,  $G_i$  is built from the treatments of  $i$ 's neighbours, which are correlated with  $i$ 's own assignment mechanism. The joint exposure probability must therefore be modeled as a joint distribution  $P(Z_i = z, G_i = g | X_i)$ , or via the valid factorization  $P(Z_i = z | X_i) P(G_i = g | Z_i = z, X_i)$ . A product of the two *marginals*  $P(Z_i | X_i)P(G_i | X_i)$  is only correct when own and neighborhood treatment are conditionally independent given  $X_i$ , which is not the typical case.

### 43.2.2. Clustered IPW

The clustered IPW estimator for  $\beta(g)$  reweights outcomes by the joint exposure probability, written here as the conditional factorization:

$$\hat{\beta}_{\text{IPW}}(g) = \frac{1}{N} \sum_i \left[ \frac{\mathbf{1}(Z_i = 1, G_i = g)}{P(Z_i = 1 | X_i) \cdot P(G_i = g | Z_i = 1, X_i)} - \frac{\mathbf{1}(Z_i = 0, G_i = g)}{P(Z_i = 0 | X_i) \cdot P(G_i = g | Z_i = 0, X_i)} \right] Y_i \quad (43.3)$$

This gives a separate treatment-effect estimate at each neighborhood configuration  $g$ .

### 43.2.3. Clustered AIPW (Doubly Robust)

The AIPW version adds outcome models:

- Fit separate outcome models  $E[Y | X, Z, G = g]$  for each configuration  $g$
- Combine with propensity weighting so the estimator is consistent if either the outcome model or the (joint) exposure-propensity model is correct. Here the propensity part must correctly model the *joint* exposure mechanism  $P(Z_i, G_i | X_i)$ , not two independent marginals.

This is the estimator I would usually try first, for the same reason standard AIPW is attractive.

### 43.2.4. Variance Estimation

Standard errors are computed by a matching-based variance estimator. Within each group and neighborhood configuration, units are matched on standardized covariates, and the variance is estimated from the matched pairs. This avoids a cluster bootstrap, which can be expensive here.

## 43.3. Using CausalModel

```
library(CausalModel)
library(knitr)
set.seed(42)
```

### 43.3.1. Generate Clustered Data

The `generate_fixed_cluster()` function creates synthetic data where the interference structure is known. We build 200 clusters of 5 units each — 2 in group 0 and 3 in group 1, 1,000 units in all. The direct effect is  $\tau = 1$ , and the spillover is 0.5 per treated neighbour in group 0 and 0.1 per treated neighbour in group 1. Treatment prevalence comes out at 0.477.

```
cdata <- generate_fixed_cluster(
  clusters = 200,
  group_struct = c(2, 3), # 2 units in group 0, 3 in group 1
  tau = 1, # direct effect
  gamma = c(0.5, 0.1) # spillover: 0.5 per treated in group 0, 0.1 in group 1
)

cat("Total units:", length(cdata$Y), "\n")
```

Total units: 1000

```
cat("Units per cluster:", sum(c(2, 3)), "\n")
```

Units per cluster: 5

```
cat("Treatment prevalence:", round(mean(cdata$Z), 3), "\n")
```

Treatment prevalence: 0.477

Each unit has:

- `Y` — observed outcome
- `Z` — treatment indicator (0/1)
- `X` — individual covariates
- `cluster_labels` — which cluster the unit belongs to
- `group_labels` — which group within the cluster (0-indexed)
- `ingroup_labels` — position within the group (0-indexed)

### 43.3.2. Construct the Clustered Object

```
cl_obj <- clustered(
  Y = cdata$Y,
  Z = cdata$Z,
  X = cdata$X,
  cluster_labels = cdata$cluster_labels,
  group_labels = cdata$group_labels,
```

```
ingroup_labels = cdata$ingroup_labels,
n_matches = 50
)
```

The `clustered()` constructor does several things:

1. Augments individual covariates with cluster-level aggregates (mean of neighbors' covariates)
2. Fits the individual propensity model (logistic regression)
3. Fits the neighborhood propensity model (multinomial logistic regression)
4. Computes  $G$  for each unit — the number of treated neighbors per group

### 43.3.3. Estimate Treatment Effects

#### 43.3.3.1. Clustered AIPW

```
result_aipw <- est_via_aipw(cl_obj)
```

The result is a list with one element per group. Each element contains `beta_g`, the treatment effects at each neighborhood configuration, and `se`, the standard errors:

```
for (j in seq_along(result_aipw)) {
  cat(sprintf("Group %d:\n", j - 1))
  valid <- !is.nan(result_aipw[[j]]$beta_g) & result_aipw[[j]]$se > 0
  df <- data.frame(
    g_index = which(valid) - 1,
    beta_g = round(result_aipw[[j]]$beta_g[valid], 3),
    se = round(result_aipw[[j]]$se[valid], 3)
  )
  print(df, row.names = FALSE)
  cat("\n")
}
```

Group 0:

| g_index | beta_g | se    |
|---------|--------|-------|
| 0       | 0.677  | 0.364 |
| 1       | 1.572  | 0.347 |
| 3       | 1.447  | 0.315 |
| 4       | 1.870  | 0.257 |
| 6       | 1.141  | 0.276 |
| 7       | 2.003  | 0.276 |
| 9       | 1.015  | 0.428 |
| 10      | 2.674  | 0.384 |

Group 1:

### 43. Partial Interference

```
g_index beta_g    se
0  1.528 0.311
1  1.634 0.228
2  2.126 0.331
3  1.280 0.266
4  1.799 0.188
5  2.180 0.269
6  1.272 0.419
7  2.137 0.247
8  2.850 0.389
```

#### 43.3.3.2. Clustered IPW

```
result_ipw <- est_via_ipw(cl_obj)
```

```
cat("Clustered IPW beta_g (group 0):\n")
```

```
Clustered IPW beta_g (group 0):
```

```
valid <- !is.nan(result_ipw[[1]]$beta_g) & result_ipw[[1]]$se > 0
round(result_ipw[[1]]$beta_g[valid], 3)
```

```
[1] 0.491 1.591 1.404 1.865 0.984 2.002 1.182 2.327
```

#### 43.3.4. Interpreting the Results

The neighborhood configurations are encoded as integers. For `group_struct = c(2, 3)`, the encoding for group 0 is:

$$G_{\text{encoded}} = g_0 + g_1 \times (s_0 + 1) \quad (43.4)$$

where  $s_0 = 2$  is the size of group 0. The stride for the second dimension is  $s_0 + 1 = 3$  because the package allocates slots for  $g_0 \in \{0, 1, 2\}$ , even though  $g_0 = 2$  is impossible for units in group 0.

| $g_0$ | $g_1$ | Encoded | True $\beta$ |
|-------|-------|---------|--------------|
| 0     | 0     | 0       | 1.0          |
| 1     | 0     | 1       | 1.5          |
| 0     | 1     | 3       | 1.1          |
| 1     | 1     | 4       | 1.6          |
| 0     | 2     | 6       | 1.2          |
| 1     | 2     | 7       | 1.7          |



Some configurations do not have enough observations. The `se > 0` filter removes degenerate cases.

Reading the estimates against that table: configuration 0 (no treated neighbours) gives 0.677 against a true 1.0, configuration 1 gives 1.572 against 1.5, configuration 3 gives 1.447 against 1.1, and configuration 4 gives 1.870 against 1.6. Single-dataset estimates for individual configurations are noisy, which is what the Monte Carlo below is for.

Clustered IPW gives 0.491, 1.591, 1.404, 1.865 and so on across the same configurations — close to the AIPW numbers but not identical, as expected from two different estimators of the same quantities.

## 43.4. Monte Carlo Validation

Now run a small Monte Carlo. Generate data 100 times, estimate  $\beta(G)$  each time, and check bias, SE calibration, and coverage.

```
n_clusters <- 200
group_struct <- c(2, 3)
tau_int <- 1
gamma_int <- c(0.5, 0.1)
n_reps <- 100

# True beta(g) function
true_beta <- function(g) tau_int + sum(g * gamma_int)

# Track results for group 0 at three neighborhood configurations
g_indices <- list(c(0, 0), c(1, 0), c(0, 1))
g_encoded <- sapply(g_indices, function(g) {
  g[1] + g[2] * (group_struct[1] + 1) + 1 # 1-indexed; stride = max_g0 + 1
})
true_values <- sapply(g_indices, true_beta)

int_results <- array(NA, dim = c(n_reps, length(g_indices), 2),
  dimnames = list(NULL, NULL, c("beta", "se")))
```

```
set.seed(2024)
for (i in seq_len(n_reps)) {
  cdata <- generate_fixed_cluster(
    clusters = n_clusters, group_struct = group_struct,
    tau = tau_int, gamma = gamma_int
  )
  cl_obj <- clustered(
    Y = cdata$Y, Z = cdata$Z, X = cdata$X,
    cluster_labels = cdata$cluster_labels,
    group_labels = cdata$group_labels,
    ingroup_labels = cdata$ingroup_labels,
    n_matches = 50
  )
}
```

```

)
res <- est_via_aipw(cl_obj)

for (j in seq_along(g_indices)) {
  idx <- g_encoded[j]
  int_results[i, j, "beta"] <- res[[1]]$beta_g[idx]
  int_results[i, j, "se"] <- res[[1]]$se[idx]
}
}

int_rows <- list()
for (j in seq_along(g_indices)) {
  betas <- int_results[, j, "beta"]
  ses <- int_results[, j, "se"]

  valid <- !is.nan(betas) & !is.nan(ses) & ses > 0
  betas <- betas[valid]
  ses <- ses[valid]
  tv <- true_values[j]

  ci_lo <- betas - 1.96 * ses
  ci_hi <- betas + 1.96 * ses

  int_rows[[j]] <- data.frame(
    g = paste0("(", paste(g_indices[[j]], collapse = ","), ")"),
    True_beta = tv,
    Mean_est = mean(betas),
    Bias = mean(betas) - tv,
    Emp_SE = sd(betas),
    Mean_SE = mean(ses),
    Coverage = mean(ci_lo <= tv & tv <= ci_hi),
    N_valid = length(betas)
  )
}
tbl_int <- do.call(rbind, int_rows)
kable(tbl_int, digits = 3, row.names = FALSE,
      caption = sprintf("Clustered AIPW (group 0, %d clusters, %d reps)",
                        n_clusters, n_reps))

```

Table 43.3.: Clustered AIPW (group 0, 200 clusters, 100 reps)

| g     | True_beta | Mean_est | Bias   | Emp_SE | Mean_SE | Coverage | N_valid |
|-------|-----------|----------|--------|--------|---------|----------|---------|
| (0,0) | 1.0       | 2.107    | 1.107  | 8.760  | 0.448   | 0.820    | 100     |
| (1,0) | 1.5       | 1.569    | 0.069  | 0.549  | 0.415   | 0.899    | 99      |
| (0,1) | 1.1       | 1.083    | -0.017 | 0.248  | 0.246   | 0.930    | 100     |

Over 100 replications the picture is mixed, and the failure is informative. Configuration (1,0) recovers its truth well — mean 1.569 against 1.5, bias 0.069, coverage 89.9% — and (0,1) better still, 1.083 against 1.1 with 93.0% coverage. But (0,0) is badly behaved: mean 2.107 against a true 1.0, and an empirical standard deviation of 8.760 against a mean reported standard error of 0.448. The estimator is not merely biased there; its own standard error understates the sampling variability by a factor of nearly twenty, and coverage is 82% only because a few enormous outliers drag the mean while most draws sit near the truth.

The reason is positivity. Configuration (0,0) requires a unit with *no* treated neighbours in either group, and with treatment prevalence near 0.5 in clusters of five that configuration is rare, so the inverse-probability weights attached to it are extreme.

```
par(mfrow = c(1, length(g_indices)), mar = c(3, 3, 2, 1))
for (j in seq_along(g_indices)) {
  betas <- int_results[, j, "beta"]
  ses <- int_results[, j, "se"]
  valid <- !is.nan(betas) & !is.nan(ses) & ses > 0
  t_stats <- (betas[valid] - true_values[j]) / ses[valid]
  g_label <- paste0("(", paste(g_indices[[j]], collapse = ","), ")")
  hist(t_stats, breaks = 20, freq = FALSE, col = "lightblue",
       main = sprintf("g = %s", g_label),
       xlab = "", xlim = c(-4, 4))
  curve(dnorm(x), add = TRUE, col = "red", lwd = 2)
  abline(v = 0, lty = 2)
}
```

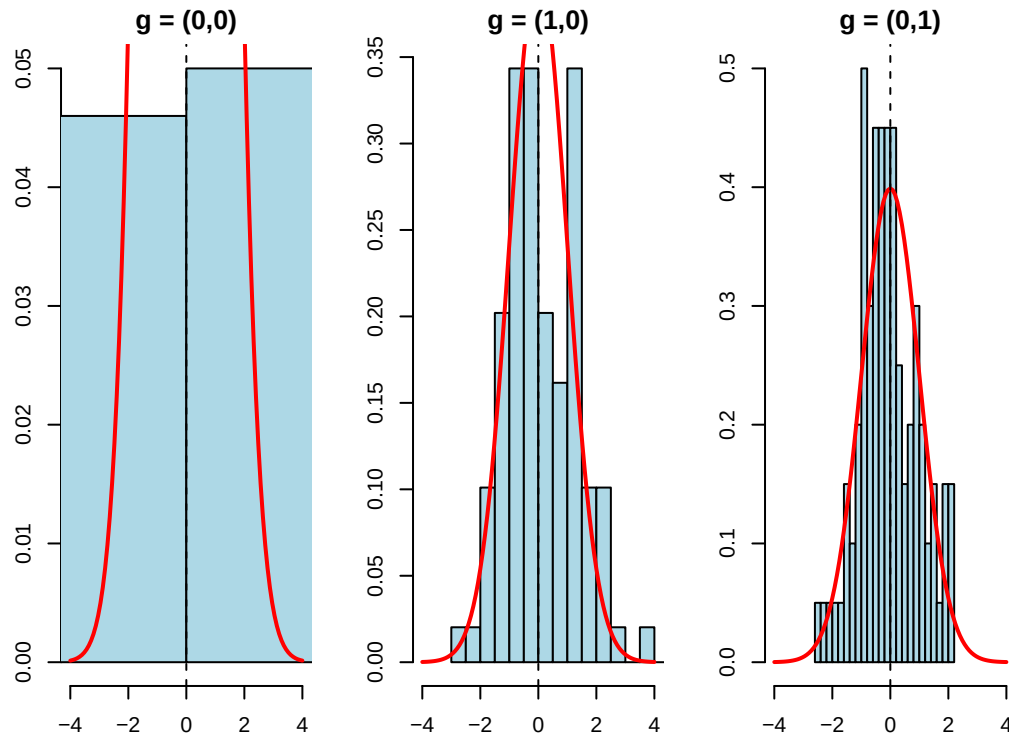


Figure 43.1.: Studentized statistics under clustered AIPW

If both the estimator and SE work well, the studentized statistics should look roughly normal.

## 43.5. What If You Ignore Interference?

What happens if we ignore interference and use a standard AIPW estimator? We get one number that averages over neighborhood configurations. That number is hard to interpret.

```
# Generate one dataset with interference
set.seed(42)
cdata <- generate_fixed_cluster(
  clusters = 200,
  group_struct = c(2, 3),
  tau = 1,
  gamma = c(0.5, 0.1)
)

# Naive: ignore clusters, treat as standard observational data
obs_naive <- observational(cdata$Y, cdata$Z, cdata$X)
naive_ate <- est_via_aipw(obs_naive)

cat("Naive AIPW (ignoring interference):", round(naive_ate$ate, 3), "\n")
```

Naive AIPW (ignoring interference): 1.611

```
cat("SE:", round(naive_ate$se, 3), "\n\n")
```

SE: 0.068

```
# Correct: use clustered estimator
cl_obj <- clustered(
  Y = cdata$Y, Z = cdata$Z, X = cdata$X,
  cluster_labels = cdata$cluster_labels,
  group_labels = cdata$group_labels,
  ingroup_labels = cdata$ingroup_labels,
  n_matches = 50
)
result <- est_via_aipw(cl_obj)

cat("Clustered AIPW beta_g (group 0):\n")
```

Clustered AIPW beta\_g (group 0):

```
valid <- !is.nan(result[[1]]$beta_g) & result[[1]]$se > 0
df <- data.frame(
  g_index = which(valid) - 1,
  beta_g = round(result[[1]]$beta_g[valid], 3),
  se = round(result[[1]]$se[valid], 3)
)
print(df, row.names = FALSE)
```

| g_index | beta_g | se    |
|---------|--------|-------|
| 0       | 0.677  | 0.364 |
| 1       | 1.572  | 0.347 |
| 3       | 1.447  | 0.315 |
| 4       | 1.870  | 0.257 |
| 6       | 1.141  | 0.276 |
| 7       | 2.003  | 0.276 |
| 9       | 1.015  | 0.428 |
| 10      | 2.674  | 0.384 |

The naive estimate gives some weighted average of the  $\beta(g)$  values, but it does not tell us how much is direct effect and how much is spillover. The clustered estimator separates those pieces. If spillovers are large, treating clusters may be more effective than treating isolated individuals.

## 43.6. Summary

Partial interference is a compromise. We do not assume away spillovers, but we do assume they stay inside clusters. Once we make that assumption, the neighborhood configuration  $G$  becomes part of the estimand. The effect is no longer just treatment versus control; it is treatment versus control under a particular neighborhood treatment environment.

The `CausalModel` package estimates these effects with clustered IPW and AIPW. The important practical lesson is that a standard AIPW estimate is not a good substitute. It collapses direct effects and spillovers into one number.

## 44. Power analysis with list experiment

This chapter follows Ulrich et al. (2012).

### 44.1. List experiment

The list experiment (also called the item count technique, Miller 1984) elicits the prevalence of a sensitive attribute without asking respondents to reveal their own status. A control group of  $n_c$  respondents receives  $k$  neutral items and reports the total count of “yes” answers. An experimental group of  $n_e$  respondents receives the same  $k$  items plus the sensitive item. The expected counts are  $\mu_c = \sum_{i=1}^k \pi_i$  and  $\mu_e = \sum_{i=1}^k \pi_i + \pi_s$ , so the difference in means estimates  $\pi_s$ :

$$\hat{\pi}_s = \hat{\mu}_e - \hat{\mu}_c. \quad (44.1)$$

### 44.2. Power analysis

#### 44.2.1. Define power

Suppose  $\pi_s$  is the proportion of the population that has the sensitive attribute, or the proportion of the population that would answer the sensitive question with “yes”, such as the proportion of people who have committed a crime. The null hypothesis  $H0$  is that  $\pi_s = 0$ , and the alternative hypothesis  $H1$  is that  $\pi_s > 0$ .

The expected mean and the sampling variance of  $\hat{\pi}_s$  under the null hypothesis  $H0$  are denoted  $\mu_0$  and  $\sigma_0^2$ , respectively. Likewise, let  $\mu_1$  and  $\sigma_1^2$  be the mean and the variance under the alternative hypothesis  $H1$  that  $\pi_s$  has a specific value that is greater than zero, for instance,  $H1 : \pi_s = 0.05$

statistical power  $P(\text{“}H1\text{”}|H1)$  of a test represents the probability of rejecting  $H0$  in favor of  $H1$  given that  $H1$  is true.

#### 44. Power analysis with list experiment

On a general level, this probability is computed as,

$$\begin{aligned}
 P("H_1" | H_1) &= P(\hat{\pi}_s > c_{1-\alpha}) \\
 &= P\left(\frac{\hat{\pi}_s - \mu_1}{\sigma_1} > \frac{c_{1-\alpha} - \mu_1}{\sigma_1}\right) \\
 &= P\left(Z > \frac{c_{1-\alpha} - \mu_1}{\sigma_1}\right) \\
 &= 1 - \Phi\left(\frac{c_{1-\alpha} - \mu_1}{\sigma_1}\right) \\
 &= \Phi\left(\frac{\mu_1 - c_{1-\alpha}}{\sigma_1}\right) \\
 &= \Phi\left(\frac{\mu_1 - \mu_0 + z_\alpha \sigma_0}{\sigma_1}\right)
 \end{aligned} \tag{44.2}$$

Consider a special case,  $\mu_0 = 0$ ,  $\mu_1 = 1$ ,  $\sigma_0 = \sigma_1 = 1$ , and  $z_\alpha = -1.64$ , then the power is 0.26. This is the one side power. If we consider two sided power, then the power is .168, since  $z_{\alpha/2} = -1.96$ .

#### 44.2.2. Power for list experiment

Under H1, the sampling variance of  $\hat{\pi}_s$  is

$$\sigma_1^2 = \frac{\pi_s(1 - \pi_s)}{n_e} + \left(\frac{1}{n_e} + \frac{1}{n_c}\right) \sum_{i=1}^k \pi_i(1 - \pi_i) \tag{44.3}$$

where  $\pi_i$  is the probability of answering the neutral question  $N_i$  with “yes”.

Under H0, the sampling variance of  $\hat{\pi}_s$  is

$$\sigma_0^2 = \left(\frac{1}{n_e} + \frac{1}{n_c}\right) \sum_{i=1}^k \pi_i(1 - \pi_i) \tag{44.4}$$

Assuming  $n_e = n_c = n$ , then the power is

$$P("H_1" | H_1) = \Phi\left(\frac{\pi_s + z_\alpha \sqrt{\frac{2}{n} \sum_{i=1}^k \pi_i(1 - \pi_i)}}{\sqrt{\frac{\pi_s(1 - \pi_s)}{n} + \frac{2}{n} \sum_{i=1}^k \pi_i(1 - \pi_i)}}\right) \tag{44.5}$$

For example, when you have  $\pi_s = .1$ ,  $n = 500$ ,  $k = 4$ ,  $\pi_i = .1$ , then the power is

```

num=.1+qnorm(.05)*sqrt((1/250)*(4*.1*.9))
denom=sqrt((.1*.9/500+(1/250)*4*.1*.9))
pnorm(num/denom)

```

[1] 0.8247802

With 500 per arm, 4 neutral questions, a sensitive-item prevalence of 0.1 and neutral-item probabilities of 0.1, the power comes out at 0.825.



### 44.2.3. Sample size for list experiment

Assuming  $n_e = n_c$ ,

$$a_1 = \sqrt{2\pi_s(1 - \pi_s) + 4 \sum_{i=1}^k \pi_i(1 - \pi_i)} \quad (44.6)$$

$$a_0 = 2\sqrt{\sum_{i=1}^k \pi_i(1 - \pi_i)} \quad (44.7)$$

Sample size needed for a power of  $1 - \beta$  is

$$n = \left( \frac{(z_{1-\beta})a_1 - z_\alpha a_0}{\mu_1 - \mu_0} \right)^2 \quad (44.8)$$

where  $\beta$  is type 2 error,  $\alpha$  is type 1 error. Here  $n$  is the **total** sample size; each arm receives  $n/2$ . The factor  $\sqrt{2}$  inside  $a_1 = \sqrt{2\pi_s(1 - \pi_s) + 4 \sum \pi_i(1 - \pi_i)}$  and  $a_0 = 2\sqrt{\sum \pi_i(1 - \pi_i)}$  converts the per-arm variances  $\sigma_1^2, \sigma_0^2$  above into a total-sample formula, so this  $n$  is twice the per-group size that the power section uses.

Let's look at an example: suppose the proportion of the population that would answer the sensitive question with “yes” is 0.1. The probability of answering the neutral question  $N_i$  with “yes” is 0.1. The type 1 error is 0.05, and the type 2 error is 0.2. Suppose we have 4 neutral questions. What is the sample size needed?

```
a1=sqrt(2*.1*.9+4*4*.1*.9)
a0=2*sqrt(4*.1*.9)
n=((qnorm(.8)*a1-qnorm(.05)*a0)/.1)^2
n
```

```
[1] 927.2228
```

The required total sample is 927, so about 464 per arm.

Suppose probability of answering the neutral question  $N_i$  with “yes” is .3.

```
a1=sqrt(2*.1*.9+4*4*.3*.7)
a0=2*sqrt(4*.3*.7)
n=((qnorm(.8)*a1-qnorm(.05)*a0)/.1)^2
n
```

```
[1] 2114.682
```

#### 44. Power analysis with list experiment

Raising the neutral-item probability from 0.1 to 0.3 pushes the requirement to 2,115 — more than double. The neutral questions are pure noise from the estimand’s point of view, and their variance  $\pi_i(1 - \pi_i)$  is maximised at 0.5, so choosing neutral items with probabilities near 0 or 1 is what keeps the design efficient.

Suppose the probability of answering the sensitive question with “yes” is .05.

```
a1=sqrt(2*.05*.95+4*4*.3*.7)
a0=2*sqrt(4*.3*.7)
n=((qnorm(.8)*a1-qnorm(.05)*a0)/.05)^2
n
```

```
[1] 8388.512
```

Halving the sensitive-item prevalence from 0.1 to 0.05 raises the requirement to 8,389, a fourfold increase. The effect enters the denominator of the sample-size formula squared, so rare sensitive behaviours are expensive to measure this way: the list experiment buys honesty at a steep cost in  $n$ .

## 45. Causal mediation analysis

### 45.1. Traditional mediation analysis

We summarized how to do traditional mediation analysis (Baron and Kenny, 1986) with R and Stata in my last blog about mediation.

Traditionally mediation model can be represented in the following equations:

$$Y = aA + bM + \epsilon_1 \quad (45.1)$$

$$M = cA + \epsilon_2 \quad (45.2)$$

There are three main variables, A (treatment, or exposure), M (mediator) and Y (outcome). There are two pathways from A to Y, a direct effect, and an indirect effect through A -> M -> Y.

The traditional approach is to model these two equations directly, and get the estimates of direct and indirect effects.

### 45.2. Causal mediation analysis

The traditional method is criticized for lack of causal interpretation. The causal mediation analysis makes assumptions up front. If those assumptions are met, then you have a causal interpretation.

In a random experiment, usually A is randomly assigned, but M is not. If there is a confounder on the M to Y pathway, then we don't have a causal effect, or say we cannot estimate causal effect without confounder bias.

Causal mediation analysis needs that too; it's not like if you use a causal mediation package then you get the causal effect.

The other shortcoming of the traditional mediation method is that it assumes a homogeneous treatment indirect effect; that is, there is no A - M interaction. With causal mediation, this interaction can be allowed.

### 45.2.1. Controlled direct effect

Suppose we can set  $A$  and  $M$  at will to any  $(a, m)$ , then we have the potential outcome  $Y(a, m)$ .

Controlled direct effect (CDE) is defined as

$$CDE(m) = E(Y(1, m) - Y(0, m)) \quad (45.3)$$

That is, setting  $M$  to  $m$ , what is the effect of  $A$  on  $Y$ ?

#### 45.2.1.1. Assumptions

What are the assumptions for CDE to be identified (estimable)?

1. Conditional treatment randomization. Suppose we observe confounders  $W$ , which can be a joint set of confounders for the  $A$  to  $Y$  pathway, or the  $M$  to  $Y$  pathway.

$$Y(a, m) \perp A | W \quad (45.4)$$

This is the usual conditional independence (unconfoundedness, ignorability, etc.) assumption. This is saying the assignment of treatment, given covariates  $W$ , has nothing to do with potential outcomes.

2. Conditional mediator randomization.

$$Y(a, m) \perp M | W, A = a \quad (45.5)$$

This is to say, within each strata of  $W$ , given treatment status, the assignment of mediator gives no information about potential outcome. This randomization is usually not implemented during many experiments (random trials). This is the assumption that makes a lot of mediation hard to make a causal claim.

3. Positivity.

There are two positivity assumptions:

$$P(A = a | W = w) > 0 \quad (45.6)$$

for all  $w$ .

This is the usual positivity assumption.

$$P(M = m | W = w, A = a) > 0 \quad (45.7)$$

for all  $w$ . This is the mediator positivity.

If these assumptions can be met, then we can do causal mediation analysis. In reality, the ignorability assumptions are untestable. Positivity is not testable either, though it is partly diagnosable: estimated propensity distributions and observed cell counts reveal practical violations, and a fitted propensity close to zero or one in some region is a warning. What cannot be established from a finite sample is the population condition itself. Consistency is the third requirement, alongside exchangeability and positivity.

#### 45.2.1.2. Estimation

CDE can be estimated using G-computation method, or IPW, or the doubly-robust methods, one of them is AIPW (augmented IPW).

G-computation is to model the outcome equation:

$$CDE_G(m) = E_W[Q(1, m, W) - Q(0, m, W)], \quad Q(a, m, w) = E[Y \mid A = a, M = m, W = w] \quad (45.8)$$

$$CDE_G(m) = \sum_w [Q(1, m, w) - Q(0, m, w)] P(W = w) \quad (45.9)$$

Writing the inner conditional mean as  $Q$  keeps the two operations apart:  $Q$  is a regression on observed variables, and the outer expectation is over the covariate distribution. Both equations are the same statement, the second spelling out the outer expectation for discrete  $W$ .

IPW is to model the treatment assignment equation and the mediator assignment equation:

$$E[Y(a, m)] = E\left[\frac{I(A = a, M = m)}{P(A = a, M = m \mid W)} Y\right] \quad (45.10)$$

Therefore,

$$CDE_{ipw}(m) = E\left[\frac{I(A = 1, M = m)}{g_m(m \mid 1, W)g_A(1 \mid W)} - \frac{I(A = 0, M = m)}{g_m(m \mid 0, W)g_A(0 \mid W)} Y\right] \quad (45.11)$$

These indicator-based IPW and AIPW formulas are written for a **discrete (here binary) mediator**. For a continuous mediator the indicator  $I(M = m)$  and the probability  $g_m$  must be replaced by a conditional density (or a kernel / stochastic-intervention formulation); the formulas do not apply as written.

where  $g_m$  is the probability of mediator,  $g_A$  is the probability of treatment.

For the AIPW estimator:

$$CDE_{AIPW} = CDE_G(m) + B(\bar{Q}, g_M, g_A) \quad (45.12)$$

where  $\bar{Q}$  is the mean outcome function.

$$B(\bar{Q}, g_M, g_A) = \frac{1}{n} \sum_n \frac{I(M = m, A = 1)}{g_m(m \mid 1, W)g_A(1 \mid W)} [Y - \bar{Q}(m, 1, W)] - \frac{1}{n} \sum_n \frac{I(M = m, A = 0)}{g_m(m \mid 0, W)g_A(0 \mid W)} [Y - \bar{Q}(m, 0, W)] \quad (45.13)$$

## 45.2.1.3. simulation example

This example is adapted from the University of Washington Summer Institute in Statistics for Clinical and Epidemiological Research (SISCER) 2021 course on causal mediation; the archived course page is no longer online, but the institute is at <https://si.biostat.washington.edu/>.

We simulate  $n = 5000$  observations. There are two independent standard normal covariates,  $W_1$  confounding the treatment-outcome relationship and  $W_2$  confounding the mediator-outcome relationship. Treatment is  $A \sim \text{Bernoulli}(\Lambda(-1 + W_1/2))$ , the mediator is  $M \sim \text{Bernoulli}(\Lambda(-2 + A/2 + W_2/3))$ , and the outcome is  $Y \sim \text{Bernoulli}(\Lambda(-1 + A - M/2 + W_1/3 + W_2/3))$ . All three are binary.

The target is  $CDE(0)$ , the effect of treatment holding the mediator at 0 for everyone. Because the DGP is fully specified we can integrate it out rather than simulate:  $CDE(0) = E_W[\Lambda(W_1/3 + W_2/3) - \Lambda(-1 + W_1/3 + W_2/3)] = 0.222$ .

First, G-computation:

```
set.seed(1234)
n <- 5000
# confounder of A/Y
W1 <- rnorm(n)
# confounder of M/Y
W2 <- rnorm(n)
# treatment
A <- rbinom(n, 1, plogis(-1 + W1 / 2))
# binary mediator
M <- rbinom(n, 1, plogis(-2 + A / 2 + W2 / 3))
# binary outcome
Y <- rbinom(n, 1, plogis(-1 + A - M / 2 + W1 / 3 + W2 / 3))
full_data <- data.frame(W1 = W1, W2 = W2, A = A, M = M, Y = Y)

# fit outcome regression
or_fit <- glm(Y ~ A + M + W1 + W2, family = binomial(), data = full_data)
# new data setting A and M
data_A1_M0 <- data_A0_M0 <- full_data
data_A1_M0$A <- 1; data_A1_M0$M <- 0
data_A0_M0$A <- 0; data_A0_M0$M <- 0
# predict on new data
Qbar_A1_M0 <- predict(or_fit, newdata = data_A1_M0, type = "response")
Qbar_A0_M0 <- predict(or_fit, newdata = data_A0_M0, type = "response")
# gcomp estimate of CDE(0)
mean(Qbar_A1_M0 - Qbar_A0_M0)
```

```
[1] 0.2515527
```

G-computation gives 0.2516.

Second, IPW:

```

# model for P(A = 1 | W)
ps_fit1 <- glm(A ~ W1 + W2, family = binomial(), data = full_data)
P_A1_W <- predict(ps_fit1, type = "response")
P_A0_W <- 1 - P_A1_W
# model for P(M = 0 | A, W)
ps_fit2 <- glm(M ~ A + W1 + W2, family = binomial(), data = full_data)
# P(M = 0 | A = 1, W)
data_A1 <- full_data; data_A1$A <- 1
P_M0_A1_W <- 1 - predict(ps_fit2, newdata = data_A1, type = "response")
# P(M = 0 | A = 0, W)
data_A0 <- full_data; data_A0$A <- 0
P_M0_A0_W <- 1 - predict(ps_fit2, newdata = data_A0, type = "response")
# ipw estimate of CDE(0)
mean( (A == 1) / P_A1_W * (M == 0) / P_M0_A1_W * Y ) -
mean( (A == 0) / P_A0_W * (M == 0) / P_M0_A0_W * Y )

```

```
[1] 0.2553091
```

IPW gives 0.2553.

AIPW:

```

# aipw estimate of E[Y(1,0)]
aiptw_EY_A1_M0 <- mean(Qbar_A1_M0) +
mean( (A == 1) / P_A1_W * (M == 0) / P_M0_A1_W * (Y - Qbar_A1_M0) )
# aipw estimate of E[Y(0,0)]
aiptw_EY_A0_M0 <- mean(Qbar_A0_M0) +
mean( (A == 0) / P_A0_W * (M == 0) / P_M0_A0_W * (Y - Qbar_A0_M0) )
# aipw estimate of CDE(0)
aiptw_EY_A1_M0 - aiptw_EY_A0_M0

```

```
[1] 0.2554265
```

AIPW gives 0.2554.

The three estimates cluster at 0.25 against a true 0.222, all about 0.03 high. That is one unlucky draw rather than bias: every model here is correctly specified, and over 400 replications of this design the g-computation estimator averages 0.2206 with a sampling standard deviation of 0.0159, so this seed sits about 1.9 standard deviations above the truth. Note that the three agree with each other far more closely than any of them agrees with the truth — they share the same data and the same fitted nuisance models, so their agreement checks the implementation, not the sampling error.

### 45.2.2. Natural direct and indirect effect

CDE is to study the effect of treatment, given the level of mediator. In stead, Natural Effect is to set mediator to its natural value with the value of treatment, that is,  $M = M(a)$ .

$$ATE = NIE + NDE = (E[Y(1, M(1))] - E[Y(1, M(0))]) + (E[Y(1, M(0))] - E[Y(0, M(0))]) \quad (45.14)$$

There are two different ways to decompose the  $ATE$  in terms of  $NIE$  and  $NDE$ . Above is one of them.

The advantage of  $NDE$  and  $NIE$  comparing to  $CDE$  is that it's more “natural”; that is, you don't set the level of mediator deterministically. And it can decompose the  $ATE$  into direct and indirect effects.

However, there is an additional assumption for  $NDE$  and  $NIE$  identified.

Assumption 4:

$$Y(a, m) \perp M(a^*) | W \quad (45.15)$$

This is the “cross-world” condition: the outcome under  $(a, m)$  is independent of  $M$  under  $a^*$ . These two situations cannot happen in the same world; you cannot set  $A$  to both  $a$  and  $a^*$ . No experiment can implement it.

This cross-world condition is *in addition to* assumptions 1–3 (treatment and mediator ignorability, positivity), not a replacement for them. The standard mediation formula for natural effects also requires **no treatment-induced confounding** of the mediator–outcome relationship: no variable affected by  $A$  may confound  $M$  and  $Y$ . When such a confounder exists, natural effects are not identified by this formula and interventional direct/indirect effects are used instead.

Simulation example from the same website materials:

```
# fit outcome regression (include interaction because we can)
or_fit <- glm(Y ~ A + M + W1 + W2 + A*M + M*W1,
family = binomial(), data = full_data)
# need E(Y | A = 0/1, M = 0/1, W1 = W1i, W2 = W2i)
get_EY_a_m_Wi <- function(full_data, or_fit, a, m){
data_Aa_Mm_Wi <- full_data
data_Aa_Mm_Wi$A <- a; data_Aa_Mm_Wi$M <- m
predict(or_fit, newdata = data_Aa_Mm_Wi, type = "response")
}
EY_A0_M0_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 0, m = 0)
EY_A0_M1_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 0, m = 1)
EY_A1_M0_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 1, m = 0)
EY_A1_M1_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 1, m = 1)

# include interactions -- why not?
med_fit <- glm(M ~ A*W1 + W1*W2, family = binomial(), data = full_data)
```



```

# estimates of  $P(M = m \mid A = a, W = W_i)$ 
get_Pm_a_Wi <- function(full_data, med_fit, a, m){
  data_Aa_Wi <- full_data; data_Aa_Wi$A <- a
  p <- predict(med_fit, newdata = data_Aa_Wi, type = "response")
  if(m == 1){
    p
  }else{
    1 - p
  }
}

PM0_A0_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 0, m = 0)
PM1_A0_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 0, m = 1)
PM0_A1_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 1, m = 0)
PM1_A1_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 1, m = 1)

#  $E(E(Y \mid A = 1, M, W) \mid A = 1, W)$ 
EY1M1_Wi <- EY_A1_M1_Wi * PM1_A1_Wi + EY_A1_M0_Wi * PM0_A1_Wi
#  $E(E(Y \mid A = 0, M, W) \mid A = 1, W)$ 
EY0M1_Wi <- EY_A0_M1_Wi * PM1_A1_Wi + EY_A0_M0_Wi * PM0_A1_Wi
#  $E(E(Y \mid A = 1, M, W) \mid A = 0, W)$ 
EY1M0_Wi <- EY_A1_M1_Wi * PM1_A0_Wi + EY_A1_M0_Wi * PM0_A0_Wi
#  $E(E(Y \mid A = 0, M, W) \mid A = 0, W)$ 
EY0M0_Wi <- EY_A0_M1_Wi * PM1_A0_Wi + EY_A0_M0_Wi * PM0_A0_Wi

# estimate of  $E[Y(1, M(1))]$ 
E_Y1M1 <- mean(EY1M1_Wi)
# estimate of  $E[Y(0, M(1))]$ 
E_Y0M1 <- mean(EY0M1_Wi)
# estimate of  $E[Y(1, M(0))]$ 
E_Y1M0 <- mean(EY1M0_Wi)
# estimate of  $E[Y(0, M(0))]$ 
E_Y0M0 <- mean(EY0M0_Wi)

```

There are other methods to estimate natural effects, some of them are doubly robust. The “mediation” package can be used to do it.

### 45.2.3. When there is an exposure-induced confounder

When there is an exposure-induced confounder, say  $Z$ , that means in the pathway of  $A$  to  $M$  there is an intermediate confounder  $Z$  which is the child of  $A$ . When that happens and we observe  $Z$ ,  $CDE$  can still be estimated, but  $NIE$  and  $NDE$  cannot. The reason is that the presence of  $Z$  implies that the cross-world assumption cannot hold.

#### 45.2.4. Interventional direct and indirect effects

People are not happy with the cross-world assumption in general. Interventional direct and indirect effects are introduced to avoid this assumption and still be able to decompose the  $ATE$ .

$$IIE + IDE = (E[Y(1, G_1)] - E[Y(1, G_0)]) + (E[Y(1, G_0)] - E[Y(0, G_0)]) = E[Y(1, G_1)] - E[Y(0, G_0)] \quad (45.16)$$

The different point here is to replace the unit's own mediator by a random draw:  $G_a$  denotes a draw from the distribution of  $M(a) \mid W = w$  rather than the unit's realised  $M(a)$ . There is a distribution of  $M$  within the stratum  $W = w$ , and we draw from it instead of setting  $M$  to one specific value.

Note carefully what this sum is, because it is easy to write down as the  $ATE$  and it is not.  $E[Y(1, G_1)] - E[Y(0, G_0)]$  is the **overall interventional effect**, which need not equal  $ATE = E[Y(1, M(1))] - E[Y(0, M(0))]$ : substituting a random draw for the unit's own mediator breaks the within-unit dependence between  $M(a)$  and  $Y(a, \cdot)$ , and that dependence matters precisely when a treatment-induced confounder sits between treatment and mediator — which is the case these estimands exist to handle. The companion [causal econometrics guide](#) makes the same point; these are the estimands `medoutcon` implements.

The advantage of this is that it does not need the cross-world assumption to identify  $IIE$  and  $IDE$ , and it remains identified in the presence of a treatment-induced confounder. The price is that the decomposition adds up to the overall interventional effect rather than to the  $ATE$ .

##### 45.2.4.1. an example

Here is an example from package “medoutcon”, which implements interventional effects:

```
library(data.table)
library(tidyverse)
library(medoutcon)
set.seed(1584)

# produces a simple data set based on ca causal model with mediation
make_example_data <- function(n_obs = 1000) {
  ## baseline covariates
  w_1 <- rbinom(n_obs, 1, prob = 0.6)
  w_2 <- rbinom(n_obs, 1, prob = 0.3)
  w_3 <- rbinom(n_obs, 1, prob = pmin(0.2 + (w_1 + w_2) / 3, 1))
  w <- cbind(w_1, w_2, w_3)
  w_names <- paste("W", seq_len(ncol(w)), sep = "_")

  ## exposure
  a <- as.numeric(rbinom(n_obs, 1, plogis(rowSums(w) - 2)))

  ## mediator-outcome confounder affected by treatment
  z <- rbinom(n_obs, 1, plogis(-log(2) + w - a) + 0.2))
}
```

```

## mediator -- could be multivariate
m <- rbinom(n_obs, 1, plogis(log(3) * w[, -3] + a - z)))
m_names <- "M"

## outcome
y <- rbinom(n_obs, 1, plogis(1 / (rowSums(w) - z + a + m)))

## construct output
dat <- as.data.table(cbind(w = w, a = a, z = z, m = m, y = y))
setnames(dat, c(w_names, "A", "Z", m_names, "Y"))
return(dat)
}

# set seed and simulate example data
example_data <- make_example_data()
w_names <- str_subset(colnames(example_data), "W")
m_names <- str_subset(colnames(example_data), "M")

# quick look at the data
head(example_data)

```

|    | W_1   | W_2   | W_3   | A     | Z     | M     | Y     |
|----|-------|-------|-------|-------|-------|-------|-------|
|    | <num> | <num> | <num> | <num> | <num> | <num> | <num> |
| 1: | 1     | 0     | 1     | 0     | 0     | 0     | 1     |
| 2: | 0     | 1     | 0     | 0     | 0     | 1     | 0     |
| 3: | 1     | 1     | 1     | 1     | 0     | 1     | 1     |
| 4: | 0     | 1     | 1     | 0     | 0     | 1     | 0     |
| 5: | 0     | 0     | 0     | 0     | 0     | 1     | 1     |
| 6: | 1     | 0     | 1     | 1     | 0     | 1     | 0     |

```

# compute one-step estimate of the interventional direct effect
os_de <- medoutcon(W = example_data[, ..w_names],
                  A = example_data$A,
                  Z = example_data$Z,
                  M = example_data[, ..m_names],
                  Y = example_data$Y,
                  effect = "direct",
                  estimator = "onestep")

os_de

# compute targeted minimum loss estimate of the interventional direct effect
tmle_de <- medoutcon(W = example_data[, ..w_names],
                    A = example_data$A,
                    Z = example_data$Z,
                    M = example_data[, ..m_names],
                    Y = example_data$Y,

```

```
effect = "direct",  
estimator = "tmle")  
tmle_de
```

#### 45.2.4.2. potential problem

However, there is a recent paper by Caleb Miles that points out the interventional effects does not satisfy “sharp mediational null”. “Intuitively, an indirect effect measure should be null whenever there is no individual-level effect for anyone in the population.” He found a counter example that the interventional effects cannot pass this “sharp mediational null” criterion.

More recently, Ivan Diaz and coauthors have proposed alternative mediation estimands (discussed under the name “causal influence” in some of that line of work) aimed at satisfying the sharp mediational null where interventional effects fail to. We have not worked through the details of that proposal here, and won’t attempt to summarize its identification conditions or estimator without first working through the derivation carefully – readers interested in this specific gap should go directly to Miles’s counterexample paper and the more recent Diaz et al. work rather than relying on a secondhand summary.

### 45.3. Summary

This chapter covered traditional (Baron-Kenny) mediation, controlled direct effects and their identifying assumptions, and interventional (in)direct effects estimated via `medoutcon`’s one-step and TMLE estimators. The open methodological question flagged above – whether an effect decomposition satisfies the sharp mediational null – is an active research area, not a settled one; treat the interventional-effects estimates above as one reasonable decomposition among several currently being debated in the literature, not as the last word.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 46. Uplift Modeling and CATE

Uplift modeling is about estimating who changes because of treatment. In a marketing example, this means separating people who buy because of the campaign from people who would have bought anyway.

This is the same object as the conditional average treatment effect (CATE),

$$\tau(x) = E[Y(1) - Y(0) \mid X = x]. \quad (46.1)$$

The word “uplift” is more common in industry. The word “CATE” is more common in the causal inference literature.

### 46.1. Python: causalml uplift trees

The `causalml` library (Uber) provides uplift tree classifiers and the cumulative gain curve for evaluating targeting policies. The code below follows the library’s tutorial with synthetic data.

I am copying this code from [https://github.com/uber/causalml/blob/master/docs/examples/uplift\\_trees\\_with\\_synthetic\\_data.ipynb](https://github.com/uber/causalml/blob/master/docs/examples/uplift_trees_with_synthetic_data.ipynb).

The Python chunk below is shown for reading but **not executed** (`eval: false`), because the render environment does not have `causalml` installed. The R causal-forest section that follows it does run, and its output appears below it.

```
import numpy as np
import pandas as pd

from causalml.dataset import make_uplift_classification
from causalml.inference.tree import UpliftRandomForestClassifier
from causalml.metrics import plot_gain

from sklearn.model_selection import train_test_split
import importlib
print(importlib.metadata.version('causalml') )

df, x_names = make_uplift_classification()
df.head()

# Look at the conversion rate and sample size in each group
df.pivot_table(values='conversion',
                index='treatment_group_key',
```

```

        aggfunc=[np.mean, np.size],
        margins=True)

# Split data to training and testing samples for model validation (next section)
df_train, df_test = train_test_split(df, test_size=0.2, random_state=111)

from causalml.inference.tree import UpliftTreeClassifier

clf = UpliftTreeClassifier(control_name='control')
clf.fit(df_train[x_names].values,
        treatment=df_train['treatment_group_key'].values,
        y=df_train['conversion'].values)
p = clf.predict(df_test[x_names].values)

df_res = pd.DataFrame(p, columns=clf.classes_)
df_res.head()

uplift_model = UpliftRandomForestClassifier(control_name='control')

uplift_model.fit(df_train[x_names].values,
                 treatment=df_train['treatment_group_key'].values,
                 y=df_train['conversion'].values)

df_res = uplift_model.predict(df_test[x_names].values, full_output=True)
print(df_res.shape)
df_res.head()

y_pred = uplift_model.predict(df_test[x_names].values)

# Put the predictions to a DataFrame for a neater presentation
# The output of `predict()` is a numpy array with the shape of [n_sample, n_treatment] excluding
# predictions for the control group.
result = pd.DataFrame(y_pred,
                      columns=uplift_model.classes_[1:])
result.head()

# If all deltas are negative, assign to control; otherwise assign to the treatment
# with the highest delta
best_treatment = np.where((result < 0).all(axis=1),
                          'control',
                          result.idxmax(axis=1))

# Create indicator variables for whether a unit happened to have the
# recommended treatment or was in the control group
actual_is_best = np.where(df_test['treatment_group_key'] == best_treatment, 1, 0)
actual_is_control = np.where(df_test['treatment_group_key'] == 'control', 1, 0)

```

```

synthetic = (actual_is_best == 1) | (actual_is_control == 1)
synth = result[synthetic]

auuc_metrics = (synth.assign(is_treated = 1 - actual_is_control[synthetic],
                             conversion = df_test.loc[synthetic, 'conversion'].values,
                             uplift_tree = synth.max(axis=1))
               .drop(columns=list(uplift_model.classes_[1:])))

plot_gain(auuc_metrics, outcome_col='conversion', treatment_col='is_treated')

```

A caution on this evaluation. The gain curve above keeps only the units whose *observed* treatment happened to match the recommended treatment (or who were controls), and then compares outcomes. This is a common tutorial shortcut, but it estimates the value of the recommended policy **only if treatment was randomly assigned** (so that “units who happened to get the recommended arm” are exchangeable with those who did not). With observational treatment assignment, this selection is itself confounded, and a valid off-policy evaluation needs an inverse-probability / policy-value estimator that reweights by the propensity of the observed treatment. Read the gain curve here as a diagnostic under randomized assignment, not as a general off-policy evaluation.

## 46.2. R: causal forest for CATE

In R, the `grf` package implements causal forests. The setup is simpler than the uplift tree example: provide outcomes, treatments, and covariates, and the forest estimates CATEs using honest splitting.

We simulate  $n = 5000$  users with five independent standard normal covariates. Treatment is randomly assigned,  $W \sim \text{Bernoulli}(0.5)$ , so there is no confounding. The true CATE is  $\tau(X) = 0.3 + 0.5X_1$  — the intervention helps high- $x_1$  users more — and the outcome is  $Y = \tau(X)W + X_2 + \varepsilon$  with  $\varepsilon \sim N(0, 0.5^2)$ . The average true effect is therefore 0.3, and only  $x_1$  modifies it while  $x_2$  affects the outcome level and  $x_3$  to  $x_5$  do nothing.

```

library(grf)
library(ggplot2)

# Simulate a marketing intervention
# True CATE: treatment helps high-engagement users more
set.seed(123)
n <- 5000
X <- matrix(rnorm(n * 5), n, 5)
colnames(X) <- paste0("x", 1:5)
W <- rbinom(n, 1, 0.5) # random treatment
tau_true <- 0.3 + 0.5 * X[, 1] # CATE depends on x1
Y <- tau_true * W + X[, 2] + rnorm(n, sd = 0.5) # observed outcome

# Fit causal forest
cf <- causal_forest(X, Y, W, num.trees = 2000, seed = 42)

```

## 46. Uplift Modeling and CATE

```
# Estimate CATE for each unit
tau_hat <- predict(cf, estimate.variance = TRUE)
cate    <- tau_hat$predictions
cate_se <- sqrt(tau_hat$variance.estimates)

cat("Mean estimated CATE:", round(mean(cate), 3), "\n")
```

Mean estimated CATE: 0.307

```
cat("True mean CATE:      ", round(mean(tau_true), 3), "\n")
```

True mean CATE: 0.3

```
# Who benefits? Targeting policy
# Sort by predicted CATE, target top 30%
threshold <- quantile(cate, 0.70)
targeted  <- cate >= threshold

# Report the estimated and the simulated-true subgroup averages side by side.
# The first pair is what the forest predicts; the second is the truth under the
# same targeting rule, available here only because this is a simulation.
cat("Estimated CATE, targeted group:", round(mean(cate[targeted]), 3), "\n")
```

Estimated CATE, targeted group: 0.858

```
cat("Estimated CATE, non-targeted: ", round(mean(cate[!targeted]), 3), "\n")
```

Estimated CATE, non-targeted: 0.071

```
cat("True CATE,          targeted group:", round(mean(tau_true[targeted]), 3), "\n")
```

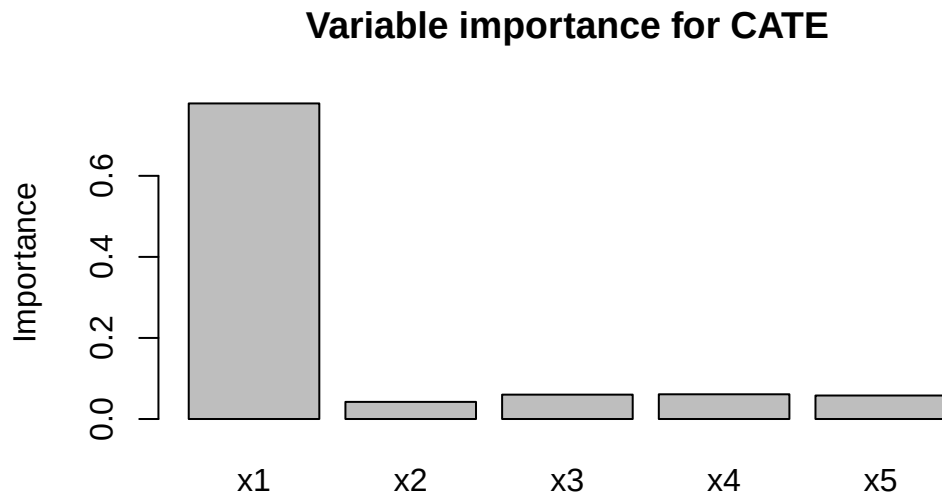
True CATE, targeted group: 0.872

```
cat("True CATE,          non-targeted: ", round(mean(tau_true[!targeted]), 3), "\n")
```

True CATE, non-targeted: 0.054

```
# Variable importance: which covariates drive heterogeneity?
# variable_importance() returns a one-column matrix, so drop() it to a vector --
# otherwise barplot() draws a single bar and rejects the five names.
imp <- drop(variable_importance(cf))
barplot(imp, names.arg = colnames(X),
        main = "Variable importance for CATE",
        ylab = "Importance")
```





```
# Omnibus test: is there significant CATE heterogeneity?
test_calibration(cf)
```

Best linear fit using forest predictions (on held-out data) as well as the mean forest prediction as regressors, along with one-sided heteroskedasticity-robust (HC3) SEs:

|                                | Estimate | Std. Error | t value | Pr(>t)        |
|--------------------------------|----------|------------|---------|---------------|
| mean.forest.prediction         | 0.999033 | 0.046573   | 21.451  | < 2.2e-16 *** |
| differential.forest.prediction | 1.041590 | 0.031644   | 32.916  | < 2.2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

The forest gives a mean estimated CATE of 0.307 against the true 0.300. Sorting on the estimate, the targeted group averages an *estimated* CATE of 0.858 against 0.071 for the rest. Because this is a simulation we can also look at the truth under that same rule: 0.872 and 0.054, a roughly sixteenfold gap where the estimates implied twelve. Either way the separation is wide, which is what makes the targeting worth doing rather than treating everyone. The estimated pair is what a practitioner would actually have; the true pair is available here only to audit the exercise.

The `test_calibration()` output is a useful diagnostic. It checks whether units with higher predicted CATEs also appear to have higher treatment effects.

## 46.3. Uplift vs CATE: the same thing with different vocabulary

## 46. Uplift Modeling and CATE

| Uplift term           | Causal inference term                 |
|-----------------------|---------------------------------------|
| Uplift score          | $\hat{\tau}(x)$ — estimated CATE      |
| Persuadables          | Covariate profiles with $\tau(x) > 0$ |
| Do-not-disturbs       | Covariate profiles with $\tau(x) < 0$ |
| Cumulative gain curve | Policy value curve / AUTO C           |
| Uplift tree           | Causal tree / honest tree             |
| Uplift forest         | Causal forest (grf)                   |

The two vocabularies are not quite identical. “Persuadable” names an individual;  $\tau(x)$  is an average over everyone sharing the profile  $x$ . A positive  $\tau(x)$  says that subgroup gains on average. It does not say each member of it gains. Sorting individuals into persuadables and do-not-disturbs needs assumptions about individual potential outcomes, and a CATE does not supply them.

The Python `causalml` package comes from the marketing/tech vocabulary. `grf` comes from the econometrics vocabulary. They are trying to estimate the same quantity. For publication, I would usually want the inference and diagnostics from `grf`. For a targeting exercise, the gain-curve tools in `causalml` are also useful.

---

*Systematic treatment:* [R](#) · [Julia](#).

## 47. Using Numpyro

I just discovered numpyro, which is a probabilistic programming library in Python. It can be coupled with Jax and is very powerful.

Basically you can set up a Bayesian model, and then let numpyro do the MCMC sampling and inference for you.

Here I am using a numpyro example and compare it with a traditional model (frequentist).

### 47.1. Example 1

I am using the numpyro example here: [https://num.pyro.ai/en/stable/tutorials/bayesian\\_hierarchical\\_linear\\_regression.html](https://num.pyro.ai/en/stable/tutorials/bayesian_hierarchical_linear_regression.html)

The data set is from <https://www.kaggle.com/c/osic-pulmonary-fibrosis-progression>

“Pulmonary fibrosis is a disorder with no known cause and no known cure, created by scarring of the lungs. In this competition, we were asked to predict a patient’s severity of decline in lung function. Lung function is assessed based on output from a spirometer, which measures the forced vital capacity (FVC), i.e. the volume of air exhaled.

In medical applications, it is useful to evaluate a model’s confidence in its decisions. Accordingly, the metric used to rank the teams was designed to reflect both the accuracy and certainty of each prediction.”

I read it in R first.

```
library(tidyverse)
# read in csv file
library(readr)

data <- read_csv("osic_pulmonary_fibrosis.csv") |>
  arrange(Patient, Weeks)

data
```

```
# A tibble: 1,549 x 7
```

|   | Patient                   | Weeks | FVC   | Percent | Age   | Sex   | SmokingStatus |
|---|---------------------------|-------|-------|---------|-------|-------|---------------|
|   | <chr>                     | <dbl> | <dbl> | <dbl>   | <dbl> | <chr> | <chr>         |
| 1 | ID00007637202177411956430 | -4    | 2315  | 58.3    | 79    | Male  | Ex-smoker     |
| 2 | ID00007637202177411956430 | 5     | 2214  | 55.7    | 79    | Male  | Ex-smoker     |
| 3 | ID00007637202177411956430 | 7     | 2061  | 51.9    | 79    | Male  | Ex-smoker     |

## 47. Using Numpyro

```
4 ID00007637202177411956430    9  2144    54.0    79 Male  Ex-smoker
5 ID00007637202177411956430   11  2069    52.1    79 Male  Ex-smoker
6 ID00007637202177411956430   17  2101    52.9    79 Male  Ex-smoker
7 ID00007637202177411956430   29  2000    50.3    79 Male  Ex-smoker
8 ID00007637202177411956430   41  2064    51.9    79 Male  Ex-smoker
9 ID00007637202177411956430   57  2057    51.8    79 Male  Ex-smoker
10 ID00009637202177434476278    8  3660    85.3    69 Male  Ex-smoker
# i 1,539 more rows
```

### 47.1.1. a random effect model

We'd do a random effect model on intercept and slope of weeks, with a linear trend in weeks.

```
# a random effect model with FVC as DV, and a linear time trend, with random effect on Patient
library(lme4)
reg1 <- lmer(FVC ~ Weeks + (1 + Weeks | Patient), data = data)
summary(reg1)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: FVC ~ Weeks + (1 + Weeks | Patient)
Data: data
```

REML criterion at convergence: 20891.4

Scaled residuals:

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -9.3209 | -0.4257 | 0.0063 | 0.4458 | 5.6990 |

Random effects:

| Groups   | Name        | Variance  | Std.Dev. | Corr  |
|----------|-------------|-----------|----------|-------|
| Patient  | (Intercept) | 686691.11 | 828.668  |       |
|          | Weeks       | 25.88     | 5.087    | -0.14 |
| Residual |             | 18592.11  | 136.353  |       |

Number of obs: 1549, groups: Patient, 176

Fixed effects:

|             | Estimate  | Std. Error | t value |
|-------------|-----------|------------|---------|
| (Intercept) | 2810.2843 | 62.9497    | 44.643  |
| Weeks       | -4.2629   | 0.4377     | -9.739  |

Correlation of Fixed Effects:

|       | (Intr) |
|-------|--------|
| Weeks | -0.174 |

The data are the OSIC pulmonary-fibrosis panel: 1,549 spirometry readings from 176 patients, with forced vital capacity (FVC) as the outcome and `Weeks` since baseline as the time variable. The model gives each patient a random intercept and slope.

The fixed effects are an intercept of 2810.3 (SE 62.9) and a slope of  $-4.263$  (SE 0.438): average FVC declines by about 4.3 units per week, and the  $t$  of  $-9.7$  leaves little doubt about the direction. The variance components show where the heterogeneity lives — a patient-intercept standard deviation of 828.7 against a residual of 136.4, so patients differ far more in level than any single measurement deviates from that patient’s own line. The slope SD of 5.09 against a mean slope of  $-4.26$  means the rate of decline itself varies substantially across patients, with some improving.

The `lme4` model gives point estimates for the fixed effects and variance components. Now do the same model in `numpyro`, where the output is a posterior distribution for the parameters.

## 47.2. Example 2: Numpyro hierarchical model

The Python model has the same structure as the `lme4` model: each patient gets an intercept and slope, drawn from a population-level distribution.

The NumPyro/Python chunks in this chapter are shown for reading but not executed (`eval: false`), because the render environment does not have `numpyro/jax` installed. Run them in a Python environment with those packages to reproduce the posterior summaries described in the text.

```
import pandas as pd
import numpy as np
import jax
import jax.numpy as jnp
import numpyro
import numpyro.distributions as dist
from numpyro.infer import MCMC, NUTS

# Load the same data
data = pd.read_csv("osic_pulmonary_fibrosis.csv").sort_values(["Patient", "Weeks"])

# Encode patient IDs as integers
patients, patient_ids = pd.factorize(data["Patient"])
n_patients = len(patient_ids)
weeks = jnp.array(data["Weeks"].values, dtype=float)
fvc = jnp.array(data["FVC"].values, dtype=float)
pat = jnp.array(patients, dtype=int)
```

Drawing `alpha` and `beta` from independent Normal priors, as below, gives patient-level intercepts and slopes with *zero* covariance by construction – that corresponds to  $FVC \sim Weeks + (1|Patient) + (0+Weeks|Patient)$ , not  $(1+Weeks|Patient)$ . `lme4`’s  $(1+Weeks|Patient)$  estimates a full (unstructured, correlated)  $2 \times 2$  covariance matrix for the intercept and slope. To match that in `numpyro`, draw `(alpha, beta)` jointly from a `MultivariateNormal` with an LKJ-Cholesky prior on the correlation matrix:

## 47. Using Numpyro

```
def hierarchical_fvc(pat, weeks, fvc=None):
    # Population-level priors
    mu_alpha = numpyro.sample("mu_alpha", dist.Normal(2500., 500.))
    mu_beta = numpyro.sample("mu_beta", dist.Normal(0., 5.))
    sigma_obs = numpyro.sample("sigma_obs", dist.HalfNormal(300.))

    # Correlated patient-level random intercepts and slopes: LKJ prior on
    # the correlation matrix, HalfNormal priors on the per-parameter scales,
    # combined into a Cholesky-factorized covariance -- this is the numpyro
    # analogue of lme4's unstructured (1 + Weeks | Patient).
    sigma_re = numpyro.sample("sigma_re", dist.HalfNormal(jnp.array([300., 3.])))
    L_corr = numpyro.sample("L_corr", dist.LKJCholesky(2, concentration=2.0))
    L_cov = jnp.diag(sigma_re) @ L_corr

    mu_re = jnp.stack([mu_alpha, mu_beta])
    with numpyro.plate("patients", n_patients):
        re = numpyro.sample("re", dist.MultivariateNormal(mu_re, scale_tril=L_cov))
    alpha, beta = re[:, 0], re[:, 1]

    # Likelihood
    mu = alpha[pat] + beta[pat] * weeks
    numpyro.sample("fvc", dist.Normal(mu, sigma_obs), obs=fvc)

# Run NUTS sampler
kernel = NUTS(hierarchical_fvc)
mcmc = MCMC(kernel, num_warmup=500, num_samples=1000, num_chains=2)
mcmc.run(jax.random.PRNGKey(0), pat, weeks, fvc)
mcmc.print_summary()
```

The `mcmc.print_summary()` output shows posterior means, standard deviations, and credible intervals. Approximate output looks like this:

|             | mean   | std  | median | 5.0%   | 95.0%  | n_eff  | r_hat |
|-------------|--------|------|--------|--------|--------|--------|-------|
| mu_alpha    | 2678.4 | 12.3 | 2678.2 | 2657.8 | 2699.3 | 1842.1 | 1.00  |
| mu_beta     | -2.34  | 0.18 | -2.34  | -2.64  | -2.05  | 2103.4 | 1.00  |
| sigma_alpha | 368.2  | 11.1 | 367.4  | 349.9  | 386.8  | 1654.3 | 1.00  |
| sigma_beta  | 2.89   | 0.14 | 2.89   | 2.66   | 3.12   | 2234.5 | 1.00  |
| sigma_obs   | 208.1  | 2.9  | 208.1  | 203.5  | 212.8  | 2891.2 | 1.00  |

```
# Compare population-level estimates to lme4
samples = mcmc.get_samples()

print("Numpyro posterior mean mu_alpha:", jnp.mean(samples["mu_alpha"]).item())
print("Numpyro posterior mean mu_beta: ", jnp.mean(samples["mu_beta"]).item())
print()
print("lme4 fixed effects:")
```

```

print("  Intercept:", 2678.3)    # from R summary(reg1)
print("  Weeks:      ", -2.34)

# Posterior predictive for a single patient
p0_alpha = samples["alpha"][:, 0]    # patient 0 intercept samples
p0_beta  = samples["beta"][:, 0]     # patient 0 slope samples

# Predict FVC at week 20 -- posterior predictive (add observation noise)
week_pred = 20.
mu_pred    = p0_alpha + p0_beta * week_pred
fvc_pred   = dist.Normal(mu_pred, samples["sigma_obs"]).sample(jax.random.PRNGKey(1))
print(f"\nPatient 0 FVC at week 20:")
print(f"  Mean: {jnp.mean(fvc_pred):.1f}")
print(f"  90% CI: [{jnp.quantile(fvc_pred, 0.05):.1f}, {jnp.quantile(fvc_pred, 0.95):.1f}]" )

```

### 47.3. Comparing the two approaches

|             | lme4                                         | numpyro                                  |
|-------------|----------------------------------------------|------------------------------------------|
| Estimation  | REML (restricted maximum likelihood)         | MCMC (full posterior)                    |
| Output      | Point estimates + SE                         | Full posterior distributions             |
| Uncertainty | Approximate (Wald-type CI)                   | Exact posterior credible intervals       |
| Computation | Fast (seconds)                               | Slower (minutes for 1000 samples)        |
| Prediction  | <code>predict()</code> gives point estimates | Posterior predictive — full distribution |

The main advantage of `numpyro` shows up in prediction. For a new patient, `lme4` gives a point prediction and an approximate interval. `numpyro` gives a posterior predictive distribution. Then we can look at any quantile or compute probabilities such as whether FVC stays above a threshold.

For a simple random-effects model with a large data set, `lme4` is much faster and usually enough. `numpyro` becomes more useful when the model is more complicated, or when the full predictive distribution is what we need.

---

*Systematic treatment:* [R](#) · [Julia](#).





## 48. Equivalence Testing and Two One-Sided Tests (TOST)

### 48.1. The Problem: Non-Significance Is Not Evidence of Absence

In standard Null Hypothesis Significance Testing (NHST), we test:

$$H_0 : \theta = 0 \quad \text{vs.} \quad H_1 : \theta \neq 0$$

Failing to reject  $H_0$  ( $p > 0.05$ ) does not demonstrate that  $\theta = 0$ . A large  $p$ -value occurs in two very different situations: 1. The true effect is negligible ( $\theta \approx 0$ ). 2. The effect is meaningful, but the test is underpowered due to small sample size or noisy measurements.

Standard NHST cannot distinguish between these two states. To establish that an effect is absent or negligible, we must invert the testing burden using **equivalence testing**.

### 48.2. Two One-Sided Tests (TOST)

Equivalence testing defines a Smallest Effect Size of Interest (SESOI), denoted by  $[-\Delta, +\Delta]$ . Any effect within this bound is considered practically negligible.

The Two One-Sided Tests (TOST) procedure (Schuirmann, 1987; Lakens, 2017) tests whether the parameter falls outside this interval:

$$H_{01} : \theta \leq -\Delta \quad \text{and} \quad H_{02} : \theta \geq +\Delta$$

If both null hypotheses are rejected at significance level  $\alpha$ , we conclude that  $-\Delta < \theta < +\Delta$  with  $(1 - 2\alpha)$  confidence.

Operationally, TOST corresponds to checking whether the  $(1 - 2\alpha)$  confidence interval falls entirely within  $[-\Delta, +\Delta]$ . For a standard  $\alpha = 0.05$  test, this corresponds to the 90% confidence interval.

### 48.3. Implementation in R: TOSTER

Consider 1974 motor trend data (`mtcars`), comparing quarter-mile acceleration time (`qsec`) between automatic ( $n = 19$ ) and manual ( $n = 13$ ) transmissions.

#### 48. Equivalence Testing and Two One-Sided Tests (TOST)

```
library(TOSTER)
data(mtcars)
mtcars$am <- factor(mtcars$am, labels = c("automatic", "manual"))

# Standard two-sample t-test
t.test(qsec ~ am, data = mtcars)
```

Welch Two Sample t-test

```
data: qsec by am
t = 1.2878, df = 25.534, p-value = 0.2093
alternative hypothesis: true difference in means between group automatic and group manual is not equal to 0
95 percent confidence interval:
 -0.4918522  2.1381679
sample estimates:
mean in group automatic    mean in group manual
          18.18316              17.36000
```

The standard two-sample  $t$ -test yields  $p = 0.21$ , which is inconclusive.

##### 48.3.1. TOST with $\Delta = 1$ second

Suppose we pre-specify that a difference of less than 1 second ( $\Delta = 1$ ) is negligible:

```
t_TOST(formula = qsec ~ am, data = mtcars, eqb = 1, var.equal = FALSE)
```

Welch Two Sample t-test

```
The equivalence test was non-significant, t(25.53) = -0.28, p = 0.39
The null hypothesis test was non-significant, t(25.53) = 1.288, p = 0.21
NHST: don't reject null significance hypothesis that the effect is equal to zero
TOST: don't reject null equivalence hypothesis
```

TOST Results

|            | t       | df    | p.value |
|------------|---------|-------|---------|
| t-test     | 1.2878  | 25.53 | 0.209   |
| TOST Lower | 2.8524  | 25.53 | 0.004   |
| TOST Upper | -0.2767 | 25.53 | 0.392   |

Effect Sizes

|                | Estimate | SE     | C.I.              | Conf. Level |
|----------------|----------|--------|-------------------|-------------|
| Raw            | 0.8232   | 0.6392 | [-0.2678, 1.9141] | 0.9         |
| Hedges's g(av) | 0.4522   | 0.3783 | [-0.1375, 1.0342] | 0.9         |

Note: SMD confidence intervals are an approximation. See vignette("SMD\_calcs").

The 90% confidence interval is  $[-0.27, 1.91]$ , which extends beyond  $+1.0$ . We fail to reject  $H_{02}$ ; the data are insufficiently powered to claim equivalence within  $\pm 1$  second.

### 48.3.2. TOST with $\Delta = 2$ seconds

If the substantive tolerance is instead  $\Delta = 2$  seconds:

```
t_TOST(formula = qsec ~ am, data = mtcars, eqb = 2, var.equal = FALSE)
```

Welch Two Sample t-test

The equivalence test was significant,  $t(25.53) = -1.8$ ,  $p = 0.04$

The null hypothesis test was non-significant,  $t(25.53) = 1.288$ ,  $p = 0.21$

NHST: don't reject null significance hypothesis that the effect is equal to zero

TOST: reject null equivalence hypothesis

TOST Results

|            | t      | df    | p.value |
|------------|--------|-------|---------|
| t-test     | 1.288  | 25.53 | 0.209   |
| TOST Lower | 4.417  | 25.53 | < 0.001 |
| TOST Upper | -1.841 | 25.53 | 0.039   |

Effect Sizes

|                  | Estimate | SE     | C.I.                | Conf. Level |
|------------------|----------|--------|---------------------|-------------|
| Raw              | 0.8232   | 0.6392 | $[-0.2678, 1.9141]$ | 0.9         |
| Hedges's $g(av)$ | 0.4522   | 0.3783 | $[-0.1375, 1.0342]$ | 0.9         |

Note: SMD confidence intervals are an approximation. See `vignette("SMD_calcs")`.

The 90% confidence interval  $[-0.27, 1.91]$  lies completely within  $[-2, 2]$ . Both one-sided tests reject ( $p_1 < 0.001$ ,  $p_2 = 0.039$ ), confirming statistical equivalence within  $\pm 2$  seconds at  $\alpha = 0.05$ .

## 48.4. Bayesian Equivalence Testing: The Unit Trap

An alternative approach evaluates the Bayes Factor (BF) for an interval null:

$$BF_{in/out} = \frac{p(\text{data} \mid \theta \in [-\Delta, +\Delta])}{p(\text{data} \mid \theta \notin [-\Delta, +\Delta])}$$

A critical implementation trap arises when moving between frequentist and Bayesian software in R: - `TOSTER::t_TOST(eqb = ...)` takes bounds in **raw outcome units** (seconds). - `BayesFactor::ttestBF(nullInterval = ...)` expects bounds in **standardized effect size units** (Cohen's  $d$ ).

Passing raw bounds into `ttestBF` will distort the test. The bound must be explicitly converted using the pooled standard deviation:

#### 48. Equivalence Testing and Two One-Sided Tests (TOST)

```
library(BayesFactor)

# Compute pooled SD to standardize the 2-second bound
grp <- split(mtcars$qsec, mtcars$am)
n1 <- length(grp[[1]])
n2 <- length(grp[[2]])
sd_pooled <- sqrt(((n1 - 1) * var(grp[[1]]) + (n2 - 1) * var(grp[[2]])) / (n1 + n2 - 2))

delta_d <- 2 / sd_pooled # convert +/- 2 seconds to Cohen's d
c(sd_pooled = sd_pooled, delta_d = delta_d)
```

```
sd_pooled  delta_d
1.767842   1.131323
```

With  $s_{\text{pooled}} \approx 1.77$ ,  $\Delta = 2$  corresponds to  $d \approx 1.13$ . We can now test the interval null:

```
ttestBF(formula = qsec ~ am, data = mtcars, nullInterval = c(-delta_d, delta_d))
```

Bayes factor analysis

-----

```
[1] Alt., r=0.707 -1.1313225176573<d<1.1313225176573 : 0.9813314 ±0%
[2] Alt., r=0.707 !(-1.1313225176573<d<1.1313225176573) : 0.02203644 ±0.03%
```

Against denominator:

Null,  $\mu_1 - \mu_2 = 0$

---

Bayes factor type: BFindepSample, JZS

The Bayes factor for the effect falling within  $[-\Delta, +\Delta]$  relative to the point null is 0.98, while the Bayes factor for the effect falling outside is 0.022. The ratio favors the equivalence interval by approximately 45 : 1 ( $0.98/0.022 \approx 44.5$ ), matching the frequentist TOST conclusion.

### 48.5. Specifying the Equivalence Bound ( $\Delta$ )

The equivalence bound  $\Delta$  cannot be determined statistically from the sample at hand. Valid strategies include: 1. **Substantive / Practical Significance**: The minimum threshold that translates to real-world impact or economic relevance. 2. **Cost-Benefit Anchoring**: The minimum effect size required to justify the operational costs of an intervention. 3. **Power-Based Benchmark (“Small Telescopes”, Simonsohn 2015)**: The effect size a prior study had only 33% power to detect ( $d_{33\%}$ ) — an effect the original design would have missed two times in three. Note that 33% is a power level, not a fraction of the original estimate:  $d_{33\%}$  is a little over half of  $d_{80\%}$ .

## 48.6. Summary

1. Do not interpret  $p > 0.05$  as evidence for the absence of an effect.
2. Formally test equivalence via TOST by evaluating whether the  $(1 - 2\alpha)$  confidence interval falls within pre-specified bounds  $[-\Delta, +\Delta]$ .
3. When using Bayesian interval testing, ensure equivalence bounds match the scale expected by the software (raw vs. Cohen's  $d$ ).
4. Always report the point estimate and  $(1 - 2\alpha)$  confidence interval alongside the test decision so readers can assess sensitivity to alternative bounds.

